Cuprins

0 Introducere **1**

0.1 Sisteme de calcul . . . 2
0.2 Sistemul de operare . . . 4
0.2.1 Alegerea sistemului de operare . . . 5
0.3 Lumea Linux . . . 7
0.3.1 Linux și Unix . . . 8
0.3.2 GNU/Linux. Distribuții . . . 8
0.3.3 Mașina virtuală de suport . . . 9
0.3.4 Instalarea Linux . . . 11
0.4 Sumar . . . 12

1 Interfața cu utilizatorul **13**

1.1 Interfața grafică . . . 15
1.1.1 Interfețe native și interfețe web . . . 15
1.1.2 Utilizabilitate și experiența utilizatorului . . . 17
1.2 Interfața grafică în Linux . . . 18
1.2.1 Desktop Environments . . . 18
1.2.2 Sistemul de ferestre (Window System) . . . 19
1.2.3 Funcționalități ale interfeței grafice în Linux . . . 22
1.3 Interfața grafică în mașina virtuală de suport . . . 23
1.4 Sumar . . . 26

2 Utilizarea sistemului de fișiere **27**

2.1 Noțiuni de bază . . . 27
2.1.1 Definiții . . . 28
2.1.2 Structura ierarhică a sistemului de fișiere . . . 30
2.1.3 Căi relative și căi absolute . . . 33
2.2 Formatul fișierelor . . . 35
2.2.1 Atributele fișierelor . . . 37
2.3 Operații uzuale asupra fișierelor și directoarelor . . . 38
2.3.1 Afișarea și schimbarea directorului curent . . . 39
2.3.2 Listarea fișierelor . . . 40
2.3.3 Afișarea conținutului fișierelor . . . 42
2.3.4 Crearea fișierelor/directoarelor . . . 43
2.3.5 Copiere / mutare / redenumire / ștergere . . . 44
2.3.6 Căutarea fișierelor . . . 47
2.3.7 Arhivarea și dezarhivarea fișierelor . . . 50
2.3.8 Backup . . . 51
2.4 Redirectarea intrării sau ieșirii . . . 52
2.5 Tipuri de sisteme de fișiere . . . 55

2.5.1 Integritatea datelor . . . 56
2.5.2 Alegerea unui sistem de fișiere . . . 57
2.6 Anexă: Comenzi pentru lucrul cu fișiere în Windows . . . 57
2.7 Sumar . . . 57

3 Pachete software 60
3.1 Pachete software . . . 61
3.1.1 Tipuri de pachete . . . 63
3.2 Gestiunea pachetelor . . . 64
3.2.1 Operații cu pachete . . . 65
3.2.2 Pachete în sistemele de operare mobile . . . 66
3.3 Gestiunea pachetelor în Linux . . . 66
3.3.1 Gestiunea pachetelor format DEB . . . 67
3.3.2 Gestiunea pachetelor format RPM . . . 69
3.3.3 Gestiunea pachetelor în alte formate . . . 70
3.4 Sisteme de distribuție cu tot cu sistem . . . 71
3.4.1 Medii specifice pentru limbaje de programare . . . 74
3.5 Anexă: Instalarea unui pachet din surse . . . 76
3.6 Sumar . . . 79

4 Procese 80
4.1 Programe și procese . . . 82
4.2 Resursele și atributele unui proces . . . 83
4.2.1 Atributele unui proces . . . 85
4.2.2 Utilitare pentru urmărirea proceselor . . . 86
4.2.3 Starea proceselor . . . 90
4.2.4 Prioritatea proceselor . . . 92
4.3 Ierarhia de procese . . . 94
4.3.1 Ierarhia de procese în Linux/Unix . . . 96
4.3.2 Ierarhia de procese în Windows . . . 96
4.3.3 Foreground și background . . . 97
4.3.4 Procesul init . . . 101
4.4 Procese și fișiere . . . 102
4.4.1 Descriptori de fișiere . . . 102
4.4.2 Redirectarea în/din fișiere . . . 106
4.5 Operații cu procese. Interacțiunea între procese . . . 107
4.5.1 Încheierea unui proces . . . 107
4.5.2 Semnale . . . 108
4.5.3 Înlănțuirea comenzilor . . . 110
4.5.4 Comunicarea prin pipe-uri . . . 110
4.6 Interactivitatea proceselor . . . 111
4.6.1 Terminale . . . 112
4.6.2 Procese neinteractive . . . 113
4.6.3 Procese daemon . . . 113
4.6.4 Detașarea de terminal . . . 114
4.6.5 screen, tmux, byobu, dtach . . . 116
4.7 Investigarea proceselor . . . 118
4.8 Anexă: Sistemul de fișiere procfs . . . 119
4.9 Anexă: Internele pornirii unui proces. Loading . . . 120
4.10 Sumar . . . 121

5 Utilizatori 123
5.1 Utilizatori de sistem . . . 124
5.1.1 Permisiuni și privilegii . . . 125
5.2 Utilizatorul administrativ (superuser) . . . 126
5.2.1 Utilizatorul administrativ în Linux (root) . . . 127
5.2.2 Utilitarul su . . . 128
5.2.3 Alternative la root în Linux . . . 128
5.3 Operații cu utilizatori . . . 132
5.3.1 Atribute ale utilizatorilor. Informații despre utilizatori . . . 133
5.3.2 Obținerea de informații despre utilizatori . . . 134
5.3.3 Gestiunea utilizatorilor și grupurilor . . . 136
5.3.4 Gestiunea utilizatorilor și grupurilor în Windows . . . 137
5.4 Autentificare. Gestiunea parolelor . . . 139
5.4.1 Baza de date de parole . . . 139
5.4.2 Gestiunea parolelor . . . 141
5.4.3 Autentificare centralizată . . . 143
5.5 Accesul la sistemul de fișiere . . . 144
5.5.1 Gestiunea permisiunilor în Windows . . . 144
5.5.2 Gestiunea permisiunilor în Linux . . . 146
5.5.3 Comenzi pentru gestiunea permisiunilor . . . 150
5.5.4 Biți speciali de acces: setuid, setgid, sticky . . . 152
5.6 Anexă: Resetarea parolei în Linux . . . 153
5.7 Use case: Resetarea parolei în Windows . . . 155
5.8 Sumar . . . 155

6 Dezvoltarea aplicațiilor 156
6.1 Introducere . . . 156
6.2 Limbaje de programare . . . 156
6.3 Limbaje compilate . . . 159
6.3.1 Exemple de limbaje de programare compilate . . . 162
6.3.2 Pascal . . . 162
6.3.3 Alte limbaje . . . 162
6.4 Limbaje interpretate . . . 163
6.4.1 Exemple de limbaje de programare interpretate . . . 163
6.5 Limbaje hibride . . . 163
6.5.1 Exemple de limbaje de programare hibride . . . 164
6.6 Dezvoltarea unui program . . . 166
6.6.1 Scrierea codului sursă . . . 166
6.6.2 Editoare în mod text . . . 166
6.6.3 Editoare în interfață grafică . . . 167
6.6.4 Medii integrate de dezvoltare (IDE) . . . 168
6.6.5 Biblioteci și framework-uri . . . 170
6.6.6 Rularea programului . . . 170
6.6.7 Automatizarea procesului de dezvoltare . . . 171
6.7 Medii de dezvoltare . . . 171
6.7.1 C / C++ . . . 171
6.7.2 Java . . . 174
6.7.3 C# . . . 175
6.7.4 Python . . . 175

6.7.5 Node.js . . . 176
6.8 Depanarea programelor . . . 177
6.9 Sisteme de management și versionare a codului sursă . . . 178
6.10 Licențe pentru programe . . . 179
6.10.1 GNU GPL . . . 180
6.10.2 GNU LGPL . . . 180
6.10.3 MIT . . . 180
6.11 Sumar . . . 180

7 Interfața în linia de comandă 182
7.1 Shellul. Funcționarea shellului . . . 182
7.1.1 Interacțiunea cu shellul . . . 183
7.1.2 Facilități shell . . . 184
7.1.3 Funcționarea shellului . . . 188
7.1.4 Comenzi interne și comenzi externe . . . 190
7.2 Funcționalități shell . . . 191
7.2.1 Operatori shell . . . 191
7.2.2 Subshelluri . . . 191
7.2.3 Variabile shell . . . 192
7.2.4 Expandări . . . 194
7.2.5 Escaping . . . 197
7.3 Pornirea și personalizarea shellului . . . 198
7.3.1 Configurarea la pornire a shellului Bash . . . 198
7.3.2 Personalizarea promptului . . . 199
7.3.3 Multiplexoare de terminal . . . 200
7.4 Expresii regulate . . . 200
7.5 Prelucrare de text de bază: filtre de text și one linere . . . 202
7.5.1 Generatoare de text . . . 202
7.6 Utilitare avansate . . . 204
7.6.1 find . . . 204
7.6.2 sed . . . 205
7.6.3 awk . . . 206
7.7 Sumar . . . 207

8 Componente hardware 208
8.1 Clasificarea sistemelor de calcul (hardware-ului) . . . 209
8.1.1 Arhitectura von Neumann . . . 209
8.1.2 Arhitecturi de procesor . . . 210
8.1.3 Arhitectura x86 și arhitectura ARM . . . 212
8.1.4 Forme constructive ale hardware-ului . . . 213
8.2 Componentele unui sistem de tip desktop . . . 215
8.3 Interacțiunea hardware - sistem de operare . . . 218
8.3.1 Drivere . . . 219
8.3.2 Controllere . . . 220
8.4 Rolul extensiilor (plăcilor) auxiliare ale unui sistem de calcul . . . 222
8.4.1 Placa video . . . 224
8.4.2 Placa de sunet . . . 225
8.4.3 Placa de rețea . . . 225
8.5 Facilități moderne în hardware . . . 227
8.5.1 Virtualizarea în hardware . . . 227

8.5.2 Trusted Platform Module . . . 227
8.6 Abstractizarea dispozitivelor în Linux . . . 228
8.7 Vizualizarea componentelor hardware pe Linux vs Windows . . . 230
8.7.1 Linux . . . 230
8.7.2 Windows . . . 232
8.7.3 Android . . . 232
8.8 Vizualizarea versiunilor de drivere . . . 235
8.8.1 Vizualizarea driverelor pe Linux . . . 235
8.8.2 Vizualizarea driverelor pe Windows . . . 236
8.9 Sumar . . . 236

9 Pornirea sistemului 238
9.1 Secvența de boot . . . 238
9.2 Firmware de boot . . . 241
9.2.1 BIOS și UEFI . . . 242
9.2.2 Alte firmware-uri de boot . . . 244
9.2.3 Configurare dispozitive bootabile . . . 245
9.2.4 Secure Boot . . . 246
9.3 Dispozitiv de boot . . . 246
9.3.1 Scheme de partiționare: MBR și GPT . . . 247
9.3.2 Încărcarea bootloaderului pentru BIOS și MBR . . . 247
9.3.3 Încărcarea bootloaderului pentru UEFI . . . 248
9.3.4 Denumiri discuri și partiții . . . 248
9.4 Bootloader . . . 249
9.4.1 GRUB . . . 250
9.5 Pornirea kernelului și drivere . . . 253
9.5.1 Kernelul și driverele în Linux . . . 253
9.6 Pornirea init și a serviciilor de startup în Linux . . . 256
9.6.1 Pornirea terminalelor de login . . . 257
9.7 Anexă: Resetarea parolei administrative . . . 259
9.8 Anexă: Crearea unui stick USB bootabil cu Linux . . . 259
9.9 Anexă: Crearea unui stick USB bootabil cu Windows . . . 260
9.10 Anexă: Adăugarea unei intrări GRUB pentru UDPCast . . . 261
9.11 Anexă: Booting în Windows . . . 263
9.12 Concluzii . . . 263

10 Administrarea spațiului de stocare 264
10.1 Tipuri de discuri . . . 266
10.1.1 Hard Disk Drive . . . 266
10.1.2 SSD . . . 268
10.2 Partiționarea dispozitivelor de stocare . . . 269
10.2.1 Sisteme de fișiere . . . 272
10.3 Anexă: Adăugarea unui disc mașinii virtuale . . . 276
10.4 Anexă: Investigarea spațiului de stocare . . . 276
10.5 Anexă: Partiționare, formatare și montare în Linux . . . 280
10.6 Anexă: Backup periodic . . . 285
10.6.1 Backup periodic în Linux . . . 285
10.6.2 Backup periodic în Windows . . . 286
10.7 Anexă: Montarea dispozitivelor . . . 287
10.7.1 Crearea unui disc ce are ca suport un fișier . . . 288

10.8 Anexă: Logical Volume Manager . . . 289
10.9 Anexă: Replicarea datelor . . . 291
10.10 Sumar . . . 293

11 Rețelistică și Internet 295
11.1 Servicii de Internet . . . 297
11.2 Funcționarea Internetului . . . 299
11.2.1 Internet Protocol (IP) . . . 300
11.2.2 Nume în Internet. DNS . . . 301
11.3 Echipamente de rețea . . . 303
11.4 Adresarea IP. Stiva TCP/IP . . . 303
11.4.1 Adresarea IP . . . 305
11.4.2 Adresarea TCP . . . 306
11.4.3 Limita de adrese IP . . . 307
11.5 Configurări de rețea . . . 308
11.5.1 Interfețe de rețea. Investigația configurației . . . 309
11.5.2 DHCP . . . 313
11.5.3 Tipuri de configurare . . . 314
11.6 Configurarea rețelei în Linux . . . 315
11.6.1 Inspectarea configurației . . . 315
11.6.2 Configurarea grafică. NetworkManager . . . 316
11.6.3 Configurarea DNS . . . 316
11.6.4 Configurarea în linia de comandă . . . 317
11.6.5 ifcfg (Fedora / RedHat) . . . 320
11.6.6 Interfețe active/inactive . . . 321
11.6.7 Configurarea temporară. Comanda ip . . . 322
11.6.8 Configurarea DHCP. Comanda dhclient . . . 324
11.6.9 Sumar . . . 324
11.7 Aplicații de rețea. Webul . . . 325
11.7.1 Serviciul web . . . 325
11.7.2 Alte servicii . . . 327
11.8 Anexă: Configurări de rețea folosind suita ifconfig/route . . . 329
11.9 Sumar . . . 331

12 Securitatea sistemului 333
12.1 Fundamentele securității informatice . . . 334
12.1.1 Obiectivele securității . . . 335
12.1.2 Noțiuni de securitate . . . 336
12.1.3 Principii de securitate . . . 338
12.1.4 Modelul subiect-obiect. Permisiuni de acces . . . 340
12.2 Securitatea datelor . . . 341
12.2.1 Securitatea în sistemul de fișiere . . . 342
12.2.2 Confidențialitatea datelor . . . 342
12.2.3 Integritatea datelor . . . 346
12.3 Securitatea accesului. Autentificare . . . 347
12.3.1 Gestiunea parolelor . . . 348
12.4 Securitatea transferului . . . 350
12.4.1 Identitate . . . 351
12.4.2 Semnături digitale. Certificate digitale . . . 351
12.4.3 Transport Layer Security (TLS) . . . 355

12.4.4 Secure Shell . . . 357
12.5 Securitatea proceselor . . . 363
12.6 Sumar . . . 366

13 Automatizarea sarcinilor 367
13.1 Perspective pentru scripting . . . 369
13.2 Dezvoltarea scripturilor . . . 371
13.2.1 „Hello, World!” în scripting și shebang . . . 371
13.2.2 Conținutul unui script shell . . . 372
13.2.3 Depanarea unui script shell . . . 373
13.2.4 Exemple de scripturi shell . . . 373
13.3 Funcționalități pentru scripturi shell . . . 375
13.3.1 Variabile . . . 375
13.3.2 Controlul fluxului . . . 377
13.3.3 Funcții . . . 387
13.4 Facilități avansate și bune practici în shell scripting . . . 388
13.4.1 Robustețe . . . 388
13.4.2 Lizibilitate . . . 391
13.4.3 Eficiență . . . 392
13.4.4 Portabilitate . . . 394
13.4.5 Alte funcționalități . . . 394
13.4.6 Resurse suplimentare . . . 395
13.5 Automatizarea la nivelul sistemelor informatice . . . 395
13.5.1 Gestiunea serviciilor . . . 395
13.5.2 Planificarea sarcinilor . . . 398
13.6 Anexă: Utilitare pentru interacțiunea cu programe interactive . . . 400
13.7 Anexă: Gestiunea configurației . . . 402
13.8 Anexă: Bune practici în automatizare și scripting . . . 402
13.9 Sumar . . . 403

14 Mașini virtuale 404
14.1 Concepte de virtualizare . . . 404
14.1.1 Clasificarea virtualizării . . . 406
14.1.2 Aplicații/hipervizoare pentru rularea mașinilor virtuale . . . 407
14.1.3 Containere (lightweight virtualization) . . . 409
14.2 Operații cu mașini virtuale . . . 410
14.2.1 Configurarea rețelei virtuale . . . 411
14.2.2 Servicii de integrare . . . 412
14.3 Virtualizarea și tehnologiile cloud . . . 412
14.4 Emulare și virtualizare . . . 413
14.5 Anexă: Crearea unei mașini virtuale în VirtualBox . . . 414
14.6 Anexă: OpenStack în UPB . . . 419
14.6.1 Accesarea OpenStack Dashboard . . . 419
14.6.2 Crearea unei perechi de chei SSH . . . 419
14.6.3 Crearea unui keypair . . . 420
14.6.4 Crearea unei mașini virtuale . . . 420
14.6.5 Accesarea mașinii virtuale . . . 422
14.6.6 Ștergerea mașinii virtuale . . . 422
14.7 Anexă: Rularea unui sistem de operare compilat pentru ARM pe x86 . . . 423
14.8 Sumar . . . 424

15 Sisteme încorporate **425**
15.1 Microcontrolere și calculatoare 426
15.1.1 Microcontrolere 426
15.1.2 Calculatoare integrate 428
15.2 Sisteme simple de intrare ieșire 431
15.3 Magistrale de comunicare 434
15.3.1 Comunicare prin fir 436
15.3.2 Comunicarea wireless 438
15.3.3 Sisteme Gateway 439
15.4 Dezvoltarea programelor pentru sisteme integrate 439
15.5 Internet of Things 440
15.5.1 Implementări de sisteme IoT 442
15.5.2 Edge/Fog Computing 443
15.5.3 Securitatea IoT 444
15.6 Sumar 444

Bibliografie **445**

Abrevieri

ACL – Access Control Lists
ACPI – Advanced Configuration and Power Interface
AD – Active Directory
ADC – Analog to Digital Converter
BCD – Boot Configuration Data
BIOS – Basic Input/Output System
BLE – Bluetooth Low Energy
CAN – Controller Area Network
CI – Continuous Integration
CISC – Complex Instruction Set Computing
CMOS – Complementary Metal-Oxide-Semiconductor
CPU – Central Processing Unit
CSM – Compatibility Support Module
CTF – Capture the Flag
DAC – Digital to Analog Converter
DHCP – Dynamic Host Configuration Protocol
DMA – Direct Memory Access
DTB – Device Tree Blob
EDVAC – Electronic Discrete Variable Automatic Computer
ELF – Executable and Linking Format
ESP – EFI System Partition
FAT32 – File Allocation Table 32
FSB – Front-Side Bus
GB – gigabytes
GCC – GNU Compiler Collection
GNU – GNU's Not Unix
GNU GPL – GNU General Public License
GNU LGPL – GNU Lesser General Public License
GPIO – General Purpose Input Output
GPT – GUID Partition Table
GPU – Graphics Processing Unit
GRUB – GRand Unified Bootloader
GUI – Graphical User Interface
HDD – Hard Disk Drive
HTTP – Hypertex Transfer Protocol
I/O – Input/Output
IDE – Integrated Development Environment
IFS – Input Field Separator
Inter-Integrated Circuit – ,

IoT – Internet of Things
IP – Internet Protocol
ISA – Instruction Set Architecture
IT – Information Technology
IT&C – Information Technology and Communications
JDK – Java Development Kit
JIT – just-in-time
JRE – Java Runtime Environment
KVM – Kernel Virtual Machine
LDAP – Lightweight Directory Access Protocol
LED – Light-Emitting Diode
LVM – Logical Volume Manager
LXC – Linux Containers
MB – megabytes
MBR – Master Boot Record
MHz – megaherți
MISO – Master In Slave Out
MIT – Massachusetts Institute of Technology
MOSI – Master Out Slave In
MU – Memorty Unit
NAS – Network Attached Storage
NTFS – New Technology File System
PC – Personal Computer
PCI – Peripheral Component Interconnect
PDF – Portable Document Format
PID – Process Id
POSIX – Portable Operating System Interface
POST – Power-On Self Test
PWC – Pulse Width Modulation
PXE – Preboot eXecution Environment
QEMU – Quick Emulator
RAID – Redundant Array of Independent / Inexpensive Disks
RAM – Random Access Memory
RISC – Reduce Instruction Set Computing
ROM – Read-Only Memory
SAM – Security Account Manager
SAS – Serial attached SCSI
SATA – Serial Advanced Technology Attachment
SFP – Small form-factor pluggable transceiver
SPI – Serial Peripheral Interface
SSD – Solid State Drive
SSH – Secure shell
TB – terabytes
TPM – Trusted Platform Module
UAC – User Acount Control
UEFI – Unified Extensible Firmware Interface
UID – User Id
USB – Universal Serial Bus
UUID – Universally Unique Identifier

VMM – Virtual Machine Monitor

Capitolul 0

Introducere

Viața celor mai multora dintre noi este strâns legată de tehnologie. Avem peste tot în jur și folosim sisteme informatice; cu ajutorul acestora ne informăm, comunicăm, ducem la bun sfârșit activități profesionale și personale.

Un sistem informatic este compus din sisteme de calcul, infrastructură și date. Sistemele de calcul sunt echipamentele cu putere de procesare pe care le folosim direct sau indirect (servere, laptopuri, telefoane mobile); infrastructura înseamnă rețeaua, sistemele de stocare, interfețele de acces.

Acest elemente formează lumea informației digitale, sau lumea IT & C (Information Technology and Communication). Este lumea care definește revoluția tehnologică a ultimelor decenii, lumea care definește funcționarea majorității domeniilor: economie, politică, educație, societate.

Oricare dintre noi este un utilizator de tehnologie informatică. În general, facem acest lucru prin intermediul dispozitivelor și sistemelor de calcul personale: sistem desktop, laptop, smartphone, smart TV, smartwatch. Un sistem de calcul (*computer system*) sau un dispozitiv de calcul este o componentă fizică ce poate prelua, prelucra și reda informație. De exemplu, un smart TV primește comenzi prin telecomandă, preia conținut video de pe Netflix, îl prelucrează și apoi îl redă utilizatorului pe ecran. Un laptop preia acțiuni de la utilizator ce pot rezulta în pornirea unei aplicații de tip browser web și accesarea platformei Facebook pe care utilizatorul să o urmărească.

Un utilizator cu profil tehnic, care dorește o utilizare mai eficientă sau mai sigură a sistemului de calcul, sau care dorește personalizarea acestuia, va urmări să înțeleagă dedesubturile funcționării acestuia.

Această carte este destinată înțelegerii funcționării unui sistem de calcul din perspectiva unui utilizator cu profil tehnic, accentul fiind pus pe sistemul de operare, o componentă software esențială a sistemului de calcul. Majoritatea noțiunilor prezentate sunt comune tuturor sistemelor de calcul și sistemelor de operare; însă ne vom concentra pe sisteme de calcul de tipul desktop / laptop și pe sistemul de operare Linux.

0.1 Sisteme de calcul

Așa cum am precizat mai sus, un sistem de calcul este un echipament care preia informații, le prelucrează și le livrează mai departe. Îl mai numim și calculator. În engleză termenul este de *computer/computing system* sau *computer*. Definiția este destul de relaxată pentru a include un număr mare de dispozitive: de la servere cu mii de procesoare și putere de prelucrare imensă până la dispozitive portabile precum smartwatch-uri. În general, pe parcursul acestei cărți vom folosi interschimbabil noțiuni precum sistem de calcul, dispozitiv de calcul, dispozitiv, platformă de calcul, echipament.

Pe parcursul unei zile putem folosi mai multe sisteme de calcul din posesia noastră sau a altora:

- avem pe birou un sistem desktop în care dezvoltăm aplicații
- avem o consolă pentru jocuri
- avem un smart TV pe care urmărim cele mai recente episoade din serialul preferat
- avem un ruter Wi-Fi cu care avem acces la Internet
- avem un laptop pe care navigăm pe Internet, lucrăm cu documente, testăm aplicații
- avem un dispozitiv mobil inteligent (*smartphone*) cu care ascultăm muzică, discutăm cu cei apropiați, accesăm platforme de socializare
- avem un asistent activat vocal (*smart speaker*, precum Google Home sau Amazon Alexa) care poate fi comandat pentru a furniza informații, pentru a reda o melodie, pentru a consulta calendarul
- avem un smartwatch pe care avem informații despre starea de sănătate, avem remindere și avem acces la calendar și informații despre vreme
- avem un computer de bord în mașină cu care ascultăm muzică din telefon, cu care accesăm hărți pentru navigație
- accesul pe platforme de hărți sau de socializare sau de conținut video sau audio înseamnă folosirea Internet-ului și a unor sisteme de calcul de tip server aflate la distanță într-un data center întreținut de companii precum Google, Amazon sau Facebook

Aceste sisteme de calcul sunt folosite de majoritatea utilizatorilor, indiferent de pregătirea lor tehnic. Un utilizator cu profil tehnic tehnic, care lucrează sau este pasionat de lumea calculatoarelor (IT&C) va folosi și alte sisteme de calcul. Iar folosirea unui sistem de calcul duce, prin Internet, la folosirea altor sisteme de calcul.

Un sistem de calcul este compus din hardware (componente fizice) și software (programe sau aplicații). Cele două componente funcționează împreună; hardware-ul oferă puterea de calcul în vreme ce software-ul oferă o interfață flexibilă și prietenoasă utilizatorului, ca în Figura 1.

Cea mai importantă caracteristică a software-ului este flexibilitatea. Se poate configura o aplicație existentă pentru a funcționa diferit sau se pot instala noi aplicații. Nu este nevoie de o schimbare de echipament cum ar fi în cazul hardware-ului.

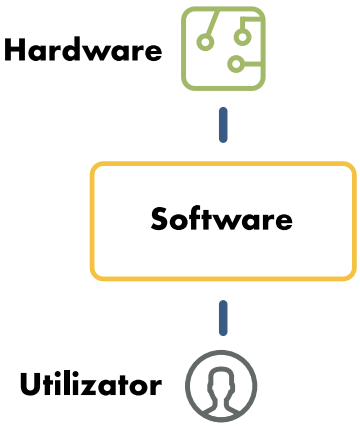


Figura 1: Sistemul de calcul și utilizatorul

În general, spunem că un sistem de calcul este compus din hardware și software-ul necesar utilizării de bază. Alte componente software ce vor fi adăugate ulterior de utilizator nu fac parte din sistemul de calcul. Din nou, definiția este relaxată, nu există o graniță clară între ce componente software sunt necesare utilizării de bază și ce componente software sunt necesare unei utilizări specifice.

Componenta esențială a software-ului unui sistem de calcul, și subiectul principal al acestei cărți, este sistemul de operare. Sistemul de operare gestionează resursele fizice/hardware ale sistemului de calcul și facilitează folosirea acestora de aplicații. Figura 2 prezintă organizarea unui sistem de calcul, cu aplicațiile rulând peste sistemul de operare.

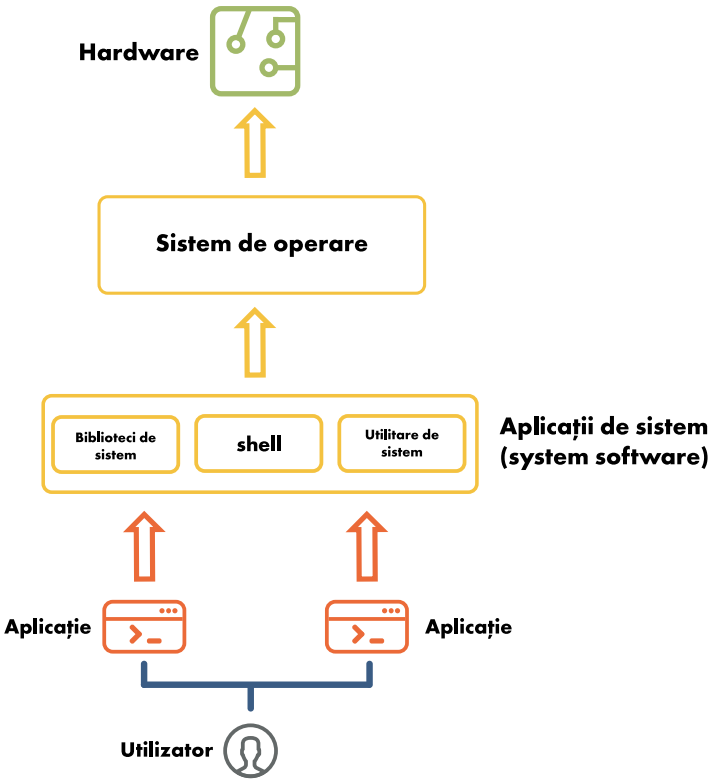


Figura 2: Componentele software într-un sistem de calcul

O categorie aparte de aplicații sunt aplicațiile de sistem (*system programs*, *system software*). Aceste componente software oferă suport suplimentar celorlalte aplicații; o

aplicație de sistem importantă este shellul. Shellul oferă utilizatorului interfața cu care acesta să poată porni și interacționa cu alte aplicații. Acesta poate fi grafic sau în linie de comandă. Mai multe despre shell și interfața cu utilizatorul vom discuta în Capitolul 1.

În continuare vom detalia sistemul de operare.

0.2 Sistemul de operare

Un sistem de operare este un set de programe folosite pentru a gestiona resursele hardware ale sistemului. Sistemul de operare ascunde complexitatea hardware-ului de aplicații. Un dezvoltator de aplicații nu este preocupat de diferențele hardware între diferitele platforme, ci doar de interfața pe care o oferă sistemul de operare.

Spunem că sistemul de operare îndeplinește următoarele roluri:

- abstractizare hardware: sistemul de operare oferă o interfață pentru aplicații; indiferent de specificul hardware, interfața va fi aceeași
- portabilitate: aceeași aplicație (de pildă browserul web Firefox sau editorul Vim) poate rula nemodificată pe același sistem de operare aflat pe configurații hardware diferite ale sistemului (cu procesoare sau discuri sau plăci de rețea diferite)
- mediere: sistemul de operare asigură accesul echitabil al mai multor aplicații la resursele hardware; dacă două aplicații folosesc simultan Internetul, sistemul de operare va transfera pe rând informațiile către/de la acestea folosind placa de rețea a sistemului, având în vedere să nu amestece fluxurile de transfer
- securitate/izolare: fiecare aplicație rulează izolat de celelalte; o aplicație nu va folosi neautorizat și nu va corupe, accidental sau cu intenție, resursele altei aplicații

Unele dispozitive nu au sistem de operare, acestea fiind în general, dispozitive cu rol specific (*embedded systems*) care rulează o singură aplicație. Exemplu de astel de dispozitive sunt un proiector, un bec inteligent, un senzor de lumină sau de temperatură. Dispozitivele și sistemele de calcul au însă un sistem de operare, sau o componentă software alternativă.

Componenta centrală a sistemului de operare, cea care gestionează și abstractizează resursele hardware este **nucleul sistemului de operare**, numit și **kernel**. Nucleul sistemul de operare este o componentă critică, coruperea ducând la coruperea întregului sistem. Nucleul abstractizează resursele hardware oferind aplicațiilor o interfață numită interfața de apeluri de sistem (*system call interface*), la fel ca în Figura 3. O aplicație care are nevoie de acces la resursele hardware sau o anumită funcționalitate implementată de sistemul de operare va invoca un apel de sistem prin interfața de apeluri de sistem.

Utilizatorul nu este preocupat de obicei de sistemul de operare și nu interacționează cu acesta. În afară de câteva informații de forma „pe laptop am Windows” sau „am telefon cu Android”, utilizatorul nu are nevoie să afle mai multe. Lucrurile se schimbă pentru un utilizator cu profil tehnic; pentru acesta este interesant să înțeleagă sistemul de operare și particularitățile sale pentru a putea folosi mai eficient, mai sigur sau mai pe gustul său sistemul de calcul.

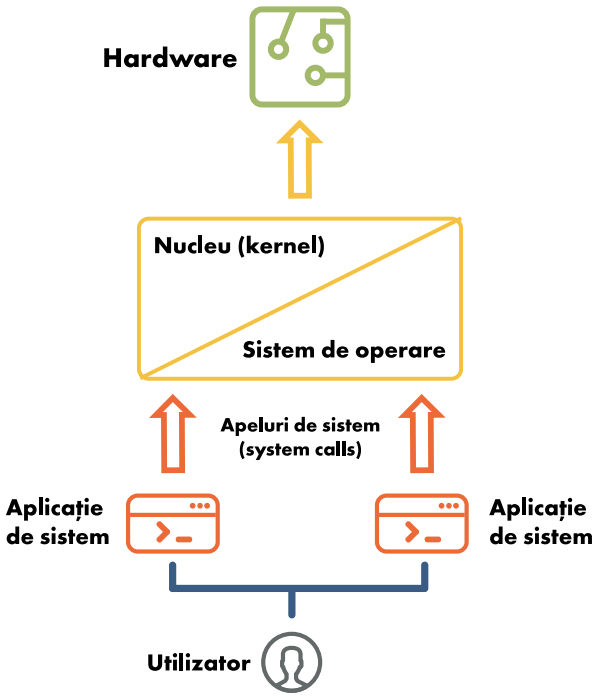


Figura 3: Interfața de apeluri de sistem

Din punctul de vedere al utilizării, putem considera următoarea clasificare a sistemelor de operare:

- sisteme de operare generaliste, folosite pe sisteme server, desktop sau laptop: Microsoft Windows, Apple macOS, distribuții GNU/Linux, FreeBSD
- sisteme de operare pentru dispozitive mobile: Android, Apple iOS
- sisteme de operare specializate: pentru echipamente de rețea (Cisco iOS, Juniper Junos OS, DD-WRT), pentru smartwatch-uri (Apple watchOS, Wear OS), pentru smart TV (Tizen, webOS, Apple tvOS), pentru console de jocuri (Sony Orbis OS, Microsoft Xbox OS, Nintendo Horizon) etc.

Clasificarea este una evolutivă, nu strictă. Pot fi adăugate tipuri noi pe măsură ce anumite dispozitive apar. Dacă am fi făcut o clasificare similară în 2005, de exemplu, a doua categorie (sisteme de operare pentru dispozitive mobile) nu ar fi existat.

E de remarcat că sistemele de operare sunt legate între ele deși rulează pe sisteme de calcul radical diferite. De exemplu, GNU/Linux, Android, DD-WRT, Wear OS, Tizen, webOS au la bază kernelul Linux; adică sunt sisteme de operare derivate din același kernel. La fel este cazul produselor Apple (macOS, iOS, watchOS, tvOS): au la bază sistemul de operare Darwin. Pentru un utilizator tehnic, cunoașterea unui sistem de operare înseamnă că va avea o bază de cunoștințe pentru diferitele platforme pe care acesta rulează.

0.2.1 Alegerea sistemului de operare

Alegerea unui anumit sistem de operare ține de mai mulți factori, important fiind ce urmărește utilizatorul și pe ce sistem de calcul va rula sistemul de operare. Cu toate acestea, pentru o bună parte din dispozitive, întrebarea apare rar sau deloc. De

exemplu, pentru un cumpărator de smart TV sau de consolă de jocuri nu va conta sistemul de operare în alegerea produsului, va conta produsul în sine. Puțini utilizatori sunt preocupați de sistemul de operare care rulează pe ruterul lor wireless, e important să funcționeze corespunzător. În general, întrebări și alegeri au loc în cazul sistemelor de operare generaliste (server/desktop/laptop) și a dispozitivelor mobile inteligente (smartphone).

În cazul dispozitivelor mobile inteligente (smartphones), cele mai răspândite sisteme de operare sunt Apple iOS și Android. Alegerea între cele două ține de mai multe criterii. Internetul abundă de comparații între cele două. În plus, în vreme ce Apple controlează dispozitivele și sistemul de operare de pe iPhone, ecosistemul Android este mult mai vast. Cineva poate alege un anumit telefon cu Android ținând cont mai mult de diferențele hardware decât de cele software (mai reduse). Un sumar al avantajului fiecărei platforme este mai jos:

- Android: Dispozitivele Android sunt mai diverse și, în general, mai ieftine. Un utilizator poate decide ce telefon să își achiziționeze dintr-o plajă largă de opțiuni. Aplicațiile Android se pot dezvolta pe orice platformă folosind, uzual, mediul Android Studio, în vreme ce aplicațiile iOS se dezvoltă doar pe macOS folosind mediul Xcode.

- iOS: Dispozitivele iPhone, care rulează iOS, sunt produse uniform de Apple și în general, dispun de o mai bună integrare a componentelor software/hardware. Ecosistemul Apple oferă avantaje: o integrare foarte bună cu alte componente și tehnologii Apple (macOS, tvOS, iCloud, iTunes), versiuni puține și stabile de sisteme de operare.

Sistemele de operare generaliste rulează pe sisteme de calcul puternice, care cuprind o unitate de calcul și periferice precum monitor, tastatură, mouse. Aceste sisteme pot fi servere, folosite pentru prelucrări de date în general de organizații, sau sisteme desktop sau laptop, folosite de utilizatori obișnuiți. Dintre sistemele de operare generaliste cele mai cunoscute și folosite sunt Microsoft Windows, Apple macOS și GNU/Linux.

La fel ca în cazul Android/iOS, pe Internet sunt numeroase articole comparative între cele trei sisteme de operare generaliste de mai sus, cu avantajele și dezavantajele fiecăruia. De avut în vedere că migrarea tot mai multor aplicații în zona web înseamnă că diferențele dintre sisteme de operare sunt din ce în ce mai reduse. Adesea un utilizator se va descurca la fel de bine și pe Windows și pe macOS și pe Linux pentru că va petrece o bună parte din timp folosind un browser web, care oferă o interfață relativ comună. Mai jos sumarizăm avantajele fiecăruia și cazurile frecvente de utilizare:

- Microsoft Windows: Este cel mai răspândit sistem de operare pe sistemele desktop, multe sisteme desktop/laptop venind cu Microsoft Windows preinstalat. Are integrare cu produsele enterprise Microsoft, folosite în companii și organizații mari. Cele mai multe aplicații dezvoltate rulează pe Windows. Jocurile sunt dezvoltate în special să ruleze pe Windows.

- Apple macOS: Vine preinstalat pe dispozitive produse de Apple (MacMini, MacBook) și vine preconfigurat. Oferă un mediu de lucru mai aproape de Linux pentru dezvoltatorii tehnici. În general urmărește utilizabilitate, atât la nivel software și interfața cât și la nivel hardware: greutate laptop, aspect, tastatură, durată baterie.

- GNU/Linux: Este cel mai răspândit sistem de operare pe sistemele server, fiind fiabil și cu multe aplicații pentru dezvoltatori și administratori. Oferă o plajă diversă de distribuții, poate fi personalizat și configurat. Este un sistem open source, adică este accesibil codul sursă, un foarte bun mod de învățare și de participare în cadrul comunității de dezvoltatori.

Un utilizator va opta pentru folosirea unui sistem și în funcție de preferințe personale și de nevoie; de exemplu, poate nu este mulțumit cu un anumit sistem de operare, dar o anumită aplicație rulează numai pe acela; sau la locul de muncă i se cere să folosească un anumit sistem de operare.

Poate apărea situația în care un utilizator are nevoie să folosească 2 sisteme de operare, pentru nevoi sau preferințe diferite. În această situație are două opțiuni:

1. dual-install/dual-boot: pe același sistem de calcul instalează două sisteme de operare diferite și pornește alternativ unul sau altul. Detalii tehnice prezentăm în Capitolul 9.
2. mașină virtuală: instalează un sistem de operare într-o mașină virtuală și folosește acea mașină virtuală din sistemul de operare principal. Detalii tehnice prezentăm în Capitolul 14.

În general, un sistem desktop sau laptop vine cu un sistem de operare preinstalat. Utilizatorul are posibilitatea instalării altui sistem de operare, a instalării dual-boot sau a instalării unei mașini virtuale. Imediat după instalare, utilizatorul va face configurațiile de bază: crearea unui cont de utilizator, configurarea datei, schemei de tastatură, personalizarea interfeței grafice. Apoi va folosi sistemul în scopul dorit, folosind aplicații deja existente sau instalând aplicații noi.

În această carte vom prezenta noțiuni conceptuale ce se aplică în general, pe toate tipurile de sisteme de calcul și sisteme de operare. Pentru aspecte demonstrative, vom folosi ca suport Linux pe desktop. Aceasta pentru că sistemul de operare și distribuțiile bazate pe Linux, fiind open source, oferă acces rapid și deschis la resursele interne ale sistemului. În Linux, ca utilizator cu profil tehnici, putem investiga ușor procese, fișiere, procesul de boot, sistemul de operare, aspecte de securitate. Și putem face parte din comunitățile open source de dezvoltatori din lumea Linux.

0.3 Lumea Linux

Din punct de vedere tehnic, numele Linux este numele nucleului sistemului de operare, numele kernelului. În limbaj comun însă, când spunem Linux, ne referim la sistemul de operare și aplicațiile care rulează pe un sistem de calcul.

Linux nu rulează doar pe sisteme desktop și servere. Android are la bază nucleul Linux. Foarte multe smart TV-uri folosesc sisteme de operare bazate pe Linux. Rutere wireless folosesc Linux. Alte tipuri de dispozitive precum interfața de comandă a unei mașini inteligente, precum un ceas inteligent sau precum un asistent activat vocal (smart speaker) folosesc Linux.

Răspândirea Linux la toate nivelurile de dispozitive se datorează codului sursă deschis al nucleului Linux și al aplicațiilor din lumea Linux. Codul sursă deschis (*open source*),

numit și software liber (*free software*), înseamnă că acest cod este dezvoltat și distribuit printr-o licență software specifică, licență care oferă oricui acces la implementarea programelor și posibilitatea să modifice acele programe și să contribuie cu aceste modificări la îmbunătățirea lor. În acest fel multe aplicații din lumea Linux, inclusiv nucleul Linux, sunt create de comunități de dezvoltatori care de obicei colaborează virtual, cu ajutorul Internetului. O companie poate prelua codul sursă al acestor aplicații, le poate împacheta și livra împreună cu dispozitivele sale. Este cazul dispozitivelor ce rulează Android sau a smart TV-urilor cu nucleu Linux.

Aplicații care au cod sursă deschis nu sunt apanajul lumii Linux. Apple are componente software open source, la fel și Microsoft.

Lumea Linux este centrată pe componente software open source, este formată din comunități deschise de dezvoltatori, testeri, proiectanți, administratori etc. și oferă resurse și suport de documentare pentru oricine. Este lumea ideală pentru suport tehnic în educație, pentru a obține cunoștințe tehnice despre sisteme de calcul și sisteme de operare. De aceea, în această carte vom folosi Linux ca principal suport în demonstrarea conceptelor prezentate.

0.3.1 Linux și Unix

Adesea vom întâlni termenul de Unix sau sisteme de operare bazate pe Unix sau comenzi Unix. Deși tehnic cele două nume sunt diferite, pentru majoritatea conceptelor pot fi folosite interschimbabil.

Unix este folosit ca termen umbrelă pentru familia sistemelor de operare care au arhitectura software și modul de utilizare similare cu sistemul de operare UNIX (de obicei scris cu majuscule) creat la Bell Labs în anii 1970 de Dennis Ritchie și Ken Thompson. UNIX-ul creat atunci a cunoscut o evoluție constantă iar ideile sale au fost preluate în alte sisteme de operare. De exemplu sistemele de operare din familia BSD (FreeBSD, NetBSD, OpenBSD, DragonFly BSD) au la bază cod sursă din versiunile vechi de Unix. Sistemul de operare Darwin folosit de macOS se bazează pe Unix și este cod derivat din familia BSD.

Linux este un sistem de operare bazat pe Unix dar scris de la zero. Este un proiect software pornit în 1991 de Linus Torvalds căruia apoi i s-au alăturat o comunitate de dezvoltatori din Internet. Numele Linux vine de la prenumele creatorului (Linus) căruia i s-a adăugat celebrul sufix **x** care marchează apartenența la Unix.

Linux a cunoscut o dezvoltare continuă din momentul creării sale ajungând să fie prezent acum pe o plajă largă de dispozitive. Linux este dezvoltat actualmente ca produs open source de o comunitate extinsă de dezvoltatori din Internet. Mulți dintre aceștia lucrează la companii mari (precum Intel, Samsung, IBM, Microsoft) pentru a se asigură că echipamentele și aplicațiile acestora funcționează bine împreună cu Linux.

0.3.2 GNU/Linux. Distribuții

Denumirea Linux se referă doar la nucleul sistemului de operare (kernel). Pentru a putea funcționa corespunzător, un sistem de operare are nevoie de software de bază și de

aplicații pentru a rula: un shell, browser web, player multimedia, suită de utilitare pentru dezvoltare etc. Multe dintre aceste aplicații provin din proiectul GNU[1], un proiect pornit de Richard Stallman în anii '80 pentru a crea o alternativă deschisă la Unix. Întrucât nucleul lipsea, Linux a îndeplinit acest rol. De aceea, sistemele de operare și suitele software formate din nucleul Linux și aplicațiile din proiectul GNU poartă denumirea de distribuții GNU/Linux.

O distribuție este un cumul de componente software care sunt instalate pe un sistem de calcul și apoi sunt folosite. O distribuție GNU/Linux folosește nucleul Linux, aplicații din proiectul GNU și alte aplicații din alte proiecte. O denumire corectă ar fi distribuție GNU/Linux/others. Din motive de expeditivitate, adesea, inclusiv în această carte, vom folosi termenul distribuție Linux.

O distribuție diferă de alte distribuții prin aplicațiile pe care le conține, modul în care pot fi gestionate aceste aplicații (instalate, dezinstalate, configurate), configurații ale sistemului (denumiri de fișiere, versiuni), hardware-ul pentru care se oferă suport și interfață. În Figura 4[2] sunt screenshot-uri din patru distribuții renumite: Ubuntu, Fedora, Arch, Mint.

Fiind una diversă și descentralizată, lumea Linux are multe distribuții. Alegerea unei distribuții ține de factori obiectivi (consum de resurse, hardware pe care rulează, aplicații prezente, personalizare) dar și de unii personali (ce am folosit, ce mi-a plăcut, ce folosesc prietenii mei). Deși interfața poate diferi, elementele software din spate sunt aceleași, așa că un utilizator tehnic nu va avea probleme mari în a se acomoda cu o distribuție nouă.

Distribuțiile Linux pot fi clasificate în familii de distribuții. O familie are la bază o distribuție (care cuprinde anumite aplicații, un mod propriu de gestiune a aplicațiilor, configurări particulare) din care sunt derivate alte distribuții. De exemplu, familia RedHat are la bază distribuția RedHat din care sunt derivate Fedora, CentOS. Familia Debian are la bază distribuția Debian din care sunt derivate Ubuntu și Mint.

Descrieri și comparații între distribuții, argumente pentru a alege una găsiți în număr mare pe Internet și pe site-ul DistroWatch[3]. Un sumar al celor mai cunoscute distribuții Linux sunt în Tabelul 1.

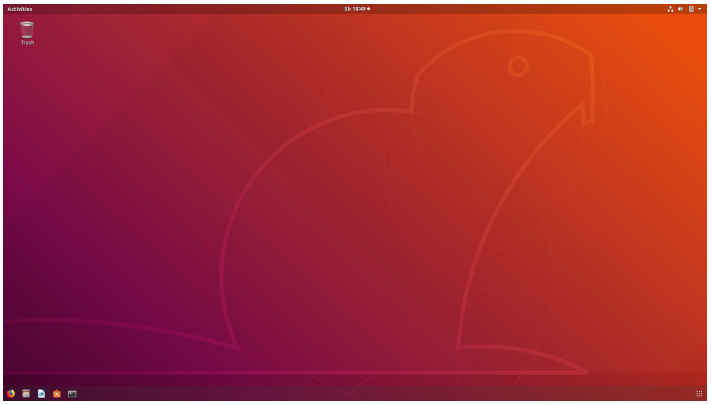
0.3.3 Mașina virtuală de suport

O dată cu această carte oferim o mașină virtuală de suport pe care rulăm părțile aplicative pe care le prezentăm. Mașina virtuală de suport se poate descărca de la adresa TODO și rulează distribuția Ubuntu 18.04. Am optat pentru această distribuție pentru că este printre cele mai cunoscute și pentru că familia Debian cuprinde un număr mare de distribuții (printre care și Ubuntu). Șansele să vă întâlniți cu o distribuție din familia Debian sunt foarte mari.

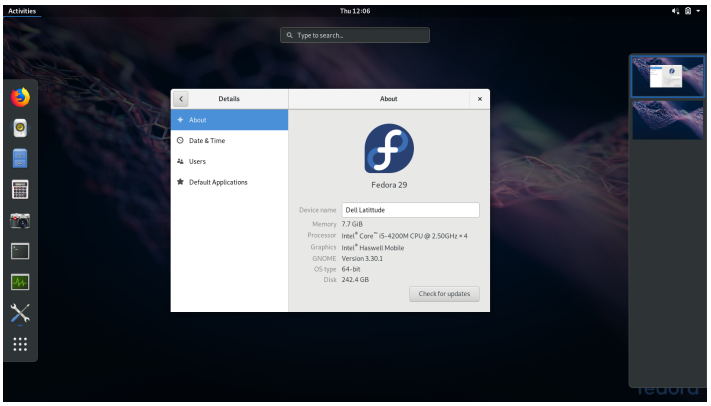
[1] https://www.gnu.org/

[2] Fedora: https://commons.wikimedia.org/wiki/File:Fedora_29_(2018,_10)_running_GNOME_Shell_3.30_(2018,_09)_under_Wayland.png (CC BY-SA 4.0)
Arch: https://en.wikipedia.org/wiki/File:KDE_Arch.png (CC BY-SA 4.0)
Mint: https://commons.wikimedia.org/wiki/File:Screenshot_from_linux_mint_18_start_menu.png (CC BY-SA 4.0)

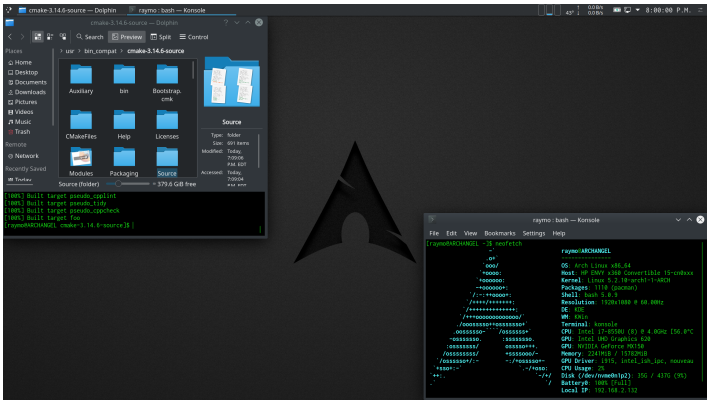
[3] https://distrowatch.com/



(a) Ubuntu



(b) Fedora



(c) Arch

(d) Mint

Figura 4: Distribuții Linux

Tabelul 1: Distribuții Linux

Distribuție	Site	Pachete	Interval de lansare
Debian	http://www.debian.org	`.deb` (`apt`, `dpkg`)	nefixat
Ubuntu	http://www.ubuntu.com	`.deb` (`apt`, `dpkg`)	6 luni (aprilie și octombrie)
Fedora	https://getfedora.org/	`.rpm` (`dnf`, `rpm`)	6 luni (aprilie și octombrie)
Mint	http://linuxmint.com	`.deb` (`apt`, `dpkg`)	nefixat
Arch	http://www.archlinux.org/	`.deb` (`pacman`)	continuu (*rolling*)
CentOS	http://www.centos.org/	`.rpm` (`dnf`, `rpm`)	aliniat cu Red Hat Enterprise Linux, nefixat
Kali	http://www.debian.org	`.deb` (`apt`, `dpkg`)	continuu (rolling)

Mașina virtuală poate fi descărcată și pornită în soluțiile de virtualizare VirtualBox sau VMware. Contul de utilizator este `student` cu parola `student`.

0.3.4 Instalarea Linux

Spre deosebire de Windows sau macOS, puține sisteme desktop sau laptop vin cu Linux preinstalat. Așa că majoritatea utilizatorilor vor trebui să instaleze Linux. Așa cum am amintit și mai sus, Linux poate fi instalat direct pe sistemul fizic (singur sau dual-boot cu Windows) sau într-o mașină virtuală. Instalarea pe sistemul fizic are avantajul performanței, pe când cel în mașina virtuală avantajul ușurinței în instalare și a flexibilității.

Nu vom insista pe instalarea Linux. Din nou, resursele din Internet acoperă copios acest subiect. O precizare este că, la fel ca orice alt sistem de operare, este nevoie de un mediu de instalare: un CD-ROM/DVD-ROM bootabil sau, mai probabil, un stick USB bootabil. Astfel, pentru a instala Linux, un utilizator va urma pașii:

1. Va descărca un fișier imagine ISO de pe Internet conținând imaginea distribuției Linux care va fi instalată. De exemplu, de la adresa http://releases.ubuntu.com/18.04/ se descarcă fișierul ISO pentru Ubuntu 18.04.

2. Va crea un mediu bootabil de la fișierul imagine ISO. Va folosi o aplicație de tip ISO image burner ca să scrie imaginea pe CD-ROM/DVD-ROM sau o aplicație precum Unetbootin pentru a scrie imaginea pe un stick USB.

3. Va porni (boota) sistemul de tip desktop/laptop de pe mediul bootabil creat mai sus.

4. Va realiza instalarea efectivă. Pașii sunt destul de simpli, majoritatea putând fi realizați prin apăsarea butoanelor de tip *Next* în ecranul de instalare.

0.4 Sumar

Avem în jur și folosim sisteme informatice compuse din sisteme de calcul interconectate.

Un sistem de calcul este compus din hardware și software. Sisteme de calcul sunt atât laptopuri și dispozitive personale câte și dispozitive de rețea, componente în mașini, smart TV-uri.

Sistemul de operare e componenta software centrală ce asigură accesul utilizatorului (prin aplicațiilor software) la resursele hardware.

În această carte folosim Linux, un sistem de operare open source cu o comunitate largă și care facilitează dobândirea de competențe și cunoștințe tehnice.

Din punct de vedere istoric, Linux este un sistem de operare din familia Unix.

De obicei folosim Linux pe sisteme desktop în forma unei distribuții GNU/Linux ce include sistemul de operare, aplicațiile și interfața. Similar Linux, majoritatea componentelor software sunt open source.

Mașina virtuală de suport a acestei cărți rulează distribuția Ubuntu 18.04.

Linux poate fi instalat lângă Windows în mod dual-boot sau poate fi instalat într-o mașină virtuală, precum cea pusă la dispoziție ca suport al acestei cărți.

Capitolul 1

Interfața cu utilizatorul

Un sistem de calcul oferă resurse de calcul utilizatorului. Utilizatorul folosește o interfață de lucru furnizată de sistemul de calcul pentru activitățile sale: navigare pe Internet, dezvoltare de aplicații, creare de conținut, scriere de documente. Interfața cu utilizatorul este oferită de o aplicație a sistemului de operare numită **shell**. Prin interacțiunea cu shellul utilizatorul își realizează activitățile și folosește resursele sistemului de calcul. De exemplu un shell grafic va prezenta utilizatorului butoane, meniuri și ferestre pe care acesta le va folosi; un shell text va prezenta utilizatorului un prompt unde acesta va introduce comenzi. Cu aceste elemente (butoane, meniuri, ferestre, comenzi), utilizatorul va interacțiunea cu sistemul de operare și cu aplicațiile.

Figura 1.1 prezintă legătura dintre componentele sistemului de calcul. Componentele hardware sunt gestionate de sistemul de operare; peste sistemul de operare rulează aplicații, una dintre aplicații fiind shellul, aplicația care interfațează utilizatorul cu celelalte aplicații, cu serviciile sistemului de operare și resursele hardware.

Majoritatea aplicațiilor au la rândul lor o interfață. De exemplu jocurile pe calculator au o interfață grafică unde utilizatorul/jucătorul își gestionează resurse, creează unități, navighează harta; un browser web are o interfață în care se introduce adresa unui site și are zonă de redare a componentelor paginii web (text, butoane, imagini); aplicația Matlab, pentru calcule matematice complexe, oferă un prompt în care dezvoltatorul introduce comenzi specifice. În general, numim shell aplicația care mediază interacțiunea utilizatorului cu sistemul de operare; într-un sens mai larg, putem spune și că aplicațiile de mai sus (jocul, browser-ul web, Matlab) oferă un shell, acea interfață.

Interfețele cu utilizatorul, shellurile, vin de obicei în două moduri: **shelluri grafice** (*Graphical User Interface* - GUI) și **shelluri text/linie de comandă** (*Command Line Interface* - CLI). În general, interfața grafică este mai intuitivă, are avantajul ușurinței în utilizare și acomodare, un aspect mai plăcut, ușor personalizabil. Interfața în linia de comandă este ceva mai dificil de învățat și de folosit, mai anostă ca aspect, dar oferă acces mai complet și mai detaliat la serviciile sistemului. Interfața în linia de comandă permite automatizarea prin scripturi, lucru pe care îl vom discuta în Capitolul 13.

În Figura 1.2 avem două screenshoturi: unul cu un shell GUI și unul cu shell CLI din distribuția Ubuntu 18.04. Shellul GUI oferă meniuri, butoane și ferestre, în vreme ce shellul CLI oferă un prompt unde se pot introduce comenzi. Shellul GUI este, în fapt, interfața grafică pe care o vedem într-o instalare obișnuită Ubuntu.

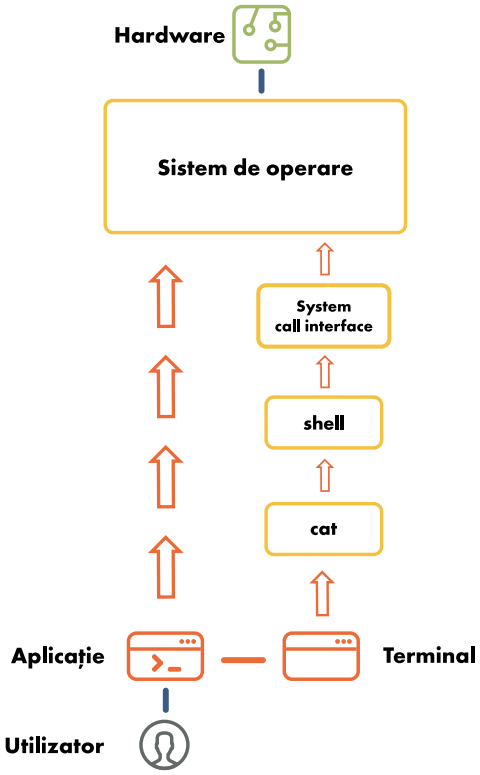
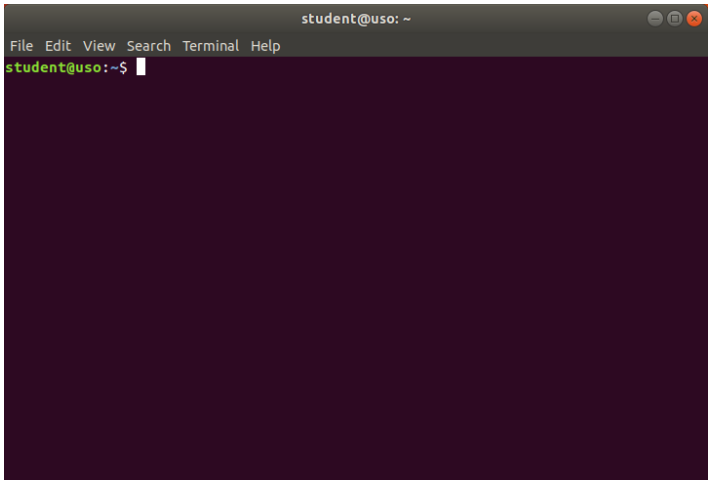


Figura 1.1: Componentele sistemului de calcul

(a) Shell GUI



(b) Shell CLI

Figura 1.2: Shell GUI și CLI în Ubuntu 18.04

În capitolul curent ne vom concentra pe interfața grafică. Interfața în linia de comandă o vom discuta detaliat în Capitolul 7.

1.1 Interfața grafică

Interfața grafică este comună pe majoritatea sistemelor desktop și este universală pe dispozitive mobile, dispozitive de tip smartwatch sau smart TV.

În mod clasic interfața grafică este compusă din elementele **WIMP**: *Window*, *Icon*, *Menu*, *Pointer*. Aceste elemente se regăsesc în interfețele cu ferestre comune sistemelor desktop. Pe sistemele mobile sau pe smart TV sau smartwatch, elementele sunt similare, cu diferența că, în mod uzual, nu sunt prezente mai multe ferestre simultan pe ecranul prezentat utilizatorului.

Pointerul din interfața grafică este controlat cu mouse-ul sau o telecomandă specifică sau este îndeplinit de deget pe ecranul tactil (*touchscreen*) al dispozitivelor mobile. Cu ajutorul acestuia se activează (prin click) elemente din interfața grafică: iconuri, meniuri, butoane, sau se poate face *drag and drop* sau alte acțiuni de aranjare a interfeței grafice.

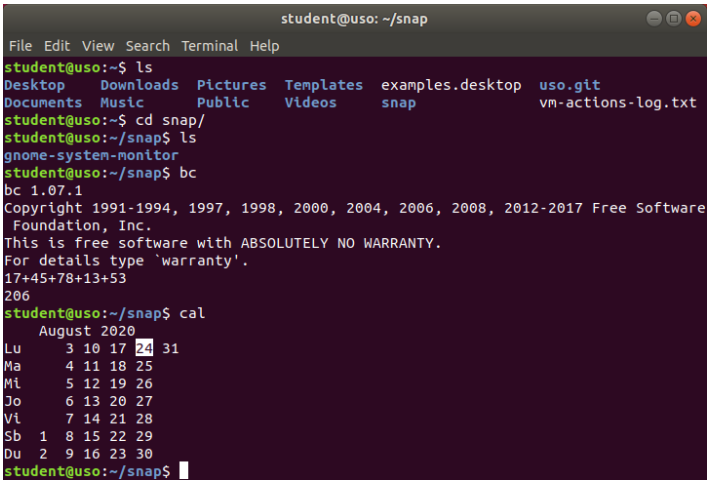
Interfața grafică poate fi personalizată: scheme de culori, fonturi, dimensiuni, plasare elemente grafice pot fi modificate. Uzual noțiunea de „temă” include o agregare de fonturi, culori și plasare a elementelor; un utilizator poate selecta între diferite teme pentru a alege interfața grafică potrivită. Această personalizare poate fi făcută atât pentru shellul sistemului de operare cât și pentru interfața grafică expusă de diferite aplicații. Adică pentru shellul sistemului de operare putem alege o dimensiune a iconurilor și o schemă de culori, în vreme ce pentru interfața unei aplicații precum Firefox (browser web) putem alege folosirea anumitor imagini pentru iconuri sau ascunderea anumitor meniuri.

Pentru accelerarea acțiunilor în interfața grafică, există în mod uzual scurtături de tastatură. Este o practică frecventă în lumea jocurilor de calculator (unde celebrele actions-per-minute (APM)) se bazează pe folosirea scurtăturilor din tastatură. În interfața în linia de comandă scurtăturilor sunt folosite pentru gestiunea ferestrelor, accesarea rapidă a unor elemente din meniuri.

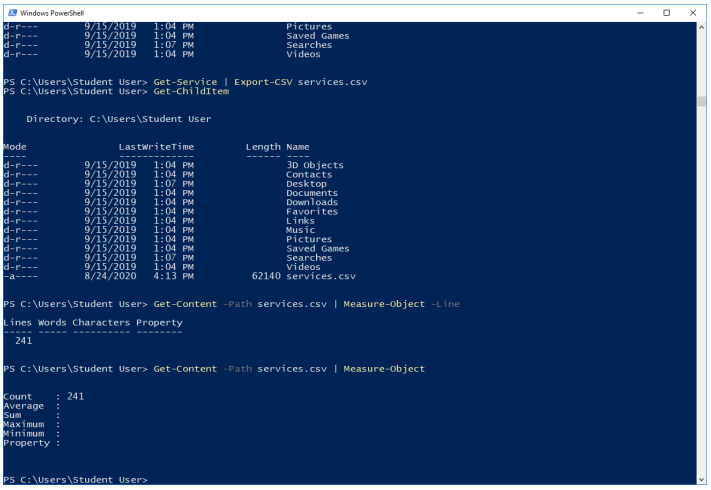
În instalările implicite, sistemele de operare desktop oferă interfața grafică. Pentru tipurile de acțiuni care necesită interfața în linia de comandă (precum acțiuni mai specifice, sau acțiuni de automatizare sau acțiuni care nu se pot realiza altfel), se pot porni shelluri CLI din interfața grafică. Într-un mediu GNOME se poate porni o fereastră GNOME Terminal, în într-un mediu grafic Windows se poate porni o fereastră PowerShell. În Figura 1.3 sunt screenshot-uri cu două ferestre din mediul GUI în care rulează un shell CLI, una în Linux și alta în Windows. Este modul uzual în care rulăm comenzi în sistemele desktop moderne.

1.1.1 Interfețe native și interfețe web

Interfețele web sunt tot forme de interfața grafică, folosind componente precum meniuri, iconuri, pointer, butoane. Spunem că o aplicație are interfață GUI dacă rulează de sine



(a) Linux



(b) Windows

Figura 1.3: Shell CLI (terminal) în Linux și în Windows

stătătoare pe un sistem de calcul; o numim **aplicație nativă**. Altfel, aplicațiile web oferă o interfață grafică web (numită și WebUI - *web user interface*) în cadrul unui browser.

De avut în vedere că în ultimii ani asistăm la o migrare spre interfețe grafice web. Sistemele de operare desktop moderne oferă mai puține aplicații preinstalate, majoritatea elementele grafice (WIMP) fiind migrate în aplicații web. Adesea, un utilizator va porni un browser și din browser își va executa majoritatea acțiunilor: urmărit filme, ascultat muzică, lucrat la documente folosind suita Google Docs sau Office 360, comunicat online (Facebook Messenger, WhatsApp), publicat conținut (Facebook, Pinterest), inclusiv dezvoltat de aplicații.

Aplicațiile web sunt uzuale mediului desktop. În mod obișnuit, pe dispozitivele mobile sau de tip smart TV sau altele, aceste aplicații sunt aplicații native. În continuare pot fi accesate și prin browser, dar forma uzuală este cea de aplicație nativă. Adică pe un mediu desktop vom accesa din browser aplicația Facebook sau YouTube, pe când pe un dispozitiv mobil aceste aplicații sunt native; se poate accesa o aplicație web și pe un dispozitiv mobil prin intermediul unui browser, dar nu este o acțiune obișnuită pentru utilizator.

Motivul migrării aplicațiilor desktop în aplicații web este portabilitatea: dat fiind un browser (Mozilla Firefox, Google Chome sau altul), aplicațiile web se vor comporta la fel indiferent de sistemul de operare sau distribuția rulată. Altfel, este nevoie de o aplicație nativă pentru fiecare sistem de operare sau distribuție. În zona dispozitivele mobile sau smart TV, furnizorul dispozitivului oferă și sistemul de operare iar aplicațiile sunt create doar pentru acesta.

1.1.2 Utilizabilitate și experiența utilizatorului

În proiectarea interfeței grafice este esențial ca aceasta să fie cât mai intuitivă și ușor de folosit. Noțiunea de experiența utilizatorului (*user experience* - UX) este centrală pentru aplicațiile și dispozitivele cu interfața grafică. O interfața încărcată, cu elemente greu accesibile sau neintuitiv plasate va fi respinsă de utilizator care va alege altă aplicație sau altă platformă.

Organizațiile care dezvoltă interfețe grafice stabilesc principii și recomandări de urmat. Acest principii țin cont de natura umană și comportamentul utilizatorilor, în general din studii specifice, și se regăsesc în documente numite *Human Interface Guidelines* (Ghiduri de interfațare om-calculator). Exemple de organizații și astfel de documente sunt:

- GNOME: *GNOME Human Interface Guidelines*[1]
- Apple: *Apple Human Interface Guidelines*[2]
- Android: *Android Design Guidelines*[3]
- Microsoft: *Microsoft User Interface Principles*[4]

[1] https://developer.gnome.org/hig/stable/
[2] https://developer.apple.com/design/human-interface-guidelines/
[3] https://developer.android.com/design
[4] https://docs.microsoft.com/en-us/windows/win32/appuistart/-user-interface-principles

O aplicație sau un dispozitiv de succes va trebuie să țină cont și de interfața grafică, nu numai de funcționalitățile și performanța sa. Aspectele ce țin de experiența utilizatorului sunt cu atât mai importante pe dispozitivele mobile unde interfața grafică tactilă este universală. În aceeași măsură, aplicațiile și site-urile web necesită interfețe intuitive pentru satisfacția utilizatorilor acestora. Un utilizator nemulțumit sau încurcat de interfață nu va folosi aplicația, lucru care se traduce în general în pierderi financiare. De aceea, aspectele ce țin de experiența utilizatorului sunt esențiale pentru dezvoltatorii de aplicații pe dispozitive mobile sau aplicații web.

1.2 Interfața grafică în Linux

Așa cum am precizat și în Capitolul 0, Linux (sau mai bine spus, distribuțiile software bazate pe Linux) rulează pe o plajă largă de sisteme și dispozitive: server, desktop/laptop, mobile (Android), smart TV, smart watch, rutere/echipamente de rețea etc. Fiecare dintre aceste sisteme oferă o interfață utilizatorului:

- interfață grafică tactilă, în cazul dispozitivelor mobile sau smart watch
- interfață grafică și telecomandă, în cazul smart TV
- interfață în linie de comandă sau interfață web de configurare, în cazul echipamentelor de rețea
- interfață în linie de comandă, în cazul serverelor
- interfață grafică și în linie de comandă, în cazul sistemelor desktop/laptop

În cele ce urmează vom prezenta interfața grafică în cazul sistemelor desktop/laptop. Deși nu este obișnuit, această interfață poate fi prezentă și în cazul sistemelor server. Interfața în linia de comandă va fi prezentată în formă de comenzi Linux în restul capitolelor și detaliat în Capitolul 7.

1.2.1 Desktop Environments

Suita de componente software care oferă interfața grafică utilizatorului într-un sistem de operare formează un mediu desktop (*desktop environment*), așa numitul shell GUI. Putem spune că toate sistemele de operare oferă un desktop environment pe majoritatea tipurilor de dispozitive. Termenul a fost însă popularizat și este folosit cu precădere în sistemele de operare din lumea Unix/Linux.

În lumea Linux posibilitatea de personalizare a distribuțiilor software este foarte mare, inclusiv a mediului desktop folosit. O distribuție Linux oferă o configurație implicită de mediu desktop, care poate fi însă modificată.

Alegerea unui mediu desktop ține de consumul de resurse, de posibilitățile de personalizare și preferința și istoricul utilizatorului. Un mediu desktop oferă un anume aspect, o selecție și configurare a componentelor software, și aplicații specifice pentru acțiuni uzuale (navigare în sistemul de fișiere, navigare web, redare multimedia, editor text, emulator de terminal).

Exemple de medii desktop în Linux sunt GNOME, KDE, MATE, Cinnamon, Budgie, Xfce. O comparație a acestora și referințe la articole comparative se găsește pe Wikipedia[1]. Aplicațiile grafice de bază (precum browerul în sistemul de fișiere, editor text, player multimedia) sunt parte a fiecărui mediu desktop, fiecare mediu cu particularitățile sale. Alte aplicații grafice sunt instalate, majoritatea nefiind specifice unui mediu desktop; de exemplu, Mozilla Firefox este un browser web iar VLC este un player multimedia care nu sunt specifice unei anumite distribuții.

1.2.2 Sistemul de ferestre (Window System)

Redarea elementelor grafice ale unei aplicații ce rulează peste sistemul de operare este gestionată de sistemul de ferestre. Sistemul de ferestre este un ansamblu software care interacționează cu sistemul de operare, componente ale mediului desktop și cu aplicațiile. Fiecare sistem de operare (nu doar Linux) are un sistem de ferestre. În Windows, este reprezentat de Desktop Windows Manager; în macOS, este reprezentat de Quartz și Core Graphics.

Sistemul de ferestre este compus în mod uzual din:

- manager de ferestre/afișare (*window/display manager*): gestionează așezarea ferestrelor pe ecran, uzual încluzând bare de meniuri.
- widget/GUI toolkit: colecție de biblioteci care oferă elementele grafice (numite widgeturi), folosit de managerul de ferestre.
- server de ferestre/afișare (*window/display server*): interacționează printr-un protocol specific cu aplicațiile grafice, cu sistemul de operare și cu managerul de ferestre pentru redarea componentelor grafice.

Schematic, componentele sistemului de ferestre sunt indicate în Figura 1.4[2].

Delimitarea între componentele sistemului de ferestre din Figura 1.4 nu este strictă, mai ales în sistemele de operare diferite de Linux.

În mod implicit, managerii de ferestre folosiți implicit pe majoritatea distribuțiilor Linux sunt flotanți (*floating/stacking window manager*). Adică ferestrele pot fi suprapuse și mișcate în funcție de preferințele utilizatorilor. Un alt tip de manager de ferestre este cel bazat pe secțiuni (*tiling window manager*); în acest caz fiecare fereastră ocupă un spațiu fix în ecran, fără a se suprapune cu alte ferestre; ecranul este secționat cu ferestre diferite ocupând un spațiu nesupraspus cu altele. Exemple de tiling window managers sunt i3[3], awesome[4], bspwm[5]. Unii manageri sunt dinamici permițând tranziția între modul floating și modul tiling.

În Linux, în mod tradițional, sistemul de ferestre folosit este X Window System[6], sau, mai simplu, X. În ultima perioadă a căpătat tracțiune alternativa Weyland/Weston[7], apărută

[1] https://en.wikipedia.org/wiki/Comparison_of_X_Window_System_desktop_environments

[2] https://commons.wikimedia.org/wiki/File:Schema_of_the_layers_of_the_graphical_user_interface.svg (CC BY-SA 3.0)

[3] https://i3wm.org/

[4] https://awesomewm.org/

[5] https://github.com/baskerville/bspwm

[6] https://www.x.org/wiki/

[7] https://wayland.freedesktop.org/

by Shmuel Csaba Otto Traian; GNU FDL 1.2+ and CC-BY-SA 3.0+; created 2013-10-29

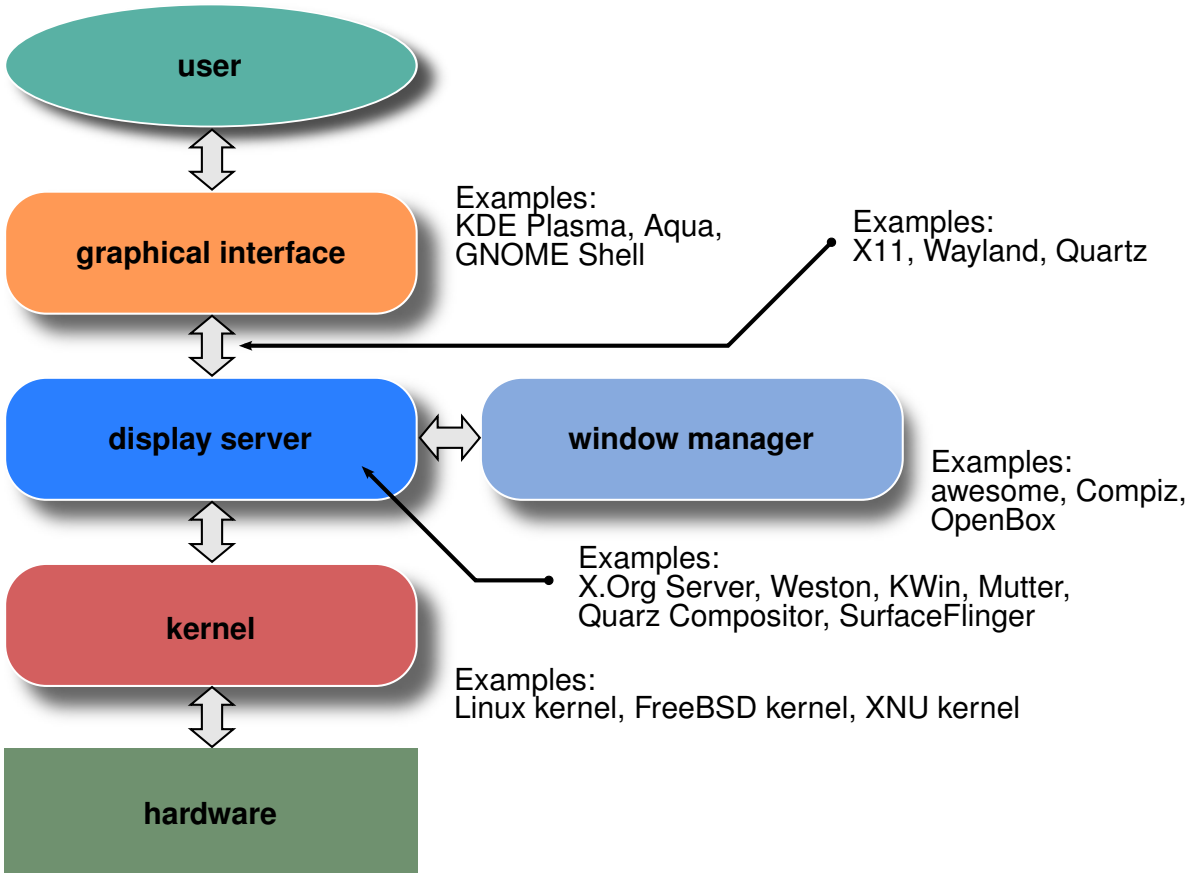


Figura 1.4: Componentele sistemului de ferestre

ca reacție la complexitatea X. Weyland/Weston se dorește a fi o soluție mai simplă și cu proiectare mai sigură. Weyland/Weston nu este încă adoptat pe scară largă, astfel că multe distribuții sau medii desktop preferă folosirea X, care este mai stabil.

Aspectul comun al X și Weyland este interacțiunea componentelor. X și Weyland sunt de fapt specificații de protocoale și implementări de referință ale acestora pentru interacțiunea între serverul de ferestre (window server) și aplicațiile grafice (clienți grafici).

Schema de funcționare a X/Weyland este prezentată în Figura 1.5[1]

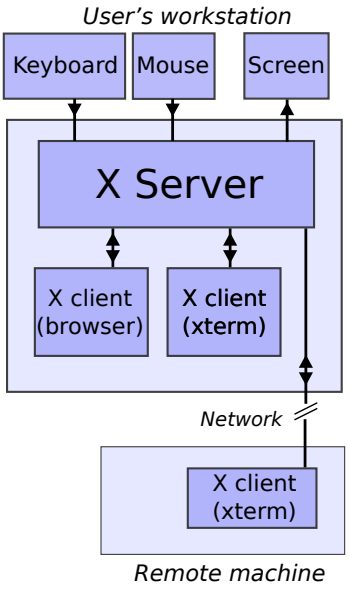


Figura 1.5: Funcționarea sistemului de ferestre în Linux (X sau Weyland)

În interacțiunea indicată în Figura 1.5, o aplicație precum Mozilla Firefox este client grafic (*X client* în figură). Folosind protocolul X/Weyland, aplicația transmite către serverul X/Weyland (*X server* în figură) cerințele sale de afișare grafică pe ecran (*Screen* în figură). Aceste cerințe sunt rezultate de obicei din acțiunile utilizatorului, cu ajutorul mouse-ului sau tastaturii (*Keyboard* și *Mouse* în figură) sau a altor dispozitive. Aceste acțiuni ajung la aplicație ștot prin intermediul serverului X. Practic, aplicația client se ocupă de resursele sale de calcul lăsând serverului grafic responsabilitatea interacțiunii cu utilizatorul.

Una dintre funcționalitățile X, absentă din Weyland, este folosirea peste rețea, indicată în Figura 1.5 (*Network*). În această situație, aplicația client rulează pe un sistem la distanță (*Remote machine* în figură), iar serverul X rulează pe sistemul local (*User's workstation* în figură). Interacțiunea între aplicația client și aplicația server are loc prin rețea prin intermediul protocolului X. Practic, pașii urmați sunt:

1. Utilizatorul folosește echipamentele locale (mouse, tastatură) pentru a executa acțiuni.
2. Acțiunile utilizatorului sunt capturate de serverul X local.
3. Serverul X local transmite aceste transmise de server prin rețea către aplicația client ce rulează la distanță.

[1] https://commons.wikimedia.org/wiki/File:X_client_server_example.svg (CC BY-SA 3.0)

4. Aplicația client execută aceste acțiuni și generează un rezultat care să fie afișat (pixeli, culori, ferestre etc.)
5. Rezultatul este transmis de aplicația client prin rețea către serverul X.
6. Serverul X redă rezultatul pe afișajul local.
7. Utilizatorul vede pe afișajul local rezultatul acțiunilor sale inițiale.

În secvența de mai sus, întrucât aplicația client rulează pe sistemul de la distanță, consumul de resurse (procesor, memorie, sistem de fișiere) se face pe acel sistem. Sistemul local (pe care rulează serverul X) ocupă resurse doar pentru interacțiune cu utilizatorul (ecran, tastatură, mouse).

Aplicații grafice pot fi rulate la distanță și dacă sistemul local rulează Windows sau macOS. Pentru aceasta este nevoie de instalarea unui server X nativ Windows, precum XMing[1], sau a unui server nativ macOS, precum XQuartz[2].

În mod uzual, din rațiuni de securitate, rularea la distanță de aplicații X grafice se realizează peste un canal SSH, protocol despre care vom discuta în Secțiunea 12.4.4.

1.2.3 Funcționalități ale interfeței grafice în Linux

Pe lângă elementele grafice de tipul WIMP, interfața grafică în Linux oferă utilizatorilor și alte funcționalități.

O astfel de funcționalitate este prezența de de spații de lucru (*workspace*) multiple, sau desktopuri multiple. Într-o sesiune de mediu grafic pot exista mai multe workspace-uri/desktopuri, fiecare cu ferestrele sale; utilizatorul poate migra între workspace-uri pentru a compartimenta aplicațiile și ferestrele în funcție de nevoi. Depinzând de mediul desktop, numărul de workspace-uri este static (fixat într-o configurație) sau dinamic (se poate crea rapid un workspace nou). În oricare situație, acțiunile sunt: tranziție la un alt workspace sau mutare fereastră într-un alt workspace. O funcționalitate similară este prezentă în macOS, numită Mission Control, și în Windows, de la Windows 10.

Pentru a executa acțiuni rapide în mediul grafic sunt utile scurtături de tastatură (*keyboard shortcuts*). În general, orice mediu grafic oferă scurtături pentru pornire de aplicații, închidere de aplicații, deschidere și închidere de tab-uri de aplicație, minimizat ferestre, accesat meniuri, schimbat între ferestre. Aceste scurtături depind de sistemul de operare și mediul desktop folosit. O listă a scurtăturilor folosite pe mașina virtuală de suport a acestei cărți (Ubuntu 18.04) este în Tabelul 1.1.

O parte dintre acțiunile utilizatorului pot fi realizate doar sau mai rapid în interfața în linia de comandă. Pentru acesta, din mediul grafic se pot porni rapid aplicații shell în linia de comandă, în forma emulatoarelor de terminal *terminal emulator*. Un emulator de terminal este o aplicație care prezintă utilizatorului o fereastră. În această fereastră rulează un shell CLI unde utilizatorul poate rula comenzi. De exemplu, în Figura 1.3, prima imagine este o fereastră de emulator de terminal în GNOME (aplicația `gnome-terminal`) în care rulează shellul Bash; în mod similar, cealaltă imagine este o fereastră Windows

[1] https://sourceforge.net/projects/xming/
[2] https://www.xquartz.org/

în care rulează shellul PowerShell. În macOS acest rol în are aplicația Terminal, care este preinstalată în sistemul de operare. Sunt și alte emulatoare de terminal care pot fi instalate în locul sau pe lângă cele implicite. Mai multe informații despre terminale, emulatoare de terminal și shell vom prezenta în Capitolul 7.

1.3 Interfața grafică în mașina virtuală de suport

Distribuțiile, mediile desktop, managerii de ferestre și aplicațiile Linux sunt diverse. Este dificil de găsit un numitor comun de aspecte generice, comune tuturor. În această secțiune vom prezenta informații specifice mașinii virtuale de suport a cărții, mașină care rulează Ubuntu 18.04.

Așa cum am precizat mai sus, pentru acțiuni rapide în interfața grafică a unui sistem de operare (și, în general, a unei aplicații) sunt utile scurtăturile de tastatură. Tabelul 1.1 conține cele mai comune scurtături în mașina virtuală.

Tabelul 1.1: Scurtături de tastatură în mașina virtuală de suport

Scurtătură	**Efect**	**Aplicații în care are efect**
`Alt+F2`	deschiderea promptului de rulare comenzi	în mediul grafic (nu într-o anume aplicație)
`Alt+Ctrl+t`	deschiderea unui emulator de terminal	în mediul grafic (nu într-o anume aplicație)
`Ctrl+n`	deschiderea unei aplicații identice	în majoritatea aplicațiilor grafice
`Ctrl+t`	deschiderea unui nou tab	în aplicații cu taburi (browser web, browser de fișiere)
`Ctrl+Shift+t`	deschiderea unui nou tab	în emulatorul de terminal
`Ctrl+w`	închiderea unui tab (sau a ultimei ferestre)	în aplicații cu taburi (browser web, browser de fișiere, terminal)
`Alt+1, Alt+2, ...`	nagivarea între taburi	în aplicații cu taburi (browser web, browser de fișiere, terminal)
`Alt+Tab`	nagivarea între adrese	în mediul grafic (nu într-o anume aplicație)
`Ctrl+l`	accesarea barei de adrese	în aplicații cu bare de adrese (browser web, browser de fișiere)
`Alt+F4`	închiderea unei ferestre de aplicație	în toate aplicațiile
`Alt+Ctrl+d`	afișarea desktopului (cu minimizarea tuturor ferestrelor)	în mediul grafic (nu într-o anume aplicație)
`Alt+F10`	maximizare / minimizare aplicații (comutare)	în toate aplicațiile
`Alt+Ctrl+săgeți`	navigarea între workspace-uri	în mediul grafic (nu într-o anume aplicație)
`Alt+Ctrl+l`	închidere ecran (*lock screen*)	în mediul grafic (nu într-o anume aplicație)

O scurtătură frecventă de tastatură, întâlnită pe majoritatea distribuțiilor și mediilor desktop, este `Alt+F2` folosită pentru pornirea rapidă a unei aplicații, introducând numele comenzii, la fel ca în Figura 1.6. Funcționalitatea este numită *application launcher*. Echivalentul în Windows este `Windows+r`, iar în macOS `Command+Space`.

Sunt mai multe scurtături de tastatură, majoritatea fiind configurabile. Pentru vizualizarea și, eventual, configurarea acestora în mașina virtuală se accesează `System Settings` → `Devices` → `Keyboard`; va rezulta un ecran precum cel

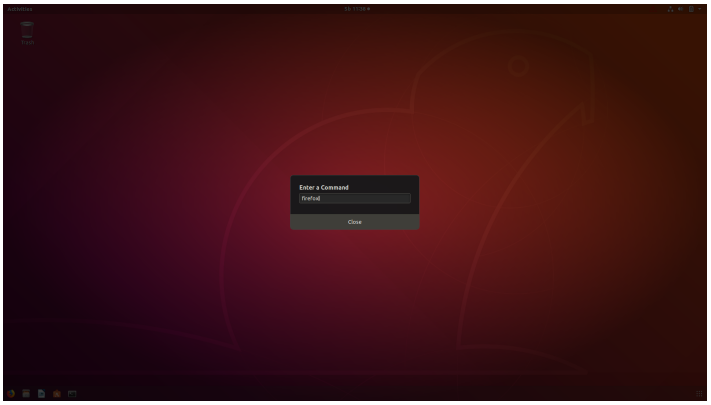


Figura 1.6: Rularea unei aplicații din mediul grafic în Linux (*application launcher*)

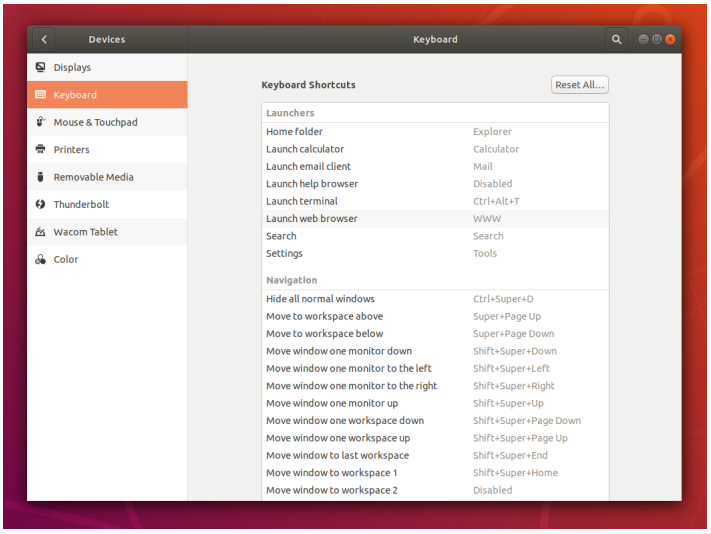


Figura 1.7: Vizualizarea și configurarea scurtăturilor de tastatură

din Figura 1.7. Aplicația *System Settings* se accesează din interfața grafică sau prin rularea comenzii `gnome-control-center`.

O acțiune frecventă la instalarea sistemului de calcul este configurarea schemei de tastatură (*keyboard layout*). Dacă dorim adăugarea unei noi scheme de tastatură, acest lucru se face prin accesarea meniului de tastatură din pașii `System Settings` → `Region & Language`. Vom ajunge la un ecran precum cel din Figura 1.8, unde putem configura noi scheme de tastatură.

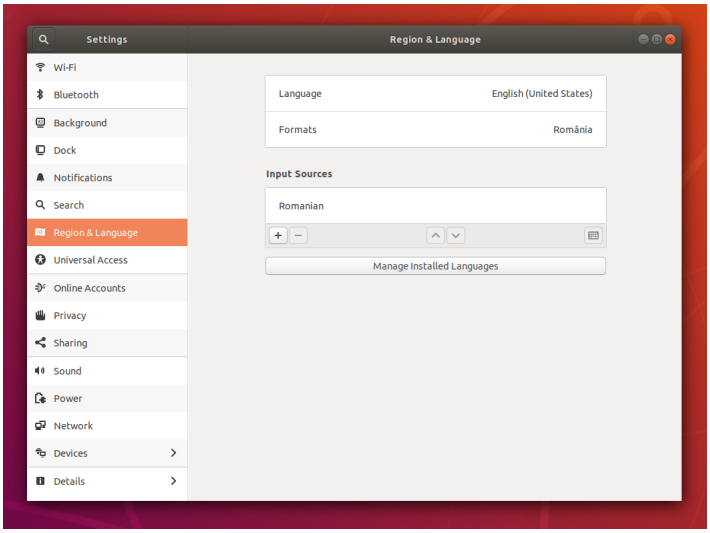


Figura 1.8: Configurarea schemei de tastatură (keyboard layout)

În general configurațiile se pot realiza din meniurile aplicațiilor sau din aplicația *System Settings* (pornită din interfața grafică sau cu ajutorul comenzii `gnome-control-center`. Anumite configurații (precum dezactivarea introducerii parolei la autentificare) necesită accesarea interfeței de configurare. Pentru aceasta folosim comandda `dconf-editor` pentru a porni aplicația *Dconf Editor*, la fel ca în Figura 1.9, cu care avem acces la o plajă largă de opțiuni de configurare.

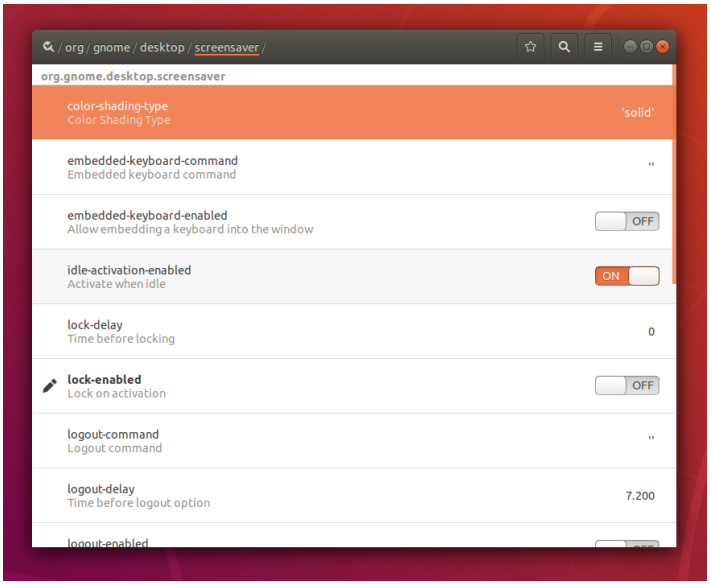


Figura 1.9: Opțiuni de configurare în mediul grafic (dconf-editor)

În mod implicit, în Linux, desktopul conține puține scurtături de desktop (*desktop shortcuts*) sau alte iconuri. Aceasta pentru că aplicațiile pot fi pornite rapid folosit scurtătura de tip *application launcher* (`Alt+F2`). Dacă însă dorim să creăm o scurtătură de desktop, putem face acest fie creând manual un fișier cu extensia `.desktop`[1] sau folosind comanda `gnome-desktop-item-edit`, ca mai jos:

```
1 student@uso:~$ gnome-desktop-item-edit ~/Desktop/Skype.desktop
```

La rularea acestei comenzi, va apărea o fereastră precum cea din Figura 1.10 unde vom introduce informațiile legate de aplicația Skype pentru care dorim crearea scurtăturii: calea către executabil, numele, iconul. În final va fi creat fișierul `Skype.desktop` care va apărea pe desktop și cu care vom putea porni, ca scurtătură de desktop, aplicația Skype.

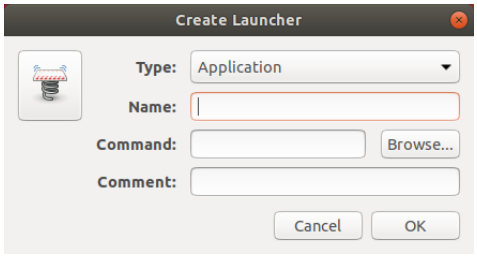


Figura 1.10: Configurarea unei scurtături de desktop

1.4 Sumar

Sistemul de operare oferă o interfață (shell) pentru interacțiunea utilizatorului cu aplicațiile și sistemul de operare.

Shellul poate fi grafic (GUI) sau în linia de comandă. Shellul grafic este mai intuitiv și preferat majorității utilizatorilor. Interfața în linia de comandă permite acces la un spațiu mai mare de opțiuni și acțiuni și posibilitatea automatizării prin scripturi.

Interfața grafică este compusă din elemente grafice acționate cu mouse-ul sau tactil, sub acronimul WIMP (*Window, Icon, Menu, Pointer*).

Interfața grafică trebuie să fie simplă și intuitivă. Aplicațiile și sistemele de operare urmăresc recomandări de proiectare a interfeței pentru a oferi o experiență bună utilizatorului.

În Linux interfața grafică vine în formă de mediu desktop (*desktop environment*), o suită software cu shell grafic, aplicații grafice și opțiuni de configurare specifice. Exemple sunt GNOME și KDE.

Mediul grafic se bazează pe un sistem de ferestre. În Linux, în mod tradițional este folosit sistemul de ferestre X. În ultimi ani, a prins tracțiune sistemul de ferestre Weyland/Weston.

Mașina virtuală de suport a acestei cărți folosește un sistem de operare Ubuntu 18.04 cu interfață grafică GNOME.

[1] https://linuxconfig.org/how-to-create-custom-desktop-files-for-launchers-on-linux

Capitolul 2

Utilizarea sistemului de fișiere

În fiecare zi lucrăm cu diverse fișiere, precum poze, texte sau melodii. De exemplu, le căutăm prin directoare, le deschidem, le modificăm, le închidem, le ștergem, le scoatem apoi din Recycle Bin deoarece le șterseserăm din greșeală, și așa mai departe.

Fișierele sunt, astfel, partea a mai vizibilă și utilizată a sistemului de operare. Ne dorim să avem acces rapid și ușor la fișiere, să le putem localiza rapid și să stocăm cât mai multe date. Aceste nevoi duc la nevoia organizării fișierelor. Pentru a permite căutare rapidă și operații ușor de realizat cu fișierele, acestea sunt organizate într-o structură ierarhică, numită **sistem de fișiere**.

2.1 Noțiuni de bază

Sistemul de fișiere este una dintre componentele centrale ale sistemului de operare, care ne ajută să organizăm cantități impresionante de informații, procese și colaboratori. Sistemul de fișiere are două roluri importante în sistemul de operare:

- În primul rând, oferă posibilitatea **controlului unei cantități tot mai mari de documente**, permițându-ne să găsim un anumit fișier printre mii, milioane sau chiar miliarde de alte fișiere, în sistemele distribuite. Această contribuție este detaliată în Secțiunea 2.1.2 privind structura ierarhică.
- În al doilea rând, sistemele de fișiere **asigură separarea resurselor între utilizatorii multipli ai unui sistem de calcul**, fie cei umani sau non-umani. Vom discuta importanța acestei trăsături în Secțiunea 5.5 dedicată permisiunilor.

Sistemele de fișiere sunt diverse, **construite și optimizate pentru diferite contexte de utilizare**. Nu există un sistem de fișiere optim pentru toată lumea. Pentru a alege un sistem de fișiere trebuie să știm care sunt prioritățile în funcționarea sistemului. De exemplu, creșterea resurselor de stocare a dus la dispariția crizelor de spațiu și a problematicii comprimării în sistemele personale; aceasta este însă tot mai relevantă în arhitecturile *cloud*. În zilele noastre, un utilizator stochează și folosește filme, poze, jocuri pe calculator, documente, mașini virtuale, arhive de informații; acestea ocupă spațiu considerabil pe un sistem laptop sau pe un dispozitiv mobil inteligent sau în Dropbox; acest spațiu ocupat nu mai este însă considerat o problemă dat fiind starea tehnologiilor și serviciilor de stocare.

De asemenea, cerințele ca datele să fie integre și valide (să nu fie corupte) este foarte importantă în sistemele ce lucrează cu date critice, dar mai puțin importantă în sistemele personale, care au mai multă nevoie de simplitate și flexibilitate. Printre criteriile după care putem compara și alege sistemele de fișiere se numără:

- asigurarea integrității datelor;
- separarea eficientă a resurselor între diferiți utilizatori;
- securizarea datelor prin setarea permisiunilor diferențiate de acces;
- volumul gestionat: facilitatea în lucrul cu fișiere foarte mari sau cu un număr foarte mare de fișiere;
- comprimarea fișierelor (*file compression*) pentru a maximiza spațiul de stocare pe disc (comprimarea înseamnă că aceleași date vor ocupa mai puțin spațiu, dar vor necesita efortul procesorului pentru a le decomprima la fiecare utilizare);
- optimizarea spațiului de stocare prin gestiunea fișierelor duplicate și a zonelor ineficient scrise pe disc;
- gestiunea posibilelor erori prin jurnalizare și reversibilitate, adică menținerea unei liste a modificărilor ce permite revenirea la o stare anterioară în cazul apariției unei erori.

2.1.1 Definiții

Știm, intuitiv, ce sunt fișierele: sunt documentele noastre electronice, precum pozele, melodiile, proiectele pentru facultate sau programele executabile.

> Fișierul (*file*) reprezintă o formă de organizare digitală a informațiilor, având forma unei înșiruiri de octeți.

Informațiile sunt organizate în **fișiere** în vederea utilizării lor printr-o aplicație și a stocării lor de durată. În afara fișierelor create de utilizatorii umani, există și fișiere create de utilizatori automați. Unele dintre acestea sunt esențiale pentru funcționarea sistemului de calcul și sunt ascunse de utilizatorii obișnuiți.

Fișierele sunt organizate la rândul lor în **directoare**.

> Un director (*folder* sau *directory*) reprezintă o colecție de fișiere și subdirectoare, identificată printr-un nume.

Dacă putem înțelege un fișier prin analogie cu o **foaie** pe care sunt scrise informații, putem înțelege un director prin analogie cu un **dosar** care conține file de hârtie dar și alte dosare. Un director, ca și un dosar, poate fi și gol.

Această analogie este utilă dar poate fi și înșelătoare. Dintr-o perspectivă tehnică, directoarele sunt tot fișiere. Ele nu „conțin” efectiv fișierele pe care le organizează, așa cum un dosar conține foi, ci doar numele lor - fiind similare cu o foaie pe care am scris o listă de documente. Prin urmare, directoarele în Linux sunt niște fișiere speciale care servesc organizării altor fișiere și directoare.

Înțeles pentru *folder* (director): Conceptul mai general de director (*folder*), preluat din engleză și în limbajul nostru, se poate referi la forme de organizare a informațiilor care nu au corespondent în sistemul de fișiere. De exemplu, interfețele de email pot permite organizarea mesajelor pe *foldere*. Acestea nu vor fi regăsite în structura ierarhică a sistemului de fișiere, rămânând accesibile doar prin intermediul interfeței în care au fost create.

Fișierele reprezintă informații digitale inscripționate pe mediile fizice de stocare (hard disk, USB stick, DVD etc). Mediile de stocare pot fi considerate spații continue de octeți, pe care putem înscrie multe fișiere, de dimensiuni variabile. Pentru a putea citi sau scrie fișiere pe un mediu de stocare este necesar să cunoaștem sistemul de fișiere utilizat pentru organizarea acestuia.

Sistemul de fișiere este o parte a sistemului de operare ce se ocupă cu numele și atributele fișierelor, pe care le stochează într-o structură ierarhică. Sistemul de fișiere oferă o metodă de organizare fizică și logică a fișierelor într-un mediu de stocare:

- stocarea fișierelor ca o înșiruire de octeți reprezintă organizarea fizică;
- modul în care sunt adresate fișierele reprezintă organizarea logică.

Sistemul de fișiere ajută astfel utilizatorul să stocheze și să organizeze datele digitale pentru acces facil. Figura 2.1 prezintă rolul sistemului de fișiere în organizarea datelor; orgnizarea este de obicei ierarhică așa cum este prezentat în Secțiunea 2.1.2.

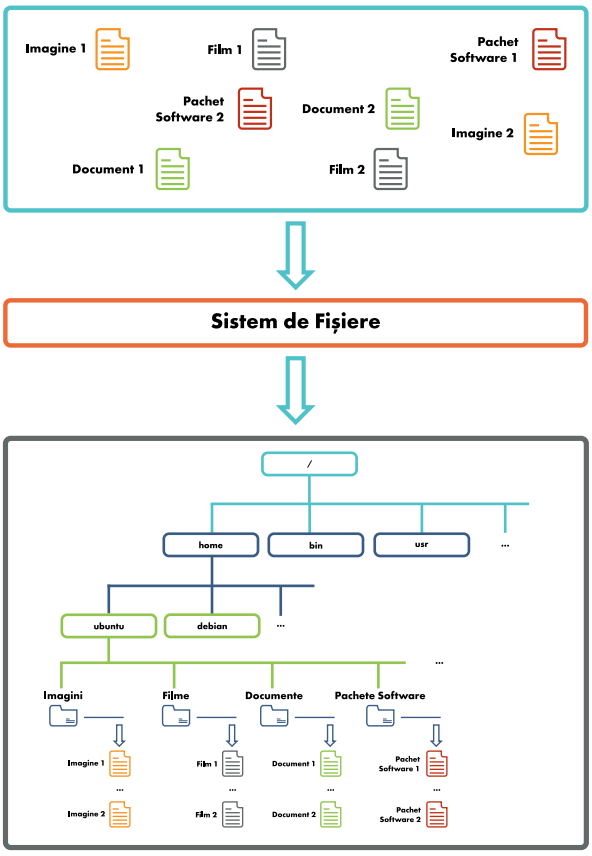


Figura 2.1: Sistemul de fișiere ca organizator de date digitale

Fișierele sunt folosite de sistemul de operare pentru a organiza atât datele provenite

de la utilizator, cât și cele generate de sistem. În sistemul de fișiere sunt stocate și organizate poze, documente, notițe ale utilizatorului, dar și fișiere de configurare, fișiere de tip jurnal (*log files*), baze de date necesare funcționării sistemului.

Deoarece este important ca utilizatorii să poată accesa fișierele stocate, sistemul de operare pune la dispoziție o interfață pentru a putea lucra cu sistemul de fișiere. În funcție de preferințe, există două tipuri de interfețe, așa cum descris în Capitolul 1: interfața în linia de comandă (CLI) expusă de interpretorul de comenzi (shellul CLI) și interfața grafică (GUI) prezentată de exemplu de un browser de sistem de fișiere.

2.1.2 Structura ierarhică a sistemului de fișiere

De la an la an avem tot mai multe fișiere pe calculatoare și telefoane. Acumulăm poze, video-uri, melodii, precum și documente de la școală sau de la birou. Ce-ar însemna să găsim o poză într-o colecție de un milion de poze? Sistemul de fișiere oferă o interfața ierarhică care permite organizarea și căutarea informației.

O structură **ierarhică**, sau arborescentă, apare atunci când fișierele sunt organizate în directoare (*folders*). Un director poate conține mai multe fișiere dar și alte directoare, fiecare dintre care poate conține mai multe fișiere dar și alte directoare și tot așa, până când ultimul director va conține doar fișiere sau va fi gol. Un director este analog unei crengi dintr-un arbore, pe care pot crește alte crengi dar și frunze (fișierele). Pe frunze nu crește nimic.

Să examinăm următorul exemplu. Pentru a găsi o poză anume (de exemplu, `selfiecumotanul-mai2018.jpg`) într-o colecție de 1.000.000 de poze, utilizatorul ar trebui să ceară sistemului de fișiere să parcurgă toate pozele până când găsește numele fișierului căutat. În cel mai rău caz, va trebui să parcurgă toate cele 1.000.000 de nume.

Ce se întâmplă însă dacă apelăm la o organizare pe trei niveluri, grupând pozele câte 100, la fel ca în Figura 2.2? Pe primul nivel, vom avea 100 de directoare. În fiecare dintre ele, includem 100 de subdirectoare. Apoi, în fiecare subdirector includem 100 de poze. Astfel, am stocat într-o ierarhie cu trei niveluri `100 directoare * 100 subdirectoare * 100 poze = 1.000.000 poze`.

Cum putem găsi acum poza? Adresa ei ne va indica unde anume se află, în arborele de fișiere. De exemplu, dacă adresa sa completă este: Directorul `99/cat-pics/selfiecumotanul-mai2018.jpg`, sistemul de fișiere va căuta întâi directorul `99` printre cele 100 de directoare de pe primul nivel. Apoi, va căuta `cat-pics` printre cele 100 de subdirectoare din directorul 99. Apoi, în cele din urmă, va căuta poza dorită. În cel mai rău caz, sistemul de fișiere va realiza, de trei ori la rând, o căutare între 100 de elemente.

Prin structura arborescentă cu 3 niveluri am înlocuit deci o căutare într-un sac cu 1.000.000 elemente, pe care o singură persoană o realizează în ore dacă nu zile, cu o căutare în trei pungute cu 100 de elemente fiecare, care este ușor de realizat chiar și manual, de o singură persoană.

Prin organizarea arborescentă pe directoare, un număr mare de fișiere pot fi gestionate la nivelul unei singure persoane. Sistemul de fișiere transformă astfel complexitatea

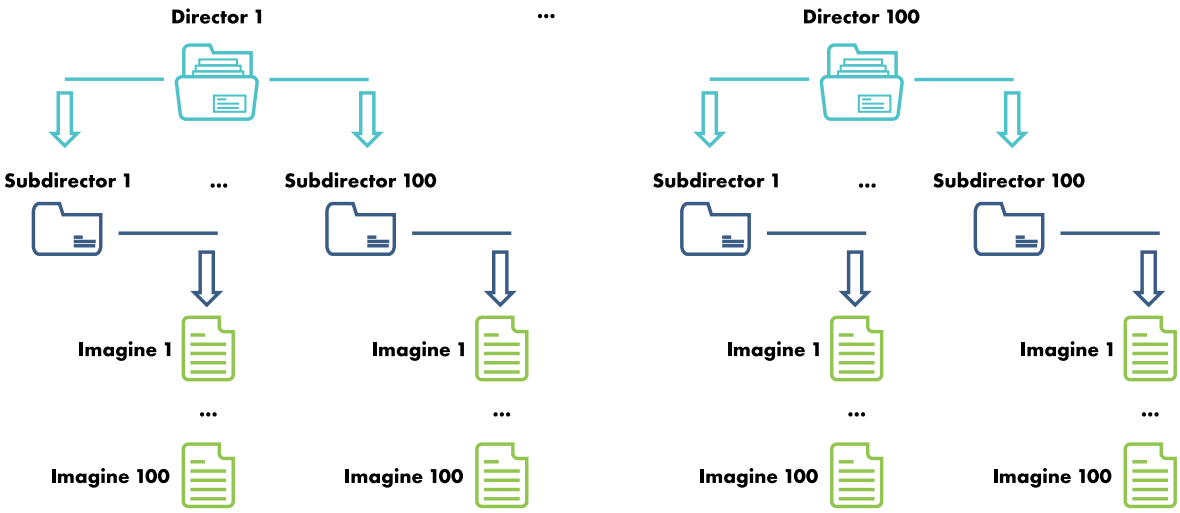


Figura 2.2: Exemplu de organizare ierarhică pe 3 niveluri a unui set de 1.000.000 poze

volumului imens de informații și o face accesibilă pentru utilizatorul uman. Mai mult despre găsirea fișierelor se află în Secțiunea 2.3.6 privind comenzile de căutare (**`find`**, **`locate`**, **`whereis`**, **`which`** și **`type`**).

Există însă un cost pentru acest control sporit al complexității. Fiecare director este un element suplimentar, creat de sistemul de fișiere, care trebuie să fie și el stocat undeva și ocupă astfel resurse. Aceste directoare nu există în structura plată, în care economisim astfel spațiu. În exemplul de mai sus, în structura ierarhică avem în total 100*100=10.000 directoare, create special pentru a organiza cele 1.000.000 de poze. Prin urmare, raportându-ne la numărul inițial de fișiere, pentru a controla mai bine informațiile plătim un cost de 10.000 / 1.000.000 = 1%.

Prezentăm în continuare exemple concrete de organizare a fișierelor în sistemele de operare moderne.

În Tabelul 2.1 este prezentată structura ierarhică în Linux.

În Figura 2.3 prezentăm cum arată grafic o ierarhie a sistemului de fișiere. Observăm că în rădăcină se află directoarele `home/`, `bin/`, `usr/` etc. În directorul `home/` se află subdirectoarele `ubuntu/` și `myuser/` ș.a.m.d.

> **Folosire caracter / (*slash*):** Atunci când folosim nume de directoare în Linux vom prefera să folosim sufixul `/` (*slash*) pentru a indica faptul că sunt directoare.

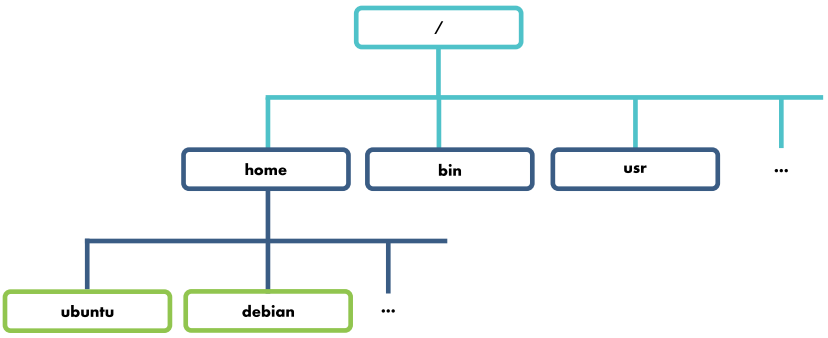


Figura 2.3: Ierarhia într-un sistem de fișiere din mediul Linux

Tabelul 2.1: Ierarhia într-un sistem de fișiere din mediul Linux

Director	Conținut
`/`	directorul rădăcină (*root directory*) – directorul cel mai cuprinzător, care conține celelalte directoare, considerate analoage trunchiului și ramurilor.
`/bin`	comenzi esențiale necesare bootării, întreținerii și depanării sistemului
`/boot`	fișiere necesare bootării, precum imaginea kernel-ului
`/dev`	fișiere speciale utilizate pentru accesul direct la dispozitivele hardware sau logice ale sistemului
`/etc`	fișiere pentru configurarea sistemului, precum `inittab`, `fstab` și `hosts`
`/home`	fișierele fiecărui utilizator din sistem - datele unui utilizator se găsesc în `/home/username`
`/media`	subdirectoare în care se montează unitățile optice, floppy etc.
`/mnt`	subdirectoare în care se montează alte sisteme de fișiere
`/opt`	pachete de aplicații de dimensiuni mari, accesibile tuturor utilizatorilor
`/proc`	sistem virtual de fișiere din care se obțin informații despre sistem și aplicațiile care rulează la un moment dat
`/root`	directorul home al utilizatorului root
`/sbin`	comenzi de bază accesibile numai utilizatorului root
`/tmp`	fișiere temporare
`/usr`	aplicații pentru uzul normal al sistemului de operare - `/usr/local` conține aplicațiile instalate/compilate de utilizator
`/var`	fișiere al căror conținut se schimbă foarte des, precum log-uri, fișiere temporare, cache (date reutilizabile), spool (date neprocesate)

Ierarhia sistemului de fișiere în macOS este apropiată celei din Linux.

Structura fișierelor în Windows diferă față de cea din Linux. Aceasta este mai simplă și majoritatea directoarelor importante se află în `C:\Windows`, așa cum se observă în Tabelul 2.2.

Deși în Linux și în macOS avem un singur director rădăcină, Windows are simultan pentru fiecare sistem de fișiere câte un director rădăcină:

- `A`, `B`: de obicei sunt rezervate pentru floppy disk-uri
- `C`: partiția de pe hard disk; pot exista mai multe, cărora li se asociază litere în ordine

Tabelul 2.2: Ierarhia într-un sistem de fișiere din Windows

Director	Conținut
`C:\`	directorul rădăcină
`C:\Windows`	Windows-ul și fișierele aferente
`C:\Documents and Settings`	configurările utilizatorilor și date specifice acestora
`C:\Program Files`	aplicații
`C:\Windows\System32`	drivere și fișiere de configurare Windows
`C:\Documents and Settings\username\My Documents`	datele unui utilizator (aceasta este calea implicită, ea poate fi modificată)

- `D` (sau următoarea literă disponibilă după partițiile de pe hard disk-uri): se referă la CD-ROM/DVD-Rom

Windows alocă literele în funcție de partiții, nu după sistemul de fișiere. Dacă se modifică sistemul de fișiere de pe o partiție, litera asignată partiției va rămâne aceeași. Pe o partiție se poate afla la un moment dat un singur sistem de fișiere.

În Tabelul 2.3 de mai jos avem o comparație între căile importante din sistemele de operare cele mai cunoscute.

Tabelul 2.3: Comparație între căile sistemelor de operare

Descriere	Windows	Linux	Mac OS
rădăcină	`C:`	`/`	`/`
director home	`C:\Documents and Settings\username`	`/home/username`	`/Users/username`
aplicații	`C:\Program Files`	`/bin`; `/sbin`; `/usr/bin`; `/usr/sbin`; `/usr/local/bin`;	`/opt/*/bin` `/Applications`; `/bin`; `/sbin`
configurări ale sistemului	Windows Registry	directoare specifice fiecărei aplicații, aflate în home-ul utilizatorului; `/etc`	`/Users/username/Library`; `/etc`

Observăm că în Linux înșiruirea de directoare este separată de caracterul `/` (*slash*), în timp ce în mediile Windows se folosește `\` (*backslash*).

2.1.3 Căi relative și căi absolute

Revenind la exemplul din Figura 2.3, dorim să găsim poza `selfiecumotanul-mai2018.jpg`, care se află în subdirectorul `cat-pics`, în directorul `99`. Pentru a putea accesa poza, avem două posibilități: putem sa folosim o cale relativă sau o cale absolută. Alegerea căii pe care o vom folosi depinde de unde ne aflăm în momentul respectiv în ierarhia de fișiere și unde se află fișierul căutat. O cale este echivalentul unei adrese pentru identificarea fișierului.

Calea absolută reprezintă adresa completă a fișierului, începând cu directorul rădăcină. Astfel, o cale absolută va începe cu `/` (*slash*) sau `~` (tildă, *tilde*) în cazul Linux/macOS sau cu `C:`, `D:` etc, în cazul Windows.

Calea relativă este o cale ce pornește din directorul curent. Pornind din directorul curent se construiește o cale către fișierul destinație dorit. O cale relativă **nu** începe cu `/` (*slash*) sau `~` (tildă, *tilde*) în cazul Linux/macOS sau cu `C:`, `D:` etc, în cazul Windows.

Directorul *home*: Simbolul `~` este o prescurtare în Linux/macOS pentru directorul *home* al utilizatorului. De obicei acesta este `/home/<username>/` în Linux și în `/Users/<username>/` în macOS.

În fiecare director se găsesc **două directoare speciale**: `.` (punct) și `..` (punct punct). Directorul `.` (*punct*) indică spre același director, directorul curent. `..` (*punct punct*) indică spre directorul părinte în ierarhia de fișiere și directoare.

Pentru a ne întoarce în ierarhia de fișiere pas cu pas, ne folosim de `..` pentru a ajunge în directorul părinte. Putem să înlănțuim mai multe grupări `..` (de ex. `../../..`) pentru a ne întoarce mai sus în ierarhie. Avem un exemplu de comenzi în Listing 2.1.

```
1  student@uso:~$ pwd
2  /home/student
3  student@uso:~$ cd uso.git/labs/
4  student@uso:~/uso.git/labs$ pwd
5  /home/student/uso.git/labs
6  student@uso:~/uso.git/labs$ cd ..
7  student@uso:~/uso.git$ pwd
8  /home/student/uso.git
9  student@uso:~/uso.git$ cd ../..
10 student@uso:/home$ pwd
11 /home
12 student@uso:/home$ cd
13 student@uso:~$ pwd
14 /home/student
15 student@uso:~$ cd /usr/local
16 student@uso:/usr/local$ pwd
17 /usr/local
```

Listing 2.1: Referința .. către directorul părinte

În exemplu de mai sus am folosit comanda **`pwd`** (*print working directory*) pentru a afișa directorul curent (numit și director de lucru). Directorul este indicat și în promptul comenzii, între caracterul `:` (două puncte) și caracterul `$` (dolar); simbolul ~ (tildă) este echivalent pentru directorul home al utilizatorului, în cazul de mai sus `/home/student/`. Comanda **`cd`** (*change directory*) schimbă directorul curent; comanda primește ca argument o cale care poate fi cale relativă sau absolută.

La linia 6 din exemplu de mai sus am schimbat directorul de lucru în directorul părinte folosind comanda **`cd ..`** și am urcat un nivel în ierarhia de directoare; observăm că este afișată o cale mai scurtă la linia 8 (`/home/student/uso.git`) față de calea anterioară de la linia 5 (`/home/student/uso.git/labs`). La linia 9 este o cale

relativă (`../..`) care urcă două niveluri în ierarhie. La linia 12 avem comanda **`cd`** fără argumente; această folosire a comenzii schimbă directorul de lucru în directorul home al utilizatorului, în cazul nostru `/home/student/`. La linia 15 avem o cale absolută (care începe cu `/` (*slash*)): `/usr/local`; comanda **`cd`** primește ca argument această cale și schimbă directorul curent în `/usr/local`.

Până acum am prezentat doar când folosim directorul special `..` (punct punct).

De multe ori, folosim construcția `.` (punct), care indică directorul curent, pentru comenzi ce execută scripturi/programe din acel director. Dacă în directorul curent avem un executabil numit `list_permissions`, atunci îl vom putea rula folosind comanda:

```
1 student@uso:~$ ./list_permissions
```

Aceasta înseamnă să se execute executabilul `list_permissions` din directorul curent.

Construcția `.` (punct) poate fi folosită în situații în care dorim să referim directorul curent. De exemplu dacă dorim să copiem un fișier în directorul curent rulăm o comandă similară cu cea de pe linia 5 din Listing 2.2.

```
1 student@uso:~$ pwd
2 /home/student
3 student@uso:~$ ls
4 Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
    examples.desktop  uso.git  vm-actions-log.txt
5 student@uso:~$ cp /etc/passwd .
6 student@uso:~$ ls
7 Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
    examples.desktop  passwd  uso.git  vm-actions-log.txt
```

Listing 2.2: Referința . către directorul curent

În exemplu de mai sus am folosit comanda **`ls`** (*list*) pentru a afișa conținutul directorului curent (chiar directorul home `/home/student`). Apoi, la linia 5 am folosit comanda **`cp`** (*copy*) pentru a copia fișierul `/etc/passwd` (cale absolută), dat ca primul argument, în directorul curent, reprezentat de construcția `.` (punct), dat ca al doilea argument. Apoi, la afișarea conținutului directorului (folosind comanda **`ls`**) vedem prezența fișierului `passwd`, acum copiat. În exemplu am folosit construcția `.` (punct) pentru a referi directorul curent, destinația comenzii de copiere.

2.2 Formatul fișierelor

Din perspectiva utilizatorului, fișierele se împart în diverse categorii, precum muzică, poze, jocuri și altele. Toate acestea sunt văzute de calculator ca o colecție de biți ce trebuie prelucrați pentru a putea fi redați. Calculatorul prelucrează fișierele în funcție de **formatul** acestora, pentru a ști ce programe sunt necesare pentru a le putea deschide și pentru a putea lucra cu ele.

Pentru început, putem clasifica fișierele în două mari categorii: fișiere de tip text (*text file*) și fișiere binare (*binary file*).

- **Fișierele de tip text** conțin linii compuse din caractere citibile (litere, cifre, semne de punctuație) fără să conțină elemente ce trebuie să fie interpretate de un

program (precum grafice, cod executabil etc). Fișierele text pot conține text simplu (*plain text*), având extensia `.txt`, sau care conți cod sursă (de exemplu cu extensia `.c` sau `.java`) sau formate de prezentare precum HTML.

- **Fișierele binare** sunt toate fișierele care nu sunt de tip text, putând reprezenta: imagini, programe executabile, melodii, fișiere comprimate etc.

Formatul (sau tipul) fișierelor se referă la modul de codificare a informației în fișier, care permite apoi redarea sau utilizarea informației prin intermediul unei interfețe sau a unei aplicații. Formatul fișierului specifică felul în care informația va fi codificată în biți, în mediul digital.

Formatul fișierului este, de regulă, asociat cu **extensia** acestuia. Extensia reprezintă sufixul de la finalul numelui fișierului, separat de numele fișierului printr-un punct. Exemple de extensii sunt: `.txt` (fișiere text), `.tex` (document sursă LaTeX), `.mp3` (format audio), `.bmp` (imagine tip *bitmap*), `.png` (imagine tip *Portable Network Graphic*), etc.

Este important să remarcăm că, **dacă schimbăm manual extensia unui fișier, nu înseamnă că i-am schimbat tipul**. Formatul fișierului ține de proprietățile conținutului acestuia și nu se schimbă dacă modificăm numele sau extensia. Dacă, de exemplu, avem un fișier text în care am pus versurile cântecului „The Kinslayer" al formației Nightwish, și schimbăm numele fișierului din `kinslayer.txt` în `kinslayer.mp3`, fișierul text nu devine dintr-o dată cântec și, desigur, nu îl vom putea deschide cu o aplicație de tip MP3 player.

Fișierele pot fi convertite dintr-un format într-altul, de regulă în interiorul categoriilor mari de conținut. De exemplu, putem converti un fișier `.wav` într-un fișier `.mp3`, sau un fișier `.bmp` într-un fișier `.png`, sau un fișier `.doc` într-un fișier `.pdf`. Este posibil să convertim și fișiere în formate ce par foarte diferite, de exemplu un fișier `.txt` într-un fișier `.mp3`, apelând la soluții automate text-to-speech. Convertirea se realizează nu prin schimbarea manuală a extensiei, ci prin intermediul unei aplicații cu care deschidem fișierul și îl transformăm în formatul dorit.

Pentru a vedea de ce tip este un fișier în Linux, dincolo de extensie, folosim comanda **`file`**. Mai jos avem exemple de folosire a comenzii **`file`** pentru determinarea tipului unui fișier. Pentru fiecare fișier primit ca argument de comanda **`file`** sunt indicate informații despre tipul fișierului: executabil, imagine de kernel, script shell, fișier comprimat, fișier imagine. Listing 2.3 conține exemple de rulare a comenzii **`file`**.

```
1 student@uso:~$ file /bin/ls
2 /bin/ls: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
  dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/
  Linux 3.2.0, BuildID[sha1]=9567f9a28e66f4d7ec4baf31cfbf68d0410f0ae6,
  stripped
3 student@uso:~$ file /boot/vmlinuz-4.15.0-29-generic
4 /boot/vmlinuz-4.15.0-29-generic: Linux kernel x86 boot executable bzImage
  , version 4.15.0-29-generic (buildd@lgw01-amd64-057) #31-Ubuntu SMP Tue
  Jul 17 15:39:52 UTC 2018, RO-rootFS, swap_dev 0x7, Normal VGA
5 student@uso:~$ file /etc/init.d/ssh
6 /etc/init.d/ssh: POSIX shell script, ASCII text executable
7 student@uso:~$ file /usr/share/man/man1/cp.1.gz
8 /usr/share/man/man1/cp.1.gz: gzip compressed data, max compression, from
  Unix
9 student@uso:~$ file /usr/share/pixmaps/htop.png
```

```
10 /usr/share/pixmaps/htop.png: PNG image data, 128 x 128, 8-bit/color RGBA,
    non-interlaced
```

Listing 2.3: Aflarea tipului unui fișier (comanda file)

În Listing 2.3 am analizat cinci fișiere folosind comanda **`file`** și au rezultat următoarele tipuri:

- fișierul `/bin/ls` este un executabil de tip ELF (*Executable and Linking Format*)
- fișierul `/boot/vmlinuz-4.15.0-29-generic` este o imagine de nucleu (*kernel*) de sistem de operare
- fișierul `/etc/init.ssh` este un script shell
- fișierul `/usr/share/man/man1/cp.1.gz` este un fișier comprimat de format GZIP
- fișierul `/usr/share/pixmaps/htop.png` este un fișier imagine PNG

Comanda **`file`** lucrează independent de extensia fișierului. Acest lucru este avantajos deoarece extensia poate să fie atribuită greșit de către utilizator, precum putem vedea și în Listing 2.4. Chiar dacă extensia fișierul a fost schimbată din `.jpg` în `.txt`, comanda **`file`** detectează în continuare tipul corect al fișierului: fișier de tip imagine JPEG.

```
1 student@uso:~$ file photo.jpg
2 photo.jpg: JPEG image data, JFIF standard 1.01
3 student@uso:~$ mv photo.jpg fisier.txt
4 student@uso:~$ file fisier.txt
5 fisier.txt: JPEG image data, JFIF standard 1.01
```

Listing 2.4: Detectarea tipului unui fișier cu extensie greșită

2.2.1 Atributele fișierelor

Pentru a controla funcționarea sistemelor de fișiere este importantă distincția dintre **date** și **metadate**. Un fișier conține efectiv informații sau date, precum muzică, versuri, imagini sau bilete de avion.

> **Metadatele** sunt informații despre informații: cantitatea informațiilor, data ultimei accesări, cine le-a creat, cine le-a modificat, unde anume au fost editate ultima dată.

Sistemele de fișiere se bazează pe crearea și gestionarea metadatelor referitoare la fișiere. Aceste metadate poartă denumirea de **atribute**. Ele fac posibilă accesarea eficientă și securizată a informațiilor.

În Linux putem afișa atribute ale fișierelor folosind comanda **`ls`** cu opțiunea `-l`; opțiunea înseamnă *long listing* și este folosită pentru afișarea detaliată (cu atribute a fișierelor), ca în Listing 2.5. În Listing 2.5 vedem pe coloane, atribute ale fișierelor, precum: permisiuni, informații legate de posesie (utilizator), dimensiune, data ultimei modificări, numele fișierului.

```
1 student@uso:~$ ls -l
2 total 60
3 drwxr-xr-x  2 student student 4096 aug  6 17:41 Desktop
```

```
4  drwxr-xr-x  3 student student 4096 aug 20 21:00 Documents
5  drwxr-xr-x  2 student student 4096 aug  6 17:41 Downloads
6  drwxr-xr-x  2 student student 4096 aug  6 17:41 Music
7  drwxr-xr-x  2 student student 4096 aug  6 17:41 Pictures
8  drwxr-xr-x  2 student student 4096 aug  6 17:41 Public
9  drwxr-xr-x  2 student student 4096 aug  6 17:41 Templates
10 drwxr-xr-x  2 student student 4096 aug  6 17:41 Videos
11 -rw-r--r--  1 student student 8980 aug  6 17:37 examples.desktop
12 -rw-r--r--  1 student student 2506 sep 30 11:28 passwd
13 drwxr-xr-x 15 student student 4096 aug 24 14:52 uso.git
14 -rw-r--r--  1 student student 4827 aug 21 14:37 vm-actions-log.txt
```

Listing 2.5: Afișarea atributelor fișierelor (folosind ls)

Cele mai simple atribute ale unui fișier sunt numele, dimensiunea și tipul. Tipul fișierului este indicat de primul caracter din rezultatul rulării comenzii **`ls -l`**, și poate fi:

- `-` = regular file
- `b` = block special file
- `c` = character special file
- `d` = directory
- `l` = symbolic link
- `n` = network file
- `p` = fIFO
- `s` = socket

În rularea din Listing 2.5 avem fișiere (primul caracter este `-`) și directoare (primul caracter este `d`).

Alte atribute importante descriu permisiunile, adică operațiile pe care diferite tipuri de utilizatori le pot realiza asupra respectivului fișier. Informații detaliate privind **tipurile de utilizatori** și configurarea permisiunilor acestora se regăsesc în Capitolul 5.

- `r` = permisiunea de a citi fișierul
- `w` = permisiunea de a scrie în fișier
- `x` = permisiunea de a executa fișierul
- `-` = absența permisiunii

2.3 Operații uzuale asupra fișierelor și directoarelor

Sistemul de fișiere ne permite realizarea mai multor tipuri de operațiuni asupra fișierelor. Operațiile uzuale asupra fișierelor includ: afișarea și schimbarea directorului, afișarea conținutului fișierului, listarea fișierelor dintr-un director, crearea fișierelor sau a directoarelor, copierea, mutarea, redenumirea sau ștergerea acestora, precum și arhivarea/dezarhivarea și realizarea unei versiuni de backup.

O problemă poate apărea dacă doi utilizatori doresc să citească sau să modifice același fișier simultan, deoarece nu este clar dacă acțiunile celor doi nu se vor încurca reciproc.

De aceea, una dintre responsabilitățile sistemului de fișiere este să mențină separarea resurselor resurselor între utilizatori. Vom prezenta detaliat aceste aspecte în Capitolul 5.

2.3.1 Afișarea și schimbarea directorului curent

Pentru a afișa directorul curent folosim comand **pwd** (*print working directory*). De asemenea, dacă nu s-au efectuat manual schimbări asupra prompt-ului bash, aceste va afișa implicit directorul în care ne aflăm. Dacă dorim să ne mutăm în alt director, vom folosi comanda **cd** <**cale**>. Comanda **cd** are ca parametru o cale absolută sau relativă către destinația în care vrem să ajungem.

Urmărim cum funcționează cele două comenzi – **pwd** și **cd** – în Listing 2.6. Vedem cum se modifică prompt-ul shellului atunci când schimbăm directorul curent. Atunci când schimbăm directorul,

```
1  student@uso:~$ pwd
2  /home/student
3  student@uso:~$ cd ..
4  student@uso:/home$ pwd
5  /home
6  student@uso:/home$ cd ../usr/bin/
7  student@uso:/usr/bin$ pwd
8  /usr/bin
9  student@uso:/usr/bin$ cd .
10 student@uso:/usr/bin$ pwd
11 /usr/bin
12 student@uso:/usr/bin$ cd /
13 student@uso:/$ pwd
14 /
15 student@uso:/$ cd
16 student@uso:~$ pwd
17 /home/student
18 student@uso:~$ cd /usr/bin
19 student@uso:/usr/bin$ pwd
20 /usr/bin
21 student@uso:/usr/bin$ cd /home
22 student@uso:/home$ pwd
23 /home
24 student@uso:/home$ cd -
25 /usr/bin
26 student@uso:/usr/bin$ pwd
27 /usr/bin
28 student@uso:/usr/bin$ cd ~
29 student@uso:~$ pwd
30 /home/student
31 student@uso:~$ cd ./././.
32 student@uso:~$ pwd
33 /home/student
34 student@uso:~$ cd ../..
35 student@uso:/$ pwd
36 /
```

Listing 2.6: Schimbarea directorului curent

În exemplu de mai sus:

- **cd ..** (comanda de la linia 3) ne întoarce în directorul părinte

- **`cd ../usr/bin`** (comanda de la linia 6) este o cale relativă care pornește din directorul părinte al directorului curent
- **`cd .`** (comanda de la linia 9) nu modifică directorul, deoarece **`.`** (punct) face referire la directorul curent
- **`cd ~`** (comanda de la linia 28) ne deplasează în directorul home al utilizatorului curent; ~ (tildă) este echivalentul directorului home pentru utilizatorul curent
- **`cd`** (comanda de la linia 15) schimbă directorul tot în directorul home
- **`cd -`** (comanda de la linia 24) ne întoarce în directorul în care ne aflam anterior

2.3.2 Listarea fișierelor

Acum că știm să navigăm dintr-un director în altul, ne interesează să afișăm conținutul acestora. Comanda pe care o folosim este **`ls [opțiuni] <cale>`**. Dacă dorim să listăm conținutul directorului curent, executăm comanda **`ls`** fără a mai fi nevoie să specificăm calea.

În continuare găsim opțiunile folosite frecvent pentru această comandă:

- `-l` = afișează detalii despre fiecare director/fișier, precum dimensiunea, utilizator, grup, data modificare, drepturi de acces
- `-a` = afișează toate fișierele, inclusiv cele ascunse (cele care încep cu **`.`** (caracterul punct))
- `-h` = afișează dimensiunea fișierelor în format ușor de înțeles, respectiv dimensiunea în octeți este înlocuită cu dimensiunea în Kocteți/Mocteți/Gocteți dacă depășește un anumit ordin de mărime

Un exemplu de utilizare a opțiunii `-l` a comenzi **`ls`** se găsește în Listing 2.7.

```
1 student@uso:~$ ls -l uso.git/README.md
2 -rw-r--r-- 1 student student 146 aug 20 20:57 uso.git/README.md
```

Listing 2.7: Opțiunea -l a comenzii ls

În output-ul de pe linia 2 din Listing 2.7:

- primul caracter ne spune tipul fișierului, în cazul de față fișier obișnuit
- următoarele 3 grupuri de caractere (`rwx`) reprezintă permisiunile de acces, pe care le vom detalia în Capitolul 5
- `student` este utilizatorul fișierului
- `student` (a doua apariție) este grupul de care aparține fișierul
- `146` este dimensiunea fișierului, în octeți
- `aug 20 20:57` este data ultimei modificări a fișierului
- `20:28` este ora ultimei modificări
- `uso.git/README.md` este numele fișierului

Listing 2.8 conține alte exemple de folosire a comenzii **`ls`**, cu următoarele exeplicații:

- comanda de la linia 19 (**`ls -l`**) afișează toate informațiile (`-l`) ale tuturor fișierelor, ascunse și neascunse, din directorul curent; dimensiunea fișierelor este afișată în format inteligibil (*human-readable*) (`-h`); observăm că sunt sunt afișate și directoarele standard **`.`** și **`..`**
- comanda de la linia 38 primește o cale relativă pentru a-i afișa conținutul
- comanda de la linia 41 afișează toate fișierele, ascunse și neascunse, din rădăcina ierarhiei de fișiere și directoare (`/`)

```
1  student@uso:~$ ls
2  Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
     examples.desktop  passwd  uso.git  vm-actions-log.txt
3  student@uso:~$ ls ~
4  Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
     examples.desktop  passwd  uso.git  vm-actions-log.txt
5  student@uso:~$ ls -l
6  total 60
7  drwxr-xr-x  2 student student 4096 aug  6 17:41 Desktop
8  drwxr-xr-x  3 student student 4096 aug 20 21:00 Documents
9  drwxr-xr-x  2 student student 4096 aug  6 17:41 Downloads
10 drwxr-xr-x  2 student student 4096 aug  6 17:41 Music
11 drwxr-xr-x  2 student student 4096 aug  6 17:41 Pictures
12 drwxr-xr-x  2 student student 4096 aug  6 17:41 Public
13 drwxr-xr-x  2 student student 4096 aug  6 17:41 Templates
14 drwxr-xr-x  2 student student 4096 aug  6 17:41 Videos
15 -rw-r--r--  1 student student 8980 aug  6 17:37 examples.desktop
16 -rw-r--r--  1 student student 2506 sep 30 11:28 passwd
17 drwxr-xr-x 15 student student 4096 aug 24 14:52 uso.git
18 -rw-r--r--  1 student student 4827 aug 21 14:37 vm-actions-log.txt
19 student@uso:~$ ls -lh
20 total 60K
21 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Desktop
22 drwxr-xr-x  3 student student 4,0K aug 20 21:00 Documents
23 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Downloads
24 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Music
25 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Pictures
26 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Public
27 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Templates
28 drwxr-xr-x  2 student student 4,0K aug  6 17:41 Videos
29 -rw-r--r--  1 student student 8,8K aug  6 17:37 examples.desktop
30 -rw-r--r--  1 student student 2,5K sep 30 11:28 passwd
31 drwxr-xr-x 15 student student 4,0K aug 24 14:52 uso.git
32 -rw-r--r--  1 student student 4,8K aug 21 14:37 vm-actions-log.txt
33 student@uso:~$ ls -a
34 .              .bash_history  .config   .gconf      .local    .
   sudo_as_admin_successful  .vboxclient-display.pid      .viminfo
   Downloads  Templates         uso.git
35 ..             .bash_logout   .dbus     .gitconfig  .mozilla  .tmux
                        .vboxclient-draganddrop.pid  .vimrc     Music
   Videos            vm-actions-log.txt
36 .ICEauthority  .bashrc        .emacs    .gnupg      .profile  .tmux.conf
                    .vboxclient-seamless.pid     Desktop    Pictures
   examples.desktop
37 .bash_aliases  .cache         .emacs.d  .lesshst    .ssh      .vboxclient
   -clipboard.pid  .vim                          Documents  Public     passwd
38 student@uso:~$ ls ../..
39 bin   cdrom  etc   initrd.img      lib    lib64   lost+found  mnt  proc
   run   snap  swapfile  tmp  var      vmlinuz.old
```

```
40 boot  dev    home  initrd.img.old  lib32  libx32  media       opt  root
   sbin  srv   sys       usr  vmlinuz
41 student@uso:~$ ls -a /
42 .   bin   cdrom  etc   initrd.img      lib    lib64   lost+found  mnt
   proc  run   snap  swapfile  tmp  var      vmlinuz.old
43 ..  boot  dev    home  initrd.img.old  lib32  libx32  media       opt
   root  sbin  srv   sys       usr  vmlinuz
```

Listing 2.8: Folosirea comenzii ls

O opțiune folosită a comenzii **ls** este -R, care afișează recursiv directoarele și fișierele ce au ca rădăcină directorul dat ca argument, exemplificată în Listing 2.9. Prin recursiv înțelegem că trece prin toate directoarele pornind de la directorul dat ca argument.

```
1  student@uso:~$ ls -R /usr/local/lib/
2  /usr/local/lib/:
3  python2.7  python3.6
4
5  /usr/local/lib/python2.7:
6  dist-packages  site-packages
7
8  /usr/local/lib/python2.7/dist-packages:
9
10 /usr/local/lib/python2.7/site-packages:
11
12 /usr/local/lib/python3.6:
13 dist-packages
14
15 /usr/local/lib/python3.6/dist-packages:
```

Listing 2.9: Listarea recursivă a conținutului unui director

Dacă dorim să folosim alte opțiuni, putem să consultăm **man ls** sau **ls --help**.

2.3.3 Afișarea conținutului fișierelor

Pentru a putea vedea ce conține un fișier, folosim comanda **cat** <**nume fișier**>, ca în Listing 2.10.

```
1  student@uso:~$ cat /etc/default/saned
2  # Defaults for the saned initscript, from sane-utils
3
4  # To enable under systemd please read README.Debian
5  # Set to yes to start saned under SysV
6  RUN=no
7
8  # Set to the user saned should run as
9  RUN_AS_USER=saned
```

Listing 2.10: Afișarea conținutului unui fișier

Comanda va întoarce conținutul întregului fișier, indiferent de lungime. Pentru ca utilizatorul să poată naviga prin tot output-ul comenzii, putem folosi comanda **less**, ca mai jos:

```
1  student@uso:~$ less /etc/X11/Xsession
```

Comanda **less** are o interfață asemănătoare cu cea a editorului **vim** și permite navigarea în ambele direcții, linie cu linie.

2.3.4 Crearea fișierelor/directoarelor

Pentru a crea un fișier gol, avem două metode. Prima metoda este utilizarea comenzii **`touch <nume fișier>`**, iar a doua se bazează pe o funcționalitate a shellului numită redirectare în fișier, ca în Listing 2.11, cu următoarele explicații:

- comanda de la linia 1 folosește prima metodă; creează un fișier gol cu numele `cats.txt` în directorul curent
- comanda de la linia 2 folosește a doua metodă; redirectează (`>`) ieșirea unei comenzi nule într-un fișier, ducând la trunchierea fișierului, dacă există, sau, în cazul nostru, ducând la crearea fișierului gol `dogs.txt`
- vedem din rezultatul rulării comenzii `ls -l` de la linia 5 că ambele fișiere sunt create cu dimensiune 0, sunt goale

```
1 student@uso:~/Downloads$ touch cats.txt
2 student@uso:~/Downloads$ > dogs.txt
3 student@uso:~/Downloads$ ls
4 cats.txt  dogs.txt
5 student@uso:~/Downloads$ ls -l
6 total 0
7 -rw-rw-r-- 1 student student 0 sep 30 12:20 cats.txt
8 -rw-rw-r-- 1 student student 0 sep 30 12:20 dogs.txt
```

Listing 2.11: Crearea unui fișier

Comanda **`touch`** are în principal rolul pentru a actualiza data ultimei accesări sau modificări asupra fișierului; dacă un fișier nu există, acesta este creat cu data curentă; crearea fișierului este un rol secundar al comenzii **`touch`**. În Listing 2.12 folosim comanda **`touch`** pentru a schimba data ultimei modificări din `aug 20 20:57` în `sep 30 12:24`.

```
1 student@uso:~$ ls -l uso.git/README.md
2 -rw-r--r-- 1 student student 146 aug 20 20:57 uso.git/README.md
3 student@uso:~$ touch uso.git/README.md
4 student@uso:~$ ls -l uso.git/README.md
5 -rw-r--r-- 1 student student 146 sep 30 12:24 uso.git/README.md
```

Listing 2.12: Schimbarea datei ultimei modificări a unui fișier

2.3.4.1 Crearea legăturilor simbolice

Exemplu următor prezintă crearea legăturilor simbolice, un mod prin care același fișier poate fi referit din două căi, echivalentul scurtăturilor.

```
1 student@uso:~$ cat uso.git/README.md
2 uso
3 ===
4
5    * Directorul ''lab02'' contine toate fisierele si structura de
  directoare necesare rezolvarii laboratorului 2 de catre studenti
6 student@uso:~$ ln -s uso.git/README.md readme_link
7 student@uso:~$ ls -l readme_link
8 lrwxrwxrwx 1 student student 17 sep 30 12:28 readme_link -> uso.git/
  README.md
```

```
9  student@uso:~$ cat readme_link
10 uso
11 ===
12
13    * Directorul ''lab02'' contine toate fisierele si structura de
   directoare necesare rezolvarii laboratorului 2 de catre studenti
```

În exemplul de mai sus am creat intrarea `readme_link` care este o legătură simbolică spre fișierul **`uso.git/README.md`**. Dacă afișăm, folosind comanda **`cat`**, fișierul sau legătura simbolică, obținem același conținut: legătura simbolică referă același fișier.

Legăturile (*link-urile*) numite și legături simbolice (*symbolic links*) sunt create cu ajutorul comenzii `ln` căreia îi transmitem ca argument opțiunea `-s`.

2.3.4.2 Crearea directoarelor

Pentru a crea un director folosim comanda **`mkdir`** ca în Listing 2.13.

```
1  student@uso:~$ mkdir games
2  student@uso:~$ ls -d games/
3  games/
4  student@uso:~$ ls -ld games/
5  drwxrwxr-x 2 student student 4096 sep 30 17:28 games/
6  student@uso:~$ ls games/
7  student@uso:~$ mkdir games/warcraft
8  student@uso:~$ mkdir games/lol
9  student@uso:~$ ls games/
10 lol  warcraft
11 student@uso:~$ mkdir -p games/heroes/3/nighon/mutare
12 student@uso:~$ ls -ld games/heroes/3/nighon/mutare
13 drwxrwxr-x 2 student student 4096 sep 30 17:32 games/heroes/3/nighon/
   mutare
```

Listing 2.13: Crearea directoarelor

În Listing 2.13 am creat directorul `games/` iar în cadrul său am creat subdirectoarele `warcraft/` și `lol/`. Comanda **`ls`** afișează conținutul unui diretor; dacă dorim să afișăm informații despre director în sine folosim comanda opțiunea `-d` a comenzii **`mkdir`**; o formă frecventă este folosirea opțiunii `-ld` pentru afișarea informațiilor despre director în format lung, cu atribute ale directorului. În partea finală a exemplului am folosit opțiunea `-p` a comenzii **`mkdir`** pentru a crea un director cu toate directoarele părinte necesare chiar dacă acestea nu există.

În Tabelul 2.4 sunt sumarizate comenzile de creare a tipurilor de intrări în sistemul de fișiere.

2.3.5 Copiere / mutare / redenumire / ștergere

Operațiile frecvente cu fișiere sunt copierea, mutarea sau redenumirea și ștergerea. Copierea înseamnă că un fișier sau director este duplicat în alt loc, și se găsește acum în două locuri. Mutarea / Redenumirea înseamnă cu un fișier sau director este acum mutat din locul său inițial în alt loc în sistemul de fișiere; spre deosebire de copiere,

Tabelul 2.4: Comenzi pentru crearea intrărilor în sistemul de fișere

Tip de intrare	Comandă
fișier obișnuit (regular file)	**touch <nume_fișier>** sau **> <nume_fișier>**
director	**mkdir <nume_director>**
legătură (link)	**ln -s <destinatie> [<nume_legătură>]**

acum fișierul / directorul se găsește într-un singur loc. Ștergerea înseamnă eliminarea acelui fișier sau director din sistemul de fișiere; nu se va mai găsi în nici un loc.

Comenzile de copiere și mutare primesc doi parametri:

- primul parametru reprezintă sursa de unde dorim să copiem/mutăm
- al doilea parametru reprezintă destinația unde dorim să copiem/mutăm

> **Observație:** Primul parametru (sursa) al comenzilor din acest subcapitol poate fi reprezentată și prin globbing în sistemul de fișiere, despre care vom discuta detaliat în Capitolul 7.

2.3.5.1 Copierea

Comanda folosită pentru a copia un fișier este **cp** [**opțiuni**] <**sursă**> <**destinație**>. În Listing 2.14 este prezentat un exemplu de copiere a unui fișier și unul de copiere a unui director. Pentru copierea unui director folosim opțiunea `-r` a comenzii **cp** pentru copiere recursivă (care trece prin toate subdirectoarele).

```
1  student@uso:~$ cp passwd passwd-copy
2  student@uso:~$ ls -l passwd passwd-copy
3  -rw-r--r-- 1 student student 2506 sep 30 11:28 passwd
4  -rw-r--r-- 1 student student 2506 sep 30 16:48 passwd-copy
5  student@uso:~$ cmp passwd passwd-copy
6  student@uso:~$ cp uso.git/labs/01-fs/wiki/basics.wiki .
7  student@uso:~$ cmp basics.wiki uso.git/labs/01-fs/wiki/basics.wiki
8  student@uso:~$ cp -r uso.git/labs/ .
9  student@uso:~$ diff -r uso.git/labs/ labs/
10 student@uso:~$
```

Listing 2.14: Copierea fișierelor și directoarelor

În Listing 2.14, în primă fază facem o copie, în același director a fișierul `passwd` în fișierul `passwd-copy`. Apoi folosim comanda **ls -l** pentru a vedea că cele două fișiere au aceeași dimensiune (`2506`). Comanda **cmp** compară două fișiere și, dacă sunt identice, nu afișează nimic. Apoi copiem un fișier dintr-o cale relativă (`uso.git/labs/01-fs/wiki/basics.wiki`) în directorul curent, indicat de construcția **.** (punct). Folosim comanda **cmp** pentru a valida că cele două fișiere sunt identice. Apoi copiem un director dintr-o cale relativă (`uso.git/labs/`) în directorul curent. Pentru a compara două directoare și pentru a demonstra că sunt identice,

folosim comanda **diff** cu opțiunea `-r`, pentru parcurgere recursivă a argumentelor de comparat.

2.3.5.2 Mutarea / Redenumirea

Comanda folosită pentru a muta/redenumi un fișier este **mv** [**opțiuni**] <**sursă**> <**destinație**>. Mutarea se va face implicit recursiv și va păstra toate atributele fișierelor. Când se execută comanda **mv**, se schimbă doar părintele fișierului (sau directorului) pe care îl mutăm. Spre deosebire de comanda **cp**, comanda **mv** nu primește opțiunea `-r` pentru mutarea unui director; o intrare este mutată indiferent dacă este director sau fișier.

Listing 2.15 conține exemple de folosire a comenzii **mv**.

```
student@uso:~$ mv basics.wiki 01-fs-basics.wiki
student@uso:~$ ls basics.wiki 01-fs-basics.wiki
ls: cannot access 'basics.wiki': No such file or directory
01-fs-basics.wiki
student@uso:~$ mv passwd-copy Downloads/
student@uso:~$ ls passwd-copy
ls: cannot access 'passwd-copy': No such file or directory
student@uso:~$ ls Downloads/passwd-copy
Downloads/passwd-copy
student@uso:~$ mv labs/ labs-old
student@uso:~$ ls labs
ls: cannot access 'labs': No such file or directory
student@uso:~$ ls labs-old/
00-intro  01-fs  02-process  03-user  04-appdev  05-cli  06-hw-boot  07-
storage  08-net  09-vm  10-sec  11-ctf
```

Listing 2.15: Mutarea / Redenumirea fișierelor și directoarelor

În Listing 2.15 am redenumit fișierul `basics.wiki` în `02-fs-basics.wiki`. Am mutat fișierul `passwd-copy` în subdirectorul `Downloads/`, dat prin cale relatativă. Și am redenumit directorul `labs/` în `labs-old/`. La fiecare operație de mutare/redenumire am verificat că vechea intrare nu mai este disponibilă, dar este disponibilă noua intrare.

2.3.5.3 Ștergerea fișierelor/directoarelor

Ștergerea unei intrări înseamnă eliminarea acelei intrări și a conținutului său din sistemul de fișiere. Comanda cea mai folosită pentru ștergerea fișierelor și directoarelor este **rm** [**opțiuni**] <**cale**>. Opțiunile cele mai des folosite pentru această comandă sunt:

- `-r/-R`: se utilizează atunci când se dorește ștergerea recursivă a unui director
- `-f`: se utilizează pentru a șterge forțat fișierele

Este important să fim atenți când folosim această comandă și opțiunile ei, pentru a nu șterge informații utile din greșeală și apoi să nu le mai putem recupera. Una dintre cele mai întâlnite erori este **rm -rf /**, care șterge recursiv toate fișierele începând cu rădăcina.

> **Observație:** Dacă dorim să ștergem un director gol, putem să folosim comanda **rmdir** <**director_gol**>

Listing 2.16 conține exemple de folosire a comenzilor **rm** și **rmdir**.

```
1  student@uso:~$ ls
2  02-fs-basics.wiki  Desktop  Documents  Downloads  Music  Pictures  Public
     Templates  Videos  examples.desktop  labs-old  passwd  readme_link  uso
   .git  vm-actions-log.txt
3  student@uso:~$ rm 02-fs-basics.wiki
4  student@uso:~$ rm passwd
5  student@uso:~$ rm readme_link
6  student@uso:~$ rm -fr labs-old
7  student@uso:~$ rmdir Downloads/
8  rmdir: failed to remove 'Downloads/': Directory not empty
9  student@uso:~$ rm Downloads/*
10 student@uso:~$ rmdir Downloads/
11 student@uso:~$ ls
12 Desktop  Documents  Music  Pictures  Public  Templates  Videos  examples.
   desktop  uso.git  vm-actions-log.txt
```

Listing 2.16: Ștergerea fișierelor și directoarelor

În Listing 2.16 am șters, folosind comanda **rm** fișierele `basics-wiki` și `passwd` și legătura simbolică `readme_link`. Apoi am șters recursiv (folosind opțiunea `-fr` (de la *force and recursive*) directorul `labs-old/`. Comanda **rmdir** poate șterge un director doar dacă acesta este gol; pentru aceasta am șters toate fișierele din directorul `Downloads/` folosind construcția `Downloads/*`; apoi am folosit comanda **rmdir** pentru a șterge directorul. Am folosit comanda **ls** la început și la sfârșit pentru a demonstra ștergerea fișierelor.

2.3.6 Căutarea fișierelor

Structura ierarhică a sistemului de fișiere permite găsirea rapidă a unui fișier parcurgând pe rând subdirectoarele care-l conțin. Chiar și așa sunt situații în care nu se știe precis unde este localizat un fișier sa un director și este utilă căutarea acestuia. Pentru a căuta un fișier după numele său există două tipuri de căutări:

- **căutare indexată** în care fișierele sunt inspectate și informațiile despre localizarea lor sunt reținute într-o bază de date cu fișierele indexată;
- **căutare exhaustivă** în cadrul unei zone din ierarhia sistemului de fișiere, trecând prin toate fișierele

Pentru căutare indexată folosim, în Linux, comanda **locate** (și comanda **updatedb**). Pentru căutare exhaustivă folosim, în Linux, comanda **find**. Le prezentăm în continuare.

2.3.6.1 Comanda locate

Comanda **locate** caută în toate fișierele indexate într-o bază de date locală. Avantajul său este căutarea rapidă în sistemul de fișiere. Are două dezavantaje:

1. Baza de date trebuie actualizată periodic pentru a indexa modificările din sistemul de fișiere. Actualizarea se face folosind comanda **updatedb**.
2. Căutarea se face strict după nume, nu și după alte atribute ale fișierelor.

Rezultatul rulării comenzii este o listă cu toate fișierele al căror nume conține șirul de caractere trimis ca argument.

În Listing 2.17 se caută toate fișierele al căror nume conține șirul `pwd`.

```
student@uso:~$ locate pwd
/bin/pwd
/etc/.pwd.lock
/lib/modules/4.15.0-29-generic/kernel/drivers/watchdog/hpwdt.ko
/lib/modules/4.15.0-32-generic/kernel/drivers/watchdog/hpwdt.ko
/lib/modules/4.15.0-34-generic/kernel/drivers/watchdog/hpwdt.ko
/sbin/pam_extrausers_chkpwd
/sbin/unix_chkpwd
/snap/core/4917/bin/pwd
[...]
```

Listing 2.17: Căutarea fișierelor folosind locate

Unele distribuții Linux folosesc comanda **slocate** în locul comenzii **locate**, care afișează doar fișierele din directoarele în care utilizatorul curent are drepturi de acces.

2.3.6.2 Comanda find

Pentru căutarea în ierarhia de directoare al fișierelor se folosește comanda **find**, comandă ce parcurge exhaustiv ierarhia de directoare. Comanda permite căutarea folosind diferite criterii, precum numele fișierului, utilizator, grup, tip, permisiuni, dimensiune, dată și altele. Exemple de opțiuni ale comenzii **find** pentru criterii de căutare:

- `-name` - căutare după numele fișierelor
- `-type` - căutare după tipul intrării (fișier, director, legătură simbolică)
- `-size` - căutare după dimensiunea fișierelor

Mai jos avem câteva exemple de folosire a comenzii **find**:

- în Listing 2.18 căutăm în ierarhia **/usr/** toate legăturile simbolice
- în Listing 2.19 căutăm în ierarhia **/usr/** toate fișierele care au numele `stat`
- în Listing 2.20 obținem fișierele care au o dimensiune mai mare de `500KB`

```
student@uso:~$ find /usr/ -type l
/usr/lib/pm-utils/sleep.d/95hdparm-apm
/usr/lib/libgjs.so.0
/usr/lib/bfd-plugins/liblto_plugin.so
/usr/lib/sudo/libsudo_util.so
/usr/lib/sudo/libsudo_util.so.0
/usr/lib/gold-ld/ld
/usr/lib/systemd/user/dbus-org.bluez.obex.service
/usr/lib/systemd/user/graphical-session.target.wants/update-notifier-
release.path
```

```
10 /usr/lib/systemd/user/graphical-session.target.wants/update-notifier-
   crash.path
11 /usr/lib/systemd/user/graphical-session.target.wants/unicast-local-avahi.
   path
12 [...]
```

Listing 2.18: Afișarea link-urilor simbolice din ierarhia /usr/

```
1 student@uso:~$ find /usr -name stat
2 /usr/bin/stat
3 /usr/src/linux-headers-4.15.0-29-generic/include/config/f2fs/stat
4 /usr/src/linux-headers-4.15.0-32-generic/include/config/f2fs/stat
5 /usr/src/linux-headers-4.15.0-34-generic/include/config/f2fs/stat
```

Listing 2.19: Afișarea fișierelor cu numele stat din ierarhia /usr/

```
1 student@uso:~$ find /usr/ -size +500k
2 /usr/lib/gnome-shell/gnome-shell-portal-helper
3 /usr/lib/gnome-shell/libgnome-shell.so
4 /usr/lib/p7zip/7zr
5 /usr/lib/xorg/modules/drivers/intel_drv.so
6 /usr/lib/xorg/Xorg
7 /usr/lib/debug/lib/x86_64-linux-gnu/libnsl-2.27.so
8 /usr/lib/debug/lib/x86_64-linux-gnu/libmvec-2.27.so
9 /usr/lib/debug/lib/x86_64-linux-gnu/ld-2.27.so
10 /usr/lib/debug/lib/x86_64-linux-gnu/libm-2.27.so
11 /usr/lib/debug/lib/x86_64-linux-gnu/libc-2.27.so
12 [...]
```

Listing 2.20: Afișarea fișierelor cu o dimensiune mai mare de 500KB din ierarhia /usr/

2.3.6.3 Căutarea comenzilor

Uneori dorim să identificăm o comandă sau executabilul aferent unei comenzi. Pentru aceasta folosim comenzile **`whereis`**, **`which`** și **`type`**.

Comanda **`whereis`** este utilizată pentru căutarea locurilor corespunzătoare unei comenzi în sistemul de fișiere De exemplu, pentru a localiza comanda **`ls`**, folosim:

```
1 student@uso:~$ whereis ls
2 ls: /bin/ls /usr/share/man/man1/ls.1posix.gz /usr/share/man/man1/ls.1.gz
```

Comanda afișează calea către executabil și către pagina de manual a comenzii.

Dacă dorim doar obținerea căii către executabilul aferent comenzii, folosim comanda **`which`**. De exemplu, comanda următoare afișează calea către executabilul aferent comenzii **`chmod`**:

```
1 student@uso:~$ which chmod
2 /bin/chmod
```

Utilizarea comenzii **`type`** duce la determinarea modului de interpretare a altei comenzi, de exemplu comandă integrată în shell (numită și internă sau *builtin*), comandă externă sau alias. Mai multe despre comenzi în shell, interne și externe, vor fi prezentate în Capitolul 7.

Un exemplu de comandă integrată in shell este comanda **`cd`** în vreme ce o comanda externă, care are un executabil asociat, este comanda **`cat`**, așa cum putem vedea în Listing 2.21.

```
1 student@uso:~$ type cd
2 cd is a shell builtin
3 student@uso:~$ type cat
4 cat is /bin/cat
5 student@uso:~$ type ls
6 ls is aliased to 'ls --color=auto'
```

Listing 2.21: Tipul unei comenzi

În cazul unui alias de comandă, comanda **type** va preciza că este un alias așa cum reiese din liniile 5 și 6 din Listing 2.21.

2.3.7 Arhivarea și dezarhivarea fișierelor

Prin procesul de arhivare, mai multe fișiere și directoare sunt strânse la un loc într-un fișier unic, realizându-se de obicei și reducerea dimensiunii (compresie). În general noțiunea de arhivare se referă doar la lipirea datelor într-un singur loc, în vreme ce compresia se referă la codificarea datelor pentru a reduce dimensiunea fișierului rezultat.

În cazul compresiei se creează un dicționar care conține secvențe de octeți care se repetă mai des și codificarea lor pe mai puțini octeți. Pe lângă dicționar, în fișierul rezultat este conținut și cuprinsul fișierelor. Fiind folosite referințele către dicționar a secvențelor lungi, se realizează astfel o economie de spațiu. Cu cât fișierele conțin mai multe date repetitive, cu atât dimensiunile fișierului comprimat rezultat scad.

Utilitare precum **zip** realizează atât arhivare cât și compresie; utilitarul **tar** realizează doar arhivare, iar utilitarul **gzip** realizează doar compresie; de obicei utilitarele **tar** și **gzip** sunt folosite la comun pentru a realiza și arhivarea și compresia.

Numele utilitarului **tar** este acronimul *tape archiver*, provenind de la faptul că, la origine, rezultatul era transferat pe benzi magnetice. Fișierele `.tar` au în componență fișierele inițiale necomprimate precum și informațiile legate de modul lor de extragere (spre exemplu: de unde până unde se găsește un fișier în cadrul arhivei). Din această cauză fișierele `.tar` au o dimensiune mai mare decât suma dimensiunilor fișierelor care îl alcătuiesc.

Listing 2.22 prezintă exemple de folosire ale comenzii **tar**.

```
1  student@uso:~$ tar cvf 01-fs.tar uso.git/labs/01-fs/
2  uso.git/labs/01-fs/
3  uso.git/labs/01-fs/wiki/
4  uso.git/labs/01-fs/wiki/basics.wiki
5  uso.git/labs/01-fs/wiki/concepts.wiki
6  uso.git/labs/01-fs/wiki/demo.wiki
7  [...]
8  student@uso:~$ tar tf 01-fs.tar
9  uso.git/labs/01-fs/
10 uso.git/labs/01-fs/wiki/
11 uso.git/labs/01-fs/wiki/basics.wiki
12 uso.git/labs/01-fs/wiki/concepts.wiki
13 uso.git/labs/01-fs/wiki/demo.wiki
14 [...]
15 student@uso:~$ cd /tmp/
16 student@uso:/tmp$ tar xf ~/01-fs.tar
```

```
17 student@uso:/tmp$ ls uso.git/labs/01-fs/
18 support  wiki
```

Listing 2.22: Folosirea comenzii tar

Mai sus am creat o arhivă `.tar` (linia `1`), am listat conținutul ei (linia `8`) și apoi am dezarhivat-o în directorul `/tmp/` (liniile `15-16`).

Opțiunile comenzii **`tar`**, folosite mai sus, sunt:

- `c` (pentru *create*): pentru a crea arhiva
- `x` (pentru *extract*): dezarhivează
- `t` (pentru *list*): listează conținut
- `v` (pentru *verbose*): arată ce se întâmplă
- `f nume_arhiva.tar` (pentru *file*): numele arhivei

> **Observație:** `f` și `nume_arhiva.tar` reprezintă un singur parametru – astfel, când folosim `f` pentru a indica un fișier, acesta trebuie să se afle ultimul în lista de parametri.

Cele mai folosite utilitare în Linux care realizează comprimarea unui fișier sunt:

- **`gzip`**: are o viteză mai ridicată de compresie, dar o rată de mai scăzută (fișiere rezultat mai mari)
- **`bzip2`**: are viteză mai scăzută de compresie, dar o rată de compresie mai ridicată

Comanda **`tar`** poate să utilizeze oricare dintre programele de compresie de mai sus și se folosește de parametrul `z` pentru gzip și `j` pentru bzip2 ca în Listing 2.23.

```
1 student@uso:~$ tar czf 01-fs.tar.gz uso.git/labs/01-fs/
2 student@uso:~$ tar cjf 01-fs.tar.bz2 uso.git/labs/01-fs/
3 student@uso:~$ file 01-fs.tar.*
4 01-fs.tar.bz2: bzip2 compressed data, block size = 900k
5 01-fs.tar.gz:  gzip compressed data, last modified: Sun Sep 30 17:02:19
  2018, from Unix
```

Listing 2.23: Arhivarea cu compresie folosind tar

În Listing 2.23 am folosit, respectiv, opțiunea `z` (în cadrul opțiunii `czf`) pentru a comprima folosind **`gzip`** (linia `1`), și apoi opțiunea `j` (în cadrul opțiunii `cjf`) pentru a comprima folosind **`bzip2`** (linia `2`). Apoi am folosit comanda **`file`** pentru a verifica tipul fișierelor și am confirmat tipul de compresie aferent celor două fișiere rezultate (liniile `3-5`).

2.3.8 Backup

Backupul este utilizat pentru a păstra într-un loc separat o copie a datelor, copie ce poate fi folosită pentru a le recupera în cazul în care, din diferite motive, suportul original nu mai poate fi folosit. Adesea omitem sau amânăm realizarea unei versiuni de backup a datelor, putând ajunge la pierderea informațiilor în situații de defecțiune. De aceea, în ultimii ani sunt foarte populare soluțiile de backup automat în Cloud. Astfel,

utilizatorii pot folosi soluții precum Google Drive, Dropbox, Microsoft OneDrive pentru a sincroniza automat conținutul directoarelor în care lucrează cu un spațiu de stocare cloud. Dacă utilizatorul lucrează pe mai multe dispozitive (desktop, laptop, mobil etc.) poate fi configurată sincronizarea tuturor acestor dispozitive cu cloudul, facilitând astfel transferul documentelor.

Chiar dacă realizăm sincronizarea automată prin Cloud, este util să creăm și backupuri periodice pe alte suporturi fizice. De exemplu, în cazul în care datele noastre sunt afectate de erori grave, datorită malware sau ștergerii accidentale, aceste erori se pot propaga și în Cloud, dar versiunile salvate anterior pe suporturi fizice distincte vor fi protejate.

Tabelul 2.5 prezintă câteva metode de backup și situațiile când sunt ele potrivite.

Tabelul 2.5: Metode pentru backup

Metoda	**Descriere**
tar + gzip/bzip2	Metodă foarte simplu de aplicat. Devine greu de folosit pentru dimensiuni mari de date. Permite compresia datelor.
dd	Metoda simplu de folosit și independentă de sistemul de fișiere. Permite păstrarea intactă a structurii sistemului de fișiere. Inflexibilă când vine vorba de recuperarea datelor. Utilă pentru cantități mari de date.
rsync	E asemănătoare comenzii cp dar la care s-a adăugat suport de sincronizare intre mai multe computere. Permite replicarea structurii de fișiere (inclusiv permisiuni) între 2 computere.
rdiff-backup	Este un wrapper peste rsync. Adaugă suport pentru backupuri incrementale, adică: la un moment dat se realizează un backup complet pentru un director (asemănător rsync-ului); backup-urile incrementale salvează doar modificările ce s-au făcut de la ultimul backup pana în prezent, indiferent de tipul backup-ului; în acest fel se poate reveni la orice stare anterioara, în măsura în care s-a realizat cel puțin un backup incremental la acea stare.

Informații detaliate despre backupuri periodice folosind **`rsync`** vor fi prezentate în Secțiunea 10.6.1.

2.4 Redirectarea intrării sau ieșirii

Așa cum veți întâlni în Secțiunea 4.4.1, o aplicație care rulează folosește niște intrări speciale (standard); în folosirea acestor intrări și a altor fișiere folosește descriptori de fișiere.

Există 3 **fișiere speciale** utilizate de programele în execuție pentru a interacționa cu utilizatorii:

- intrare standard (*standard input*, *stdin*): acesta reprezintă locul de unde se citesc datele de intrare de către program (de obicei tastatura)
- ieșire standard (*standard output*, *stdout*): acesta este fișierul în care se scriu datele de ieșire (de obicei este consola curentă)
- ieșire de eroare standard (*standard error*, *stderr*): în acest fișier se scriu mesajele de eroare de către program (de obicei tot în consola curentă)

Un descriptor de fișier reprezintă un indice asociat unui fișier deschis de o aplicație. Cele 3 fișiere speciale de mai sus au următorii descriptori de fișier:

- `stdin` are descriptorul cu indexul 0
- `stdout` are descriptorul cu indexul 1
- `stderr` are descriptorul cu indexul 2

Restul fișierelor deschise de aplicații au un descriptor de fișier mai mare sau egal cu 3. În C/C++ există 3 variabile de tipul `FILE *` cu numele `stdin`, `stdout`, `stderr`. Ele au același rol ca cele descrise mai sus și sunt folosite ca orice alte variabilă de tip `FILE *`.

În unele situații, utilizatorul poate dori să modifice intrarea sau ieșirea pentru o aplicație. De exemplu, utilizatorul își poate dori ca în loc să obțină datele de intrare de la tastatură pentru un program, să le obțină dintr-un fișier. Aceste operații se pot realiza doar la nivelul descriptorilor. În shell sunt permise comenzi cu o sintaxă specială asupra descriptorilor standard modificați; pot fi întâlnite cazurile de redirectare din Tabelul 2.6.

Tabelul 2.6: Metode de redirectare

Sursă	**Destinație**	**Exemplu comandă**
intrare (*stdin*)	Fișier	**`./program < fișier_intrare`**
ieșire (*stdout*)	Fișier	**`./program > fișier_ieșire`**
eroare (*stderr*)	Fișier	**`./program 2> fișier_erori`**
eroare (*stderr*)	iesire (stdout)	**`./program 2>&1`**
eroare & ieșire	Fișier	**`./program > fișier_ieșire_și_erori 2>&1`**

Pentru a redirecta ieșirea (către exteriorul programului) se folosește caracterul `>` (semnul *mai mare*), în timp ce pentru a redirecta intrarea se folosește caracterul `<` (semnul *mai mic*) (către program), iar pentru redirectarea ieșirii de erori se folosește descriptorul `2` de fișier urmat de caracterul `>`.

Sintaxa `&1` se folosește atunci când se dorește redirectarea către ieșirea standard (*stdout*), astfel încât construcția `2>&1` redirectează ieșirea de eroare standard către ieșirea standard. Astfel, dacă dorim redirectarea ieșirii de erori și a ieșirii standard către un fișier trebuie să redirectăm mai întâi ieșirea de erori către stdout (`2>&1`) și apoi să redirectăm ieșirea standard într-un fișier (`fișier_erori_și_ieșiri`).

Listing 2.24 conține redirectarea intrării standard comenzii **`mail`** din conținutul fișierului `message.txt`. Utilizatorului `gabriel` i se va trimite un email cu informațiile din fișier.

```
1 student@uso:~$ mail gabriel < message.txt
```

Listing 2.24: Redirectarea intrării standard

Similar, Listing 2.25 conține comanda care va scrie în fișierul `listare` toate fișierele și directoarele ce se află în directorul curent.

```
1  student@uso:~$ ls > listare
2  student@uso:~$ cat listare
3  01-fs.tar
4  01-fs.tar.bz2
5  01-fs.tar.gz
6  Desktop
7  Documents
8  Downloads
9  Music
10 Pictures
11 Public
12 Templates
13 Videos
14 examples.desktop
15 games
16 listare
17 uso.git
18 vm-actions-log.txt
```

Listing 2.25: Reidrectarea ieșirii standard

Pentru a adăuga la sfârșitul fișierului rezultatul unei comenzi folosim operatorul $>>$ ca în Listing 2.26.

```
1 student@uso:~$ date >> listare
2 student@uso:~$ cat listare
3 [...]
4 vm-actions-log.txt
5 duminica 30 septembrie 2018, 20:17:50 +0300
```

Listing 2.26: Redirectarea cu adăugare (append)

În Listing 2.27 este redirectată ieșirea de eroare standard. Comanda de la linia 1 încearcă să copieze un fișier fără a spune unde dorește să-l copieze, comandă ce va genera o eroare care va fi redirectată în fișierul `errors.out`.

```
1 student@uso:~$ cp listare 2> errors.out
2 student@uso:~$ cat errors.out
3 cp: missing destination file operand after 'listare'
4 Try 'cp --help' for more information.
```

Listing 2.27: Redirectarea ieșirii de eroare standard

Putem redirecta și ieșirea standard și ieșirea de eraore standard în același fișier. În Listing 2.28, atât ieșirea standard cât și ieșirea de eroare standard ale comenzii **`strace ls`** de pe linia 1 vor fi redirectate în fișierul `strace-all.out`.

```
1 student@uso:~$ strace ls > strace-all.out 2>&1
2 student@uso:~$ cat strace-all.out
3 execve("/bin/ls", ["ls"], 0x7ffd95189350 /* 36 vars */) = 0
4 brk(NULL)                               = 0x5578b9d96000
```

```
5 access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or
  directory)
6 access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or
  directory)
7 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
8 fstat(3, {st_mode=S_IFREG|0644, st_size=91329, ...}) = 0
9 [...]
```

Listing 2.28: Redirectarea ieșirii standard și ieșirii de eroare standard în același fișier

Tabelul 2.7 prezintă câteva redirectări folosind fișiere speciale.

Tabelul 2.7: Redirectări folosind fișiere speciale

Comanda	Efect
./program 2> /dev/null	mesajele de la ieșirea de eroare standard nu sunt afișate
./program > /dev/null 2>&1	nici un mesaj nu este afișat
> new_file	creează un fișier gol cu numele `new_file`
cat /dev/null > new_file	creează un fișier cu același conținut ca `/dev/null`, adică un fișier gol, indentic ca mai sus

Detalii despre internele redirectării și ce se întâmplă la nivelul unei aplicații găsiți în Secțiunea 4.4.2.

2.5 Tipuri de sisteme de fișiere

În timp au apărut mai multe tipuri de sisteme de fișiere, majoritatea dezvoltându-se în mediul open-source. Utilizatorii au opțiunea de a alege sistemul de fișiere preferat cu care doresc să lucreze, mai ales în cazul în care folosesc partiții multiple pentru mai multe sisteme de operare.

Pentru mai multe informații despre crearea, montarea și repararea unui sistem de fișiere, precum și lucrul cu partiții, citiți Capitolul 10. În Tabelul 2.8 de mai jos se găsesc unele dintre cele mai importante sisteme de fișiere utilizate în prezent, alături de sistemele de operare în care operează.

Tabelul 2.8: Sisteme de operare și sisteme de fișiere

SO **Sistem fișiere**	**Windows**	**Linux**	**Mac OS**
FAT32	Nativ	Nativ	Nativ
NTFS	Nativ (dupa WinNT)	prin ntfs-3g	prin ntfs-3g
Ext2/Ext3/Ext4	Driver third-party	Nativ	-
HFS+	-	Nativ	Nativ
APFS	-	-	Nativ
ISO9660	Nativ	Nativ	Nativ
UDF	Nativ	Nativ	Nativ

Termenul „nativ” semnifică faptul că suportul este oferit de driverele ce însoțesc sistemul de operare.

Sistemele de fișiere pot fi clasificate după locul în care datele sunt stocate. Tabelul 2.9 prezintă succint această clasificare.

Tabelul 2.9: Clasificarea sistemelor de fișiere după suportul datelor

Tip	Exemplu	Descriere
sisteme de fișiere cu suport fizic	FAT32, NTFS, Ext4, APFS	se regăsesc de obicei pe un mediu de stocare
sisteme de fișiere virtuale	procfs, devfs, SSHFS	conțin fișiere/date generate de SO (informații despre sistem) sau de o altă componentă software (alte surse)
sisteme de fișiere pentru rețea	NFS, SMB	utilizate pentru accesul la fișiere aflate în rețea

Sistemele de fișiere virtuale (precum `procfs`) sunt sisteme care nu au suport fizic pe disc. Accesul prin comenzi la aceste sisteme de fișiere duce de obicei la date care se găsesc în memorie. Despre sistemul de fișiere `procfs` vor fi prezentate detalii în Secțiunea 4.8.

2.5.1 Integritatea datelor

Cărțile din bibliotecă se îngălbenesc, caietele se șifonează și scrisul își pierde conturul. Fișierele digitale se pot și ele degrada, datorită unor erori. Sistemele de fișiere au un rol vital în asigurarea integrității datelor (*data integrity*), adică în prevenirea, detectarea și remedierea coruperii datelor. Coruperea datelor (*data corruption*) se referă la degradarea neintenționată a datelor datorită erorilor umane, erorilor de transmisie, defecțiunilor mediului fizic de stocare sau a diferitelor deficiențe în procesare. Securizarea datelor (*data security*) se referă la prevenirea atacurilor intenționate asupra datelor și de asemenea necesită sprijinul sistemului de fișiere.

Un exemplu de metodă prin care sistemele de fișiere pot asigura integritatea datelor se referă la calcularea unei sume de control (*checksum*). Pentru un anumit set de date, o sumă de control este un număr care descrie pe scurt acel set, diferențiindu-l suficient de mult de un set foarte similar dar un pic diferit. De exemplu, o sumă de control poate consta efectiv în însumarea valorilor numerice asociate cu caracterele respective într-un cod, precum valorile lor numerice din codul ASCII. Dacă se modifică o literă din greșeală, suma va diferi și eroarea va fi detectată.

Sistemul de fișiere trebuie să fie capabil să păstreze integritatea datelor și în situația unui eșec (*failure*). În cele mai multe cazuri această funcționalitate se implementează prin **jurnalizare**, care permite și aducerea sistemului la o stare anterioară eșecului. Prin această activitate se creează un jurnal unde sunt păstrate toate modificările efectuate asupra unui sistem de fișiere, fiecare dintre aceste modificări fiind mai întâi scrisă în jurnal și apoi realizată (modificările putând apărea în mai multe locuri din sistemul de fișiere).

Operațiile din jurnal sunt executate la diferite intervale de timp de către driver-ul sistemului pentru actualizarea stării discului. După ce toate modificările asociate unei operații sunt executate, aceasta se șterge din jurnal, astfel că, la apariția unui eșec (cum ar fi întreruperile de curent, defecțiunile fizice ale dispozitivului etc.), vor putea fi executate operațiile care au fost începute, dar s-au întrerupt, deoarece ele sunt încă prezente în jurnal, urmând să fie reexecutate în întregime.

2.5.2 Alegerea unui sistem de fișiere

Atunci când alegem un sistem de fișiere, criteriile cele mai căutate sunt:

- disponibilitatea: dacă sistemul respectiv poate fi folosit în mai multe sisteme de operare, sau doar în unul singur;
- gradul de siguranță: dacă asigură jurnalizare sau nu – majoritatea sistemelor din prezent folosesc jurnalizarea, alte măsuri luate pentru a permite integritatea datelor;
- restricții speciale date de modul în care datele din sistemul de fișiere sunt organizate;de exemplu: FAT32 folosește doar 32 biți pentru stocarea dimensiunii unui fișier, deci dimensiunea maximă a unui fișier este 4GB (mai puțin decât o imagine de DVD, fapt ce îl face nepractic pentru o parte din aplicațiile multimedia)
- optimizări de performanță, ce sunt realizate în funcție ori de tipul suportului de stocare, ori de dimensiunea fișierelor etc.

Tabelul 2.10 realizează o analiză sumară a caracteristicilor sistemelor de fișiere. Unitatea TiB reprezintă un tebibyte, adică 2^{40} octeți, iar un EiB reprezintă un exbibyte, adică 2^{60} octeți.

2.6 Anexă: Comenzi pentru lucrul cu fișiere în Windows

Tabelul 2.11 prezintă comenzile echivalente în Windows pentru operațiile de lucru cu sistemul de fișiere. Acestea pot fi folosite în prompt-urile Windows precum PowerShell sau Command Prompt.

2.7 Sumar

TODO

Tabelul 2.10: Caracteristici ale sistemelor de fișiere mai cunoscute

Nume	**Sisteme de operare**	**Dimensiune maximă fișier**	**Jurnalizare**	**Observații**
FAT32	Windows / Linux / Mac OS	4 GB	nu	Cel mai folosit sistem de fișiere - întâlnit în mod special pe USB stick-uri, fără drepturi de acces; Windows-ul limitează la creare dimensiunea unei partiții la 32GB, dar poate citi partiții mai mari realizate și formatate cu aplicații third-party folosit pentru a asigura compatibilitatea cu dispozitive sau sisteme mai vechi
NTFS	Windows / Linux / Mac OS	16TiB	da	Singurul sistem de fișiere pentru Windows recomandat de Microsoft. Singurul sistem de fișiere pentru Windows cu securitate.
Ext2/Ext3	Linux / Windows	16GiB - 64TiB	ext2-nu ext3-da	Sistemul de fișiere considerat cel mai stabil datorită unei istorii de dezvoltare foarte lungi; ext3 e compatibil cu versiunea anterioară ext2, aduce jurnalizare față de ext2
Ext4	Linux / Windows / MacOS	16TiB	da	Succesorul lui Ext3, îmbunătățind performanța, stabilitatea și capacitatea de stocare. Este adecvat și pentru stocarea datelor critice, datorită preciziei ridicate ale marcajelor temporale.
HFS+	Mac OS / Linux	16 EiB	da*	* în Linux, HFS+ este suportat fără jurnalizare
APFS	MacOS, iOS	8 EiB	da	Este optimizat pentru dispozitivele de stocare tip flash și solid drive, și pune accentul pe criptare.
ISO9660	Win / Linux / Mac OS	În funcție de implementare	nu	Sistem de fișiere utilizat în principal pe CD-uri, organizare internă concepută pentru ca datele să fie citibile ușor
UDF	Win / Linux / Mac OS	16 EiB	da	Sistem de fișiere utilizat în principal pe mediile optice, cu suport atât pentru scriere cât și pentru citire

Tabelul 2.11: Echivalențe comenzi Linux și Windows

Comanda Linux	Comanda Windows	Descriere
`comanda --help`	`comanda /?`	afișează informații despre comandă
`cd`	`cd`	schimbă directorul curent
`pwd`	`chdir`	afișează directorul curent
`clear`	`cls`	șterge ecranul consolei curente
`cp`	`copy`	copiază un fișier
`rm`	`del`	șterge un fișier
`ls`	`dir`	afișează conținutul directorului curent
`vim`	`edit`	editează un fișier text
`exit`	`exit`	închide shellul curent
`diff`	`fc`	compară două fișiere și afișează diferențele între ele
`find`	`find`	caută fișiere
`mkfs (mke2fs)`	`format`	formatează un disc
`free`	`mem`	afișează informații despre memoria liberă și cea ocupată
`mkdir`	`mkdir`	creează un nou director
`mv`	`move`	mută un fișier
`mv`	`ren`	redenumește un fișier
`date`	`time`	afișează ora sistemului
`diff`	`fc`	afișează diferențele dintre două fișiere

Capitolul 3

Pachete software

Spre deosebire de hardware, software-ul are marele avantaj al flexibilității: pe un sistem hardware putem rula diferite tipuri de software, putem adăuga și șterge software, putem modifica ușor. Atunci când avem o nevoie, putem fie să dezvoltăm software nou sau putem instala software existent în sistem. Vom prezenta partea de dezvoltare software, de aplicații, în Capitolul 6, și ne vom concentra în acest capitol pe partea de instalare și configurare de aplicații existente.

Atunci când un dezvoltator sau o firmă software produce o aplicație nouă, este interesul acestora să ajungă cât mai eficient la utilizator. Această acțiune, de distribuire a software-ului, necesită ca aplicația și datele sale să fie agregate într-o formă care să fie instalabilă de utilizator. Această formă este numită pachet software. După instalarea unui pachet software utilizatorul va putea să ruleze aplicația pe sistemul său.

Pentru ca o aplicație să ruleze, aceasta are nevoie de un fișier executabil și fișiere de date și fișiere de configurare. De exemplu, un browser web dispune de un fișier executabil, are fișier de date în care reține parole de acces la site-uri și bookmark-uri și are o zonă de configurare precum pagina de start, prezența unor intrări în meniu, directorul implicit în care se descarcă fișierele. Aceste fișiere sunt împachetate de cei care se ocupă de dezvoltare. Fișierul rezultat în urma împachetării, pachetul software, este apoi transmis utilizatorului pentru instalare.

Instalarea unui pachet înseamnă că fișierul executabil, fișierele de date și fișierele de configurare sunt dispuse în sistemul local de fișiere de unde vor fi folosite. De exemplu, pe Windows, toate fișierele sunt de obicei dispuse într-un subdirector din directorul `C:\Program Files\`. Pe macOS toate fișierele sunt dispuse într-un subdirectorul din directorul `/Applications/`. Pe Linux fișierele sunt distribuite în funcție de tipul lor: în `/usr/bin/` fișierele executabile, în `/usr/share/` și `/var/` fișiere de date, în `/etc/` fișiere de configurare.

Cea mai simplă formă de împachetare a unei aplicații este o arhivă. O arhivă este un fișier reprezentând o colecție de fișiere puse unul lângă altul. Arhiva nu are informații dedicate despre aplicație, astfel încât instalarea aplicației presupune dezarhivarea. Avantajul folosirii arhivelor este simplitatea. Dezavantajul este gestiunea dificilă a pachetelor: dezinstalarea devine greoaie, pentru că trebuie șterse manual toate fișierele; unele pachete pot fi incompatibile unele cu altele și trebuie instalate alte versiuni.

Forma modernă de livrare a aplicațiilor o reprezintă pachetele software. Un **pachet**, spre deosebire de o arhivă simplă, conține informații despre aplicație, precum descrierea pachetului, versiune, dependențe de alte pachete, pachetele similare; numim aceste informații **metadatele pachetului**. Un pachet respectă un format; acest format este cunoscut de o aplicație specifică de gestiune a pachetelor, numită **manager de pachete**.

În anumite situații dorim anumite versiuni de pachete. Aceste versiuni riscă să afecteze celelalte pachete instalate, sau să necesite configurări mai complexe. Sau să aibă nevoie de alte pachete și diferite versiuni care duce la o problemă numită „dependency hell" Din acest motiv o formă de livrare a pachetelor este „toate-în-unul" (*all-in-one*) cu toate dependențele în aceeași arhivă. Vom vorbi despre acestea în Secțiunea 3.4.

În cele ce urmează vom prezenta partea de pachete software în forma lor tradițională și apoi vom discuta despre pachete *all-in-one*.

3.1 Pachete software

Un **pachet software** este o colecție de fișiere: executabile, de date, de configurare, de documentație. Un pachet este distribuit și apoi instalat pe un sistem pentru a adăuga o nouă funcționalitate acelui sistem. În urma instalării pachetului, fișierele din colecție ajung în sistemul de fișiere. Fișierele executabile sunt cele folosite pentru rula aplicațiile în formă de procese, așa cum vom preciza în Secțiunea 4.1. Fișierele de date sunt baze de date, imagini, sunete, filme folosite de aplicație pentru a rula. Fișierele de configurare permit modificarea parametrilor de funcționare a aplicației. Fișierele de documentație vor fi citite de utilizator pentru a ști cum să folosească și cum este proiectată aplicația.

Tipurile de fișiere care vor fi instalate reprezintă datele unui pachet. Pe lângă date, un pachet conține și metadate, adică informații despre acele date. **Metadatele unui pachet** sunt folosite pentru gestiunea versiunilor și dependențelor acelui pachet. De exemplu un pachet Debian (`.deb`) conține două fișiere comprimate: `data.tar.xz/` și `control.tar.xz` ca în Listing 3.1.

```
1 student@uso:~$ ar tf /var/cache/apt/archives/grep_3.1-2build1_amd64.deb
2 debian-binary
3 control.tar.xz
4 data.tar.xz
```

Listing 3.1: Conținutul unui pachet Debian (.deb)

Pentru a vedea metadatele pachetului, putem desface pachetul și vedea conținutul. Mai simplu, putem folosi o comandă dedicată ca în Listing 3.2.

```
1  student@uso:~$ dpkg -I /var/cache/apt/archives/grep_3.1-2build1_amd64.deb
2   new Debian package, version 2.0.
3   size 158964 bytes: control archive=1416 bytes.
4      1219 bytes,    29 lines      control
5       746 bytes,    13 lines      md5sums
6   Package: grep
7   Version: 3.1-2build1
8   Architecture: amd64
9   [...]
10  Pre-Depends: libc6 (>= 2.14), libpcre3
11  Depends: dpkg (>= 1.15.4) | install-info
```

```
 Suggests: libpcre3 (>= 7.7)
 [..]

student@uso:~$ apt show grep
Package: grep
Version: 3.1-2build1
[...]
Pre-Depends: libc6 (>= 2.14), libpcre3
Depends: dpkg (>= 1.15.4) | install-info
Suggests: libpcre3 (>= 7.7)
[...]
```

Listing 3.2: Metadatele unui pachet Debian (.deb)

Conținutul unui pachet, cuprinzând datele și metadatele acestuia, este descris în Figura 3.1.

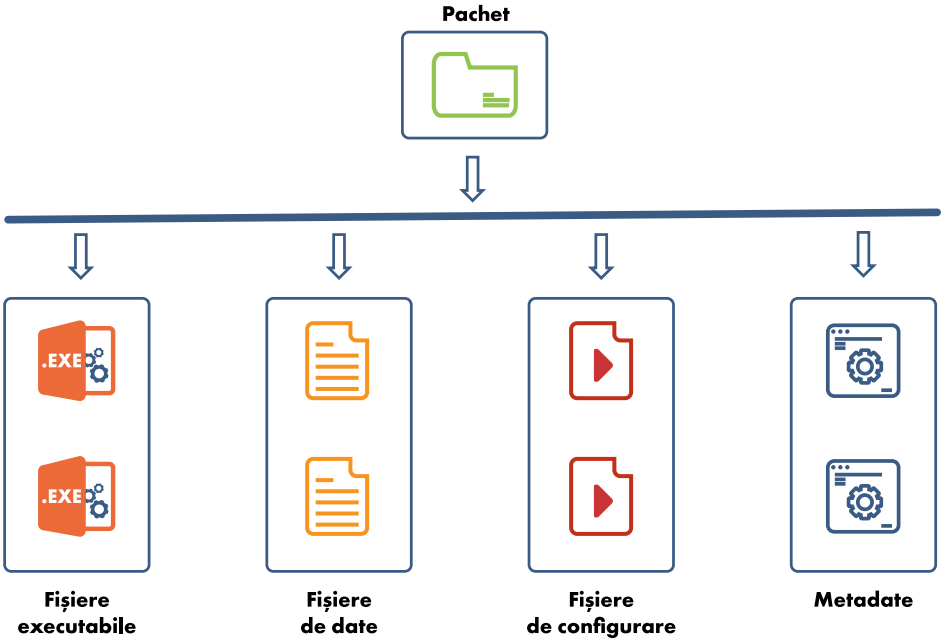


Figura 3.1: Conținutul unui pachet

Metadatele au rol în informarea utilizatorului și să asigure funcționarea corespuzătoare a aplicațiilor pe sistemelor. Pentru o funcționare corespunzătoare pachetele trebuie să fie compatibile cu sistemul și lucreze bine împreună: un pachet are nevoie de o anumită versiune de sistem de operare și de pachete să fie instalate pentru a funcționa. De exemplu, pachetul `grep`, analizat în Listing 3.2, analizat mai sus, are nevoie de pachetele `libc6` (versiune mai mare sau egală cu `2.14`) și `libpcre3`.

În plus, un pachet software evoluează și numărul său de versiune crește. Acea versiune necesită o anumită versiune a altui pachet sau altă versiune de sistem de operare. Prin analizarea versiunii, vom ști dacă acel pachet este instalabil pe un sistem dat și ce pachete (și versiuni) trebuie să fie instalate.

Spunem că un pachet are dependențe de alte pachete. Fără ca acele pachete să fie instalate, aplicația din pachet nu va funcționa. Este responsabilitatea unui **manager de pachete** să asigure instalarea și a pachetelor de care pachetul curent depinde; numim aceste pachete **dependențe**. Vom prezenta soluții de tip manager de pachete în secțiunile următoare.

Modul în care sunt organizate datele și metadatele într-un pachet software determină formatul acestuia și tipul de pachet. Fiecare sistem de operare și fiecare distribuție are un format de pachete și aplicații specifice pentru gestiunea acelui tip de pachete. Aplicațiile specifice pentru gestiunea pachetelor se ocupă de instalarea, dezinstalarea și investigarea pachetelor, acțiuni ce vor fi descrise în Secțiunea 3.2.1.

3.1.1 Tipuri de pachete

Așa cum e prezentat în Figura 3.1, în general un pachet este o arhivă de date și metadate, o colecție. Indiferent de tipul de pachet, în mod uzual se poate investiga folosind o soluție de dezarhivat.

Un prim tip de pachete software sunt arhive simple care conțin datele aplicației. Așa cum am precizat mai sus, această formă este simplă dar nu permite dezinstalarea facilă și rezolvarea dependențelor. Din acest motiv nu este foarte întâlnită, preferându-se tipuri de pachete care conțin metadate care sunt gestionate de aplicațiile specifice.

Atunci când nu se dorește folosirea unei aplicații specifice de gestiunea pachetelor, se pot livra pachetele în format de arhivă auto-extractivă (*self extracting archive*) sau pachet executabil auto-extractiv (*self-extracting executable*). Acest pachet este rulat și apoi are loc instalarea.

Aplicațiile de pe sistemele Windows nu sunt livrate în general, cu un format de instalare. De multe ori aplicațiile au un fișier numit `setup.exe` care este folosit pentru instalarea și configurarea aplicației. Alte aplicații pot fi instalate dintr-un pachet software de tip MSI (*Microsoft Install*), folosite de aplicația de instalare numită Windows Installer.

Pe macOS aplicațiile sunt livrate în pachete de tip imagine de disk, fișiere cu extensia `.dmg` sau fișiere de tip `.pkg`. Fișierele `.pkg` au avantajul că pot să conțină scripturi instalare și de configurare; fișiere `.dmg` sunt imagini de disc care conțin aplicația care se instalează de obicei prin copierea ei, fără o operație de instalare.

În Linux există cea mai mare variație de tipuri de pachete și de soluții pentru gestiunea acestora. În vreme ce la macOS și la Windows există o singură organizație responsabilă de gestiunea sistemului de operare (Apple, respectiv Microsoft), lumea Linux este diversă și are multe surse de aplicații, multe distribuții care livrează acele aplicații în pachete și care întrețin soluțiile de gestiunea lor. Cele mai răspândite formate de pachete sunt formatul `.deb` (în distribuțiile bazate pe Debian) și formatul `rpm` (în distribuțiile bazate pe RedHat). Le vom prezenta detaliat, respectiv în Secțiunea 3.3.1 și Secțiunea 3.3.2.

Sistemele de operare mobile (precum Android și iOS) folosesc un depozit (*repository*) central de pachete: Google Play pentru Android și AppStore pentru iOS. Aplicația dedicată de pe telefonul mobil descarcă și instalează pachetul din depozit. Formatul folosit pe iOS este formatul `.ipa` (*iPhone Archive*) iar pe Android este formatul `.apk` (*Android Package*).

Indiferent de sistem de operare, distribuție și aplicație, formatele de pachete au caracteristici comune: conțin date și metadate, sunt un fișier de tip arhivă, există o aplicație dedicată pentru gestiunea acestora: instalare, dezinstalare, configurare, investigare. În continuare vom prezenta cum gestionăm aceste pachete în sistemele de operare moderne.

3.2 Gestiunea pachetelor

Pentru a putea fi instalat pe sistemele utilizatorilor finali, un pachet trebuie să fie distribuit către aceștia. Utilizatorii pot accesa manual un site sau un depozit de pachete (*package repository*) de unde descarcă pachetul și unde apoi îl instalează. Sau pot folosi o aplicație specifică pentru localizarea, descărcarea și instalarea pachetelor. Aceste aplicații poartă numele de manager de pachete (*package manager*).

Un **depozit de pachete** (*package repository*) este locul sunt stocate pachetele software. În general un sistem de operare sau o distribuție de sistem de operare are un repository sau mai multe cu pachetele corespunzătoare. De exemplu, pentru Ubuntu există repository-uri pentru fiecare versiune: un repository pentru Ubuntu 18.04 Bionic Beaver și un repository pentru Ubuntu 18.10 Cosmic Cuttlefish. Pe sistemele de operare mobile putem considera Google Play sau Apple AppStore repository-urile de pachete. Depozitele conțin pachetele în formatul specific distribuției sau sistemului de operare; de exemplu pachete `.deb` pentru distribuțiile Debian/Ubuntu, pachete `.rpm` pentru distribuțiile RedHat, sau pachete `.ipa` în Apple AppStore.

Un repository este identificat printr-un URL (*Uniform Resource Locator*). De exemplu URL-ul pentru un repository Ubuntu este http://archive.ubuntu.com/ubuntu/. Acest URL este folosit de managerul de pachete pentru a afla lista de pachete și pentru a descărca pachetele.

Managerul de pachete este instalat într-un sistem de operare/distribuție de la bun început. Pentru a fi folosit se configurează URL-ul repository-ului / repository-urilor folosite. Apoi utilizatorul folosește managerul pentru a instala sau dezinstala pachete. Managerul ține cont de URL-ul repository-ului și de dependențele pachetelor pentru instalare și dezinstalarea comandată de utilizator la fel ca în Figura 3.2.

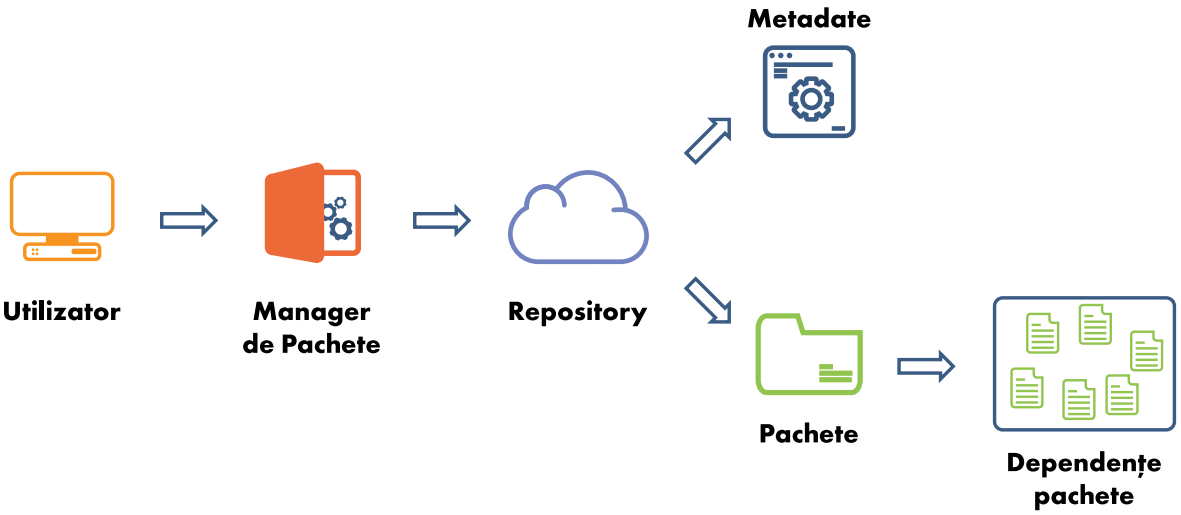


Figura 3.2: Folosirea managerului de pachete

În momentul instalării pachetelor managerul de pachete reține fișierele care au fost instalate pentru a putea să le șteargă la dezinstalare. Tot în momentul instalării, managerul de pachete va investiga dependențele pachetului care se dorește instalat pentru a le instala și pe acestea. Tipurile de operații efectuate de managerul de pachete, la comanda utilizatorului, sunt descrise în Secțiunea 3.2.1.

Noțiunea de manager de pachete este răspândită îndeosebi în distribuții Linux/BSD. Cu toate acestea există soluții de acest tip pe Windows, precum Chocolatey [1]. Pe macOS se poate folosi Homebrew [2] sau MacPorts [3]. Spre deosebire de distribuțiile Linux/BSD, soluțiile de tip manager de pachete de pe Windows și macOS nu sunt distribuite oficiale de companiile producătoare, Microsoft și Apple.

Tabelul 3.1 conține cele mai cunoscute tipuri de managere de pachete cu tipurile de pachete și distribuțiile pe care le folosesc, împreună cu aplicațiile și comenzile specifice.

Tabelul 3.1: Managere și tipuri de pachete

Manager de pachete	Tip de pachet	Sistem de operare / distribuție	Comenzi specifice
APT	`.deb`	distribuții bazate pe Debian	**`apt`**, **`aptitude`**, **`dpkg`**
YUM & DNF	`.rpm`	distribuții bazate pe RedHat	**`dnf`**, **`rpm`**
ZYpp	`.rpm`	distribuții bazate pe openSUSE	**`zypper`**, **`rpm`**
Pacman	`.tar`	Arch	**`pacman`**, **`pactree`**, **`paccache`**
Portage	`.tar.gz`	Gentoo	**`emerge`**, **`equery`**, **`eix`**
Port	`.txz`	FreeBSD	**`pkg`**
Homebrew	`.tar.gz`	macOS	**`brew`**
MacPorts	`.tbz2`	macOS	**`port`**
Chocolatey	`.nupkg`	Windows	**`choco`**

3.2.1 Operații cu pachete

Un utilizator realizează operații cu pachete prin intermediul comenzilor oferite de un manager de pachete. Operațiile tipice sunt de instalare, dezinstalare, actualizarea versiunii (*update*), investigare de pachete. Un utilizator nu trebuie să cunoască anatomia funcționării unui manager de pachete sau formatul pachetelor. Utilizatorul trebuie să știe cum configurează repository-urile unui manager de pachete și apoi să instaleze sau să dezinstaleze pachetele.

Operațiile uzuale realizate de un utilizator sunt descrise în continuare. Vom descrie în detaliul modul în care se realizează aceste operații pe distribuțiile bazate pe Debian și cele bazate pe RedHat în Secțiunea 3.3.1, respectiv Secțiunea 3.3.2. Pentru alte distribuții/sisteme de operare și, deci, alte managere de pachete, agregăm comenzile uzuale în tabele în Secțiunea 3.3.3.

Cele mai directe operații realizate cu pachete sunt instalarea și dezinstalarea pachetelor. Instalarea presupune că un pachet este despachetat și conținutul său este distribuit în sistemul de fișiere, eventual împreună cu rularea unui scripturi de configurare. Dezinstalarea înseamnă ștergerea din sistemul de fișiere a conținutului unui pachet împreună cu rularea unor scripturi de deconfigurare. Prin intermediul unui manager de pachete, instalarea poate însemna descărcarea pachetului dorit dintr-un repository de pachete împreună cu pachete de care acesta depinde. În cazul interacțiunii cu repository-ul de pachete putem opta pentru descărcarea pachetului fără instalare.

Pentru a menține sistemul la zi, atât cu cele mai noi caracteristici ale pachetelor cât și cu rezolvarea anumitor probleme (*bug*-uri) de programare sau de securitate, se

[1] https://chocolatey.org/
[2] https://brew.sh/
[3] https://www.macports.org/

recomandă acțiunea de actualizare a pachetelor (*update*). Operația de actualizare înseamnă descărcarea din depozit a celor mai recente versiuni și înlocuirea versiunilor mai vechi în sistemul de fișiere.

De multe ori nu știm ce pachet să instalăm, dat fiind numărul mare al acestora și faptul că mai multe pachete pot face același lucru: mai multe editoare, mai multe navigatoare web, mai multe playere de muzică. De aceea o operație frecventă este căutarea unui pachet după cuvinte cheie relevante. Căutarea se face de regulă în metadatele pachetelor din depozitul de pachete.

Alte acțiuni, mai puțin frecvente, sunt:

- descărcarea surselor acelui pachet pentru a le putea investiga ulterior
- listarea pachetele instalate
- căutarea unui pachet al cărui nume se potrivește unei expresii
- vizualizarea conținutului unui pachet instalat, adică ce fișiere din sistemul de fișiere corespund acelui pachet
- identificarea pachetului care conține un anumit fișier din sistemul de fișiere

Aceste acțiuni au comenzi și opțiuni specifice fiecărui manager de pachete.

3.2.2 Pachete în sistemele de operare mobile

Operațiile de mai sus au corespondent și în cazul sistemelor de operare mobile. Prin intermediul aplicațiilor GooglePlay [1] și AppStore [2] utilizatorii interacționează cu depozitele de pachete oferite de Google și de Apple, numite magazine de aplicații (*app store*). Într-un magazin de aplicații, utilizatorii caută pachete sau instalează pachete.

În urma unei comenzi de instalare, transparent utilizatorului, aplicația GooglePlay sau AppStore descarcă din depozit pachetul în format respectiv (`.apk` sau `.ipa`). După instalare, aplicația arele fișiere executabile, de date, de configurare, de documentație în sistemul de fișiere și este, în general, accesibilă prin intermediul unei icoane pe ecranul telefonului mobil.

Depozitele de pachete folosite de aplicațiile GooglePlay și AppStore nu pot fi configurate, sunt preconfigurate în aplicație. Cu toate acestea pot fi folosite aplicații cu depozite neoficiale de aplicații, precum Cydia pe iOS [3]. De avut în vedere, însă, că folosirea unor depozite neaprobate de producătorul dispozitivului mobil reprezintă un risc de securitate; instalarea acestor aplicații poate însemna instalarea de aplicații malițioase (*malware*).

3.3 Gestiunea pachetelor în Linux

În Linux, pachetele sunt colectate în depozite (*repository*-uri) pentru fiecare distribuție. Fiecare distribuție are apoi sub-depozite pentru fiecare versiune a distribuției,

[1] https://play.google.com/store
[2] https://www.apple.com/ios/app-store/
[3] https://cydia-app.com/

sub-depozit ce conține versiunile de pachete corespunzătoare. Sistemul de gestiunea a pachetelor, cuprinzând managerul de pachete și alte componente, este cel responsabil pentru interacțiunea cu depozitul de pachete, la comanda utilizatorului.

Repository-ul de pachete este identificat printr-un URL care poate fi local (de exemplu CD-ROM) sau poate fi la distanță (acesta este modul uzual). Managerul de pachete are o zonă de configurare în care putem specifica URL-urile repository-urilor folosite.

Pentru interacțiunea cu sistemul de gestiune a pachetelor, există aplicații dedicate. Acestea pot fi grafice sau în linia de comandă. De exemplu, pe sistemele bazate pe Debiam, există aplicația grafică Synaptic și utilitarul în linia de comandă **`apt`**. La fel, pe Fedora și alte distribuții, există aplicația grafică PackageKit și utilitarul în linia de comandă **`dnf`**. Aceste aplicații interacționează cu sistemul de gestiune a pachetelor, cu repository-uri și instalează pachete. Pentru operarea directă asupra pachetelor aceste aplicații folosesc o comandă dedicată care cunoaște formatul pachetelor: **`dpkg`** pe sistemele derivate din Debian și **`rpm`** pe sistemele derivate din RedHat.

În continuare vom detalia comenzile folosite pe sistemele derivate din Debian (cu pachete DEB) și pe sistemele derivate din RedHat (cu pachete RPM).

3.3.1 Gestiunea pachetelor format DEB

Sistemele derivate din Debian folosesc pachete în format DEB, cu extensia `.deb`. Aceste pachete sunt arhive `.ar` care conțin datele și metadatele.

Sistemul de gestiune a pachetelor este APT (*Advanced Package Tool*). Configurația acestuia se găsește în directorul `/etc/apt/`. În fișierul `/etc/apt/sources.list` sunt configurate URL-uri de depozite de pachete.

```
1  student@uso:~$ cat /etc/apt/sources.list
2  [...]
3  deb http://ro.archive.ubuntu.com/ubuntu/ bionic main restricted
4  deb-src http://ro.archive.ubuntu.com/ubuntu/ bionic main restricted
5  deb http://ro.archive.ubuntu.com/ubuntu/ bionic-updates main restricted
6  deb-src http://ro.archive.ubuntu.com/ubuntu/ bionic-updates main
   restricted
7  deb http://ro.archive.ubuntu.com/ubuntu/ bionic universe
8  deb-src http://ro.archive.ubuntu.com/ubuntu/ bionic universe
9  deb http://ro.archive.ubuntu.com/ubuntu/ bionic-updates universe
10 deb-src http://ro.archive.ubuntu.com/ubuntu/ bionic-updates universe
11 [...]
```

Listing 3.3: Configurație APT (Debian)

Listing 3.3 prezintă o selecție a fișierului de configurație `/etc/apt/sources.list` de pe o distribuție Ubuntu 18.04 Bionic Beaver. Fișierul conține o listă de URL-uri de repository-uri. URL-uri precizează dacă repository-ul este folosit pentru descărcarea pachetelor (liniile care încep cu `deb`) sau pentru descărcarea surselor (liniile care încep cu `deb-src`). După URL urmează versiunea distribuției (`bionic`) și apoi sunt componente de repository unde sunt clasificate pachetele (`main`, `restricted`, `universe`).

Interacțiunea cu APT se face în mod uzual în linia de comandă prin intermediul utilitarului **`apt`**. Listing 3.4 prezintă operații frecvente folosind comanda `apt`. Fiecare comandă

este prefixată de un comentariu care explică rolul acelei comenzi.

```
# Cauta pachete care au in nume sau in descrire sirul "svg".
student@uso:~$ apt search svg

# Instaleaza pachetul inkscape.
# Sau actualizeaza pachetul daca este deja instalat si exista o versiune
mai noua.
student@uso:~$ sudo apt install inkscape

# Afiseaza informatii despre pachetul inkscape (instalat sau nu).
student@uso:~$ apt show inkscape

# Dezinstaleaza pachetul inkscape.
student@uso:~$ sudo apt remove inkscape

# Actualizeaza informatiile locale despre pachete.
student@uso:~$ sudo apt update

# Actualizeaza pachetele sistemului.
student@uso:~$ sudo apt upgrade

# Descarca pachetul inkscape fara sa fie instalat.
student@uso:~$ sudo apt -d remove inkscape

# Descarca sursele pachetului inkscape.
student@uso:~$ apt source inkscape
```

Listing 3.4: Operații frecvente cu apt

Majoritatea acțiunilor realizate folosind utilitarul **apt** sunt privilegiate; de aceea sunt prefixate de comanda **sudo**. Excepție fac, în Listing 3.4, acțiune de căutare a unui pachet și cea de descărcare a surselor pachetului. Acestea sunt exceptate pentru că nu modifică fișiere de sistem.

Pentru operațiile din spate, de lucru cu pachete, utilitarul **apt** apelează uilitarul **dpkg**. Utilitarul **dpkg** are informații despre pachete și despre instalările din sistem. Listing 3.5 prezintă operații frecvente folosind utilitarul **dpkg**. Fiecare comandă este prefixată de un comentariu care explică rolul acelei comenzi.

```
# Afiseaza pachetele instalate local.
student@uso:~$ dpkg -l | grep '^ii'

# Cauta pachetele instalate local dupa expresia book.
student@uso:~$ dpkg -l *book*

# Identifica pachetul care contine fisierul /usr/bin/ssh.
student@uso:~$ dpkg -S /usr/bin/ssh

# Afiseaza continutul pachetului inkscape, instalat local.
student@uso:~$ dpkg -L inkscape
```

Listing 3.5: Operații frecvente cu dpkg

3.3.2 Gestiunea pachetelor format RPM

Similar distribuțiilor derivate din Debian, distribuțiile derivate din RedHat folosesc pachete RPM (RPM Package Manager) cu extensia `.rpm`. Aceste pachete sunt arhive `cpio` care conțin datele și metadatele.

Sistemul de gestiune a pachetelor pe Fedora este DNF (*Dignified yum*), care este o îmbunătățire a vechiului sistem care numea YUM (*Yellowdog Updater Modified*) care la rândul său era o îmbunătățire de la YUP (*Yellowdog Updater*); Yellowdog a fost o distribuție Linux care a creat sistemul YUP. Versiunile mai vechi (și mai stabile) de Red Hat Enterprise Linux (RHEL) sau CentOS (derivat din RHEL) folosesc în continuare YUM; probabil la un moment dat vor tranzita spre DNF. Configurația DNF se găsește în directorul `/etc/dnf/`. În directorul `/etc/yum/yum.repos.d/` sunt configurate URL-uri repository-uri depozite de pachete. Listing 3.6 prezintă o configurație uzuală.

```
1 [student@uso:~]$ cat /etc/yum.repos.d/fedora.repo
2 [fedora]
3 name=Fedora $releasever - $basearch
4 failovermethod=priority
5 #baseurl=http://download.fedoraproject.org/pub/fedora/linux/releases/
  $releasever/Everything/$basearch/os/
6 metalink=https://mirrors.fedoraproject.org/metalink?repo=fedora-
  $releasever&arch=$basearch
7 enabled=1
8 [...]
```

Listing 3.6: Configurație de repository YUM/DNF (RedHat)

Interacțiunea cu DNF se face în mod uzual în linia de comandă prin intermediul utilitarului **`dnf`**. La fel ca în cazul Debian, Listing 3.7 prezintă operații frecvente folosind utilitarul **`dnf`**. Fiecare comandă este prefixată de un comentariu care explică rolul acelei comenzi.

```
1  # Afiseaza pachetele instalate local.
2  [student@uso:~]$ dnf list installed
3
4  # Afiseaza lista de repository-uri folosite.
5  [student@uso:~]$ dnf repolist all
6
7  # Cauta pachete care au in nume sau in descrire sirul "svg".
8  [student@uso:~]$ dnf search svg
9
10 # Instaleaza pachetul inkscape.
11 [student@uso:~]$ sudo dnf install inkscape
12
13 # Afiseaza informatii despre pachetul inkscape (instalat sau nu).
14 [student@uso:~]$ dnf info inkscape
15
16 # Dezinstaleaza pachetul inkscape.
17 [student@uso:~]$ sudo dnf remove inkscape
18
19 # Actualizeaza pachetul inkscape.
20 [student@uso:~]$ sudo dnf update inkscape
21
22 # Actualizeaza pachetele sistemului.
23 [student@uso:~]$ sudo dnf upgrade
24
```

```
25 # Descarca pachetul inkscape fara sa fie instalat.
26 [student@uso:~]$ sudo dnf download inkscape
27
28 # Descarca sursele pachetului inkscape.
29 [student@uso:~]$ dnf download --source inkscape
```

Listing 3.7: Operații frecvente cu dnf

Pentru operațiile din spate, de lucru cu pachete, utilitarul **`dnf`** folosește utilitarul **`rpm`**. Utilitarul **`rpm`** are informații despre pachet și despre instalările din sistem. Listing 3.8 prezintă operații frecvente folosind utilitarul **`rpm`**. Fiecare comandă este prefixată de un comentariu care explică rolul acelei comenzi.

```
1  # Afiseaza pachetele instalate local.
2  [student@uso:~]$ rpm -qa
3
4  # Afiseaza informatii despre pachetul inkscape.
5  [student@uso:~]$ rpm -qi inkscape
6
7  # Identifica pachetul care contine fisierul /usr/bin/ssh.
8  [student@uso:~]$ rpm -qf /usr/bin/ssh
9
10 # Afiseaza continutul pachetului inkscape, instalat local.
11 [student@uso:~]$ rpm -ql inkscape
```

Listing 3.8: Operații frecvente cu rpm

3.3.3 Gestiunea pachetelor în alte formate

În continuare prezentăm în format tabelar comenzi comparative între diferite sisteme de gestiune a pachetelor descrise în Tabelul 3.1. Pentru completitudine vom include și APT și DNF. O coloană va descrie tipul de sistem de gestiune a pachetelor iar alte coloane vor prezenta comenzile corespunzătoare fiecărui tip de operație. Din limitări de spațiu sunt mai multe tabele cu aceeași primă coloană. În tabele vom folosi numele de pachet generic `hello`; când apare cuvântul `hello` într-o comandă presupunem că e vorba de numele unui pachet.

Sunt prezente următoarele tabele:

- Tabelul 3.2 prezintă fișierul de configurare pentru sistemul de gestiune a pachetelor și comenzile pentru configurarea acestuia.
- Tabelul 3.3 prezintă comenzile pentru actualizarea informațiilor despre pachete și actualizarea sistemului (*update*).
- Tabelul 3.4 prezintă comenzile pentru instalarea și dezinstalarea pachetelor.
- Tabelul 3.5 prezintă comenzile pentru descărcarea pachetelor (fără instalare) și descărcarea codului sursă al pachetelor.
- Tabelul 3.6 prezintă comenzile pentru afișarea de informații și conținut al pachetelor.

Tabelul 3.2: Configurare repository

Sistem de pachete	Fișier / director de configurare	Utilitar(e) de configurare
APT	`/etc/apt/sources.list`	**apt**, **dpkg**
DNF / YUM	`/etc/dnf/dnf.conf`, `/etc/yum.repos.d/`	**dnf**, **rpm**
Pacman	`/usr/local/etc/pacman.conf`	**pacman**
Portage	`/etc/portage`	**emerge**
ZYpp	`/etc/zypp/zypp.conf`	**zypper**, **rpm**
Port	`/etc/portsnap.conf`	**portsnap**, **pkg**
Homebrew	N/A	**brew**
MacPorts	`/etc/macaports/`	**port**
Chocolatey	se folosește **choco config**	**choco**

Tabelul 3.3: Actualizare / Upgrade

Sistem de pachete	Actualizare cache / versiuni	Upgrade sistem / pachete
APT	**apt update**	**apt upgrade**
DNF / YUM	**dnf makecache**	**dnf update**
Pacman	**pacman -Fy**	**pacman -Syu**
Portage	**emerge --sync**	**emerge --update --deep world**
ZYpp	N/A	**zypper update**
Port	**portsnap fetch update**	**pkg upgrade**
Homebrew	**brew update**	**brew upgrade**
MacPorts	**port selfupdate**, **port sync**	**port upgrade outdated**
Chocolatey	N/A	**choco upgrade all**

Tabelul 3.4: Instalare / Dezinstalare

Sistem de pachete	Căutare	Instalare	Dezinstalare
APT	**apt search hello**	**apt install hello**	**apt remove hello**
DNF / YUM	**dnf search hello**	**dnf install hello**	**dnf remove hello**
Pacman	**pacman -Ss hello**	**pacman -S hello**	**pacman -R hello**
Portage	**emerge --search hello**	**emerge hello**	**emerge -C hello**
ZYpp	**zypper search hello**	**zypper install hello**	**zypper remove hello**
Port	**pkg search hello**	**pkg install hello**	**pkg remove hello**
Homebrew	**brew search hello**	**brew install hello**	**brew uninstall hello**
MacPorts	**port search hello**	**port install hello**	**port uninstall hello**
Chocolatey	**choco search hello**	**choco install hello**	**choco uninstall hello**

Tabelul 3.5: Descărcare pachet / sursă

Sistem de pachete	Descărcare (fără instalare)	Descărcare sursă pachet
APT	**apt install -d hello**	**apt source hello**
DNF / YUM	**dnf download hello**	**dnf download --sourcehello**
Pacman	**pacman -Sw hello**	**asp export hello**
Portage	**emerge --fetchnly hello**	N/A
ZYpp	**zypper install --download-only hello**	**zypper source-install hello**
Port	**pkg fetch hello**	**pkg fetch hello**
Homebrew	N/A	**brew fetch hello**
MacPorts	N/A	**port fetch hello**
Chocolatey	**choco download hello**	N/A

3.4 Sisteme de distribuție cu tot cu sistem

Atunci când instalăm un pachet pe un sistem trebuie să avem grijă să fie compatibil cu celelalte pachete, adică să fie versiuni de pachete care pot lucra împreună. În unele situații avem nevoie de o anumită versiune, lucru imposibil de realizat câtă vreme versiunile altor pachete sunt incompatibile. Ne interesează să avem un mediu relativ

Tabelul 3.6: Căutare / Listare

Sistem de pachete	Căutare pachet local	Conținut pachet instalat	Pachet al unui fișier existent
APT	`dpkg -l hello`	`dpkg -L hello`	`dpkg -S /usr/bin/hello`
DNF / YUM	`rpm -qi hello`	`rpm -ql hello`	`rpm -qf /usr/bin/hello`
Pacman	`packman -Qs hello`	`packman -Ql hello`	`packman -Qo hello`
Portage	N/A	`emerge files hello`	`emerge belongs /usr/bin/hello`
ZYpp	`rpm -qi hello`	`rpm -ql hello`	`rpm -qf /usr/bin/hello`
Port	`pkg info hello`	`pkg info -l hello`	`pkg provides /usr/bin/hello`
Homebrew	`brew ls | grep -i hello`	`brew ls hello`	N/A
MacPorts	`port list`	`port contents hello`	`port provides /usr/bin/hello`
Chocolatey	`choco search --local-only hello`	N/A	N/A

izolat în care să avem instalat pachetul: o ierarhie în care pachetul să fie instalat împreună cu dependențele sale fără legătură cu ce este deja instalat la nivelul sistemului.

Această abordare de instalare numită *all-in-one* folosește un alt sistem de gestiune decât cel al sistemului. Sistemul de gestiune a pachetelor sistemului este la nivelul global al sistemului; soluțiile all-in-one lucrează independent de acesta. Soluțiile all-in-one permit actualizarea (*update*) mai facilă a pachetelor prin actualizarea directorului în care sunt instalate. Suplimentar permit configurări de securitate care pot fi realizate granular, doar la nivelul pachetului în cauză, nu la nivelul sistemului.

Un caz de utilizare pentru aceste sisteme de gestiune a pachetelor sunt soluțiile IoT (Internet of Things) despre care vom discuta în Capitolul 15. Aceasta pentru că sistemele IoT au de obicei roluri foarte bine definite și dorim să instalăm ușor o aplicație software cu toate dependențele sale.

În aceste sisteme există un pachet (o arhivă) care conține toate informațiile necesare rulării aplicație: aplicația, depdendențe, metadate, scripturi de instalare și configurare.

Sistemele Ubuntu folosesc Snapcraft [1], o soluție care înglobează aplicațiile în pachete; un pachet se numește **snap**. Un serviciu local numit **`snapd`** este responsabil de instalarea unei aplicații folosind utilitarul **`snap`**. Listing 3.9 prezintă aplicațiile instalate implicit în format **snap** pe Ubuntu 18.04.

```
1 student@uso:~$ snap list
2 Name                  Version                    Rev   Tracking
  Publisher   Notes
3 core                  16-2.45.3.1                9804  latest/stable
  canonical   core
4 core18                20200724                   1885  latest/stable
  canonical   base
5 gnome-3-26-1604       3.26.0.20200529            100   latest/stable/
  canonical   -
6 gnome-3-28-1804       3.28.0-17-gde3d74c.de3d74c 128   latest/stable
  canonical   -
```

[1] https://snapcraft.io/

```
7  gnome-3-34-1804       0+git.3009fc7              36    latest/stable
   canonical   -
8  gnome-calculator      3.36.0+git9.96b95fd2       748   latest/stable/
   canonical   -
9  gnome-characters      v3.34.0+git8.a46106b       550   latest/stable/
   canonical   -
10 gnome-logs            3.34.0                     100   latest/stable/
   canonical   -
11 gnome-system-monitor  3.36.0-12-g35f88a56d7      148   latest/stable/
   canonical   -
12 gtk-common-themes     0.1-36-gc75f853            1506  latest/stable/
   canonical   -
```

Listing 3.9: Pachete instalate în format snap pe Ubuntu 18.04

Pe un sistem pot coexista pachete instalate folosind managerul de pachete al sistemului și pachete snap.

Listing 3.10 prezintă operații uzuale folosind utilitarul **`snap`**.

```
1  # Cauta pachete care au in nume sau in descrire sirul "svg".
2  student@uso:~$ snap find svg
3
4  # Instaleaza pachetul inkscape.
5  student@uso:~$ sudo snap install inkscape
6
7  # Afiseaza informatii despre pachetul inkscape (instalat sau nu).
8  student@uso:~$ snap info inkscape
9
10 # Dezinstaleaza pachetul inkscape.
11 student@uso:~$ sudo snap remove inkscape
12
13 # Actualizeaza pachetul inkscape.
14 student@uso:~$ sudo snap refresh inkscape
15
16 # Actualizeaza pachetele sistemului.
17 student@uso:~$ sudo snap refresh
18
19 # Descarca pachetul inkscape fara sa fie instalat.
20 student@uso:~$ snap download inkscape
```

Listing 3.10: Operații uzule cu snap

Alte soluții de tip all-in-one sunt Flatpak [1] și AppImage [2]. Flatpak creează un mediu izolat folosind o formă de virtualizare de aplicație, similar unui container, descris în Secțiunea 14.1.3. În acest mediu izolat rulează aplicații. AppImage este o soluție care permite instalarea unei aplicații fără a fi nevoie de cont privilegiat; există aplicații care sunt descărcabile în format de pachet AppImage și instalabile astfel de un utilizator neprivilegiat.

Un avantaj al folosirii soluțiilor all-in-one este că folosesc versiuni mai recente ale aplicațiilor. În vreme ce aplicațiile care folosesc managerul de pachete al sistemului vor folosi versiunea disponibilă în repository, care ține de versiunea curentă a distribuției.

[1] https://www.flatpak.org/
[2] https://appimage.org/

3.4.1 Medii specifice pentru limbaje de programare

Limbaje de programare precum Python sau Ruby permit dezvoltarea de aplicații în aceste limbaje și distribuirea acestora ca pachete. Pentru aceasta cele două limbaje au sisteme de gestiune ale pachetelor: **`pip`** pentru Python și **`gem`** pentru Ruby.

Sistemele de gestiune a pachetelor pentru limbaje de programare sunt independente de sistemele de gestiune a pachetelor la nivel de sistem. Folosesc în mod similar sistemul de fișiere, dar se ocupă de gestiunea acestor noi pachete.

Pachetele specifice Python sau Ruby se pot găsi însă și în repository-ul sistemului de gestiune a pachetelor la nivelul sistemului. Astfel că, dacă se instalează pachete în ambele sisteme (cel al sistemului și **`pip`** sau **`gem`**), pot apărea conflicte.

În mod ideal, vom folosi doar pachetele de la nivelul sistemului. Dar sunt situații în care ne dorim să folosim **`pip`** sau **`gem`**:

- dorim să instalăm pachetele într-o zonă locală (un mediu virtual) pentru o aplicație, fără a fi afectate de pachetele din sistem
- pachetele respective nu se găsesc în repsitory-ul sistemului de gestiune a pachetelor
- pachetele de la nivelul sistemului au versiuni nepotrivite (prea vechi)
- dorim să instalăm pachetele doar la nivelul utilizatorului curent, nu la nivelul sistemului

Pentru a instala **`pip`** și **`gem`** pe un sistem Ubuntu folosim comenzile din Listing 3.11.

```
1 student@uso:~$ sudo apt install python3-pip python-dev
2
3 student@uso:~$ sudo apt install rubygems
```

Listing 3.11: Instalarea Python pip și Ruby gem pe Ubuntu

Listing 3.12 conține exemple de folosire a utilitarului **`pip`** iar Listing 3.13 conține exemple de folosire a utilitarului **`gem`** pentru căutarea, instalarea, listarea și dezinstalarea de pachete.

```
1  # Afiseaza pachetele instalate local.
2  student@uso:~$ pip3 list
3
4  # Cauta pachete care au in nume sau in descrire sirul "pwn".
5  student@uso:~$ pip3 search pwn
6
7  # Instaleaza pachetul pwntools.
8  student@uso:~$ sudo pip3 install pwntools
9
10 # Afiseaza informatii despre pachetul pwntools (instalat sau nu).
11 student@uso:~$ pip3 show pwntools
12
13 # Dezinstaleaza pachetul pwntools.
14 student@uso:~$ sudo pip3 uninstall pwntools
15
16 # Actualizeaza pachetul pwntools.
17 student@uso:~$ sudo pip3 install -U pwntools
18
19 # Actualizeaza pachetele sistemului.
```

```
student@uso:~$ sudo pip3 install -U

# Descarca pachetul pwntools fara sa fie instalat.
student@uso:~$ pip3 install -d pwntools

# Instaleaza pachetul pwntools doar pentru utilizatorul curent.
student@uso:~$ pip3 install --user pwntools
```

Listing 3.12: Gestiunea pachetelor folosind pip

```
# Afiseaza pachetele instalate local.
student@uso:~$ gem list

# Cauta pachete care au in nume sau in descrire sirul "rake".
student@uso:~$ gem search pwn

# Instaleaza pachetul rake.
student@uso:~$ sudo gem install rake

# Afiseaza informatii despre pachetul rake (instalat sau nu).
student@uso:~$ gem list -d rake

# Dezinstaleaza pachetul rake.
student@uso:~$ sudo gem uninstall rake

# Actualizeaza pachetul rake.
student@uso:~$ sudo gem update rake

# Actualizeaza pachetele sistemului.
student@uso:~$ sudo gem update

# Descarca pachetul rake fara sa fie instalat.
student@uso:~$ gem fetch rake

# Instaleaza pachetul rake doar pentru utilizatorul curent.
student@uso:~$ gem install --user-install rake
```

Listing 3.13: Gestiunea pachetelor folosind gem

Pentru a preveni conflicte, sau pentru a lucra la un proiect specific în Python sau Ruby, se pot crea medii specifice unde se instalează pachete folosind ruby sau gem. Aceste medii se numesc medii virtuale și sunt similare celor create de sisteme precum Snapcraft sau Flatpack; aceste medii se mai numesc și *sandbox*-uri. Aceste medii sunt create cu ajutorul utilitarelor **`virtualenv`** pentru Python și **`bundler`**. Mediile create folosind **`virtualenv`** și **`bundler`** sunt îndeosebi utile dezvoltatorilor în Python și Ruby. Aceștia vor crea un astfel de mediu izolat de lucru, în care vor instala versiunile de pachete necesare pentru dezvoltarea aplicației lor.

Pentru a fi disponibile aceste utilitare, trebuie să instalăm utilitarele respective. Putem opta pentru a le instala chiar folosind **`pip`** și **`gem`** ca în Listing 3.14.

```
student@uso:~$ sudo pip3 install virtualenv

student@uso:~$ sudo gem install bundler
```

Listing 3.14: Instalarea utilitarelor virtualenv și rvm

Listing 3.15 conține secvențe de comenzi pentru crearea unui mediu virtual Python și instalarea unui pachet. Aceste pachete sunt instalate doar în mediul virtual din directorul `myenv/` fără a fi afectate de pachetele instalate în sistem.

```
student@uso:~$ virtualenv myenv

student@uso:~$ cd myenv/
student@uso:~/myenv$ ls -F
bin/  lib/  pyvenv.cfg

student@uso:~/myenv$ source bin/activate

(myenv) student@uso:~/myenv$ pip3 install grip
Collecting grip
  Downloading grip-4.5.2.tar.gz (145 kB)
[...]

(myenv) student@uso:~/myenv$ ls bin/grip
bin/grip

(myenv) student@uso:~/myenv$ grip
[...]

(myenv) student@uso:~/myenv$ deactivate
student@uso:~/myenv$
```

Listing 3.15: Crearea unui mediu virtual (Python și Ruby)

O soluție similară pentru JavaScript (Node.js [1]) folosește utilitarul **npm** [2] ca sistem de gestiune a pachetelor și **nvm** [3] pentru crearea unui mediu izolat.

3.5 Anexă: Instalarea unui pachet din surse

Modul uzual de a obține o aplicație este prin intermediul unui pachet software și a unui sistem de gestiune a pachetelor ca mai sus. Aceste pachete conțin de obicei executabile și cod compilat pentru a face cât mai rapid procesul de compilare.

Cu toate acestea, există situații în care suntem interesați de obținerea codului sursă al unei aplicații și compilarea acesteia și instalarea manuală din codul compilat. În mod evident, doar pentru aplicațiile open source, la care avem acces la codul sursă. În trecut, această formă era frecventă pentru că distribuțiile erau puține și nu aveau toate pachetele; singurul mod de a instala pachetul era obținând sursele și compilându-l. În zilele noastre, nu mai este cazul; majoritatea distribuțiilor au foarte multe pachete în depozitele lor de pachete. Rămân însă câteva motive pentru care ne-am dori instalarea din surse:

- aplicația este suficient de obscură încât să nu se găsească în depozitul de pachete al distribuției

[1] https://nodejs.org/en/
[2] https://www.npmjs.com/
[3] https://github.com/nvm-sh/nvm

- dorim să editǎm codul sursă al aplicației pentru a adǎuga anumite funcționalități punctuale nevoilor noastre; acest scenariu e mai degrabă util unui dezvoltator, nu unui utilizator obișnuit
- aplicația este în dezvoltare, nu este încă lansată și nu există pachete; acest scenariu este de asemenea util unui dezvoltator
- vrem să compilăm aplicația într-un anumit mod, folosind funcționalități de compilare care țin cont de particularități ale sistemului (de exemplu arhitectura procesorului sau componente hardware specifice)

Instalarea din codul sursă presupune obținerea unei arhive care are codul sursă al aplicației sau clonarea repository-ului sursă al aplicației. În cadrul codului sursă astfel obținut se regăsesc în mod uzual un fișier `README` cu indicații de instalare și scripturi sau componente de compilare.

În general pentru compilare se folosesc sisteme de build care automatizează procesul. Acestea pot fi Make, Cmake, Scons, Maven, Ant, Gradle depinzând de limbajul de programare folosit și de preferințele dezvoltatorului. Mai multe informații despre sistemele de build se găsesc în Capitolul 6.

În cazul aplicațiilor scrise în limbajele C/C++ cel mai frecvent sistem de build este cel bazat pe Make, împreună cu utilitarele complementare. Acest sistem se numește Autotool, sau sistemul de build GNU.

Ca exaemplu, Listing 3.16 conține secvența de comenzi care duce la instalarea OpenSC [1], un set de aplicații și biblioteci pentru a lucra cu smart carduri.

```
1  student@uso:~$ cd Downloads/
2  student@uso:~/Downloads$ wget https://github.com/OpenSC/OpenSC/releases/
   download/0.20.0/opensc-0.20.0.tar.gz
3  [...]
4  Saving to: 'opensc-0.20.0.tar.gz'
5
6  student@uso:~/Downloads$ tar xzf opensc-0.20.0.tar.gz
7  student@uso:~/Downloads$ cd opensc-0.20.0/
8  student@uso:~/Downloads/opensc-0.20.0$ ls
9  aclocal.m4  bootstrap.ci  config.guess  config.sub  configure.ac  depcomp
     etc         ltmain.sh  MacOSX       Makefile.in   missing  packaging
   src          tests          win32
10 bootstrap   compile       config.h.in   configure   COPYING       doc
        install-sh  m4         Makefile.am  Makefile.mak  NEWS     README
       test-driver  version.m4.ci
11
12 student@uso:~/Downloads/opensc-0.20.0$ ./configure
13 [...]
14 configure: error: winscard.h is required for pcsc
15
16 student@uso:~/Downloads/opensc-0.20.0$ apt-file search winscard.h
17 libpcsclite-dev: /usr/include/PCSC/winscard.h
18 libwine-dev: /usr/include/wine/windows/winscard.h
19 libwine-development-dev: /usr/include/wine-development/wine/windows/
   winscard.h
20 mingw-w64-common: /usr/share/mingw-w64/include/winscard.h
21 mingw-w64-i686-dev: /usr/i686-w64-mingw32/include/winscard.h
22 mingw-w64-x86-64-dev: /usr/x86_64-w64-mingw32/include/winscard.h
```

[1] https://github.com/OpenSC/OpenSC/wiki

```
23
24 student@uso:~/Downloads/opensc-0.20.0$ sudo apt install libpcsclite-dev
25 [...]
26
27 student@uso:~/Downloads/opensc-0.20.0$ ./configure
28 [..]
29 configure: creating ./config.status
30 config.status: creating Makefile
31 [...]
32
33 student@uso:~/Downloads/opensc-0.20.0$ ls
34 aclocal.m4    compile       config.h.in    config.sub    COPYING  etc
        ltmain.sh  Makefile     Makefile.mak  packaging  stamp-h1
   version.m4.ci
35 bootstrap     config.guess  config.log     configure     depcomp  install
   -sh  m4         Makefile.am  missing       README     test-driver  win32
36 bootstrap.ci  config.h      config.status  configure.ac  doc      libtool
     MacOSX     Makefile.in  NEWS          src        tests
37
38 student@uso:~/Downloads/opensc-0.20.0$ make
39 make  all-recursive
40 make[1]: Entering directory '/home/student/Downloads/opensc-0.20.0'
41 Making all in etc
42   [...]
43   CC       compat_getopt_main.o
44   CC       compat_dummy.lo
45   CC       compat_strlcat.lo
46   [...]
47
48 student@uso:~/Downloads/opensc-0.20.0$ ls src/tools/
49 cardos-tool                 goid-tool.c                   openpgp-tool.
   c                       opensc-notify.plist.in
   pkcs15-crypt
50 [...]
51 student@uso:~/Downloads/opensc-0.20.0$ sudo make install
52 [...]
53 student@uso:~/Downloads/opensc-0.20.0$ sudo ldconfig
54 student@uso:~/Downloads/opensc-0.20.0$ cardos-tool
55 No smart card readers found.
56 student@uso:~/Downloads/opensc-0.20.0$ pkcs15-tool
57 Usage: pkcs15-tool [OPTIONS]
58 Options:
59       --version                Print OpenSC package version
60       --list-info              List card information
```

Listing 3.16: Instalarea OpenSC din surse

În secvența de comenzi pentru instalarea din surse de mai sus, am urmat pașii:

1. liniile `1-X`: am descărcat arhiva ce conține sursele OpenSC
2. liniile `X-Y`: am despachetat arhiva
3. liniile `X-Y`: am văzut că nu am avem fișier `Makefile` dar există fișierul `configure`
4. liniile `X-Y`: am rulat scriptul `configure`; am observat că nu există în sistemul fișierul header `winscard.h`
5. liniile `X-Y`: am căutat fișierul header `winscard.h` în pachete folosind **`apt-file`**;

am descoperit pachetul `libpcslite-dev` pe care l-am instalat

6. liniile `X-Y`: am rulat din nou scriptul `configure`; acum a rulat cu succes
7. liniile `X-Y`: am observat prezența fișierului `Makefile` generat din rularea cu succes a scriptului `configure`
8. liniile `X-Y`: am rulat comanda **`make`** pentru a compila sursele și a obține fișierele executabile și bibliotecile dorite
9. liniile `X-Y`: am observat existența fișierelor executabile dorite în directorul `src/tools/`, precum `cardos-tool` sau `pkcs15-crypt`
10. liniile `X-Y`: am instalat pachetele în sistem folosind comanda **`make install`** și actualizând cacheul de biblioteci (**`ldconfig`**), astfel că executabilele generate (**`cardos-tool`**, **`pkcs15-tool`** sunt acum folosibile)

După cum se observă, compilarea din surse necesită cunoștințe tehnice mai avansate legate de sisteme de build, despre care vom discuta în Capitolul 6. De aceea sunt de evitat pentru majoritatea utilizatorilor. Pentru aproape toate scenariile acestora, sistemele de gestiune ale pachetelor prezentate mai sus vor fi suficiente.

3.6 Sumar

Un dezvoltator sau firmă de dezvoltare are nevoie de o formă de distribuție a aplicației create pentru a ajunge la utilizatori. Uzual este în formă de pachete software.

Pachetele conțin date și metadate care sunt prelucrate de un sistem de gestiune a pachetelor pentru instalare, dezinstalare.

Sistemele de gestiune sunt aplicații care cunosc formatul pachetelor. Exemplu sunt APT sau YUM pe Linux.

Pachetele se găsesc într-un depozit (*repository*) sau magazin de apicații de unde aplicația de gestiune a pachetelor le obține. Pachetul este obținut dintr-un repository și instalat.

Gestiunea pachetelor poate fi dificilă, pot fi versiuni incompatibile. O soluție este folosirea sistemelor *all-in-one*.

Capitolul 4

Procese

Un proces este o aplicație ce se execută în sistemul de operare, folosind resursele sistemului, pentru a realiza o acțiune cerută de utilizator. Acțiunile utilizatorului pot fi să deschidă o pagină web, să creeze o imagine, să asculte o melodie. Utilizatorul face aceste acțiuni folosind aplicații precum un browser web, o suită Office, un player audio.

La nivelul sistemului de operare, aceste aplicații sau comenzi pornite de utilizator sunt procese. Atunci când o aplicație sau un program rulează, spunem că face acest lucru în contextul unui proces. Spunem despre procese că **rulează** sau că **se execută** sau că **se află în execuție**. Procesele sunt entități active care, atunci când rulează, folosesc resursele sistemului: procesor, memorie, sistem de fișiere.

Înțelegerea proceselor este esențială pentru orice utilizator pentru a asigura buna funcționare a sistemului; atunci când sistemul nu este responsiv, cauza probabil este un proces care consumă excesiv resursele sistemului. Un dezvoltator de aplicații va folosi cunoștințe despre procese pentru a proiecta aplicații performante care să folosească cât mai eficient resursele sistemului. Un inginer de sistem va monitoriza procesele sistemului pentru a se asigura de buna funcționare a acestuia și pentru a preveni atacuri de securitate sau abuz de resurse.

O parte dintre procese sunt pornite de la acțiuni ale utilizatorilor, precum rularea unei aplicații grafice sau a unei comenzi. Numim aceste procese **interactive** pentru că interacționează cu utilizatorul pe parcursul rulării lor. Alte procese sunt pornite de sistem și nu interacționează cu utilizatorul. Acestea au rol în gestiunea sistemului sau oferirea de servicii; de exemplu conectarea la rețea în momentul în care există o rețea wireless disponibilă sau actualizarea ceasului sistemului sau indexarea fișierelor pentru căutare rapidă. Aceste procese sunt **neinteractive**. Le numim **servicii** sau, pe Unix, procese **daemon**. Vom discuta mai în detaliu în Secțiunea 4.6.3.

Unele aplicații pot fi compuse din mai multe procese. De exemplu, un browser web precum Google Chrome pornește un proces pentru fiecare tab; un server web care servește cereri în rețea poate avea mai multe procese care fac servirea; o fereastră de terminal are un proces shell deschis pentru fiecare tab; mai mult, într-o fereastră de terminal pot rula mai multe procese așa cum vom vedea în Secțiunea 4.3.3. Altfel spus, când folosim interfața grafică, o fereastră de aplicație care rulează poate avea în spate mai multe procese. Sistemul de operare folosește noțiunea de proces; este preocuparea dezvoltatorului de aplicații dacă o aplicație folosește, când rulează, unul sau mai multe procese. Figura 4.1 este o perspectivă a proceselor și resurselor folosite de acestea.

procese și cerințe: Esențial este că orice aplicație care rulează în sistem, folosește unul sau mai multe procese pentru a satisface cerințele utilizatorului.

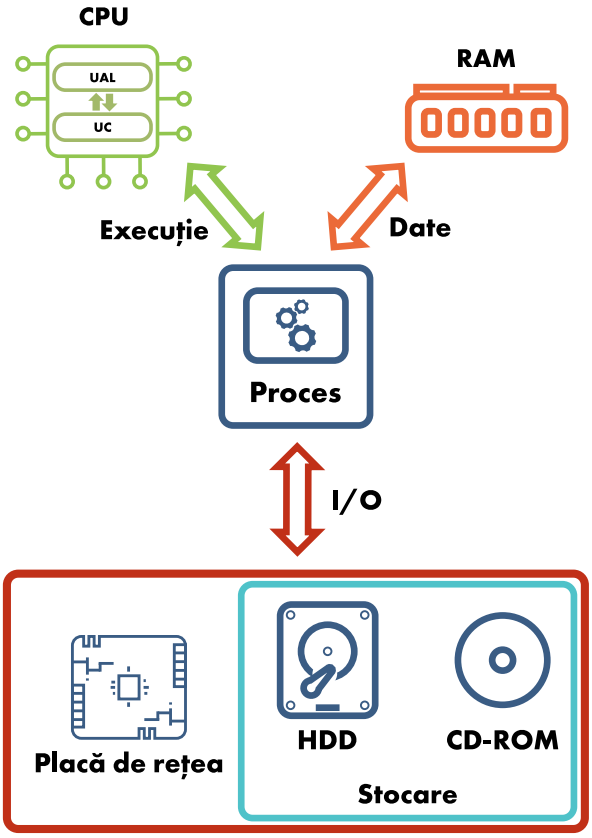


Figura 4.1: Procese și resursele lor

Pentru utilizator și sistemul de operare, procesele sunt necesare ca formă de încapsulare. Un proces execută o anumită acțiune fără a afecta alt proces. În general, procesele au resurse proprii oferite de sistemul de operare, dar pot și partaja anumite resurse. Acest lucru este util pentru a realiza acțiuni ce nu pot fi obținute cu un singur proces. De exemplu, un proces descărcă un fișier video, alt procese îl convertește într-un format adecvat, alt proces îl vizualizează. Vom vedea mai multe despre comunicarea și interacțiunea între procese în Secțiunea 4.5

Concret, sistemul de operare oferă:

- izolare a resurselor proceselor, asigurându-se că, în mod normal, un proces nu poate accesa sau corupe resursele altui proces. În acest fel sistemul este menținut integru și fiecare proces se execută corespunzător.
- arbitrare a accesului la resurse partajate: dacă două sau mai multe procese accesează o resursă comună (de exemplu un fișier), atunci accesele trebuie să fie bine ordonate și, de exemplu, să nu suprascrie un proces informațiile celuilalt.
- mecanisme explicite de comunicare între procese: procesele comunică între ele pentru realizarea unei acțiuni mai complexe. Vom discuta în Secțiunea 7.1.2.3, respectiv Secțiunea 4.5.4 despre semnale și pipe-uri, două mecanisme de bază de comunicare între procese.
- comportament echitabil al proceselor, prin asigurarea că un proces nu utiliza prea

multe resurse: un proces poate utiliza foarte multă memorie sau foarte mult procesor și împiedica alte procese să ruleze, iar sistemul de operare are grijă să aloce fiecărui proces resurse în mod echitabil

4.1 Programe și procese

Am precizat mai sus că un proces rulează și folosește resursele sistemului pentru o anumită. Pentru aceasta, un proces stochează în memorie instrucțiunile pe care trebuie să le execute. Cum ajung aceste instrucțiuni în memorie?

Instrucțiunile (numite și codul) executate de un proces sunt puse în memoria sa în momentul creării sale. Un proces este creat dintr-un program; un program este un fișier executabil care conține codul ce va fi executat. În momentul în care pornim o aplicație sau rulăm o comandă, un program este încărcat (*loaded*) în memorie și se obține un proces. Încărcarea programului înseamnă, așadar, copierea instrucțiunilor ce vor fi executate din fișierul executabil în memoria viitorului proces și pornirea procesului. Mai multe informații despre magia din spatele acțiunii de încărcare (*loading*) se găsesc în Secțiunea 4.9.

De exemplu, atunci când pornim browserul web Firefox, se creează un proces cu care interacționăm. Putem vedea acest proces în Linux/Unix/macOS folosind utilitarul **`ps`** sau putem să-l vizualizăm în Windows folosind Task Manager; în momentul vizualizării putem să vedem care este programul (fișierul executabil) din care a fost creat procesul Firefox, așa cum se vede în Listing 4.1. În Listing 4.1 prima rulare este în Linux, iar a doua în macOS. În cazul Linux, programul (fișierul executabil) aferent procesului Firefox este `/usr/lib/firefox/firefox` iar în cazul macOS, programul (fișierul executabil) este `/Applications/Firefox.app/Contents/MacOS/firefox`.

```
1 student@uso:~$ ps -ef | grep firefox
2 student   2637     1  0 01:32 tty1     00:02:00 /usr/lib/firefox/firefox
  -new-window
3 [...]
4 student@uso_macos:~$ ps -ef | grep Firefox
5 501 66908     1   0 12:36PM ??         0:14.07 /Applications/Firefox.app/
  Contents/MacOS/firefox
6 [...]
```

Listing 4.1: Fișierul executabil al unui process

> Un proces este, așadar, un program în execuție care folosește resursele sistemului de calcul pentru a realiza una sau mai multe acțiuni. Mai spunem că un proces este un program căruia i s-a atașat un context de execuție. Un program este o entitate statică, un fișier executabil în sistemul de fișiere. Un proces este, pe de altă parte, o entitate dinamică, una care rulează și care folosește resursele sistemului.

Pentru un proces spunem că programul executabil din care a fost creat este **imaginea procesului** (*process image*). Mai multe procese pot fi create din același program executabil. De exemplu, în Google Chrome orice tab este un proces creat din același program; de asemenea, orice tab de terminal creează un nou proces shell; la fel, un utilizator poate porni un proces Vim într-un terminal și un proces Vim în alt terminal. În

toate aceste situații, se vor crea procese distincte pronind de la același executabil, adică procese ce au aceeași imagine.

Figura 4.2 arată cum unul sau mai multe procese sunt create dintr-un fișier executabil (imaginea procesului).

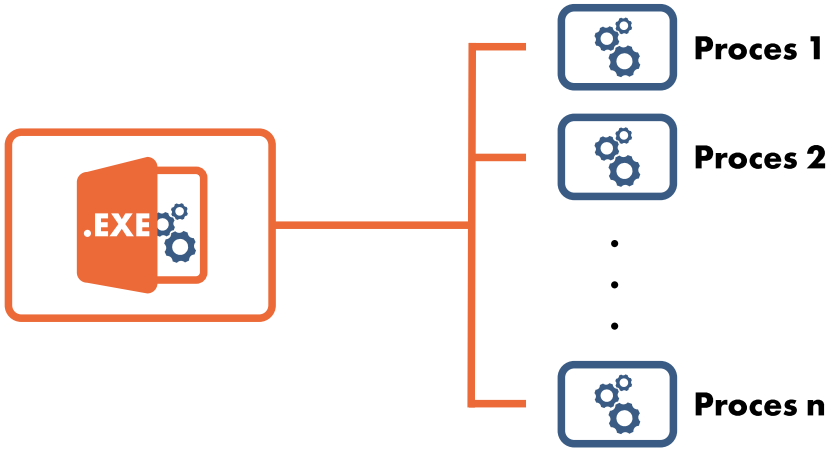


Figura 4.2: Procese și programe (executabile)

Întrucât mai multe procese pot fi create din același fișier executabil, este tehnic incorect să ne referim la procesul Firefox sau procesul Apache. Putem folosi termenul **un** proces Firefox sau **un** proces Apache. Numele unui proces identifică imaginea de proces, executabil din care a fost creat; numele nu este folosit pentru a identifica în mod unic un proces. Procesul este identificat la nivelul sistemului de un număr unic, numit *Process Id* sau PID. Prezentăm mai multe despre atributele unui proces, inclusiv identificatorul său, în secțiunea următoare.

4.2 Resursele și atributele unui proces

Un proces rulează și folosește resursele sistemului de calcul: memorie, procesor, elemente de input/output (I/O): disc, rețea, tastatură etc. Un proces folosește memoria sistemului pentru a stoca instrucțiunile din executabil și a stoca date prelucrate. Instrucțiunile sunt apoi executate pe procesor, iar procesorul operează cu date din memorie. Pe parcursul execuției, procesul comunică și cu elemente de I/O sau cu alte procese. Figura 4.3 arată modul în care un proces folosește resursele sistemului.

Unui proces i se alocă resurse (de exemplu zone de memorie) în momentul creării (încărcării sale, numit și *load-time*) și în momentul rulării (numit și *run-time*). Aceste zone pot fi modificate la *run-time*, la cererea procesului însuși, prin alocare dinamică (de exemplu `malloc()`). Pe măsură ce rulează, unui proces îi sunt alocate alte resurse. De exemplu un proces deschide și lucrează cu fișiere sau canale de comunicare în rețea sau mecanisme de comunicare cu alte procese.

Este important să putem investiga și monitoriza resursele folosite de un proces pentru a identifica procesele care acaparează resurse excesiv și care pot îngreuna sistemul. Putem modifica parametri și atribute ale proceselor pentru a afecta cât de multe resurse utilizează. De aceea sistemele de operare oferă primitive de listare și monitorizare a proceselor, precum utilitarele **`ps`** și **`top`** în Linux sau aplicația Task Manager în

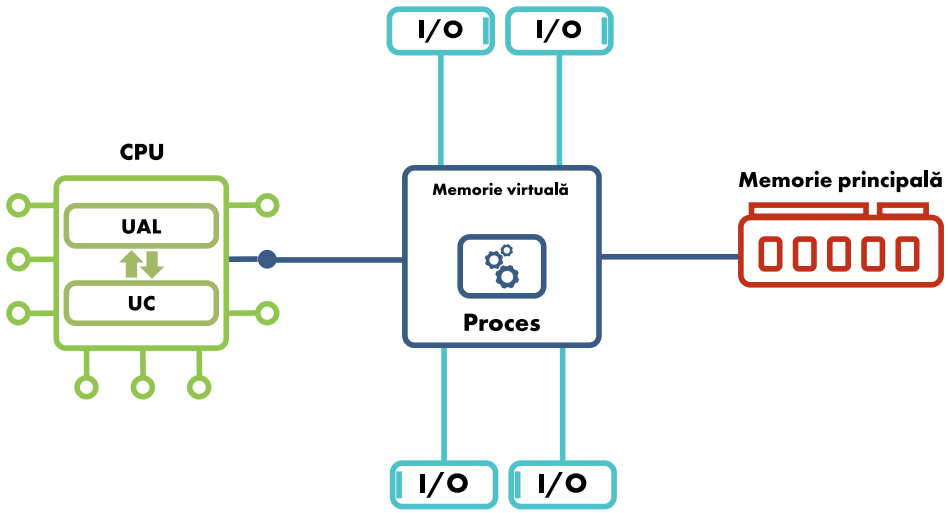


Figura 4.3: Procese și resursele sistemului

Windows. Listing 4.2 conține exemple de rulare a comenzii ps, iar Figura 4.4 și Figura 4.5 conțin screenshot-uri, respectiv cu rularea **`top`** (în Linux) și Task Manager (în Windows). Vedem în aceste figuri și secvențe resursele de procesor, memorie și I/O (de obicei disc) folosite de proces.

```
student@uso:~$ ps
  PID TTY          TIME CMD
 9585 pts/1    00:00:00 bash
 9610 pts/1    00:00:00 ps
student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   9585  9584  0 13:58 pts/1    00:00:00 -bash
student   9611  9585  0 14:00 pts/1    00:00:00 ps -f
student@uso:~$ ps -e -f
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 oct02 ?        00:00:05 /sbin/init splash
root         2     0  0 oct02 ?        00:00:00 [kthreadd]
root         4     2  0 oct02 ?        00:00:00 [kworker/0:0H]
root         6     2  0 oct02 ?        00:00:00 [mm_percpu_wq]
root         7     2  0 oct02 ?        00:00:03 [ksoftirqd/0]
root         8     2  0 oct02 ?        00:00:03 [rcu_sched]
[...]
student@uso:~$ ps -o pid,ppid,user,rss,%cpu
  PID  PPID USER       RSS %CPU
 9585  9584 student   5216  0.0
 9622  9585 student   3528  0.0
student@uso:~$ ps -e -o pid,ppid,user,rss,%cpu
  PID  PPID USER       RSS %CPU
    1     0 root      7292  0.0
    2     0 root         0  0.0
    4     2 root         0  0.0
    6     2 root         0  0.0
[...]
 1439  1018 student   3988  0.0
 1443  1018 student   3484  0.0
 1447     1 root     70092  0.0
[...]
```

Listing 4.2: Investigarea proceselor folosind ps

În Listing 4.2 este folosit utilitarul **ps** cu diferite opțiuni. Astfel, comanda de la linia 1 adfișează procesele din terminalul curent, la linia 5 se rulează comanda pentru afișarea proceselor de la terminalul curent în format complet (*full*), la linia 9 se rulează comandă pentru afișarea tuturor proceselor (`-e` pentru *everything* în format complet, la linia 18 se rulează comanda pentru afișarea, pentru procesele din terminalul curent, a atributelor PID, PPID, utilizator, memoria rezidentă și procentul de procesor consumat, iar la linia 22 la fel ca la comanda anterioară doar că pentru toate procesele sistemului.

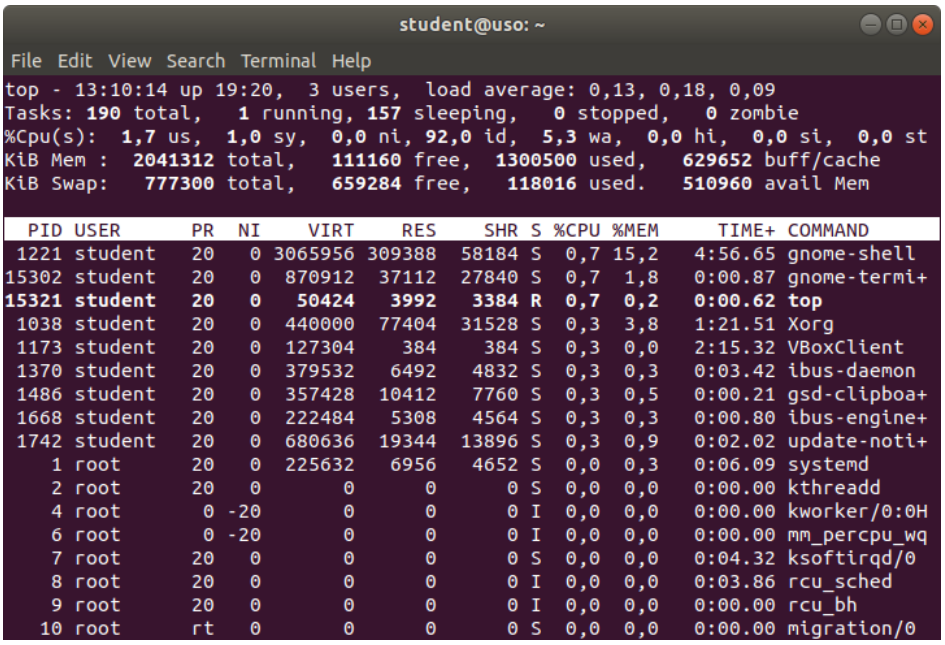


Figura 4.4: Utilitarul top

Mai multe informații despre investigarea proceselor și monitorizarea resurselor utilizate se găsesc în Secțiunea 4.7.

4.2.1 Atributele unui proces

Un proces are atribute stabilite în mod normal la pornirea sa (la *load-time*): identificatorul procesului, procesul părinte, utilizatorul, prioritatea de rulare. Unele atribuite sunt stabilite sau modificate la run-time. De exemplu, prioritatea unui proces poate fi modificată pentru a afecta accesul la resurse; un proces cu prioritate mai bună va avea acces mai des la procesor sau la disc decât procese cu prioritate mai slabă.

Atributele unui proces au rolul de identifica un proces, de a stabili ce și cât de multe resurse poate folosi și de a contabiliza resursele utilizate. O listă de atribute de bază ale unui proces împreună cu o descriere a lor se găsește în Tabelul 4.1.

Atributul PID este identificatorul unic al procesului la nivelul sistemului de operare. Fiecare proces primește un identificator în momentul creării, atribut ce nu se schimbă pe parcursul rulării. În general, comenzile sau funcțiile care lucrează cu un proces primesc ca argument PID-ul acestui proces. De exemplu, dacă vrem să terminăm un proces, vom folosi comanda **kill** cu argument PID-ul acelui proces, ca în Listing 4.3. În Listing 4.3 se observă că există un proces generat din programul **sleep** cu PID-ul

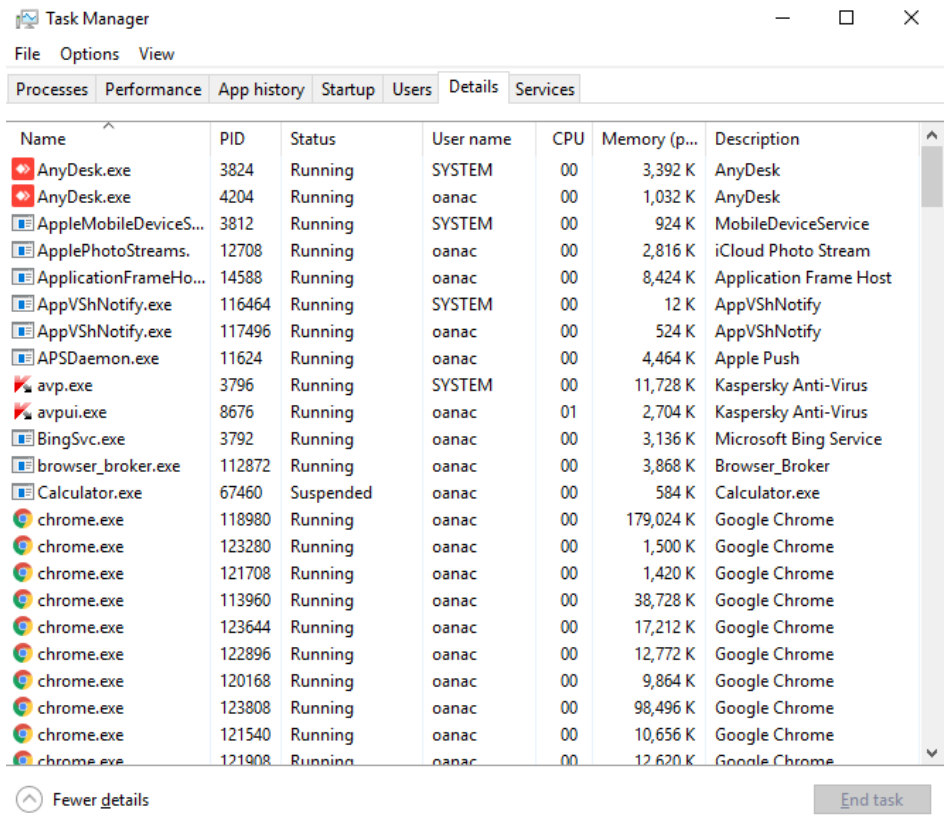


Figura 4.5: Utilitarul Task Manager

`9982`. Apoi procesul este omorât folosind comanda **`kill`** având ca argument PID-ul acelui proces.

```
1 student@uso:~$ ps -e | grep 9982
2  9982 pts/2    00:00:00 sleep
3 student@uso:~$ kill 9982
4 student@uso:~$ ps -e | grep 9982
```

Listing 4.3: Terminarea unui proces folosind comanda kill

Atributul UID este identificatorul utilizatorului ce deține procesul. Un proces cu un anumit UID va avea acces la resursele acelui utilizator. În mod normal, procesele aceluiași utilizator au acces la aceleași resurse. Utilizatorii sunt modul principal de separare/izolare între procese. Mai multe detalii vom prezenta în Capitolul 5.

4.2.2 Utilitare pentru urmărirea proceselor

Sistemul de operare oferă utilitare și comenzi pentru a afișa procese și pentru a urmări atributele și resursele lor.

Utilitarele din această categorie sunt de două tipuri:

- cele care afișează un snapshot al momentului (procese active în acest moment și atribute ale lor)
- cele care monitorizează procesele sistemului

Tabelul 4.1: Atributele unui proces

atribut	rol	momentul atribuirii	modificabil
PID	identificare proces	pornire	nu
PPID	identificare părinte proces	pornire	da
program executabil	imaginea procesului (cod și date)	pornire	nu
UID/GID (utilizator/grup)	permisiuni proces	pornire	nu (cu excepția unor situații punctuale)
prioritate statică (*nice*)	importanța în accesul resurselor	pornire	da
terminal	interfața de comunicare cu utilizatorul	pornire	da
fișiere deschise	lucrul cu fișiere	rulare	da
stare	accesul curent la procesor	rulare	da
timp de rulare pe procesor	contabilizare consum de procesor	rulare	da
memorie consumată	contabilizare consum de memorie	rulare	da
spațiu virtual de adrese	harta memoriei unui proces	pornire	da

Din prima categorie fac parte, în Linux, utilitarele **`ps`**, **`pgrep`**, **`pidof`**, **`pstree`**, **`pmap`**, **`lsof`**. Din a doua categorie fac parte **`top`**, **`htop`**, **`iotop`**, **`sysstat`**. Pe Windows utilitarele precum Task Manager, Process Explorer monitorizează procesele sistemului.

Am prezentat în Figura 4.4 și Figura 4.5 screenshot-uri cu utilitarele **`top`** și Task Manager, utilitare de monitorizare a proceselor. Monitorizarea proceselor e utilă pentru a urmări procesele din sistem, a observa tendințe, a vedea consumul de resurse și pentru a investiga de ce sistemul sau un proces funcționează anevoios. Mai multe informații despre monitorizarea și investigarea proceselor vom prezenta în Secțiunea 4.7.

Mai jos sunt descrise utilitarele pentru afișarea de informații de tip snapshot despre procese:

- **`ps`** este principalul utilitar de afișare de informații despre procese. La o rulare simplă afișează procesele din terminalul curent. Poate afișa selectiv procese și atribute ale acestora.

- **`pidof`** afișează PID-ul proceselor care au un anumit program (imagine de proces)

- **pgrep** are funcționalitatea utilitarului **pidof** extinsă: afișează procesele care corespund unei anumite condiții. Condiția poate fi „aparține unui anumit utilizator”, „are un anumit program ca imagine”, „are un anumit proces părinte”, „rulează într-un anumit terminal”, la fel ca în exemplele de mai jos.
- **pstree** afișează ierarhia de procese a sistemului, îl vom detalia în Secțiunea 4.3.1.
- **pmap** (*process map*) afișează harta memoriei unui proces, adică zonele de memorie ocupate de acesta. Este un utilitar adecvat în special pentru programatori și pentru cei care sunt interesați de internele sistemelor de operare, nu insistăm pe el.
- **lsof** (*list open files*) este utilitarul care afișează fișierele deschise de un proces. Îl vom folosi în Secțiunea 4.4

Așa cum am precizat, utilitarul **ps** este utilitarul principal pentru listarea proceselor și a atributelor acestora. Am prezentat exemple de investigare a proceselor folosind utilitarul **ps** în Listing 4.2. Listing 4.4 conține scenarii frecvente de folosire a comenzii **ps** cu argumentele aferente. Comenzile rulate în Listing 4.4 au următorul efect:

- linia 1: listarea tuturor proceselor din sistem
- linia 8: listarea tuturor proceselor fără afișarea capului de tabel
- linia 15: listarea cu mai multe atribute a proceselor din sistem
- linia 22: listarea proceselor ce aparțin utilizatorului `student`
- linia 29: listarea proceselor ce nu aparțin utilizatorului `student`
- linia 36: listarea proceselor ce au imagine (program) **VBoxClient**
- linia 43: listarea doar a PID-urilor, comenzii de pornire, procentul de procesor acaparat și memorie utilizate pentru toate procesele
- linia 50: listarea doar a PID-urilor, comenzii de pornire și a stării pentru procesele ce aparțin utilizatorului `student`
- linia 57: listarea doar a PID-urilor, comenzii de pornire, procentul de procesor acaparat și memorie utilizate pentru toate procesele sortate după procentul de procesor acaparat pentru procesele ce aparțin utilizatorului `student`

```
1  student@uso:~$ ps -e
2    PID TTY          TIME CMD
3      1 ?        00:00:05 systemd
4      2 ?        00:00:00 kthreadd
5      4 ?        00:00:00 kworker/0:0H
6      6 ?        00:00:00 mm_percpu_wq
7  [...]
8  student@uso:~$ ps -e --no-header
9      1 ?        00:00:05 systemd
10     2 ?        00:00:00 kthreadd
11     4 ?        00:00:00 kworker/0:0H
12     6 ?        00:00:00 mm_percpu_wq
13     7 ?        00:00:03 ksoftirqd/0
14 [...]
15 student@uso:~$ ps -e -f
16 UID        PID  PPID  C STIME TTY          TIME CMD
```

```
root         1     0  0 01:04 ?        00:00:05 /sbin/init splash
root         2     0  0 01:04 ?        00:00:00 [kthreadd]
root         4     2  0 01:04 ?        00:00:00 [kworker/0:0H]
root         6     2  0 01:04 ?        00:00:00 [mm_percpu_wq]
[...]
student@uso:~$ ps -u student
  PID TTY          TIME CMD
 1018 ?        00:00:01 systemd
 1019 ?        00:00:00 (sd-pam)
 1032 ?        00:00:00 gnome-keyring-d
 1036 tty1     00:00:00 gdm-x-session
[...]
student@uso:~$ ps -N -u student
  PID TTY          TIME CMD
    1 ?        00:00:05 systemd
    2 ?        00:00:00 kthreadd
    4 ?        00:00:00 kworker/0:0H
    6 ?        00:00:00 mm_percpu_wq
[...]
student@uso:~$ ps -C VBoxClient
  PID TTY          TIME CMD
 1148 ?        00:00:00 VBoxClient
 1149 ?        00:00:00 VBoxClient
 1159 ?        00:00:00 VBoxClient
 1160 ?        00:00:00 VBoxClient
[...]
student@uso:~$ ps -e -o pid,cmd,%cpu,rss
  PID CMD                         %CPU   RSS
    1 /sbin/init splash            0.0  7292
    2 [kthreadd]                   0.0     0
    4 [kworker/0:0H]               0.0     0
    6 [mm_percpu_wq]               0.0     0
[...]
student@uso:~$ ps -u student -o pid,cmd,%cpu,rss
  PID CMD                         %CPU   RSS
 1018 /lib/systemd/systemd --user  0.0  5552
 1019 (sd-pam)                     0.0   260
 1032 /usr/bin/gnome-keyring-daem  0.0  4608
 1036 /usr/lib/gdm3/gdm-x-session  0.0  3928
[...]
student@uso:~$ ps -u student -o pid,cmd,%cpu,rss --sort -%cpu
  PID CMD                         %CPU   RSS
 1221 /usr/bin/gnome-shell         0.4 292668
 8637 /usr/bin/python3 /usr/bin/u  0.4 80232
 2637 /usr/lib/firefox/firefox -n  0.2 264472
 2717 /usr/lib/firefox/firefox -c  0.2 115660
[...]
```

Listing 4.4: Folosirea comenzii ps

Dacă la un moment dat avem nevoie de PID-urile proceselor care au un anumit program ca imagine, vom folosi comanda:

```
student@uso:~$ ps -o pid -C VBoxClient
  PID
 1148
 1149
 1159
 1160
[...]
```

La fel, dacă avem nevoie de PID-urile proceselor unui anumit utilizator, vom folosi comanda:

```
student@uso:~$ ps -o pid -u student
  PID
 1018
 1019
 1032
 1036
[...]
```

Observăm că avem inclusiv antetul afișării, deși ne dorim doar PID-urile. Pentru aceasta, folosim caracterul = după numele argumentului `pid`, în forma `pid=` care dezactivează afișarea header-ului:

```
student@uso:~$ ps -o pid= -C VBoxClient
 1148
 1149
 1159
 1160
[...]
student@uso:~$ ps -o pid= -u student
 1018
 1019
 1032
 1036
[...]
```

De avut în vedere că același lucru poate fi obținut, mai simplu, folosind comanda **`pgrep`**:

```
student@uso:~$ pgrep VBoxClient
1148
1149
1159
1160
[...]
student@uso:~$ pgrep -u student
1018
1019
1032
1036
[...]
```

Atunci când dorim să obținem doar PID-urile anumitor procese, este mai simplu să folosim **`pgrep`** în loc de **`ps`**.

4.2.3 Starea proceselor

Într-unul din exemplele de folosire a utilitarului **`ps`** de mai sus am vorbit despre starea unui proces. Un proces are o stare care arată dacă acesta rulează sau nu pe procesor.

Un proces are nevoie de unul sau mai multe procesoare pentru a rula. Întrucât, de cele mai multe ori, sunt mai multe procese decât procesoare, nu toate procesele pot rula. Astfel unele procese rulează pe procesor, altele așteaptă să ruleze pe procesor; când un proces ajunge să ruleze pe un procesor spunem că este planificat (*scheduled*) să ruleze pa acel procesor. Iar unele procese pot fi blocate (*sleeping*) în așteptarea unei

operații de input/output. Simplificat, avem precizate stările și tranzițiile între stările unui proces în Figura 4.6.

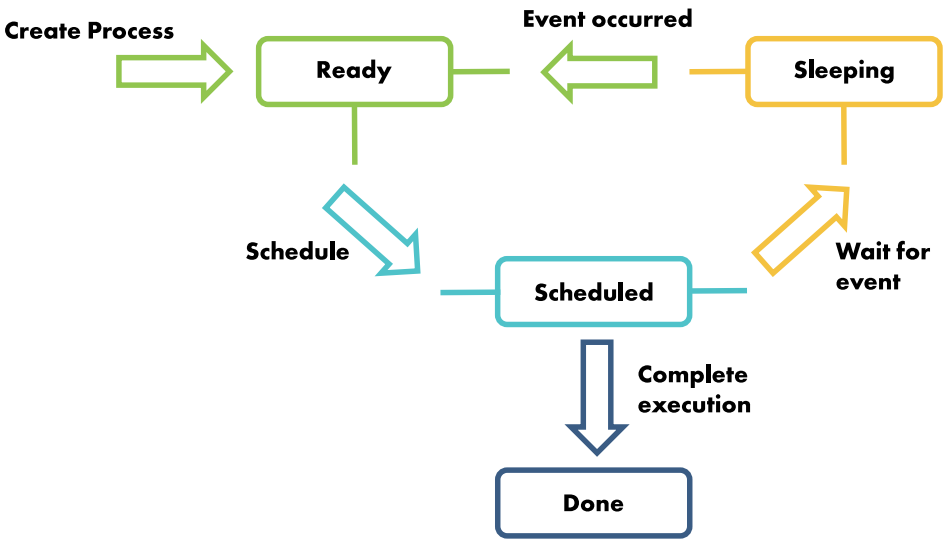


Figura 4.6: Starea proceselor

Putem investiga starea proceselor cu utilitare de urmărire și monitorizare. Acest lucru este util pentru a vedea dacă un proces este blocat și pentru ca apoi să investigam cauza pentru care este blocat. Sau să vedem cât de multe procese sunt active, gata să ruleze. În Listing 4.5 am afișat PID-ul, imaginea, starea, timpul de rulare și procentul curent de procesor ocupat pentru toate procesele din sistem, sortate în ordinea inversă a procentului de procesor ocupat. În rezultatul comenzii **`ps`**, `R` înseamnă *runnable* (nu *running*), adică fie rulează atunci pe procesor (starea *running*) fie poate fi pregătit să ruleze (starea *ready*).

```
1 student@uso:~$ ps -e -o pid,cmd,state,time,%cpu --sort -%cpu
2   PID CMD                         S     TIME %CPU
3  8637 /usr/bin/python3 /usr/bin/u S 00:00:22  3.6
4  9042 /usr/lib/snapd/snapd        S 00:00:05  1.2
5  1221 /usr/bin/gnome-shell        S 00:04:03  0.4
6  2717 /usr/lib/firefox/firefox -c S 00:02:05  0.3
7  2637 /usr/lib/firefox/firefox -n S 00:02:01  0.2
```

Listing 4.5: Monitorizarea proceselor folosind ps

Un sistem este cu atât mai încărcat cu cât are mai multe procese gata să ruleze, dar care nu au fost planificate. Aceste procese se cheamă *ready* sau *runnable* așa cum sunt prezente în Figura 4.6. Când multe procese sunt *ready* înseamnă că vor acapara un procesor imediat ce acesta devine disponibil și vor ține procesorul și sistemul încărcat.

Noțiunea de încărcare a unui sistem (numită și *load*) este dată de numărul de procese *ready*. Utilitarul **`uptime`** ne afișează încărcarea unui sistem în ultimul minut, în ultimele 5 minute și în ultimele 15 minute, ca în exemplul de mai jos:

```
1 student@uso:~$ uptime
2 13:48:40  up 9 days 16:13,  5 users,  load average: 1.61, 2.05, 2.81
```

Încărcarea unui sistem (*load average*) este deci `1.61` (în ultimul minut), `2.05` (în ultimele 5 minute), `2.81` în ultimele 15 minute. Valoarea încărcării este corelată cu

numărul de procese în starea *ready*: pregătite de execuție dar care încă nu pot rula pentru că procesoarele sistemului sunt ocupate.

Utilitarul **`top`** afișează, în partea superioară, informații despre încărcarea sistemului:

```
top - 13:58:20 up 14:02,  2 users,  load average: 0,71, 1,31, 0,92
Tasks: 190 total,   1 running, 156 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1,3 us,  0,3 sy,  0,2 ni, 97,9 id,  0,3 wa,  0,0 hi,  0,0 si,
0,0 st
KiB Mem :  2041312 total,    70676 free,  1287316 used,   683320 buff/
cache
KiB Swap:   777300 total,   699476 free,    77824 used.   531880 avail
Mem
```

4.2.4 Prioritatea proceselor

Procesele concurează la folosirea procesorului. Sistemul de operare planifică un proces *ready* pe procesor ținând cont de prioritatea sa. Procesele cu prioritate mai bună sunt planificate mai des și rulează mai mult timp pe procesor. Prioritatea unui proces este afectată de comportamentul acestuia (procesele mai „flămânde” primesc o prioritate mai slabă) și poate fi afectată de utilizator.

În Windows, utilizatorul poate afecta prioritatea unui proces folosind Task Manager, așa cum apare în Figura 4.7.

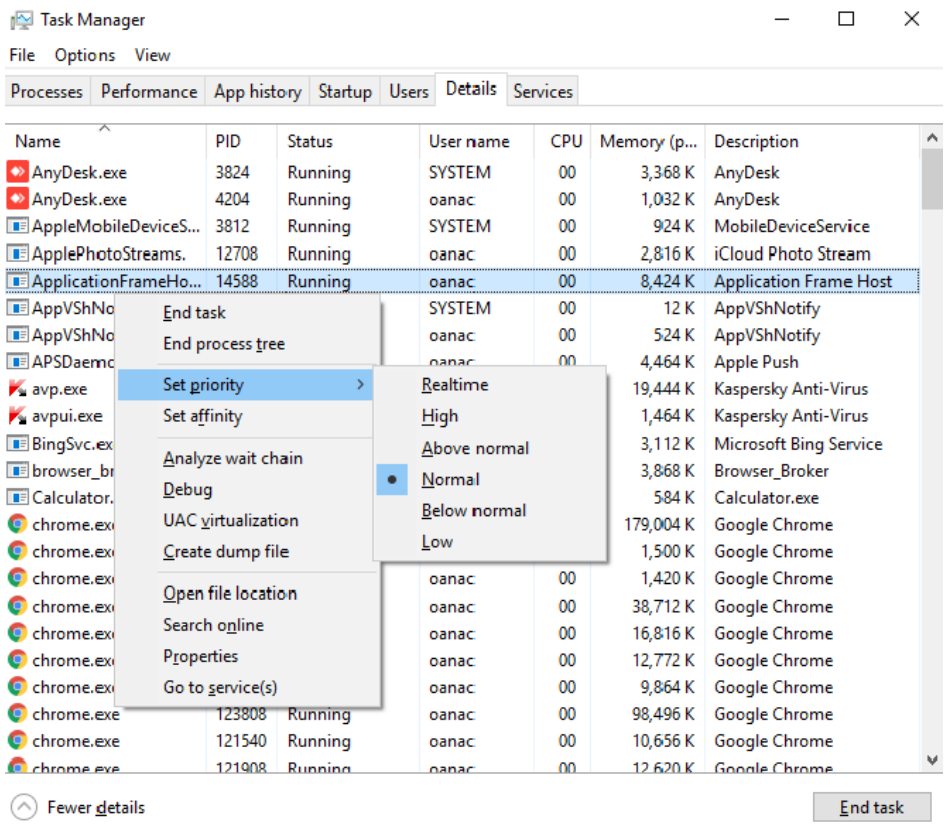


Figura 4.7: Priorități de procese în Windows

În Linux, putem modifica prioritatea unui proces prin schimbarea atributului `nice` al procesului. Valoarea `nice` arată cât de „drăguț” este acel proces cu alte procese. O

valoare mai mare a `nice` înseamnă că procesul este mai drăguț, deci lasă alte procese să fie planificate; o valoare mai mică a `nice` înseamnă că procesul nu este drăguț, deci „va lua fața" altor procese. Aceste lucru înseamnă că o valore `nice` mică înseamnă un proces cu prioritate mai bună, iar o valoare `nice` mare înseamnă un proces cu prioritate mai slabă.

> **Exprimări legate de prioritatea proceselor:** Din cauză că există această inconsecvență între valoarea `nice` și prioritate folosim exprimarea „prioritate mai bună" și „prioritate mai slabă". Folosirea exprimării „prioritate mai mare" și „prioritate mai mică" ar putea produce confuzie între prioritate și valoarea `nice`.

În mod implicit un proces pornește cu valoarea `nice` 0, o valoare neutră. Valoarea poate fi modificatăla pornirea procesului (*load-time*) sau în timp ce rulează (*run-time*). Un utilizator obișnuit (neprivilegiat) poate doar crește valoarea `nice` a unui proces pe care îl deține, adică în sensul slăbirii priorității procesului. Doar un utilizator privilegiat poate scădea valoarea `nice` a unui proces, adică în sensul îmbunătățirii priorității procesului.

Pentru a modifica prioritatea unui proces la pornire (*load-time*) folosim utilitarul **`nice`**. Pentru a modifica prioritatea unui proces la rulare (*run-time*) folosim utilitarul **`renice`**. Listing 4.6 conține exemple de comenzi care modifică prioritatea unui proces; după fiecare comandă **`nice`** / **`renice`** folosim **`ps`** pentru a vedea acum noua prioritate a procesului.

În Listing 4.6 am folosit comanda **`renice`** pentru a modifica valoarea `nice` a unui proces. Comanda poate fi folosită de un utilizator neprivilegiat doar pentru a crește, nu și pentru a scădea valoarea `nice` a unui proces. Pentru scădea valoarea `nice` a unui proces folosim contul privilegiat, cu ajutorul comenzii **`sudo`**. Similar, comanda **`nice`** poate porni un proces cu valoarea `nice` modificată.

```
1  student@uso:~$ sleep 1000 &
2  [1] 10384
3  student@uso:~$ ps -C sleep -o pid,uid,nice,cmd
4    PID   UID  NI CMD
5  10384  1000   0 sleep 1000
6  student@uso:~$ renice +10 10384
7  10384 (process ID) old priority 0, new priority 10
8  student@uso:~$ ps -C sleep -o pid,uid,nice,cmd
9    PID   UID  NI CMD
10 10384  1000  10 sleep 1000
11 student@uso:~$ renice +0 10384
12 renice: failed to set priority for 10384 (process ID): Permission denied
13
14 student@uso:~$ sudo renice -20 10384
15 10384 (process ID) old priority 0, new priority -20
16 student@uso:~$ ps -C sleep -o pid,uid,nice,cmd
17   PID   UID  NI CMD
18 10384  1000 -20 sleep 1000
19
20 student@uso:~$ nice -n +10 sleep 1000 &
21 [1] 10410
22 student@uso:~$ ps -C sleep -o pid,uid,nice,cmd
23   PID   UID  NI CMD
24 10410  1000  10 sleep 1000
25
26 student@uso:~$ nice -n -10 sleep 1000 &
```

```
[1] 10412
student@uso:~$ nice: cannot set niceness: Permission denied

student@uso:~$ sudo nice -n -10 sleep 1000 &
[1] 10413
student@uso:~$ ps -C sleep -o pid,uid,nice,cmd
  PID   UID  NI CMD
10414     0 -10 sleep 1000
```

Listing 4.6: Modificarea priorității unui process (nice)

Prioritatea unui proces este principalul mijloc prin care un proces poate folosi mai mult sau mai puțin procesorul unui sistem. Dacă vrem ca un proces să folosească mai mult resursele sistemului, vom îmbunătăți prioritatea acestuia; dacă un proces utilizează prea mult resursele sistemului (*resource hog*) și vrem să-l „temperăm”, îi slăbim prioritatea; la nevoie un astfel de proces este omorât.

4.3 Ierarhia de procese

Un proces este creat la comanda utilizatorului sau la un eveniment de declanșare (o nouă conexiune în rețea, expirarea unui interval de timp). Așa cum am precizat un proces este creat la **load-time** dintr-un fișier executabil (numit imagine de proces); după creare procesului îi sunt alocate resurse (precum timp de procesor și zone de memorie) și rulează (**run-time**).

> Crearea unui proces dintr-un executabil se mai numește *loading* (încărcare executabilului în memorie), iar momentul creării se numește **load-time**.

Un proces este creat prin intermediul unui alt proces, numit **proces părinte** (*parent process*). Un proces poate crea oricâte **procese copil** (*child process*), în limita resurselor sistemului. Un proces poate avea, însă, un singur proces părinte. Pentru crearea unui proces, procesul părinte folosește o interfață specifică a sistemului de operare: folosește grupul de apeluri `fork()` și `exec()` în Linux și `CreateProcess()` pe Windows. Nu insistăm pe aceste apeluri în această carte; câteva detalii găsiți în Secțiunea 7.1.3.2. Vizual, crearea unui proces este indicată în Figura 4.8.

În general, procesul care creează un nou process este un shell. Shellul poate fi grafic (precum **Windows Explorer**) sau poate fi în linia de comandă (precum **Bash**). De exemplu, în mediul grafic, atunci când folosim dublu click pe o icoană de pe ecran, pornim un proces; acel proces este creat de shellul grafic, care define procesul părinte al noului proces. Altfel, în linia de comandă, shellul creează un proces nou la introducerea unei comenzi. Prezentăm mai multe detalii în Capitolul 7.

Legătura proces părinte - proces copil este utilă pentru a afla informații legate de încheierea unui proces. Un proces își poate încheia execuția în mai multe moduri: ajunge la sfârșitul zonei de execuție, este omorât de alt proces, execută o acțiune nevalidă. Procesul părinte este cel care poate furniza informații despre condițiile de încheiere ale unui proces copil.

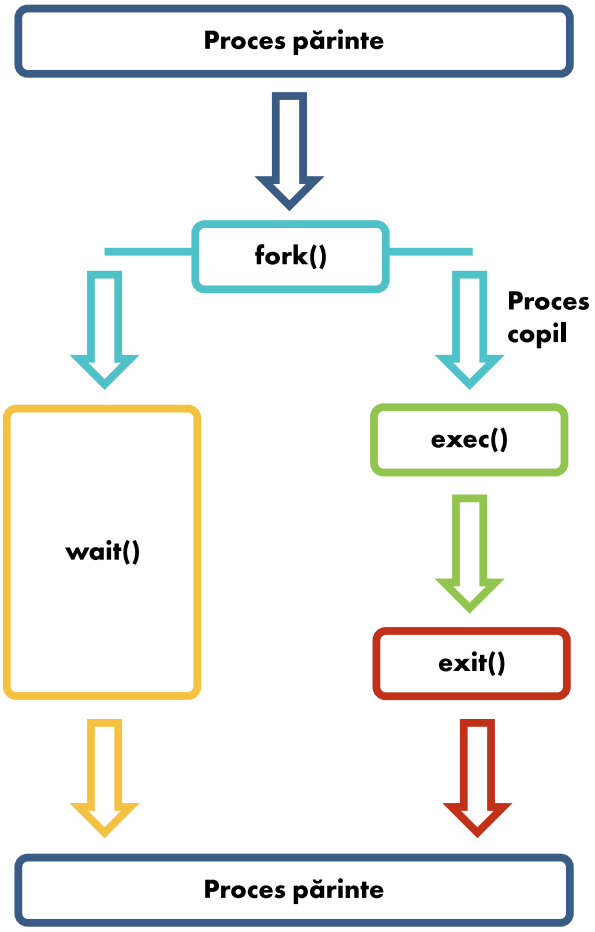


Figura 4.8: Crearea unui proces

4.3.1 Ierarhia de procese în Linux/Unix

În Linux, întrucât un proces creează alt proces care la rândul său poate crea alt proces, procesele sunt organizate într-o ierarhie de procese, într-o aborescență. De exemplu la o rulare a comenzii **pstree** putem vedea ierarhia de procese în Linux ca în Listing 4.7. Cu opțiunea `-p`, comanda **pstree** afișează și PID-ul proceselor.

```
student@uso:~$ pstree -A
systemd-+-ModemManager---2*[{ModemManager}]
        |-NetworkManager-+-2*[dhclient]
        |                `-2*[{NetworkManager}]
        |-2*[VBoxClient---VBoxClient---{VBoxClient}]
        |-VBoxClient---VBoxClient
        |-VBoxClient---VBoxClient---2*[{VBoxClient}]
        |-VBoxService---7*[{VBoxService}]
        |-accounts-daemon---2*[{accounts-daemon}]
[...]
student@uso:~$ pstree -A -p
systemd(1)-+-ModemManager(447)-+-{ModemManager}(466)
           |                   `-{ModemManager}(475)
           |-NetworkManager(473)-+-dhclient(9735)
           |                     |-dhclient(9769)
           |                     |-{NetworkManager}(573)
           |                     `-{NetworkManager}(581)
           |-VBoxClient(1148)---VBoxClient(1149)---{VBoxClient}(1155)
           |-VBoxClient(1159)---VBoxClient(1160)
           |-VBoxClient(1166)---VBoxClient(1167)---{VBoxClient}(1168)
           |-VBoxClient(1172)---VBoxClient(1173)-+-{VBoxClient}(1174)
           |                                     `-{VBoxClient}(1175)
[...]
```

Listing 4.7: Ierarhia de procese în Linux

Observăm că în vârful ierarhiei, în rădăcina proceselor, se găsește procesul **systemd**, procesul cu PID-ul 1. **systemd** este o implementare de proces **init**. În mod generic, spunem că primul proces al sistemului, rădăcina ierarhiei de procese, este **init**, cu diferite forme de implementare. Implementarea folosită curent în majoritatea distribuțiilor Linux este **systemd**.

init este primul proces al sistemului, procesul care pornește serviciile sistemului și procesele de bază. În Listing 4.7 observăm că un proces are un singur proces părinte și oricâte procese copil. Ierarhia nu are foarte multe niveluri. În general procesul init creează servicii de bază și shelluri, iar shellurile creează alte procese. Detalii despre procesul init prezentăm în Secțiunea 4.3.4

În mod obișnuit, un proces dat are, de la creare până la încheiere, un proces părinte. Se poate întâmpla însă ca un proces să rămână „orfan” adică procesul părinte să își încheie execuția înaintea sa. În acest caz, în Linux, procesul init „adoptă” procesul rămas orfan și devine noul proces părinte. Vom prezenta detalii în Secțiunea 4.3.4.

4.3.2 Ierarhia de procese în Windows

În Windows, în mod similar, un proces creează alt proces. Cu toate acestea ierarhia de procese în Windows este o ierarhie cu legături mai slabe. În vreme ce în Linux, un

proces părinte are privilegii specifice de comunicare cu un proces copil (de exemplu comunicare prin operatorul *pipe*), în Windows un proces poate comunica în același mod cu un proces părinte și cu un proces cu care nu este conectat ierarhic. Un proces are o referință (*handle*) către un proces pe care l-a creat, dar acea referință poate fi transferată altui proces, afectând ierarhia. Mai mult, în Windows orice proces poate obține informații despre încheierea unui alt proces, spre deosebire de Linux unde doar procesul părinte poate obține informații.

O imagine a ierarhiei de procese în Windows putem obține folosind utilitarul Process Explorer, așa cum vedem în Figura 4.9

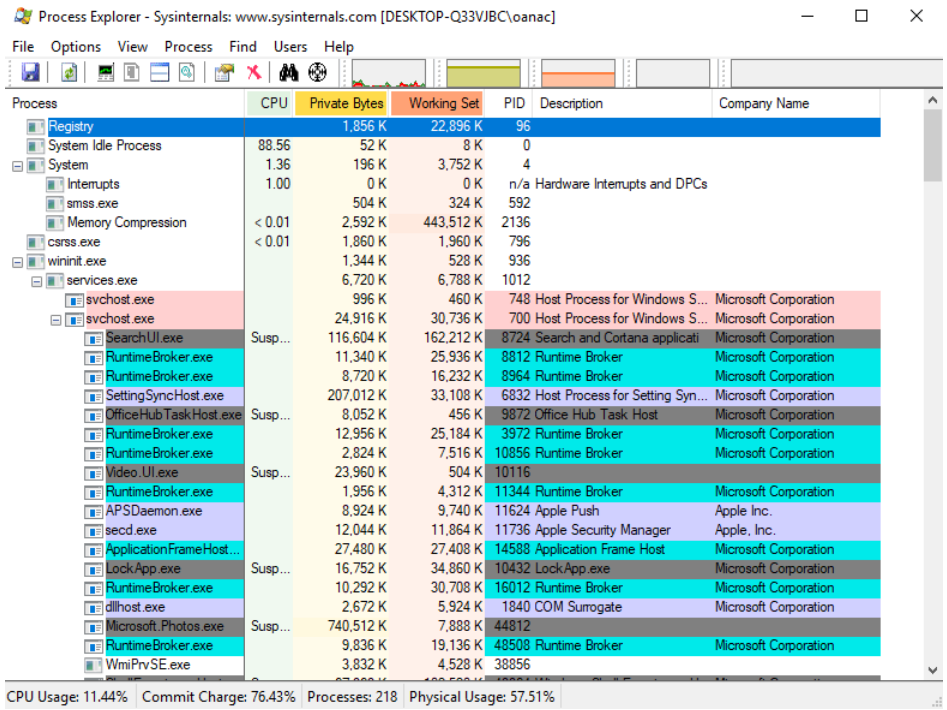


Figura 4.9: Vizualizare ierarhie de procese folosind Process Explorer

Shellul Windows este un shell grafic reprezentat de procesul Explorer, prezent în Figura 4.9. Acest proces creează aplicațiile/procesele pornite în mediul grafic în Windows.

4.3.3 Foreground și background

Un proces shell în linia de comandă creează procese în momentul introducerii de comenzi din partea utilizatorului. Procesul nou creat și shellul folosesc simultan terminalul, adică modul în care utilizatorul poate transmite informații la ieșirea standard și modul în care se afișează mesaje la ieșirea standard. Dacă atât procesul nou creat cât și shellul afișează informații la ieșirea standard, aceste afirmații vor fi agregate și afișate la terminal. Dacă însă trimitem informații la intrarea standard, prin terminal, doar procesul nou creat la va accesa.

Spunem că, în cadrul terminalului, avem un singur proces care deține controlul intrării standard, adică un singur proces care este în **foreground**. În general, modul de funcționare a shellului, detaliat în Secțiunea 7.1, este:

1. shellul citește de la intrarea standard (din terminal) comenzi și opțiuni ale utilizatorului
2. shellul creează un nou proces pornind de la comanda introdusă
3. procesul nou creat rulează în **foreground** și are controlul terminalului (și a intrării acestuia)
4. procesul nou creat își încheie execuția; shellul, în calitate de proces părinte, reține informații despre încheierea execuției
5. shellul redobândește controlul terminalului și reîncepem acțiunea de la punctul 1

Acest mod de funcționare devine problematic în momentul în care procesul nou creat nu își încheie rapid execuția: fie rulează mai mult, fie este o aplicație grafică folosită interactiv de utilizartor. În această situație, procesul nou creat „acaparează" terminalul și împiedică shellul să citească noi comenzi și să creeze noi proces. De exemplu dacă introducem comanda **`firefox`**, shellul va crea un proces Firefox care va acapara terminalul.

Pentru a trece de această problemă și pentru a permite shellului rularea continuă de comenzi și crearea de mai multe procese, există un mod de folosire a terminalului numit **background**. Background este modul în care un proces cedează accesul la intrarea terminalului curent; procesul poate rula în continuare dar nu mai are acces la informații furnizate de utilizator.

Întrucât un singur proces poate avea acces la intrarea terminalului, putem avea un singur proces în foreground. Putem avea însă oricâte procese în background. În background proceswle se pot găsi în două stări: rulând (*running*) sau suspendate (*stopped*, *suspeneded*, *paused*). Decizia de a suspenda și de a scoate un proces din starea suspendat aparține utilzatorului. Un proces suspendat nu se poate găsi în foreground, ci doar în background.

Un proces poate rula de la început în background sau poate fi transferat în background după pornire. De exemplu, pentru a rula un proces în background folosim operatorul `&` imediat după comanda aferentă și parametrii acesteia. În Listing 4.8, în prima rulare a comenzii, procesul Firefox a fost pornit în foreground, iar apoi, folosind operatorul `&`, procesul Firefox a fost pornit în background. Comanda **`jobs`** este folosită pentru a afișa job-urile din shellul curent, adică procesele care se găsesc în background.

```
1 student@uso:~$ firefox
2 student@uso:~$ firefox &
3 [1] 10533
4 student@uso:~$ jobs
5 [1]+  Running                 firefox &
```

Listing 4.8: Pornirea unui process în background

În cazul rulării unei comenzi în mod simplu, fără operatorul `&`, procesul pornește în foreground. Poate fi adus ulterior în background folosind combinația de taste `Ctrl+z`. Această combinație de taste are ca efect suspendarea procesului. Întrucât procesul nu poate rula suspendat în foreground, este trecut în background, ca în Listing 4.9.

```
1 student@uso:~$ sleep 100
2 ^Z
3 [1]+  Stopped                 sleep 100
```

```
4 student@uso:~$ jobs
5 [1]+  Stopped                 sleep 100
```

Listing 4.9: Suspendarea unui process în background

Observăm așadar că folosirea operatorului `&` duce un proces în background în starea rulând, pe când folosirea combinației de taste `Ctrl+z` duce un proces în background în starea suspendat. O dată dus un proces în background acesta poate fi readus în foreground folosind comanda **`fg`**, ca în Listing 4.10. În Listing 4.10 rulăm o comandă obișnuit și procesul rezultat rulează în background. Ulterior, folosim `Ctrl+z` pentru a plasa procesul în background. Apoi folosim **`fg`** pentru a-l readuce în foreground. În momentul în care este în background, procesul apare ca job, afișat în rezultatul rulării comenzii **`jobs`**. Când un proces este în background, acesta acaparează terminalul; în acest caz, alte informații trimise la intrare (precum introducerea comanda **`ls`**) nu sunt interpretate de shell.

```
1  student@uso:~$ sleep 100
2  ^Z
3  [1]+  Stopped                 sleep 100
4  student@uso:~$ jobs
5  [1]+  Stopped                 sleep 100
6  student@uso:~$ fg
7  sleep 100
8  ls
9  ^C
10 student@uso:~$ jobs
11 student@uso:~$
```

Listing 4.10: Readucerea unui process în foreground

Un proces dus în background în starea suspendat folosind combinația de taste `Ctrl+z` poate fi apoi trecut în starea *running*, tot în background. Facem acest lucru folosind comanda **`bg`**, ca în Listing 4.11. În Listing 4.11 se observă că, după folosirea combinației de taste `Ctrl+z`, procesul ajunge în starea `Stopped` în background. Apoi, prin folosirea comenzii `bg` acesta ajunge în starea `Running` în background.

```
1 student@uso:~$ sleep 100
2 ^Z
3 [1]+  Stopped                 sleep 100
4 student@uso:~$ jobs
5 [1]+  Stopped                 sleep 100
6 student@uso:~$ bg
7 [1]+ sleep 100 &
8 student@uso:~$ jobs
9 [1]+  Running                 sleep 100 &
```

Listing 4.11: Trecerea unui proces în starea running în background

Sumarizând, Figura 4.10 prezintă modul în care poate rula un proces în background/foreground: ce comenzi și operatori sunt folosiți în fiecare caz, ce stări au procesele și cum pot tranzita între stări (*running*, *stopped*) și între moduri de rulare (foreground, background).

Am precizat că mai multe procese pot rula în background. Procesele care rulează în background se numesc joburi și au un identificator de job pentru shellul curent. Dacă avem mai multe joburi dorim să controlăm starea unui job folosind comenzile **`fg`** și **`bg`**, atunci vom adăuga ca parametru către aceste comenzi identificatorul jobului, așa cum

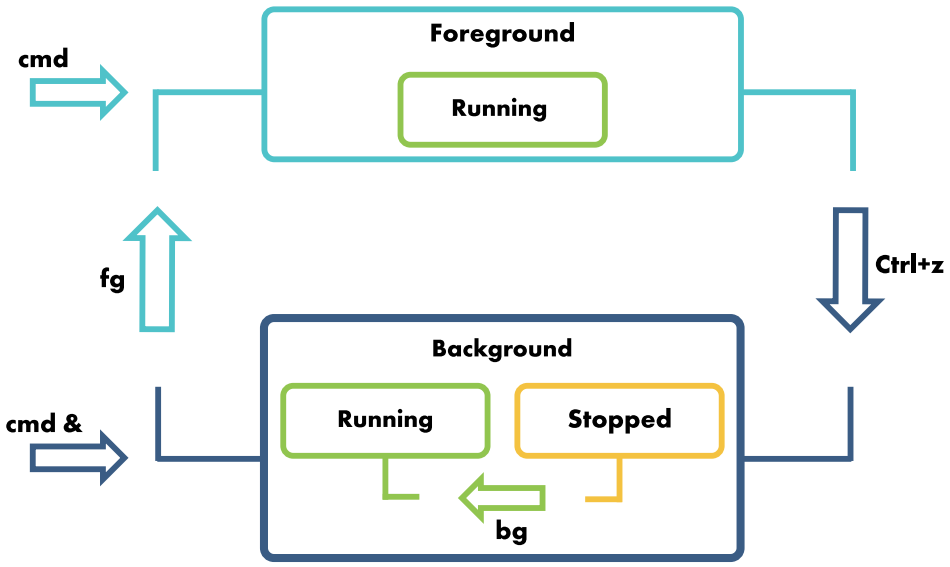


Figura 4.10: Foreground și background

facem în Listing 4.12. În Listing 4.12 am folosit construcțiile `%1` și `%2` pentru a opera joburile cu indexul 1 și 2 din background. Am trimis indexul ca argument comenzilor **`bg`** și **`fg`**.

```
student@uso:~$ sleep 100 &
[1] 10962
student@uso:~$ sleep 200
^Z
[2]+  Stopped                 sleep 200
student@uso:~$ jobs
[1]-  Running                 sleep 100 &
[2]+  Stopped                 sleep 200
student@uso:~$ bg %2
[2]+ sleep 200 &
student@uso:~$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
student@uso:~$ fg %2
sleep 200
^Z
[2]+  Stopped                 sleep 200
student@uso:~$ jobs
[1]-  Running                 sleep 100 &
[2]+  Stopped                 sleep 200
student@uso:~$ fg %1
sleep 100
^C
student@uso:~$ jobs
[2]+  Stopped                 sleep 200
```

Listing 4.12: Gestiunea job-urilor

Un scenariu util pentru trecerea unui proces în background este când pornim un proces GUI din shell și pierdem în shell accesul la terminal. De exemplu am pornit procesul Emacs (grafic) în foreground. Pentru a putea readuce shellului controlul terminalului, vom trece procesul Emacs în background (suspendat) folosind combinația de taste `Ctrl+z` și vom muta apoi procesul din starea supendat în starea running folosind comanda bg. Adică la fel în Listing 4.13.

```
student@uso:~$ emacs
^Z
[1]+  Stopped                 emacs
student@uso:~$ bg
[1]+ emacs &
student@uso:~$ jobs
[1]+  Running                 emacs &
```

Listing 4.13: Transferul unui proces GUI din foreground în background

În mod obișnuit, dacă un shell se închide (se rulează comanda **`exit`** sau combinația `Ctrl+d` sau este închisă fereastra terminalului în care rulează), procesele active în terminalul respectiv sunt, de asemenea, omorâte. Sunt omorâte procesele aflate în background și, eventual, procesul aflat în foreground. Dacă dorim să menținem anumite procese active după încheierea procesului shell, avem opțiuni de lucru; vom discuta despre aceste opțiuni în Secțiunea 4.6.3.

4.3.4 Procesul init

După cum am indicat în Secțiunea 4.3.1, în vârful ierarhiei proceselor din Linux se găsește procesul **init**. Acesta este primul proces al sistemului și creatorul primelor procese. Serviciile de bază ale sistemului, shellurile inițiale, mediul grafic sunt pornite direct sau indirect din procesul init. Spunem că un sistem Linux a bootat în momentul creării procesului init. În această secțiune discutăm minimal despre init, cu accent pe rolul său în gestiunea proceselor sistemului. Detalii despre pornirea sistemului până la pornirea init, și detalii despre init și configurarea sa prezentăm în Capitolul 9.

Pe lângă rolul în pornirea proceselor inițiale, init are și rol în întreținerea sistemului prin adoptarea proceselor orfane. Un proces orfan este un proces al cărui proces părinte și-a încheiat execuția. Pentru a menține ierarhia de procese, acest proces trebuie să aibă un proces părinte; acest proces părinte este procesul init. În codul sursă de suport al cărții în directorul `code/04-process/code/parent-child/` este un exemplu în care creăm un proces părinte (implementat fișierul sursă C `parent.c`) care apoi creează un proces (implementat în fișierul sursă C `child.c`). Listing 4.14 conține o secvență în care rulăm programul ce creează procesul părinte, program care apoi creează procesul copil. În Listing 4.14 în prima parte am pornit procesul `parent` care apoi a pornit și procesul `child`. Apoi din alt terminal urmărim cele două procese și observăm că procesul copil (`child`) cu PID-ul `11305` are ca părinte procesul părinte (`parent`) cu PID-ul `11304`. Apoi omorâm procesul părinte și investigăm din nou procesul copil. Observăm că acum procesul părinte este procesul `init` cu PID-ul `1`.

```
# start process
student@uso:~/.../code/04-process/parent-child$ ./parent
parent: Parent process started. Creating child process ...
parent: Child process created. Now going to sleep ...
child: Child process started. Now going to sleep ...

# investigate process
student@uso:~$ ps -f -C parent
UID        PID  PPID  C STIME TTY          TIME CMD
student  11304 10765  0 11:04 pts/1    00:00:00 ./parent
student@uso:~$ ps -f -C parent,child
```

```
12 UID        PID  PPID  C STIME TTY          TIME CMD
13 student  11304 10765  0 11:04 pts/1    00:00:00 ./parent
14 student  11305 11304  0 11:04 pts/1    00:00:00 ./child
15 student@uso:~$ kill 11304
16 student@uso:~$ ps -f -C parent,child
17 UID        PID  PPID  C STIME TTY          TIME CMD
18 student  11305     1  0 11:04 pts/1    00:00:00 ./child
```

Listing 4.14: Adoptarea unui proces orfan de către proces init

Procesul init este un proces esențial în Linux, vârful ierarhiei de procese. De aceea, dacă dintr-un motiv sau altul procesul init moare sau este omorât, sistemul de operare devine neutilizabil.

Pentru a evita ca un singur proces să aibă rolurile procesului init, pe sistemele Ubuntu a apărut soluția folosirii mai multor procese init: câte un proces init pentru fiecare sesiune. Când o sesiune dată este creată, se creează un proces init dedicat acelei sesiuni, care va fi vârful ierarhiei de procese din acea sesiune. Acest proces init al sesiune adoptă procesele orfane din acea sesiune. Mai multe despre această caracteristică sunt prezentate în Secțiunea 13.5.1.

4.4 Procese și fișiere

Așa cum am precizat în Capitolul 2, fișierele sunt componenta cea mai vizibilă din sistemul de operare. Fișierele sunt folosite pentru a stoca informații care sunt ulterior accesate, prelucrate sau transferate.

Accesul de orice fel la fișiere se realizează prin intermediul unui proces. O comandă precum **`cat f.txt`** duce la crearea unui proces din executabilul `/bin/cat`, proces care apoi accesează fișierul `f.txt` și îi afișează conținutul. Procesele folosesc foarte des fișiere: pentru configurare, pentru a scrie informațiile generate, pentru a citi informații pe care să le prelucreze. De exemplu un proces al unui joc va citi datele despre scenariul care se va juca și informații despre un personaj dintr-un fișier. Un compilator va deschide fișiere cod sursă pentru a le compila și fișiere obiect unde să stocheze codul obiect generat.

Când un proces deschide un fișier (de exemplu folosind apelul `fopen()` în C), primește un handle către acel fișier. Un handle (reprezentat de `FILE *` în C) este interfața prin care procesul poate lucra cu fișierul. Acel handle este identificat de o intrare într-o tabelă a procesului numită **tabela descriptorilor de fișiere**. Aceasta este tabela prin care un proces gestionează fișierele pe care le folosește, așa cum este prezentat în Figura 4.11. Indexul intrării din acea tabelă este numit **descriptor de fișier** și este identificatorul folosirii fișierului în cadrul procesului.

4.4.1 Descriptori de fișiere

Un descriptor de fișiere identifică pentru un proces fișierul pe care îl operează. Pornind de la descriptorul de fișiere, procesul accesează intrarea aferentă în tabela de descriptori de fișiere, și folosind informațiile de acolo folosește fișierul. Un descriptor de fișier poate

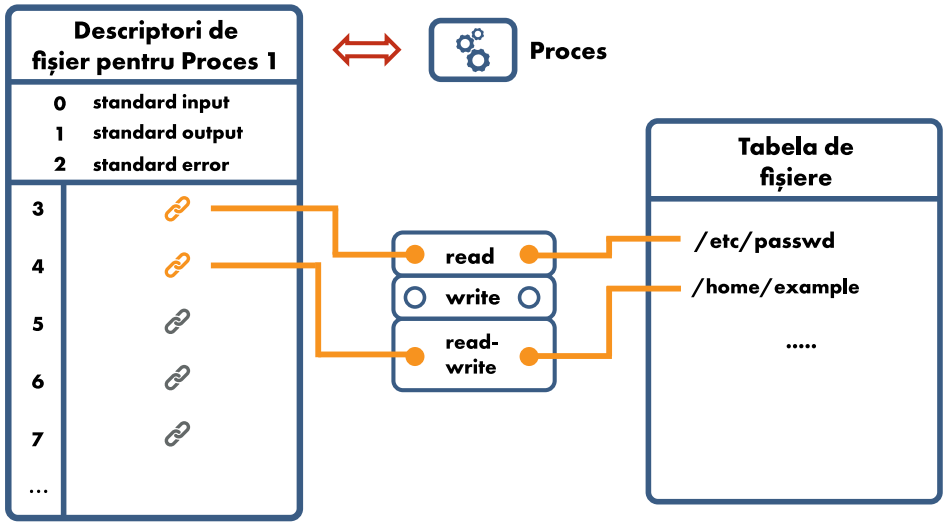


Figura 4.11: Tabela descriptorilor de fișiere a unui proces

referi și alte tipuri de intrări: poate referi un dispozitiv de terminal sau poate referi un socket pentru comunicare în rețea.

Tabela de descriptori de fișiere are o dimensiune limitată (în Linux 1024 de intrări) astfel că un proces nu poate deschide un număr infinit de fișiere. Acesta e un mod care să prevină folosirea abuzivă de resurse ale sistemului, deschizând foarte multe fișiere din cadrul unui proces; adică să prevină un atac de tipul *denial of service*. În momentul deschiderii unui fișier i se alocă o intrare din tabelă, iar fișierul va fi referit prin indexul din tabelă, acela devenind descriptorul său. În momentul închiderii unui fișier, intrarea respectivă va fi eliberată.

Pentru a investiga tabela de descriptori de fișiere a unui proces folosim utilitarul **`lsof`** ca în Listing 4.15. În Listing 4.15 `9585` este PID-ul procesului investigat, în cazul de față este un proces shell, creat din executabilul `/bin/bash`. Utilitarul `lsof` afișează mai multe informații, incluzând biblioteci încărcate. Aici sunt relevante ultimele intrări, care au pe coloana `FD` numere (`0u`, `1u`, `2u`, `255u`).

```
1  student@uso:~$ lsof -p 9585
2  COMMAND  PID    USER   FD   TYPE DEVICE SIZE/OFF   NODE NAME
3  bash    9585 student  cwd    DIR    8,1     4096 681167 /home/student
4  bash    9585 student  rtd    DIR    8,1     4096      2 /
5  bash    9585 student  txt    REG    8,1  1113504 786441 /bin/bash
6  bash    9585 student  mem    REG    8,1    47568 923818 /lib/x86_64-linux
   -gnu/libnss_files-2.27.so
7  bash    9585 student  mem    REG    8,1    97176 923812 /lib/x86_64-linux
   -gnu/libnsl-2.27.so
8  bash    9585 student  mem    REG    8,1    47576 923829 /lib/x86_64-linux
   -gnu/libnss_nis-2.27.so
9  bash    9585 student  mem    REG    8,1    39744 923814 /lib/x86_64-linux
   -gnu/libnss_compat-2.27.so
10 bash    9585 student  mem    REG    8,1 11861184 662217 /usr/lib/locale/
   locale-archive
11 bash    9585 student  mem    REG    8,1  2030544 923728 /lib/x86_64-linux
   -gnu/libc-2.27.so
12 bash    9585 student  mem    REG    8,1    14560 923751 /lib/x86_64-linux
   -gnu/libdl-2.27.so
13 bash    9585 student  mem    REG    8,1   170784 923886 /lib/x86_64-linux
   -gnu/libtinfo.so.5.9
```

```
14 bash    9585 student  mem    REG    8,1   170960 923700 /lib/x86_64-linux
   -gnu/ld-2.27.so
15 bash    9585 student  mem    REG    8,1    26376 790765 /usr/lib/x86_64-
   linux-gnu/gconv/gconv-modules.cache
16 bash    9585 student    0u   CHR  136,1      0t0      4 /dev/pts/1
17 bash    9585 student    1u   CHR  136,1      0t0      4 /dev/pts/1
18 bash    9585 student    2u   CHR  136,1      0t0      4 /dev/pts/1
19 bash    9585 student  255u   CHR  136,1      0t0      4 /dev/pts/1
```

Listing 4.15: Investigarea descriptorilor de fișiere pentru un process (lsof)

Primii 3 indecși din tabela de descriptori de fișiere (0, 1 și 2) sunt întotdeauna alocați pentru descriptorii standard de interacțiune cu procesul: intrarea standard (*standard input*), ieșirea standard (*standard output*) și ieșirea de eroare standard (*standard error*). Din acest motiv primul descriptor de fișier folosibil este descriptorul 3; în momentul în care un proces deschide primul său fișier, acestuia îi va fi alocat descriptorul 3. Astfel că primul fișier deschis va avea indexul 3. Alocarea și eliberarea indecșilor (adică descriptorilor de fișier) sunt descrise, respectiv, în Figura 4.12 și Figura 4.13.

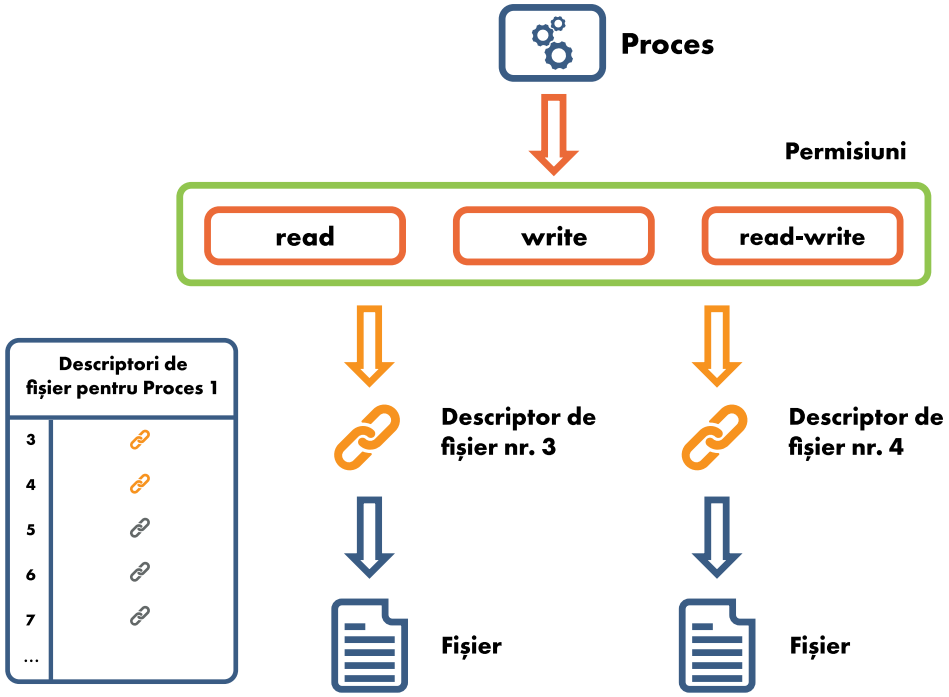


Figura 4.12: Alocarea unui descriptor de fișier

Fiecare proces are o tabelă de descriptori de fișier proprie. Astfel că o intrare 3 într-o tabelă va referi alt fișier decât o intare 3 din tabela altui proces. Descriptorii standard referă, în general, terminalul curent al procesului, care poate diferi între procese distincte.

Când folosim un editor, vom considera natural ca procesul corespunzător editorului să aibă o referință în tabela de descriptori pentru fișierul pe care îl editează. Adică dacă folosim comanda **`vim message.txt`** pentru a edita fișierul `message.txt` folosind editorul Vim, bănuim că procesul corespunzător va avea referință către fișier. Dacă investigăm folosind comanda **`lsof`**, obținem scenariul din Listing 4.16. În Listing 4.16 am pornit utilitarul Vim și apoi l-am trecut în background. Am investigat tabela sa de descriptori de fișiere. Indexul `3` este alocat unui fișier temporar de salvare a datelor în cazul în care aplicația are probleme; nu avem referință directă la fișierul `a.txt`.

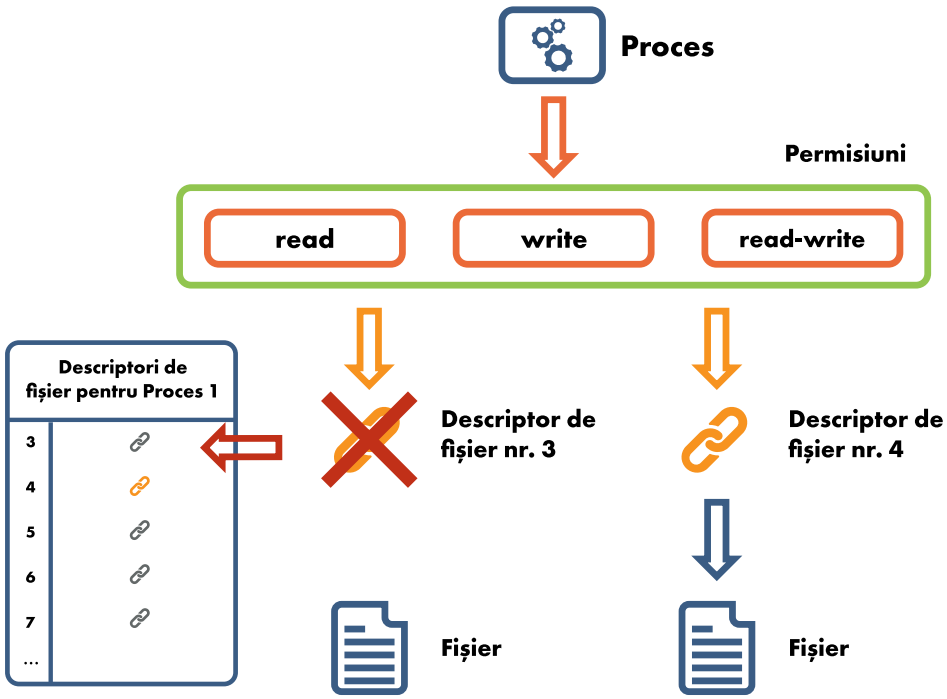


Figura 4.13: Eliberarea unui descriptor de fișier

```
1  student@uso:~$ vim a.txt
2
3  [1]+  Stopped                 vim a.txt
4  student@uso:~$ pidof vim
5  11414
6  student@uso:~$ lsof -p 11414
7  COMMAND   PID    USER   FD   TYPE DEVICE SIZE/OFF   NODE NAME
8  vim     11414 student  cwd    DIR    8,1     4096 681167 /home/student
9  vim     11414 student  rtd    DIR    8,1     4096      2 /
10 vim     11414 student  txt    REG    8,1  2880744 687896 /usr/bin/vim.nox
11 [...]
12 vim     11414 student    0u   CHR  136,2      0t0      5 /dev/pts/2
13 vim     11414 student    1u   CHR  136,2      0t0      5 /dev/pts/2
14 vim     11414 student    2u   CHR  136,2      0t0      5 /dev/pts/2
15 vim     11414 student    3u   REG    8,1     4096 680064 /home/student/.a
   .txt.swp
```

Listing 4.16: Investigarea descriptorilor de fisiere pentru un editor

În general editoarele nu folosesc referințe la fișiere; editoarele citesc conținutul fișierelor în memorie și acțiunile de editare ale utilizatorului se traduc în modificări în memorie, nu în fișierul de pe disc; doar la comenzi de salvare sunt deschise fișierele, scrise informațiile pe disc, iar apoi iar se închide referința la fișier. Acest lucru se întâmplă pentru a avea viteză sporită în lucrul cu editorul: discul este mult mai lent ca memoria și este mai eficient să fie folosită memoria în editare. Spunem că un editor creează un buffer de memorie pentru fiecare fișier deschis, buffer al cărui conținut este transferat periodic către fișier: fie la acțiuni explicite de salvare din partea utilizatorului fie, dacă este cazul, la acțiuni de salvare automată (*autosave*).

4.4.2 Redirectarea în/din fișiere

Am văzut că descriptorii de fișiere pot să refere terminalul, cum este cazul descriptorilor standard (0, 1 și 2) sau pot să refere fișiere. Legăturile descriptorilor standard cu terminalul nu sunt fixe, pot fi modificate. Modificarea referinței unui descriptor poartă numele de „redirectare" sau „redirectarea unui descriptor de fișier" (*redirecting a file descriptor*), așa cum am precizat în Secțiunea 2.4.

În mod uzual, redirectarea înseamnă modificarea descriptorilor standard să refere un fișier (sau altceva) în loc de terminal. De exemplu, redirectăm descriptorul 1 (ieșirea standard) al unui proces ca să nu mai refere terminalul ci fișierul `lshw.out`, cum este cazul comenzii din Listing 4.17. Această operație, de redirectare, este utilă pentru a putea salva ieșirea unei comenzi pentru prelucrarea ulterioară sau pentru a transfera un fișier către intrarea unei comenzi. Este cazul exemplului din Listing 4.17 unde dorim să salvăm ieșirea comenzii de investigare a hardware-ului unui sistem.

```
1 student@uso:~$ lshw > lshw.out
2 student@uso:~$ cat lshw.out
3 uso
4     description: Computer
5     width: 64 bits
6     capabilities: vsyscall32
7 [...]
```

Listing 4.17: Redirectarea ieșirii standard (stdout)

În general redirectările apar în comenzi înlănțuite, mai ample, numite one-linere. Într-un one liner rezultatul unei comenzi ajunge intrare pentru altă comandă, așa cum vom vedea în Secțiunea 4.5.4.

Putem urmări redirectarea descriptorilor cu ajutorul comenzii **`lsof`**. În Listing 4.18 toți descriptorii standard au fost redirecați în/din fișierele `out`, `err` și `in`, lucru precizat și de rezultatul rulării comenzii **`lsof`**.

```
1  student@uso:~$ sleep 100 > out 2> err < in
2  ^Z
3  [1]+  Stopped                 sleep 100 > out 2> err < in
4  student@uso:~$ pidof sleep
5  11432
6  student@uso:~$ lsof -p 11432
7  COMMAND   PID    USER   FD   TYPE DEVICE SIZE/OFF   NODE NAME
8  sleep   11432 student  cwd    DIR    8,1     4096 681167 /home/student
9  sleep   11432 student  rtd    DIR    8,1     4096      2 /
10 sleep   11432 student  txt    REG    8,1    35000 786587 /bin/sleep
11 sleep   11432 student  mem    REG    8,1 11861184 662217 /usr/lib/locale/
   locale-archive
12 sleep   11432 student  mem    REG    8,1  2030544 923728 /lib/x86_64-
   linux-gnu/libc-2.27.so
13 sleep   11432 student  mem    REG    8,1   170960 923700 /lib/x86_64-
   linux-gnu/ld-2.27.so
14 sleep   11432 student    0r   REG    8,1        0 680071 /home/student/in
15 sleep   11432 student    1w   REG    8,1        0 680069 /home/student/
   out
16 sleep   11432 student    2w   REG    8,1        0 680072 /home/student/
   err
```

Listing 4.18: Investigarea redirectării descriptorilor standard

4.5 Operații cu procese. Interacțiunea între procese

Un proces ia naștere dintr-un executabil (un program) prin intermediul unui alt proces, de obicei shellul, care este procesul părinte. În linia de comandă folosim comenzi care vor conduce la crearea de procese. Shellul folosește o interfață specifică a sistemului de operare pentru a crea un proces și devine părintele acelui proces. Apoi procesul rulează până la încheierea sa, folosind resurse puse la dispoziție de sistemul de operare.

4.5.1 Încheierea unui proces

Un proces își încheie execuția din diferite cauze. Există următoarele situații pentru încheierea unui proces:

- atingerea sfârșitului programului (încheierea funcției `main()` în C)
- apelul unei rutine de încheiere (precum funcția `exit()` în C)
- întâlnirea unei condiții neașteptate: accesarea unei zone nevalide de memorie, rularea unei intrucțiuni nevalide
- omorârea sa de un alt proces

Terminare (omorârea) unui proces se realizează de un alt proces care are permisiuni; în general, acest lucru înseamnă un proces care aparține aceluiași utilizator sau utilizatorului administrativ. În Windows terminarea unui proces are loc prin transmiterea unei notificări, numite excepție, de la un proces la altul. În mod similar, în Linux, terminaraea unui proces are loc prin transmiterea unui semnal către acesta. Primirea unui semnal duce, în multe situații (nu toate), la terminarea procesului. Transmiterea unui semnal se face prin intermediul utilitarelor din familia **`kill`**. În Listing 4.19 comanda **`kill`** trimite un semnal procesului **`sleep`**, care are PID-ul `11450` ducând la terminarea procesului. După folosirea comenzii **`kill`** procesul creat a fost omorât. Vom detalia semnale și folosire comenzii **`kill`** în Secțiunea 7.1.2.3.

```
1 student@uso:~$ sleep 100  # run command
2 Terminated # after kill command below
3
4 # kill process
5 student@uso:~$ pidof sleep
6 11450
7 student@uso:~$ kill 11450
```

Listing 4.19: Terminarea unui proces (kill)

Indiferent de caz, procesul terminat transmite către procesul părinte informații despre cum și-a încheiat execuția (*exit status*) și un cod de ieșire (*exit code*). În Linux, un cod de ieșire 0 înseamnă că programul s-a încheiat cu succes, în vreme ce alt cod de ieșire înseamnă încheiere cu insucces sau eroare. Listing 4.20 conține exemple de investigare a codului de eroare după încheierea unui proces. Construcția `$?` afișează în shell codul de ieșire al ultimei comenzi rulate. În exemplu de mai sus, atunci când comanda **`grep`** reușește (adică găsește șirul primit ca prim argument în fișierul primit ca al doilea argument), valoarea construcției `$?` este 0 (succes). Altfel este 1 (insucces).

```
1 student@uso:~$ grep student /etc/passwd > /dev/null
```

```
2 student@uso:~$ echo $?
3 0
4 student@uso:~$ grep profesor /etc/passwd > /dev/null
5 student@uso:~$ echo $?
6 1
```

Listing 4.20: Investigarea codului de ieșire al unui process

4.5.2 Semnale

Semnalele sunt, în Linux, mecanisme de interacțiune între procese. Un proces trimite un semnal unui alt proces pentru a-i semnaliza acestuia o situație deosebită. Pentru ca un proces să poată trimită semnale altui proces trebuie să aibă permisiuni adecvate; în mod normal acest lucru înseamnă să aparțină aceluiași utilizator ca cel al procesului care primește semnalul sau să aparțină utiizatorului `root`

Semnalele sunt, la bază, mecanisme de notificare din partea unui proces către altul, dar sunt folosite în multe situații pentru omorârea unui proces. De exemplu semnalul `SIGSTOP` suspendă un proces (și îl trece în background), iar semnalul `SIGTERM` îl omoară. În Listing 4.21 este pornită comanda **`sleep 100`** și apoi sunt trimise semnale procesului nou creat din alt terminal.

```
1  student@uso:~$ sleep 100
2
3  # after kill -STOP
4  [1]+  Stopped                 sleep 100
5  student@uso:~$ fg
6  sleep 100
7  # after kill -TERM
8  Terminated
9
10 # student@uso:~$ pidof sleep
11 11537
12 student@uso:~$ kill -STOP 11537
13 student@uso:~$ kill -TERM 11537
```

Listing 4.21: Trimiterea de semnale unui process

Un semnal este identificat de un număr sau de un nume. Comanda **`kill`** poate primi ca argument atât numele cât și numărul unui semnal. În Listing 4.22 afișăm numerele și numele semnalelor; apoi prezentăm trei comenzi care pot fi folosit echivalente pentru a transmite semnalul `SIGUSR1` unui proces.

```
1 student@uso:~$ kill -l
2  1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5)
  SIGTRAP
3  6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10)
  SIGUSR1
4 11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15)
  SIGTERM
5 16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20)
  SIGTSTP
6 21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25)
  SIGXFSZ
7 26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30)
  SIGPWR
```

```
8  31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37)
   SIGRTMIN+3
9  38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42)
   SIGRTMIN+8
10 43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47)
   SIGRTMIN+13
11 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52)
   SIGRTMAX-12
12 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57)
   SIGRTMAX-7
13 58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62)
   SIGRTMAX-2
14 63) SIGRTMAX-1  64) SIGRTMAX
15
16 student@uso:~$ kill -USR1 $(pidof sleep)
17 student@uso:~$ kill -10 $(pidof sleep)
18 student@uso:~$ kill -SIGUSR1 $(pidof sleep)
```

Listing 4.22: Identificarea semnalelor

Un semnal are asociată o acțiune pe care o întreprinde procesul care-l recepționează. Am văzut mai sus că această acțiune este suspendarea procesului în cazul semnalului `SIGSTOP` și omorârea procesului în cazul `SIGTERM`. Această acțiune implicită poate fi modificată programatic pentru un proces dat; nu vom discuta acest lucru aici, vedeți câteva informații în pagina de manual **`man 7 signal`**. De exemplu, un proces poate alege să ignore anumite semnale. Cu toate acestea, două semnale nu pot fi ignorate: semnalul `SIGSTOP` și semnalul `SIGKILL` care duc, respectiv, la suspendarea și omorârea procesului.

Dacă dorim să transmitem semnale mai multor procese putem folosi comanda **`pkill`**. Aceasta permite transmiterea unui semnale către procesele care au un anumit atribut; de exemplu către procesele care aparțin unui anumit utilizator. În Listing 4.23 cele trei comenzi trimit semnalul implicit (`SIGTERM`), respectiv tuturor proceselor **`sleep`**, tuturor proceselor utilizatorului `dan` și tuturor proceselor al căror proces părinte este `18291`.

```
1 student@uso:~$ pkill sleep
2 student@uso:~$ pkill -u dan
3 student@uso:~$ pkill -P 18291
```

Listing 4.23: Trimiterea de semnale mai multor procese (pkill)

Pe anumite platforme Unix există și comenzile **`killall`** sau **`skill`**. Acestea pot fi folosite, de asemenea, pentru a transmite semnale (sau a omorî) mai multe procese, cu opțiuni mai reduse decât cele ale comenzi **`pkill`**.

Un semnal este transmis de un proces altui proces pentru notificare sau de sistemul de operare altui proces pentru semnalizarea unei condiții speciale. Dacă dorim să trimitem un semnal unui proces din linia de comandă, avem două opțiuni: folosirea comenzilor din familia kill (**`kill`**, **`pkill`**, **`skill`**, **`killall`**) sau folosirea unor combinații de taste în terminalul curent. Există trei combinații de taste care transmit semnale procesului aflat în foreground în terminalul curent:

- `Ctrl+c`: transmite semnalul `SIGINT` (*interrupt*), cu rolul de a întrerupe execuția procesului curent
- `Ctrl+z`: transmite semnalul `SIGTSTP` (*terminal stop*), cu rolul de a suspenda execuția procesului curent și de a-l trece în background

- `Ctrl+textbackslash`: transmite semnalul `SIGQUIT`, cu rol similar semnalului `SIGINT`, de a întrerupe execuția procesului curent

Semnalele `SIGQUIT` și `SIGINT` au rol similar, diferența fiind că semnalul `SIGQUIT` (transmis folosind combinația de taste `Ctrl+\`) este mai puternic decât semnalul `SIGINT` (transmis folosind combinația de taste `Ctrl+c`). Mai puternic înseamnă că pot exista procese care ignoră semnalul `SIGINT` dar nu semnalul `SIGQUIT`; astfel că dacă folosim combinația de taste `Ctrl+c` și nu are loc terminarea procesului, vom folosi combinația de taste `Ctrl+\` cu șanse mai mari de reușită.

Pe scurt, atunci când vrem să omorâm un proces aflat în foreground folosim, în ordine:

- combinația de taste `Ctrl+c`
- combinația de taste `Ctrl+\`
- trimiterea semnalului `SIGKILL` din alt terminal

Adică pornim de la prima opțiune și, dacă aceea nu merge, încercăm pe a doua și apoi pe treia.

> **Folosire `Ctrl+c`:** Rețineți să folosiți `Ctrl+c` (sau `Ctrl+\`) pentru a omorî un proces aflat în foreground.
> Nu folosiți `Ctrl+z` pentru că atunci procesul ajunge în background și rămâne în viață, consumând resurse ale sistemului.

4.5.3 Înlănțuirea comenzilor

În unele situații dorim să rulăm mai multe procese unul după altul și să folosim o singură linie de comandă. De exemplu să compilăm o aplicație și apoi să o instalăm și apoi să o rulăm. Pentru aceasta, shellul ne pune la dispoziție operatori de înlănțuire de comenzi.

Cel mai simplu operator este cel de înlănțuire necondiționată (`;` , punct și virgulă). Acesta rulează două comenzi una după alta indiferent de codul de ieșire. Alți doi operatori sunt cei de înlănțuire condiționată (`||` și `&&`), care condiționează rularea celei de-a doua comenzi de codul de ieșire al primei. Astfel:

- **`comm1 || comm2`** rulează comanda **`comm2`** doar dacă **`comm1`** s-a încheiat cu insuccess
- **`comm1 && comm2`** rulează comanda **`comm2`** doar dacă **`comm1`** s-a încheiat cu succes

În exemplul de mai jos se rulează executabilul `main_args` din directorul curent doar dacă a reușit comanda de compilare `make`:

```
1 student@uso:~/.../code/04-process/main-args$ make && ./main_args
```

4.5.4 Comunicarea prin pipe-uri

Atunci când înlănțuim comenzi, urmărim, de obicei, transferul de informație de la o comandă la alta: rezultatul unei comenzi să fie folosit de altă comandă. Transferul de

informație se poate face în mod simplu, printr-un fișier: o comandă redirectează ieșirea într-un fișier iar altă comandă redirectează intrarea din acel fișier, ca în Listing 4.24. În Listing 4.24 vrem să aflăm câte procese sunt active în sistem numărând liniile afișate de comanda **ps**; număratul este realizat de comanda **wc -l**. Pentru aceasta redirectăm ieșirea comenzii **ps** în fișierul `out` pe care apoi îl redirectăm la intrarea comenzii **wc**.

```
1 student@uso:~$ ps -e --no-header > out
2 student@uso:~$ wc -l < out
3 191
```

Listing 4.24: Transferul datelor de la o comandă la altă comandă prin redirectare

Același lucru poate fi obținut cu ajutorul operatorului `|` (*pipe*) din shell. Acest operator transferă ieșirea standard a unei comenzi către intrarea standard a altei comenzi. Adică, în loc de Listing 4.24 să avem Listing 4.25.

```
1 student@uso:~$ ps -e --no-header | wc -l
2 192
```

Listing 4.25: Transferul datelor de la o comandă la altă comandă prin pipe

În unele situații avem mai multe comenzi înlănțuite prin operatorul pipe într-o singură linie de comandă. În Listing 4.26 am extras din ierarhia `/usr/include` cele 5 fișiere care conțin cel mai des șirul `FILE` (pentru lucrul cu fișîere în limbajul C).

```
1 student@uso:~$ grep -rwc FILE /usr/include/ | grep -v ':0' | sort -n -t
  ':' -k2 | tail -5
2 /usr/include/stdio_ext.h:10
3 /usr/include/unistd.h:10
4 /usr/include/c++/7/profile/impl/profiler_trace.h:20
5 /usr/include/x86_64-linux-gnu/bits/stdio2.h:20
6 /usr/include/stdio.h:82
```

Listing 4.26: Comenzi multiple înlănțuite prin pipe (one-liner)

Astfel de comenzi înlănțuite, care pot folosi și redirectări și alți operatori de înlănțuire sunt numite **one-linere**. Sunt un mod rapid de a combina mai multe comenzi existente pentru un efect nou. Vom discuta în detaliu despre one-linere în Secțiunea 7.5.1.2

> **Filozofia înlănțuirii comenzilor:** Înlănțuirea comenzilor face parte din filozofia Unix: „Do one thing, do one thing well!”. Un proces face un singur lucru și îl face bine. Pentru acțiuni mai complexe sunt combinate comenzi/procese într-un one-liner.

4.6 Interactivitatea proceselor

Din punctul de vedere al interacțiunii cu utilizatorul, clasificăm procesele în interactive și neinteractive. **Procesele interactive** au nevoie de date de intrare de la utilizator (de la tastatură sau de la mouse) și oferă informații pe ecran. **Procesele neinteractive** rulează fără intervenția utilizatorului, le mai numim **procese batch**. Procesele interactive sunt utile pentru a prelua comenzi de la utilizator; cele neinteractive sunt utile pentru gestiunea și buna funcționare a sistemului (de exemplu obținerea adresei de rețea sau

actualizarea timpului sistemului), pentru oferirea de servicii (de exemplu un server web) sau pentru automatizarea sarcinilor (de exemplu să fie indexate pentru căutare fișierele din sistem la un moment de timp).

Unul dintre cele mai importante tipuri de procese interactive dintr-un sistem este **shellul**, adică procesul ce primește intrare de la utilizator și comandă apoi sistemul de operare sau alte procese. Un shell poate fi grafic sau în linia de comandă. Un shell în linia de comandă (numit interpretor de comenzi) va primi un șir de la intrare de la utilizator, va interpreta acel șir într-o comandă (sau mai multe comenzi înlănțuite) și va crea un proces sau mai multe procese. Vom detalia funcționarea shellului în Capitolul 7.

4.6.1 Terminale

Un shell rulează în cadrul unui **terminal**, care este interfața acestuia de interacțiune cu utilizatorul. Descriptorii standard ai shellului (intrarea standard, ieșirea standard, ieșirea de eroare standard) referă terminalul. Altfel spus, când transmitem comenzi shellului, le tastăm la terminal, care este asociat descriptorului de intrare standard al shellului, care apoi interpretează șirul introdus. Terminalul este, așadar, un dispozitiv care intermediază transmiterea intrării și ieșirii de la utilizator prin tastatură/monitor către shell. Figura 4.14 prezintă legătura utilizator, terminal, shell.

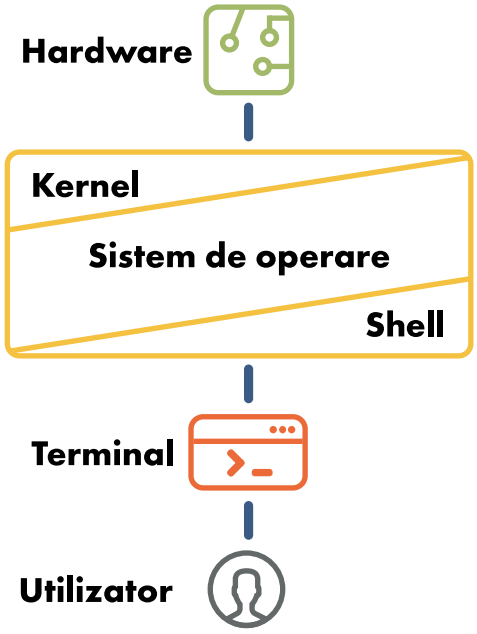


Figura 4.14: Utilizator, terminal, shell

Terminalul este identificat printr-un nume și are asociată, în Linux, o intrare în directorul `/dev`. Comanda `tty` ne indică terminalul curent pentru un shell. Putem să identificăm terminalul unui proces (inclusiv shellul) folosind comanda **`ps`**. De exemplu, în Listing 4.27 terminalul aferent shellului curent este `/dev/pts/1`.

```
1 student@uso:~$ tty
2 /dev/pts/1
3 student@uso:~$ ps -f
4 UID          PID  PPID  C STIME TTY          TIME CMD
5 student    10765 10764  0 10:50 pts/1    00:00:00 -bash
```

```
6 student  11724 10765  0 11:49 pts/1    00:00:00 ps -f
```

Listing 4.27: Identificarea terminalului unui proces

Procesele create dintr-un shell, adică procesele copil ale shellului, vor moșteni acest terminal. Procesele care se află la un moment dat în rulare în acel shell vor partaja ieșirea standard. Dar doar procesul aflat în foreground (shellul sau altul) va avea acces la intrarea standard. Așa cum am precizat în Secțiunea 4.3.3 procesul din foreground „acaparează" intrarea standard dată de terminal.

În Listing 4.28 arătăm cum procesele create din shellul curent au același terminal. Observăm că terminalul (coloana `TTY`) pentru procesul shell (`-bash`) și pentru procesele copil create de acesta (procesul **`sleep`** și **`ps`**) este același: `pts/1`.

```
1 student@uso:~$ sleep 100 &
2 [1] 11728
3 student@uso:~$ ps -f
4 UID        PID  PPID  C STIME TTY          TIME CMD
5 student  10765 10764  0 10:50 pts/1    00:00:00 -bash
6 student  11728 10765  0 11:50 pts/1    00:00:00 sleep 100
7 student  11729 10765  0 11:50 pts/1    00:00:00 ps -f
```

Listing 4.28: Terminalul proceselor pornite din shell

4.6.2 Procese neinteractive

Am precizat mai sus că procesele neinteractive nu au nevoie de interacțiune cu utilizatorul și servesc bunei funcționări a sistemului și automatizării sarcinilor. Nu există definiții și clasificări unanim acceptate pentru procesele neinteractive, așa că facem aici o trecere în revistă a principalelor denumiri.

Un proces care rulează în background (*background process*) este un proces care nu are acces la terminal pentru intrarea standard. Acesta rulează în general neinteractiv. Procesul care rulează în background poate fi readus în foreground și recuperează accesul la terminal și la intrarea standard.

Un proces care rulează în background poate fi detașat de la terminal. Detașarea de terminal înseamnă că acel proces pierde orice legătură cu terminalul dat, legătură ce nu poate fi restaurată. Descriptorii standard ai unui proces detașat de terminal nu mai referă terminalul, ci referă fișiere sau alte tipuri de intrări (pipe, socketi). În listarea proceselor sistemului putem observa că anumite procese nu au terminal asociate. Acestea sunt procese neinteractive, care rulează în background, detașate de terminal.

4.6.3 Procese daemon

O bună parte dintre procesele care rulează neinteractiv, în background, detașate de terminal se numesc **procese daemon**. În anumite descrieri, orice proces care rulează neinteractiv, în background, detașat de terminal este un proces daemon. Într-o definiție mai riguroasă, un proces daemon are ca proces părinte procesul init, descriptorii săi standard referă intrarea `/dev/null` în Linux și oferă un anumit serviciu în sistem (de exemplu procesul daemon **`atd`** este folosit pentru planificarea execuției unui proces la

un moment dat). Din acest motiv, procesele daemon se mai numesc servicii; în Windows procesele care rulează neinteractiv, în background, detașate de terminal, se numesc servicii. De asemenea, în Linux, gestiunea serviciilor se face folosind comanda **`service`**, așa cum vom prezenta în Capitolul 9.

Interacțiunea cu aceste procese se face prin intermediul semnalelor, fișierelor de configurare și fișierelor de tip jurnal (*log*). Exemple de procese daemon sunt:

- procesul init
- procese de tip servicii: **`cron`**, servicii de rețea
- alte procese rulate de utilizator pentru a rula fără un terminal asociat

Concluzionând, în general un proces daemon are următoarele caracteristici:

- rulează în background, fără o formă de interacțiune directă cu utilizatorul
- oferă un serviciu sistemului: planificare, indexare de fișiere
- nu are un terminal asociat
- are ca proces părinte init

Pentru a face o listă aproximativă a proceselor daemon într-un sistem Linux, rulăm comanda:

```
1  student@uso:~$ ps -f --ppid 1
2  UID          PID  PPID  C STIME TTY          TIME CMD
3  root         207     1  0 oct05 ?        00:00:02 /lib/systemd/systemd-
   journald
4  root         230     1  0 oct05 ?        00:00:00 /lib/systemd/systemd-
   udevd
5  systemd+     302     1  0 oct05 ?        00:00:00 /lib/systemd/systemd-
   resolved
6  root         440     1  0 oct05 ?        00:00:00 /usr/sbin/acpid
7  root         444     1  0 oct05 ?        00:00:00 /usr/lib/udisks2/udisksd
8  root         447     1  0 oct05 ?        00:00:00 /usr/sbin/ModemManager
9  syslog       450     1  0 oct05 ?        00:00:00 /usr/sbin/rsyslogd -n
10 [...]
```

> **Aplicații grafice interactive:** Aplicațiile din mediul grafic nu au un terminal asociat dar nu le putem numi procese daemon întrucât au o interfață (grafică) de interacțiune cu utilizatorul, deci sunt procese interactive.

4.6.4 Detașarea de terminal

În general, procesele daemon sunt create la pornirea sistemului, de procesul init. Pe lângă serviciile pornite de init/sistem, un utilizator poate decide să transforme un proces în proces daemon: neinteractiv, detașat de terminal. În felul acesta un utilizator va permite unui proces să ruleze independent de shellul curent iar dacă shellul curent este închis, acel proces continuă să ruleze. Numim această acțiune **daemonizarea unui proces**.

Un caz de utilizare pentru daemonizarea unui proces este rularea unei comenzi complexe la distanță. Să presupunem că ne dorim să pornim o descărcare de fișier prin Bittorrent pe un server la distanță, accesat prin SSH. Vom urma pașii:

1. Ne conectăm prin SSH la serverul de la distanță.
2. Pornim procesul **bittorrent** detașat de terminal.
3. Închidem conexiunea la shellul curent. Procesul **bittorrent** continuă execuția.
4. Ulterior ne reconectăm prin SSH și putem investiga rularea procesului.

Partea dificilă este punctul 3 în care închiderea shellului nu duce la închiderea execuției procesului **bittorrent**. Pentru a realiza acest lucru sunt două opțiuni:

- Daemonizarea procesului **bittorrent**: rulare în background, detașat de terminal
- Folosirea unui utilitar care să creeze o sesiune de terminal detașabilă în care procesul rulează în continuare interactiv

În prima categorie sunt utilitarele **nohup**, **disown**, despre care vom discuta în continuare. În a doua categorie sunt utilitarele **screen**, **tmux**, **byobu**, **dtach** despre care vom discuta în Secțiunea 4.6.5.

4.6.4.1 nohup, disown

Utilitarele **nohup** și **disown** permit daemonizarea unui proces pornit din shell. Strict tehnic, după cum vom vedea, procesele create astfel nu sunt daemoni pentru că nu au neapărat descriptorii standard direcționați către intrarea `/dev/null` și nu au că părinte procesul init de la început. Însă, după cum am precizat, denumirile în zona proceselor neinteractive sunt mai relaxate, așa că vom folosi termenul de daemonizare.

Pentru a înțelege funcționarea utilitarelor **nohup** și **disown**, e nevoie de cunoașterea pașilor din momentul închiderii unui proces shell. Atunci când terminalul aferent unui shell este închis, shellul își încheie, de asemenea, execuția. Atunci shellul trimite semnalul `SIGHUP` proceselor aflate în background, iar procesele își încheie execuția la primirea acestui semnal. Pentru a preveni ca un proces aflat în background să își încheie execuția în momentul închiderii terminalului, trebuie ca acesta să ignore semnalul `SIGHUP`.

Pentru ignorarea semnalului `SIGHUP` de un proces, putem folosi utilitarul **nohup**. Comanda **nohup** prefixează o altă comandă care creează un proces ce ignoră semnalul `SIGHUP`. În Listing 4.29 rulăm comanda **sleep 100** folosind utilitarul **nohup**. Acum procesul **sleep** nou creat va ignora semnalul `SIGHUP`; închiderea terminalului și shellului nu va duce la încheierea execuției sale. După închiderea shellului, procesul este adoptat de init, care devine noul părinte al procesului.

```
1 student@uso:~$ nohup sleep 100 &
2 [2] 11772
3 student@uso:~$ nohup: ignoring input and appending output to 'nohup.out'
4
5 student@uso:~$
```

Listing 4.29: Ignorarea semnalului SIGHUP

Întrucât procesul rulat sub **nohup** va pierde controlul terminalului, nu poate folosi terminalul pentru ieșirea standard sau ieșirea de eroare standard. Dacă nu precizăm explicit redirectare, ieșirea standard și ieșirea de eroare standard vor fi redirectate în fișierul `nohup.out`. Altfel, putem explicit preciza locul unde dorim redirectarea ieșirii standard și ieșirii de eroare standard, de multe ori fiind vorba de `/dev/null`. În Listing 4.30 rulăm un client Bittorrent (cu utilitarul **transmission-cli**) folosind **nohup** cu redirectarea ieșirii standard și a ieșirii de eroare standard în `/dev/null`.

```
1 student@uso:~$ sudo apt install transmission-cli
2 [...]
3 student@uso:~$ wget http://releases.ubuntu.com/18.04/ubuntu-18.04.1-
  desktop-amd64.iso.torrent
4 [...]
5 student@uso:~$ nohup transmission-cli ubuntu-18.04.1-desktop-amd64.iso.
  torrent > /dev/null 2>&1 &
6 [1] 12482
```

Listing 4.30: Ignorarea semnalului nohup și detașarea de terminal

Dacă am uitat să pornim un proces folosind comanda **nohup**, putem în continuare configura shellul să nu îi trimită acestuia semnalul `SIGHUP` folosind comanda **disown**. Practic comanda **nohup** afectează procesul nou creat și acesta ignoră semnalul `SIGHUP`, pe când comanda **disown** afectează shellul și nu trimite semnalul unui proces din background. În Listing 4.31 folosim comanda **disown** iar procesul aflat în background va dispărea din lista de joburi. Acest proces va fi persistent și după încheierea procesului shell.

```
1  student@uso:~$ transmission-cli ubuntu-18.04.1-desktop-amd64.iso.torrent
   > /dev/null 2>&1
2
3  ^Z
4  [1]+  Stopped                 transmission-cli ubuntu-18.04.1-desktop-
   amd64.iso.torrent > /dev/null 2>&1
5  student@uso:~$ bg
6  [1]+ transmission-cli ubuntu-18.04.1-desktop-amd64.iso.torrent > /dev/
   null 2>&1 &
7  student@uso:~$ jobs
8  [1]+  Running                 transmission-cli ubuntu-18.04.1-desktop-
   amd64.iso.torrent > /dev/null 2>&1 &
9  student@uso:~$ disown
10 student@uso:~$ jobs
11 student@uso:~$
```

Listing 4.31: Detașarea de shell folosind disown

4.6.5 screen, tmux, byobu, dtach

Folosirea **nohup** și **disown** are dezavantajul pierderii interactivității. Se creează procese daemon, detașate de terminal și care rulează în background. Uneori avem nevoie să rulăm o aplicație interactivă de la care să ne „deconectăm” și la care să ne reconectăm din nou. Acest lucru îl putem realiza folosind utilitarele **screen**, **tmux**, **byobu** sau **dtach**.

screen, **tmux**, **byobu** sunt numite **multiplexoare de terminal**. Au mai multe funcționalități cele mai utile fiind posibilitatea creării de „subferestre de terminal” într-o

singură sesiune și posibilitatea detașării de la sesiunea de terminal și apoi reatașarea la aceasta. În acest capitol ne interesează al doilea rol. Când pornim oricare dintre utilitare se creează o sesiune de terminal de la care ne putem detașa și la care apoi ne putem reatașa. Listing 4.32 conține un exemplu de folosire a **tmux** pentru a crea o sesiune de terminal și apoi pentru a ne detașa de la ea. Argumentul `list-sessions` al comenzii **tmux** afișează sesiunile existente. Argumentul `attach-session` duce la reatașarea la sesiunea existentă.

```
1 student@uso:~$ tmux
2 # Use Ctrl+b d to detach.
3 [detached (from session 0)]
4 student@uso:~$ tmux list-sessions
5 0: 1 windows (created Sat Oct  6 12:39:41 2018) [179x38]
6 student@uso:~$ tmux attach-session
```

Listing 4.32: Folosirea tmux

screen, **tmux**, **byobu** sunt utilitare similare cu diverse caracteristici, dar cu rulare similară în ceea ce privește pornirea unui proces interactiv și detașarea de la terminal. Uzual, urmăm pașii de mai jos:

1. Se pornește o sesiune de terminal folosind **screen**, **tmux** sau **byobu**.
2. Se rulează comenzile în acel terminal.
3. Se face detașarea de la terminal.
4. Se poate, ulterior, realizare reconectarea la terminal.

Shellul în care a fost pornit terminalul **screen**, **tmux** sau **byobu** poate fi închis, terminalul rămânând activ. Apoi din alt shell se poate face reconectarea. Acest lucru este foarte util pentru scenariul precizat mai sus, al pornirii unui client Bittorrent la distanță, într-o conexiune SSH.

În Tabelul 4.2 sunt prezentate cele mai frecvente comenzi pentru **tmux**, **byobu** și **screen**. **tmux** și **byobu** sunt aplicații mai bogate în funcționalități și le recomandăm în defavoarea **screen**. De avut în vedere că multe opțiuni pot fi personalizate.

Tabelul 4.2: Comparație de comenzi de multiplexor de terminal

funcționalitate	tmux	byobu	screen
creare fereastră nouă	`Ctrl+b c`	`F2`	`Ctrl+a c`
detașare	`Ctrl+b d`	`Shift+F6`	`Ctrl+b c`
listare sesiuni	`tmux list-sessions`	`byobu list-sessions`	`screen -ls`
reatașare sesiune	`tmux attach [session-name]`	`byobu attach [session-name]`	`screen -r [session-id]`
schimbare fereastră	`Ctrl+b <window-id>`	`F3, F4`	`Ctrl+a <window-id>`
închidere sesiune	`Ctrl+b :kill-session`	`Ctrl+a \`	`Ctrl+a \`

Dacă ne interesează strict rularea unei sesiuni de terminal, nu și partea de multiplexare și subferestre, putem folosi comanda **dtach**. Comanda creează o sesiune de terminal și rulează o comandă în cadrul ei (uzual shellul) și apoi ne putem deconecta din acea sesiune. Listing 4.33 conține un exemplu de folosire a comenzii **dtach**; detalii găsiți în pagina de manual a comenzii. În prima comandă din Listing 4.33 creăm o sesiune shell nouă folosind socket-ul numit `this_socket`. Apoi, în cadrul sesiunii deschise, folosim

combinația de taste `Ctrl+\` pentru a ne detașa. Apoi a doua comandă ne permite reconectarea la sesiunea shell deschisă.

```
student@uso:~$ dtach -c this_socket /bin/bash
# Use Ctrl+\ to detach
[detached]
student@uso:~$ dtach -a this_socket
```

Listing 4.33: Folosirea dtach

4.7 Investigarea proceselor

Procesele folosesc resursele sistemului de calcul pentru realizarea unor acțiuni. Se poate întâmpla să avem nevoie să eliberăm resurse sau ca sistemul sau un proces să funcționeze anevoios. În această situație e util să investigăm procesele sistemului și să identificăm procesele care consumă resurse. Apoi aceste procese pot fi omorâte, sau analizate sau li se pot actualiza prioritățile.

Un prim mod de a investiga procesele sistemului este cu ajutorul utilitarelor de listare a proceselor sau monitorizare a lor. Utilitarul **`ps`** este utilitar de bază în Linux. Pentru monitorizare folosim **`top`** sau **`htop`**. Pe Windows putem folosi Task Manager sau Process Explorer.

Utilitarele **`top`** și **`htop`** oferă o interfață interactivă de afișare periodică a informațiilor despre procese. Sun utile pentru o privire dinamică a proceselor sistemului. Simpla rulare a comenzilor **`top`** sau **`htop`** duce la afișarea unui ecran. În acel ecran putem folosi apoi diferite opțiuni pentru a selecta anumite procese sau anumite atribute sau moduri de sortare.

Am prezentat exemple și screenshoturi de folosire a acestor utilitare în Secțiunea 4.2.1.

Similar utilitarelor **`top`** și **`htop`** de investigare a folosirii procesorului și memoriei, putem folosi utilitarul **`iotop`** pentru investigarea folosirii discului (doar în mod privilegiat, este nevoie de folosirea **`sudo`**):

```
student@uso:~$ sudo iotop
```

Atunci când dorim să investigăm fișierele sau socketurile deschise de un proces, putem folosi utilitarul **`lsof`**, pe care l-am prezentat și în Secțiunea 4.4 Pentru monitorizarea folosirii resurselor unui proces, în afara utilitarului **`top`**, putem folosi utilitarele din familia **`sysstat`**. Acestea afișează periodic informații despre procentul de procesor ocupat, procentul de memorie ocupată, utilizarea de I/O. Monitorizarea (continuă) este utilă pentru a verifica trenduri în utilizarea resurselor. Monitorizarea este o formă de asigurare a securității unui sistem de calcul, așa cum vom prezenta în Capitolul 12.

În cazul în care dorim o investigare amănunțită a funcționării unui program/proces putem urmări apelurile de bibliotecă sau de sistem realizate. Pentru aceasta folosim, respectiv, utilitarele **`ltrace`** și **`strace`**. Utilitarele pot rula un program pentru a crea un proces nou și a-l investiga, și pot fi atașate unui proces existent. De exemplu, pentru a vedea ce operații de scriere realizează comanda **`ls`**, folosim comenzile **`strace`** și **`ltrace`** ca în Listing 4.34.

```
1  student@uso:~$ strace -e write ls > /dev/null
2  write(1, "Desktop\nDocuments\nDownloads\nMusi"..., 191) = 191
3  +++ exited with 0 +++
4  student@uso:~$ ltrace -e fwrite_unlocked ls > /dev/null
5  ls->fwrite_unlocked("Desktop", 1, 7, 0x7feee1c54760)
                                                        = 7
6  ls->fwrite_unlocked("Documents", 1, 9, 0x7feee1c54760)
                                                      = 9
7  ls->fwrite_unlocked("Downloads", 1, 9, 0x7feee1c54760)
                                                      = 9
8  ls->fwrite_unlocked("Music", 1, 5, 0x7feee1c54760)
                                                         = 5
9  ls->fwrite_unlocked("Pictures", 1, 8, 0x7feee1c54760)
                                                       = 8
10 [...]
11 +++ exited (status 0) +++
```

Listing 4.34: Investigarea apelurilor de sistem și de blbliotecă

4.8 Anexă: Sistemul de fișiere procfs

În Linux, utilitarele de lucru cu procese își iau informațiile din sistemul de fișiere virtual **procfs**, accesibil din `/proc`. Un sistem de fișiere virtual este un sistem de fișiere care nu are suport pe disc, ci rezidă în memorie. procfs este o interfață oferită de sistemul de operare în forma unor fișiere de unde pot fi obținute informații despre procese. Un indiciu că este folosit este numele pachetului ce conține utilitarele **`ps`**, **`pgrep`** etc.: pachetul `procps`.

> **/proc doar în Linux:** Sistemul de fișiere procfs se găsește doar în Linux, nu și pe alte Unix-uri.

Investigăm `/proc` în Listing 4.35. În liniile 1-7 observăm că `/proc` este compus din fișiere cu nume obișnuite și directoare format numeric: Fiecare director cu format numeric corespunde unui proces, numărul reprezentând PID-ul procesului. În cadrul fiecărui acest director se găsesc fișiere și subdirectoare ce conțin informații despre procesul respectiv așa cum se observă din liniile 9-13.

```
1  student@uso:~$ ls /proc
2  1      1183   1456   1655  276   474        bus          mounts
3  10     1188   1457   1668  28    479        cgroups      mtrr
4  1010   1197   1459   1683  281   488        cmdline      net
5  1018   12     14636  17    284   493        consoles     pagetypeinfo
6  1019   1202   1464   1742  287   496        cpuinfo      partitions
7  1032   1205   1467   176   288   498        crypto       sched_debug
8  [...]
9  student@uso:~$ ls /proc/2395
10 attr             exe        mounts         projid_map    status
11 autogroup        fd         mountstats     root          syscall
12 auxv             fdinfo     net            sched         task
13 cgroup           gid_map    ns             schedstat     timers
14 [...]
```

Listing 4.35: Informații din /proc

Dintre intrările din Listing 4.35 selectăm câteva și rolurile lor:

- **`exe`**: legătură simbolică către executabilul aferent procesului
- **`fd`**: director cu tabela de descriptori de fișiere a procesului
- **`status`**: informații despre starea procesului
- **`task`**: director cu firele de execuție (*threadurile*) procesului

Un utilizator poate, astfel, obține informații despre procese fie investigând fișierele din `/proc`, fie folosind utilitarele **`ps`**, **`pgrep`**, **`top`**, **`lsof`**. De avut în vedere că `/proc` se găsește doar în Linux, pe când utilitarele sunt disponibile, într-un mod sau altul în majoritatea Unix-urilor.

În afara directoarelor aferente, `/proc` deține și alte intrări pentru investigarea sistemului. De exemplu:

- `/proc/filesystems`
- `/proc/interrupts`
- `/proc/ioports`
- `/proc/sys/`
- `/proc/cmdline`
- `/proc/cpuinfo`
- `/proc/meminfo`

Aceste intrări sunt consultate de utilitare puse la dispoziție în Linux precum **`mount`**, **`lscpu`**, **`free`**, **`df`**, **`lsblk`** și altele.

> **Folosire procfs:** Dacă, în Linux, doriți să aflați sau să prelucrați informații despre procese sau despre sistem și utilitarele existente nu vă ajută, puteți să urmăriți intrările din `/proc`.

4.9 Anexă: Internele pornirii unui proces. Loading

Un proces este creat de un alt proces (de obicei un proces shell) dintr-un program executabil. Spunem că procesul este un program aflat în execuție, iar programul este imaginea procesului. Crearea unui proces dintr-un program executabil poartă numele de **loading**. Momentul în care are loc loadingul se cheamă **load time**. Iar crearea procesului este facilitată de o componentă a sistemului de operare numită **loader**.

Loaderul interpretează programul executabil, apoi alocă memoria aferentă pentru proces și copiază din programul executabil datele și instrucțiunile în memorie. În același timp loaderul face legătura cu biblioteci dinamice și le încarcă la nevoie în memorie, pentru rularea corespunzătoare a procesului. După aceasta se stabilește ca procesul să înceapă execuția de la **punctul de intrare** (*entry point*) care este o funcție similară funcției `main()`. Apoi procesul începe să ruleze din momentul în care este planificat pe un procesor.

Acțiunile unui loader sunt prezentate în Figura 4.15.

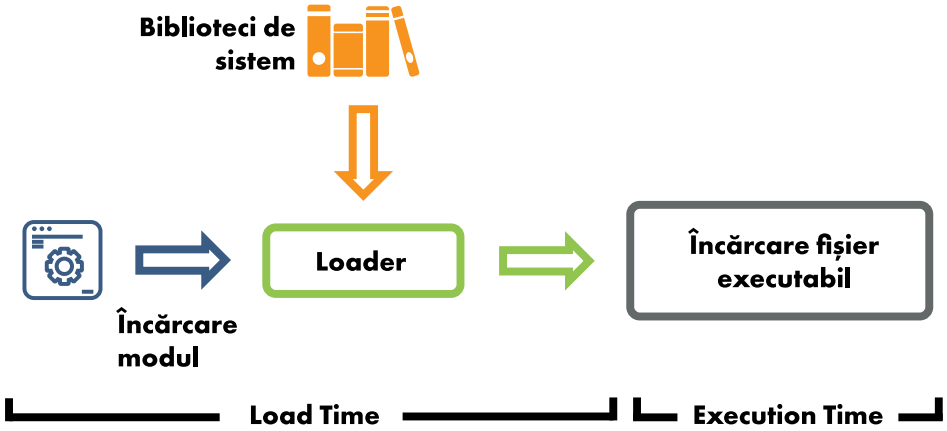


Figura 4.15: Acțiunile loader-ului

Două sau mai multe procese pot fi rulate din același program executabil. Funcționarea acestora diferă însă încă de la load-time în funcție de doi factori: argumentele programului și variabilele de mediu. Argumentele sunt transmise de utilizator/shell în momentul rulării unei comenzi. Iar variabilele de mediu sunt variabile proprii shellului care sunt moștenite de procesele create de acesta. Mai multe informații vom prezenta în Capitolul 7.

Astfel, pe lângă pașii descriși mai sus, loaderul plasează pe stiva procesului (accesibil funcției `main()`) argumentele programului și variabilele de mediu. Acestea sunt apoi disponibile programului din argumentele funcției `main()` (în C). Găsiți o exemplificare a investigării variabilelor de mediu într-un program C în codul sursă de suport al cărții în directorul `code/04-process/main-args/`.

După load-time, în momentul în care procesul rulează intră în faza run-time. Aici procesul folosește resursele sistemului, comunică cu exteriorul și cu alte procese și poate fi monitorizat și controlat de utilizator.

4.10 Sumar

Pe un sistem de calcul procesele sunt necesare pentru realizarea acțiunilor utilizatorilor.

Un proces execută o acțiune izolată de alte procese și folosește resursele sistemului (procesor, memorie, I/O) pentru realizarea acțiunii.

Un proces este creat dintr-un program executabil, numit imaginea procesului. Mai multe procese pot fi create din aceeași imagine de executabil.

Procesele au atribute de identificare, de contabilizare a resurselor folosite și de stabilire a ce resurse și cât de mult pot folosi. Un proces aparține unui utilizator și poate folosi resursele acelui utilizator.

Există o ierarhie de procese, shellul creează majoritatea proceselor. Un proces poate avea mai multe procese copil și un singur proces părinte. În vârful ierarhiei este procesul init.

În cadrul unei sesiuni de shell putem avea un proces în foreground și mai multe procese în background.

Procesele comunică/interacționează pentru a realiza o acțiune. Putem redirecta ieșirea și intarea unui proces în fișiere. Și putem redirecta ieșirea unui proces la intrarea altui proces folosind pipe-uri.

Procesele își încheie execuția de la sine, sau pot fi omorâte de alte procese. În Linux, procesele sunt omorâte de alte procese prin intermediul semnalelor.

Unele procese sunt interactive, altele sunt batch/daemoni/servicii. Daemonii nu au un terminal atașat și rulează în mod neinteractiv, independent de procesele shell existente.

Este util să monitorizăm procesele și resursele folosite de acestea pentru a detecta abuzuri de folosire a resurselor sau pentru a identifica sursele de încărcare ale sistemului.

În Linux, informațiile despre procese se găsesc în sistemul de fișiere procfs. De aici sunt interpretate de utilitarele de lucru cu procese.

Capitolul 5

Utilizatori

Sistemul de operare oferă resurse pe care utilizatorii să le poată folosi. În mod obișnuit, utilizatorii creează procese (aplicații) care folosesc resursele sistemului (procesor, memorie, I/O) pentru a realiza o activitate. Utilizatorii stochează și transferă date în/din sistemul de calcul, de exemplu folosind sistemul de fișiere sau rețeaua.

Accesul la un sistem de calcul se realizează prin intermediul unui **cont de utilizator** (*user account*). Un utilizator fizic (o persoană) folosește un cont de utilizator pentru a se identifica în sistem și pentru a folosi resurse cu permisiunile aferente acelui cont de utilizator. În ceea ce urmează vom folosi, în general, termenul de utilizator ca fiind un cont de utilizator pe un sistem de calcul.

Sistemele de operare moderne sunt în general multi-utilizator (*multi-user*), adică permit existența mai multor conturi de utilizator pe un sistem și rularea simultană a mai multor procese aparținând diferiților utilizatori. Mai mulți utilizatori pot simultan deschide shelluri locale și rula comenzi pe un sistem prin conexiuni de rețea la sistem, precum folosind SSH, despre care vom discuta în Capitolul 12. Un cont de utilizator este separat de alt cont astfel că procesele unui utilizator sunt separate/izolate de procesele altuia, la fel și resursele folosite de procese (precum fișierele).

Informațiile despre conturile de utilizatori sunt utile nu doar utilizatorilor avansați, ci și utilizatorilor obișnuiți. Deși utilizatorul obișnuit are în general un singur cont pe sistemul propriu, informațiile despre conturile de utilizator îi sunt necesare pentru a înțelege ce acțiuni poate executa pe sistemul de operare, pentru ce acțiuni are nevoie de accesul la un cont privilegiat. De asemenea, poate să creeze la nevoie un cont pentru un prieten sau o rudă pe un sistem al său. Sau se poate autentifica pe sistemul altcuiva și va trebui să aibă grijă ce informații lasă pe acel sistem. Pentru un utilizator cu permisiuni speciale (*power user*) sau un administrator, informațiile despre și operațiile cu utilizatori sunt esențiale pentru a asigura accesul mai multor utilizatori fizici la un sistem dat, pentru a se asigura de separarea resurselor între aceștia și între procesele acestora și, în general, pentru a garanta securitatea sistemului.

Din punctul de vedere al sistemului de operare, un cont de utilizator este reprezentat de obicei sub forma unui nume de utilizator (*username*). Acesta este un șir de caractere cu care este identificat utilizatorul. Pentru a permite accesul în sistem, acestui nume de utilizator i se asociază o formă de autentificare, precum parolă sau autentificare biometrică. Dacă utilizatorul fizic furnizează în interfața de acces la sistem un nume de utilizator valid și forma de autentificare corectă, i se permite accesul în sistem.

Până acum am prezentat conturi de utilizator de sistem, adică acele conturi care permit crearea de procese și dețin resurse precum fișiere. Există și conturi de utilizatori de aplicații, precum conturile de utilizatori de aplicații web. Aceste conturi permit accesul la resurse care sunt gestionate de aplicație, nu de sistemul de operare. De asemenea, aceste conturi nu permit crearea de procese sau accesul la resursele sistemului; permit doar accesul gestionat de aplicație la resursele aplicației.

Indiferent de tipul de cont de utilizator (de sistem sau de aplicație) principiul de folosire este același: se folosește un nume de utilizator și o formă de autentificare pentru a permite accesul acestuia la resursele sistemului sau aplicației. Toate numele de utilizator și formele de autentificare valide sunt menținute într-o bază de date. Când un utilizator fizic transmite aceste date, sistemul sau aplicația le verifică dacă sunt prezente în baza de date; în caz afirmativ se oferă accesul la resurse. Acest mod de folosire este descris în Figura 5.1.

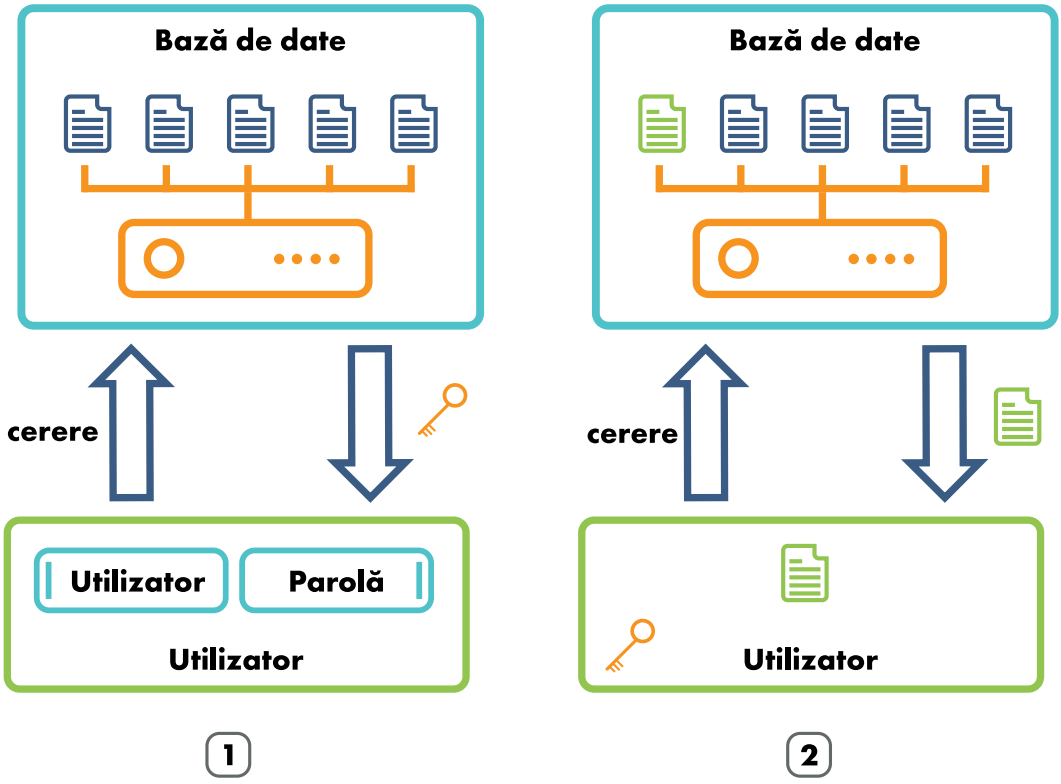


Figura 5.1: Bază de date de access

În continuare ne vom referi în general la conturile de utilizator de sistem, cu precizarea că multe concepte se aplică și la conturile de utilizator de aplicație.

5.1 Utilizatori de sistem

Așa cum am precizat mai sus, conturile de utilizator de sistem sunt folosite pentru a obține acces la resursele gestionate de sistemul de operare. Conturile de utilizator permit obținerea unui shell și rularea de comenzi care creează procese.

De obicei, un sistem de operare oferă un prompt grafic sau text de autentificare unde vor fi completate numele de utilizator și apoi, uzual, parola.

În momentul autentificării unui utilizator de sistem, acesta are acces complet la o parte din sistemul de fișiere numită **directorul home al utilizatorului**, reprezentând directorul personal al acestuia. Fiecare sistem de operare are, în general, un astfel de director pentru utilizatorul de sistem (`/home/student` pe Linux, `C:\Users\student` pe Windows, `/Users/student` pe macOS). După autentificare, utilizatorul poate crea procese care apoi accesează fișiere sau alte resurse ale utilizatorului.

5.1.1 Permisiuni și privilegii

Odată autentificat în sistem, un utilizator are anumite permisiuni de folosire a sistemului. De exemplu, un utilizator obișnuit poate crea fișiere doar în directorul său home, dar nu poate instala pachete. Un alt utilizator poate avea permisiunea de a instala pachete. Sistemul de operare, pe baza unor configurări, permite unui utilizator anumite acțiuni.

În unele sisteme și aplicații există noțiunea de rol. Un utilizator poate avea un rol care îi atribuie anumite permisiuni. De exemplu, rolul de `Administrator` înseamnă, în general, într-o instalare de platformă precum Moodle[1], permisiuni complete în cadrul platformei; un rol de `Profesor` înseamnă controlul conținutului cursurilor; un rol de `Student` înseamnă posibilitatea accesării conținutului.

În general, sistemele oferă anumite permisiuni utilizatorilor. Structurarea acestor permisiuni este esențială pentru asigurarea securității sistemului. Dacă un utilizator are mai multe permisiuni decât are nevoie, acesta va putea abuza sau exploata sistemul. Dacă un atacator din exterior obține permisiunile unui utilizator (impersonează utilizatorul), pagubele pe care le provoacă vor fi proporționale cu permisiunile utilizatorului.

Din punctul de vedere al conceptelor de securitate, utilizatorul este un **agent** iar resursele puse la dispoziție de sistem sunt **obiecte**. **Politica de securitate** (*security policy*) a sistemului precizează permisiunile și dictează ce agenți au acces la ce obiecte. În acest context, apar adesea termenii de **permisiune** și **privilegiu**.

> În general, spunem că permisiunile sunt informații aferente obiectului care stabilesc ce agenți pot face tipuri de acțiuni, în vreme ce privilegiile sunt certificate aferente agentului care stabilesc ce acțiuni poate acesta întreprinde. Mai multe informații găsiți în Capitolul 12.

În sistemele de operare de uz general (Linux/Unix, Windows, macOS) există, în general, **utilizatori obișnuiți** (neprivilegiați) și **utilizatori privilegiați**.

> Utilizatorii neprivilegiați au acces complet la o parte din sistemul de fișiere (directorul home) și pot crea procese care folosesc aceste resurse. Utilizatorii privilegiați pot face acțiuni care afectează starea globală a sistemului: instalare de pachete, adăugare/ștergere de utilizatori, repornirea sistemului, configurarea datei sistemului, schimbarea permisiunilor sistemului, configurarea rețelei etc.

[1] http://www.moodle.org

În general, există un utilizator al sistemului care are permisiuni complete. De obicei ne referim la acest utilizator ca *superuser*. Pe sistemele Linux/Unix este numit `root`, în vreme ce pe sistemele Windows este numit `Administrator`. În multe situații, pentru realizarea unei operații privilegiate, se accesează contul de *superuser*.

5.2 Utilizatorul administrativ (superuser)

Utilizatorul administrativ, *superuserul*, deține controlul complet al sistemului. Toate operațiile privilegiate, inaccesibile unui utilizator obișnuit neprivilegiat, sunt permise acestuia. Un utilizator obișnuit va accesa acest cont special (dacă știe parola sau forma de autentificare a acestuia) pentru a realiza o operație privilegiată.

Faptul că superuserul are acces complet la sistem îl face foarte periculos de folosit. Dacă un atacator obține acces neautorizat la contul de superuser, atunci sistemul este compromis. La fel, dacă rulăm greșit o comandă privilegiată, aceasta poate afecta iremediabil sistemul.

> **Comanda rm -fr /:** Comanda **`rm -fr /`** este celebră pentru că poate fi folosită pentru a șterge tot sistemul de fișiere. Dacă este folosită de contul de `root` această comandă șterge conținutul sistemului de fișiere începând de la directorul rădăcină (`/`). Aveți grijă în folosire acestei comenzi și, ori de câte ori ștergeți un fișier, aveți grijă ce argumente transmiteți comenzii **`rm`**.

De aceea, sistemele de operare moderne pun la dispoziția utilizatorul metode de realizare de acțiuni privilegiate care să nu necesite accesarea directă a contului privliegiat. Astfel că, în folosirea superuserului, există următoarele recomandări de folosire:

- Să fie folosit contul doar când este absolută nevoie, pentru operații privilegiate. Nu vă autentificați ca superuser pe interfața grafică.
- Să fie o parolă / formă de autentificare cât mai greu de accesat în mod neautorizat.
- Să fie evitată folosirea superuser-ului când există forme alternative de executare de acțiuni privilegiate.

Sistemele de operare moderne oferă opțiuni alternative de executare a acțiunilor privilegiate. În Linux/Unix se poate folosi **`sudo`**, bitul setuid, capabilități; le vom prezenta în Secțiunea 5.2.3.

În Windows, începând cu Windows Vista se folosește *User Account Control* (UAC), prezentat în Figura 5.2. *User Account Control* este o tehnologie care oferă un prompt care trebuie confirmat ori de câte ori se dorește realizarea unei acțiuni privilegiate. Doar utilizatorilor configurați specific li se permite rularea de acțiuni privilegiate prin User Account Control.

Figura 5.2: User Account Control

5.2.1 Utilizatorul administrativ în Linux (root)

Utilizatorul administrativ în Linux este numit `root`. Utilizatorul `root` are acces complet la resursele sistemului și poate face orice operații. În general, accesarea contului de `root` este evidențiată în linia de comandă prin prezența simbolului `#` (*diez*) în cadrul promptului ca în Listing 5.1.

```
1 student@uso:~$ sudo su
2 [sudo] password for student:
3 root@uso:/home/student#
```

Listing 5.1: Prompt neprivilegiat și privilegiat (root)

În secvența de mai sus, utilizatorul inițial este `student`, iar promptul său este marcat de simbolul `$` (*dolar*) la finalul promptului. Atunci când folosim comanda `sudo su` și după ce introducem parola utilizatorului `student`, obținem un shell privilegiat, marcat de simbolul `#` (*diez*) la finalul promptului.

Accesarea unui cont (inclusiv a contului `root`) se poate face în două moduri:

- autentificarea în sistem cu acel cont (la un prompt de login în linia de comandă sau în interfața grafică)
- schimbarea utilizatorului curent în linia de comandă prin folosirea comenzii **`su`** (*substitute user*)

În general, nu este recomandată autentificarea în sistem folosind contul de `root`. Orice aplicație care va fi lansată din acest cont va avea acces complet asupra resurselor sistemului, devenind astfel o posibilă zonă de atac a acestuia. Este recomandat să folosiți comanda **`su`** pentru schimbarea temporară a utilizatorului și obținerea unui shell de `root`, folosirea comenzilor privilegiate și apoi revenirea în consola utilizatorului neprivilegiat folosind comanda **`exit`**, așa cum este ilustrat în Listing 5.2.

```
1 student@uso:~$ su
2 Password:
3 root@uso:/home/student# apt install deluge
4 [...]
5 root@uso:/home/student# exit
```

```
6 exit
7 student@uso:~$
```

Listing 5.2: Accesarea contului root

În Listing 5.2 am folosind comanda `su` urmată de introducerea parolei utilizatorului `root` pentru a accesa contul privilegiat. Acum putem executa acțiuni privilegiate, precum instalarea pachetului `deluge`. La final folosim comanda `exit` pentru a încheia sesiunea curentă privilegiată și a reveni la un shell de utilizator neprivilegiat (`student`).

5.2.2 Utilitarul su

Utilitarul **`su`** este folosit pentru schimbarea utilizatorului, nu neapărat în `root`. Dacă **`su`** nu primește nici un argument sau primește ca argument `root`, va schimba utilizatorul în `root`. Altfel, va schimba în utilizatorul primit ca parametru. Așa cum apare în Listing 5.3, la rularea comenzii **`su`** se cere parola noului utilizator; fără introducerea parolei acelui utilizator nu se poate schimba utilizatorul.

```
1  student@uso:~$ su teacher
2  Password:
3  teacher@uso:/home/student$ exit
4  student@uso:~$ su - teacher
5  Password:
6  teacher@uso:~$ logout
7  student@uso:~$ su -l teacher
8  Password:
9  teacher@uso:~$ logout
10 student@uso:~$ su -l teacher -c whoami
11 Password:
12 teacher
```

Listing 5.3: Schimbarea contului de utilizator folosind comanda su

Liniile 4 și 7 din Listing 5.3 prezintă o formă uzuală de rulare a comenzii **`su`** cu opțiunea `-l` sau `-`; această formă schimbă mediul de lucru al viitoarei comenzi, incluzând directorul home. După cum se poate observa, există o diferență între linia `3` și liniile `6` și `9`, care folosesc opțiunile `-` și `-l`: după schimbarea utilizatorului se schimbă și directorul curent în directorul home al noului utilizator. Acest lucru se observă în promptul expus de shell.

Utilitarul **`su`** permite rularea unei comenzi fără a deschide un shell prin intermediul opțiunii `-c`, ca în liniile `10-12` din Listing 5.3. În felul acesta se rulează direct comanda primită ca argument și apoi se revine la shellul utilizatorului curent, fără a se deschide un shell al noului utilizator.

5.2.3 Alternative la root în Linux

Așa cum am precizat anterior, utilizatorul `root` deține controlul întregului sistem. Dacă un atacator obține acces neautorizat la contul `root`, sistemul va fi compromis. Pe de altă parte, privilegiile pe care le deține utilizatorul `root` sunt necesare pentru administrarea sau funcționarea sistemului, precum configurarea unei interfețe de rețea. De aceea

sistemele de operare moderne oferă opțiuni de granularizare a privilegiilor. În Linux există câteva astfel de opțiuni printre care amintim bitul setuid, capabilități și utilitarul **sudo**.

Bitul setuid (*set-user-id-on-execution*) este un bit asociat unui fișier executabil. Atunci când executăm acel fișier procesul nou creat nu va aparține utilizatorului care a rulat comanda, ci utilizatorului care deține fișierul, în mod obișnuit utilizatorul `root`. Listing 5.4 conține un exemplu de folosire a bitului `setuid`. Observăm în permisiunile fișierelo `/bin/ping`, `/bin/su`, `/usr/bin/sudo` prezența literei `s`, însemnând bitul setuid.

```
1 student@uso:~$ ls -l /bin/ping
2 -rwsr-xr-x 1 root root 64424 Mar 10  2017 /bin/ping
3 student@uso:~$ ls -l /bin/su
4 -rwsr-xr-x 1 root root 44664 Jan 25  2018 /bin/su
5 student@uso:~$ ls -l /usr/bin/sudo
6 -rwsr-xr-x 1 root root 149080 Jan 31  2020 /usr/bin/sudo
```

Listing 5.4: Bitul set-user-id-on-execution (setuid)

Prezența bitului `setuid` pe un executabil deținut de `root` (de exemplu **passwd**, **su**, **sudo**) înseamnă că acele comenzi pot fi rulate de orice utilizator pentru a crea procese privilegiate. Vom explica în Secțiunea 5.4 de ce aceste comenzi sunt privilegiate.

Bitul `setuid` are dezavantajul că procesul nou creat rulează cu privilegii complete de `root`. Dacă acel proces este exploatat, atacatorul va deține un proces cu permisiuni de `root` și sistemul va fi compromis. De aceea, pe Linux există un alt mecanism numit **capabilități**. Capabilitățile sunt un concept de bază în sisteme de operare însemnând atașarea unui privilegiu unui agent (unui obiect). De exemplu, pe sistemele Linux moderne, executabilul aferent utilitarului ping are atașată capabilitatea `cap_net_raw`. Această capabilitate înseamnă că procesele create din executabilul **ping** vor avea privilegiul de a crea resurse privilegiate necesare utilitarului **ping**.

Spre deosebire de bitul de setuid, capabilitățile sunt granulare. Privilegiile aferente contului `root` sunt separate în capabilități. Un proces are atașate, la nevoie, doar capabilitățile de care are nevoie; dacă acel proces este exploatat atunci acțiunile posibile ale atacatorului sunt limitate la ce privilegii oferă capabilitățile, nu la toate privilegiile contului de `root`.

Exemple de capabilități pe Linux sunt:

- `cap_net_raw`: permite folosirea de socketi speciali (numiți *raw*)
- `cap_net_admin`: permite configurarea rețelei
- `cap_fowner`: permite configurare de permisiuni pe orice fișier
- `cap_kill`: permite transmiterea de semnale către orice procese

Documentație despre capabilități: Suportul de capabilități este o componentă avansată a sistemului de operare, care permite dezvoltatorilor de aplicații configurarea de privilegii granulare. Pentru mai multe detalii, urmăriți pagina de manual, folosind comanda **man 7 capabilities**.

5.2.3.1 Utilitarul sudo

Bitul `setuid` și capabilitățile sunt mecanisme care oferă privilegii unor executabile și a proceselor pornite din acestea sau, în cazul capabilităților, și mai granular, unor procese. Nu permit ușor atașarea de privilegii unui utilizator sau configurarea personalizată (*custom*) a acțiunilor care pot fi executate de acel utilizator. Pentru aceasta, soluția folosită pe sistemele Unix este **sudo**.

sudo este un utilitar care permite rularea unor comenzi în numele altui utilizator, în general `root`. **sudo** are o configurare granulară și permite precizarea explicită a ce comenzi să fie rulate. Spre deosebire de utilitarul **su**, la rularea utilitarului **sudo** nu este nevoie de cunoașterea parolei noului utilizator, ci a utilizatorului curent.

În mod tipic, când rulăm **sudo**, îl succedăm cu comanda pe care vrem să o executăm ca `root`, ca în exemplele din Listing 5.5. Liniile `1` și `3` arată rularea unei comenzi în mod privilegiat prin prefixarea acesteia cu `sudo`. Adesea dorim obținerea unui shell de `root` pentru a rula mai multe comenzi privilegiate. Pentru aceasta rulăm tipic una dintre cele două comenzi de pe liniile `4-8` din Listing 5.5: `sudo su` sau `sudo bash`.

```
1 student@uso:~$ sudo apt install inkscape
2 [...]
3 student@uso:~$ sudo adduser dean
4 student@uso:~$ sudo su
5 root@uso:/home/student# exit
6 student@uso:~$ sudo bash
7 root@uso:~# exit
8 student@uso:~$
```

Listing 5.5: Folosirea sudo

Utilitarul **sudo** permite unui utilizator să ruleze comenzi ca `root` (sau ca alt utilizator) printr-o configurație din fișierul de configurare `/etc/sudoers`. Listing 5.6 conține secvențe din fișierul de configurare `/etc/sudoers`. De exemplu, conform liniei de configurare `2` utilizatorul `student` poate rula comenzi ca `root`.

```
1 # User privilege specification
2 student ALL=(ALL:ALL) ALL
3 teacher ALL=(ALL:ALL) /bin/cat
4
5 # Allow members of group admin to execute any command.
6 %admin  ALL=(ALL) ALL
7
8 # Allow members of group sudo to execute any command.
9 %sudo        ALL=(ALL) ALL
```

Listing 5.6: Secvențe din fișierul /etc/sudoers

În mod obișnuit, fișierul `/etc/sudoers` are linii de configurare care permit utilizatorilor din grupul `admin` sau grupul **sudo** să poată rula comenzi ca `root`, precum liniile `5` și `9` din Listing 5.6.

În această situație, dacă dorim ca un utilizator să poată rula comenzi ca `root` îl adăugăm în grupul admin sau în grupul **sudo**, nu este nevoie de editarea fișierului `/etc/sudoers`.

Folosirea contului de root: E important când e folosit `root` și când se obține consolă de `root` și ce utilizatori au acces la **sudo**. Avem grijă când acordăm unui utilizator *permisiuni de sudo*, adică posibilitatea de a rula comenzi ca `root`, lucru care poate duce la compromiterea sistemului.

sudo permite configurări mai avansate în fișierul `/etc/sudoers`; de exemplu linia de configurare `3` din Listing 5.6 permite utilizatorului `teacher` rularea doar a comenzii `cat` ca `root`, permițând vizualizarea tuturor fișierelor din sistem, ca în Listing 5.7. Utilizatorul `teacher` nu poate vizualiza conținutul fișierului `/etc/sudoers` (folosind comanda **cat**). Dar poate dacă prefixează cu `sudo` comanda de afișare.

```
1 teacher@uso:~$ cat /etc/sudoers
2 cat: /etc/sudoers: Permission denied
3 teacher@uso:~$ sudo cat /etc/sudoers
4 [sudo] password for teacher:
5 #
6 # This file MUST be edited with the 'visudo' command as root.
```

Listing 5.7: Rularea unei comenzi privilegiate cu sudo

Fișierul `/etc/sudoers` este un fișier important în sistem și de aceea există mai multe restricții și convenții pentru accesarea acestuia. Astfel, deoarece configurația sa afectează întreg sistemul de operare, editarea fișierului este permisă numai utilizatorului privilegiat. De asemenea, pentru a preveni editarea eronată (ce poate duce sistemul într-o stare de eroare imposibil de rezolvat), se recomandă folosirea comenzii **visudo** pentru editare. Pentru editarea sa, fie rulăm comanda **visudo** ca `root`, fie comanda **sudo visudo**.

Folosire /etc/sudoers: Fișierul `/etc/sudoers` permite configurări avansate. Puteți parcurge pagina de manual **man sudoers** sau resursele din Internet[a].

[a] https://www.digitalocean.com/community/tutorials/how-to-edit-the-sudoers-file-on-ubuntu-and-centos

5.2.3.1.1 sudo și redirectare O situație frecventă este să redirectăm rezultatul rulării unei comenzi într-un fișier privilegiat (la care nu avem de obicei acces). De exemplu, dacă dorim să activăm rutarea pe un sistem Linux, vom rula, ca `root`, comanda:

```
1 student@uso:~$ echo 1 > /proc/sys/net/ipv4/ip_forward
```

În momentul în care folosim un utilizator neprivilegiat dar care poate folosi **sudo**, am fi tentați să încercăm să rulăm comanda:

```
1 student@uso:~$ sudo echo 1 > /proc/sys/net/ipv4/ip_forward
```

Acest lucru însă nu funcționează pentru că, deși comanda **echo 1** rulează ca `root`, comanda **sudo**, al cărei output este redirectat, rulează ca utilizator neprivilegiat. Adică redirectarea cu operatorul `>` are loc pentru comanda **sudo**, rulată de utilizator neprivilegiat, și nu va reuși.

Soluția la această situație este să nu folosim redirectare ci să folosim o comandă care primește ca argument acel fișier, iar comanda să ruleze privilegiat ca **sudo**. Adică soluția este folosirea comenzii **tee**, ca mai jos:

```
1 student@uso:~$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

În forma de mai sus comanda **tee** este rulată ca `root` prin intermediul utilitarului **sudo**. Comanda scrie informația primită la intrarea standard (adică `1`) în fișierul primit ca argument (`/proc/sys/netc/ipv4/ip_forward`) în mod privilegiat.

> **Folosire `tee` pentru a scrie într-un fișier privilegiat:** Atunci când dorim să scriem într-un fișier privilegiat fără a accesa o consolă de `root`, folosim comanda tee cu **sudo**. Comanda **tee** are și opțiunea `-a` pentru adăuga informație la sfârșitul fișierului (*append*); în mod obișnuit, comanda suprascrie conținutul fișierului primit ca argument.

5.3 Operații cu utilizatori

În sistemul de operare un utilizator este reprezentat de procesele și resursele acestuia. Procesele și resursele (de exemplu fișierele) sunt marcate ca aparținând utilizatorului.

Pentru un utilizator fizic (o persoană), un utilizator este identificat printr-un nume de utilizator (*username*); atunci când ne autentificăm, pentru acces folosim numele de utilizator. La nivelul sistemului de operare, însă, un utilizator este identificat printr-un număr, numit în mod uzual *user identifier* sau **UID**, similar cum un proces este identificat printr-un *process identifier* (PID).

Procesele și fișierele unui utilizator au în atributele acestora câmpul UID. Pentru procese, acest câmp oferă informații despre ce operații asupra resurselor sistemului de operare pot fi executate de proces, în timp ce câmpul UID al fișierelor identifică utilizatorul care le deține. În Listting 5.8 afișăm UID-ul proceselor folosind comanda **ps** și listăm UID-ul unui fișier folosind comand **stat**. Observăm că UID-ul utilizatorului `student` este `1000`: liniile `3`, `4` și `9`.

```
1  student@uso:~$ ps -o pid,ppid,uid,user,cmd
2    PID  PPID   UID USER     CMD
3  24104 24103  1000 student  -bash
4  24610 24104  1000 student  ps -o pid,ppid,uid,user,cmd
5  student@uso:~$ stat vm-actions-log.txt
6    File: vm-actions-log.txt
7    Size: 4827            Blocks: 16         IO Block: 4096   regular file
8  Device: 801h/2049d      Inode: 659107      Links: 1
9  Access: (0644/-rw-r--r--)  Uid: ( 1000/ student)   Gid: ( 1000/ student)
10 Access: 2020-08-08 16:32:44.086193999 +0300
11 Modify: 2018-08-21 14:37:44.316527432 +0300
12 Change: 2018-08-21 14:37:44.320529432 +0300
13  Birth: -
```

Listing 5.8: Afișare UID pentru procese și fișiere

De obicei, un proces poate avea acces la o resursă dacă UID-ul procesului este marcat în cadrul resursei ca având acces.

5.3.1 Atribute ale utilizatorilor. Informații despre utilizatori

Un utilizator are atribute care îi permit identificarea în sistem, autentificarea și accesul la resurse. Așa cum am precizat mai sus, cele mai importante două atribute sunt numele de utilizator (*username*) și identificatorul de utilizator (*user identifier*, UID).

Un utilizator de sistem are acces la un director home (un subdirector în sistemul de fișiere unde are acces complet). Pentru utilizatorul `student` acesta este, în mod uzual, `/home/student` (pe Linux), `/Users/student` (pe macOS) sau `C:\Users\student` (pe Windows).

Pentru autentificare, un utilizator are asociată o parolă sau o altă formă de autentificare. Aceasta trebuie furnizată la un prompt de login (interfață în linia de comandă sau grafică) pentru a fi permis accesul în sistem în contul acelui utilizator.

Pe sistemele Unix, un utilizator are asociat un shell de autentificare (*login shell*). Este programul care este rulat în momentul autentificării utilizatorului pe un sistem; programul este, de obicei, un shell în linia de comandă.

Pentru gestiunea mai flexibilă a accesului la resurse, **utilizatorii sunt grupați**. Un utilizator face parte din unul sau mai multe grupuri. Astfel că, un administrator care dorește ca mai mulți utilizatori să aibă acces la o anumită resursă va crea un grup, va adăuga utilizatorii la acel grup și va oferi grupului permisiuni asupra resursei respective. Un grup poate conține 0 sau mai mulți utilizatori, iar un utilizator poate aparține unui sau mai multor grupuri, așa cum este indicat în Figura 5.3.

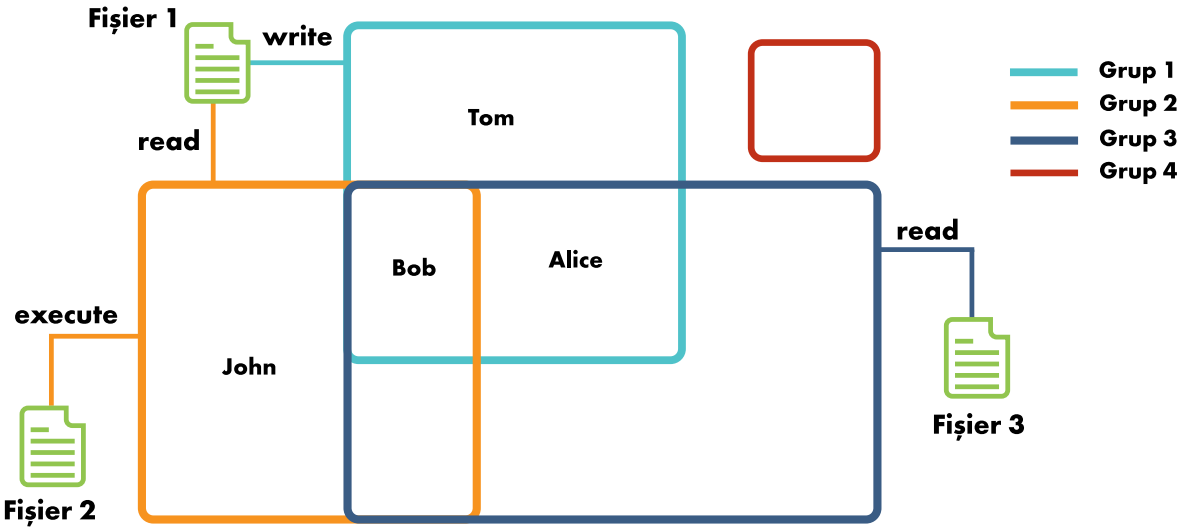


Figura 5.3: Asocierea utilizatori la grupuri

De exemplu, în Windows, există grupul `Administrators`. Un utilizator care este parte din grupul `Administrators` are acces complet la sistem.

Aceste atribute sunt definite în fișiere de configurare sau de tip bază de date. Aceste atribute pot fi, de obicei, vizualizate de orice utilizator, dar modificate doar de un utilizator privilegiat. Mai multe informații prezentăm în Secțiunea 5.3.1

5.3.2 Obținerea de informații despre utilizatori

Informațiile despre utilizatori (nume de utilizator, identificator de utilizator, director home) pot fi obținute folosind interfețele grafice aferente unui sistem de operare sau distribuții sau folosind utilitare în linia de comandă.

În Windows, pentru obținerea de informații despre utilizatori folosim în mod obișnuit interfața grafică: *Control Panel* → *User Accounts* → *User Accounts*.

5.3.2.1 Obținerea de informații despre utilizatori în Linux

În Linux, informațiile despre utilizatori sunt stocate în fișiere de configurare dedicate. În scenarii de administrare mai complexe (care nu fac subiectul acestei cărți), aceste informații sunt stocate într-o baza de date. Fișierele de configurare în care sunt reținute informații despre utilizatori sunt indicate în Tabelul 5.1.

Tabelul 5.1: Informații despre utilizatori

Fișier	Rol	Informații
`/etc/passwd`	informații utilizatori	nume de utilizator, UID, director home, shell de login, GID
`/etc/shadow`	parole utilizatori	nume de utilizator, parolă criptată, informații expirare parolă
`/etc/group`	informații grupuri	nume grup, GID, utilizatori aferenți

Fișierele de configurare au formă tabelară cu coloane: fiecare linie este o intrare în tabel iar fiecare coloană este un atribut al acelei intrări. Mai jos sunt prezentate câteva linii din fiecare fișier de configurare și explicațiile pentru rolul acestora. Detalii despre fișierele `/etc/passwd` și `/etc/shadow` prezentăm în Secțiunea 5.4.

Putem obține informații despre utilizatori direct din fișierele de configurare de mai sus, așa cum facem în Listing 5.9.

```
1  student@uso:~$ grep student /etc/passwd
2  student:x:1000:1000:Student User,,,:/home/student:/bin/bash
3  student@uso:~$ grep student /etc/group
4  adm:x:4:syslog,student
5  cdrom:x:24:student
6  sudo:x:27:student
7  dip:x:30:student
8  plugdev:x:46:student
9  lpadmin:x:116:student
10 student:x:1000:
11 sambashare:x:126:student
12 student@uso:~$ sudo grep student /etc/shadow
13 [sudo] password for student:
14 student:$6$VNq0Dxyc$[...].:17749:0:99999:7:::
```

Listing 5.9: Investigarea fișierelor de configurare pentru utilizatori

Altfel, informații despre utilizatori obținem folosind utilitare precum cele din Tabelul 5.2. Aceste utilitare investighează fișierele de configurare din Tabelul 5.1.

Tabelul 5.2: Utilitare de investigare utilizatori

Utilitar	Rol	Fișiere investigate
id	informații despre utilizator	/etc/passwd, /etc/group
groups	grupurile utilizatorului curent	/etc/group
users, **w**, **who**	utilizatorii autentificați în sistem acum	/var/run/utmp
whoami	numele utilizatorului curent	N/A
finger, **pinky**	informații complete despre un utilizator	/etc/passwd, /etc/group

În Listing 5.10 sunt câteva exemple de folosire ale comenzilor de mai sus.

```
1  student@uso:~$ id
2  uid=1000(student) gid=1000(student) groups=1000(student),4(adm),24(cdrom)
   ,27(sudo),30(dip),46(plugdev),116(lpadmin),126(sambashare)
3  student@uso:~$ id teacher
4  uid=1001(teacher) gid=1001(teacher) groups=1001(teacher)
5  student@uso:~$ groups
6  student adm cdrom sudo dip plugdev lpadmin sambashare
7  student@uso:~$ groups teacher
8  teacher : teacher
9  student@uso:~$ users
10 student student
11 student@uso:~$ whoami
12 student
13 student@uso:~$ su - teacher -c whoami
14 Password:
15 teacher
16 student@uso:~$ finger
17 Login     Name           Tty      Idle  Login Time   Office     Office
   Phone
18 student   Student User  *:0              Sep  4 17:42 (:0)
19 student   Student User   pts/0           Sep 19 20:05 (192.168.56.1)
20 student@uso:~$ finger teacher
21 Login: teacher                          Name: Teacher User
22 Directory: /home/teacher                Shell: /bin/bash
23 Never logged in.
24 No mail.
25 No Plan.
26 student@uso:~$ pinky
27 Login    Name                 TTY      Idle   When             Where
28 student  Student User        ?:0       ?????  2020-09-04 17:42 :0
29 student  Student User         pts/0           2020-09-19 20:05
   192.168.56.1
```

Listing 5.10: Investigarea utilizatorilor prin comenzi

În Linux, anumiți utilizatori au configurat ca shell de login **/bin/false** sau **/bin/nologin**. Aceasta înseamnă că acești utilizatori nu se pot autentifica în sistem și nu pot obține un shell de login. Acești utilizatori mai sunt numiți și utilizatori de sistem, iar ceilalți sunt numiți utilizatori interactivi. Utilizatorii de sistem sunt cei care pot

avea procese dar nu pot obține shell de login. Cei interactivi pot obține shell și pot rula apoi comenzi. La o instalare simplă, o foarte bună parte dintre utilizatorii unui sistem Linux sunt utilizatori de sistem, așa cum se observă în Listing 5.11.

```
1  student@uso:~$ grep /bin/false /etc/passwd
2  speech-dispatcher:x:111:29:Speech Dispatcher,,,:/var/run/speech-
   dispatcher:/bin/false
3  whoopsie:x:112:117::/nonexistent:/bin/false
4  hplip:x:118:7:HPLIP system user,,,:/var/run/hplip:/bin/false
5  gnome-initial-setup:x:120:65534::/run/gnome-initial-setup/:/bin/false
6  gdm:x:121:125:Gnome Display Manager:/var/lib/gdm3:/bin/false
7  vboxadd:x:999:1::/var/run/vboxadd:/bin/false
8
9  student@uso:~$ grep nologin /etc/passwd
10 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
11 bin:x:2:2:bin:/bin:/usr/sbin/nologin
12 sys:x:3:3:sys:/dev:/usr/sbin/nologin
13 games:x:5:60:games:/usr/games:/usr/sbin/nologin
14 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
15 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
16 [...]
```

Listing 5.11: Afișarea utilizatorilor de sistem

5.3.3 Gestiunea utilizatorilor și grupurilor

Gestiunea utilizatorilor și grupurilor, însemnând crearea, ștergerea sau modificarea utilizatorilor și grupurilor poate fi făcută, în mod normal, doar de un utilizator privilegiat. Operațiile de gestiune a utilizatorilor sunt:

- crearea unui utilizator: un nou cont de utilizator, identificat de un nume de utilizator (*username*) și un identificator de utilizator (UID) este disponibil în sistem
- ștergerea unui utilizator: un cont de utilizator este șters din sistem, nu mai pot fi create procese cu acest utilizator
- crearea unui grup: este creat un grup nou în care pot fi adăugați utilizatori
- ștergerea unui grup: este șters un grup existent
- adăugarea unui utilizator la un grup
- ștergerea unui utilizator dintr-un grup
- schimbarea parolei unui utilizator
- modificarea atributelor unui utilizator: username, login shell, director home

Aceste operații de gestiune duc la modificarea fișierelor de configurare sau de tip baze de date din sistem care rețin informațiile despre grupuri și utilizatori. Operațiile sunt realizate prin utilitare specifice fie din interfața grafică fie din linie de comandă, ca în Listing 5.12.

5.3.4 Gestiunea utilizatorilor și grupurilor în Windows

În Windows, gestiunea utilizatorilor și grupurilor o facem din interfața din panoul de control. Aceasta este uzual accesibilă din *Control Panel* → *User Accounts* → *User Accounts*, ca în Figura 5.4.

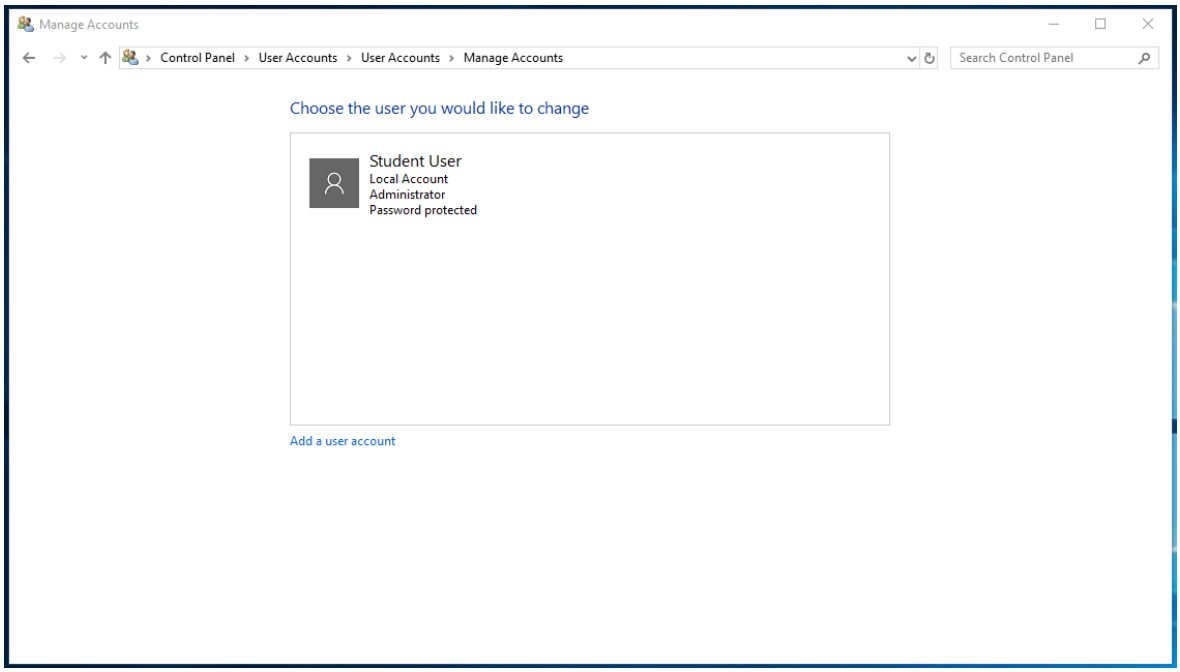


Figura 5.4: Gestiunea utilizatorilor în Windows

Pe lângă interfața grafică, uzuală, Windows oferă și utilitare în linia de comandă pentru gestiunea utilizatorilor folosind interfața **`netsh`**[1].

5.3.4.1 Gestiunea utilizatorilor și grupurilor în Linux

În Linux există interfețe grafice pentru gestiunea utilizatorilor în funcție de mediul desktop folosit (GNOME, KDE, XFCE etc.) În mod tradițional, gestiunea se face în linia de comandă cu ajutorul utilitarelor specifice, indicate în Tabelul 5.3.

Pe distribuțiile bazate pe Debian (Debian, Ubuntu, Linux Mint etc.) există un set de utilitare mai ușor de folosit care înglobează comenzile de mai sus, indicate în Tabelul 5.4

În Linux/Unix există două tipuri de grupuri: **grupuri primare** și **grupuri secundare**. Un utilizator aparține unui singur grup primar și oricâtor grupuri secundare. Când un utilizator este creat, i se precizează grupul primar aferent. Utilitarul usermod poate fi folosit pentru modificarea grupurilor din care face parte un utilizator; utilitarul are opțiuni distincte pentru modificarea grupului primar și pentru modificarea grupurilor secundare.

Un utilizator neprivilegiat are posibilitatea de a-și modifica anumite informații proprii: shellul de login, numele de utilizator, datele personale, parola. Pentru aceasta folosește utilitarele din Tabelul 5.5.

[1] https://docs.microsoft.com/en-us/windows-server/networking/technologies/netsh/netsh-contexts

Tabelul 5.3: Utilitare de gestiune a utilizatorilor

Operație	Utilitare	Fișiere modificate
adăugare utilizator	**useradd**	/etc/passwd, /etc/shadow, /etc/group
ștergere utilizator	**userdel**	/etc/passwd, /etc/shadow, /etc/group
modificare utilizator	**usermod**	/etc/passwd, /etc/shadow, /etc/group
adăugare grup	**groupadd**	/etc/group
ștergere grup	**groupdel**	/etc/group
modificare grup	**groupmod**	/etc/group
modificare shell	**chsh**	/etc/passwd
modificare informații utilizator	**chfn**	/etc/passwd
schimbare parolă	**passwd**	/etc/shadow

Tabelul 5.4: Utilitare wrapper Debian

Utilitar wrapper	Utilitar de bază
adduser	**useradd**
deluser	**userdel**
addgroup	**groupadd**
delgroup	**groupdel**

Tabelul 5.5: Utilitare pentru modificare informații proprii

Operație	Utilitar
schimbare shell de login	**chsh**
schimbare informații	**chfn**
schimbare parolă	**passwd**

Un grup care este grupul primar al unui utilizator nu poate fi șters. Întâi trebuie să fie șters utilizatorul, apoi grupul.

În Listing 5.12 este prezentat un scenariu de lucru cu utilizatori și grupuri în care sunt folosite comenzile specifice din Linux.

```
1 student@uso:~$ sudo groupadd courses
2 student@uso:~$ sudo adduser teacher courses
```

```
Adding user ‘teacher’ to group ‘courses’ ...
Adding user teacher to group courses
Done.
student@uso:~$ users teacher
student@uso:~$ id teacher
uid=1001(teacher) gid=1001(teacher) groups=1001(teacher),1002(courses)
student@uso:~$ sudo adduser george
[...]
student@uso:~$ id george
uid=1002(george) gid=1003(george) groups=1003(george)
student@uso:~$ sudo adduser george courses
Adding user ‘george’ to group ‘courses’ ...
Adding user george to group courses
Done.
student@uso:~$ id george
uid=1002(george) gid=1003(george) groups=1003(george),1002(courses)
student@uso:~$ sudo deluser george courses
Removing user ‘george’ from group ‘courses’ ...
Done.
student@uso:~$ id george
uid=1002(george) gid=1003(george) groups=1003(george)
student@uso:~$ sudo delgroup courses
Removing group ‘courses’ ...
Done.
student@uso:~$ id teacher
uid=1001(teacher) gid=1001(teacher) groups=1001(teacher)
student@uso:~$ sudo deluser george
Removing user ‘george’ ...
Warning: group ‘george’ has no more members.
Done.
student@uso:~$ id george
id: ’george’: no such user
```

Listing 5.12: Gestiunea utilizatorilor și grupurilor în Linux

Gestiunea parolelor este o componentă critică a sistemului de operare și o vom prezenta în secțiunea de mai jos.

5.4 Autentificare. Gestiunea parolelor

Atunci când un utilizator dorește accesul la sistemul de operare trebuie să se autentifice. Autentificarea presupune, în mod tipic, furnizarea unui nume de utilizator (*username*) și a unei parole (*password*). Acest lucru este valabil și pentru aplicații (aplicații web, jocuri, aplicații de baza de date, conturi bancare). Există și alte moduri de autentificare precum folosirea de informații biometrice (amprentă digitală, retină), dispozitiv de tip token de autentificare, autentificare cu cheie publică. În această secțiune vom prezenta doar autentificarea pe bază de parole.

5.4.1 Baza de date de parole

Pentru ca autentificarea să reușească, adică utilizatorului să îi fie permis accesul în sistem, trebuie ca numele de utilizator și parola să fie valide. Sistemul de operare deține

o bază de date cu perechi *(nume de utilizator, parolă)*. Autentificarea reușește dacă numele de utilizator și parola introduse de utilizator se găsesc în baza de date.

Baza de date de parole este modificată de acțiunile de lucru cu utilizatori. Adică:

- Se adaugă o nouă intrare în baza de date de parole, adică o nouă pereche *(nume de utilizator, parolă)* în momentul adăugării unui utilizator nou în sistem.
- Se șterge o intrare existentă în baza de date de parole în momentul ștergerii unui utilizator în sistem.
- Se modifică o intrare existentă în baza de date atunci când un utilizator își schimbă parola.

Baza de date de parole este o componentă critică a sistemului iar accesul la aceasta trebuie să fie permis numai pentru acțiuni bine definite: autentificare, adăugarea/ștergerea unui utilizator, schimbarea parolei. Altfel, dacă ar fi permise și alte acțiuni, un utilizator neprivilegiat ar accesa baza de date și ar afla parolele altor utilizatori și ar putea folosi conturile acestora, cel mai periculos fiind folosirea conturilor de utilizatori privilegiați. De aceea, accesul la baza de date este controlat strict.

Chiar și cu un control strict, se poate întâmpla ca un defect în sistem să permită unui utilizator neprivilegiat accesul la baza de date de parole. De aceea parolele nu sunt reținute în format citibil (*plain text*), ci sunt criptate. Dacă un atacator obține în mod nevalid accesul la baza de date de parole, acesta va trebui să decripteze parolele, lucru dificil de realizat. Atacuri celebre în lume au avut ca rezultat obținerea bazelor de date de parole[1], deci criptarea acestora e foarte importantă.

În Linux, baza de date de utilizatori este reprezentată de fișierul `/etc/passwd`. Întrucât multe comenzi au nevoie de accesul la acest fișier pentru a obține UID-ul când se știe numele de utilizator sau pentru a afla shellul utilizatorului curent, acest fișier nu poate reține parola. Dacă ar fi așa, orice comenzi uzuale ar putea obține parolele utilizatorilor. Baza de date de parole este reprezentată în Linux de fișierul `/etc/shadow`. Fișierul `/etc/shadow` poate fi accesat doar de utilizatori privilegiați așa cum putem vedea în Listing 5.13.

```
1 student@uso:~$ ls -l /etc/passwd
2 -rw-r--r-- 1 root root 2620 Sep 19 21:08 /etc/passwd
3 student@uso:~$ ls -l /etc/shadow
4 -rw-r----- 1 root shadow 1573 Sep 19 21:08 /etc/shadow
```

Listing 5.13: Permisiuni de acces pe fișierele de parole

O intrare în fișierul `/etc/shadow` conține numele de utilizator, parola criptată și informații legate de gestiunea parolei, așa cum se observă în Listing 5.14.

```
1 student:$6$VNq0Dxyc$[...].:17749:0:99999:7:::
```

Listing 5.14: Conținutul fișierului /etc/shadow

Pentru informații detaliate despre formatul intrării și al parolei criptate, consultați paginile de manual folosind comenzile **`man shadow`** și **`man crypt`**.

În Windows, baza de date de parole se numește Security Account Manager (SAM), și se găsește în `\%SystemRoot%\system32\config\SAM`. Accesul la SAM are loc prin

[1] https://en.wikipedia.org/wiki/List_of_data_breaches

intermediul Windows Registry și este, similar cu `/etc/shadow`, permis numai acțiunilor bine definite. Similar cu `/etc/shadow`, parolele sunt reținute criptate.

5.4.2 Gestiunea parolelor

Un utilizator își poate gestiona parola proprie, iar un utilizator administrativ poate gestiona parola tuturor utilizatorilor. Gestiunea parolelor înseamnă în mod uzual schimbarea lor dar sunt și alte acțiuni. O parolă are asociată o durată de viață după care expiră; odată expirată, o parolă trebuie să fie schimbată. Altă acțiune legată de parole este blocarea parolei, astfel încât autentificarea cu parolă devine inaccesibilă. Sumarizând, există următoarele acțiuni de gestiune a parolelor:

- schimbarea parolei
- stabilirea duratei de viață a unei parole
- blocarea parolei

În Windows se folosește interfața grafică pentru gestiunea parolelor. Se accesează *Control Panel* → *User Accounts* → *User Accounts* → *Manage Accounts* → *Change an Account* → *Channge password* ca în Figura 5.5.

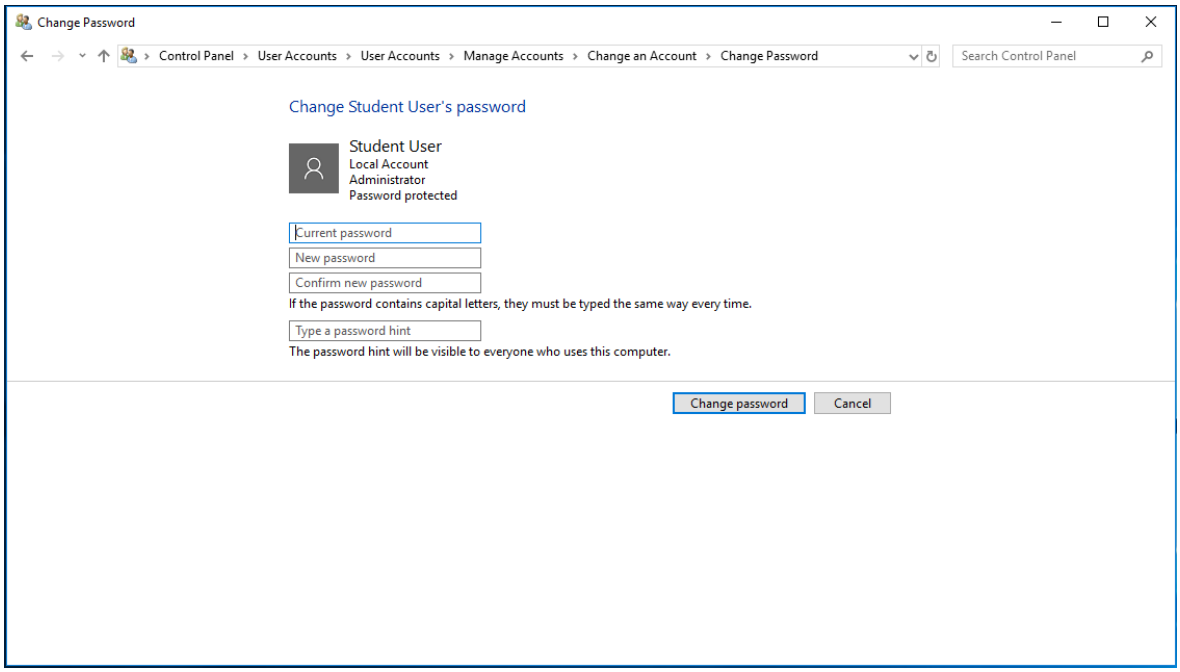


Figura 5.5: Gestiunea parolelor în Windows

În Linux, modul uzual de gestiune a parolei este prin linia de comandă folosind utilitarul **`passwd`**. Cu ajutorul utilitarului **`passwd`**, un utilizator neprivilegiat își poate gestiona parola proprie, iar unul privilegiat parola tuturor utilizatorilor. Atunci când un utilizator neprivilegiat își gestionează parola folosind utilitarul **`passwd`** trebuie să introducă parola la promptul afișat. În Listing 5.15 avem exemple de folosire a comenzii **`passwd`** pentru gestiunea conturilor.

```
1 student@uso:~$ passwd
2 Changing password for student.
```

```
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully

root@uso:~# passwd teacher
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@uso:~#
```

Listing 5.15: Gestiunea conturilor în Linux folosind passwd

Expirarea parolelor și blocarea parolelor poate fi utilă pentru crearea de conturi temporare. Cineva primește un cont și apoi nu mai poate să acceseze prin parolă acel cont pentru că nu mai este parte a unei organizații, are de plătit o sumă de bani sau altă situație.

De avut în vedere că o parolă blocată nu înseamnă că acel cont nu poate fi folosit. Un utilizator se poate autentifica la acel cont folosind alte forme de autentificare, precum autentificare cu chei publice SSH. Dacă dorim să dezactivăm acel cont și să nu mai poată fi folosit cu nici o formă de autentificare, putem folosi comanda **`usermod`** ca în Listing 5.16.

```
student@uso:~$ sudo usermod --expiredate 1 teacher
student@uso:~$ su - teacher
Password:
Your account has expired; please contact your system administrator
su: Authentication failure
student@uso:~$
```

Listing 5.16: Dezactivarea unui cont

Comanda configurează contul să expire la *1 ianuarie 1970* făcându-l, astfel, inutilizabil.

Utilitarul **`passwd`** este utilitar interactiv, necesită intrarea de la utilizator. Ceea ce îl face dificil de folosit pentru automatizare; de exemplu, dacă dorim să creăm `100` de conturi pe un sistem cu anumite parole. Pentru aceasta putem folosi utilitarul **`chpasswd`**, ca în Listing 5.17.

```
student@uso:~$ echo "teacher:woof2PhaiNiphooS" | sudo chpasswd
```

Listing 5.17: Schimbarea neinteractivă a parolei folosind chpasswd

Utilitarul **`chpasswd`** primește la intrarea standard linii de forma `username:password` și le schimbă în mod neinteractiv, folosibil în scripturi. Mai sus am schimbat parola utilizatorului `teacher` la valoarea `woof2PhaiNiphooS`.

Dincolo de folosirea lor într-un sistem de operare, pentru o persoană, parolele sunt importante și trebuie avut grijă cum se folosesc. Așa cum vom discuta detaliat în Secțiunea 12.3.1, folosirea parolelor poate fi problematică pentru că:

- folosim foarte multe parole și, în mod ideal, trebuie să fie diferite între ele
- parolele pot fi slabe și ușor de ghicit
- parolele trebuie să fie schimbate periodic, altfel șansele de fi ghicite/decriptate cresc

- folosim parolele de pe dispozitive multiple: desktop, laptop, dispozitiv mobil

Pentru o gestiune corespunzătoare a parolelor există următoarele recomandări:

- Parolele să fie lungi (cel puțin 10 caractere) și să conțină caractere diferite. Puteți apela la un generator de parole precum utilitarul **`pwgen`**.
- Nu notați parolele în clar pe hârtii sau în fișiere de unde pot fi ușor citite.
- Folosiți un utilitar de tipul *password manager* care să rețină parolele și să minimizeze riscul expunerii lor la un atacator. În același timp, să ajute să fie accesibile de pe dispozitive multiple.

5.4.3 Autentificare centralizată

De multe ori în cadrul unei organizații există mai multe tipuri de aplicații și interfețe pe care le folosim: un site web, un wiki, un cont Linux, un site pentru cursuri, un sistem de gestiune a codului sursă. Este problematic pentru un utilizator să aibă un cont și o parolă diferită pentru fiecare acest tip de aplicație; utilizatorul poate uita din parole, poate le va nota undeva, sau le va face simple ca să le țină minte.

Soluția la această situație este autentificarea centralizată. Autentificarea centralizată presupune existența unui serviciu de autentificare care gestionează baza de date de parole, serviciu accesat de diferitele aplicații, ca în Figura 5.6.

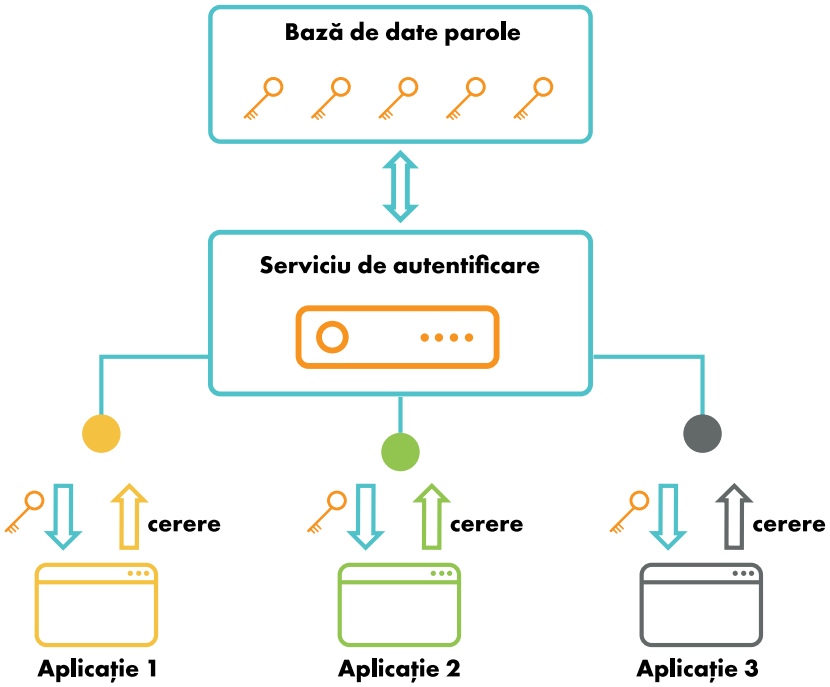


Figura 5.6: Autentificare centralizată

În situația autentificării centralizate, un utilizator folosește același nume de utilizator și aceeași parolă pentru toate aplicațiile. Aplicațiile trimit mai departe numele de utilizator și parola către serviciul de autentificare și, dacă intrările respective sunt valide în baza de date, oferă accesul utilizatorului la aplicație.

În lumea Windows autentificarea centralizată se bazează pe serviciul AD (*Active Directory*) iar în lumea Linux se bazează pe serviciul LDAP (*Lightweight Directory Access Protocol*). Aceste subiecte sunt avansate și nu fac subiectul acestei cărți. Important este înțelegerea nevoii de autentificare centralizată și principalele soluții pentru asigurarea acesteia.

5.5 Accesul la sistemul de fișiere

Conturile de utilizator sunt folosite în sistemul de operare pentru a separa resursele la care are acces fiecare utilizator. Una dintre cele mai folosite resurse este sistemul de fișiere; fiecare utilizator dispune de un director home la care are permisiuni complete.

În general, unui utilizator i se asociază permisiuni pentru o anumită resursă. De exemplu, utilizatorul `ana` poate modifica fișierul `note.xls` dacă îi sunt asociate permisiuni de modificare ale acelui fișier. La nivel conceptual, un agent sau un subiect are anumite permisiuni de acces al unei resurse sau unui obiect, așa cum este descris în Figura 5.7.

Schema de mai jos este o schemă generală. În cazul particular al secțiunii curente, agentul este utilizatorul, iar resursa este fișierul. Mai precis, agentul este un proces aparținând unui utilizator. Spunem că un proces poate accesa un anumit fișier dacă utilizatorul căruia aparține procesul are permisiunile aferente la acel fișier.

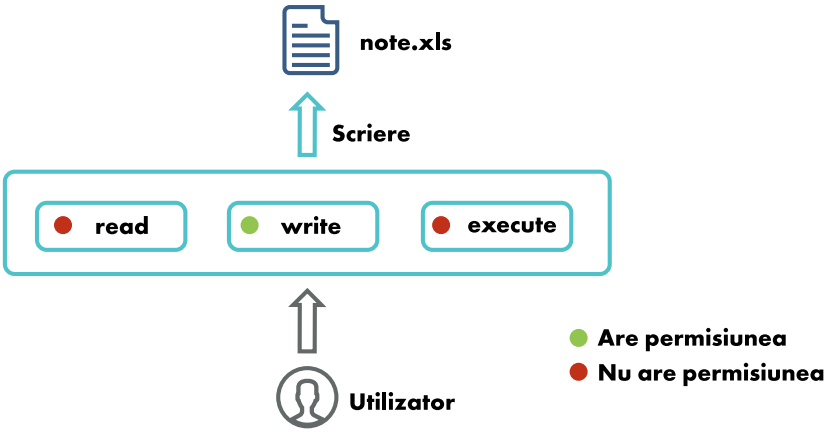


Figura 5.7: Utilizatori, fișiere, permisiuni

În multe dintre sistemele de operare moderne, informațiile despre permisiuni sunt reținute în fiecare fișier. În cadrul fișierului se reține metadate, printre care și permisiunile de acces pentru utilizatori. Modul în care sunt structurate aceste metadate ține de sistemul de operare.

5.5.1 Gestiunea permisiunilor în Windows

În Windows permisiunile sunt reținute în forma unor liste de access (*access control lists*, ACL). Adică se creează o listă de utilizatori sau grupuri de utilizatori și permisiuni aferente, ca în Figura 5.8.

În momentul accesului unui fișier de către un proces, se obține utilizatorul căruia aparține procesul. Apoi, se caută utilizatorul în lista de acces a fișierului. Dacă există și are

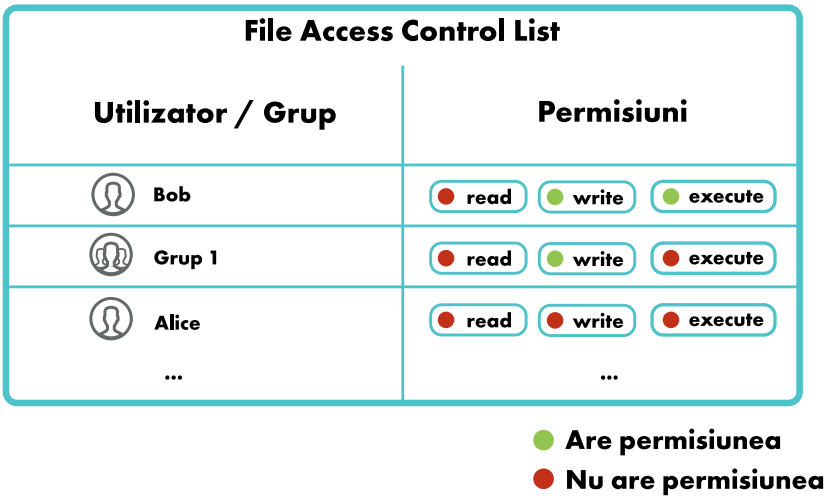


Figura 5.8: Liste de acces (Windows)

asociate permisiuni adecvate, atunci se permite accesul procesului la fișier. Dacă nu există sau dacă nu are permisiuni, accesul nu este permis. De obicei în Windows, există o intrare în lista de acces numită `Everyone` care precizează permisiunile pentru toți utilizatorii. Dacă, în urma căutării, un utilizator nu este găsit în listă, atunci se folosesc permisiunile pentru intrarea `Everyone`. Figura 5.9 verifică modul în care sunt analizate permisiunile în Windows.

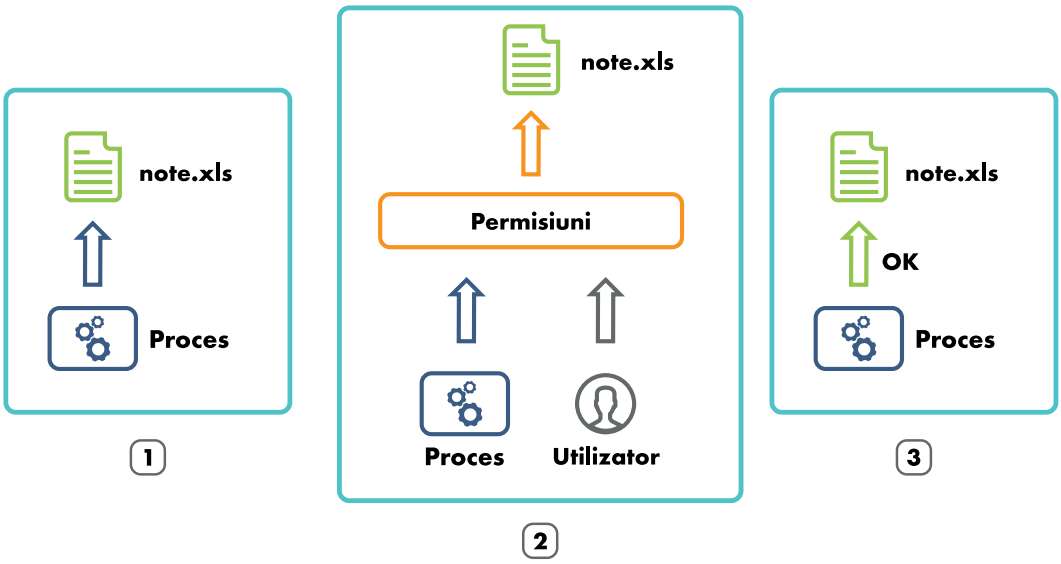


Figura 5.9: Verificarea permisiunilor în Windows

În Windows permisiunile sunt prezente doar pe sistemul de fișiere NTFS, nu și pe FAT32. Permisiunile sunt în mod tipic configurate folosind interfața grafică: *click dreapta pe fișier* → *Properties* → *Security*, obținând o fereastră precum cea din Figura 5.10.

Pentru un utilizator dat pot fi precizate mai multe tipuri de permisiuni, dintre care amintim:

- `List Folder/Read Data`: utilizatorul poate lista conținutul unui director și datele unui fișier
- `Create Files/Write Data`: utilizatorul poate crea fișiere într-un director și poate scrie date într-un fișier

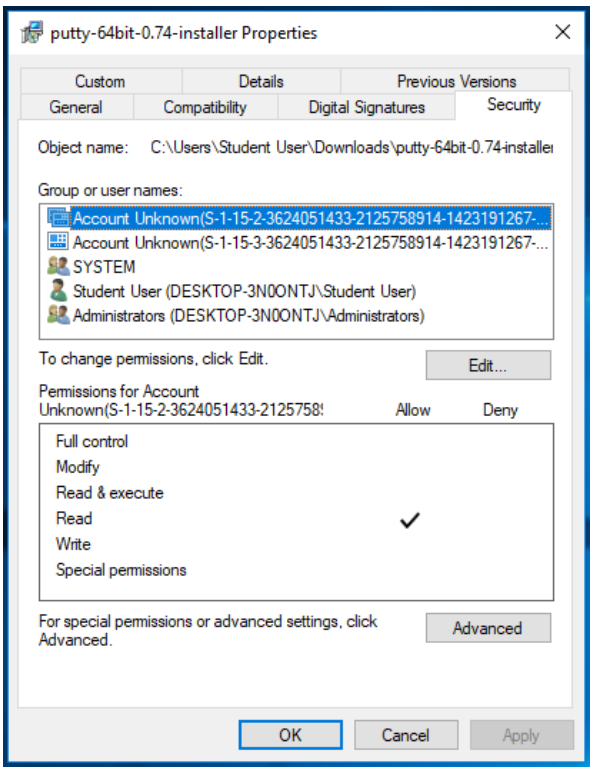


Figura 5.10: Permisiuni în Windows

- `Traverse Folder/Execute File`: utilizatorul poate traversa un director și poate executa un fișier

Aceste permisiuni sunt similare permisiunilor `read`, `write` și `execute` pe care le vom prezenta în continuare pentru Linux. O listă completă a permisiunilor în Windows, pentru NTFS, găsiți pe site-ul care prezintă sistemul de fișiere NTFS[1].

5.5.2 Gestiunea permisiunilor în Linux

În Linux fiecare fișiere are metadate. Metadatele relevante pentru permisiuni sunt:

- `user` (utilizator): utilizatorul fișierului
- `group` (grup): grupul fișierului
- permisiuni: informații despre ce tipuri accese sunt permise și cui

În Linux permisiunile sunt reținute într-un mod simplificat. Există 3 entități pentru care se pot configura permisiuni și 3 tipuri de permisiuni, ținând cont de metadatele fișierului. Entitățile sunt:

- `user` (utilizator): utilizatorul fișierului
- `group` (grup): grupul fișierului
- `others` (ceilalți): ceilalți utilizatori, diferiți de utilizatorul fișierului și care nu fac parte din grupul fișierului

[1] http://www.ntfs.com/ntfs-permissions-file-folder.htm

Permisiunile în Linux sunt, de asemenea, simplificate la 3 tipuri:

- `read` (citire): fișierul poate fi citit, directorul poate fi listat
- `write` (scriere): fișierul poate fi modificat, pot fi adăugate / șterse intrări în director
- `execute` (execuție): fișierul poate fi executat, directorul poate fi parcurs

Permisiunile în Linux sunt verificate în cazul în care un proces accesează un fișier, urmând pașii:

1. Se obține utilizatorul procesului care accesează fișierul.
2. Se compară utilizatorul procesului cu utilizatorul fișierului. Dacă sunt aceiași atunci se obțin permisiunile pentru `user` (utilizator) și se verifică dacă acțiunea procesului este acceptată de permisiuni. Dacă da, accesul este permis; dacă nu, se trece la pasul următor.
3. Se obțin toate grupurile din care face parte utilizatorul procesului. Se compară, pe rând, fiecare dintre aceste grupuri cu grupul fișierului. Dacă există un grup care să corespundă grupului fișierului, atunci se obțin permisiunile pentru `group` (grup) și se verifică dacă acțiunea procesului este acceptată de permisiuni. Dacă da, accesul este permis; dacă nu, se trece la pasul următor.
4. Se obțin permisiunile pentru `others` (ceilalți) și se verifică dacă acțiunea procesului este acceptată de permisiuni. Dacă da, accesul este permis; dacă nu, accesul nu este permis.

Fie următoarea situație de metadate pentru un fișier `curs.txt`:

- utilizator: `ana`
- grup: `students`
- permisiuni: citire, scriere pentru utilizator, citire pentru grup, nici o permisiune pentru ceilalți

Utilizatorul `corina` încearcă accesarea pentru citire a fișierului folosind comanda **`cat curs.txt`**. Presupunem că utilizatorul `corina` aparține mai multor grupuri între care și `students`, grupul fișierului `curs.txt`. Se urmează pașii:

1. Se obține utilizatorul procesului care accesează procesul: `corina`.
2. Se compară utilizatorul procesului (`corina`) cu utilizatorul fișierului (`ana`). Nu corespund. Se trece la pasul următor.
3. Se obțin toate grupurile din care face parte utilizatorul procesului (`corina`). Grupul fișirului `curs.txt` este `students`, grup din care face și utilizatorul `corina`. În această situație verificăm dacă acțiunea este permisă. Acțiunea este de citire, se găsește în permisiunile pentru grup (citire), și atunci comanda **`cat curs.txt`** reușește.

În situația de mai sus, dacă am fi rulat o comandă care modifica fișierul `curs.txt`, nu am fi reușit, pentru că permisiunile pentru grup nu includeau *scriere*.

Mai jos prezentăm ce permisiuni sunt necesare pentru a putea rula comenzi de lucru cu sistemul de fișiere:

- Pentru a putea traversa un director, pentru ca acesta să fie parte a unei căi este suficient ca acesta să aibă permisiuni de execuție. Adică dacă dorim să funcționeze comanda **`cd /a/b/c/d/`**, trebuie ca directoarele `a`, `b`, `c` și `d` să aibă permisiuni de execuție.
- Pentru a putea lista conținutul unui director, adică folosirea comenzii **`ls`**, trebuie ca acel director să aibă permisiuni de citire și de execuție. Pe un director care are doar permisiuni de citire nu se poate folosi comanda **`ls`**.
- Pentru a putea afișa conținutul unui fișier, trebuie ca acel fișier să aibă permisiuni de citire iar directorul care-l conține să aibă permisiuni de execuție.
- Pentru a putea edita un fișier, trebuie ca acel fișier să aibă permisiuni de scriere (nu neapărat și de citire) iar directorul care-l conține să aibă permisiuni de execuție.
- Pentru a putea șterge un fișier sau un director, trebuie ca acel fișier sau director să se găsească într-un director care să aibă permisiuni de scriere și execuție.
- Pentru a putea crea un fișier sau un director, trebuie ca acel fișier sau director să fie create într-un director care să aibă permisiuni de scriere și execuție.
- Pentru a putea rula un fișier executabil, trebuie ca acel fișier să aibă permisiuni de citire și execuție iar directorul care-l conține să aibă permisiuni de execuție.

5.5.2.1 Formatul permisiunilor

În mod uzual, permisiunile sunt listate pentru un fișier dat în forma din Figura 5.11.

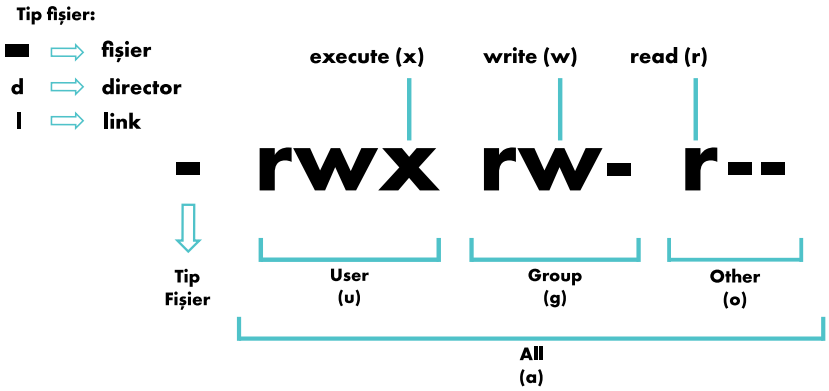


Figura 5.11: Permisiuni în Linux

Adică permisiunile sunt listate ca un șir de nouă caractere:

- primele trei caractere sunt permisiunile pentru `user` (utilizator)
- următoarele trei caractere sunt permisiunile pentru `group` (grup)
- ultimele trei caractere sunt permisiunile pentru `others` (ceilalți)

```
1 student@uso:~$ ls -l invoice.pdf
2 -rw-rw-r-- 1 dan staff 0 Sep 19 21:40 invoice.pdf
3 student@uso:~$ ls -l export_grades
4 -rwxr-x--- 1 dan adm 0 Sep 19 21:41 export_grades
5 student@uso:~$ stat invoice.pdf
6   File: invoice.pdf
```

```
  Size: 0               Blocks: 0          IO Block: 4096   regular empty
 file
Device: 801h/2049d      Inode: 658479      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1002/     dan)   Gid: (   50/   staff)
Access: 2020-09-19 21:40:40.242001000 +0300
Modify: 2020-09-19 21:40:40.242001000 +0300
Change: 2020-09-19 21:41:55.110001000 +0300
 Birth: -
student@uso:~$ stat export_grades
  File: export_grades
  Size: 0               Blocks: 0          IO Block: 4096   regular empty
 file
Device: 801h/2049d      Inode: 683312      Links: 1
Access: (0750/-rwxr-x---)  Uid: ( 1002/     dan)   Gid: (    4/     adm)
Access: 2020-09-19 21:41:26.414001000 +0300
Modify: 2020-09-19 21:41:26.414001000 +0300
Change: 2020-09-19 21:42:00.170001000 +0300
 Birth: -
```

Listing 5.18: Listare permisiuni pe fișiere

Astfel, în Listing 5.18 avem următoarele:

- fișierul `invoice.pdf` are:
  - permisiuni de citire și scriere pentru `user`, adică pentru utilizatorul `dan`
  - permisiuni de citire și scriere pentru `group`, adică pentru grupul `staff`
  - permisiuni de citire pentru ceilalți
- fișierul `export_grades` are:
  - permisiuni de citire, scriere și execuție pentru `user`, adică pentru utilizatorul `dan`
  - permisiuni de citire și execuție pentru `group`, adică pentru grupul `adm`
- nici o permisiune pentru ceilalți

Pentru gestiunea mai rapidă a permisiunilor, acestea sunt gestionate în format pe biți. Adică fiecare dintre cele nouă caractere de permisiuni ocupă un bit; bitul este 1 dacă permisiunea este activă și pe 0 dacă este inactivă. Adică avem corespondențe de forma:

- `rw-r--r--` corespunde `110100100`
- `rwxr-x---` corespunde `111101000`
- `rw-rw----` corespunde `110110000`

Pentru a fi mai rapid, grupurile de biți se grupează câte trei (trei biți pentru `user`, trei biți pentru `group`, trei biți pentru `others`) și cei trei biți se transformă într-o cifră în octal (baza 8); de aceea mai numim reprezentarea octală. Avem, așadar, corespondența:

- `rw-r--r--` corespunde `110100100`, adică `644` în octal
- `rwxr-x---` corespunde `111101000`, adică `750` în octal
- `rw-rw----` corespunde `110110000`, adică `660` în octal

Pentru că este ușor de modificat și rapid de folosit, gestiunea permisiunilor se face, adesea, în format octal.

5.5.3 Comenzi pentru gestiunea permisiunilor

Pentru gestiunea permisiunilor avem comenzi de afișare a metadatelor de fișierului (utilizator, grup și permisiuni) și comenzi de modificare a metadatelor.

Pentru afișarea metadatelor, cele două comenzi folosite sunt **`ls -l`** și **`stat`** ca în Listing 5.18. În rezultatul rulării comenzilor avem utilizatorul, grupul și permisiunile.

Comenzile de modificare sunt **`chown`**, **`chgrp`** și **`chmod`**. Primele două comenzi (**`chown`** și **`chgrp`**) modifică utilizatorul și grupul, iar **`chmod`** modifică permisiunile. Comanda **`chown`** poate modifica și utilizatorul și grupul unui fișier, comanda **`chgrp`** doar grupul. Cele două comenzi pot fi rulate doar de utilizatorul privilegiat; doar acesta poate modifica utilizatorul și grupul unui fișier. Listing 5.19 conține exemple de folosire a comenzilor **`chown`** și **`chgrp`**.

```
student@uso:~$ ls -l  invoice.pdf
-rw-rw-r-- 1 dan staff 0 Sep 19 21:40 invoice.pdf
student@uso:~$ sudo chown student invoice.pdf
student@uso:~$ ls -l  invoice.pdf
-rw-rw-r-- 1 student staff 0 Sep 19 21:40 invoice.pdf
student@uso:~$ sudo chown teacher:gdm invoice.pdf
student@uso:~$ ls -l invoice.pdf
-rw-rw-r-- 1 teacher gdm 0 Sep 19 21:40 invoice.pdf
student@uso:~$ sudo chgrp avahi invoice.pdf
student@uso:~$ ls -l invoice.pdf
-rw-rw-r-- 1 teacher avahi 0 Sep 19 21:40 invoice.pdf
```

Listing 5.19: Modificarea utilizatorului și grupului unui fișier

Comanda **`chmod`** modifică permisiunile fișierelor. Comanda poate fi rulată de utilizatorul privilegiat sau de cel ce deține fișierul (*owner*), adică utilizatorul fișierului; altfel spus, un utilizator își poate modifica permisiunile fișierelor proprii. Comanda de modificare primește o construcție de modificare a permisiunilor și numele fișierului/fișierelor pentru care dorim schimbarea permisiunilor. Listing 5.20 conține exemple de folosire a comenzii **`chmod`**:

```
student@uso:~$ ls -l road-trip.png
-rw-rw-r-- 1 student student 0 Sep 19 21:47 road-trip.png
student@uso:~$ chmod g-w,o+w road-trip.png
student@uso:~$ ls -l road-trip.png
-rw-r--rw- 1 student student 0 Sep 19 21:47 road-trip.png
student@uso:~$ chmod a-r road-trip.png
student@uso:~$ ls -l road-trip.png
--w-----w- 1 student student 0 Sep 19 21:47 road-trip.png
student@uso:~$ chmod u=rw,g=r,o= road-trip.png
student@uso:~$ ls -l road-trip.png
-rw-r----- 1 student student 0 Sep 19 21:47 road-trip.png
student@uso:~$ chmod 644 road-trip.png
student@uso:~$ ls -l road-trip.png
-rw-r--r-- 1 student student 0 Sep 19 21:47 road-trip.png
```

Listing 5.20: Modificarea permisiunilor unui fișîer

Forma în octal a permisiunilor la **`chmod`** este preferată atunci când vrem să configurăm un nou set de permisiuni unui fișier. Când vrem să actualizăm permisiunile și modificările sunt mici, este de preferat să folosim forma în octal a permisiunilor.

5.5.3.1 Permisiuni de creare pentru fișiere

Atunci când creăm un fișier sau director, folosind, de exemplu, comenzile **`touch`** și **`mkdir`**, le sunt asociate metadate implicite la creare. Utilizatorul și grupul fișierului vor fi utilizatorul și grupul procesului, care sunt, de obicei, utilizatorul și grupul primar al utilizatorului care a lansat comanda. Stabilirea permisiunilor la creare ține de **masca de creare a shellului** (numită și `umask`).

Masca de creare a shellului este o valoare pe biți care este folosită astfel:

- Pentru fișiere se face un **ȘI logic** între valoarea `666` (octal) și masca inversată.
- Pentru directoare se face un **ȘI logic** între valoarea `777` (octal) și masca inversată.

Deosebirea între fișiere și directoare apare pentru că directoarele au nevoie de permisiuni de execuție pentru toate operațiile. Iar fișierele nu au nevoie de permisiuni de execuție; dacă este cazul acestea trebuie acordate explicit.

Exemple sunt prezente în Tabelul 5.6.

Tabelul 5.6: Permisiuni de creare fișier în funcție de mască

Mască de creare	Mască inversată	Permisiuni de creare fișier	Permisiuni de creare director
022	755	644	755
002	775	664	775
077	700	600	700

Afișarea și modificarea măștii shellului se face folosind comanda **`umask`**. Fără parametru această comandă afișează masca de creare, iar cu parametru modifică masca de creare, ca în Listing 5.21.

```
1  student@uso:~$ umask
2  0002
3  student@uso:~$ touch ana.txt
4  student@uso:~$ ls -l ana.txt
5  -rw-rw-r-- 1 student student 0 Sep 19 21:50 ana.txt
6  student@uso:~$ mkdir bogdan.dir
7  student@uso:~$ ls -ld bogdan.dir
8  drwxrwxr-x 2 student student 4096 Sep 19 21:50 bogdan.dir
9  student@uso:~$ umask 0022
10 student@uso:~$ umask
11 0022
12 student@uso:~$ touch corina.txt
13 student@uso:~$ ls -l corina.txt
14 -rw-r--r-- 1 student student 0 Sep 19 21:51 corina.txt
15 student@uso:~$ mkdir dan.dir
16 student@uso:~$ ls -ld dan.dir
17 drwxr-xr-x 2 student student 4096 Sep 19 21:51 dan.dir
```

Listing 5.21: Masca de creare a fișierelor (umask)

Tot în Listing 5.21 (liniile `9-17`) sunt exemple de modificare a umask-ului și crearea de fișiere și directoare pentru a vedea cum modificarea valorii umask duce la modificarea măștii de creare a shellului și, astfel, a permisiunilor de creare pentru fișiere și directoare.

5.5.4 Biți speciali de acces: setuid, setgid, sticky

Atunci când afișăm masca de creare a shellului, observăm că sunt patru cifre octale afișate. În mod normal ar trebui să fie trei cifre afișate, cu permisiuni pentru `user`, `group` și `others`. Prima cifră afișată reprezintă un set special de trei de biți numiți `setuid`, `setgid` și `sticky`, și sunt activați în această ordine. Astfel, pentru permisiunile de mai jos avem explicațiile:

- `4755`: este activat bitul `setuid`, utilizatorul are permisiuni complete, grupul are permisiuni de citire și execuție, ceilalți au permisiuni de citire și execuție
- `2755`: este activat bitut `setgid`, celelalte permisiuni sunt ca mai sus
- `1755`: este activat bitul `sticky`, celelalte permisiuni rămân la fel

Bitul `setuid` are sens când este folosit pe fișiere executabile. Adaugă funcționalitatea de set user ID on execution. Când un fișier executabil are activat bitul `setuid`, procesul creat din acest fișier executabil va avea utilizatorul ale acestui fișier (de obicei `root`) nu al utilizatorului care a rulat comanda. Mai concret, dacă rulăm ca utilizator `student` executabilul **`myexec`** care are activat bitul `setuid` și are ca utilizator `root`, atunci procesul creat din executabilul **`myexec`** va rula ca utilizatorul `root`.

Bitul `setuid`, prezentat și în Secțiunea 5.2.3 este folosit atunci când anumite utilitare necesită privilegii pentru accesarea unor fișiere speciale atunci când rulează. În Listing 5.22, executabilul `/usr/bin/passwd` are activat bitul setuid pentru că procesul creat va trebui să acceseze fișierul `/etc/shadow`, fișier accesibil doar utilizatorilor privilegiați.

```
1  student@uso:~$ ls -l /usr/bin/passwd
2  -rwsr-xr-x 1 root root 59640 Jan 25  2018 /usr/bin/passwd
3  student@uso:~$ stat /usr/bin/passwd
4    File: /usr/bin/passwd
5    Size: 59640             Blocks: 120        IO Block: 4096   regular
   file
6  Device: 801h/2049d        Inode: 656195      Links: 1
7  Access: (4755/-rwsr-xr-x)  Uid: (    0/    root)   Gid: (    0/    root)
8  Access: 2018-08-21 11:41:23.741306286 +0300
9  Modify: 2018-01-25 17:09:22.000000000 +0200
10 Change: 2018-08-06 17:34:29.848000000 +0300
11  Birth: -
```

Listing 5.22: Folosirea bitului setuid

Vedem în rezultatul rulării comenzilor prezența cifrei `4` pe prima poziție în forma octală a permisiunilor și a caracterului `s` în forma literală a permisiunilor în locul caracterului de execuție pentru utilizator. Aceasta înseamnă prezența bitului `setuid` în permisiunile fișierului.

Bitul `setgid` are sens când este folosit pe executabile și pe directoare. Adaugă funcționalitatea de *set group ID on execution*. Pe executabile are același efect ca bitul `setuid` doar că pentru grup.

Când un director are activat bitul `setgid`, viitoarele subdirectoare și fișiere ale acestuia vor avea ca grup în metadate grupul directorului, nu grupul utilizatorului care a rulat comanda. Viitoarele subdirectoare vor moșteni bitul `setgid` astfel că ce se creează nou în ierarhie va avea mereu același grup. Este util pentru a crea un spațiu partajat

în care diferiți utilizatori să creeze fișiere dar să rămână accesibile între ei câtă vreme fac parte din același grup. Mai concret, dacă directorul `top/` are grupul `heroes` și are activat bitul `setgid`, viitoarele fișiere și subdirectoare vor avea tot grupul `heroes`, indiferent ce utilizatori au creat intrările.

Listing 5.23 prezintă cum activarea bitului `setgid` pe un director duce la crearea de intrări care moștenesc grupul directorului.

```
1 student@uso:~$ ls -ld top/
2 drwxr-xr-x 2 student heroes 4096 Sep 19 21:54 top/
3 student@uso:~$ sudo chmod g+s top/
4 student@uso:~$ ls -ld top/
5 drwxr-sr-x 2 student heroes 4096 Sep 19 21:54 top/
6 student@uso:~$ mkdir top/alamar
7 student@uso:~$ ls -ld top/alamar
8 drwxr-sr-x 2 student heroes 4096 Sep 19 21:55 top/alamar
```

Listing 5.23: Folosirea bitului setgid

Bitul `sticky` are sens când este folosit pe directoare la care au acces de scriere mai mulți utilizatori, de exemplu directorul `/tmp`. Adaugă funcționalitatea de *restricted deletion flag*. Acest lucru înseamnă că un utilizator nu poate șterge fișierele altui utilizator, deși are permisiuni de scriere pe directorul care conține acele fișiere.

Directorul `/tmp` este cel care are activat bitul `sticky` pentru că oferă permisiuni complete tuturor utilizatorilor. Deși un utilizator are permisiuni complete pe acel director, nu poate șterge fișiere care nu-i aparțin, așa cum avem prezentat în Listing 5.24.

```
1 student@uso:~$ ls -ld /tmp/
2 drwxrwxrwt 13 root root 4096 Sep 19 21:54 /tmp/
```

Listing 5.24: Folosirea bitului sticky

5.6 Anexă: Resetarea parolei în Linux

Se poate întâmpla să fie uitată parola administrativă pe un sistem care rulează Linux. În această situație este nevoie de resetarea parolei contului `root`. Pentru aceasta avem două opțiuni:

- modificarea procesului de boot și înlocuirea procesului init
- folosirea unui stick USB sau CD bootabil

Procesul de boot va fi prezentat detaliat în Capitolul 9.

În prima variantă, în ecranul bootloader-ului GRUB editățm linia de boot și adăugăm șirul `init=/bin/bash`, ca în Figura 5.12.

Apoi folosim combinația de taste `Ctrl+x` pentru a boota sistemul de operare. În această situație primul proces pornit de sistem nu va fi init ci va fi un shell cu permisiuni de `root`. În acest shell vom rula comenzile din Listing 5.25 pentru schimbarea parolei utilizatorului `root`.

```
1 root@none:~# mount -o remount /
2 root@none:~# passwd
```

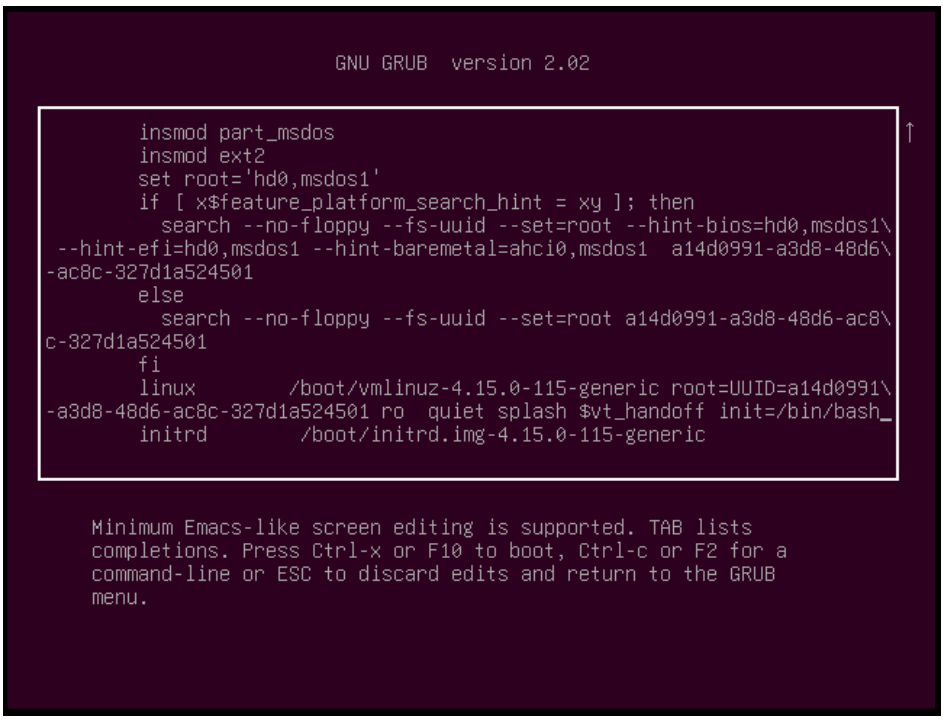


Figura 5.12: Pornirea în shell privilegiat din GRUB

```
3 [...]
```

Listing 5.25: Schimbarea parolei contului root în shellul privilegiat

Mai sus am folosit comanda mount pentru a remonta sistemul de fișiere rădăcină în modul *read-write* și a putea astfel modifica fișierul `/etc/shadow` cu ajutorul comenzii **`passwd`**.

Apoi repornim sistemul fie din buton, fie folosind combinația de taste `Ctrl+Alt+Del`. Nu vom putea folosi comenzi precum **`reboot`** sau **`poweroff`**.

În a doua variantă, folosim un CD/stick USB bootabil cu o distribuție Linux live (precum Ubuntu). După ce sistemul va porni distribuția live, vom realiza următorii pași:

1. Montăm sistemul de fișiere rădăcină al sistemului Linux instalat (pentru care vrem să resetăm parola contului `root`).
2. Folosim comanda **`chroot`** pentru a modifica sistemul de fișiere rădăcină în care rulăm în sistemul de fișiere al sistemului Linux instalat
3. Folosim comanda **`passwd`** pentru a schimba parola contului `root` în sistemul de fișiere rădăcină al sistemului Linux instalat.
4. Revenim la sistemul de fișiere al sistemului live, părăsind shellul obținut prin **`chroot`**.
5. Demontăm sistemul de fișiere rădăcină al sistemului Linux instalat.
6. Repornim sistemul.

Pașii de mai sus sunt reproduși în comenzile din Listing 5.26.

```
1 root@ubuntu:~# mkdir /mnt/rootfs
2 root@ubuntu:~# mount /dev/sda1 /mnt/rootfs
```

```
3 root@ubuntu:~# chroot /mnt/rootfs
4 root@ubuntu:~# passwd
5 [...]
```

Listing 5.26: Resetarea parolei contului root cu live CD

5.7 Use case: Resetarea parolei în Windows

Pentru resetarea parolei unui utilizator în Windows, cel mai important fiind utilizatorul `Administrator`, se poate folosi utilitarul **`chntpw`** (de la change NT password). Acesta este inclus în mai multe CD-uri de recuperare (rescue CDs).

O opțiune este folosirea Ultimate Boot CD[1], un CD bootabil cu utilitare diverse de diagnostic și reparație la nivelul sistemului de operare. Ultimate Boot CD conține PartedMagic[2] care, la rândul său, conține utilitarul **`chntpw`**. Utilitarul **`chntpw`** oferă interfață în linia de comandă pentru modificarea parolelor utilizatorilor în Windows.

Pentru a nu ocupa spațiu din această carte cu detalii, recomandăm să parcurgeți tutorialul cu screenshot-uri despre folosirea Ultimate Boot CD pentru resetarea parole în Windows[3].

5.8 Sumar

Sistemele de operare moderne sunt sisteme multi-utilizator (*multi-user*). Un sistem dispune de mai mulți utilizatori care coexistă simultan. Procesele care rulează în sistemul de operare rulează cu permisiunilor acelor utilizatori.

Utilizatorii de sistem sunt acei utilizatori care au acces la resursele unui sistem de operare: sistem de fișiere, rețea, pot rula procese. Sistemul oferă o bază de date folosită pentru autentificarea utilizatorilor în sistem.

Există un utilizator privilegiat (numit generic *superuser*) care are acces complet la sistem. Există mecanisme alternative de a oferi privilegii utilizatorilor obișnuiți (neprilegiați). Gestiunea utilizatorilor, gestiunea rețelei, gestiunea pachetelor sunt acțiuni privilegiate.

Sistemul de operare oferă permisiuni de acces la sistemul de fișiere, componenta cea mai vizibilă. Permisiunile permit accesul unor utilizator la un fișier dar nu a altor utilizatori.

Modul uzual de accesare a sistemelor de operare moderne este prin intermediul unei parole. Majoritatea sistemelor oferă opțiuni de resetare a parolei în cazul în care aceasta este pierdută.

[1] http://www.ultimatebootcd.com
[2] https://partedmagic.com
[3] https://www.top-password.com/blog/reset-forgotten-windows-password-with-ultimate-boot-cd/

Capitolul 6

Dezvoltarea aplicațiilor

6.1 Introducere

În momentul de față avem o varietate mare de dispozitive electronice cu care interacționăm: de la sisteme complexe, cum ar fi calculatoarele personale sau laptop-urile, la telefoanele mobile sau chiar ceasurile inteligente pe care le folosim intens zilnic. Deși foarte diferite, toate aceste sisteme electronice au un lucru în comun: toate rulează software.

Având în vedere marea varietate a sistemelor fizice pe care rulează aplicațiile pe care le dezvoltăm, este foarte important să înțelegem toate aspectele dezvoltării unui program și toate opțiunile pe care le avem la dispoziție în acest proces. Astfel, putem face alegerile corecte pentru a dezvolta eficient o aplicație adaptată dispozitivului fizic pe care va rula.

De exemplu, dacă dezvoltăm o aplicație ce va rula pe un ceas inteligent, trebuie să ținem cont că avem resurse limitate comparativ la o aplicație ce va rula pe un sistem desktop. În cazul unui ceas inteligent, avem mai puțină memorie și putere de procesare și, mai mult, trebuie să ținem cont de consumul de energie pe care îl are dispozitivul.

Alegerea unui limbaj potrivit pentru un program este esențială pentru succesul acestuia. În funcție de tipul aplicației, trebuie ales limbajul adecvat având în vedere proprietăți precum: viteza de execuție, ușurința de scriere a codului, numărul de biblioteci existente, utilitarele existente, portabilitatea și comunitatea din jurul limbajului.

6.2 Limbaje de programare

Un **limbaj de programare** definește un set de reguli folosite pentru a formula instrucțiuni pe care calculatorul să le execute. Regulile limbajului constau în cuvinte cheie, instrucțiuni care pot fi folosite și gramatica acestora, care se mai numește și sintaxă.

Folosind un limbaj de programare, putem defini operații pe care dorim să le executăm într-o anumite ordine. Acesta este practic programul pe care îl scriem. Operațiile sunt rulate pe procesorul sistemului pe care dorim sa îl programăm, dar înainte de a fi executate, ele trec printr-un proces de transformare. Fiecare procesor cunoaște un

anumit set de instrucțiuni pe care le poate executa. Codul scris folosind aceste instrucțiuni se numește **cod mașină** și este singurul limbaj pe care un procesor îl înțelege și îl execută. Prin urmare, programul scris de noi într-un limbaj de programare este transformat în cod mașină pentru a putea fi înțeles și executat.

La început, calculatoarele erau niște mașini cu capacitate de calcul mult mai redusă, care puteau executa operații simple, în general stocate pe benzi magnetice sau cartele perforate, ca în Figura 6.1. Odată cu evoluția procesoarelor și a mediilor de stocare, a apărut limbajul de asamblare, folosit pentru a scrie programe mai complexe. **Limbajul de asamblare** este un limbaj specific fiecărui procesor și are în general o corespondență de aproximativ unu-la-unu între instrucțiunile de asamblare și cele are procesorului. Practic, putem considera că fiecare instrucțiune a limbajului de asamblare este tradusă printr-o altă instrucțiune a procesorului. Codul sursă scris în limbaj de asamblare este transformat în cod mașină de un asamblor.

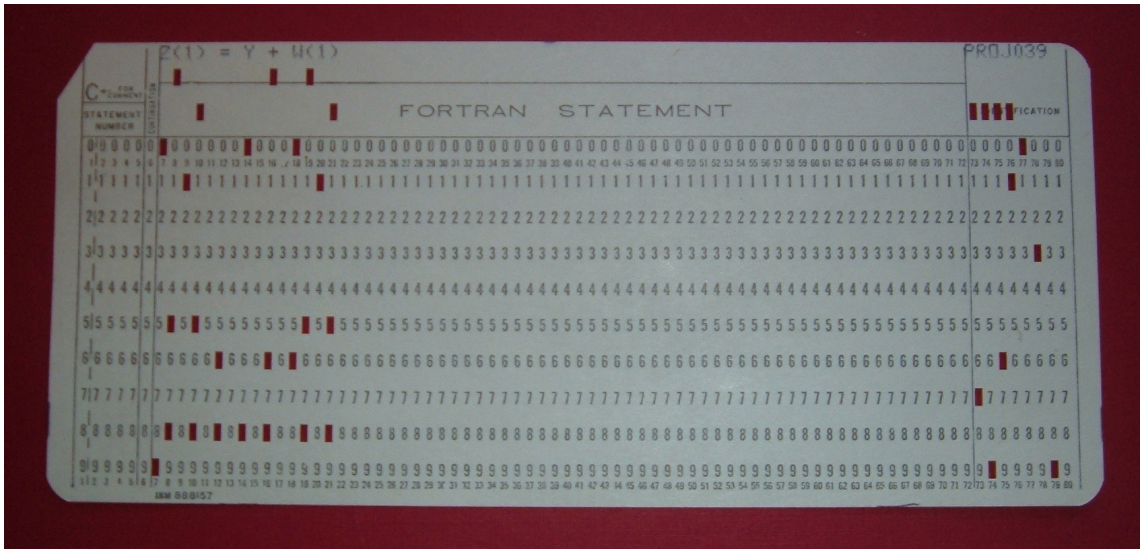


Figura 6.1: Program scris pe cartelă perforată[1]

În general, producătorul unui procesor definește și limbajul de asamblare specific acestuia, pentru a oferi o variantă lizibilă a instrucțiunilor procesorului. Scopul limbajului de asamblare a fost de a permite scrierea de aplicații într-o manieră ușoară, la început acesta fiind singura modalitatea de a dezvolta programe. Deși mai ușor de folosit decât codul binar, și limbajul de asamblare este dificil de urmărit mai ales pentru programe lungi. De aceea, în timp, au apărut limbajele mai complexe din punct de vedere al logicii, limbaje ce folosesc comenzi apropiate ca structură de limbajul natural, și care sunt mai ușor de urmărit. Acestea permit programatorilor să se concentreze pe logica aplicației, nu pe limbajul efectiv și implementarea acestuia. Ele se numesc **limbaje de nivel înalt**.

În prezent, majoritatea aplicațiilor sunt dezvoltate folosind limbaje de programare de nivel înalt, care sunt mult mai ușor de "citit" de către programatori. Totuși, pentru dezvoltarea de aplicații care controlează dispozitive hardware și care trebuie să fie foarte eficiente în folosirea resurselor (ex. drivere) se folosește uneori limbajul de asamblare, deși cazurile acestea sunt destul de rare.

[1] https://en.wikipedia.org/wiki/Computer_programming_in_the_punched_card_era#/media/File:FortranCardPROJ039.agr.jpg **CC BY-SA 2.5**

În mare parte, limbajele de programare folosite în zilele noastre sunt limbaje de nivel înalt. Astfel, programatorii folosesc o sintaxă apropiată de limbajul natural pentru dezvoltarea aplicațiilor, ceea ce ușurează întregul proces (se folosesc cuvinte precum *while*, *if*, *else*). Atunci când scriem un program, totul pornește de la ideea clară a ce vrem să obținem prin acel program. Mai exact, trebuie să stabilim clar ce vrem să facă programul. După aceea, următorul pas este să structurăm ideea și să o *traducem* în limbajul de programare folosit. Așa obținem programul final ce apoi poate fi rulat, proces sumarizat în Figura 6.2.

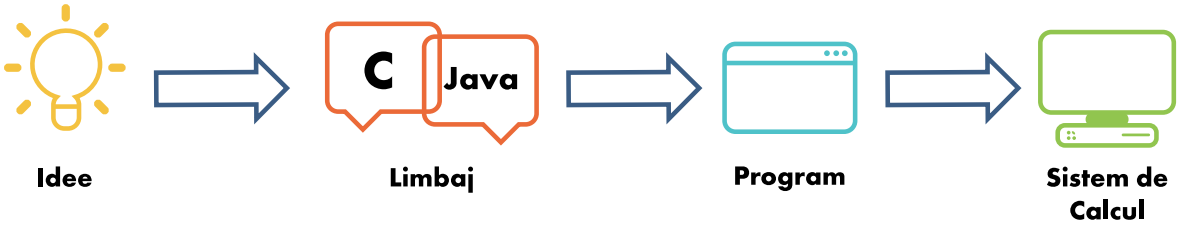


Figura 6.2: De la idee la program

Fișierele pe care programatorii le scriu se numesc **fișiere cod sursă**, ele conțin practic instrucțiuni scrise într-un anume limbaj de programare, de cele mai multe ori, limbaj de nivel înalt. Codul sursă scris astfel nu poate fi executat pentru că procesorul nu îl înțelege. Cum am menționat deja, procesorul înțelege doar codul mașină. Drept urmare, pentru a executa programul, e necesar să transformăm codul sursă în cod mașină.

Procesul de transformare a codului sursă în cod mașină se numește **compilare**.

Bazat pe modul în care codul de nivel înalt este transformat și rulat pe procesor, putem clasifica limbajele de programare în limbaje compilate sau interpretate. Diferența între cele două este că pentru a rula un limbaj compilat, acesta este întâi transformat în cod mașină, rezultând un fișier executabil, care apoi poate fi rulat oricând. Pe de altă parte, un limbaj interpretat este executat direct, prin intermediul unui interpretor. Interpretorul este cel care ia fiecare instrucțiune, o transformă în cod mașină iar aceasta este executată imediat.

Fiecare dintre cele două tipuri de a rula cod sursă, compilare sau interpretare, are avantaje și dezavantaje de care este important să ținem cont când alegem limbajul de programare pentru dezvoltarea unei aplicații.

Principalul avantaj al unui limbaj compilat este viteza de execuție. Odată generat fișierul executabil, acesta va fi rulat de către procesor de oricâte ori dorim, astfel compilarea este un proces separat de execuția programului și are loc o singură dată. În schimb, în cazul unui limbaj interpretat, la fiecare rulare, codul sursă va fi interpretat linie cu linie de către interpretor, acesta fiind de fapt cel care execută acțiunile programului, proces care e mai lent.

Unele interpretoare mai eficiente vor încerca sa compileze parțial codul sursă și să îl execute direct pe procesor. Practic, în acest caz, codul sursă se compilează la fiecare rulare.

Pe de altă parte, un limbaj interpretat are mai multă portabilitate. Cum am precizat deja, codul mașină rezultat în urma compilării poate rula doar pe un anumit tip de procesor. Deci pentru a rula aceeași aplicație pe arhitecturi (procesoare) diferite, avem nevoie de un fișier executabil pentru fiecare tip de arhitectură. În aceea ce privește limbajele

interpretate, putem folosi aceleași fișiere care sunt transformate de către interpretor în codul mașină specific procesorului.

Majoritatea limbajelor de programare apărute în ultima perioadă sunt considerate limbaje hibride. Ele îmbină elemente ale ambelor procese, compilare și interpretare, pentru a obține atât portabilitate, cât și viteză în execuție.

Este important de reținut că indiferent de limbaj (compilat, interpretat sau hibrid), procesorul știe să ruleze doar cod mașină, așa că orice cod sursă va fi la un moment dat transformat în cod mașină, iar pentru programatori este important dacă limbajul folosit este unul compilat, interpretat sau hibrid. Este important să cunoaștem tipul unui limbaj de programare și avantaje și dezavantaje sale pentru a putea alege în funcție de nevoile pe care le avem.

De exemplu, dacă ne dorim să dezvoltăm un joc tip curse de mașini, avem nevoie de rapiditate în execuție, față de o aplicație de organizare a lucrului în echipă, unde ne dorim să putem acoperi cât mai multe platforme.

6.3 Limbaje compilate

Limbajele compilate sunt cele unde codul sursă este transformat în fișiere executabile, ce sunt apoi rulate pe procesor, în urma unui proces care se numește compilare. Astfel, instrucțiunile specifice limbajului de programare sunt transformate de compilator în instrucțiuni pe care procesorul le înțelege.

În procesul de compilare codul sursă este prelucrat și trece prin mai multe etape intermediare ca în Figura 6.3.

1. compilare: codul sursă este transformat în cod în limbaj de asamblare
2. asamblare: codul în limbaj de asamblare este transformat în cod mașină;
3. link-editare (sau linking): se fac legăturile către fișiere externe care conțin simboluri sau funcții apelate din fișierul sursă; dacă avem o funcție definită într-un fișier și folosită în altul, implementarea funcției trebuie legată de apelul ei.

În teorie, pentru fiecare limbaj de programare ar trebui să existe un compilator care să cunoască gramatica specifică acestuia, instrucțiunile specifice unui procesor și care să facă transformarea din cod sursă în cod mașină.

Pentru că există un număr foarte mare de limbaje și un număr mare de procesoare diferite, în practică, nu avem cate un compilator diferit. În general, există framework-uri de compilator care au suport pentru mai multe limbaje de programare (de exemplu C sau C++ sau D), respectiv alte componente care au suport pentru mai multe arhitecturi de procesor (de exemplu x86 sau ARM sau MIPS). Astfel, un compilator poate genera cod pentru diferite arhitecturi pornind de la diferite tipuri de limbaje. În general aceste compilatoare au un limbaj intern, numit limbaj intermediar care facilitează aceste traduceri din limbaj în cod mașină de procesor, așa cum este indicat în Figura 6.4.

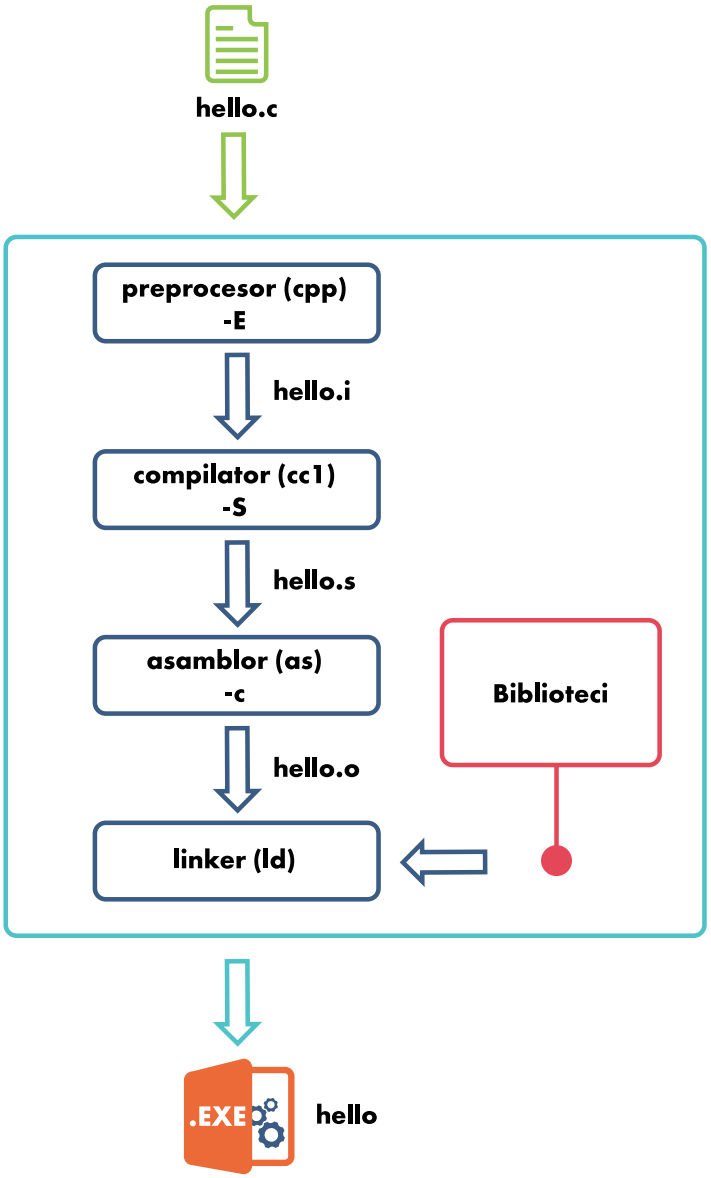


Figura 6.3: Etapele compilării

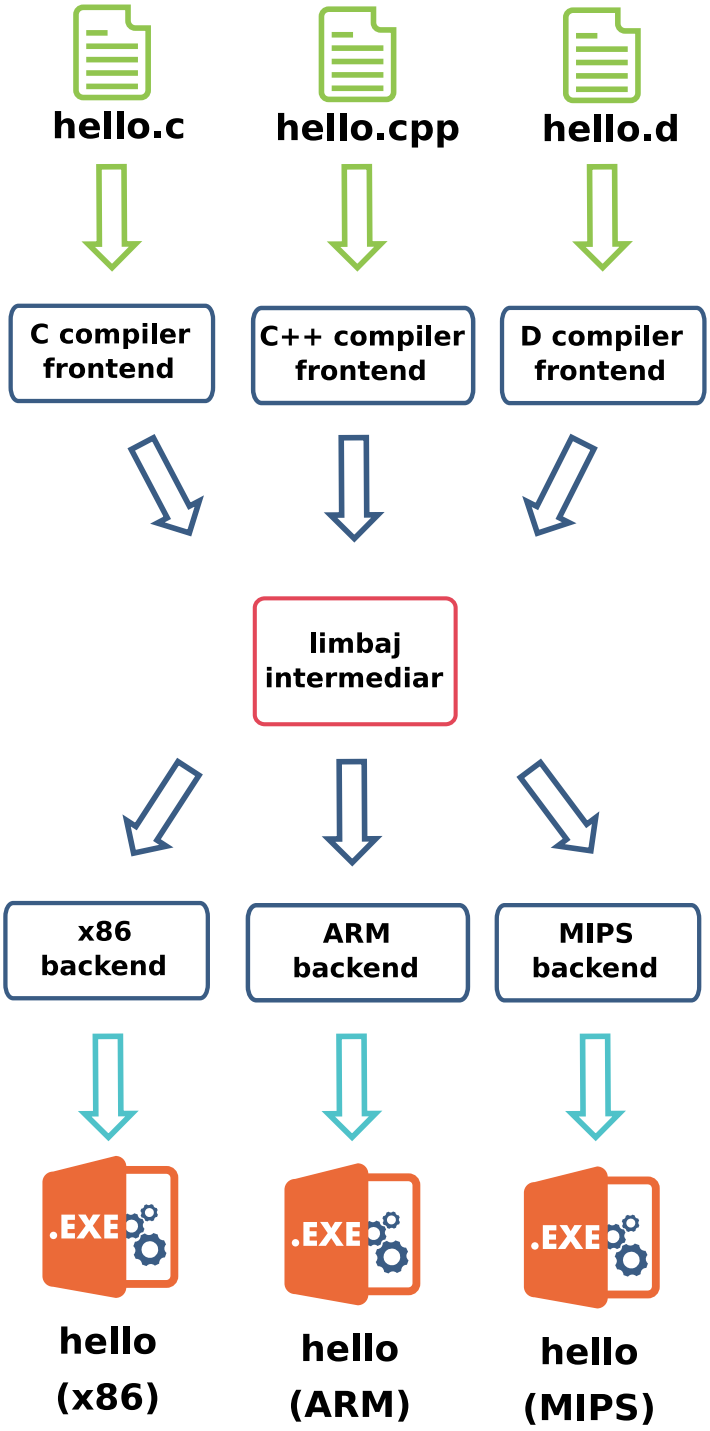


Figura 6.4: Traducerea în limbaj intermediar

6.3.1 Exemple de limbaje de programare compilate

Deși am prezentat avantajele și dezavantajele folosirii unui limbaj de programare compilat, dacă alegem că aceasta este opțiunea potrivită, avem un număr mare de limbaje compilate din care putem alege. În continuare vom descrie câteva din cele mai folosite și particularitățile de care trebuie să ținem cont în luarea unei decizii.

6.3.1.1 C

C este un limbaj de programare care a fost dezvoltat în anii 1970 pentru a fi folosit la implementarea sistemului de operare Unix. Până la apariția limbajului C, sistemele de operare erau scrise în limbaj de asamblare. C aduce un nivel de abstractizare peste limbajul de asamblare, care ușurează scrierea programelor.

Pentru că limbajul permite accesarea ușoară a resurselor hardware, aplicațiile dezvoltate în C prezintă un risc mai mare de securitate, dacă acestea nu sunt implementate cu atenție. Din aceleași considerente și probabilitatea de a genera erori în timpul implementării este mai mare. În plus, C are un nivel de abstractizare foarte redus, adică nu avem la dispoziție funcții care să execute operații complexe, de aceea nu este un limbaj folosit pentru prototipare. Este recomandat să folosim C dacă stăpânim bine limbajul și nu riscăm să generăm erori și dacă ne dorim o aplicație care să consume cât mai puține resurse.

Preponderent, limbajul C este folosit pentru a dezvolta sisteme de operare și drivere pentru acestea.

6.3.1.2 C++

C++ este un limbaj de programare de nivel înalt gândit pentru dezvoltarea de aplicații. Este o adaptare a limbajului C pentru a-l face mai ușor de folosit. În comparație cu alte limbaje mai noi, C++ este un limbaj mai complex și mai dificil de utilizat. Totuși, un număr mare de aplicații existente axate pe performanță sunt implementate folosind C++.

6.3.2 Pascal

Pascal a fost dezvoltat în anii 1960, având ca scop principal dezvoltarea de programe pentru descrierea de algoritmi. Limbajul a fost gândit pentru educație, pentru a fi utilizat în predarea algoritmilor.

Deși nu este un limbaj pe care să îl luam în calcul în dezvoltarea aplicațiilor, pentru că există alte limbaje mai moderne, poate fi în continuare luat în calcul în scopuri didactice.

6.3.3 Alte limbaje

Alte limbaje compilate sunt D, Go (folosit pentru scrierea sistemului de containere Docker) și Rust (folosit de Firefox pentru Servo, motorul de randare a paginilor web)

6.4 Limbaje interpretate

Limbajele interpretate sunt cele care sunt transformate în cod mașină în timpul execuției.

Pentru a putea rula o aplicație dezvoltată într-un limbaj de programare interpretat, avem nevoie de un program care ia fiecare linie din codul sursă și o transformă în cod mașină. Acest program se numește interpretor.

Diferența între o aplicație compilată și una interpretată este că spre deosebire de compilator, care generează un întreg fișier executabil care poate fi apoi rulat, interpretorul ia fiecare linie, generează codul mașină aferent ei, codul este rulat pe procesor, după care operația este reluată. Deși acest proces crește timpul de execuție al aplicației, sunt avantaje pe care limbajele interpretate le au față de cele compilate.

În primul rând, programele scrise în astfel de limbaje pot fi direct rulate pe orice arhitectură, atâta timp cât există un interpretor instalat. Acest lucru înseamnă că putem copia codul sursă de pe un sistem care rulează Windows pe unul care rulează macOS. În plus, pentru că toate instrucțiunile trec prin interpretor, limbajele interpretate nu oferă acces direct la resursele hardware, ceea ce le face mai sigure, dar și mai ineficiente. De aceea astfel de limbaje sunt recomande pentru dezvoltarea de aplicații care nu țin de sistemul de operare.

Pe de altă parte, limbajele interpretate presupun ca orice mașină care va rula aplicația dezvoltată, trebuie să conțină codul sursă ce va fi rulat. Sunt cazuri în care nu se dorește publicarea codului sursă ar fi de preferat un limbaj compilat, care permite distribuirea unui executabil.

6.4.1 Exemple de limbaje de programare interpretate

În prezent utilizarea limbaje de programare pur interpretate este în scădere. Acestea au ajuns să fie înlocuite de alte limbaje de programare, majoritatea hibride. Totuși, dacă vom ajunge să lucrăm la aplicații complexe dezvoltate cu ani în urmă, există posibilitatea să ajungem să scriem cod sursă în limbaje precum PHP sau Perl.

PHP este un limbaj folosit preponderent pentru dezvoltarea de servere web. Mai nou este înlocuit de Node.js, dar încă mai putem găsi aplicații web dezvoltate în PHP.

Perl e un alt limbaj interpretat folosit pentru dezvoltare web. Este considerat predecesorul PHP, deci este tot mai rar întâlnit.

6.5 Limbaje hibride

Limbajele de programare hibride îmbină elemente de compilare cu elemente de interpretare, pentru a rezulta în aplicații portabile, rapide și sigure.

Pentru că principalul dezavantaj al programelor interpretate este timpul mare de execuție, una din îmbunătățirile aduse interpretoarelor este posibilitatea de a transforma codul sursă într-un cod intermediar, care se numește **bytecode**, care va fi apoi interpretat. Bytecode-ul este practic codul mașină al interpretorului, deci

transformarea acestuia în cod mașină este mult mai rapidă. Practic, la rularea unui program, este generat un fișier intermediar care conține bytecode, iar interpretorul ia apoi instrucțiuni din bytecode și le transformă în cod mașină. Putem deci considera că avem două etape în procesul de rulare: compilare către bytecode și apoi interpretare.

Pentru că în acest caz interpretorul are propriul limbaj mașină, acesta nu cunoaște un limbaj de programare anume, ci el transformă bytecode-ul specific în cod mașină, interpretorul se mai numește și **mașină virtuală** (ex. Java Virtual Machine). Un avantaj pe care mașina virtuală îl aduce este posibilitatea de a genera bytecode-ul și de a distribui fișiere bytecode în locul codului sursă. Astfel codul sursă al aplicației nu este făcut public odată cu distribuția aplicației.

Odată generat bytecode-ul, acesta poate fi transformat în cod mașină în două moduri diferite. Fie se ia tot bytecode-ul și se transformă în cod mașină, după care codul mașină este executat (compilare *ahead-of-time*, sau JIT), fie se iau bucăți din bytecode și se transformă în cod mașină în timpul rulării (compilare *just-in-time*).

Compilarea *ahead-of-time* funcționează ca procesul normal de compilare, doar că se transformă bytecode în cod mașină. Odată generat codul mașină acesta este executat. Avantajul acestei metode este timpul rapid de execuție. În schimb, timpul de pornire a aplicației crește (timpul necesar pentru a transforma bytecode în cod mașină).

Compilarea *just-in-time* (JIT) presupune ca bucăți de cod care urmează să fie executate să fie transformate în cod mașină în timpul rulării. Interpretorul analizează blocul de cod care urmează să fie executat și îl transformă în cod mașină.

În plus, interpretorul are anumite mecanisme care îi permit să țină cont de factori precum frecvența de execuție a unui anumit bloc de cod și eficientizează procesul prin stocarea în memorie a codului mașină, pentru a nu transforma același cod de mai multe ori, deci făcând procesul și mai eficient. Totuși, spre deosebire de compilarea ahead-of-time, JIT are un timp de execuție mai mare, dar timpul de pornire al aplicației scade.

6.5.1 Exemple de limbaje de programare hibride

În general, limbajele pe care le întâlnim cel mai frecvent astăzi sunt limbaje hibride, precum Java, Python sau JavaScript. Cum multe din aplicațiile pe care le folosim zi de zi au devenit aplicații web, limbajele orientate spre așa ceva au crescut în popularitate.

6.5.1.1 Java

Java este un limbaj de programare folosit pentru dezvoltarea de aplicații enterprise care a fost dezvoltat de Sun Microsystems. Oracle a cumpărat Sun Microsystems, așa că astăzi Java este întreținut de către Oracle. Paradigma pe care se bazează Java este programarea orientată pe obiecte. Acesta facilitează scrierea unui cod cât mai modularizat și refolosibil.

Idee de bază care a stat la baza limbajului Java a fost portabilitatea. Se dorea scrierea unui program o singură dată și rularea lui pe multe platforme diferite fără a face vreo modificare și fără a fi necesară distribuirea sursei. Inițial a fost folosit în programe de

navigare sub forma de *Java Applets*, mici aplicații scrie care se putea include într-o pagină.

Fiind destinat dezvoltării de aplicații pentru companii, platforma Java este are două componente, compilatorul care generează bytecode-ul (Java Development Kit - JDK) și mașina care rulează bytecode-ul (Java Runtime Environment - JRE). Astfel pentru distribuția aplicației nu este necesară distribuția codului sursă.

6.5.1.2 C#

C# este un limbaj de programare similar cu Java, dezvoltat de Microsoft. Este principalul limbaj de dezvoltare pentru platforma Windows, fiind integrat inclusiv în motoare de jocuri (de exemplu Unity).

6.5.1.3 Python

Python este un limbaj dezvoltat inițial cu scopul de a fi folosit pentru a învăța programare, principalul avantaj fiind ca oferă o lizibilitatea mare codului. A devenit foarte cunoscut datorită unor proiecte precum GNOME (inițial) și OpenStack, precum și datorita bibliotecilor de matematică avansate.

Python este folosit pentru automatizarea administrării sistemului, prin scripturi simple. Este de asemenea foarte folosit în educație, codul sursă fiind ușor de urmărit. În plus, Python impune reguli de indentare, ceea ce îl face potrivit ca prim limbaj de programare care obișnuiește elevii cu necesitatea de a indenta codul. Limbajul mai este folosit în cercetare, find o variantă mai „ușoară” față de alte limbaje științifice cum ar fi Matlab, R sau Julia.

O altă utilizare a limbajului Python este pentru dezvoltarea de servere web, cu ajutorul bibliotecilor Flask și Django.

6.5.1.4 JavaScript

JavaScript este un limbaj folosit pentru dezvoltarea de aplicații web. Are o sintaxa similară cu limbajele C și Java. Deși se numește JavaScript, în afară de sintaxa similară, nu are nimic în comun cu Java.

La început JavaScript a fost folosit pentru partea de interfață a aplicațiilor web, interpretorul de JavaScript fiind integrat în browsere. Odată cu apariția platformei Node.js, aplicațiile JavaScript pot fi rulate și în afara browserului. Astfel, JavaScript este folosit pentru aplicații simple, pentru automatizarea operațiilor, dar mai ales pentru dezvoltarea de servere web, prin intermediul bibliotecii Express.

În principiu, dacă ne dorim să dezvoltăm o aplicație web, JavaScript, împreună cu Node.js sunt o opțiune importantă de luat în calcul.

6.6 Dezvoltarea unui program

Până acum am discutat despre limbajele de programare uzuale și cum acestea se diferențiază între ele și despre cum codul sursă pe care noi îl scriem rulează pe procesor. Întrebarea pe care urmează să o punem este: *ce presupune dezvoltarea unui program*?

6.6.1 Scrierea codului sursă

În primul rând, dezvoltarea unui program constă în scrierea efectivă a codului sursă; practic, acest pas se rezumă la a crea fișiere text și a le edita. De aceea, aplicația principală de care avem nevoie în dezvoltarea unui program este un **editor de text**.

În ceea ce privește editoarele de text, avem un număr mare de editoare diferite din care putem alege, pornind de la unele foarte simple care au ca scop editarea unui fișier text, la aplicații complexe adaptate dezvoltării de programe. În mod normal, orice editor poate fi folosit pentru scrierea de cod, chiar și Notepad din Microsoft Windows, totuși este de preferat să folosim unul adaptat scrierii de cod, pentru a ne ușura munca.

Editoarele adaptate dezvoltării de programe vin cu anumite caracteristici care ne ajută în procesul de dezvoltare cum ar fi indentarea automată a codului, evidențierea cuvintelor cheie, numerotarea liniilor, gruparea secvențelor de cod.

Ținând cont de numărul mare de limbaje de programare existente, fiecare cu particularitățile sale, este imposibil ca un editor să cuprindă toate aspectele fiecărui limbaj. De aceea, majoritatea acestor editoare permit instalarea de extensii care aduc funcționalități în plus sau aduc funcționalități specifice pentru un anumit limbaj.

În continuare vom descrie cele mai cunoscute editoare folosite pentru scrierea programelor.

6.6.2 Editoare în mod text

6.6.2.1 Vim

Vim este un editor care funcționează în linie de comandă, fiind versiunea îmbunătățită a vi, editor care se găsește pe toate sistemele Unix. Este un editor foarte configurabil, dar nu foarte ușor de folosit pentru începători. Acest lucru este datorat lipsei interfeței grafice, tot meniul fiind transpus prin comenzi. Deși pentru utilizatorii experimentați acest lucru poate reprezenta un avantaj aducând rapiditate în folosire, pentru începători, lipsa cursorului și necesitatea de a memora comenzi îngreunează folosirea editorului.

Chiar dacă există multe variante de editoare mai intuitive, vă recomandam familiarizarea cu Vim, sau chiar cu varianta redusă, vi, tocmai pentru că este unul din puținele editoare disponibile pe orice sistem Unix.

Pentru familiarizarea cu Vim și comenzile de bază, recomandăm parcurgerea tutorialului **`vimtutor`**. Este suficient să rulați comanda **`vimtutor`** în terminal și tutorialul va începe.

Deși putem găsi vi pe orice sistem Unix, doar unele distribuții vin cu editorul vim instalat. Pentru a instala Vim pe sistemele Debian/Ubuntu, rulăm comanda **`sudo apt install vim`**.

6.6.2.2 Alte editoare

Alte editoare similare cu Vim, care funcționează tot în linie de comandă, sunt Nano, Pico și Emacs. La fel ca Vim, este se găsesc frecvent pe sistemele Unix, sunt folosite la editarea rapidă, cum ar fi editarea unor fișiere de configurare.

6.6.3 Editoare în interfață grafică

6.6.3.1 Sublime Text

Sublime Text este un editor de cod avansat. Permite crearea sau editarea facilă a unor programe complexe, alcătuite din mai multe fișiere. Pe lângă posibilitatea de a edita structura de fișiere și directoare a proiectului, editorul recunoaște limbajele de programare cele mai folosite și evidențiază cuvintele cheie specifice fiecărui limbaj. În plus, recunoaște variabilele definite de utilizator și le sugerează acestuia în timpul scrierii codului.

Pe lângă caracteristicile de bază, cum am menționat și mai sus, Sublime Text oferă suport pentru extensii care aduc funcționalități cum ar fi integrarea cu sisteme de versionare a codului (ex. Git), recunoașterea unor limbaje de programare mai puțin cunoscute sau diverse teme.

Pentru a instala Sublime Text, trebuie să urmăm pașii descriși pe pagina web dedicată editorului (https://www.sublimetext.com/3). În momentul de față este recomandată folosirea versiunii 3, care a fost lansată drept versiune stabilă de curând. Editorul nu este gratis, el poate fi folosit pentru o perioadă limitată, după care este necesară achiziționarea unei licențe de folosire.

6.6.3.2 Atom

Atom este un editor asemănător cu Sublime Text, dezvoltat de GitHub. Principalul avantaj pe care acesta îl are față de Sublime Text este că are sursa deschisă și nu se percepe nicio taxă pentru folosire, deci poate fi considerat o variantă mai accesibilă față de Sublime Text.

Informații despre instalarea editorului Atom se găsesc pe pagina acestuia: https://atom.io/.

6.6.3.3 Visual Studio Code

Visual Studio Code (VSCode) este un editor de cod dezvoltat de Microsoft. Similar cu Atom, editorul are sursa deschisă și poate fi folosit gratuit.

Precum cele două editoare descrise mai sus, VSCode poate fi folosit pe sisteme Windows, Linux și macOS. Pentru a-l instala urmăm pașii descriși la adresa https://code.visualstudio.com/Download/.

6.6.4 Medii integrate de dezvoltare (IDE)

Am vorbit despre editoare și importanța lor în dezvoltarea programelor. Pe lângă editoare avansate, avem posibilitatea de a folosi medii integrate de dezvoltare. La o primă vedere, acestea sunt similare cu editoarele de text. IDE -urile au, în schimb, anumite funcționalități avansate, multe din ele fiind adaptate unui singur limbaj de programare, sau unui număr redus de limbaje. În primul rând, orice IDE are integrat un compilator sau interpretor pentru limbajul sau limbajele de programare pe care le suportă și la o simplă apăsare de buton programul este rulat.

Având în vedere că editoare precum Sublime Text sau Visual Studio Code vin cu un număr mare de extensii care pot adăuga funcționalități specifice unui anumit limbaj de programare, diferența între un editor avansat și un IDE este de multe ori greu de sesizat. Astfel, unele din funcționalitățile specifice unui IDE se regăsesc și în editoarele avansate, cum ar fi depistarea erorilor, căutarea avansată a unei variabile, afișarea definiției funcției utilizate, sau chiar rularea programului la o apăsare de buton.

Pe de altă parte, există multe funcționalități pe care un IDE le aduce în plus. De exemplu, în ceea ce privește interpretarea codului sursă pe măsura ce acesta e scris, majoritatea editoarelor pot să identifice variabilele declarate, dar nu și tipul acestora. De aceea un IDE va oferi sugestii mai relevante programatorului. Un alt avantaj este posibilitatea de a refactoriza codul, adică putem alege să schimbăm numele unei variabile și schimbarea va avea loc în toate fișierele, peste tot unde variabila este folosită.

Multe dintre limbajele de programare existente sunt folosite pentru dezvoltarea de aplicații grafice, cum ar fi Objective-C pentru aplicații iOS. Pentru aceste cazuri, IDE-urile oferă un mediu vizual pentru dezvoltarea interfeței grafice în care programatorul poate să construiască vizual interfața și codul specific este generat automat, ca în Figura 6.5.

Un alt avantaj important al mediilor de dezvoltare integrate sunt metodele avansate de a depana programele. Folosind sistemul de depanare integrat în IDE, putem configura puncte în care să oprim execuția programului, după care putem inspecta valoarea curentă a variabilelor folosite sau valorile întoarse de funcții. Mai mult, putem chiar modifica valorile unor variabile pentru a influența execuția programului.

În general, pentru fiecare limbaj de programare ales, avem un IDE adaptat acestuia. Există totuși și destul de multe IDE-uri ce suportă mai multe limbaje. Se recomandă folosirea unui IDE dedicat limbajului de programare folosit, dacă acesta există, pentru că ne ușurează mult munca. Un IDE ne permite să ne concentrăm pe scrierea codului, fără a ne consuma timp cu a rula comenzi pentru compilarea, rularea sau depanarea acestuia.

Mai departe vom menționa câteva medii de dezvoltare importante pentru cele mai folosite limbaje de programare.

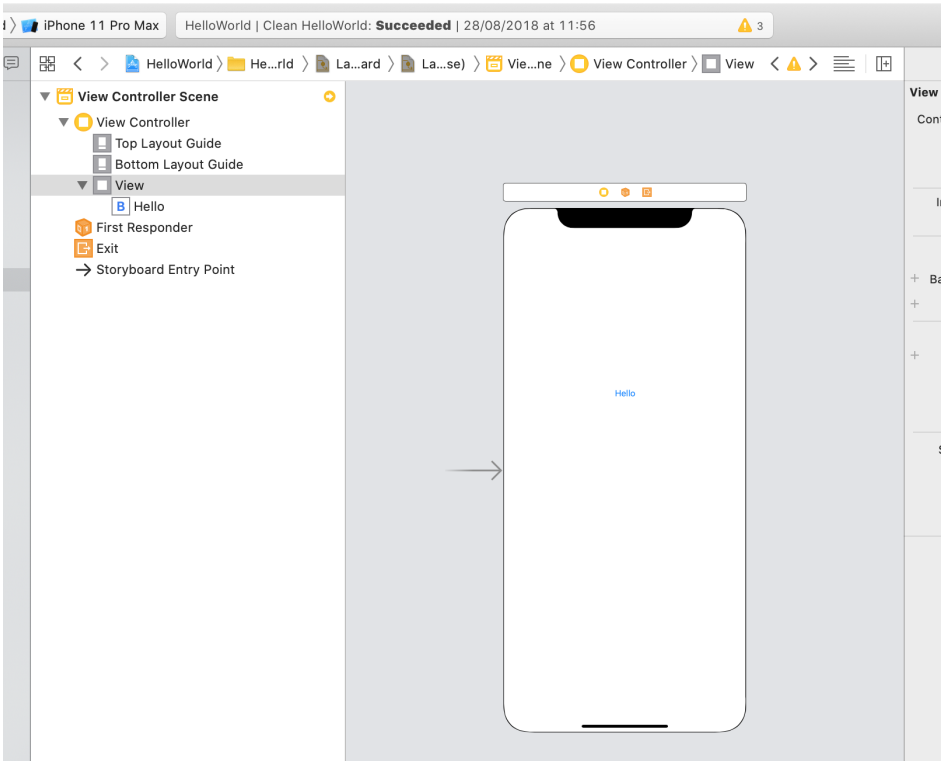


Figura 6.5: Construirea vizuală a interfeței unei aplicații într-un IDE

6.6.4.1 Microsoft Visual Studio și MonoDevelop

Microsoft Visual Studio este editorul dedicat dezvoltării în C/C++ și C#. Pentru că este suportat doar pe sisteme Windows sau macOS, pentru sisteme Linux recomandăm folosirea IDE-ului MonoDevelop. MonoDevelop este open source, dar este susținut de Microsoft, împreună cu Microsoft Visual Studio.

6.6.4.2 Eclipse

Eclipse este unul din IDE-urile cele mai folosite pentru dezvoltarea de aplicații Java. Înainte de apariția Android Studio, era folosit chiar pentru dezvoltarea de aplicații Android, prin instalarea unei extensii.

6.6.4.3 NetBeans

NeatBeans e un IDE cu sursă deschisă specializat pentru limbajul Java. El oferă o modalitate ușoară de a crea interfețe grafice pentru aplicații Java, împreună cu editorul pentru dezvoltarea programelor.

6.6.4.4 Xcode

Xcode este IDE-ul dezvoltat de Apple pentru crearea în principal de aplicații pentru iOS și macOS. Este adaptat pentru C, Objective-C și Swift. Objective-C e un limbaj de

programare ce extinde limbajului C, folosit pentru dezvoltarea de aplicații iOS și macOS. Swift este succesorul limbajului Objective-C.

O caracteristică importantă a Xcode este editorul de interfețe grafice integrat (storyboardul), care permite generarea vizuală a interfețelor grafice și conectarea mai multor ferestre fără necesitatea de a scrie cod.

În ceea ce privește editorul pentru scrierea codului sursă, oricare din editoarele descrise mai sus suportă un număr mare de limbaje de programare și este adaptat pentru acestea. Recomandarea noastră este să încercați mai multe editoare și să alegeți pe cel pe care îl considerați cel mai comod în folosire, iar pentru limbajele de programare care au un IDE dedicat, se recomandă folosirea IDE-ului.

6.6.5 Biblioteci și framework-uri

Un alt aspect de care trebuie să ținem cont în dezvoltarea programelor este folosirea bibliotecilor și a framework-urilor.

Bibliotecile sunt colecții de resurse pe care le integrăm în aplicațiile noastre pentru a ne ușura procesul de dezvoltare. De exemplu, dacă dorim să dezvoltăm o aplicație Python care face cereri HTTP , vom folosi biblioteca *requests* care expune funcția *get()* pentru a face o cerere. Fără a folosi biblioteca, ar trebui să scriem noi tot codul care se conectează la un server, generează pachetul pentru cerere, îl trimite, apoi preia și interpretează răspunsul.

O unealtă asemănătoare bibliotecilor sunt framework-urile. Și acestea sunt unelte care ușurează procesul de dezvoltare, dar ele oferă un schelet pe care aplicația îl urmează pentru a beneficia de proprietățile framework-ului. Practic putem să ne gândim că biblioteca e un modul care odată integrat în aplicație aduce funcții sau constante noi pe care le putem folosi când e cazul, în timp ce framework-ul impune anumite funcții sau elemente ce trebuie implementate în aplicație și aplicația trebuie să se muleze după acea structură.

Pentru a putea integra biblioteci și framework-uri în aplicațiile pe care le dezvoltăm, acestea trebuie să existe pe sistemul pe care lucrăm. Vom vedea mai departe, că de multe ori fiecare platformă de dezvoltare specifică unui limbaj vine cu propriul sistem de instalare a acestor module.

6.6.6 Rularea programului

Odată ce am scris codul sursă și am instalat bibliotecile sau framework-urile folosite, pasul final este să rulăm programul. Complexitatea acestui pas depinde de complexitatea programului obținut. Pentru un program simplu, cu un fișier care conține codul sursă și care nu folosește nicio bibliotecă, în general, este suficient să specificăm fișierul sursă și compilatorul sau interpretorul se ocupă de restul procesului. Listing 6.1 conține comanda de compilare și de rulare a unui program simplu.

```
1 student@uso:~$ gcc main.c -o main
2 student@uso:~$ ./main
```

Listing 6.1: Compilarea și rularea unui program simplu

În cazul unui program complex care conține mai multe fișiere, vom rula mai multe comenzi, ca în Listing 6.2. Linia 1 și linia 2 conțin comenzi de compilare, linia 3 este comanda de linking, iar linia 4 este comanda de rulare.

```
1 student@uso:~$ gcc -c utils.c
2 student@uso:~$ gcc -c main.c
3 student@uso:~$ gcc utils.o main.o -o main
4 student@uso:~$ ./main
```

Listing 6.2: Compilarea și rularea unui program din mai multe surse

6.6.7 Automatizarea procesului de dezvoltare

Cum am observat mai sus, pentru aplicații complexe, compilarea sau rularea acestora constă în rularea unei secvențe de comenzi. Toate aceste comenzi trebuie rulate indiferent dacă se aduc modificări majore programului, sau dacă se schimbă doar o linie de cod. Astfel, nu e greu să ne dăm seama că avem nevoie de o modalitate de a automatiza acțiunile necesare compilării și rulării codului, astfel încât să rulăm o comandă simplă care să *se ocupe de tot*. Practic, putem să ne gândim că fișierele care alcătuiesc programul pe care l-am dezvoltat sunt ingredientele necesare pentru o prăjitură, iar prăjitura este executabilul sau chiar rezultatul rulării programului. Astfel, similar unei rețete care descrie pașii necesari pentru a obține prăjitura, vom scrie un fișier care conține pașii necesari obținerii rezultatului programului, iar utilitarul în cauză este bucătarul care urmează rețeta.

Unele dintre utilitarele cele mai folosite pentru automatizarea acestor taskuri sunt Make, Grunt, Gulp.

6.7 Medii de dezvoltare

Pentru fiecare limbaj de programare folosit avem nevoie de mediul de dezvoltare aferent (aplicații, biblioteci, compilator și/sau interpretor), pentru a putea rula aplicațiile pe care le scriem. Bineînțeles că putem scrie codul sursă în orice editor, fără a folosi nici un alt program adiacent, dar am obține un fișier cu instrucțiuni inutil, pe care nu îl putem compila sau interpreta. De aceea, în primul rând, în funcție de tipul limbajului, avem nevoie de un compilator sau un interpretor, care să ne permită acest lucru.

Pe lângă compilator sau interpretor, pentru folosirea unui limbaj de programare avem nevoie să instalăm bibliotecile folosite.

Mai departe vom analiza particularități în instalarea compilatorului sau a interpretorului pentru mai multe limbaje de programare.

6.7.1 C / C++

Pentru limbajul C, compilatorul cel mai cunoscut este GCC (*GNU Compiler Collection*). Conceput inițial pentru a compila limbajul C, în momentul de față GCC a fost extins pentru a suporta compilarea și a altor limbaje, cum ar fi C++ sau Java.

Pentru sistemele derivate din Debian, există un pacet special numit *build-essential*, instalabil folosind comanda:

```
student@uso:~$ sudo apt install build-essential
```

Pentru sistemele derivate din Redat, precum Fedora, CentOS etc. trebuie instalate pachetele separat folosind comanda:

```
student@uso:~$ sudo dnf install make gcc g++
```

Toate aceste pachete vor instala și limbajul C++.

GCC poate fi instalat și pe sisteme Windows, prin intermediul unei aplicații care care simulează un sistem Unix (Cygwin, MinGW etc.).

Pentru sisteme Windows, compilatorul de baza este integrat în aplicația Microsoft Visual C++. Microsoft Visual C++ este un ecosistem de programare și constă într-un editor cu funcții avansate (sugestii de completare a codului, sugestii de posibile erori etc.) împreună cu compilatorul de C și C++ și se recomandă folosirea acestuia pentru dezvoltarea de programe C.

Pentru C++ se poate folosi aceeași colecție de compilatoare ca pentru limbajul C, adică GCC. Cum am menționat mai sus, în momentul de față GCC suportă compilarea limbajului C++.

La folosirea gcc, sistemul analizează fișierele de intrare și hotărăște dacă limbajul compilat este C sau C++. Pentru a eficientiza procesul, se poate folosi direct g++, adică compilatorul de C++ integrat în gcc, ca în Listing 6.3.

```
student@uso:~$ g++ main.cpp -o main
student@uso:~$ ./main
```

Listing 6.3: Exemplu compilare si rulare C++

De asemenea, pentru sisteme Windows mediul de dezvoltare recomandat rămâne același. Microsoft Visual C++ este recomandat și pentru dezvoltarea de aplicații C++.

6.7.1.1 Dezvoltarea în C/C++

Cum mai menționat anterior, compilatorul cel mai folosit pentru C și C++ este gcc/g++. Pentru folosirea compilatorului este suficient să rulăm comanda **`gcc`** cu parametrii aferenți.

În procesul de compilare codul sursă este prelucrat și trece prin mai multe etape intermediare:

1. preprocesare: se înlocuiesc macro-uri, se numerotează fiecare linie de cod; rezultatul e un fișier cu extensia **.i**
2. compilare: codul sursă este transformat în cod în limbaj de asamblare; rezultatul e un fișier cu extensia **.s**
3. asamblare: codul în limbaj de asamblare este transformat în cod mașină; rezultatul e un fișier cu extensia **.o**

4. link-editare: se fac legăturile către fișiere externe care conțin simboluri sau funcții apelate din fișierul sursă; dacă avem o funcție definită într-un fișier și folosită în altul, implementarea funcției trebuie legată de apelul ei;

La simpla rulare a comenzii **gcc**, se trece prin toate etapele menționate. De exemplu, pentru un program C simplu (`main.c` fără nicio bibliotecă) vom rula comenzile indicate în Listing 6.1.

În schimb, pentru programe mai complexe, alcătuite din mai multe fișiere, e necesar să generăm fișierul obiect aferent fiecărui fișier sursă și apoi sa le link-editǎm pentru a obține executabilul final, așa cum am indicat în Listing 6.2.

6.7.1.2 Automatizarea compilării în C/C++

Cum am menționat mai devreme, este obositor să rulăm toate comenzile de mai sus pentru fiecare modificare adusă în codul sursă. De aceea există posibilitatea de a crea un fișier care conține toate aceste comenzi și care le poate rula pe toate printr-un singur apel.

Utilitarul de automatizare cel mai folosit pentru aplicațiile C/C++ este utilitarul Make. Pentru a folosi Make e suficient să creăm un fișier cu numele `Makefile` în structura programului nostru. La rularea comenzii **make**, utilitarul găsește acel fișier și execută instrucțiunile descrise.

Un fișier `Makefile` urmează formatul din Listing 6.4.

```
regula: dependente
<TAB> comanda
```

Listing 6.4: Formatul unui fișier Maefile

Fișierul poate să conțină mai multe astfel de linii, precum fișierul `Makefile` din Listing 6.5.

```
build: utils.o hello.o help.o
	gcc utils.o help.o hello.o -o hello

all:
	gcc simple_hello.c -o simple

utils.o: utils.c
	gcc -c utils.c

hello.o: hello.c
	gcc -c hello.c

help.o: help.c
	gcc -c help.c

clean:
	rm -f *.o hello
```

Listing 6.5: Exemplu Makefile

Regula reprezintă numele unei instrucțiuni. La simpla rulare a comenzii **make** prima regulă din fișier este cea care va fi executată. În schimb, dacă vom scrie **make clean**, se va executa comanda aferentă regulii `clean`.

Dependențele sunt fișiere și/sau reguli necesare pentru a putea rula o comandă. Practic, la rularea unei comenzi se verifică dacă fișierul din dependență există. Dacă acesta există, putem rula comanda, dacă nu, se caută o altă regulă cu acel nume și se va rula acea comandă (dacă dependențele ei sunt îndeplinite, dacă nu, se continuă lanțul de dependențe). Bineînțeles, sunt cazuri în care dependențele pot să lipsească.

Acum să luăm cazul programului dezvoltat mai sus, care depinde de mai multe fișiere și să presupunem că am schimbat o linie într-un fișier. La rularea comenzii **`make`**, toate fișierele ar fi refăcute, ceea ce nu este eficient. De aceea, **`make`** ține cont de încă un aspect la verificarea dependențelor. Dacă dependența (fișierul) există, **`make`** va urmări lanțul de dependențe până ajunge la fișierele de baza ale acesteia. Dacă fișierele au fost modificate după ce dependența a fost creată, aceasta va fi regenerată, în caz contrar nu se va regenera dependența.

6.7.2 Java

Fiind un limbaj hibrid, pentru Java sistemul de execuție a codului sursă este mai complex. Cum am menționat mai sus, în general limbajele hibride trec printr-o etapă de compilare în urma căreia rezultă un fișier bytecode, iar apoi acel cod este interpretat.

Pentru majoritatea limbajelor hibride (Python, JavaScript) există un sistem care conține atât compilatorul de bytecode cât și interpretorul, sau mașina virtuală. Pentru Java, cele două elemente sunt separate clar, avem compilatorul care generează codul sursă în bytecode și mașina virtuală ce execută fișierele bytecode. Cele două se instalează separat, ca două aplicații diferite și independente. Astfel, putem avea un sistem care doar generează fișierele bytecode și un alt sistem care le execută.

În ceea ce privește platformele pentru a compila și a rula Java, putem alege între două platforme principale: OracleJDK sau OpenJDK. Cum sugerează și numele, OracleJDK este dezvoltată de Oracle, în timp ce OpenJDK este o platformă open source și este special concepută pentru sisteme Unix. Pe lângă JDK (Java Development Kit), putem instala OracleJRE sau OpenJRE (Java Runtime Environment).

În timp ce JDK este sistemul complet ce compilează și execută aplicații Java, JRE este doar mașina virtuală care rulează bytecode-ul generat de către compilator. Ea este inclusă în JDK, dar poate fi instalată și separat. Astfel, putem avea sisteme care doar rulează aplicații Java și care necesită bytecode-ul generat de JDK.

Pe sisteme Linux OpenJDK este într-un pachet cu numele de forma `openjdk-X-jdk`, unde `X` este numele versiunii. De exemplu, pe Ubuntu 18.04 pachetul este `openjdk-8-jdk`.

Pentru sisteme care rulează Windows, se recomandă instalarea OracleJDK. Deși se găsesc versiuni OpenJDK compatibile cu Windows, acestea nu sunt întreținute și prezintă riscuri de securitate.

Pentru a instala OracleJDK pe un sistem Windows, accesați site-ul Oracle și urmați instrucțiunile (https://www.oracle.com/technetwork/java/javase/overview/install-windows-142126.html).

O particularitate a Java este aceea că bibliotecile sunt împachetate ca niște arhive (jar - *Java Archive*) care sunt incluse în aplicațiile pe care le dezvoltăm. Fișierul `.jar` conține

resurse diverse, de la definiții de clase la imagini sau alte fișiere multimedia. Fiecare astfel de arhivă poate conține un fișier manifest ce oferă informații despre pachet și despre utilizarea sa.

6.7.3 C#

Pentru C# sistemul de execuție este .NET, dezvoltat de Microsoft. Așadar, sistemele cu Windows au deja sistemul instalat.

Pentru sisteme Unix, se recomanda folosirea Mono, cu ajutorul pachetului `mono-devel`. Mono este platforma open-source pentru C#, susținută tot de Microsoft.

Pentru aplicațiile dezvoltate în C# putem folosi fie Mono, fie .NET. Indiferent de platforma folosită, pentru a putea instala biblioteci putem folosi utilitarul NuGet, dezvoltat de Microsoft.

La fel ca în Java, în C# bibliotecile sunt similare unei arhive.

6.7.4 Python

La fel ca Java, Python e un limbaj de programare hibrid. Diferența între cele două este că pentru Python nu se diferențiază compilatorul de mașina virtuală, cele două sunt integrate.

Pentru a folosi Python pe un sistem Unix este suficient să instalăm pachetul `python`.

Pentru sistemele Windows, site-ul *python.org* ne oferă resursele necesare pentru instalare.

La instalarea interpretorului de Python este foarte importantă versiunea acestuia. Putem alege între Python2 sau Python3, cele două fiind diferite. Python 3.0 a fost lansat în anul 2008 și prezintă o sintaxă diferită de Python 2. De aceea este important să instalăm un interpretor potrivit pentru versiunea Python pe care o folosim la dezvoltarea aplicațiilor.

Se recomandă folosirea Python3 deoarece Python2 nu mai este adus la zi.

Când vine vorba de dezvoltarea aplicațiilor folosind Python, abordarea e puțin diferită de C, pentru că Python e un limbaj interpretat. Drept urmare, pentru a putea rula programul pe care îl scriem, va trebui să rulăm interpretorul și să îi spunem acestuia care e programul pe care dorim să îl interpreteze.

Pentru că în general vom lucra la proiecte complexe, care conțin mai multe fișiere, similar cu programele din C, în Python vom avea un fișier de bază care depinde de celelalte fișiere. Astfel, pentru a rula aplicația e suficient să specificăm fișierul de pornire, ca în Listing 6.6.

```
1 student@uso:~$ python main.py
```

Listing 6.6: Exemplu rulare Python

Cum am menționat mai sus, Python oferă un număr mare de biblioteci și framework-uri care pot fi integrate în programele pe care le dezvoltăm. Pentru a putea folosi o

bibliotecă, aceasta trebuie să existe pe sistemul care rulează programul. Pentru instalarea unei biblioteci, folosim utilitarul **pip**, așa cum am precizat în Secțiunea 3.4.1.

6.7.5 Node.js

Când menționăm Node.js, ne referim deja la mediul de dezvoltare al aplicațiilor JavaScript.

JavaScript este un limbaj folosit pentru aplicații web, iar interpretoarele de JavaScript sunt integrate în browsere. De aceea a apărut Node.js, care e o platformă ce permite executarea codului JavaScript în afară browserului. Unul din interpretoarele de JavaScript este V8, dezvoltat de Google și integrat în Google Chrome. Node.js integrează acest interpretor.

Pentru a instala Node.js pe sisteme Linux, recomandăm folosirea instanței Snap ca în Listing 6.7. Platforma se poate instala și instalând pachetul `nodejs` în mod clasic, dar se va descărca o versiune foarte veche a platformei.

```
1 student@uso:~$ sudo snap install --classic node
2 node (12/stable) 12.18.4 from NodeSource, Inc. (nodesource) installed
3
4 student@uso:~$ node
5 Welcome to Node.js v12.18.4.
6 Type ".help" for more information.
7 >
```

Listing 6.7: Instalare Node.js

Pentru instalarea pe sisteme ce rulează Windows, se recomandă descărcarea kitului de instalare de pe site-ul Node.js.

În general, dacă alegem să dezvoltăm o aplicație folosind Node.js, dorim să dezvoltăm o aplicație web la care atât partea de server, cât și cea de client să fie scrise folosind JavaScript. Pentru partea de server vom folosi Node.js pentru a putea rula programul, în timp ce partea de client va fi interpretată în browser.

Când dezvoltăm o aplicație în Node.js, aspectele de care trebuie să ținem cont sunt similare cu cele care au legătură cu Python. Similar cu Python, pentru rularea codului sursă scris avem nevoie de interpretorul necesar instalat.

Pentru a rula o aplicație Node.js folosim comanda:

```
1 student@uso:~$ node main.js
```

Pentru manipularea modulelor externe se folosește utilitarul **npm**. Pentru instalarea unei biblioteci comanda necesară este **npm install** <**biblioteca**>.

Cum am menționat mai sus, de cele mai multe ori, când folosim Node.js, vom crea aplicații complexe, de cele mai multe ori, aplicații web. De aceea, după ce scriem codul sursă vom dori să îl prelucrăm. De exemplu putem alege să minimizăm codul, adică să restrângem tot codul într-un singur fișier, greu inteligibil pentru dezvoltatori, pentru a ne asigura că nu poate fi ușor copiat. Pe lângă asta, putem alege să trecem tot codul sursă printr-un utilitar care verifică dacă există greșeli, cum ar fi nedeclararea unei variabile.

Pentru că sunt multe astfel de operații pe care vrem să le aplicăm codului sursă înainte de a-l distribui, putem automatiza întregul proces. Unul din utilitarele folosite la această automatizare este **grunt**. Putem să ne gândim la **grunt** ca la un utilitar semănător cu Make, doar că operațiile pe care **grunt** le efectuează implică preprocesarea codului sursă. Similar cu un fișier `Makefile`, pentru **grunt**, vom crea un fișier numit `Gruntfile` care conține toate regulile, ca în Listing 6.8.

```
module.exports = function(grunt) {

 grunt.initConfig({
   jshint: {
     files: ['Gruntfile.js', 'src/**/*.js', 'test/**/*.js'],
     options: {
       globals: {
         jQuery: true
       }
     }
   },
   watch: {
     files: ['<%= jshint.files %>'],
     tasks: ['jshint']
   }
 });

 grunt.loadNpmTasks('grunt-contrib-jshint');
 grunt.loadNpmTasks('grunt-contrib-watch');

 grunt.registerTask('default', ['jshint']);

};
```

Listing 6.8: Exemplu cod Gruntfile

6.8 Depanarea programelor

Pe măsură ce scriem tot mai multe programe, ne dăm seama că de cele mai multe ori acestea nu funcționează cum ne dorim de la început. De multe ori se va întâmpla ca programul rulat să arunce o eroare sau să nu obțină rezultatul dorit. Deși am dori să putem spune că *e vina calculatorului*, orice eroare sau comportament eronat apare dacă noi nu am scris ceva bine în codul sursă. Erorile pot să varieze de la un simbol uitat, la accesări ilegale de memorie sau chiar erori în logica programului. Astfel, o mare parte din timpul destinat dezvoltării aplicațiilor îl dedicăm depanării. De aceea e de preferat să fim eficienți în descoperirea erorilor.

Am menționat mai sus că unul din aspectele importante ale unui IDE este posibilitatea de a seta puncte de oprire în execuția programului. Astfel, odată ce o linie de cod este executată, întregul proces se oprește și noi putem monitoriza starea curentă. Adică putem vizualiza valorile pe care le au variabilele sau adresele de memorie la care se găsesc, ca în Figura 6.6.

Unul dintre cele mai cunoscute utilitare pentru depanare este GDB. GDB oferă suport pentru toate operațiile menționate mai sus și este compatibil cu toate sistemele de operare existente.

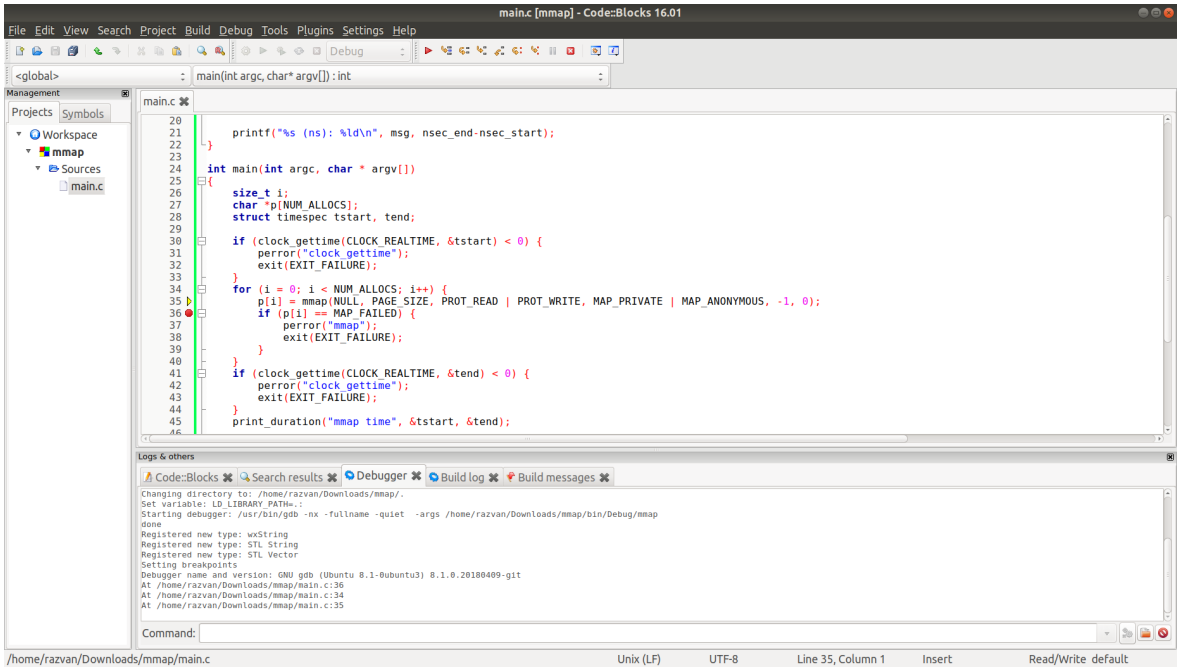


Figura 6.6: Interfață de depanare într-un IDE (coblocks)

Cea mai simplă metodă de a depana un program este să afișăm mesaje pe parcursul execuției. Astfel putem vizualiza valorile variabilelor sau blocurile de cod care se execută sau nu. Deși pare un mod primitiv de a face depanare, este cel mai la îndemână.

De fiecare dată când ceva nu funcționează, se recomandă verificarea pas cu pas a liniilor de cod și printarea rezultatului. De foarte multe ori eroarea se observă prin afișarea variabilelor.

În plus, nu este o practică eficientă să scriem zeci de linii de cod fără a testa. E recomandat să scriem în incremente mici și să ne asigurăm ca fiecare bloc funcționează. Altfel, ne vom trezi puși în fața a zeci de linii de cod care nu funcționează și va fi dificil să identificăm problema.

6.9 Sisteme de management și versionare a codului sursă

De cele mai multe ori, vom lucra la proiecte complexe, în cadrul unei echipe. Acest lucru înseamnă că vom avea multe fișiere cu cod sursă pe care le vom modifica în paralel cu alte persoane. Drept urmare, avem nevoie de o modalitate de a împărți proiectul într-un mod eficient. Deși la început acest lucru nu pare neapărat dificil, pe măsură ce proiectul crește, este tot mai greu. Pe lângă partajarea efectivă a întregii structuri a programului, avem nevoie de o modalitate de a urmări modificările aduse și cine ce modificări a făcut. Să ne gândim la situația în care suntem o echipă de 5 oameni și cineva face o modificare ce rezultă într-un comportament defectuos al programului. Având în vedere că suntem 5 oameni care au adus simultan modificări codului sursă, va fi dificil să identificăm eroarea. În cazul unui proiect avansat, problema e și mai mare.

De aceea metode precum *folosim un director online comun sau chiar trimitem sursele*

modificate prin email nu sunt recomandate pentru a lucra în echipă la un proiect și de aceea există sisteme dedicate partajării de cod sursă. Un astfel de sistem este Git.

Git este un sistem de management și versionare a codului sursă care permite partajarea unui proiect în cadrul unei echipe. Proiectul este stocat într-un depozit (*repository*). Repository-ul conține fișierele efective ale proiectului și multe alte informații despre acesta. Folosind Git, fiecare persoană lucrează la o versiune proprie a proiectului, pe care apoi o urcă online și este automat integrată în proiect. Ceilalți colaboratori descarcă modificările aduse când doresc. Sistemul urmărește modificările pe care fiecare persoană le aduce și la orice moment putem descărca o versiune anterioară a codului sursă. Pentru a realiza toate acestea avem la dispoziție următoarele operații de bază:

- **`clone`**: se clonează întreg repository-ul pe sistemul local; practic se va crea un director care conține toate fișierele aferente programului, plus niște fișiere care stochează informații despre proiect (ex, cine a creat repository-ul, ce modificări au fost aduse etc.);
- **`commit`**: se salvează toate modificările aduse proiectului; starea actuală este salvată local; dacă modificările nu sunt publicate online, acestea nu sunt vizibile colaboratorilor;
- **`push`**: se publică modificările salvate prin commit; odată publicate aceste modificări, colaboratorii pot să vizualizeze cine le-a adus și pot să urmărească schimbările linie cu linie
- **`pull`**: se descarcă local ultimele modificări publicate de ceilalți colaboratori și se integrează în versiunea locală a proiectului.

6.10 Licențe pentru programe

Un aspect important când vine vorba de dezvoltarea programelor este licența acestuia.

Licența unui program oferă informații despre dreptul de folosire și distribuție a programului.

De multe ori vom dori să publicăm programele pe care le dezvoltăm pentru a permite altor persoane să le folosească sau chiar să le integreze în proiectele proprii. Când facem acest lucru, este important să definim scopul aplicației pe care o dezvoltăm și să îi atribuim o licență corespunzătoare. Astfel, cei care vor folosi programul nostru vor ști ce pot și ce nu pot să facă cu el.

Pe de altă parte, în procesul de dezvoltare a unui program vom folosi un număr semnificativ de biblioteci și framework-uri. Când facem acest lucru este foarte important să verificăm licența fiecărui pachet extern pe care îl integrăm în aplicația noastră. Licențele modulelor externe folosite pot influența folosirea programului pe care îl dezvoltăm.

Acum că știm că e important să stabilim o licență pentru programele pe care le dezvoltăm, întrebarea care urmează este, cum stabilim aceste reguli și cum alegem licența potrivită. De foarte multe ori chiar licența unui modul extern folosit poate dicta licența pe care trebuie să o atribuim programului nostru. Ca să înțelegem mai bine ce

caracteristici poate să aibă o licență, vom menționa cele mai cunoscute și folosite licențe software.

6.10.1 GNU GPL

GNU GPL este prescurtarea de la *GNU General Public License* și de cele mai multe ori ne referim la această licență drept GPL.

Un program însoțit de o licență GPL poate fi distribuit doar dacă sursa acestuia este publică și programul poate fi integrat în orice alt program care este public și nu are nicio restricție de folosire. Practic, orice program care folosește un modul GPL trebuie să aibă la rândul lui licență GPL. În plus, biblioteca integrată poate fi modificată

6.10.2 GNU LGPL

GNU LGPL (simplu LGPL) este o licență cu restricții mai reduse față de GNU GPL. Numele vine de la *GNU Lesser General Public License*. În cazul acestei licențe programul în cauză poate fi integrat în alte programe care au orice altă licență. Singura restricție este ca modulul LGPL folosit să fie făcut public în cadrul aplicației. De asemenea, în cazul în care biblioteca LGPL este modificată și abia apoi integrată într-o aplicație, biblioteca modificată trebuie publicată sub licență LGPL.

6.10.3 MIT

MIT este o licență mai permisivă față de GPL. Orice bibliotecă care are licența MIT poate fi integrată în orice program și acesta poate fi distribuit, fără necesitatea de a face programul public. Singura condiție este ca licența MIT să apară în programul principal.

6.11 Sumar

Pe parcursul acestui capitol am prezentat principalele tipuri de limbaje de programare: compilate, interpretate și hibride. Am discutat despre avantajele și dezavantajele fiecăruia. De asemenea, am trecut în revistă principalele limbaje de programare folosite din fiecare categorie, evidențiind unde se folosește fiecare.

Sunt importante de reținut etapele dezvoltării unei aplicații. În primul rând, trebuie să hotărâm ce aplicație dorim să realizăm, cine o va folosi și care va fi mediul hardware și software în care va rula. Odată stabilite acestea, trebuie să alegem limbajul potrivit, mediul de programare și să instalăm bibliotecile necesare.

Următorul pas este scrierea codului sursă, compilarea acestuia și testarea lui. Acest proces este repetat de foarte multe ori până la definitivarea aplicației.

Odată ce aplicația funcționează corespunzător, trebuie să îi adăugăm o licență de distribuție și să facem un pachet de instalare.

Aplicația este gata de lansare.

Capitolul 7

Interfața în linia de comandă

O aplicație sau un sistem oferă uzual o interfață de interacțiune cu utilizatorul. Cu ajutorul acestei interfețe utilizatorul comandă sistemul sau aplicația pentru a-și rezolva o nevoie.

Unele aplicații, numite și programe batch, sunt neinteractive. Adică nu oferă o interfață cu utilizatorul, spunem că rulează în background. Aplicațiile interactive, pe de altă parte, oferă o interfață care poate fi grafică (*Graphical User Interface* - GUI), în linia de comandă (*Command Line Interface* - CLI) sau web (WebUI), fiecare cu avantajele sale. Un utilizator tehnic va cunoaște diferitele tipuri de interfețe, care sunt plusurile acestora și când să le folosească. Va urmări eficiența operațiilor și va folosi interfețe CLI pentru a ajunge cât mai rapid la rezultat.

În general spunem că interfața grafică este mai accesibilă, în vreme ce interfața în linia de comandă este universală (pentru diferite sisteme de operare șî aplicații) și oferă acces rapid la funcționalitățile sistemului.

Interfața grafică este folosită pe toate mediile mobile și în cele mai multe medii desktop. Elementele GUI, descrise în Secțiunea 1.1, sunt WIMP: Window, Icon, Menu, Pointer. Pe sistemele mobile se folosește un ecran tactil (*touchscreen*).

Interfața în linia de comandă este mai simplă, eficientă și cu facilități complexe. Este folosită în medii specializate, în servere și în medii desktop de către utilizatori cu profil tehnic. Interfața în linia de comandă prezintă, în general, un prompt unde utilizatorul introduce comenzi pentru a interacționa cu sistemul sau aplicația. Exemple de interfețe în linia de comandă sunt: shellul Linux, PowerShell, python shell, game consoles, MATLAB/Octave, AutoCAD.

În Capitolul 1 ne-am concentrat pe interfața grafică (GUI). În acest capitol vom insista pe interfața în linia de comandă (CLI).

7.1 Shellul. Funcționarea shellului

Așa cum am prezentat în Figura 1.1, interacțiunea utilizatorului cu sistemul de operare se realizează printr-o aplicație numită shell. Shellul poate fi GUI sau CLI, exemplificat în Figura 1.2. Deși termenul de shell se poate referi la shell GUI sau shell CLI, cel mai

adesea ne referim la shell CLI. Este cazul și acestui capitol, în care atunci când folosim termenul „shell” ne vom referi la shell CLI.

7.1.1 Interacțiunea cu shellul

În general, un shell CLI oferă utilizatorului un prompt în care acesta introduce comenzi. Un prompt este un șir de caractere care indică utilizatorului unde trebuie să tasteze comanda. Promptul poate conține informații suplimentare utilizatorului, precum numele utilizatorului, numele stației de lucru, directorul curent. Exemple de prompturi sunt în Listing 7.1; acestea pot avea orice formă, utilizatorul trebuie doar să introducă acele comenzi pe care le dorește.

```
1 student@uso:~$
2 tony@lounge%
3 C:\>
```

Listing 7.1: Prompturi de shell

O comanda este, la fel, un șir de caractere. O dată apăsată tasta `Enter` șirul de caractere ajunge la aplicația shell care interpretează șirul / comanda. Din acest motiv shellul (CLI) se numește **interpretor de comenzi** (*command interpreter*). Interpretarea comenzii duce la executarea acțiunii corespunzătoare din partea shellului. De exemplu comanda **`ls`** duce la crearea unui proces, comanda **`cd`** duce la schimbare directorului curent, comanda **`alias`** duce la crearea unui alias de comandă. Figura 7.1 sumarizează interacțiunea dintre utilizator, shell și sistemul de operare, cu realizarea unei acțiuni dorite de utilizator.

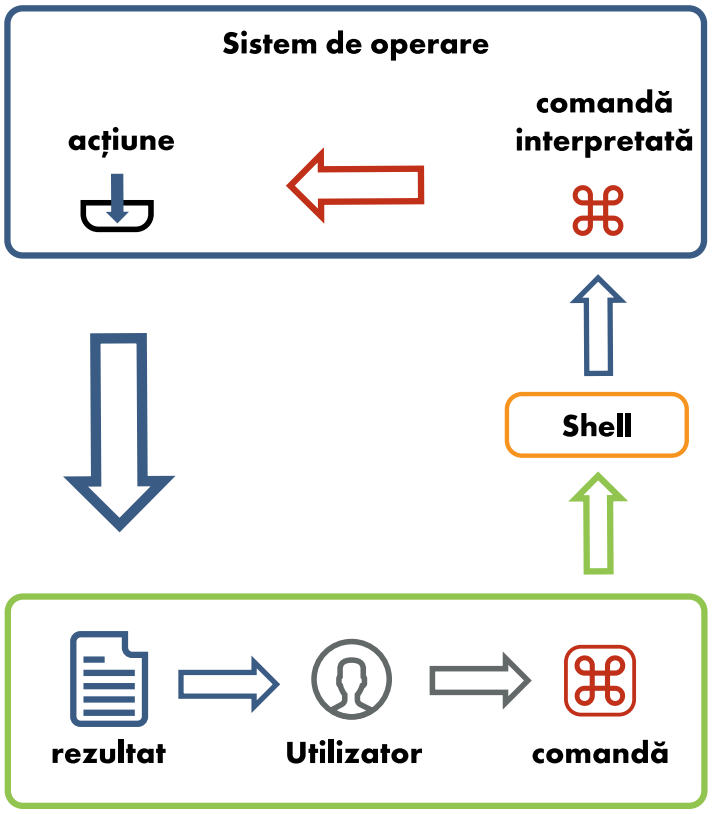


Figura 7.1: Interacțiunea utilizatorului cu shellul prin comenzi

În cazul unui shell CLI, folosim tastatura ca formă de intrare (*input*) și ecranul ca formă de ieșire (*output*). Adică introducem comenzile de la tastatură pentru a fi transmise shellului, iar rezultatul rulării comenzilor este afișat de shell la ecran, unde poate fi urmărit de utilizator.

Așa cum am precizat în Secțiunea 4.6.1, un shell rulează în cadrul unui **terminal**, care este interfața acestuia de interacțiune cu utilizatorul. Descriptorii standard ai shellului (intrarea standard, ieșirea standard, ieșirea de eroare standard) referă terminalul. Shellul este un proces care primește comenzi la intrarea standard, le interpretează și apoi afișează rezultatul la ieșirea standard. Atunci când rulăm comenzi spunem că le scriem în terminal. Când shellul interpretează comenzile spunem că rezultatul este afișat în terminal. Modul de interacțiune este indicat în Figura 4.14.

Terminalul este, așadar, un dispozitiv reprezentând canalul de comunicare între shell și utilizator. Terminalul controlează intrarea și ieșirea unui shell. În general, fiecare shell are un terminal.

7.1.2 Facilități shell

În folosirea shellului, un utilizator urmărește eficacitate și eficiență: să rezolve sarcina pe care o are de făcut și să facă acest lucru cât mai repede. Eficacitatea este atinsă având acces la comenzile adecvate și la parametrii corespunzători. Dacă o comandă nu există, un pachet poate fi instalat pentru a oferi acea comandă și pentru a extinde plaja de acțiune a shellului.

Pentru eficiență utilizatorul trebuie să tasteze cât mai rapid comenzi. Acest lucru poate presupune două tipuri de acțiuni: generarea rapidă a comenzii sau editarea rapidă a unei comenzi, modificarea ei.

Pentru generarea rapidă a unei comenzi shellul oferă facilitățile de completare a comenzilor și căutare în istoricul de comenzi.

7.1.2.1 Completarea comenzilor

Completarea comenzilor (*shell completion*, *command-line completion*) permite completarea (expandarea) automată a unei comenzi sau a unui argument fără a fi nevoie de tastarea acesteia în totalitate. Acest lucru se întâmplă folosind tasta `Tab`. Cu ajutorul tastei `Tab` putem completa fie numele unei comenzi, fie numele unui argument.

Adică scriem o parte din comandă sau argument și apoi apăsăm tasta `Tab`; dacă shellul poate deduce ce comandă sau ce argument vrem să folosim, va completa întreg numele său. Dacă sunt mai multe posibilități, la apăsarea de două ori a tastei `Tab` se afișează posibilitățile de atunci. În mod uzual, se mai scrie o secvență neambiguă și apoi se apasă iar `Tab`. În felul acesta, scriem foarte repede comenzile. Completarea comenzilor este o facilitate esențială în folosirea eficientă a shellului.

7.1.2.2 Istoricul de comenzi

Shellul menține un istoric de comenzi. Acest lucru e util pentru un utilizator care dorește să refolosească o comandă anterioară. Putem afișa întreg istoricul de comenzi al sesiunii curente de shell folosind comanda **`history`** ca în Listing 7.2.

```
student@uso:~$ history
    1  ls
    2  iostat
    3  pidstat
    4  vmstat
    5  mpstat
    6  lsof +D /dev
    7  lsof +D /home/student
    8  lsof +D /var/log/
    9  sudo lsof +D /var/log/
   10  netstat
   [...]
```

Listing 7.2: Afișarea istoricului de comenzi

Istoricul de comenzi poate fi folosit folosind combinații de taste sau comenzi specifice shellului. Listing 7.3 prezintă exemple uzuale de folosire a istoricului comenzilor.

```
1  student@uso:~$ snap install pdftk
2  error: access denied (try with sudo)
3  student@uso:~$ sudo !!
4  sudo snap install pdftk
5  [sudo] password for student:
6
7  student@uso:~$ !-13
8  ls /usr/share/pixmaps/gvim.svg
9  /usr/share/pixmaps/gvim.svg
10
11 (reverse-i-search)'ink': pinky teacher
```

Listing 7.3: Folosirea istoricului de comenzi

Cel mai direct mod de folosirea a istoricului de comenzi este prin intermediul tastelor săgeată (sus și jos) care parcurg comenzile din istoric. Aceleași funcționalități (comanda anterioară, comanda următoare) pot fi obținute din combinațiile de taste `Ctrl+p` sau `Ctrl+n` (împrumutate din Emacs).

Dacă vrem să refolosim comanda anterioară avem construcția `!!`. Aceasta este utilă în special când vrem să folosim comanda anterioară împreună cu altă comandă. De exemplu, dacă am rulat o comandă fără **`sudo`**, așa cum este cazul liniilor `1-5` din Listing 7.3. Mai general putem folosi comenzi precum `!-3` sau `!-2` ca să accesăm a `N`-a comandă anterioară din istoric; de exemplu, linia `7` din Listing 7.3 refolosește a `13`-a comandă anterioară din istoricul de comenzi.

Mai adesea, însă, folosim funcționalitatea de căutare înapoi (*reverse search*) în istoricul comenzilor. Atunci căutăm o comandă anterioară, prezentă la un moment dat în istoric, pe care o putem rula din nou sau edita pentru o nouă rulare. Ca să folosim căutarea înapoi în istoric, tastăm `Ctrl+r`. În acel moment se schimbă promptul shellului și putem introduce un șir de caractere. În momentul introducerii șirului, pe măsură ce tastăm, shellul va face căutare înapoi și va face match pe acele comenzi care corespund; este cazul liniei `11` din Figura 7.3. Putem să și ștergem din șir dacă dorim. Nu se face

match doar pe începutul comenzii, ci pe orice subșir din comandă. Dacă există mai multe comenzi care conțin subșirul introdus, putem apăsa în continuare `Ctrl+r` pentru a găsi comanda căutată. Apoi, dacă apăsăm pe taste precum săgeți sau combinații de taste pentru editare se revine în modul normal de lucru, se părăsește modul de căutare înapoi.

Facilitatea de căutare înapoi este, din nou, esențială pentru folosirea eficientă a shellului. Împreună cu command-line completion sunt facilități indispensabile unui utilizator productiv în shell.

7.1.2.3 Editarea comenzilor

După ce am generat în shell o comandă (folosindu-ne și de facilități de completare a comenzii) sau după ce am regăsit o comandă în istoricul shellului, de multe ori va trebui să o modificăm. Poate un parametru este greșit, poate vrem să schimbăm comanda dar să păstrăm parametrul, poate avem un cuvânt sau o parte a unei căi în plus.

În mod simplist, editarea unei comenzi se face parcurgând comanda cu tastele săgeți (stânga și dreapta) și folosind tastele `Backspace` și `Delete`. Anumite deplasări mai rapide în cadrul comenzii se pot face folosind tastele `Home` și `End`.

O formă mai bună de editare este folosind combinațiile de taste furnizate de shell. Aceste combinații de taste au două avantaje:

1. Nu îndepărtează degetele de pe tastele comune, așa cum le îndepărtăm atunci când folosim tastele săgeți, `Home`, `End`.
2. Permit editări mai rapide și mai complexe ale comenzii.

Tabelul 7.1 prezintă cele mai comune astfel de combinații de taste și efectul lor.

Tabelul 7.1: Combinații de taste în shell

Combinație de taste	**Efect**
`Ctrl+c`	terminarea procesului interactiv curent / anularea comenzii tastate în shell
`Ctrl+\`	terminarea procesului interactiv curent
`Ctrl+d`	livrarea EOF (*End of File*) și închiderea shellului
`Alt+.`	completarea în shell a ultimului argument al ultimei comenzi
`Ctrl+a`	plasarea cursorului de editare la începutul comenzii
`Ctrl+e`	plasarea cursorului de editare la sfârșitul comenzii
`Alt+b`	plasarea cursorului un cuvânt înapoi
`Alt+f`	plasarea cursorului un cuvânt înainte
`Ctrl+u`	ștergerea din poziția curentă până la începutul comenzii
`Ctrl+k`	ștergerea din poziția curentă până la sfârșitul comenzii
`Ctrl+d`	ștergerea literei de sub cursor (echivalent `Delete`)
`Alt+d`	ștergerea cuvântului din fața cursorului
`Alt+Backspace`	ștergerea cuvântului din spatele cursorului

Combinațiile de taste `Ctrl+c` și `Ctrl+\` sunt folosite pentru a opri procesul curent, prin transmiterea semnalului, respectiv, `SIGINT` și `SIGQUIT`, așa cum am precizat în .

Combinația de taste `Alt+.` este folosită pentru cazurile în care una dintre comenzile anterioare conține ca ultim argument o valoare refolosibilă. De exemplu, după o comandă `mkdir`, care primește ca argument numele unui director, putem folosi o comandă `cd`, urmată de apăsarea combinației de taste `Alt+.`, pentru a completa numele directorului.

Multe dintre aceste combinații de taste sunt împrumutate din editorul Emacs. Mai mult, inclusiv tastele `Ctrl+r` (pentru căutare în istoric), `Ctrl+p` (comanda anterioară) și `Ctrl+n` (comanda anterioară) sunt împrumutate tot din editorul Emacs.

Recomandăm folosirea acestor combinații de taste cât mai mult pentru editarea comenzilor. Ajută la creșterea eficienței utilizării shellului și concentrarea timpului și efortului pe rezolvarea sarcinii propuse.

7.1.2.4 Biblioteca Readline

Editarea comenzilor în shell cu ajutorul combinațiilor de taste de mai sus este disponibilă și în alte interfețe în linia de comandă precum Python shell sau MySQL shell sau SQlite shell sau altceva. Toate combinațiile de taste prezentate în tabelul Tabelul 7.1 sunt, în general, prezente și în aceste interfețe.

Acest lucru se întâmplă pentru că funcționalitatea de editare de comenzi este oferită de biblioteca Readline [1], bibliotecă la care sunt legate majoritatea shellurilor și a interfețelor în linia de comandă.

Facilitățile Readline sunt apelate de shell pentru prelucrarea intrării de la utilizator, adică a comenzilor și parametrilor ei. Biblioteca permite asocierea între comenzi Readline și o combinație de taste. De exemplu: *beginning-of-line (C-a)* înseamnă că pentru a ajunge la începutul liniei, vom tasta `Ctrl+a`.

O listă exhaustivă a comenzilor Readline și a combinației de taste (numită *keybinding*) găsiți în documentația bibliotecii Readline (online [2] sau pagina de manual) sau documentația shellului Bash (online [3] sau pagina de manual). Pentru accesarea paginilor de manual folosiți comenzile din Listing 7.4. Pentru a localiza comabinațiile de taste, căutați șirul *EDITING COMMANDS*, respectiv șirul *Command Names* în pagina de manual pentru `readline` și `bash`.

```
1 student@uso:~$ man readline
2 ... # cautare dupa sirul "EDITING COMMANDS"
3
4 student@uso:~$ man bash
5 ... # cautare dupa sirul "Command Names"
```

Listing 7.4: Documentație pentru combinațiile de taste din shell (prin Readline)

[1] https://tiswww.case.edu/php/chet/readline/rltop.html

[2] https://tiswww.case.edu/php/chet/readline/readline.html#SEC3

[3] https://www.gnu.org/software/bash/manual/html_node/Readline-Interaction.html#Readline-Interaction

7.1.3 Funcționarea shellului

Așa cum am indicat mai sus, shellul primește de la utilizator, prin intermediul terminalului, un șir de caractere pe care îl interpretează ca fiind o comandă și argumentele acesteia. Acestea sunt separate prin caracterul spațiu.

Odată obținută comanda, shellul va trebui să execute acțiunea aferentă acelei comenzi. Pentru acesta, în mod uzual, găsește un fișier executabil aferent comenzii și creează un proces din acel fișier. De exemplu, pentru comanda **`ls`** fișierul executabil este `/bin/ls`, pentru comanda **`sudo`**, fișierul executabil este `/usr/bin/sudo`.

Pentru localizarea fișierului executabil aferent unei comenzi, shellul folosește variabila de mediu `PATH`.

7.1.3.1 Variabila de mediu PATH

Despre variabile de mediu vom discuta detaliat în Secțiunea 7.2.3. Pe moment e suficient sa știm că o variabilă are un nume și o valoare.

Pentru a afișa valoarea (conținutul) unei variabile folosim comanda **`echo`** și prefixăm numele variabilei cu simbolul `$` (dolar) ca în Listing 7.5. Variabila de mediu `PATH` are o valoare ca cea de pe linia `4`.

```
1 student@uso:~$ echo $USER
2 student
3 student@uso:~$ echo $PATH
4 /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/
  usr/local/games:/snap/bin
```

Listing 7.5: Afișarea valorii variabilelor shell

Variabila `PATH` conține directoarele în care sunt căutate fișiere executabile pentru o comandă, separate prin `:` (două puncte). Shellul va prefixa, în ordine, numele comenzii cu numele fiecărui director din variabila `PATH` și va verifica existența unui fișier cu acel nume. Dacă există, va crea un proces din acel executabil, altfel trece la următorul director. Astfel, în cazul comenzii **`ls`**, dacă valoarea variabilei `PATH` e cea din exemplu de mai sus, se caută, pe rând existența fișierelor executabile `/usr/local/sbin/ls`, `/usr/local/bin/ls`, `/usr/sbin/ls` și așa mai departe.

Dacă nu găsește nici un fișier executabil, se afișează un mesaj de eroare ca în Listing 7.6.

```
1 student@uso:~$ dg
2 dg: command not found
```

Listing 7.6: Eroare localizare fișier executabil

Odată identificat fișierul executabil aferent comenzii, shellul creează un proces din acel executabil.

Dacă se transmite comanda prin cale (absolută sau relativă), nu se mai caută în `PATH`.

Directorul curent este absent din `PATH`, de aceea trebuie să rulăm explicit un executabil `./nume_executabil`.

Pentru a afla, fără a rula, care este fișierul executabil aferente unei comenzi folosim comanda **`which`** ca în Listing 7.7.

```
student@uso:~$ which ls
/bin/ls
student@uso:~$ which pdftk
/snap/bin/pdftk
student@uso:~$ which sudo
/usr/bin/sudo
```

Listing 7.7: Aflarea executabilului unei comenzi (which)

7.1.3.2 Crearea unui proces nou

Așa cum am precizat în Secțiunea 4.5, un proces nou este creat prin intermediul unui alt proces, de obicei shellul. Shellul este procesul care pornește de la o comandă, identifică fișierul executabil aferent acelei comenzi din variabila `PATH` și apoi creează un proces din acel executabil.

Procesul este creat printr-un API intern al sistemului de operare, care în Linux este dat de două funcții: `fork()` și `exec()`. `fork()` este un apel care creează un proces copil identic procesului părinte (o clonă a acestuia) iar `exec()` este apelul care modifică imaginea procesului copil cu cea din executabilul primit ca parametru.

Astfel când shellul primește comanda **`ls`** și identifică executabilul `/bin/ls`, va crea un proces copil identic (tot shell) folosind `fork()` și apoi va înlocui imagina de executabil cu `/bin/ls` folosind `exec()` așa cum am prezentat în Figura 4.8.

Procesul nou creat are ca proces părinte shellul. De exemplu, dacă într-un shell rulăm comanda **`sleep 100`** care nu face nimic timp de `100` de secunde, iar în alt shell afișam informații despre procesul creat din această comandă, vedem că procesul părinte aferent este chiar shellul, ca în Listing 7.8. Vedem că shellul în care rulăm comanda **`sleep 100`** are PID-ul `14599`, PID care este indicat ca PID al procesului părinte (`PPID`) în rularea comenzii **`ps -f -C sleep`** într-un alt shell.

```
# int-un shell
student@uso:~$ echo $$
14599
student@uso:~$ sleep 100

# in alt shell
student@uso:~$ ps -f -C sleep
UID        PID  PPID  C STIME TTY          TIME CMD
student  14619 14599  0 12:04 pts/1    00:00:00 sleep 100
```

Listing 7.8: Shellul ca proces părinte

După ce shellul creează un proces copil, shellul se blochează așteptând ca acesta să își încheie execuția. În acest timp informațiile transmise la terminal la intrarea standard sunt fie citite de procesul nou fie sunt ținute într-un buffer al terminalului. De exemplu, dacă rulăm comanda **`sleep 5`** și tastăm rapid informații, acele informații vor fi transmise shellului după încheierea procesului creat din comanda **`sleep 5`**.

Dacă dorim ca shellul să nu aștepte încheierea procesului nou creat, atunci rulăm comanda în background cu ajutorul operatorului `&`, așa cum am arătat în Secțiunea 4.3.3.

După ce procesul se încheie, shellul se deblochează. În acest moment shellul reține codul de ieșire (*exit code*) al procesului creat. Acest cod poate fi afișat cu ajutorul construcției `$?`. Un cod de ieșire cu valoarea `0` înseamnă că procesul și-a încheiat execuția cu succes. Un cod de ieșire cu altă valoare înseamnă că procesul și-a încheiat execuția cu eroare.

7.1.4 Comenzi interne și comenzi externe

Am precizat că atunci când rulăm o comandă, shellul identifică un fișier executabil aferent acelei comenzi. Numim aceste comenzi comenzi externe, întrucât executabilul aferent lor este separat de shell.

Din rațiuni de viteză și pentru că nu ar fi posibil altfel, unele comenzi sunt implementate la nivelul shellului. Adică rularea acelei comenzi nu conduce la crearea unui proces nou dintr-un executabil, ci duce la rularea unei componente de program din procesul shell curent. Aceste comenzi sunt numite comenzi interne sau *shell builtins*.

Comenzile interne sunt obligatorii pentru anumite funcționalități. De exemplu comanda **`exit`** este comandă internă. Dacă ar fi comandă externă, s-ar crea un proces nou obținut din executabilul `exit` care și-ar încheia execuția, fără a afecta shellul. La fel, comand **`cd`** este comandă internă ca să schimbe directorul curent aferent shellului. Dacă ar fi comandă externă, atunci s-ar crea un proces nou din executabilul cd care ar schimba directorul și și-ar încheia execuția, fără a afecta însă shellul.

Comenzile interne sunt folosite și din rațiuni de eficiență. Crearea unui proces nou înseamnă un cost de timp (*overhead*) la nivelul shellului.

Cu toate acestea, comenzile externe sunt prevalente pentru modularitate: o nouă funcționalitate este adăugată prin adăugarea unui nou fișier executabil, fără a fi nevoie de modificarea shellului, cum ar fi cazul comenzilor interne.

Tabelul 7.2 sumarizează diferențele între comenzi interne și comenzi externe.

Tabelul 7.2: Combinații de taste în shell

Criteriu	Comenzi interne	Comenzi externe
parte din shell	da	nu
executabil dedicat	nu	da
overhead	redus	mai mare
modularitate	da	nu

Pentru a identifica rapid tipul unei comenzi, se poate folosi comanda **`which`** pentru a vedea dacă comanda are un fișier executabil asociat. Mai robust de folosit este, însă, comanda **`type`**, așa cum este folosită în Listing 7.9.

```
1 student@uso:~$ type echo
2 echo is a shell builtin
3 student@uso:~$ type exit
4 exit is a shell builtin
5 student@uso:~$ type ls
6 ls is aliased to 'ls --color=auto --group-directories-first'
7 student@uso:~$ type ps
```

```
8  ps is hashed (/bin/ps)
9  student@uso:~$ type df
10 df is /bin/df
11 student@uso:~$ type type
12 type is a shell builtin
```

Listing 7.9: Tipul unei comenzi (type)

Comanda **`type`** este, desigur, internă, pentru a putea investiga comenzile interne ale shellului.

Când un shell rulează, va căuta întâi intern comanda introdusă de utilizator, apoi va căuta dacă este o comandă externă cu un corespondent într-un fișier executabil localizabil prin intermediul variabilei `PATH`.

7.2 Funcționalități shell

La nivel de bază, shellul permite rularea de comenzi sau rularea de comenzi cu parametri/opțiuni. Shellul are funcționalități care permit înlănțuirea și combinarea mai multor comenzi și configurarea modului în care acestea rulează.

Aceste funcționalități sunt oferite de operatori shell, variabile shell, expandări în shell și construcții agregate (*globbing*). Le prezentăm în continuare.

7.2.1 Operatori shell

Operatorii shell frecvent folosiți sunt cei de redirectare și cei de înlănțuire. Operatorii de redirectare cei mai întâlniți sunt `>`, `<`, `>>`, `2>`, prezentați în Secțiunea 2.4 Pentru înlănțuirea comenzilor avem operatorii `;`, `||`, `&&`, `|`, prezentați în Secțiunea 4.5.3.

Atunci când dorim să rulăm un proces în background folosim operatorul `&`. Acesta va trimite procesul în background, așa cum este prezentat în Secțiunea 4.3.3.

7.2.2 Subshelluri

Atunci când folosim operatorii shell, putem opta să operăm asupra unei compoziții de comenzi. De exemplu, dorim să reținem outputul a două comenzi într-un fișier. În Listing 7.10, în liniile `1-2`, redirectarea nu funcționează.

Doar rezultatul rulării comenzii **`ps`** este redirectat. Pentru a redirecta o înlănțuire de comenzi folosim un subshell, adică vom crea un proces nou shell care va rula cele două comenzi și va transfera rezultatul amândurora în fișier, ca în liniile `4-5` din Listing 7.10. Aici ambele comenzi sunt redirectate în fișierul `a.out`.

```
1 student@uso:~$ ls ; ps > a.out
2 Desktop  Documents  Downloads  examples.desktop  Music  Pictures  Public
   snap  Templates  uso.git  Videos  vm-actions-log.txt
3
4 student@uso:~$ ( ls ; ps ) > a.out
```

Listing 7.10: Subshelluri și redirectare

7.2.3 Variabile shell

Execuția shellului poate fi investigată și configurată cu ajutorul variabilelor acestuia. Variabilele shell oferă informații despre configurația shellului sau afectează execuția acestuia.

O variabilă are un nume și o valoare. Pentru a afișa valoarea unei variabile prefixăm numele cu simbolul `$`, așa cum am văzut în Secțiunea 7.1.3.1.

Există o serie de parametri speciali ai shellului care sunt asemuiți unor variabile. Nu pot fi configurați / modificați, doar citiți. Sunt prezentați în Tabelul 7.3.

Tabelul 7.3: Parametri speciali ai shellului

Nume	Efect
`$`	PID-ul procesului shell curent
`!`	PID-ul celui mai recent proces din background
`?`	codul de ieșire al celei mai recente comenzi
`_`	ultimul argument al celei mai recente comenzi

Alți parametri speciali sunt folosiți în special în shell scripting și vor fi prezentați în Secțiunea 13.3.1.

Shellul Bash are o serie de variabile predefinite, care au roluri specifice în funcționarea shellului; o parte sunt indicate în Tabelul 7.4.

Tabelul 7.4: Variabile predefinite Bash

Nume variabilă	Rol / Efect
`PATH`	căile de căutare a fișierelor executabile pentru comenzi
`PWD`	directorul curent
`USER`	numele utilizatorului curent
`HOME`	directorul home al utilizatorului curent
`SHELL`	shellul curent
`TERM`	tipul de terminal folosit

Pentru a vedea toate variabilele definite la un moment dat într-un shell Bash putem folosi comanda internă **`declare`**, ca în Listing 7.11.

```
1 student@uso:~$ declare
2 BASH=/bin/bash
3 BASH_VERSION='4.4.19(1)-release'
4 COLUMNS=185
5 DIRSTACK=()
6 EUID=1000
7 GROUPS=()
8 HISTCONTROL=ignoreboth
```

```
HISTFILE=/home/student/.bash_history
HISTFILESIZE=2000
HISTSIZE=1000
HOME=/home/student
HOSTNAME=uso
HOSTTYPE=x86_64
[...]
```

Listing 7.11: Afișarea variabilelor definite în shell

Pentru a inițializa o variabilă oarecare în shell folosim o construcție de forma `nume=valoare` ca în exemplele din Listing 7.12. Atenție, între `nume`, `=` și `valoare` nu poate exista caracterul spațiu, deoarece acesta este separatorul shellului. O construcție de forma `nume = valoare` va fi interpretată de către shell ca „rulează comanda `nume` cu argumentele `= valoare`”.

```
student@uso:~$ config=/etc/nsswitch.conf
student@uso:~$ echo $config
/etc/nsswitch.conf
```

Listing 7.12: Inițializarea variabilelor în shell

Dacă vrem să anulăm definirea unei variabile folosim comanda **`unset`** ca în Listing 7.13.

```
student@uso:~$ echo $config
/etc/nsswitch.conf
student@uso:~$ unset config
student@uso:~$ echo $config

student@uso:~$
```

Listing 7.13: Anularea definirii unei variabile

7.2.3.1 Variabile de mediu

Anumite variabile pot fi variabile de mediu. O variabilă de mediu este o variabilă care este moștenită de procesele create din shell. Multe dintre variabilele shellului indicate în Tabelul 7.4 sunt variabile de mediu. Pentru a vedea toate variabilele de mediu definite într-un shell folosim comanda **`export -p`** sau comanda **`env`** ca în Listing 7.14.

```
student@uso:~$ env
LC_ALL=en_US.UTF-8
LC_MEASUREMENT=ro_RO.UTF-8
SSH_CONNECTION=192.168.56.1 46702 192.168.56.101 22
[...]

student@uso:~$ export -p
declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/1000/bus"
declare -x HOME="/home/student"
declare -x LANG="en_US.UTF-8"
[...]
```

Listing 7.14: Afișarea variabilelor de mediu

Pentru a defini o variabilă de mediu folosim comanda **`export`** ca în Listing 7.15.

```
student@uso:~$ config=/etc/nsswitch.conf
student@uso:~$ env | grep config
student@uso:~$ export config
student@uso:~$ env | grep config
config=/etc/nsswitch.conf

student@uso:~$ export target=vm-actions-log.txt
student@uso:~$ env | grep target
target=vm-actions-log.txt
```

Listing 7.15: Definirea unei variabile de mediu

Dacă dorim ca o variabilă să nu mai fie definită va variabilă de mediu (să fie neexportată), folosim opțiunea `-n` la comanda **`export`** ca în Listing 7.11.

```
student@uso:~$ env | grep target
target=vm-actions-log.txt
student@uso:~$ export -n target
student@uso:~$ env | grep target
student@uso:~$ echo $target
vm-actions-log.txt
```

Listing 7.16: Eliminarea condiției de variabilă de mediu

În folosirea variabilelor e de avut în vedere ce variabile există deja și care este rolul lor. Pentru aceasta putem afișa toate variabilele, inclusiv cele de mediu, folosind comanda **`declare`** ca în Listing 7.11.

În general, marcăm o variabilă ca variabilă mediu atunci când dorim ca această să fie moștenită și vizibilă în alt proces. Altfel, dacă dorim să afecteze doar procesul shell curent, poate fi definită ca variabilă simplă. Dacă dorim ca o variabilă să afecteze un proces, nu este neapărat nevoie să o marcăm ca variabilă de mediu. O variabilă poate fi folosită pentru a porni un program și afectează doar rularea acelui program. De exemplu în folosirea clientului Bittorrent în linia de comandă **`transmission-cli`**, putem folosi o comandă precum cea din Listing 7.17. Prefixând rularea comenzii cu definirea unei variabile, vom crea un proces care va avea acea variabilă definită pe parcursul rulării sale; la încheierea rulării procesului, variabila nu mai există.

```
student@uso:~$ TR_DEBUG=1 transmission-cli
```

Listing 7.17: Folosirea unei variabile pentru a afecta funcționarea unui program

7.2.4 Expandări

Un shell primește la intrare comenzi care pot cuprinde parametri, operatori și variabile. După ce face separația elementelor comenzii (*parsing*), shellul realizează operațiile de expandare. Expandarea este de mai multe tipuri, detaliate în documentația Bash [1]. În Listing 7.18 sunt prezentate exemple de comenzi cu cele mai importante forme de expandare:

- Liniile `1-2` prezintă expandarea cu tildă. Expandarea caracterului $\sim$ (tildă) este folosită pentru expandarea directorului home. Caracterul tildă este înlocuit cu valoarea variabilei `HOME` a shellului curent.

[1] https://www.gnu.org/software/bash/manual/html_node/Shell-Expansions.html#Shell-Expansions

- Liniile `4-12` prezintă expandarea cu acolade. Expandarea de acolade înseamnă trecerea prin mai multe opțiuni.
- Liniile `14-19` prezintă expandarea unei variabile, caz de expandare parametrică. În cazul simplu se folosește doar caracterul `$` (*dollar*). Altfel folosim construcții care încep cu `${` (dolar-acoladă), ca să fie clar care este numele variabile. De exemplu, linia `16` va încerca afișarea valorii variabilei `config_test`, nu a valorii variabilei `config` urmată de șirul `_test`. Pentru aceasta o rulare corectă este cea de la linia `18`.
- Liniile `21-24` prezintă expandarea aritmetică. Dacă nu am folosi construcția `$((...))` pentru expandare aritmetică, nu s-ar realiza calculul matematic, ci doar s-ar afișa expresia în cauză, ca un șir.

```
1  student@uso:~/uso.git$ ls ~/Downloads/
2  idafree70_linux.run  opensc-0.20.0  opensc-0.20.0.tar.gz
3
4  student@uso:~/uso.git$ ls lab{02,03,04}
5  lab02:
6  anul_1  anul_2  demo
7
8  lab03:
9  gcc-optimizations  large-project  project  simple-gcc  static-lib  tema-
   pc  ugly
10
11 lab04:
12 batman.sh  bg-proc.sh  it-s-a-trap.sh
13
14 student@uso:~/uso.git$ echo $PATH
15 /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/
   usr/local/games:/snap/bin
16 student@uso:~/uso.git$ echo $config_test
17
18 student@uso:~/uso.git$ echo ${config}_test
19 /etc/nsswitch.conf_test
20
21 student@uso:~/uso.git$ echo 3+4
22 3+4
23 student@uso:~/uso.git$ echo $((3+4))
24 7
```

Listing 7.18: Forme de expandare în shell

Expandarea parametrică are mai multe forme, le amintim pe cele relevante în Tabelul 7.5. Presupunem că variabila `config` este inițializată la valoarea `/etc/nsswitch.conf`.

Tabelul 7.5: Expandarea parametrică în shell

Construcție	Rol / Efect	Exemplu de comandă	Rezultat comandă
`${var}`	afișează conținutul variabilei `var`	`${config}`	`/etc/nsswitch.conf`
`${#var}`	afișează numărul de caractere ale conținutului variabilei `var`	`${#config}`	`18`
`${var/str1/str2}`	înlocuirea șirului `str1` cu șirul `str2` în conținutul variabilei `var`	`${config/ns/df}`	`/etc/dfswitch.conf`
`${var#str}`	șterge șirul `str` de la începutul conținutului variabilei `var`	`${config#/*/}`	`nsswitch.conf`
`${var%str}`	șterge șirul `str` de la sfârșitul conținutului variabilei `var`	`${config%/*}`	`/etc`

7.2.4.1 Globbing

Termenul de *globbing* se referă la construcții specifice shellului care pot fi expandate la a se potrivi cu mai multe opțiuni. Se mai numește expandarea căilor (*pathname expansion*). De exemplu construcția `z*` se potrivește cu orice nume care începe cu litera `z`; este util în shell în interacțiunea cu sistemul de fișiere, ca în Listing 7.19.

```
1
2 student@uso:~/uso.git$ ls /bin/z*
3 /bin/zcat  /bin/zcmp  /bin/zdiff  /bin/zegrep  /bin/zfgrep  /bin/zforce
  /bin/zgrep  /bin/zless  /bin/zmore  /bin/znew
4 student@uso:~/uso.git$ ls /bin/*w
5 /bin/znew
```

Listing 7.19: Folosirea globbing în shell

Globbing folosește de regulă caractere speciale, așa cum sunt indicate în construcțiile din Tabelul 7.6.

Tabelul 7.6: Caractere speciale în globbing

Caracter special din globbing	Rol / Efect
`?`	se potrivește cu orice caracter o singură dată
`*`	se potrivește cu orice caracter de oricâte ori
`[...]`	se potrivește cu orice caracter din setul de caractere dintre parantezele drepte

Construcțiile ce folosesc globbing, expandarea cu acolade și expandarea tildei sunt frecvent folosite în operațiile cu sistemul de fișiere.

7.2.4.2 Expandarea comenzilor

O situație întâlnită frecvent este folosirea outputului unei comenzi ca parametru al unei alte comenzi sau reținerea acestuia într-o variabilă. De exemplu, atunci când vrem să reținem într-o variabilă numărul de procese ale unui utilizator. În acest caz folosim expandarea comenzilor ca în Listing 7.20.

```
1 student@uso:~/uso.git$ pids=$(pgrep -u student)
2 student@uso:~/uso.git$ echo $pids
3 1183 1184 1218 1222 1224 1275 1290 1492 1493 1501 1502 1509 1510 1519
  1520 1524 1526 1531 1534 1601 1606 1622 1639 1653 1663 1687 1696 1807
  1811 1820 1830 1844 1848 1852 1857 1861 1866 1870 1879 1883 1887 1891
  1895 1898 1910 1922 1926 1928 1929 1930 1933 1943 1944 1990 2031 2033
  2034 2035 2040 2058 2104 2119 2129 2138 2145 2188 3792 3884 8924 8925
  13202 13203 14455 14457 14599 14610
4 student@uso:~/uso.git$ num_proc=$(pgrep -u student | wc -w)
5 student@uso:~/uso.git$ echo $num_proc
6 78
```

Listing 7.20: Expandarea comenzilor

7.2.5 Escaping

După cum am văzut, shellul folosește caractere cu roluri dedicate în rularea sa: separare de comenzi și parametri, operatori, expandare. Le numim *metacaractere*. Aceste metacaractere pot fi însă parte a unor nume de parametri primiți de comenzi. De exemplu, în Listing 7.21 avem fișierul `USO - Curs 07.pdf` dar comanda rezultă în eroare. Acest lucru se întâmplă deoarece shellul interpretează intrarea primită ca fiind o comandă și trei parametri. Caracterul spațiu (*blank*) este un metacaracter în shell.

```
student@uso:~/uso.git$ ls USO - Curs 07.pdf
ls: cannot access 'USO': No such file or directory
ls: cannot access '-': No such file or directory
ls: cannot access 'Curs': No such file or directory
ls: cannot access '07.pdf': No such file or directory
ls udo curs 05.pdf
```

Listing 7.21: Caracterul spațiu în numele unui fișier

Pentru a împiedica shellul să interpreteze caracterele, folosim *escaping*, adică metode prin care acele caractere să fie interpretate literal, nu ca metacaractere. Există trei moduri de escaping: prin ghilimele, prin apostrofuri și prin \ (*backslash*).

În cazul escapării prin apostrofuri, caracterele plasate între două apostrofuri vor avea interpretare literală. Escaparea prin ghilimele este similară escapării prin apostrofuri, doar că nu se escapează caracterul dolar. Altfel spus construcțiile care folosesc dolar (afișarea valorii unei variabile și expandările) își păstrează semnificația între ghilimele.

Escaparea prin backslash păstrează sensul literal al caracterului imediat următor.

Listing 7.22 conține exemple de folosire a celor trei forme de escaping.

```
student@uso:~/uso.git$ ls "USO - Curs 07.pdf"
'USO - Curs 07.pdf'
student@uso:~/uso.git$ ls 'USO - Curs 07.pdf'
'USO - Curs 07.pdf'
student@uso:~/uso.git$ ls USO\ -\ Curs\ 07.pdf
'USO - Curs 07.pdf'
```

Listing 7.22: Forme de escaping în shell

Fiecare formă poate fi la rândul său folosită pentru a escapa celelalte forme, așa cum apare în Listing 7.23. Este similar cu escapingul ghilimelelor într-un șir dintr-un limbaj de programare (precum C sau Python)

```
student@uso:~/test$ touch "ana'are'mere"
student@uso:~/test$ touch 'ana"are"mere"
> ^C
student@uso:~/test$ touch 'ana"are"mere'
student@uso:~/test$ touch 'ana\are\mere'
student@uso:~/test$ ls
"ana'are'mere"  'ana"are"mere'  'ana\are\mere'
```

Listing 7.23: Forme de escaping în shell

7.3 Pornirea și personalizarea shellului

Orice utilizator are configurat un shell de login în fișierul `/etc/passwd`. După ce un utilizator se autentifică în sistem, se pornește un proces din shellul de login. În mod uzual, acest shell este `/bin/bash` pe sistemele Linux; înseamnă că utilizatorul pornește shellul Bash.

Atunci când este pornit un shell, sunt încărcate configurări pentru acesta. Configurările sunt stocate în fișiere globale și locale și au în general rolul de a stabili variabilele shellului sau alți parametri ai acestuia. De exemplu, personalizarea promptului unui shell se realizează prin configurarea variabilei `PS1` în Bash, așa cum vom vedea în Listing 7.27 din Secțiunea 7.3.2.

Locul unde se găsesc fișierele de configurare globale și locale depind de tipul de shell. Vom prezenta în continuare shellul Bash.

7.3.1 Configurarea la pornire a shellului Bash

Există mai multe moduri de pornire a shellului Bash: login/non-login sau interactiv/neinteractiv. Aceste diferențe nu le vom detalia aici, sunt prezentate în documentația Bash online [1] sau folosind comanda `man bash` și căutând în manual după șirul `INVOCATION`.

În cazul cel mai uzual (shell de login și interactiv), shellul Bash parcurge întâi fișierul global de configurare `/etc/profile`, apoi primul fișier găsit dintre `~/.bash_profile`, `~/.bash_login`, `~/.profile`.

În general, Bash va avea configurate linii precum cele din Listing 7.24 în fișierul `/etc/profile`, ceea ce înseamnă că va fi interpretat conținutul fișierului `/etc/bash.bashrc` și conținutul fișierelor din directorul `/etc/profile.d/`.

```
        if [ -f /etc/bash.bashrc ]; then
            . /etc/bash.bashrc
        fi


if [ -d /etc/profile.d ]; then
    for i in /etc/profile.d/*.sh; do
        if [ -r \$i ]; then
            . \$i
        fi
    done
fi
```

Listing 7.24: Secvență de configurare Bash (/etc/profile)

De asemenea, fișierul `~/.bash_profile` va avea configurate linii precum cele din Listing 7.25, ceea ce înseamnă că se va interpreta și conținutul fișierului `~/.bashrc`.

```
    if [ -f "\$HOME/.bashrc" ]; then
        . "\$HOME/.bashrc"
    fi
```

[1] https://www.gnu.org/software/bash/manual/html_node/Invoking-Bash.html

Listing 7.25: Secvență de configurare Bash (/.bashrc)

De aceea, dacă faceți configurări pentru personalizarea și configurarea pornirii shellului, le veți face, de obicei:

- global, la nivelul sistemului, în fișierul `/etc/bash.bashrc`
- local, la nivelul utilizatorului, în fișierul `~/.bashrc`

De exemplu, putem adăuga secvența din Listing 7.26 fie în fișierul `/etc/bash.bashrc` fie în fișierul `~/.bashrc`. Acest lucru va conduce o configurare globală sau locală care va limita directoarele din prompt la `3` (variabila de mediu `PROMPT_DIRTRIM`), respectiv la afișarea conținutului sortat după caractere ASCII (adică literele mari primele) (variabila de mediu `LC_COLLATE`).

```
1 export PROMPT_DIRTRIM=3
2 export LC_COLLATE=POSIX
```

Listing 7.26: Configurare în Bash (bashrc)

La închiderea unei sesiuni de shell acesta interpretează fișierul `~/.bash_logout`.

7.3.2 Personalizarea promptului

O funcționalitate adesea folosită în shell este personalizarea promptului. Personalizarea acestuia poate înseamna personalizarea conținutului sau a aspectului (culorilor).

Promptul afișat de shellul Bash este definit de variabila `PS1`. De exemplu pe mașina virtuală USO promptul arată ca în liniile `1-2` din Listing 7.27 cu valoarea `PS1` cea indicată. Liniile `4-7` conțin exemple de prompt-uri personalizate prin modificarea variabilei `PS1`.

```
1 student@uso:~$
2 student@uso:~$ echo $PS1
3 ${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m
  \]:\[\033[01;34m\]\w\[\033[00m\]\$
4 student@uso:[23:00:06]:~$ echo $PS1
5 ${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:[\t
  ]:\[\033[01;34m\]\w\[\033[00m\]\$
6 student@uso:[11:02 PM]:~$ echo $PS1
7 ${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:[\@
  ]:\[\033[01;34m\]\w\[\033[00m\]\$
```

Listing 7.27: Personalizarea prompt-ului

Conținutul promptului din variabila `PS1` reprezintă elemente care sunt apoi interpretate de shell, precum cele din Tabelul 7.7.

O listă completă a elementelor ce pot fi parte din variabila PS1 găsiți în documentație [1] [2] sau în pagina de manual Bash în secțiunea `PROMPTING`.

Dacă doriți să personalizați permanent promptul, va trebui să modificați valoarea variabilei `PS1` într-un fișier de configurare a shellului așa cum am precizat în

[1] https://www.gnu.org/software/bash/manual/html_node/Controlling-the-Prompt.html
[2] https://www.howtogeek.com/307701/how-to-customize-and-colorize-your-bash-prompt/

Tabelul 7.7: Elemente pentru personalizarea prompt-ului

Construcție	Efect
`\u`	numele utilizatorului
`\h`	numele stației (*hostname*)
`\w`	directorul curent
`\$`	caracterul dollar
`\t`	data curentă, în format 24H
`\@`	data curentă, în format 12H, urmată de AM / PM
`\[\033[COLORm\]`	schimbarea culorii textului din prompt

Secțiunea 7.3.1. Puteți încerca site-ul Easy Bash Prompt Generator [1] pentru o formă interactivă și vizuală de generare de conținut `PS1` pentru personalizarea promptului.

7.3.3 Multiplexoare de terminal

Atunci când dorim shelluri multiple, avem posibilitatea să deschidem sau să folosim mai multe ferestre de shell sau mai multe taburi de shell sau să folosim multiplexoare de terminal precum **`screen`**, **`tmux`** pe care le-am prezentat în Secțiunea 4.6.5.

Multiplexoarele de terminal sunt utile în situația în care vrem să rulăm aplicații interactive și apoi să ne detașăm de la terminal.

`tmux`, **`byobu`** și **`screen`** au fișiere de configurare specifice în care putem configura aspectul terminalului, combinații de taste și au facilități dedicate, precum salvarea sesiunii curente și restaurarea acesteia la alt moment de timp, detaliate în Tabelul 4.2.

7.4 Expresii regulate

În Linux majoritatea fișierelor conțin informații text; la fel, majoritatea comenzilor afișează informații text. Dorim să prelucrăm aceste informații text: extragere de linii, extragere de coloane, verificarea apariției unor secvențe.

Pentru verificarea apariției unei secvențe (acțiune numită *pattern matching*) folosim expresii regulate. O expresie regulată este un șir de caractere; la fel ca în cazul globbing, unele caractere au rol special și le numim metacaractere. O expresie regulată este folosită pentru a îngloba reprezentarea mai multor șiruri. De exemplu expresia regulată `[a-z]+,[0-9]+` se potrivește cu un nume format din litere mici urmat de virgulă și un număr.

Expresiile regulate sunt prezente în majoritatea limbajelor, într-o formă sau alta: Python, Perl, PHP, Ruby, JavaScript, în biblioteci C. În shell, utilitare specializate folosesc expresii regulate, cel mai cunoscut fiind **`grep`**; **`grep`** extrage linii ce conțin o anumită expresie regulată, ca în Listing 7.28.

```
1 student@uso:~$ grep printf /usr/include/stdio.h
```

[1] http://ezprompt.net/

```
2  extern int fprintf (FILE *__restrict __stream,
3  extern int printf (const char *__restrict __format, ...);
4  extern int sprintf (char *__restrict __s,
5  extern int vfprintf (FILE *__restrict __s, const char *__restrict
   __format,
6  extern int vprintf (const char *__restrict __format, _G_va_list __arg);
7  [...]
8
9  student@uso:~$ ps -ef | grep student
10 student   1183     1  0 Oct01 ?        00:00:00 /lib/systemd/systemd --
   user
11 student   1184  1183  0 Oct01 ?        00:00:00 (sd-pam)
12 student   1218     1  0 Oct01 ?        00:00:00 /usr/bin/gnome-keyring-
   daemon --daemonize --login
13 student   1222  1179  0 Oct01 tty1     00:00:00 /usr/lib/gdm3/gdm-x-
   session --run-script env GNOME_SHELL_SESSION_MODE=ubuntu gnome-session --
   session=ubuntu
14 student   1224  1222  0 Oct01 tty1     00:00:00 /usr/lib/xorg/Xorg vt1 -
   displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset
    -keeptty -verbose 3
15 [...]
```

Listing 7.28: Expresii regulate în grep

Expresiile regulate pot conține unul sau mai multe dintre metacaracterele prezentate în Tabelul 7.8.

Tabelul 7.8: Metacaractere în expresii regulate

Construcție	**Rol / Efect**
`.`	orice caracter
`a?`	caracterul `a` o dată sau niciodată
`a*`	caracterul `a` de oricâte ori, posibil niciodată
`a+`	caracterul `a` de oricâte ori, cel puțin o dată
`^`	început de linie
`$`	sfârșit de linie
`[...]`	orice caracter din setul de caractere dintre parantezele drepte
`\`	escaping

Dacă dorim să folosim într-o expresie regulată un metacaracter în sensul său literal, adică escaping, vom folosi backslash.

Tabelul 7.9 conține exemple de expresii regulate.

Tabelul 7.9: Exemple de construcții cu expresii regulate

Construcție	**Efect**
`[A-Z][a-z]+`	nume propriu
`[A-Z][a-z]+ [A-Z][a-z]+`	prenume și nume
`[a-zA-Z_][a-zA-Z_0-9]*`	nume de variabilă
`[0-9]{10}`	număr de telefon
`^#include +<[^>]+>$`	linie ce conține o directivă `include` în C

În cazul **grep**, în mod special, anumite caractere trebuie prefixate de backslash pentru a avea rolul lor de metacaracter: +, |, <, >, (,).

Un caz util de folosire a expresiilor regulate este atunci când vrem să extragem informații care se repetă. De exemplu să extragem dintr-o listă pe cei al căror nume și prenume începe cu aceeași literă, ca în exemplul din Listing 7.29. În acest caz, construcția `\([A-Z]\)` definește o variabilă care reține prima literă (majusculă) a unei linii. Variabila este apoi refolosită sub forma `\1` pentru a referi o nouă potrivire.

```
1 grep '^\([A-Z]\)[A-Z]\+ \1' students.txt
```

Listing 7.29: Expresii regulate cu variabile

Utilitarele avansate de prelucrare text **sed** și **awk** folosesc expresii regulate și le vom prezenta în Secțiunea 7.6.

7.5 Prelucrare de text de bază: filtre de text și one linere

Așa cum am precizat mai sus, textul este forma principală de stocare și afișare a datelor în Linux. Ceea ce face ca mare parte din utilitarele din Linux să afișeze și să primească la intrare text.

O bună parte din comenzile Linux lucrează cu text. Le grupăm în generatoare de text sau în prelucratoare de text. Utilitarele prelucratoare de text le mai numim **filtre de text**. Un filtru de text primește la intrare text, îl prelucrează și afișează, de obicei, tot text, ca în Figura 7.2.

Figura 7.2: Filtre de text

7.5.1 Generatoare de text

Exemple de generatoare text sunt **ps**, **pstree**, **ls**, **find**, **stat**, **df**, **lsblk**, **tree**, **strace**, **diff** și în general orice comenzi care afișează informații. Aceste comenzi afișează informații la ieșirea standard. Aceste informații sunt utile fie utilizatorului, fie altor comenzi care le pot prelucra, sau amândurora. Spunem, astfel, că generatoarele de comenzi au două scenarii de utilizare:

1. rezultatul rulării este util în special utilizatorului
2. rezultatul rulării lor este util în special unei prelucrări, unei comenzi prelucratoare de text

În scenariile de utilizare, dacă dorim ca rezultatul unei comenzi să fie prelucrat, vom prefera a doua categorie de utilitare. Atunci când construim one-linere sau când folosim scripturi, în general este recomandat să fie folosită a doua categorie de generatoare.

7.5.1.1 Prelucratoare de text

Prelucrarea text presupune acțiuni de forma:

- selecție: linii sau coloane
- înlocuire: înlocuirea unor părți cu alte părți
- extragere: doar anumite părți din text
- ordonare: ordonarea rezultatelor după un criteriu dat, de exemplu ordonare alfabetică
- sumarizare: obținerea unor informații numerice sau statistice
- reformatare: plasarea unor caractere în plus, reordonarea unor elemente pe o linie

Tabelul 7.10 prezintă comenzi de prelucrare și rolul lor din acțiunile de mai sus.

Tabelul 7.10: Prelucratoare de text

Comandă	Rol / efect	Tip de acțiune
`grep`	extrage liniile care se potrivesc cu o expresie regulată	extragere
`cut`	extrage anumite coloane	selecție
`nl`	afișează numărul liniei pentru fiecare linie	reformatare
`wc`	afișează numărul de linii	sumarizare
`sort`	sortează liniile	ordonare
`uniq`	elimininină liniile duplicate	extragere, reformatare
`fmt`	reformatează conținutul	reformatare
`paste`	alătură conținutul mai multor fișiere	reformatare
`join`	alătură conținutul mai multor fișiere pe baza unor informații comune	reformatare
`rev`	inversează conținutul unei linii	reformatare
`tail`	afișează doar liniile finale	selecție
`head`	afișează doar liniile inițiale	selecție
`tac`	inversează liniile unui fișier	reformatare
`tr`	translatează caractere	înlocuire
`tee`	clonează conținutul într-un alt fișier și la ieșirea standard	duplicare
`sed`	utilitare de prelucrări complexe	toate tipurile de acțiuni
`awk`	utilitare de prelucrări complexe	toate tipurile de acțiuni

7.5.1.2 One-linere

În momentul în care avem nevoie să prelucrăm text, vom combina un generator de text (sau un fișier text pur și simplu) cu una sau mai multe prelucratoare de text. Combinarea o vom face prin intermediul operatorului | (*pipe*) și vom forma one-linere (*one-liners*). O altă formă de combinare a comenzilor, și de obține de one-linere, este prin intermediul expandării comenzilor.

Exemple concrete de one linere sunt în Listing 7.30.

```
# generate password
student@uso:~$ < /dev/urandom tr -d -c 'A-Za-z_0-9' | head -c 12 ; echo
FOyfJRrEibRW

# Generate files.
student@uso:~/test$ touch $(seq -f "file-%02g.txt" 1 30)
student@uso:~/test$ ls
file-01.txt  file-04.txt  file-07.txt  file-10.txt  file-13.txt  file-16.
txt  file-19.txt  file-22.txt  file-25.txt  file-28.txt
file-02.txt  file-05.txt  file-08.txt  file-11.txt  file-14.txt  file-17.
txt  file-20.txt  file-23.txt  file-26.txt  file-29.txt
file-03.txt  file-06.txt  file-09.txt  file-12.txt  file-15.txt  file-18.
txt  file-21.txt  file-24.txt  file-27.txt  file-30.txt

# Sort by size, print largest 5 files.
student@uso:~/test$ stat -c "%n,%s" /etc/* | sort -n -k 2 -t ',' | tail
-5
/etc/mime.types,24301
/etc/brltty.conf,25341
/etc/matplotlibrc,31974
/etc/mailcap,48488
/etc/ld.so.cache,96247
```

Listing 7.30: Exemple de one-linere

Exemple de one-linere găsiți pe Internet folosind șirul de căutare „bash one-liners” [1] [2].

7.6 Utilitare avansate

În lucrul în shell, în one-linere și scripturi există cazuri de utilizare în care este foarte greu să ne descurcăm fără trei utilitare avansate: **find**, **sed** și **awk**, utilitare cu rol distinct. Le numim avansate datorită funcționalităților și opțiunilor complexe și diverse pe care le au și pentru că folosirea lor este uneori anevoioasă pentru cazuri de utilizare complicate.

7.6.1 find

Utilitarul **find** este folosit pentru listarea intrărilor într-o ierarhie din sistemul de fișiere care corespund unor anumite criterii. Listing 7.31 conține exemple de folosire a comenzii **find**.

```
student@uso:~$ find uso.git/
student@uso:~$ find uso.git/ -type f
student@uso:~$ find uso.git/ -type f -name 'a*'
student@uso:~$ find uso.git/ -type f -perm -111
student@uso:~$ find uso.git/ -type f -mtime -25

student@uso:~$ find uso.git/ -type f -perm -111 -delete
student@uso:~$ find uso.git/ -type f -perm -111 -exec ls -l {} \;
student@uso:~$ find uso.git/ -type f -exec stat -c "%s" {} \;
```

[1] http://www.bashoneliners.com/
[2] https://onceupon.github.io/Bash-Oneliner/

```
10 student@uso:~$ find uso.git/ -type f -exec stat -c "%s,%n" {} \;
11 student@uso:~$ find uso.git/ -type f -exec stat -c "%s,%n" {} \; | sort -
   n
12
13 student@uso:~$ find uso.git/ -type f -name 'a*' | xargs ls -l
```

Listing 7.31: Folosirea find

Pe lângă criterii de selectare (numite *TESTS*), indicate în liniile 1–5, **find** permite acțiuni care să fie executate pe intrările descoperite. Acțiuni uzuale sunt `-delete` și `-exec`, ca în liniile 7–11. Opțiunea cu `-exec` este echivalentă folosirii utilitarului **xargs**, ca în linia 13.

Construcțiile care folosesc **find ... -exec** și **xargs** sunt echivalente, diferențele între ele fiind minore.

7.6.2 sed

Utilitarul **sed** este folosit în principal pentru a face înlocuiri într-un fișier sau prelucrare text. Îl putem considera ca fiind o formă avansată a utilitarului **tr**.

Listing 7.32 conține exemple de folosire a comenzii **sed**.

```
1  student@uso:~$ cat test.txt
2  ana
3  are
4  mere
5  bune
6  si
7  alune
8  si
9  gutui
10 amarui
11 cu
12 puf
13 galben
14 ca
15 de
16 pui
17 student@uso:~$ sed 's/al/AL/g' < test.txt
18 ana
19 are
20 mere
21 bune
22 si
23 ALune
24 si
25 gutui
26 amarui
27 cu
28 puf
29 gALben
30 ca
31 de
32 pui
33 student@uso:~$ sed -n '/ana/,/si/p' < test.txt
34 ana
```

```
are
mere
bune
si
student@uso:~$ sed '/si/d' < test.txt
ana
are
mere
bune
alune
gutui
amarui
cu
puf
galben
ca
de
pui
```

Listing 7.32: Folosirea sed

În forma sa cea mai simplă, din linia `1`, utilitarul înlocuiește o secvență cu altă secvență. Caracterul `s` înseamnă *substitute*. Utilitarul `sed` are în spate un limbaj de programare și are și alte comenzi în afară de substituire, precum comanda `p` (*print*), pentru afișare, sau comanda `d` (*delete*) pentru ștergere.

Interesante sunt de parcurs sed oneliners [1]. Și mai interesant e de parcurs sed Sokoban [2].

7.6.3 awk

awk este în esență un limbaj de programare care poate fi folosit ca filtru de text. Putem considera awk ca fiind un limbaj intermediar între utilitarele de tip filtru de text și un limbaj de programare precum Perl sau Python. Dacă dorim să facem prelucrări text fără a intra într-un limbaj de programare complet precum Perl sau Python vom folosi awk.

În forma sa simplă, un one-liner awk are forma din Listing 7.33, cu utilitatea unui separator care poate fi o expresie regulată.

```
$ cat students.txt
VLĂDUȚU I. Liviu-Alexandru      311CC   6       3.5     5.22
GEORGIU V. Alexandra-Maria      311CC   10      10      9.67
PĂUNOIU N. Gabriel      311CC   7       6.5     3.5
BĂCÎRCEA A. Andrei      311CC   7       5.5     4.44
BOU V. Paul     311CC   7       5.75    3.6
[...]

$ awk -F '\t' '{print $1, $2;}' < students.txt
VLĂDUȚU I. Liviu-Alexandru 311CC
GEORGIU V. Alexandra-Maria 311CC
PĂUNOIU N. Gabriel 311CC
BĂCÎRCEA A. Andrei 311CC
BOU V. Paul 311CC
```

[1] http://sed.sourceforge.net/sed1line.txt
[2] https://github.com/aureliojargas/sokoban.sed/blob/master/sokoban.sed

```
[...]


$ awk -F '\t' '$2 ~ /313CC/ {print $1, $2;}' < students.txt | head -5
ZINCULESCU C. Marius-Valentin 313CC
MARCU L. Cosmin-Alexandru 313CC
DAVID A.C. Despina 313CC
PREDOI-CRISTEA I. Adrian 313CC
NICOLESCU I. Cristian 313CC
```

Listing 7.33: Folosirea awk

În forma sa programatică, awk poate executa secvețe de cod pe anumite linii ca în Listing 7.34.

```
BEGIN {
        for (i = 0; i < len; i++)
                for (j = 0; j < len; j++)
                        arr[i,j] = 0;
}
{
        if ($1 == "a")
                arr[0,0]++;
        else if ($1 == "b")
                arr[0,1]++;
        [...]
```

Listing 7.34: Forma programatică a awk

Detalii complete despre awk se găsesc parcurgând documentația [1].

7.7 Sumar

Interfața în linia de comandă este necesară utilizatorului tehnic pentru eficiență și pentru uniformitate.

Shellul este principala formă de interacțiune cu sistemul de operare. Bash este cel mai răspândit shell în Linux.

Shellul oferă funcționalități precum reverse history search, command completion și expandări pentru utilizator.

Shellul poate fi configurat pentru a eficientiza și personaliza experiența utilizatorului.

Comenzile shell și fișierele din Linux sunt centrate pe folosirea textului. Multe comenzi sunt filtre de text, comenzi care prelucrează text, la baza prelucrării textului stând expresiile regulate.

[1] https://www.gnu.org/software/gawk/manual/

Capitolul 8

Componente hardware

Hardware-ul reprezintă partea fizică a unui sistem informatic constituită din diverse componente mecanice, electronice și optice. Cu ajutorul hardware-ului se pot primi, prelucra și stoca informații. Mai generic, prin hardware înțelegem orice componentă într-un sistem complex de calcul ce poate fi atinsă, există fizic într-un loc anume (pornind de la monitor, tastatură, până la componentele de comunicație, imprimante etc.). Prin comparație, software-ul este partea logică ce controlează diverse module din hardware pentru a îndeplini sarcinile utilizatorului.

Într-o lume predominantă a software-ului, cuantificată prin numărul semnificativ mai mare de companii care fac dezvoltare, prin numărul de angajați, numărul de start-up-uri ce apar în fiecare an în comparație cu cele de hardware, cunoașterea elementelor de hardware este esențială întrucât programele dezvoltate trebuie să lucreze cu acestea. Din perspectiva utilizatorului, cunoașterea componentelor hardware este esențială din mai multe perspective: achiziție, instalare sistem de operare compatibil, instalare drivere compatibile, optimizare rulare aplicații, depanare când apar probleme cu sistemul.

La achiziția unui sistem de calcul (hardware-ul aferent), utilizatorul trebuie să reușească să identifice caracteristicile cele mai bune pentru bugetul dat. În general, nu e suficient să cunoască cele mai bune opțiuni deoarece acestea sunt, adesea, cele mai scumpe și deseori nu se încadrează în bugetul utilizatorilor. Cunoașterea componentelor hardware, tipul și prețul lor, compatibilitatea între acestea, ajută la achiziția unui sistem de calcul cu cel mai bun raport preț/calitate.

După achiziție urmează, în general, instalarea unui sistem de operare (dacă sistemul de calcul nu a venit cu unul preinstalat - dar chiar și atunci se dorește personalizarea lui). Pentru alegerea unui sistem de operare corespunzător, trebuie cunoscute caracteristicile componentelor hardware (arhitectură, memorie, dispozitive auxiliare), iar pentru buna funcționare a tuturor extensiilor hardware (plăci grafice, camere video) este necesară instalarea unui software specializat numit driver. Alegerea driverelor este foarte importantă și este strict legată de tipul dispozitivului fizic, precum și modelul acestuia. Alegerea unui driver necorespunzător nu va permite folosirea dispozitivului fizic la parametri normali sau chiar deloc, iar în cel mai rău caz poate cauza probleme cu stabilitatea sistemului (blue-screen-uri pe Windows, reset-uri spontane etc).

În următoarele secțiuni vom realiza o clasificare a sistemelor de calcul (hardware) în funcție de diverse criterii (arhitecturi, dimensiuni fizice, forme de împachetare a

componentelor) și vom descrie interacțiunea dintre dispozitivele hardware și sistemele de operare. Spre finalul capitolului vom prezenta rolul extensiilor auxiliare (plăci) și facilități moderne incluse în hardware-ul din ziua de astăzi. Capitolul se încheie cu două studii de caz legate de vizualizare componentelor hardware și a versiunilor de driver pe două sisteme de operare de uz general: Linux și Windows.

8.1 Clasificarea sistemelor de calcul (hardware-ului)

Rolul acestui capitol este dat de prezentarea componentelor hardware cu scopul de a putea alege un sistem de calcul, de a diagnostica o problema cu un sistem de calcul și de a instala software-ul potrivit. Pentru a îndeplini sarcinile dorite de utilizator, sistemul de calcul are nevoie de 3 componente principale: procesare (echivalentul creierului la om), stocare (înmagazinarea datelor procesate pentru a fi accesate ulterior - echivalentul unei cărți) și de un mecanism prin care să introducem și să preluăm date de la componenta de procesare (în termeni tehnici acesta se numește subsistemul intrare/ieșire sau I/O - de la input/output).

Un prim pas pentru îndeplinirea obiectivelor enunțate mai sus este dat de definirea unor clasificări (tipuri) ale hardware-ului din mai multe perspective. Vom porni de la o descriere logică a hardware-ului numită și `Arhitectura von Neumann`. Numele arhitecturii provine de la matematicianul și fizicianul John von Neumann, care a scris primul draft al raportului EDVAC în 1945 ce descria arhitectura unui sistem de calcul.

8.1.1 Arhitectura von Neumann

Arhitectura von Neumann reprezintă o descriere logică a unui sistem de calcul ce enumeră următoarele tipuri de componente:

- unitate de procesare / procesor (Central Processing Unit - CPU)
- unitate de memorie (Memory Unit)
- dispozitive de intrare/ieșire (Input-Output - I/O)

In Figura 8.1 este reprezentată grafic această arhitectură. Unitatea de procesare este formată din unitatea de control și unitatea aritmetică/logică. Unitatea de control are rolul de a configura și de a transmite comenzi perifericelor, respectiv unității aritmetice/logice, pentru a întreprinde acțiunile dorite de utilizator. Unitatea aritmetică/logică se ocupă în general cu realizarea calculelor, fiind optimizată pentru acest lucru. Aceasta are un rol foarte important deoarece majoritatea comenzilor utilizatorului se vor traduce în final către hardware în operații de aritmetică (ex. adunare) sau de logică (și/sau/xor). Unitatea de memorie are rolul de a servi rapid procesorul cu instrucțiunile și datele necesare realizării calculelor. Datele în general vin de la utilizator prin dispozitivele de intrare și sunt stocate temporar de unitatea de memorie pentru a putea fi folosite de unitatea de procesare. Acest flux a fost creat din cauza faptului că dispozitivele periferice (intrare/ieșire) sunt, în general, mult mai lente decât unitatea de procesare și ar încetini-o. Unitatea de memorie este, în general, volatilă (la închiderea sistemului, datele din unitatea de memorie se pierd) fiind folosită

ca un tampon pentru prelucrarea datelor. Odată prelucrate, acestea sunt trimise către dispozitivele de ieșire și folosite în diverse scopuri (ex. afișare, stocare permanentă).

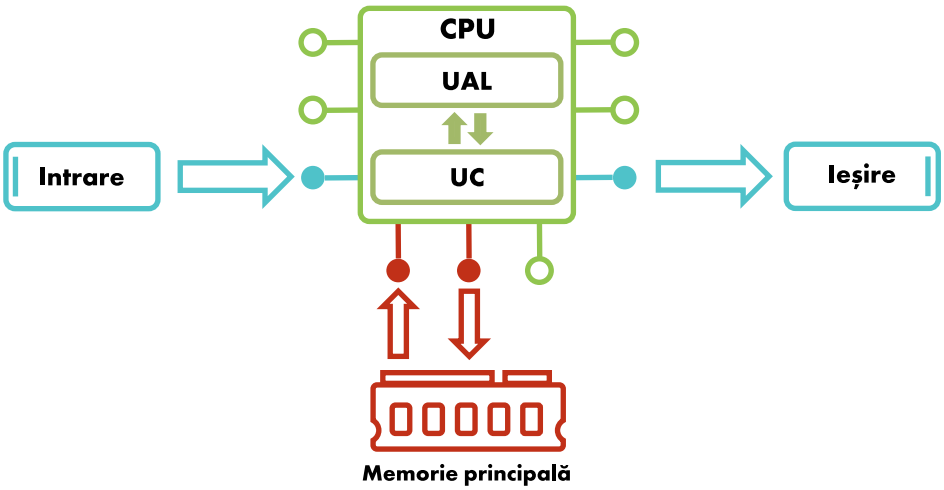


Figura 8.1: Arhitectura von Neumann

În zilele noastre, numărul unităților de procesare a crescut, precum și mărimea unității de memorie. Unitatea de memorie este partajată între toate procesoarele folosind o singură legătură între acestea denumită magistrală. Acest lucru s-a dovedit a fi un punct de congestie pentru procesoare întrucât acestea așteaptă fiecare pentru magistrală. Pentru optimizare, în implementările curente s-au introdus memorii specifice fiecărui procesor, numite unități de **memorie cache**. Un alt motiv pentru introducerea memoriei cache este faptul că memoria principală este cu câteva ordine de mărime mai înceată decât unitatea de procesare. O altă optimizare, în afară de memoria cache, a fost reprezentată de arhitectura Harvard unde s-a separat memoria de instrucțiuni (control) de memoria de date (intrare). Cu toate aceste optimizări, fiecare sistem calcul respectă arhitectura logică von Neumann, indiferent de tehnologia folosită și modul de integrare a componentelor.

În limbajul comun (de zi cu zi) folosim noțiunea de *procesor* pentru unitatea de calcul, *memorie* pentru unitatea de memorie, *I/O* pentru dispozitivele de intrare sau ieșire. *Disc/hard* reprezintă un dispozitiv I/O pentru stocarea permanentă a datelor, iar *placa de rețea* este și ea un dispozitiv I/O pentru comunicarea între diferite unități de calcul.

8.1.2 Arhitecturi de procesor

În zilele noastre dispozitivele hardware folosesc procesoare (unități de calcul) de la diverși producători. Aceste procesoare respectă în general două arhitecturi cunoscute: x86 și ARM. x86 este folosit în general în sistemele desktop, laptop-uri, servere, iar arhitectura ARM este folosită în general în dispozitivele mobile (smartphone-uri, tablete).

Pornind de la modelul von Neumann prezentat anterior, putem realiza o clasificare a unității de procesare în funcție de tipul instrucțiunilor pe care îl execută (se mai numește și set de instrucțiuni sau arhitectura sistemului, prescurtat deseori ISA , *Instruction Set Architecture*).

Astfel, în zilele noastre, sistemele de calcul respectă una din următoarele 2 arhitecturi:

- Reduced Instruction Set Computing (referită de acum încolo prin acronimul RISC)

- Complex Instruction Set Computing (referită de acum încolo prin acronimul CISC)

Înainte de a defini arhitecturile RISC/CISC vom mai introduce câteva elemente componente ale unității de procesare. Execuția instrucțiunilor se realizează asupra unor operanzi. Acești operanzi ar putea fi locații de memorie, dar acest lucru ar încetini unitatea de calcul la viteza de acces a datelor din memorie. Pentru a preveni accesul lent la date, au fost introduse registrele. Registrele sunt la baza memorii foarte rapide, de dimensiune fixă, dată de arhitectura constructivă a unității de procesare (32 biți / 64 biți - cele mai frecvente în ziua de astăzi sunt arhitecturile pe 64 de biți). Instrucțiunile au în general zero sau mai mulți operanzi referiți prin registre. Inclusiv locațiile de memorie asupra cărora se fac modificări sunt referite tot prin registre (se încarcă adresa memoriei în registru, iar acesta este folosit în execuția instrucțiunii).

Arhitectura de tip RISC este cunoscută și sub numele de arhitectură *load/store*: orice operație (de adunare, scădere) se realizează prin încărcarea conținutului memoriei în registre (*load*), iar după finalizarea operației conținutul registrelor este salvat în memorie (*store*). Instrucțiunile nu pot referi în mod direct memoria.

Arhitectura de tip CISC poate executa în aceeași instrucțiune primită de la utilizator mai multe operații numite micro-instrucțiuni: dacă dorim să adunăm două numere din memorie, putem executa o singură instrucțiune care va referi cele două locuri din memorie; în spate instrucțiunea este tradusă în mai multe micro-instrucțiuni (încărcare operanzi în registre, adunare, salvare rezultate); pentru acest exemplu avem o singură instrucțiune dată de utilizator pentru arhitectura CISC, iar pentru arhitectura RISC vom avea trei instrucțiuni specifice diferite.

Pentru ambele tipuri de arhitecturi există avantaje și dezavantaje. La RISC arhitectura internă este mult mai simplă, dar trebuie generate mai multe instrucțiuni pentru o operație, iar la CISC arhitectura internă este mai complexă, predispusă bug-urilor, dar într-o singură instrucțiune se pot îndeplini mai multe sarcini. O să observăm în continuare că avem exemple în producție din ambele tipuri, fiecare specific unui anumit domeniu.

În afară de clasificarea legată de setul de instrucțiuni (ISA - RISC vs CISC), se mai poate realiza o clasificare legată de dimensiunea adresării memoriei: câți biți sunt folosiți pentru a adresa memoria. În legătură directă cu dimensiunea adresării se află și dimensiunea registrelor întrucât acestea sunt folosite pentru a realiza adresarea. În zilele noastre avem în general două tipuri de adresări:

1. pe 32 de biți din ce în ce mai puțin folosită, dar păstrată din motive de compatibilitate

2. pe 64 de biți folosită preponderent în ziua de astăzi inclusiv în piața de mobile

Trecerea de la 32 de biți la 64 de biți s-a realizat atunci când dimensiunea memoriei a crescut mai mult de patru gigaocteți (*gigabytes*). Folosind 32 biți putem itera prin adrese de la 0 la 4 gigaocteți (2^{32}). Pentru a rezolva acest deficit s-a trecut la o arhitectură pe 64 de biți ce poate adresa semnificativ mai mult (2^{64} teoretic, 2^{48} în practică din cauza limitărilor hardware).

8.1.3 Arhitectura x86 și arhitectura ARM

Așa cum am precizat mai sus, în zilele noastre există două mari arhitecturi:

- x86 - disponibil atât pe 32 de biți (referit ca x86), cât și pe 64 de biți (referit ca x86_64 sau x64); este o arhitectură de tip CISC. Figura 8.2 prezintă un calculator cu procesor cu arhitectură x86.
- ARM - disponibil atât pe 32 de biți (referit ca ARMv7), cât și pe 64 de biți (referit ca ARMv8); este o arhitectură de tip RISC. Figura 8.3 prezintă un smartphone cu procesor cu arhitectura ARM. După cum se poate observa toate componentele sunt integrate pe aceeași plăcută, spre deosebire de șasiul din Figura 8.2 unde sunt separate componentele și dispuse pe placa de bază.

Figura 8.2: Calculator cu arhitectura x86[1]

Figura 8.3: Smartphone cu arhitectura ARM[2]

Pentru arhitectura x86 există 2 producători care împart procentele din piață: Intel și AMD (aceștia proiectează și implementează hardware-ul). Arhitectura ARM are un alt model de dezvoltare: compania ARM vinde proiectele procesorului către producătorii interesați,

[1] https://commons.wikimedia.org/wiki/File:Modified-pc-case.png **CC BY SA**
[2] https://en.wikipedia.org/wiki/Nexus_5#/media/File:Nexus_5_Front_View.png **CC BY 2.5**

iar aceștia le integrează cu alte componente periferice și le implementează (de exemplu Apple și Samsung).

Arhitectura x86 este folosită cu precădere în sisteme de tip desktop, laptop și server. Pentru fiecare tip de sistem, există versiuni diferite de procesor:

- pentru desktop avem procesoare pretabile aplicațiilor de zi cu zi
- pentru laptop avem procesoare ce respectă caracteristicile unui sistem desktop, dar dispun de un consum redus de energie
- pentru servere avem procesoare cu un număr mare de unități de procesare (sau core-uri în limbaj tehnic), cache-uri mai mari, răcire eficientă

Arhitectura ARM este folosită cu precădere în dispozitivele embedded/mobile. Majoritatea telefoanelor smart din ziua de astăzi au un procesor ARM (cei mai mari vendori fiind Apple și Samsung). În cadrul telefoanelor smart există 2 tipuri de procesoare: pe 32 de biți (majoritatea) și pe 64 de biți. Procesoarele ARMv7 au o adresare pe 32 de biți, iar procesoarele ARMv8 au o adresare pe 64 de biți. Avantajul ARM în fața x86 este consumul redus de energie și numărul ridicat de core-uri pe care îl poate oferi. La momentul scrierii, ARM are intenția de a intra pe piața serverelor concurând cu Intel și AMD. Există diverse prototipuri folosind procesoare ARMv8 ce ating performanțele procesoarelor Intel, la un cost redus. De asemenea, Intel a încercat folosirea procesoarelor x86 în telefoanele mobile smart, dar nu au reușit să creeze un produs stabil și să câștige o cotă de piață relevantă, fiindcă sunt mereu în urma procesoarelor ARM cu 1-2 ani. În acest moment, Intel a renunțat să mai investească în acest segment.

O altă diferență între cele 2 arhitecturi (x86 și ARM) o reprezintă modelul de integrare a componentelor:

- x86 are o arhitectură modulară în care utilizatorul poate înlocui separat procesorul, memoria, plăcile de procesare (grafică, sunet, rețea).
- ARM are o arhitectură integrată denumită și SoC (*System on Chip*): integratorii pun toate componentele (procesorul, memoria, plăcile de procesare) pe aceeași plăcuță, iar acestea nu pot fi schimbate separat. Vom detalia aceste sisteme în Capitolul 15.

8.1.4 Forme constructive ale hardware-ului

Sistemele de calcul vin în diverse forme constructive (*form factors* - dimensiune, forma de împachetare a componentelor) în funcție rolul și utilizarea acestora. Într-o clasificare generică avem patru forme constructive:

- server - specific centrelor de date
- desktop - specific utilizatorilor obișnuiți
- laptop - specific utilizatorilor finali, ocupând mai puțin spațiu și având un cost mai mare ca sistemele Desktop în general
- încorporat / integrat (*embedded* - specific dispozitivelor mobile (smartphone-uri / tablete) și industriale

Serverul este specific centrelor de date având o construcție robustă, sisteme de prindere speciale în locurile special amenajate, o circulație a aerului pe orizontală foarte bună (în general din față în spate). Dimensiunea serverelor este dată de numărul unităților de rack ocupate (1U = $4,445cm$). În Figura 8.4 este ilustrat un server de dimensiune 1U (1 unitate de rack), cu sisteme de prindere în lateral. După cum se poate observa aerul va circula din față în spate și niciodată pe deasupra sau dedesubt din cauza modului în care serverele sunt așezate în rack: unul peste altul fără nici un spațiu.

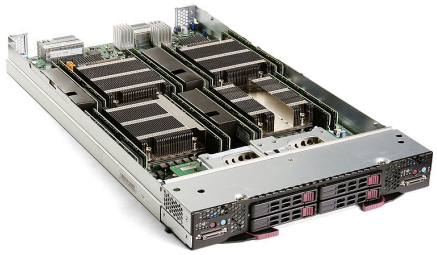


Figura 8.4: Server[1]

Desktopul este specific utilizatorilor finali având un raport preț/performanță foarte bun. După cum am precizat mai sus, și laptopul tot utilizatorului final se adresează. Avantajul desktopului în detrimentul laptopului este dat de faptul că poate fi ușor îmbunătățit, reparat și întreținut.

Desktopul vine și el în mai multe forme constructive:

- small factor
- normal factor

Small factor se referă la o carcasă compactă de mici dimensiuni ce poate fi pusă inclusiv pe birou, iar normal factor este o carcasa de dimensiuni mai mari. Avantajul small factor este după cum am precizat anterior dimensiunea redusă, iar dezavantajul este dat de faptul că suportă un număr limitat de extensii auxiliare (în general nu este loc și pentru ele în carcasă). Dezavantajul normal factor este dat de dimensiune, dar este mult mai ușor de personalizat.

Figura 8.5: Laptop[2]

Laptopurile reprezintă un alt model constructiv al unui calculator pentru a adresa portabilitatea fizică (vezi Figura 8.5). De obicei acestea conțin din fabrică toate

[1] https://commons.wikimedia.org/wiki/File:Supermicro_SBI-7228R-T2X_blade_server.jpg **CC BY-SA 4.0**

[2] https://commons.wikimedia.org/wiki/File:Lenovo_G500s_laptop-2905.jpg **CC BY-SA 4.0**

componentele dorite și este mai dificil de personalizat (în general se poate adăuga mai multă memorie, se poate schimba unitatea de stocare, dar cam atât).

Sistemele embedded (detaliate în Capitolul 15), de obicei, nu au o carcasă cu care vin de la producător. Acestea sunt niște plăcuțe care conțin toate perifericele integrate (SoC - *System on Chip*). Figura 8.6 prezintă un astfel de sistem embedded (integrat).

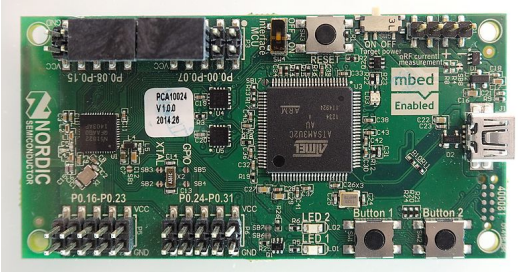


Figura 8.6: Sistem embedded (integrat)[1]

8.2 Componentele unui sistem de tip desktop

Mai sus am prezentat diferite forme constructive ale unui sistem de calcul. În acest capitol vom descrie componența unui sistem de tip desktop. Această componență este valabilă și pentru celelalte forme constructive ale sistemelor de calcul (server, laptop, embedded) doar că sunt dispuse și integrate în poziții diferite.

Componentele sunt prezentate vizual în Figura 8.7. Le enumerăm în continuare:

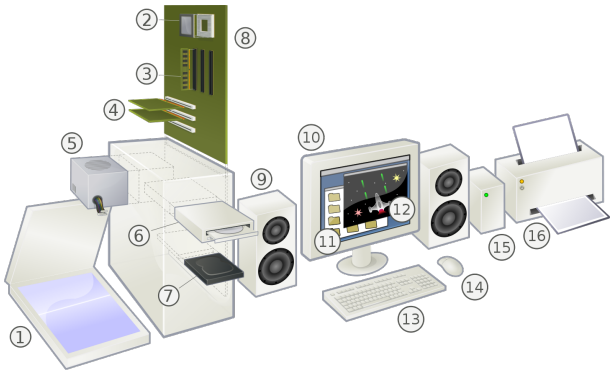


Figura 8.7: Componentele unui calculator[2]

1. scanner - cu ajutorul acesteia se pozează conținutul documentelor cu scopul de a fi digitalizate (transferate pe calculator). Acesta este un dispozitiv de intrare.
2. CPU (Figura 8.8) - procesorul (unitatea de procesare) execută instrucțiunile primite de la utilizator. Principalele caracteristici sunt frecvența (în mega/giga herți - MHz / GHz) care determină numărul de instrucțiuni executate pe secundă, mărimea memoriei cache, numărul de nuclee (un CPU poate avea mai multe unități de execuție înglobate în aceeași pastilă) și viteza cu care comunică cu

[1] https://commons.wikimedia.org/wiki/File:Embedded_World_2016,_Nordic_RF51822.jpg **CC0**
[2] https://commons.wikimedia.org/w/index.php?curid=4023664 **CC BY 2.5**

restul componentelor prin intermediul magistralei (se mai numește și frecvența magistralei FSB) .

3. memorie (Figura 8.9) - unitatea de memorie care este de tip RAM (*Random Access Memory*) și este reprezentată de o plăcuță îngustă care poate avea diferite dimensiuni exprimate în megaocteți (MB - megabytes) sau gigaocteți (GB - gigabytes). O altă caracteristică a memorie se referă la frecvența acesteia în general exprimată în megaherți (MHz) . Frecvența determină viteza cu care se realizează comunicația între memorie și magistrală, respectiv procesor.

Figura 8.8: CPU (procesorul)[1]

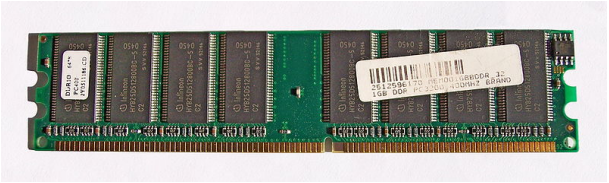


Figura 8.9: Memoria RAM[2]

4. plăci de extensie - sunt variante pornind de la plăcile de procesare a sunetului (pentru redarea acestuia), a elementelor grafice (pentru afișarea pe ecran), până la plăcile ce realizează conectivitatea cu alte calculatoare (respectiv Internetul). Vom extinde subiectul plăcilor de extensie în secțiunile următoare

5. sursă de alimentare a unității de calcul - transformă curentul alternativ în curent continuu la diferite tensiuni / intensități pentru fiecare componentă a sistemului de calcul. Principala caracteristică a surselor de alimentare este dată de putere (watt). Sursa trebuie sa fie dimensionată corespunzător pentru a reuși să acopere toți consumatorii din sistem (ex.: dacă avem un procesor mai puternic, cu o frecvență mai mare și mai multe nuclee avem nevoie de o sursă de putere mai mare)

6. unitate optică (CD/DVD - ROM) (Figura 8.10) - este un dispozitiv de intrare/ieșire persistent pentru redarea filmelor, a muzicii, dar se pot stoca și date care nu au legătură cu aria multimedia (ex. documente)

[1] https://commons.wikimedia.org/wiki/File:Intel_CPU_Core_i7_6700K_Skylake_top.jpg **CC BY SA**
[2] https://commons.wikimedia.org/wiki/File:DDR_RAM-3.jpg **Public domain**

7. unitate de stocare (HDD - *Hard Disk Drive*) (Figura 8.11) - este un dispozitiv de intrare/ieșire pentru stocarea permanentă/persistentă a datelor. De exemplu, ori de câte ori salvăm un fișier în curs de editare, acesta este scris pe unitatea de stocare. Aceasta este mult mai lentă ca unitatea de memorie și de aceea se preferă scrierea cât mai rară, în special când se întreprind acțiuni în timp real.

Figura 8.10: DVD-ROM[1]

Figura 8.11: Discul pentru stocare - HDD[2]

8. placă de bază (Figura 8.12) - reprezintă elementul central de conexiune între procesor, memorie și plăcile de extensie. Aceasta conține practic magistrala de comunicație și sloturile în care sunt introduse procesorul, memoriile și plăcile de extensie.
9. boxe conectate la placa de extensie (numită și placă de sunet) pentru a reda sunetul emis de unitatea de calcul
10. monitor - conectat la placa de extensie (numită și placă grafică sau GPU în acest caz) pentru a reda elementele grafice.
11. sistem de operare - software-ul ce este responsabil cu inițializarea și controlul componentelor hardware

[1] https://commons.wikimedia.org/wiki/File:ASUS_CD-ROM_CD-S520-A4_20080821.jpg **CC BY 3.0**
[2] https://commons.wikimedia.org/wiki/File:Hard_Drive_(11644419853).jpg **CC BY 2.0**

12. o aplicație - proces ce rulează în sistemul de operare
13. tastatură - dispozitiv de intrare prin care utilizatorul poate trimite comenzi către calculator sau poate redacta diverse documente

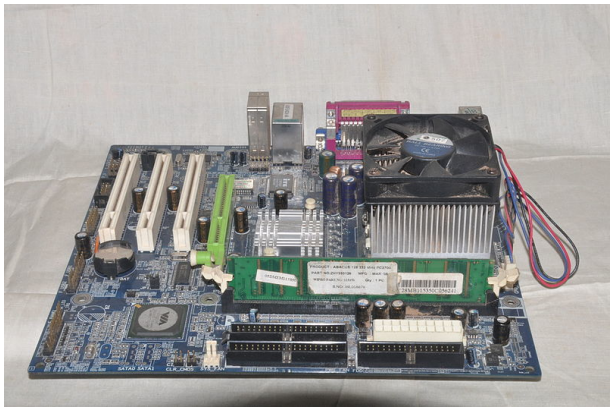


Figura 8.12: Placa de bază[1]

14. mouse - dispozitiv de intrare pentru a ușura controlul sistemului de operare și a aplicațiilor (care este posibil prin intermediul tastaturii, dar în medii grafice, operarea doar din tastatură devine foarte greoaie)
15. unitate de stocare externă (HDD extern - portabil) - are același rol ca unitatea de stocare prezentată la punctul de mai sus, cu avantajul de a fi portabilă și dezavantajul că este de obicei mai lentă.
16. imprimantă - dispozitiv care permite transpunerea documentelor sau imaginilor într-o formă fizică, pe suport de hârtie.

Acestea sunt elementele de bază ale unui sistem de calcul din zilele noastre. În mod curent există diverse extensii, dispozitive specializate ce se conectează la calculator (stick USB pentru comunicații, pentru stocare, token pentru criptare), dar folosesc aceleași interfețe ca unul dintre dispozitivele de mai sus.

8.3 Interacțiunea hardware - sistem de operare

Până la această secțiune am prezentat arhitectura logică a unui sistem de calcul, am enumerat clasificări în funcție de mai multe criterii (arhitectura unității de procesare, forma constructivă a sistemului de calcul), am iterat principalele componente ale unui sistem de tip desktop. Toate acestea trebuie controlate de o componentă software specializată: **sistemul de operare**. Sistemul de operare (*operating system* - OS) este primul program (software) ce rulează la pornirea unui sistem de calcul. Rolurile sistemului de operare sunt:

- descoperirea dispozitivelor ce formează sistemul de calcul
- inițializarea și configurarea dispozitivelor
- controlul dispozitivelor

[1] https://commons.wikimedia.org/wiki/File:Mother_board.JPG **CC BY SA 4.0**

Vom discuta despre descoperirea dispozitivelor în contextul celor două arhitecturi folosite în sistemele de uz general din ziua de astăzi (x86 și ARM). Descoperirea dispozitivelor se face folosind următoarele procedee:

- *autodiscovery* (descoperire automată de către sistemul de operare) - în general pe x86 - la pornirea sistemului rulează un program proprietar al plăcii de bază denumit BIOS (Basic Input/Output System - vezi Capitolul 9) care testează toate componentele conectate la placa de bază și certifică compatibilitatea acestora (între ele și cu placa de bază). Cel mai important lucru este dat de faptul că încarcă sistemul de operare de pe disc în memorie și pornește execuția acestuia. Din acest punct sistemul de operare este cel care va face din nou descoperirea tuturor componentelor, inițializarea și configurarea, precum și controlul acestora. Totul este făcut automat prin scanarea și enumerarea unor locații speciale din memorie pentru a identifica fiecare componentă.
- DTB (*Device Tree Blob*) - în general pe ARM - pe sistemele ARM în general nu există noțiunea de BIOS întrucât totul este integrat pe o singură plăcuță (SoC - *System on Chip*) și nu avem o configurație hardware dinamică (totul este prestabilit de la bun început). Aici există specificații clare la ce adresă trebuie încărcat sistemul de operare și de unde începe execuția acestuia. Fiind totul integrat într-un SoC, integrarea variind de la vendor la vendor (ex. adresa de început a memoriei este diferită), nu există nici un mecanism de autodiscovery. În general producătorul oferă un DTB (Device Tree Blob) în care este descrisă fiecare componentă SoC-ului. Acest DTB este încărcat în memorie împreună cu sistemul de operare, iar acesta interpretează acest DTB pentru a vedea de elemente are la dispoziție cu scopul de a le inițializa, configura și controla.

În ambele cazuri prezentate mai sus, până la rularea sistemului de operare, în practică, mai există un nivel de software numit *boot loader*. Boot loaderul este de fapt programul care rulează imediat după BIOS, în cazul x86, și imediat după pornirea sistemului, în cazul ARM. Boot loaderul are rolul de a încărca sistemul de operare în memorie. În cazul x86 cel mai folosit boot loader pe sistemele Linux este GRUB, iar pe sistemele Windows este unul proprietar. În cazul ARM avem un singur boot loader numit U-BOOT. Mai multe detalii despre boot loader, motivul pentru care acesta a fost introdus și ce opțiuni de configurare oferă, sunt prezentate în Capitolul 9.

Un exemplu simplu, care apare proeminent în viața de zi cu zi este folosirea unei imprimante. În general pentru a putea folosi imprimanta, spunem că aceasta trebuie instalată. Mai specific, trebuie instalat un driver pentru aceasta. Driverul este util sistemului de operare pentru putea controla dispozitivul (să ii dea comenzi de printare în acest caz și să primească răspunsuri de la acesta relativ la starea acțiunii de printare). Fără driver, dispozitivul nu ar putea fi folosit.

8.3.1 Drivere

O dată încărcat sistemul de operare, acesta este lansat în execuție. Din acest punct se va începe descoperirea tuturor elementelor hardware (de la procesor și memorie până la toate perifericele conectate), inițializarea și configurarea acestora, precum și controlul lor. Procesorul și memoria unui sistem de calcul sunt elementele centrale fără de care nu îl putem folosi (dacă am avea doar procesor și memorie, am putea rula

programe). Inițializarea, configurarea și controlul acestora este în datoria nucleului sistemului de operare. Celelalte elemente hardware (plăci de extensie, dispozitive externe) oferă o elasticitate foarte mare a sistemului, existând în foarte multe forme (interne/externe, se conectează pe porturi de diverse tipuri), îndeplinind obiective variate (afișare, producere sunet, imprimare foi). Acest avantaj, de a conecta extensii foarte diverse, vine la pachet cu un mare dezavantaj: software-ul (programele) necesare inițializării, configurării și controlului acestora. Toate acestea ar trebui implementate de către producătorul sistemului de operare. Acest lucru nu este sustenabil având în vedere varietatea de componente și vendori. Dezvoltatorii sistemelor de operare au venit în întâmpinarea acestei probleme prin crearea unui nou concept de program (software) numit *driver*.

Driverul este o subcomponentă a unui sistem de operare care este înglobat în acesta sau se poate încărca după ce sistemul de operare pornește. Rolul unui driver este bine definit de componenta pe care trebuie să o controleze. Avantajul introducerii noțiunii de driver în cadrul sistemului de operare este dat de faptul că fiecare vendor care dezvoltă o componentă hardware, își scrie propriul driver pentru controlul acelei componente/periferice, folosind interfața pusă la dispoziție de sistemul de operare. Astfel, dezvoltatorii sistemului de operare nu trebuie să se mai preocupe să scrie programe de control (driver) pentru fiecare dispozitiv din lumea aceasta, iar această sarcina cade în rolul vendorului respectiv.

Funcționarea unui driver și relația sa cu sistemul de operare și hardware-ul este prezentată schematic în Figura 8.13.

Pornind de la considerentele enumerate mai sus, putem identifica două tipuri de drivere:

- generice - cele care pot controla orice dispozitiv din aceeași gamă (ex. driverul de placă de sunet poate controla orice placă de sunet făcută de orice vendor). De obicei sunt dezvoltate de către producătorii sistemului de operare pentru a oferi un minim de funcționalitate.
- specifice - sunt dezvoltate de către vendorii/producătorii dispozitivelor și sunt specifice fiecărei versiuni/model de hardware. Acestea oferă toată gama de funcționalități suportată de componenta hardware în cauză.

La instalarea unui sistem de operare, în general există drivere generice pentru orice tip de dispozitiv, dar performanța / funcționalitatea sunt limitate. De aceea este recomandat ca de fiecare dată după instalarea unui sistem de operare, să navigați pe site-ul producătorului fiecărei componente să luați ultimul driver compatibil. Nu de multe ori ați observat că după instalarea sistemului de operare, experiența grafică lăsa de dorit (rezoluție mică, mișcări sacadate) până la instalarea software-ului recomandat de producător (acesta fiind driverul). Așadar recomandăm instalarea ultimelor versiuni de drivere pentru a beneficia de performanță și stabilitate maximă.

8.3.2 Controllere

Pentru inițializare, configurare și control, dispozitivele trebuie să comunice cu procesorul (unitatea de procesare) și invers. Pentru acest lucru trebuie implementat un protocol de comunicație în fiecare dispozitiv. Acest lucru ar duce la creșterea costului dispozitivului și la existența variată a protocoalelor de comunicații de la diferiți vendori.

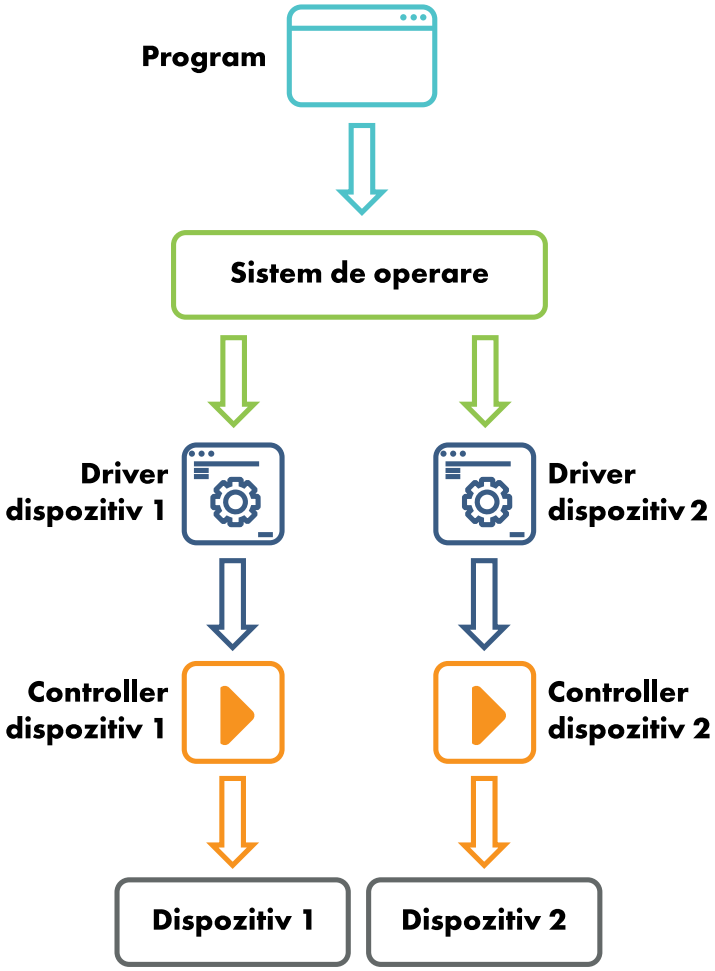


Figura 8.13: Relația sistem de operare - driver - controller - dispozitiv

Pentru a preîntâmpina acest lucru, dispozitivele au fost grupate în categorii, iar pentru fiecare categorie a fost implementat un nou element de gestiune numit **controller**.

Controllerul este cel care intermediază comunicația între dispozitive și procesor. Astfel dispozitivul trebuie să implementeze doar interfața de comunicație cu controllerul. Un alt rol al controllerului este acela de a limita impactul dispozitivelor lente asupra procesorului: dispozitivele sunt în general mult mai lente ca unitatea de procesare și, dacă ar exista comunicație directă între acestea procesorul, ar fi blocat la viteza de transfer/execuție a dispozitivului. Existând un tampon, numit controller (practic este un procesor), acesta de multe ori preia mai multe sarcini de la dispozitiv și le trimite pe toate deodată procesorului, eficientizând astfel comunicația. Figura 8.13 prezintă relația dintre sistemul de operare, drivere, controller și dispozitive, precum și fluxul de interacțiune dintre acestea.

Controllerele au și ele la rândul lor drivere (programe) care le ajută la inițializarea, configurarea și controlul acestora. Recomandarea de mai sus este valabilă și în cazul controllerelor: la instalarea sistemului de operare există controllere generice, iar recomandarea este să fie instalate drivere de la producător (disponibile de obicei fără niciun cost pe site-ul acestora).

Am prezentat până în acest punct comunicația între dispozitiv și procesor și faptul că aceasta este intermediată de controller. În rândurile ce urmează vom descrie procedeele de comunicație între dispozitive și memorie.

Un prim procedeu este DMA (*Direct Memory Access*). Controllerul primește de la procesor o adresă de memorie validă de unde va citi/va scrie datele pentru dispozitiv. Astfel procesorul nu trebuie să aștepte după dispozitiv să scrie.

Dacă dispozitivul nu dispune de un controller DMA, se poate aplica un alt procedeu și anume: *memory mapping* (maparea memoriei). Printr-un mecanism special aceeași zonă de memorie este vizibilă atât dispozitivului, cât și procesorului. Astfel dispozitivul poate citi/scrie către și dinspre memorie.

În ambele cazuri (DMA și Memory Mapping) avem nevoie de un mecanism de notificare (când a terminat de citit/scris dispozitivul). Acesta este implementat printr-un procedeu asincron numit **întrerupere**. Întreruperea este trimisă de dispozitiv (sau controller) atunci când au terminat de făcut o acțiune (citire/scriere/prelucrare). Întreruperea este un mecanism asincron, astfel procesorul va opri execuția curentă imediat ce întreruperea vine și o va trata. Cu orice întrerupere livrată, există și un context: de unde a venit, care este motivul. După tratarea întreruperii, procesorul își va continua execuția programului anterior.

8.4 Rolul extensiilor (plăcilor) auxiliare ale unui sistem de calcul

În această secțiune vom face o scurtă trecere a rolului extensiilor auxiliare (plăci de extensie) și vom prezente trei dintre cele mai importante plăci de extensie: video, de rețea și de sunet.

Plăcile de extensie se conectează folosind diverse porturi interne sau externe. Printre porturile interne putem enumera:

- PCI (*Peripheral Component Interconnect*) - este un standard vechi, iar plăcile de bază din ziua de astăzi aproape au renunțat la acest tip de slot (unele mai prezintă câte un astfel de slot din cauza prețului foarte scăzut al perifericelor)
- PCI Express (*Peripheral Component Interconnect Express*) - este urmașul PCI-ului și asigură diferite viteze de interconectare în funcție de componentele conectate (placă de rețea, placă video). În specificații veți observa că viteză este exprimată folosind notația x1, x2, x4, x8 sau x16. În funcție de viteză, se va observa că variază și dimensiunea slotului. În acest moment acesta este standardul de-facto pentru interconectarea internă oricărei componente într-un sistem de calcul. De menționat că PCI Express este abreviat ca *PCIe* sau *PCI-e*.

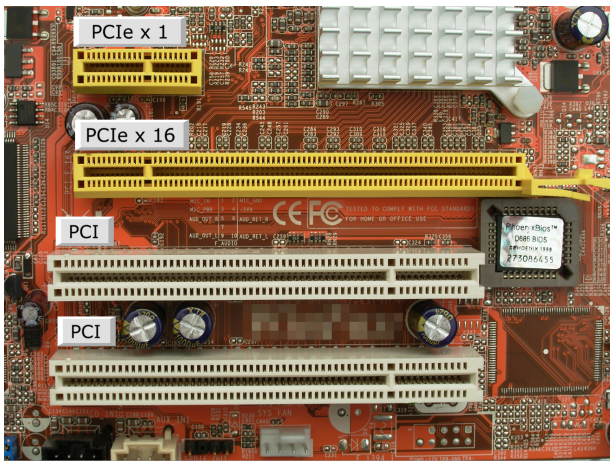


Figura 8.14: Sloturi PCI și PCIe[1]

În Figura 8.14 se poate observa cum arată un slot PCI în comparație cu unul PCIe x1 sau x16.

Portul extern cel mai des folosit pentru interconectarea dispozitivelor este USB-ul. În Figura 8.15 se poate observa forma unui port USB. Acesta vine în mai multe forme ce oferă performanțe diferite (viteză de comunicație):

- USB 1.0 - 12Mbps
- USB 2.0 - 480Mbps
- USB 3.0 - 5Gbps
- USB 3.1 - 10Gbps
- USB 3.2 - 20Gbps

Până în acest punct am discutat despre formele de conectare ale plăcilor de extensie la unitatea de calcul. Plăcile de extensie au un element central de aceeași importanță cu unitatea de procesare dintr-un sistem de calcul. Acel element central poartă numele de *chipset*. Chipsetul implementează logica centrală a dispozitivului, este cel care comunică cu sistemul de calcul.

[1] https://et.m.wikipedia.org/wiki/Fail:PCI_und_PCIe_Slots.jpg **Flexible license**

În general, driverele despre care am discutat anterior sunt dezvoltate pentru un anumit chipset, și nu pentru o placă anume. Pot exista plăci de la mai mulți vendori, care au același chipset. În acest caz același driver este compatibil și va fi dezvoltat de producătorul chipset-ului, nu te integratorul plăcii. Dăm un exemplu legat de o placă de rețea de 100 Mbps cu chipset RTL8139 (Realtek este producătorul chipset-ului): există cel puțin 2 producători (TP-Link și D-link) pentru această placă. Plăcile arată diferit, au serii diferite, dar au același chipset. Prin comparație putem lua două laptopuri produse de HP și Dell care au același procesor Intel. Aceste laptopuri vor putea rula același sistem de operare compatibil cu procesorul Intel fără nici o modificare.

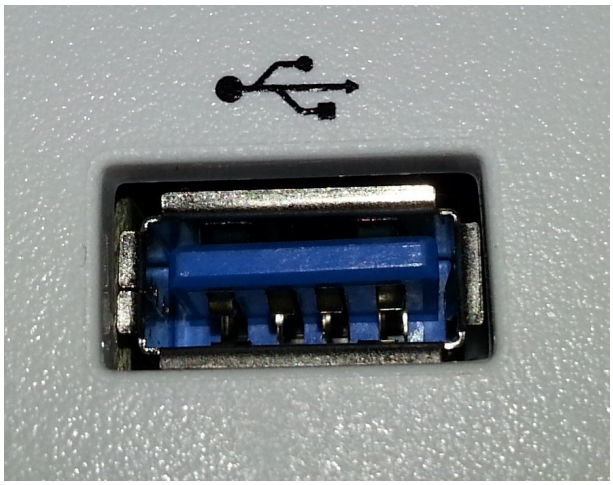


Figura 8.15: Port USB[1]

Accentuăm faptul că o placa de extensie conține un chipset care implementează logica necesară. De multe ori nu e suficient să căutăm un vendor de renume, ci trebuie să ne uităm și la un chipset de renume.

În continuare vom trece prin trei dintre cele mai importante plăci de extensie: video, sunet și rețea.

8.4.1 Placa video

Placa video (*graphics card*), este o placă de extensie ce se conectează pe portul PCIe (în general de tip x8 sau x16 pentru o viteză de comunicație ridicată) și are rolul de a afișa pe monitor (ecran) elemente grafice. Pentru afișarea elementelor grafice, în special în spațiu tri-dimensional, calcule complexe de geometrie trebuie făcute.

Pentru acest lucru, placa grafică este echipată cu unități specializate de calcul matematic. Pe lângă unitățile de calcul, placa grafică mai dispune și de propria memorie pentru a eficientiza timpul de procesare a datelor. La achiziția unei plăci video (sau mai poartă denumirea de placă grafică - GPU, *Graphics Processing Unit*) trebuie avute în vedere următoarele aspecte: portul de conectare al plăcii (PCIe și la ce viteză), capacitatea memoriei, frecvența de procesare, numărul de nuclee, precum și porturile de ieșire. În Figura 8.16 sunt prezentate formatul porturilor de ieșire ale unei plăci grafice: HDMI, DVI-I, DVI-D, DisplayPort. În general plăcile grafice sunt produse cu o combinație între aceste porturi, existând adaptoare la nevoie.

[1] https://commons.wikimedia.org/wiki/File:Blue_USB_port_without_USB_3.0_contacts.jpg **CC0**

Figura 8.16: Porturi ieșire placă grafică[1]

Atunci când veți analiza configurația unei plăci grafice, veți observa că dispune de un număr foarte mare de nuclee (de ordinul zecilor/sutelor). Din acest motiv, unitățile de prelucrare de pe plăcile grafice (GPU, *Graphics Processing Unit*) sunt folosite și în prelucrarea de date paralele folosind framework-uri specifice (ex. OpenCL). De reținut că nucleele prezente în placa grafică (GPU) nu se pot compara cu cele de pe unitatea de procesare (CPU). Nucleele de pe GPU sunt optimizate să facă operații simple aritmetice fără nici un fel de optimizare la verificări de condiție (branch-uri) și predicție. GPU-ul nu poate înlocui rolul unui procesor.

8.4.2 Placa de sunet

Placa de sunet (*sound card*) este o placă de extensie ce se poate conecta atât pe porturile interne (PCI, PCIe), cât și pe cele externe (USB) și are rolul de a procesa semnalele audio ce intră în calculator (microfoane) sau ies din calculator (boxele). În esență placa de sunet conține un convertor digital-analogic prin care convertește datele de pe calculator (digitale) în semnale analogice (sunetul emis de boxe) și invers (de la analogic - captat de microfon - la digital). În momentul în care achiziționați o placă de sunet trebuie să inspectați următoarele caracteristici:

- puterea cu care poate emite (în general în watt)
- numărul de canale, implicit și numărul de porturi disponibile: cele mai des întâlnite sunt cele denumite 2.0 care au 2 boxe sau căști (stânga - dreapta) și cele denumite 5.1 care au 5 sateliți (2 spate stânga-dreapta, 2 față stânga - dreapta, 1 față centru) și un difuzor pentru sunete joase (subwoofer).
- modalitatea de conectare la calculator (internă pe PCI/PCIe sau externă pe USB)

În Figura 8.17 este reprezentată o placă de sunet 5.1 ce se conectează pe portul intern PCI. Există câte un port de ieșire pentru fiecare zonă (față, spate, centru + subwoofer), un port de intrare pentru microfon și un port de intrare generic (pentru alte dispozitive).

8.4.3 Placa de rețea

Placa de rețea (*network interface card* - NIC) este o placă de extensie cu ajutorul căreia un sistem de calcul comunică la distanță cu alte sisteme de calcul. Prin intermediul plăcii

[1] https://commons.wikimedia.org/wiki/File:GPU_Interface.jpg **CC BY-SA 4.0**

[1] https://commons.wikimedia.org/wiki/File:A_VIA_Envy_Sound_Card_5.1_6_Channels_(VIA_VT1617A).jpg **CC BY SA 4.0**

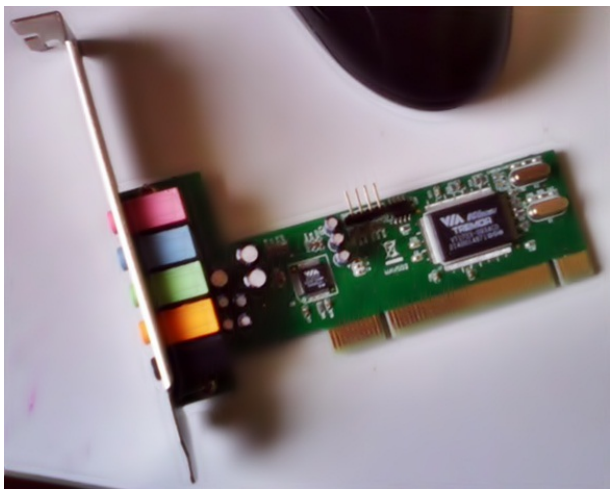


Figura 8.17: Placă de sunet[1]

de rețea se primesc/trimit date necesare utilizatorului sistemului de calcul (ex. se asigură accesul la Internet). Plăcile de rețea sunt de obicei interne conectându-se pe porturile PCI, respectiv PCIe. Există și plăci de rețea externe ce se pot conecta la portul USB, dar, de obicei, acestea sunt mai lente.

La achiziția unei plăci de rețea, trebuie ținut cont de următoarele caracteristici:

- portul pe care se conectează: este recomandat să fie PCIe dacă sistemul de calcul are un astfel de port disponibil. De asemenea trebuie verificată viteza portului PCIe (x1, x2, x4, x8, x16). În general se pot combina diferite viteze ale portului PCIe (fie că e vorba de placa de bază sau de placa auxiliară) iar viteza de funcționare va fi cea mai mică dintre cele două

- interfața disponibilă: în general cupru RJ-45 (protocol ethernet). Există și plăci cu suport de fibră optică: în descriere veți observa cuvântul cheie SFP (SFP+). Atenție pentru a utiliza suportul de fibră optică aveți nevoie de module speciale SFP ce au un cost mai ridicat decât cablul de cupru și de asemenea echipamentul în care vă conectați trebuie să suporte modul de fibră optică. În concluzie, în general veți alege portul de tip RJ-45.

- viteza de transmisie: 100Mbps, 1Gbps, 10Gbps. Cea mai răspândită în ziua de astăzi este viteza de 1Gbps.

În Figura 8.17 este reprezentată o placă de rețea 3Com cu interfață RJ-45 cu viteza 100Mbps.

Un lucru bun de menționat este acela că veți găsi diferențe foarte mari de preț pentru o placă de rețea ce funcționează la viteza de 1Gbps. De exemplu o placă de rețea Intel PCIe x1 pe 1Gbps poate ajunge la un pret de 400RON, în comparație cu o placă de rețea TP-Link cu aceleași specificații care are un preț de 50-60RON. Motivul constă în eficiența chipsetului plăcii: în cazul plăcii de rețea Intel chipsetul preia mare parte din funcția de procesare a datelor ce vin/pleacă în comparație cu TP-Link unde procesarea este lăsată în grija unității de procesare. Pentru o performanță sporită, mai ales în medii unde se realizează transmisii de date cu volum ridicat, sunt recomandate plăcile de rețea ce nu se bazează pe CPU atunci când trebuie să trimită/primească date.

8.5 Facilități moderne în hardware

În mod curent, producătorii de hardware încearcă să aducă noi facilități atât de funcționalitate, cât și de securitate. Implementarea direct în hardware a acestora aduce beneficiul performanței: execuția unei facilități în hardware este mult mai rapidă decât alternativa execuției în software, prin programe dedicate. În cadrul acestei secțiuni, vom prezenta două facilități prezente în hardware: virtualizare și TPM (*Trusted Platform Module*).

8.5.1 Virtualizarea în hardware

Pe un sistem de calcul putem rula un singur sistem de operare. Acesta trebuie să aibă control total al componentelor sistemului și nu le poate partaja cu o altă entitate. Acest lucru ne limitează la rularea unui singur sistem de operare la un moment dat. O dată cu trecerea timpului, necesitatea rulării mai multor tipuri de sisteme de operare, în același timp, pe același hardware, a crescut cu scopul de a eficientiza anumite operații (testarea unor programe pe diverse sisteme de operare) sau pentru a spori securitatea (un sistem de operare poate oferi facilități de securitate suplimentare față de altul).

Pentru a putea rula mai multe sisteme de operare în același timp s-a introdus conceptul de **virtualizare** (*virtualization*): dacă se rulează mai multe sisteme de operare în același timp, fiecare din ele va vedea o instanță virtuală a fiecărei componente a sistemului (CPU virtual, memorie virtuală, disc virtual etc.). Se realizează practic o multiplexare a fiecărei componente hardware. În anii 2005-2008 această multiplexare se realiza doar în software, fiind foarte lentă. Din anii 2007-2009 s-a introdus conceptul de virtualizare în hardware, acesta fiind suportat de către CPU. Viteza de rulare a sistemelor de operare virtualizate a crescut, având aproape aceleași performanțe ca un sistem de operare care rulează direct peste hardware. Pentru a beneficia de virtualizare, aceasta trebuie activată din BIOS (vezi Capitolul 9). De multe ori, fără activarea acestei opțiuni, soluțiile de virtualizare nu vor funcționa indicând un mesaj de eroare corespunzător.

Vom discuta mai multe despre virtualizare în Capitolul 14.

8.5.2 Trusted Platform Module

Securitatea este un alt subiect dezbătut în piața IT&C . Asigurarea securității sistemelor a devenit o prioritate pentru fiecare companie din lume (indiferent de domeniul de activitate). Instalarea de firewall-uri și antiviruși este utilă pentru protecția atacurilor din exterior (din Internet), dar nu protejează unitatea de calcul împotriva furtului fizic de date (ex.: dacă un laptop este furat, hoțul are acces la datele de pe laptop fără nicio problemă).

Pentru a preveni astfel de pierderi de date, unitățile de stocare ale calculatoarele sunt criptate integral folosind algoritmi specifici. Pentru criptare este necesară o cheie de criptare (asemănătoare unei parole). Această cheie trebuie stocată într-un loc. În timp

[1] https://commons.wikimedia.org/wiki/File:3Com-Etherlink-Network-Interface-Card-05.jpg **CC BY-SA 3.0**

Figura 8.18: Placă de rețea[1]

au fost întreprinse diverse metode de stocare a cheilor (la începutul discului, pe un stick USB), dar nu au fost suficiente. De aceea producătorii de procesoare au introdus un nou dispozitiv numit TPM.

TPM (*Trusted Platform Module*) este un standard internațional pentru un procesor criptografic capabil să stocheze chei criptografice (de criptare). O dată criptat discul, cheile vor fi stocate în acesta și protejate pe baza unei parole. Dacă unitatea de stocare va fi mutată pe alt calculator, atacatorul nu va putea vedea datele. Doar modulul TPM ce stochează acele chei va putea vizualiza conținutul unității de stocare. Modulul TPM trebuie activat din BIOS, ca și virtualizarea (vezi Capitolul 9).

8.6 Abstractizarea dispozitivelor în Linux

Pe calculatoarele ce rulează un sistem de operare bazat pe nucleul Linux, dispozitivele sunt abstractizate (se văd) cu fișiere și se află în calea absolută `/dev` din sistemul de fișiere. Intrările din `/dev` sunt numite, simplu, dispozitive. Dispozitivele din `/dev` sunt populate de frameworkul `udev`. Pe baza informațiilor oferite de nucleu, acesta creează intrările puse la dipoziție utilizatorului in `/dev`.

Fiind văzute ca fișiere, utilizatorul poate aplica aceleași operații pe care le aplică fișierelor, descrise în Secțiunea 2.3. Există două tipuri de dispozitive în calea `/dev`:

- fizice - au un corespondent hardware
- virtuale - nu au un corespondent hardware, sunt create de sistemul de operare.

Un exemplu de dispozitiv fizic este discul. Acesta este identificat printr-o intrare în sistemul de fișiere în directorul `/dev/`, intrare al cărei nume începe, de obicei, cu șirul `sd` sau `hd`, în funcție de inferfața de conectare. Listing 8.1 prezintă rularea unei

comenzi care afișează intrările din directorul `/dev/` corespunzătoare discurilor unui sistem.

```
student@uso:~$ ls -l /dev/sd*
brw-rw---- 1 root disk 8,  0 Sep 28 18:53 /dev/sda
brw-rw---- 1 root disk 8,  1 Sep 28 18:53 /dev/sda1
brw-rw---- 1 root disk 8, 16 Sep 28 18:53 /dev/sdb
brw-rw---- 1 root disk 8, 17 Sep 28 18:53 /dev/sdb1
brw-rw---- 1 root disk 8, 18 Sep 28 18:53 /dev/sdb2
brw-rw---- 1 root disk 8, 19 Sep 28 18:53 /dev/sdb3
```

Listing 8.1: Afișarea intrărilor de tip disc

După cum se poate observa, avem două discuri (`sda` și `sdb`) fiecare cu un număr diferit de partiții. Asupra acestora putem executa aceleași operații ce se aplică și fișierelor. De exemplu, în Listing 8.2 sunt citiți primi 10 octeți de pe discul `sda`.

```
student@uso:~$ sudo dd if=/dev/sda bs=1 count=10 -- | hexdump
10+0 records in
10+0 records out
10 bytes copied, 0.00132512 s, 7.5 kB/s
0000000 63eb 1090 d08e 00bc b8b0
000000a
```

Listing 8.2: Citirea datelor dintr-o intrare de tip disc

Pe lângă dispozitivele fizice, în cadrul sistemului Linux, avem și dispozitive virtuale cum ar fi:

- `/dev/zero` - la citire generează zero-uri
- `/dev/random` - la citire generaza numere aleatoare
- `/dev/null` - la scriere, preia toate caracterele și le șterge (*gaură neagră*)
- `/dev/full` - poate emula condițiile unui disc plin

Un exemplu de folosire a acestor dispozitive virtuale este prezentat în Listing 8.3. În acest exemplu sunt generate 10 octeți `0` într-un fișier. **Atenție!** Este vorba ca numărul `0` și nu de caracterul `0`; dacă am folosi comanda **`cat`** pe fișier, nu s-ar afișa nimic; de aceea am folosit **`hexdump`**, pentru a vedea numerele codificate.

```
student@uso:~$ dd if=/dev/zero of=test.zero bs=1 count=10
10+0 records in
10+0 records out
10 bytes copied, 0.00318695 s, 3.1 kB/s
student@uso:~$ hexdump test.zero
0000000 0000 0000 0000 0000 0000
000000a
```

Listing 8.3: Folosirea dispozitivelor virtuale

8.7 Vizualizarea componentelor hardware pe Linux vs Windows

8.7.1 Linux

Pentru a vizualiza dispozitivele descoperite de nucleul Linux putem folosi utilitarul **`dmesg`**, ca în Listing 8.4.

```
student@uso:~$ dmesg
[    0.000000] Linux version 4.15.0-118-generic (buildd@lgw01-amd64-039) (gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)) #119-Ubuntu SMP Tue Sep 8 12:30:01 UTC 2020 (Ubuntu 4.15.0-118.119-generic 4.15.18)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-4.15.0-118-generic root=UUID=a14d0991-a3d8-48d6-ac8c-327d1a524501 ro quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Centaur CentaurHauls
[    0.000000] x86/fpu: Supporting XSAVE feature 0x001: 'x87 floating point registers'
[    0.000000] x86/fpu: Supporting XSAVE feature 0x002: 'SSE registers'
[    0.000000] x86/fpu: Supporting XSAVE feature 0x004: 'AVX registers'
[    0.000000] x86/fpu: xstate_offset[2]:  576, xstate_sizes[2]:  256
[...]
```

Listing 8.4: Afișarea mesajelor nucleului Linux (dmesg)

După cum se poate observa, după încărcare, nucleul a început cu inspectarea zonelor de BIOS, după care a început enumerarea tuturor subcomponentelor din sistemul de calcul.

După pornirea sistemului, pentru a afișa informații despre hardware putem folosi utilitarul **`lshw`**, ce va afișa toate componentele sistemului sub formă arborescentă. Ieșirea comenzii este destul de stufoasă. O opțiune este folosirea parametrului `-short`, ce va face o sumarizare a configurației sistemului, ca în Listing 8.5.

```
student@uso:~$ lshw -short
WARNING: you should run this program as super-user
H/W path    Device      Class       Description
============================================
                        system      Computer
/0                      bus         Motherboard
/0/0                    memory      1993MiB System memory
/0/1                    processor   Intel(R) Core(TM) i7-8565U CPU @ 1.80 GHz
/0/100                  bridge      440FX - 82441FX PMC [Natoma]
/0/100/1                bridge      82371SB PIIX3 ISA [Natoma/Triton II]
/0/100/1.1              storage     82371AB/EB/MB PIIX4 IDE
/0/100/2                display     VirtualBox Graphics Adapter
[...]
```

Listing 8.5: Informații despre componentele hardware (lshw)

Pentru a afișa doar componentele ce țin de interconectarea PCI, putem folosi utilitarul **`lspci`**, ca în Listing 8.6.

```
student@uso:~$ lspci | head
```

```
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev
02)
00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II
]
00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:02.0 VGA compatible controller: InnoTek Systemberatung GmbH VirtualBox
 Graphics Adapter
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet
Controller (rev 02)
00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest
Service
00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97
Audio Controller (rev 01)
00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB
00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet
Controller (rev 02)
[...]
```

Listing 8.6: Informații despre componentele PCI (lspci)

Pentru a afișa informații despre procesor putem interoga intrarea virtuală `/proc/cpuinfo` sau pentru o afișare mai sumarizată putem folosi utilitarul **`lscpu`**, ca în Listing 8.7.

```
student@uso:~$ cat /proc/cpuinfo | head
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 142
model name      : Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz
stepping        : 12
cpu MHz         : 1992.017
cache size      : 8192 KB
physical id     : 0
siblings        : 1
[...]

student@uso:~$ lscpu | head
Architecture:        x86_64
CPU op-mode(s):      32-bit, 64-bit
Byte Order:          Little Endian
CPU(s):              1
On-line CPU(s) list: 0
Thread(s) per core:  1
Core(s) per socket:  1
Socket(s):           1
NUMA node(s):        1
Vendor ID:           GenuineIntel
[...]
```

Listing 8.7: Informații despre procesor

Pentru a afișa informații despre memorie putem interoga intrarea virtuală `/proc/meminfo` sau putem folosi utilitarul **`free`**, ca în Listing 8.8.

```
student@uso:~$ free
              total        used        free      shared  buff/cache
available
```

```
3  Mem:        2040972      915132       78544       17972     1047296
   920236
4  Swap:        777300       25600      751700
5  [...]
6
7  student@uso:~$ cat /proc/meminfo | head
8  MemTotal:        2040972 kB
9  MemFree:           78396 kB
10 MemAvailable:     920116 kB
11 Buffers:          131460 kB
12 Cached:           780896 kB
13 SwapCached:         1292 kB
14 Active:           952364 kB
15 Inactive:         764792 kB
16 Active(anon):     408200 kB
17 Inactive(anon):   414572 kB
18 [...]
```

Listing 8.8: Informații despre memorie

Pentru a monitoriza parametrii discului, putem folosi utilitarul **`hdparm`**, ca în Listing 8.9.

```
1  student@uso:~$ sudo hdparm /dev/sda
2
3  /dev/sda:
4   multcount     = 128 (on)
5   IO_support    =  1 (32-bit)
6   readonly      =  0 (off)
7   readahead     = 256 (on)
8   geometry      = 2088/255/63, sectors = 33554432, start = 0
```

Listing 8.9: Parametrii discului

8.7.2 Windows

Pe sistemul de operare Windows pentru a vedea detalii despre componentele atașate sistemului se poate folosi modulul *Device Manager* (`Start -> Run -> devmgmt.msc`).

După cum se poate observa în Figura 8.19 componentele sunt grupate, dispuse arborescent (asemănător comenzii **`lshw`** din Linux). Pentru a vedea dispozitivele, respectiv modelul acestora, se extinde grupul de interes. Pentru exemplu, vom extinde grupul *Processors*.

În Figura 8.20, am extins secțiunea *Processors* și observăm 8 intrări, întrucât avem 8 nuclee disponibile pe sistemul acesta. Ne apare și varianta de procesor, precum și frecvența acestuia (i7-4790 la frecvența 3.6Ghz).

8.7.3 Android

În mod implicit, în Android, nu există o opțiune prin care putem vizualiza componentele hardware ale unui dispozitiv (telefon/tableta). Pentru acest lucru putem folosi o aplicație ce poate fi instalată gratuit din PlayStore. O astfel de aplicație este *Android Hardware Info* (vezi Figura 8.21)

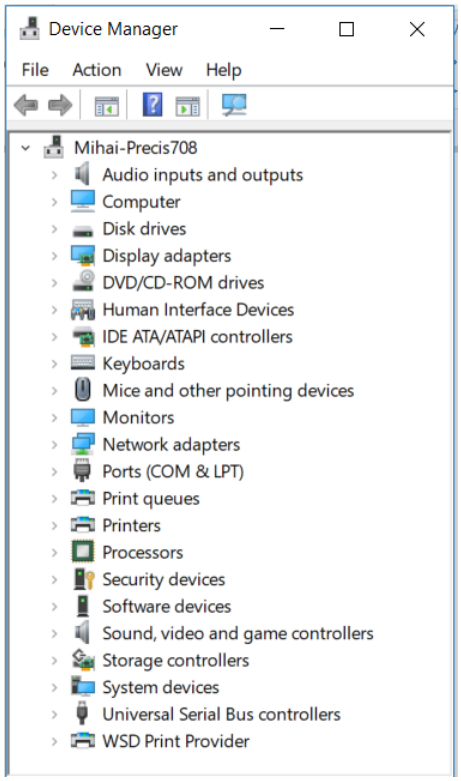


Figura 8.19: Device Manager

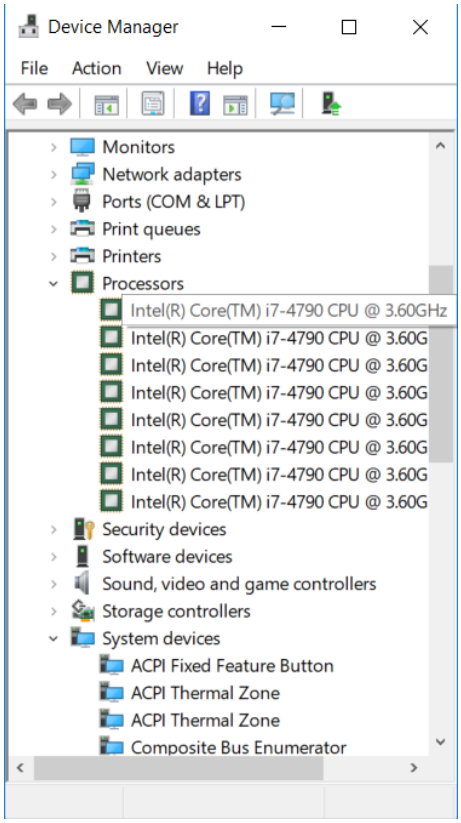


Figura 8.20: Vizualizarea procesoarelor în Device Manager

Figura 8.21: Vizualizarea componentelor pe Android

8.8 Vizualizarea versiunilor de drivere

De-a lungul capitolului am subliniat importanța programelor ce rulează pe sistemul de calcul și am accentuat modelul de dezvoltare al programelor, numite drivere, care controlează componentele sistemului. Cele mai des întâlnite probleme legate de stabilitatea unui sistem de calcul sunt cele cauzate de dispozitivele atașate, respectiv driverele instalate pentru acestea. Este util de știut cum vizualizăm ce drivere sunt active la un moment dat în sistem și care e versiunea acestora.

8.8.1 Vizualizarea driverelor pe Linux

Pe un sistem de operare cu nucleu Linux, vizualizarea driverelor se face cu ajutorul utilitarului **`lsmod`**, exemplificat în Listing 8.10.

```
student@uso:~$ lsmod | head
Module                  Size  Used by
btrfs                1150976  0
zstd_compress         163840  1 btrfs
xor                    24576  1 btrfs
raid6_pq              114688  1 btrfs
ufs                    77824  0
qnx4                   16384  0
hfsplus               106496  0
hfs                    57344  0
minix                  32768  0
[...]
```

Listing 8.10: Vizualizarea modulelor / driverelor (lsmod)

După cum se poate observa mai sus, utilitarul **`lsmod`** afișează doar driverele (sau module în terminologia Linux), dimensiunea acestora în memorie și dacă acestea sunt folosite de alte drivere. Pentru a afla informații despre versiunea unui driver/modul folosim utilitarul **`modinfo`**, ca în Listing 8.11.

```
student@uso:~$ modinfo hfsplus
filename:       /lib/modules/4.15.0-118-generic/kernel/fs/hfsplus/hfsplus
.ko
alias:          fs-hfsplus
license:        GPL
description:    Extended Macintosh Filesystem
author:         Brad Boyer
srcversion:     064BE87360260E2EE84695B
depends:
retpoline:      Y
intree:         Y
name:           hfsplus
vermagic:       4.15.0-118-generic SMP mod_unload
signat:         PKCS#7
signer:
sig_key:
sig_hashalgo:   md4
```

Listing 8.11: Informații despre un modul / driver (modinfo)

`modinfo` ne oferă informații despre plasarea fișierului de driver pe disc (`hfsplus.ko`), autorul modului, ce dependențe are precum și versiunea (`vermagic`).

8.8.2 Vizualizarea driverelor pe Windows

În cadrul sistemului de operare Windows, informațiile despre drivere se pot afla tot din *Device Manager* (`Start -> Run -> devmgmt.msc`).

Pentru a vizualiza informații despre driverul procesorului, vom da click pe unul din procesoarele listate în Figura 8.20. Va apărea o nouă fereastră ca în Figura 8.22 ce va avea un tab denumit *Driver*. Acesta ne va afișa toate informațiile relevante despre driver:

- compania care a furnizat driverul (*Driver Provider*)
- data la care a fost compilat (*Driver Date*)
- versiunea (*Driver Version*)
- semnătura (*Digital Signer*) - în general driverele pe Windows trebuie semnate de un vendor agreat de Microsoft pentru a putea fi încărcate.

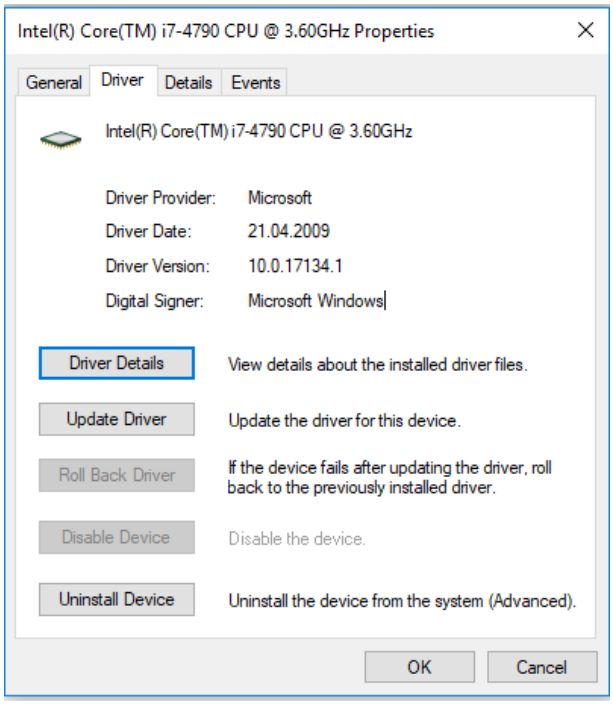


Figura 8.22: Vizualizarea unui driver pe Windows

8.9 Sumar

Sistemul de calcul este format din componente hardware necesare ca să rulăm software-ul.

Componentele hardware sunt formate din unitatea de procesare (CPU), unitatea de memorie și dispozitive de intrare/ieșire.

Sistemele de calcul se clasifică în funcție de arhitectura procesorului și de formatul constructiv (*form factor*). Fiecare arhitectură (x86, ARM) și fiecare format constructiv (server, desktop, laptop, embedded) este destinat unui anumit caz de utilizare.

Dispozitivele de intrare/ieșire îndeplinesc diferite roluri și sunt diverse. Procesorul interacționează cu dispozitivele prin intermediul controllerelor. Sistemul de operarea controlează dispozitive prin intermediul driverelor, componente software dedicate.

Sistemele de operare moderne oferă utilitare și interfețe dedicate pentru investigarea componentelor hardware și a driverelor.

Capitolul 9

Pornirea sistemului

Atunci când un sistem hardware pornește, acesta urmărește încărcarea sistemului de operare, pornirea acestuia și a proceselor de bază. Această acțiune se cheamă *booting* sau *bootstrapping*: modul în care un sistem, din momentul în care este alimentat și pornit fizic, ajunge să ofere o interfață de lucru utilizatorului. Procesul de boot/booting se referă la pașii urmați de la pornirea electrică (*power on*) până la pornirea proceselor.

Înțelegerea procesului de boot este utilă pentru un utilizator dacă dorește să își configureze modul în care este configurat hardware-ul sistemului, ce sistem de operare rulează, opțiuni de rulare a sistemului de operare și ce procese sunt create la pornire (*startup*). Pentru un utilizator avizat, a ști informații despre anatomia procesului de boot și a secvența de pași urmați este important pentru a depana probleme ce pot apărea sau pentru a îmbunătăți aspecte precum timpul de pornire, securitatea sistemului, consumul de resurse.

După pornirea fizică (power on) a sistemului, este încărcat firmware-ul de boot (numit BIOS pe arhitectura x86), apoi este identificat un dispozitiv bootabil, este încărcat bootloaderul, apoi sistemul de operare și apoi sunt pornite procesele de început (*startup processes*). În Figura 9.1 este prezentat un sumar al pașilor urmați în secvența de boot, urmând să îi detaliem în secțiunile de mai jos:

1. firmware de boot (BIOS)
2. dispozitiv bootabil
3. bootloader
4. sistem de operare (kernel) și drivere de dispozitiv
5. procese inițiale

9.1 Secvența de boot

În cadrul secvenței de boot, fiecare pas, descris în Figura 9.1, are rolul său. Unul dintre roluri este acela de a transfera controlul următorului pas: fiecare pas are capacitate limitată în procesul de pornire, următorul pas preluând rezultatul său și realizând alte

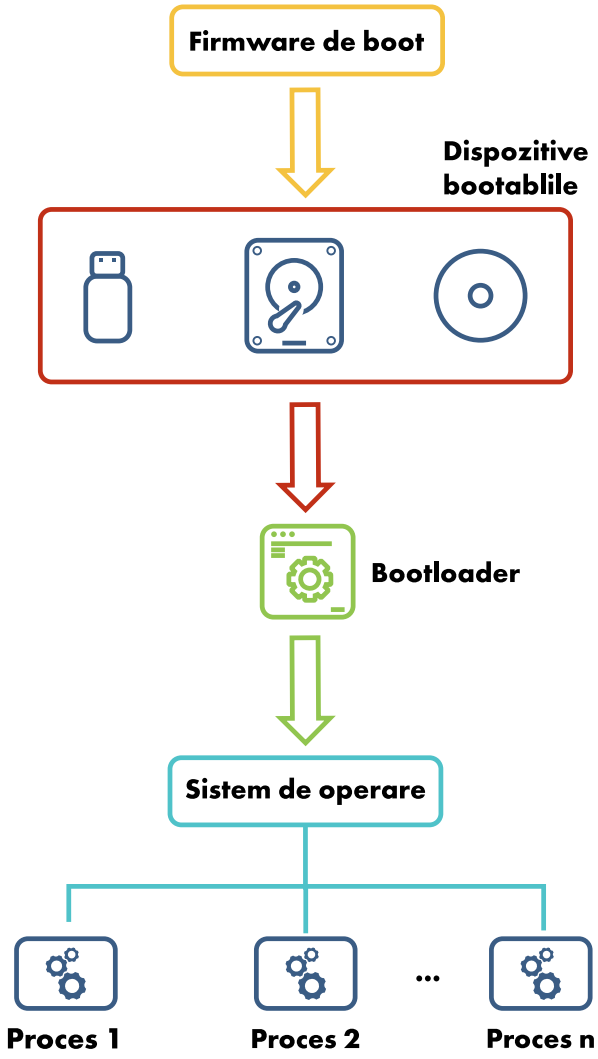


Figura 9.1: Secvența de boot pe PC

acțiuni. Alt rol al fiecărui pas este de configurare a unor opțiuni specifice în procesul de boot, așa cum vom preciza mai jos.

În momentul pornirii fizice (*power on*), sistemul încarcă firmware-ul de boot dintr-o zonă dedicată. Firmware-ul de boot este numit BIOS (sau UEFI) pe arhitectura x86. Firmware-ul de boot există pe sistemul fizic indiferent de prezența sau nu a unui sistem de operare sau dispozitiv bootabil. Firmware-ul de boot realizează verificări de bază ale funcționării hardware-ului, oferă utilizatorului posibilitatea de configurări aproape de hardware (precum activarea/dezactivarea virtualizării, configurarea vitezei de acces la memorie) și stabilește ordinea de boot: ce dispozitive și în ce ordine sunt verificate pentru a trece la următorul pas.

Firmware-ul de boot identifică dispozitivul de boot și încarcă o secvență dedicată de pe acesta. Dispozitivul de boot poate fi hard disk drive, USB flash drive, CD-ROM drive. Dispozitivul de boot nu are un rol activ, ci unul de stocare. Acesta stochează următoarea secvență de cod care să fie executată, în mod tipic bootloaderul. Bootloaderul este încărcat de firmware-ul de boot din dispozitivul de boot în memorie. Apoi îi este transferat controlul bootloaderului.

Întrucât firmware-ul de boot are, de obicei, capacități limitate de a încărca în memorie părți stocate pe un dispozitiv precum hard disk, se transferă controlul bootloaderului pentru a gestiona acest lucru. Bootloaderul urmărește încărcarea în memorie a sistemului de operare (a imaginii de kernel) de pe un dispozitiv de stocare. Bootloaderul poate oferi opțiunea utilizatorului de a decide ce sistem de operare să încarce, funcționalitate utilă pentru sistemele dual boot (când putem boota și Windows și Linux).

Sistemul de operare este pornit cu ajutorul bootloaderului din imaginea de kernel. Imaginea de kernel este un fișier care se regăsește pe un dispozitiv de stocare; din acest dispozitiv este încărcată în memorie și îi este transferat controlul. Imaginea de kernel este practic un fișier executabil care conține datele și codul sistemului de operare. Transferul controlului înseamnă execuția codului sistemului de operare. Din acest punct, sistemul de operare deține controlul sistemului și asigură buna funcționare a acestuia.

O dată încărcat sistemul de operare, acesta încarcă și driverele de dispozitiv (*device drivers*). Driverele de dispozitiv sunt reținute în fișiere similare imaginii de kernel și sunt încărcate în memorie de către sistemul de operare pentru gestionarea dispozitivelor hardware: disc, rețea, tastatură etc.

După încărcarea driverelor, sistemul de operare pornește primele procese ale sistemului (*startup processes*). Aceste procese sunt serviciile de bază ale sistemului (precum menținerea timpului sistemului, aplicarea configurațiilor de rețea) și aplicațiile de interfață cu utilizatorul (shellurile). Acum utilizatorul se poate autentifica în sistemul de operare și poate beneficia de serviciile acestuia.

Secvența de pași de bootare este comună pentru arhitecturi hardware și sisteme de operare diferite, dar fiecare pas are caracteristici și denumiri particulare. Pe arhitectura x86 firmware-ul de boot se numește BIOS sau UEFI; pe arhitectura ARM nu are o denumire specifică. Bootloaderele cele mai comune folosite pe arhitectura x86 sunt GRUB (pentru Linux) sau WinLoader (pentru Windows), în vreme ce Das U-boot e un bootloader folosit pentru platforme embedded precum ARM.

În cele ce urmează vom descrie în detaliu pașii din secvența de boot.

9.2 Firmware de boot

Firmware-ul de boot constă dintr-unul sau mai multe programe ce își înlănțuie execuția în diverse etape. Prima etapă constă, în cazul procesoarelor moderne, într-un program integrat în procesor(numit **boot ROM**). Acesta urmând să încarce bootloaderul secundar, ulterior ducând la încărcarea sistemului de operare.

În cazul platformelor Intel 64 și AMD64, firmware-ul de boot încărcat după boot ROM se află pe un chip flash de pe placa de bază, putând fi rescris la nevoie.

Protocolul de bootare Intel specifică doar că firmware-ul trebuie să se afle pe un anumit chip flash și să își înceapă execuția la o anumită adresă, numită *reset vector*. Restul este lăsat la creativitatea dezvoltatorului.

Diferența dintre bootloader și BIOS este oarecum vagă, dar, în limbaj comun, BIOS se referă la programul din flash de unde procesorul își începe execuția controlată de dezvoltator, în timp ce bootloader se referă la un program gestionat de administrator, program ce este lansat de BIOS. Dar, având în vedere că BIOS-ul poate fi un program arbitrar, există posibilitatea de a integra bootloaderul în acesta. Un exemplu este combinația dintre coreboot și GRUB2.

Pe platforme Intel, BIOS-ul are următoarele responsabilități:

- inițializează și verifică componentele hardware ale sistemului
- permite configurarea componentelor și opțiunilor hardware ale sistemului
- blochează acele configurații astfel încât să nu poată fi schimbate de sistemul de operare
- generează configurația pentru descoperirea și inițializarea hardware-lui de către sistemul de operare
- descoperă dispozitivele ce furnizează un bootloader
- lansează, posibil automat, bootloaderul

Firmware-ul de boot este rulat în momentul pornirii sistemului (adică la apăsarea pe butonul de pornire). Partea de verificare a componentelor hardware poartă numele de POST (*Power-On Self Test*) pe arhitectura x86; dacă sunt defecte hardware acestea vor fi semnalizate de firmware-ul de boot, în general prin mesaje sonore sau vizuale simple.

Firmware-ul de boot este parte a hardware-ului sistemului și vine „preinstalat”. În mod obișnuit, firmware-ul de boot este furnizat de producătorul plăcii de bază care conține și chipul flash.

Firmware-ul de boot permite realizarea de configurări la nivelul sistemului și hardware-ului. Mai jos sunt o serie de configurări permise de firmware-ul de boot:

- stabilirea unei parole de protejare a meniului de configurare
- configurarea timpul sistemului
- activarea suportului hardware de virtualizare
- stabilirea viteză de acces la memorie
- activarea bootării de pe stick USB

- configurarea ordinii de boot
- activarea/dezactivarea folosirii porturilor USB

9.2.1 BIOS și UEFI

Pe arhitectura x86, firmware-ul de boot poartă numele de BIOS (*Basic Input-Output System*). În ultima perioadă, BIOS legacy este înlocuit de UEFI (*Unified Extensible Firmware Interface*). UEFI este o formă îmbunătățită de firmware de boot. UEFI în sine reprezintă un standard gestionat de UEFI Forum din care fac parte producători de software și hardware, ceea ce duce la caracteristici comune indiferent de sistemul hardware folosit. Implementarea de referință a acestuia este EDK2[1], o soluție open source. Dar este o implementare incompletă, din cauza lipsei de suport hardware. Suportul hardware necesar pentru bootarea majorității plăcilor de bază este proprietar.

Figura 9.2 conține screenshoturi cu un ecran de boot BIOS[2] și unul de boot UEFI[3].

Meniurile de configurare pentru BIOS și UEFI sunt accesate la pornirea sistemului folosind o combinație de taste a furnizorului plăcii de bază, cel mai adesea `F2`, `F8`, `F12` sau `Esc`. La apăsarea combinației adecvate se accesează ecranul de configurare specific.

Configurațiile realizate sunt în mod uzual salvate la părăsirea modului de configurare. Aceste configurații sunt salvate într-un chip de memorie dedicat numit CMOS (de la *Complementary Metal-Oxide-Semiconductor*, tehnologia folosită pentru tranzistoarele ce compun circuitele integrate moderne). Această memorie este nevolatilă și își păstrează informațiile după resetarea sistemului. Chipul de memorie este alimentat de o baterie dedicată numită baterie CMOS, similară cu cea din Figura 9.3[4].

Dacă bateria CMOS este scoasă din soclul ei sau este epuizată, configurația se pierd și BIOS-ul bootează la niște valori implicite. Aceasta este o formă facilă de resetarea a parolei de BIOS, în cazul în care o astfel de parolă este configurată, așa cum vom preciza în Secțiunea 9.7.

Pe lângă interfața grafică mai prietenoasă, UEFI folosește o nouă schemă de partiționare a discurilor numită GPT (*GUID Partition Table*) spre deosebire de schema MBR (*Master Boot Record*) utilizată de BIOS. Aceasta îi permite o flexibilitate sporită în accesarea dispozitivului de boot și alegerea următoarei faze din procesul de boot: încărcarea bootloaderului. Vom reveni la schemele de partiționare GUID și MBR în Secțiunea 9.3.1; detalii vom prezenta și în Secțiunea 10.2.

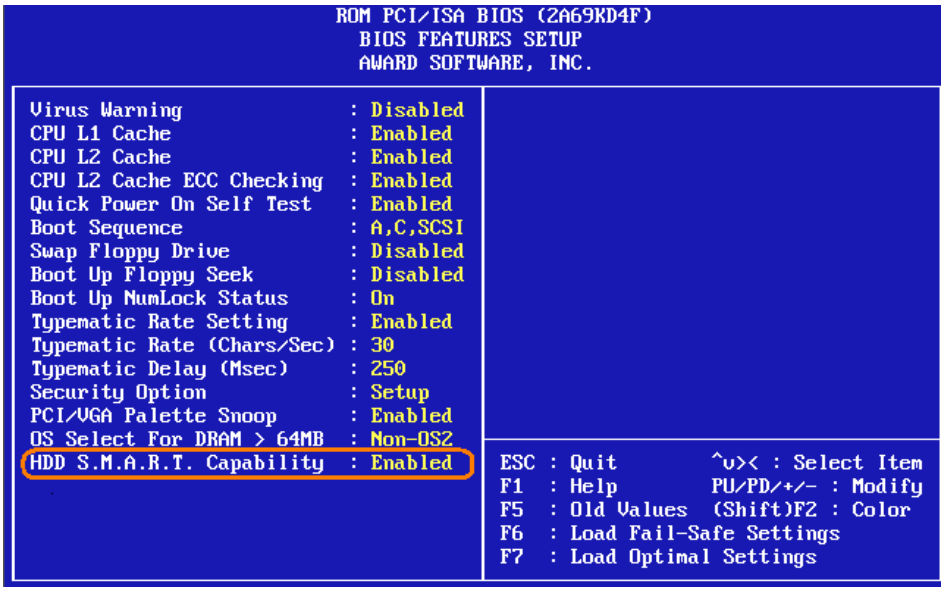
În general, UEFI permite și un mod de bootare compatibil cu BIOS. În acest mod, procesul de bootare este similar cu cel de BIOS și se folosește tot schema de partiționare MBR și modul de încărcare a bootloaderului. Altfel se folosește modul nativ de bootare UEFI. UEFI folosește o partiție dedicată numită ESP (*EFI System Partition*), formatată cu sistem de fișiere FAT, în care stochează imagini de firmware (numite imagini EFI) care reține codul bootloaderului ce va fi încărcat la următorul pas.

[1] https://github.com/tianocore/tianocore.github.io/wiki/EDK-II

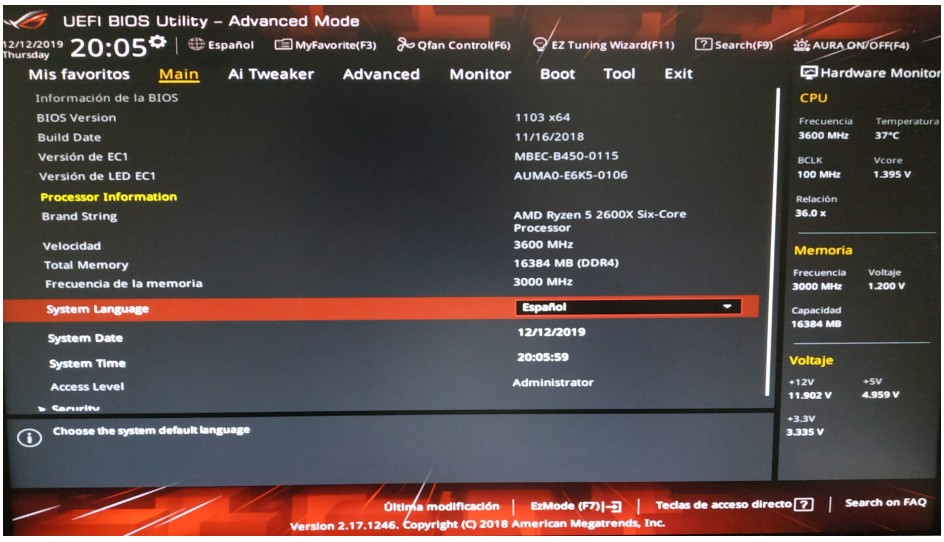
[2] https://commons.wikimedia.org/wiki/File:Bios-configuracion-smart-habilitada.png (CC BY-SA 3.0)

[3] https://commons.wikimedia.org/wiki/File:Bios_asus.jpg (CC BY-SA 4.0)

[4] https://commons.wikimedia.org/wiki/File:Yakumo_Notebook_536S_-_CR2032_backup_battery_on_motherboard-4667.jpg (© Raimond Spekking / CC BY-SA 4.0 (via Wikimedia Commons))



(a) BIOS



(b) UEFI

Figura 9.2: BIOS și UEFI

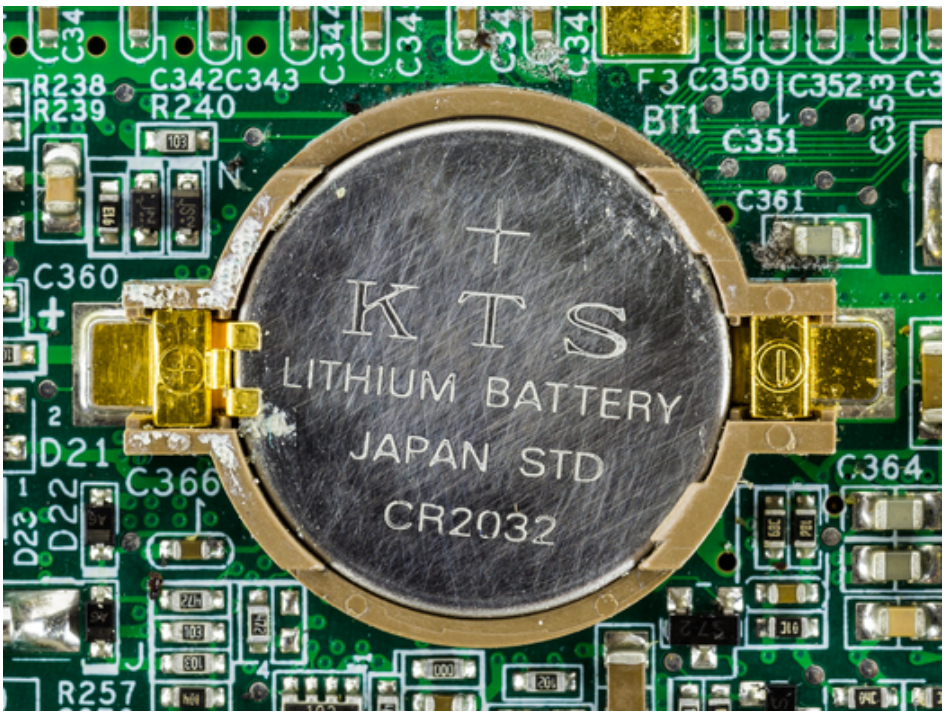


Figura 9.3: Bateria CMOS

Tabelul 9.1 prezintă principalele deosebiri[1] între BIOS și UEFI:

Tabelul 9.1: Comparație BIOS-UEFI

Criteriu	**BIOS**	**UEFI**
localizarea codului	chip EPROM pe placa de bază	fișiere `.efi` în partiția ESP
schemă de partiționare implicită	MBR	GUID
dimensiunea maximă a discului	2.2 TB	9 ZB
ecran de configurare	text	grafic
suport SecureBoot	nu	da

Datorită flexibilității, sistemele PC moderne folosesc din ce în ce mai mult UEFI, cu probabilitate mică de a mai întâlni, în următorii ani, sisteme care folosesc BIOS. BIOS rămâne însă folosit pe sistemele mai vechi și înțelegerea modului său de funcționare e utilă pentru a vedea evoluția firmware-ului de boot pe arhitectura x86.

9.2.2 Alte firmware-uri de boot

Fiecare sistem hardware care rulează un sistem de operare generalist (*general purpose*) are un firmware de boot, prima componentă rulată la pornirea sistemului. Pe arhitectura x86, de fapt pe sistemele de tipul PC (Personal Computer), acest firmware de boot este BIOS-ul, iar în ultimii ani UEFI.

Sistemele care folosesc procesoare ARM sunt mai variate, fără să aibă un format unitar de conectare a componentelor hardware. Mai mult, producătorii de sisteme care

[1] https://www.freecodecamp.org/news/uefi-vs-bios/

folosesc arhitecturi ARM sunt foarte variați. Lucru care face mai dificilă dezvoltarea de software de sistem pentru o plajă de dispozitive ARM. Totuși, în ultimii ani, EDK2 a căpătat suport pentru anumite platforme ARM.

Pe sistemele PC o alternativă de firmware de boot este coreboot (și alternativa libreboot). coreboot este o versiune liberă/deschisă de firmware de boot, ca o reacție la faptul că firmware-ul de BIOS și UEFI este în general furnizată de producător în formă binară (*binary blob*). Dincolo de aspecte de filozofie (software liber vs. proprietar), dezvoltatorii coreboot invocă posibilitatea ca furnizorii de binare BIOS/UEFI să aibă accese ascunse (*backdoor*), iar un firmware de boot deschis oferă posibilitatea verificării codului. coreboot încă folosește unele componente binare în vreme ce libreboot le înlocuiește pentru o versiune complet deschisă (având ca dezavantaj lipsa suportului pentru anumite platforme). Dezavantajul coreboot este folosirea sa redusă pe sisteme hardware, doar câteva folosind coreboot în loc de UEFI (sau BIOS). coreboot este firmware-ul preferat de Chromebook-uri.

9.2.3 Configurare dispozitive bootabile

Unul dintre rolurile firmware-ului de boot este configurarea dispozitivelor bootabile și selectarea ordinii de boot. Figura 9.4[1] afișează ecranul BIOS pentru stabilirea ordinii dispozitivelor bootabile.

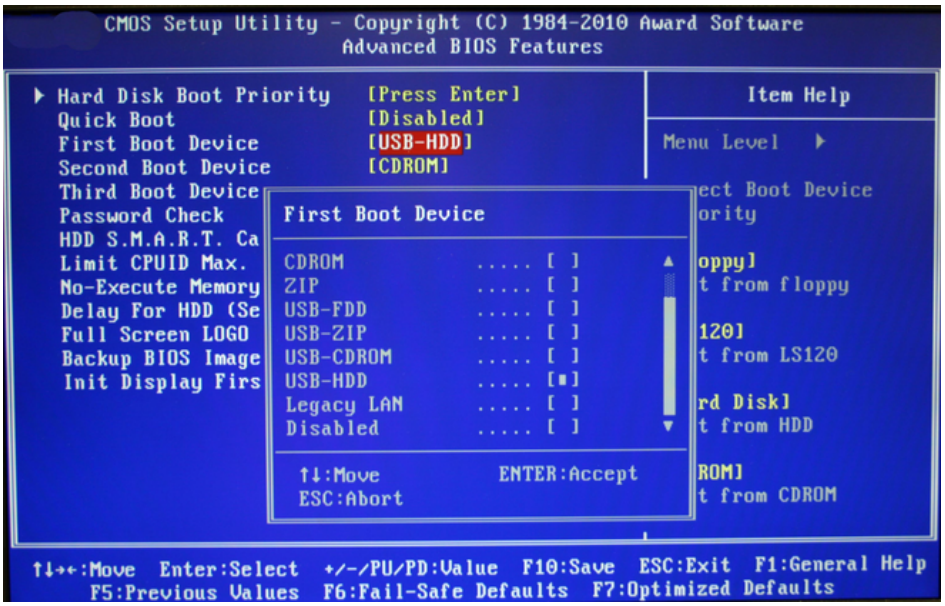


Figura 9.4: Stabilirea ordinii dispozitivelor bootabile în BIOS

Atunci când firmware-ul va trece la pasul următor va trece, în ordine, prin dispozitivele de boot configurate și va boota de pe primul disponibil. Dispozitive de boot pot fi discuri, USB flash drive-uri, CD ROM-drive sau poate fi vorba de bootare din rețea folosind PXE (*Preboot eXecution Environment*).

Un dispozitiv bootabil, identificat ca atare de firmware, are un marcaj specific: conține o secvență de cod care este rulată de firmware și care este următoarea fază a procesului

[1] https://commons.wikimedia.org/wiki/File:Bios-configuracion-orden-arranque.png (CC BY-SA 3.0)

de boot. Această secvență este, de obicei, bootloaderul, despre care vom discuta detaliat în Secțiunea 9.4.

PXE este o metodă prin care putem configura pornirea unui sistem de operare de pe un alt sistem (server) accesibil prin rețea. După o configurare specifică, pe care nu o vom detalia aici, o stație poate încărca sistemul de operare de pe alt sistem.

Despre dispozitive bootabile vom discuta mai multe în Secțiunea 9.3.

9.2.4 Secure Boot

Procesul de boot al unui sistem poate fi corupt: pot fi modificați / deturnați pași din acest proces. Dorim să avem verificare criptografică (semnătură criptografică) pentru imaginile de firmware pentru a le garanta integritatea.

Secure Boot este o tehnică ce se asigură că firmware-ul de boot și alte componente din procesul de boot sunt securizate. Secure Boot stochează semnătura componentelor și verifică la bootare că aceasta este corespunzătoare.

Secure Boot este o tehnologie UEFI, parte din standard. Există un certificat digital al entității care furnizează bootloaderul, imaginea de kernel și drivere. Mai multe despre certificate digitale vom vorbi în Capitolul 12.

Tehnologia de tipul Secure Boot este utilă în special pe dispozitive mobile de tip smartphone, unde bootloaderul și imaginea de kernel sunt furnizate de producătorul telefonului și unde se doresc garanții că acestea nu au fost modificate. Dispozitivele mobile ce folosesc Android sau iOS folosesc Secure Boot, adică mecanisme de verificare criptografică a imaginii de bootloader și a imaginii de kernel folosite. Pe Android tehnologia se numește Verified Boot, iar pe iOS se numește Secure Boot.

9.3 Dispozitiv de boot

După ce firmware-ul de boot încheie verificarea componentelor hardware, trece la următorul pas: încărcarea bootloaderului. Bootloaderul este o componentă software care încarcă sistemul de operare, oferind utilizatorului posibilitatea de selectare a sistemului de operare încărcat și de configurare a opțiunilor de pornire a acestuia.

Bootloaderul se găsește pe un dispozitiv de boot, un suport de stocare pentru imaginea de bootloader. Acest dispozitiv trebuie identificat de firmware-ul de boot, care apoi localizează și încarcă imaginea de bootloader.

Firmware-ul de boot oferă utilizatorului posibilitatea configurării dispozitivelor de boot: activarea și dezactivarea dispozitive de boot și selectarea ordinii în care acestea sunt consultate pentru bootare. Dacă, într-o anumită situație, dispozitive bootabile au fost configurate în ordinea CD-ROM drive, USB flash drive, hard disk drive, atunci firmware-ul de boot va investiga pe rând aceste dispozitive. Adică va investiga dacă există un CD-ROM în CD-ROM drive și dacă este CD-ROM bootabil, va investiga dacă porturile USB ale sistemului au conectate un USB flash drive și dacă este bootabil, și va verifica dacă hard disk drive-ul sistemului este bootabil.

Un dispozitiv este bootabil atunci când conține o imagine de bootloader care să fie încărcată în memorie și căreia să îi fie transferat controlul din partea firmware-ului de boot. Pe sisteme precum dispozitivele mobile, firmware-ul de boot are în general preconfigurat dispozitivul de boot și calea către imaginea de bootloader pe care să o încarce. Pe sistemele PC, utilizatorul poate configura atât dispozitivul de boot cât și calea către imaginea de bootloader. În cazul sistemelor PC diferă modul de încărcare a bootloaderului în funcție de firmware-ul de boot (BIOS sau UEFI) și schema de partiționare (MBR sau GPT), despre care vorbim în continuare.

9.3.1 Scheme de partiționare: MBR și GPT

Schemele de partiționare descriu partițiile pe disc și sunt folosite de firmware-ul de boot. Schemele de partiționare folosite pe sistemele PC sunt MBR și GPT.

MBR (*Master Boot Record*) este schema de partiționare clasică pe PC. În cadrul acestei scheme de partiționare primul sector de disc (de 512 octeți) conține tabela de partiții și prima parte a bootloaderului (numită 1st stage bootloader). Primul sector de disc se mai cheamă și sectorul zero sau *boot sector* sau *master boot record*, de unde și denumirea schemei de partiționare. Dată fiind dimensiunea redusă a acestui sector, doar patru partiții pot fi descrise. Numim aceste patru partiții primare. Pentru a compensa această limitare, putem alege o partiție să fie o partiție extinsă și în cadrul acelei partiții extinse să creăm oricâte partiții logice.

GPT (*GUID Partition Table*) este o schemă modernă de partiționare pe PC. În cadrul acestei scheme de partiționare există 128 de partiții, identificate în mod unic, la nivel global printr-un șir numit GUID (*global unique id*). Schema de partiționare GPT cuprinde 128 de partiții, eliminând limitarea de 4 partiții a schemei de partiționare MBR.

Un disc este bootabil dacă sectorul de boot al acestuia are ca ultimi doi octeți valoarea în hexazecimal `0x55AA`. În momentul în care acei ultimi doi octeți au acea valoare, firmware-ul de boot va putea folosi acest disc pentru a boota.

În general, asociem schema de partiționare MBR cu bootarea folosind BIOS, iar schema de partiționare GPT cu bootarea folosind UEFI.

9.3.2 Încărcarea bootloaderului pentru BIOS și MBR

Atunci când folosim schema de partiționare MBR, BIOS-ul extrage din sectorul de boot (marcat cu valoarea `0x55AA` pe ultimii doi octeți) prima parte din bootloader, numită *1st stage bootloader*. Aceasta este o secvență de cod redusă cu operații minime; este încărcată de BIOS în memorie și apoi îi este transferat controlul.

Dacă analizăm primul sector dintr-un disc, putem observa informații despre bootloader și prezența octeților `0x55AA` ca în Listing 9.1. Octeții `0x55AA` sunt ultimii 2 octeți și în cursul afișării apar referințe la `GRUB`.

```
1 student@uso:~$ sudo xxd -l 512 /dev/sda
2   [sudo] password for student:
3   [...]
4   00000170: 265a 7cbe 8e7d eb03 be9d 7de8 3400 bea2  &Z|..}....}.4...
5   00000180: 7de8 2e00 cd18 ebfe 4752 5542 2000 4765  }.......GRUB .Ge
```

```
00000190: 6f6d 0048 6172 6420 4469 736b 0052 6561  om.Hard Disk.Rea
000001a0: 6400 2045 7272 6f72 0d0a 00bb 0100 b40e  d. Error........
000001b0: cd10 ac3c 0075 f4c3 1b56 4fdf 0000 8020  ...<.u...VO.... 
000001c0: 2100 83fe ffff 0008 0000 00f0 ff01 0000  !...............
000001d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001f0: 0000 0000 0000 0000 0000 0000 0000 55aa  ..............U.
```

Listing 9.1: Sector bootabil

Spațiul disponibil în primul sector este limitat, doar `512` octeți. De aceea, această primă parte de bootloader poate face operații minime. Astfel că rolul său este să încarce partea principală din bootloader, numită și *2nd stage bootloader*, care realizează apoi încărcarea sistemului de operare, așa cum vom preciza în Secțiunea 9.4.

9.3.3 Încărcarea bootloaderului pentru UEFI

UEFI are un mod compatibil BIOS numit și CSM (*Compatibility Support Module*) care duce la bootarea într-un mod similar BIOS: încărcare *1st stage bootloader* din MBR, apoi încărcare *2nd stage bootloader*. Altfel, UEFI folosește modul nativ.

În cazul în care folosim UEFI în modul nativ, una dintre partiții este marcată specializat ca fiind partiția EFI. Această partiție se numește ESP (*EFI System Partition*). Această partiție este formatată cu sistem de fișiere FAT (*File Allocation Table*) și conține fișiere executabile într-un format specific, numit format EFI. Aceste fișiere executabile sunt numite OS loaders, sunt codul echivalent de bootloader pentru pornirea sistemului de operare. De exemplu, putem avea fișierul `/efi/boot/bootx64.efi` pentru booting pe un sistem PC.

În UEFI putem adăuga intrări pentru alte loadere. Pentru aceasta adăugăm căi către executabile EFI și acestea vor putea fi selectate cu ajutorul EFI. De exemplu, vom adăuga `/efi/fedora/grubx64.efi` pentru porni bootloaderul GRUB care va încărca imaginea de sistem de operare pentru o instalare Fedora Linux.

În Linux, pentru gestiunea intrărilor din partiția EFI se poate folosi utilitarul **`efibootmanager`**.

9.3.4 Denumiri discuri și partiții

În general, discurile și partițiile sunt, în Linux, intrări în directorul `/dev`. De exemplu `/dev/sda` este primul disc al sistemului, `/dev/sdb` al doilea disc, `/dev/sdc` al treilea disc. Apoi partițiile sunt numerotări ale acestor discuri: `/dev/sda1` este prima partiție a primului disc, `/dev/sdb3` este a treia partiție a celui de-al doilea disc.

Atunci când folosim GRUB ca bootloader, numele discurilor au alt format, pe echivalentul celor de mai jos:

- `(hd0)` este primul disc, echivalentul `/dev/sda`
- `(hd1)` este al doilea disc, echivalentul `/dev/sdb`

- `(hd0,1)` sau `hd0,msdos1` este prima partiție a primului disc, echivalentul `/dev/sda1`
- `(hd1,3)` sau `hd1,msdo3` este a treia partiție a celui de-al doilea disc, echivalentul `/dev/sdb3`

Întrucât ordinea discurilor poate diferi prin mutarea discurilor sau prin adăugarea unui disc nou, cele două moduri de denumire de mai sus sunt variabile. Pentru a menține o denumire fixă, se folosește un identificator unic al partiției un șir numit UUID (Universally Unique Identifier). Pe Linux putem obține acest identificator cu ajutorul comenzii **`blkid`** sau listând conținutul directorului `/dev/disk/by-uuid`, ca în Listing 9.2.

```
1  student@uso:~$ sudo blkid /dev/sda1
2  [sudo] password for student:
3  /dev/sda1: UUID="a14d0991-a3d8-48d6-ac8c-327d1a524501" TYPE="ext4"
   PARTUUID="df4f561b-01"
4  student@uso:~$ ls -l /dev/disk/by-uuid/
5  total 0
6  lrwxrwxrwx 1 root root 10 sep 15 00:43 a14d0991-a3d8-48d6-ac8c-327
   d1a524501 -> ../../sda1
7
8  student@uso:~$ cat /etc/fstab
9  # /etc/fstab: static file system information.
10 #
11 # Use 'blkid' to print the universally unique identifier for a
12 # device; this may be used with UUID= as a more robust way to name
   devices
13 # that works even if disks are added and removed. See fstab(5).
14 #
15 # <file system> <mount point>   <type> <options>      <dump>  <pass>
16 # / was on /dev/sda1 during installation
17 UUID=a14d0991-a3d8-48d6-ac8c-327d1a524501 /               ext4    errors=
   remount-ro 0       1
18 /swapfile                                 none            swap    sw
              0       0
```

Listing 9.2: Identificare discuri

În Listing 9.2 de mai sus, se observă că sistemul de fișiere aflat pe partiția `/dev/sda1` are ca identificator `a14d0991-a3d8-48d6-ac8c-327d1a524501`. Acest identificator este folosit și în fișierul `/etc/fstab`, fișier unde se configurează sistemele de fișiere ce vor fi montate, așa cum vom detalia în Secțiunea 10.2.1.

9.4 Bootloader

Atunci când firmware-ul de boot trebuie să treacă la următorul pas din procesul de boot, va încărca în memorie bootloaderul sau boot managerul și îi va transfera controlul procesului de boot. Bootloaderul este un program cu rolul de încărcare a sistemului de operare.

Înainte de încărcarea sistemului de operare, bootloaderul poate oferi utilizatorului opțiunea de a alege între diferite sisteme de operare și va oferi posibilitatea configurării opțiunilor de bootare a kernelului.

Când vorbim despre diferite sisteme de operare ne referim atât la sisteme de operare diferite, precum Linux sau Windows, aflate pe partiții distincte, cât și la versiuni diferite de imagini de kernel (fișiere ce conțin sistemul de operare) ale aceluiași sistem de operare, fișiere aflate pe aceeași partiție.

În cazul folosirii BIOS, încărcarea bootloaderului se face în doi pași:

1. întâi se încarcă prima fază din bootloader (numită *1st stage bootloader*), o secvență scurtă care se găsește în MBR (*Master Boot Record*)
2. ulterior, *1st stage bootloader* încarcă a doua fază din bootloader (*2nd stage bootloader*), o secvență mai detaliată care se poate afla într-un fișier pe disc, în metadatele unui sistem de fișiere sau în spațiul dintre MBR și prima partiție. Și care deține codul pentru a putea opta între diferite sisteme de operare și pentru a configura opțiunile de boot pentru kernel.

În cazul UEFI, partiția ESP (*EFI System Partition*) conține fișiere executabile `.efi`. Aceste fișiere pot conține orice funcționalitate, dar cel mai adesea conțin codul unui bootloader sau chiar nucleul sistemului de operare ce trebuie bootat. Neavând restricția de spațiu de 512 octeți din MBR, nu necesită înlănțuirea mai multor etape pentru încărcarea bootloaderului.

În general, un sistem de operare are un bootloader aferent. În cazul UEFI, la bootare, se poate selecta direct acel bootloader din partiția ESP. În cazul BIOS, însă, se încarcă mereu același 1st stage și 2nd stage bootloader. De aceea, dacă dorim să folosim două sisteme de operare diferite, bootloaderul încărcat va oferi opțiune să booteze sistemul de operare nativ, sau să încarce un alt bootloader. Opțiunea de încărcare a unui bootloader din alte bootloader se numește *chainloading*.

Concret, dacă un sistem are instalat Linux și Windows, sistemul va avea configurat bootloaderul specific Linux (GRUB) ca 1st stage și 2nd stage bootloader. Dacă utilizatorul va opta să booteze în Linux, atunci bootloaderul GRUB va încărca Linux; altfel, bootloaderul GRUB va încărca, prin chainloading, bootloaderul bootmgr care încarcă Windows.

9.4.1 GRUB

În Linux, bootloaderul folosit cel mai adesea pe PC este GRUB (*GRand Unified Bootloader*). Mai există și alte bootloadere, precum Das uBoot, folosit în sisteme încorporate.

Atunci când instalăm Linux, bootloaderul GRUB este instalat astfel: *1st stage bootloader* este în MBR, iar *1.5 stage bootloader* este pe disc în spațiul dintre MBR și prima partiție. *2nd stage bootloader* își începe execuția prin încărcarea fișierului *normal.mod*. Acesta, precum și alte module GRUB se află într-un subdirector al */boot/grub*, *i386-pc* pentru platforme BIOS, și *x86_64-efi* sau *i386-efi* pentru platforme UEFI. Din acest motiv, se recomandă rezervarea a cel puțin 2MiB intre acestea. Dacă sistemul are instalat și Windows, instalarea GRUB va detecta acest lucru și va configura chainloading pentru a permite bootarea Windows. Când folosim UEFI, GRUB va plasa un fișier imagine `.efi` în `/efi/boot/bootx64.efi`.

Instalarea corespunzătoare a GRUB va duce la afișarea unui ecran precum cel din Figura 9.5.

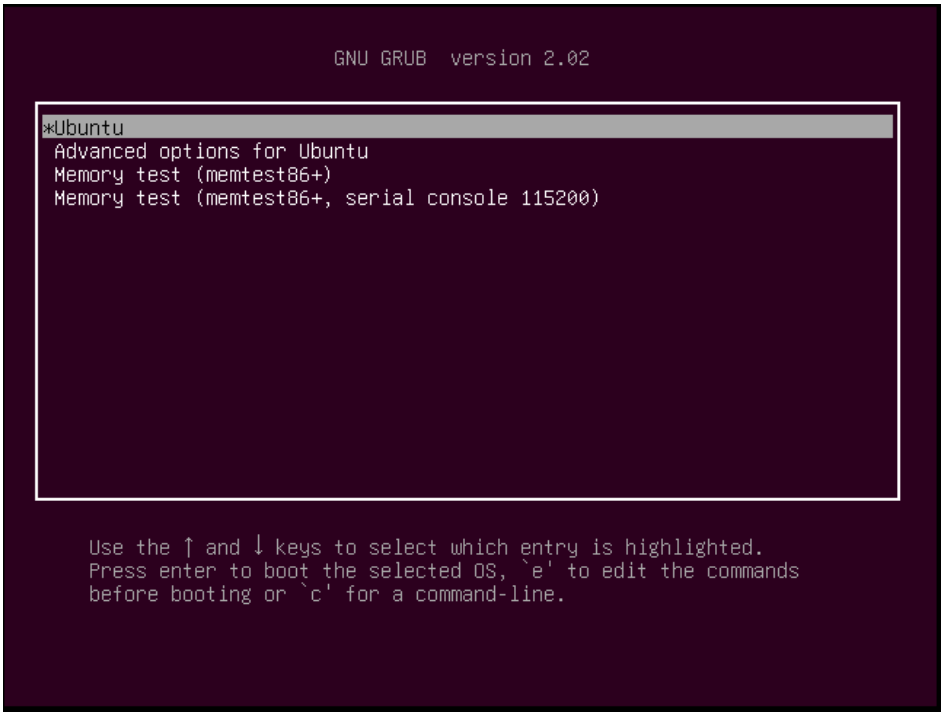


Figura 9.5: Ecran de boot GRUB

În acest ecran observăm opțiunile de boot pentru GRUB. În mod tipic, un utilizator selectează una dintre intrări și apoi apasă tasta `Enter` pentru a boota folosind acea opțiune. În Figura 9.5 avem doar opțiuni de boot pentru Linux (Ubuntu). Pe un sistem de tip dual-boot, ecranul de boot GRUB va afișa intrări și pentru Windows.

Un utilizator poate alege să editeze o intrare GRUB. În acest caz utilizatorul va apăsa tasta `e` și va accesa o nouă interfață ca în Figura 9.6. În interfața nouă putem modifica imaginea de kernel (de nucleu de sistem de operare) folosită, putem modifica partiția rădăcină folosită sau opțiuni de boot pentru kernel. O utilizare a editării opțiunilor de boot este recuperarea parolei contului `root` folosind opțiunea `init=/bin/bash` așa cum am precizat în Secțiunea 5.6. Detalii despre opțiunile de boot corespunzătoare nucleului sistemului de operare prezentăm în Secțiunea 9.5.

Opțiunile afișate de GRUB sunt încărcate din fișierul `/boot/grub/grub.cfg`, fișierul de configurare GRUB. Acest fișier nu se recomandă să fie editat manual; fișierul este generat de comanda **update-grub** din analiza automată a sistemului și a configurărilor. Comanda **update-grub** consultă partițiile sistemului, detectează sisteme de operare și imagini de kernel, consultă configurările din sistem și apoi generează fișierul `/boot/grub/grub.cfg`.

Configurările GRUB, care devin persistente după rularea comenzii **update-grub**, se realizează în fișierul `/etc/default/grub` și în fișierele din `/etc/grub.d/`. În fișierul `/etc/default/grub` sunt variabile GRUB care afectează comportamentul acestuia. Listing 9.3 din fișierul `/etc/default/grub`.

```
1 student@uso:~$ cat /etc/default/grub
2 # If you change this file, run 'update-grub' afterwards to update
3 # /boot/grub/grub.cfg.
```

Figura 9.6: Editarea unei intrări în GRUB

```
4  # For full documentation of the options in this file, see:
5  #  info -f grub -n 'Simple configuration'
6
7  GRUB_DEFAULT=0
8  GRUB_TIMEOUT_STYLE=hidden
9  GRUB_TIMEOUT=10
10 GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
11 GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
12 GRUB_CMDLINE_LINUX=""
```

Listing 9.3: Configurare GRUB: /etc/default/grub

Observăm la începutul fișierului comentariul care spune că aceste configurări sunt apoi marcate persistente în fișierul de configurare `/boot/grub/grub.cfg` după rularea comenzii **`update-grub`**. Opțiunile de mai sus reprezintă:

- `GRUB_DEFAULT`: indexul opțiunii implicite de boot a GRUB; în ecranul afișat către utilizator la bootare, dacă utilizatorul nu alege o opțiune, opțiunea implicită va fi aleasă
- `GRUB_TIMEOUT`: după cât timp de la afișarea ecranului GRUB, timp în care utilizatorul nu a atins nici o tastă, se bootează în opțiunea implicită; valoarea `-1` înseamnă așteptare infinită până la alegerea explicită a unei opțiuni
- `GRUB_TIMEOUT_STYLE`: precizează cum va fi afișat ecranul de boot până la expirarea timpului de afișare; valoarea `hidden` înseamnă că nu se afișează nici un ecran, și după 10 secunde se bootează opțiunea implicită; în acel interval de 10 secunde utilizatorul poate apăsa tasta `Esc` pentru a cauza afișarea ecranului de boot
- `GRUB_TIMEOUT_LINUX_DEFAULT`: adaugă opțiunile respective nucleului când bootează

În directorul `/etc/grub.d/` sunt fișiere de configurare pentru opțiuni GRUB. Multe fișiere sunt predefinite pentru configurări dedicate. Dacă dorim să facem configurări personale vom completa fișierele `/etc/grub.d/40_custom` sau `/etc/grub.d/41_custom`. Aceste fișiere sunt citite literal și adăugate la sfârșitul fișierului `/boot/grub/grub.cfg`.

Pentru detalii despre configurarea GRUB consultați documentația, fie online[1], fie accesând pagina **info**: **`info grub "Configuration"`**.

9.5 Pornirea kernelului și drivere

Rolul bootloaderului este de a permite utilizatorului să selecteze un sistem de operare în care să booteze, să configureze opțiunile acelui sistem de operare și apoi să booteze în sistemul de operare.

Bootloaderul va încărca un fișier numit **imagine de kernel**, care conține codul și datele sistemului de operare, și apoi va transfera controlul către acesta. Imaginea de kernel (sau imaginea de nucleu de sisteme de operare) este practic un executabil care conține implementarea nucleului (kernelului), componenta critică în funcționarea sistemului.

Nucleul realizează configurarea componentelor hardware ale sistemului, încarcă driverele de dispozitiv (*device drivere*), montează sistemul de fișiere rădăcină și apoi pornește procesul `init`, primul proces al sistemului, care, la rândul său, va porni alte procese. Un driver este un fișier ce conține codul pentru utilizarea și configurarea unui dispozitiv hardware. După investigarea hardware-ului sistemului, nucleul sistemului de operare va anunța un program dedicat de prezența noului dispozitiv, iar acest program va asigura integrarea dispozitivului în sistem.

9.5.1 Kernelul și driverele în Linux

Într-o configurație tipică, fișierul reprezentând imaginea de kernel în Linux se găsește în `/boot/vmlinuz-<versiune>`, unde `<versiune>` este versiunea de sistem de operare. Un exemplu este `/boot/vmlinuz-4.15.0-29-generic`. Acest fișier este configurat să fie încărcat de bootloader. Este încărcat în memorie și apoi executat, transferându-se controlul sistemului de operare. O investigație în linia de comandă ne oferă informații despre această imagine:

```
1 student@uso:~$ file /boot/vmlinuz-4.15.0-29-generic
2 /boot/vmlinuz-4.15.0-29-generic: Linux kernel x86 boot executable bzImage
  , version 4.15.0-29-generic (buildd@lgw01-amd64-057) #31-Ubuntu SMP Tue
  Jul 17 15:39:52 UTC 2018, RO-rootFS, swap_dev 0x7, Normal VGA
```

În cazul de față avem o imagine de kernel comprimată, de unde și precizarea `executable bzImage`(big zip Image); tot ca referință la faptul că este comprimată este și folosirea literei `z` în numele fișierului `vmlinuz-4.15.0-29-generic`. Imaginea este de tip self-extracting: după ce este încărcată în memorie de bootloader, se transferă controlul imaginii care conține o secvență inițială de cod ce decomprimă restul fișierului și apoi îl execută.

[1] https://www.gnu.org/software/grub/manual/grub/

Driverele sunt localizate, în Linux, în directorul `/lib/modules/<version>/kernel` unde `<version>` este versiunea de sistem de operare. Driverele corespunzătoare imaginii `/boot/vmlinuz-4.15.0-29-generic` se găsesc în directorul `/lib/modules/4.15.0-29-generic/kernel/`. De fapt, în acel director se găsesc modulele de kernel, componente externe nucleului care pot fi încărcate în cadrul acestuia pentru a-i extinde funcționalitatea. Numim modul de kernel orice program încărcabil în nucleu care extinde funcționalitatea acestuia. Numim drivere acele module de kernel al căror rol este de a permite utilizarea și configurarea componentelor hardware. În Linux, un modul de kernel / driver este un fișier cu extensia `.ko`:

```
student@uso:~$ file /lib/modules/4.15.0-29-generic/kernel/drivers/net/ethernet/intel/e1000/e1000.ko
/lib/modules/4.15.0-29-generic/kernel/drivers/net/ethernet/intel/e1000/e1000.ko: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), BuildID[sha1]=c2bcba22a6d98cf9d4c3735248bc48674ead034b, not stripped
```

Observăm că driverul `e1000`, pentru plăci de rețea Intel, se găsește în fișierul `e1000.ko`, fișier de tipul ELF (*Executable and Linking Format*), formatul clasic de executabil pe Linux.

Driverele sunt încărcate fie de kernel la pornirea sistemului, după identificarea componentelor hardware, fie în timp real, de programe dedicate sau de utilizator. În lucrul cu module sau drivere, utilizatorul poate realiza următoarele operații:

- listarea modulelor încărcate în nucleu, folosind utilitarul **`lsmod`**
- încărcarea unui modul de kernel nou, folosind utilitarul **`insmod`** sau utilitarul **`modprobe`**
- descărcarea unui modul de kernel existent, folosind utilitarul **`rmmod`** sau comanda **`modprobe -r`**
- afișarea de informații despre un modul de kernel folosind utilitarul **`modinfo`**

Listing 9.4 jos avem exemple de comenzi de lucru cu module de kernel:

```
student@uso:~$ sudo lsmod
Module                  Size  Used by
btrfs                1150976  0
zstd_compress         163840  1 btrfs
xor                    24576  1 btrfs
[...]
minix                  32768  0
ntfs                  102400  0
[...]

student@uso:~$ sudo modinfo e1000
filename:       /lib/modules/4.15.0-118-generic/kernel/drivers/net/ethernet/intel/e1000/e1000.ko
version:        7.3.21-k8-NAPI
license:        GPL
description:    Intel(R) PRO/1000 Network Driver
author:         Intel Corporation, <linux.nics@intel.com>
[...]

student@uso:~$ sudo rmmod minix
```

```
20
21 student@uso:~$ sudo modprobe minix
22
23 student@uso:~$ sudo modprobe -r ntfs
24
25 student@uso:~$ sudo insmod /lib/modules/4.15.0-118-generic/kernel/fs/ntfs
   /ntfs.ko
```

Listing 9.4: Lucrul cu module de kernel în Linux

Comanda **insmod** primește ca argumente calea completă către fișierul ce conține modulul de kernel în vreme ce comanda modprobe primește numele modulului și apoi îl caută în directorul `/lib/modules/<version>`.

Dacă dorim încărcate anumite versiuni de module de kernel la pornirea sistemului, sau dacă dorim să împiedicăm încărcarea anumitor module (*blacklisting*) atunci vom realiza configurări în fișierul `/etc/modules` sau în directorul `/etc/modprobe.d/`.

În directorul `/boot/` pe lângă imagini de kernel, apar fișiere numite `initrd.img-<version>`. De exemplu fișierul `/boot/initrd.img-4.15.0-29-generic`. Acest fișier este numit *ram disk image* și conține o serie de drivere necesare pentru pornirea nucleului. Atunci când nucleul pornește trebuie să încarce informații de pe disc. Pentru a le accesa are însă nevoie de driverul de disc și de driverul de sistem de fișiere. O soluție ar fi să fie adăugate în imaginea de kernel; dar, dat fiind numărul mare de discuri și de sisteme de fișiere, ar rezulta o imagine de kernel mult prea mare și cu multe drivere inutile pe sistemul curent. Soluția este adăugarea acestor drivere într-o imagine separată: *ram disk image*.

Imaginea de ram disk este încărcată de bootloader, așa cum am văzut în ecranul de configurare a opțiunii GRUB din Figura 9.6. Când nucleul bootează, are referință la această imagine în memorie și poate folosi driverul de disc și de sistem de fișiere corespunzător pentru a putea folosi discul.

Imaginea de ram disk este generată la instalarea unei versiuni noi de kernel și poate fi generată la nevoie cu ajutorul comenzii **mkinitramfs**. Nu vom detalia aici această comandă, puteți urmări pagina sa de manual sau exemple pe Internet.

După cum spuneam mai sus, în ecranul de bootloader sau în cadrul opțiunii `GRUB_CMDLINE_DEFAULT` putem preciza opțiuni pentru kernel care afectează modul în care acesta bootează și rulează. Comanda echivalentă folosită pentru bootarea kernelului, împreună cu opțiunile aferente, se găsește în fișierul `/proc/cmdline`:

```
1 student@uso:~$ cat /proc/cmdline
2 BOOT_IMAGE=/boot/vmlinuz-4.15.0-34-generic root=UUID=a14d0991-a3d8-48d6-
  ac8c-327d1a524501 ro quiet splash
```

Dintre opțiunile de boot ale nucleului, precizăm:

- opțiunile `debug`, `verbose`, `quiet` care configurează nivelul de detaliu al mesajelor de boot afișate la pornirea kernelului
- opțiunea `splash` activează splashscreenul la pornirea sistemului, în loc să afișeze mesajele de boot
- opțiunea `noacpi` dezactivează ACPI (*Advanced Configuration and Power Interface*) o componentă PC care uneori este cauzatoare de probleme

O listă completă a acestor opțiuni găsiți în documentația nucleului[1].

După ce sistemul de operare a pornit, putem afla informații despre acesta. Pe lângă aflarea liniei de comandă echivalente din fișierul `/proc/cmdline`, mai putem afla versiunea nucleului care rulează folosind **`uname`**:

```
1 student@uso:~$ uname -a
2 Linux uso 4.15.0-34-generic #37-Ubuntu SMP Mon Aug 27 15:21:48 UTC 2018
  x86_64 x86_64 x86_64 GNU/Linux
```

De asemenea, pentru depanare sau informare, nucleul transmite informații care se jurnalizează. Pentru accesarea jurnalului folosim comanda **`dmesg`**, ca mai jos:

```
1 student@uso:~$ dmesg
2 [    0.000000] Linux version 4.15.0-34-generic (buildd@lgw01-amd64-047) (
  gcc version 7.3.0 (Ubuntu 7.3.0-16ubuntu3)) #37-Ubuntu SMP Mon Aug 27
  15:21:48 UTC 2018 (Ubuntu 4.15.0-34.37-generic 4.15.18)
3 [    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-4.15.0-34-generic
  root=UUID=a14d0991-a3d8-48d6-ac8c-327d1a524501 ro quiet splash
4 [    0.000000] KERNEL supported cpus:
5 [    0.000000]   Intel GenuineIntel
6 [    0.000000]   AMD AuthenticAMD
7 [...]
```

9.6 Pornirea init și a serviciilor de startup în Linux

După ce nucleul și driverele sunt încărcate și sistemul de operare este inițializat, trebuie pornite primele procese. Pentru început se pornește primul proces, numit generic **init**, care apoi pornește alte procese. Așa cum am precizat în Secțiunea 4.3.1, procesul init este în vârful ierarhiei proceselor având ca roluri crearea primelor procese din sistemul de operare și adoptarea proceselor orfane.

În mod tradițional, init era procesul pornit din executabilul `/sbin/init` și se numea System V init, după numele unei variante importante de UNIX (UNIX System V). În mod curent implementarea cea mai răspândită de init este **systemd**, cu executabilul aflat în `/lib/systemd/systemd`:

```
1 student@uso:~$ ls -l /sbin/init
2 lrwxrwxrwx 1 root root 20 aug  6 17:34 /sbin/init -> /lib/systemd/systemd
```

Observăm că `/sbin/init` este, pe sistemele Linux moderne, un fișier de tipul legătură simbolică la fișierul `/lib/systemd/systemd`. Astfel că, pe sistemele moderne, configurarea init echivalează configurării systemd.

Procesul principal systemd rulează ca procesul cu PID-ul 1. Există posibilitatea creării unui proces per utilizator prin transmiterea opțiunii `--user` comenzii `systemd`. Acest lucru este util atunci când dorim să creăm și să gestionăm servicii la nivelul utilizatorului, la la nivelul sistemului.

Serviciile sunt procese dedicate, de obicei procese daemon (revedeți Secțiunea 4.6.3), care oferă funcționalități suplimentare sistemului sau care gestionează buna funcționare a acestuia. Serviciile sunt în general pornite și gestionate de systemd.

[1] https://www.kernel.org/doc/html/v5.9/admin-guide/kernel-parameters.html

În mod tradițional, în implementarea veche (System V) de init, serviciile se configurau și rulau cu ajutorul scripturilor de inițializare din directorul `/etc/init.d/`. Pentru a menține compatibilitatea, systemd folosește în continuare acele scripturi de inițializare, pe care le le putem localiza pe sistemele moderne:

```
student@uso:~$ ls /etc/init.d/
acpid      apport           cron          dns-clean      [...]
alsa-utils avahi-daemon     cups          gdm3           [...]
anacron    bluetooth        cups-browsed  grub-common    [...]
apparmor   console-setup.sh dbus          hddtemp        [...]
```

În systemd, serviciile, împreună cu alte componente, sunt numite `units`. Pentru a vizualiza toate unitățile systemd, folosim comanda **`systemd --list-units`**:

```
student@uso:~$ systemctl list-units | head -100
UNIT                    LOAD   ACTIVE SUB       DESCRIPTION
proc-sys-fs-[...]       loaded active waiting   Arbitrary Executable
File Formats File System Automount Point
sys-devices-[...]       loaded active plugged   VBOX_CD-ROM
sys-devices-[...]       loaded active plugged   82540EM Gigabit Ethernet
 Controller (PRO/1000 MT Desktop Adapter)
sys-devices-[...]       loaded active plugged   82801AA AC'97 Audio
Controller
sys-devices-[...]       loaded active plugged   82540EM Gigabit Ethernet
 Controller (PRO/1000 MT Desktop Adapter)
sys-devices-[...]       loaded active plugged   VBOX_HARDDISK 1
sys-devices-[...]       loaded active plugged   VBOX_HARDDISK
sys-devices-[...]       loaded active plugged   VBOX_HARDDISK 1
```

Dacă dorim să afișăm informații, oprim, porni sau reporni servicii, putem folosi fie interfața clasică `/etc/init.d/`, fie comanda **`service`**, fie comanda **`systemctl`**. Mai jos sunt comenzile în toate formele pentru afișarea stării serviciului SSH (pentru conexiune la distanță) și pentru pornirea serviciului:

```
student@uso:~$ /etc/init.d/ssh status
student@uso:~$ service ssh status
student@uso:~$ systemctl status ssh.service
student@uso:~$ sudo /etc/init.d/ssh start
student@uso:~$ sudo service ssh start
student@uso:~$ sudo systemctl start ssh.service
```

Dacă dorim să oprim sau să repornim serviciul, folosim parametrul `stop`, respectiv `restart`, în loc de `start`.

Dacă dorim adăugarea de servicii în sistem putem să le adăugăm în configurarea **`systemd`** sau putem folosi **`supervisor`**. Vom detalia în Secțiunea 13.5.1.

9.6.1 Pornirea terminalelor de login

Ulterior pornirii serviciilor inițiale (*startup processes*), init/systemd trebuie să configureze terminalele virtuale, unde utilizatorul se va putea autentifica. Aceste terminale pot fi inițializate în mod text sau în mod grafic. De obicei prin combinațiile de taste `Ctrl+Alt+F1`, `Ctrl+Alt+F2`, ..., `Ctrl+Alt+F6`. Pe un sistem fără interfață grafică se vede implicit primul terminal putându-se accesa celelalte terminale folosind combinațiile de taste `Ctrl+Alt+F2`, ..., `Ctrl+Alt+F6`. Terminalul virtual curent

poate fi schimbat și cu ajutorul comenzii **`chvt n`**, unde `n` ia valori de la 1 la numărul de terminale virtuale alocate de kernel.

Mediul grafic, în cazul în care este prezent, poate porni în locul consolei text. Acesta va fi accesibil de obicei prin `Ctrl+Alt+F1` sau `Ctrl+Alt+F7`, dar orice configurație este posibilă.

Terminalele virtuale sunt pornite și gestionate de init/systemd prin următorul flux, prezentat în Figura 9.7.

1. systemd pornește procese numite **`getty`** care reprezintă interfața de comunicare cu terminalul virtual.
2. **`getty`** afișează un prompt de utilizator unde se introduce numele de utilizator și parola.
3. **`getty`** pornește procesul **`login`** care citește parola utilizatorului și o verifică în fișierul `/etc/shadow`
4. dacă parola este corectă, procesul **`login`** pornește shellul descris ca ultima coloana din linia corespunzătoare utilizatorului în fișierul `/etc/passwd`

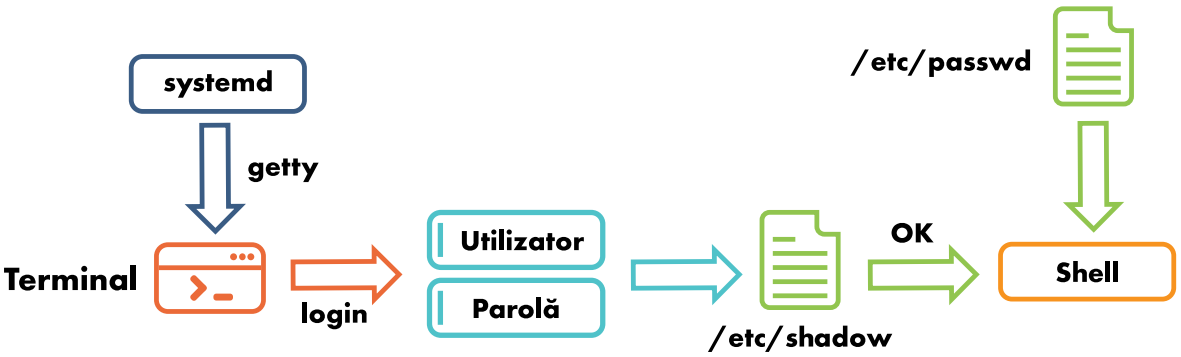


Figura 9.7: Pornirea terminalelor virtuale și a shellurilor

Putem vizualiza unitățile gestionate de systemd pentru terminalele virtuale folosind comanda de mai jos:

```
student@uso:~$ systemctl list-units | grep getty
getty@tty2.service              loaded active running   Getty on tty2
getty@tty3.service              loaded active running   Getty on tty3
getty@tty4.service              loaded active running   Getty on tty4
getty@tty5.service              loaded active running   Getty on tty5
getty@tty6.service              loaded active running   Getty on tty6
system-getty.slice              loaded active active    system-getty.slice
getty.target                    loaded active active    Login Prompts
```

Observăm în cadrul comenzii că sunt cinci terminale virtuale, de la `tty2` până la `tty6`. Putem observa și procesele aferente celor cinci terminale virtuale, gestionate de procesul **`agetty`**, o implementare de **`getty`**.

```
student@uso:~$ ps -f -C agetty
UID        PID  PPID  C STIME TTY          TIME CMD
root      4453     1  0 14:27 tty2     00:00:00 /sbin/agetty [...]
root      4454     1  0 14:27 tty3     00:00:00 /sbin/agetty [...]
root      4455     1  0 14:27 tty4     00:00:00 /sbin/agetty [...]
root      4456     1  0 14:27 tty5     00:00:00 /sbin/agetty [...]
root      4457     1  0 14:27 tty6     00:00:00 /sbin/agetty [...]
```

După o autentificare corectă la un terminal virtual este pornit shellul. Shellul poate fi pornit și dintr-un emulator de terminal din interfața grafică și dintr-un pseudo-terminal creat printr-o conexiune SSH. În toate situațiile shellul folosește niște fișiere de configurare specifice care personalizează experiența utilizatorului în linia de comandă, așa cum am prezentat în Secțiunea 7.3.1.

9.7 Anexă: Resetarea parolei administrative

Este posibil ca pentru un sistem să pierdem parola administrativă, a contului root sau a contului Administrator și să fie nevoie de resetarea acesteia. În Secțiunea 5.6 și Secțiunea 5.7 am precizat modurile uzuale de resetare a parolei administrative și în Linux și în Windows. În Linux folosim opțiunea `init=/bin/bash` la bootarea sistemului de operare, editând intrarea în bootloader, sau folosim unui USB stick bootabil sau CD-ROM. În Windows folosim Ultimate Boot CD.

În Linux este posibil să configurăm parolă de acces la bootloader (GRUB) și atunci opțiunea cu editarea intrării în bootloader nu funcționează. Rămâne însă o opțiune folosirea unui USB stick bootabil care nu necesită editarea opțiunilor.

Chiar și așa, este posibil să fie dezactivat din firmware-ul de boot folosirea USB sau CD-ROM pentru bootare. Sau să fie configurată parolă de acces la firmware care nu permite editarea opțiunilor și, deci, adăugarea opțiunii de boot USB sau CD-ROM. Pentru aceasta avem soluția desfacerii unității fizice a sistemului și extragerea bateriei CMOS care va duce la resetarea configurării firmware-ului de boot la o valoare implicită, fără parolă. Ulterior vom activa bootarea de pe USB sau CD-ROM și vom reseta parola administrativă folosind un USB stick sau CD-ROM bootabil. Evident, resetarea parolei firmware-ului de boot prin extragerea bateriei CMOS necesită acces fizic la sistem.

O formă mai directă de a trece de parola de firmware de boot sau cea de bootloader este detașarea discului din unitate și atașarea acestuia la o altă unitate. Apoi se bootează acel sistem cu un stick USB sau CD-ROM bootabil și se resetează parola. Pentru a preveni acest lucru o soluție este criptarea discului, lucru ce face imposibilă orice modificare fără accesul la parolă.

9.8 Anexă: Crearea unui stick USB bootabil cu Linux

Pentru a instala un sistem de operare Linux pe un PC, avem nevoie de un mediu de boot. În mod obișnuit, acest mediu va fi un stick USB bootabil. Pentru început acest stick trebuie să fie partiționat și apoi marcată partiția principală ca fiind bootabilă. Pașii de pregătire ai mediului de boot sunt:

1. Ștergerea vechilor componentele de pe stickul USB. Presupunând că stickul USB este accesibil prin intrarea `/dev/sdb`, rulăm comanda

```
1 wipefs -a /dev/sdb
```

   `wipefs` va detecta și va șterge toate semnăturile de sisteme de fișiere și tabelele de partiții prezente pe `/dev/sdb`.

2. Crearea tabelei de partiții și apoi unei partiții FAT32 și marcarea acesteia ca fiind bootabilă. Pentru aceasta folosim utilitarul **gparted** (cu interfața grafică), accesând opțiunile:
   (a) `Device` → `Create Partition Table`
   (b) click dreapta pe spațiu → `New`
   (c) click dreapta pe partiție → `Manage Flags` → `boot`

În acest moment avem un stick USB cu o partiție FAT32 bootabilă.

Pentru scrierea de informații pe stickul USB, descărcăm un fișier `.iso` cu distribuția Linux dorită. Apoi folosim utilitarul **unetbootin** (pe orice platformă)[1], cu interfață grafică. Înainte de folosirea sa, trebuie să fie montat stickul USB, fie automat fie printr-o comandă de forma:

```
student@uso:~$ sudo mkdir /mnt/sdb1
student@uso:~$ sudo mount /dev/sdb1 /mnt/sdb1
```

În interfața **unetbootin** selectăm fișierul `.iso` pe care dorim să îl scriem și destinația unde este montat stickul, adică `/mnt/sdb1` în cazul nostru.

O alternativă, care permite crearea unui stick USB cu Linux folosind Windows, este folosirea utilitarului Rufus[2] urmărind indicațiile de aici[3].

Având în vedere că majoritatea distribuțiilor de Linux oferă imagini ISO hibride, care pot fi scrise și pe mediu optic (CD-ROM, DVD-ROM) și pe dispoztive de stocare în masă (stick USB, HDD, SSD), se mai poate folosi comanda:

```
        dd if=my.iso of=/dev/sdb bs=1M
```

9.9 Anexă: Crearea unui stick USB bootabil cu Windows

Pentru crearea unui stick bootabil cu Windows, pregătim mediul de instalare (stickul USB) în mod similar cu instalarea Linux, cu o diferență: partiția trebuie să fie formatată NTFS.

Apoi se urmează pașii de mai jos, detaliați și aici[4]:

1. Se montează fișierul `.iso` cu imaginea de Windows.

```
student@uso:~$ mkdir iso-mnt/
student@uso:~$ sudo mount -o loop windows.iso iso-mnt/
```

2. Se montează stickul USB:

```
student@uso:~$ sudo mkdir /mnt/sdb1
student@uso:~$ sudo mount /dev/sdb1 /mnt/sdb1
```

[1] https://unetbootin.github.io
[2] https://rufus.akeo.ie
[3] https://tutorials.ubuntu.com/tutorial/tutorial-create-a-usb-stick-on-windows
[4] https://askubuntu.com/a/487970

3. Se copiază tot conținutul din imaginea de Windows pe stickul USB:

```
student@uso:~$ sudo cp -r iso-mnt/* /mnt/sdb1/
```

4. Se instalează GRUB pe stickul USB:

```
sudo grub-install --target=i386-pc --boot-directory="/mnt/sdb1/boot"
 /dev/sdb
```

5. Se generează fișierul `/mnt/sdb1/boot/grub/grub.cfg` cu următorul conținut:

```
echo "If you see this, you have successfully booted from USB :)"
insmod ntfs
insmod search_fs_uuid
search --no-floppy --fs-uuid <UUID> --set root
ntldr /bootmgr
boot
```

   iar în loc de `<UUID>` scriem identificatorul pe care îl obținem folosind comanda:

```
student@uso:~$ sudo blkid /dev/sdb1
```

6. Demontăm stickul USB și fișierul `.iso`:

```
student@uso:~$ sudo umount /mnt/sdb1
student@uso:~$ sudo umount iso-mnt/
```

Acum putem folosi stickul pentru a boota Windows.

9.10 Anexă: Adăugarea unei intrări GRUB pentru UDPCast

Pentru a demonstra o configurare particulară pentru GRUB, vom descrie instalarea UDPcast pe disc. Va putea fi bootat de pe disc prin GRUB. UDPCast[1] este o imagine bootabilă de CD care permite transferul prin rețea al unui disc de la o sursă către mai multe destinații, asigurând astfel o clonare a discului.

Pentru a adăuga o intrare în GRUB pentru UDPCast și pentru a boota astfel în GRUB fără a fi nevoie de un stick USB sau un CD-ROM bootabil, urmăm pașii:

1. Obținem fișierul `.iso` UDPCast:

```
student@uso:~$ wget http://www.udpcast.linux.lu/20120424/udpcd.iso
```

2. Montăm fișierul `.iso`:

```
student@uso:~$ mkdir iso-mnt/
student@uso:~$ sudo mount -o loop udpcd.iso iso-mnt/
student@uso:~$ ls iso-mnt/
ISOLINUX.BIN  ISOLINUX.CFG boot.catalog  initrd  linux pxelinux.0
```

   Din conținutul fișierului `.iso` ne interesează fișierele `initrd` și `linux`, adică imaginea de ram disk și imaginea de kernel.

[1] http://www.udpcast.linux.lu

3. Copiem imaginea de ram disk si cea de kernel într-un director din `/boot`:

```
student@uso:~$ sudo mkdir /boot/udpcast
student@uso:~$ sudo cp iso-mnt/{initrd,linux} /boot/udpcast
student@uso:~$ ls /boot/udpcast
initrd linux
```

4. Adăugăm configurarea corespunzătoare în fișierul `/etc/grub.d/40_custom`

```
student@uso:~$ sudo blkid /dev/sda1
/dev/sda1: UUID="a14d0991-a3d8-48d6-ac8c-327d1a524501" TYPE="ext4"
PARTUUID="df4f561b-01"
student@uso:~$ sudo vi /etc/grub.d/40_custom
student@uso:~$ cat /etc/grub.d/40_custom
#!/bin/sh
exec tail -n +3 $0
# This file provides an easy way to add custom menu entries.  Simply
 type the
# menu entries you want to add after this comment.  Be careful not
to change
# the 'exec tail' line above.

menuentry 'UDPCast' {
        insmod part_msdos
        insmod ext2
        set root='(hd0,msdos1)'
        search --no-floppy --fs-uuid --set=root a14d0991-a3d8-48d6-
ac8c-327d1a524501
        linux   /boot/udpcast/linux root=01:00
        initrd  /boot/udpcast/initrd
}
```

   În configurarea GRUB am folosit UUID-ul partiției `/dev/sda1`, partiția unde se găsește directorul `/boot` (`a14d0991-a3d8-48d6-ac8c-327d1a524501`) pe care l-am descoperit cu ajutorul comenzii blkid. Am folosit construcția `(hd0,msdos1)` pentru a identifica prima partiție din primul hard disk, adică `/dev/sda1`.

5. Actualizăm configurația GRUB:

```
student@uso:~$ sudo update-grub
Generating grub configuration file ...
[...]
done
student@uso:~$ grep UDPCast /boot/grub/grub.cfg
menuentry 'UDPCast' {
```

   În ultima comandă am verificat că intrarea a fost adăugată în fișierul de configurare GRUB: `/boot/grub/grub.cfg`.

6. Demontăm fișierul `.iso`:

```
student@uso:~$ sudo umount iso-mnt
student@uso:~$ sudo rmdir iso-mnt
```

7. Repornim sistemul și apoi vom avea în GRUB intrarea aferentă UDPCast.

9.11 Anexă: Booting în Windows

În Windows 7, 8 și 10, bootarea se bazează pe trei componente:

- Windows Boot Manager (`Bootmgr.exe`)
- bootloaderul sistemului de operare (`Windload.exe`)
- loaderul unui sistem suspendat (`Winresume.exe`)

A treia componentă este folosită doar pentru încărcarea unui sistem de operare suspendat/hibernat.

Windows Boot Manager este o componentă generică ce permite selectarea între mai multe instanțe de sistem de operare Windows de pe un sistem dat. Windows Boot Manager prezintă utilizatorului opțiunile de sistem de operare pe care să booteze; după ce utilizatorul alege opțiunea respectivă, Windows Boot Manager încarcă bootloaderul aferent și acesta, ulterior, va încărca sistemul de operare.

Opțiunile de boot, însemnând ce sisteme de operare sunt disponibile și ce opțiuni de boot sunt folosite pentru un sistem de operare, sunt reținute într-o zonă numită BCD (*Boot Configuration Data*), zonă ce poate fi editată folosind **`BCDEdit.exe`**, un utilitar inclus în Windows.

Detalii despre procesul de boot în Windows găsiți în documentație[1].

9.12 Concluzii

Un sistem de calcul parcurge mai mulți pași din momentul pornirii acestuia (apăsării pe butonul de *power-on*) până la pornirea primelor procese interactive, în general shellul.

Un utilizator are nevoie să știe procesul de boot pentru a putea depana probleme ce pot apărea, pentru a configura componentele pornite și opțiunile acestora.

În general un sistem de calcul dispune de un firmware de boot, încărcat în memorie și rulat imediat după pornirea sistemului. Acesta încarcă și transferă controlul unui bootloader care încarcă și transferă controlul sistemului de operare. Sistemul de operare configurează hardware-ul și pornește primele procese din sistem. Procesul de boot se consideră încheiat când utilizatorul are acces la sistem, prin intermediul unui shell (CLI sau GUI).

Conceptual pașii sunt similari, dar la nivel practic apar diferențe între tipurile de sisteme de calcul și sisteme de operare.

[1] https://docs.microsoft.com/en-us/windows-hardware/drivers/devtest/introduction-to-boot-options

Capitolul 10

Administrarea spațiului de stocare

În cadrul unui sistem de calcul, unitatea de procesare (procesorul, CPU) este elementul central, cel care realizează operațiile necesare, respectiv cerute, de către utilizator. Pentru a putea realiza aceste operații, unitatea de procesare are nevoie de un mecanism prin care să fie aduse datele la aceasta cât mai rapid și de un mecanism prin care să furnizăm rezultatele înapoi utilizatorului. Cel mai apropiat mecanism de intrare/ieșire a datelor necesare/rezultate este memoria (de tip RAM și de tip cache). Principalul dezavantaj al memoriei o reprezintă volatilitatea: la pierderea alimentării cu energie electrică, datele stocate în memorie se pierd.

Pentru a preîntâmpina această problemă, a fost introdus un nou nivel de intrare/ieșire a datelor în care capacitatea de înmagazinare a datelor este net superioară, iar la întreruperea alimentării cu energie electrică, datele se păstrează. Acest nou nivel de intrare/ieșire poartă denumirea, în general, de dispozitiv de stocare.

Dispozitivele de stocare au ca principal dezavantaj viteza de operare, de aceea nu le folosim direct în relația cu unitatea de procesare (datele trec prin memorie înainte de a ajunge la procesor). Un alt dezavantaj al dispozitivelor de stocare îl reprezintă modul de organizare a datelor pe acesta: capacitatea este net superioară comparativ cu a unei memorii (gigabytes vs. terabytes/petabytes) iar o adresare liniară, ca în cazul memoriei, nu ar fi posibilă. În cadrul acestui capitol vom studia tehnici de organizare a datelor pe dispozitivele de stocare (vezi Figura 10.1).

Așadar, stocarea este și ea un element central în operațiile cu sistemul de calcul. Orice informație procesată ar trebui stocată persistent pe un dispozitiv de stocare, după cum a fost prezentat și în Capitolul 8.

Dispozitivele hardware de stocare sunt alcătuite din două componente: controller și disc. Controllerul este asemenea unei unități de procesare din cadrul unui sistem de calcul, dar specializat în controlul și managementul unităților de stocare efective (discului). Controllerele au rolul de a centraliza logica de acces la unitățile de stocare și de a nu duplica logica în cadrul fiecărui disc. De asemenea, deoarece discurile sunt cu câteva ordine de mărime mai lente comparativ cu unitatea de procesare, controllerul acționează ca un element intermediar de control: procesorul dă comanda controllerului, acesta execută toate operațiile cerute și când sunt finalizate, anunță unitatea principală de execuție (procesorul). Astfel procesorul nu așteaptă după unitățile de stocare, ci execută alte operații relevante în tot acest timp.

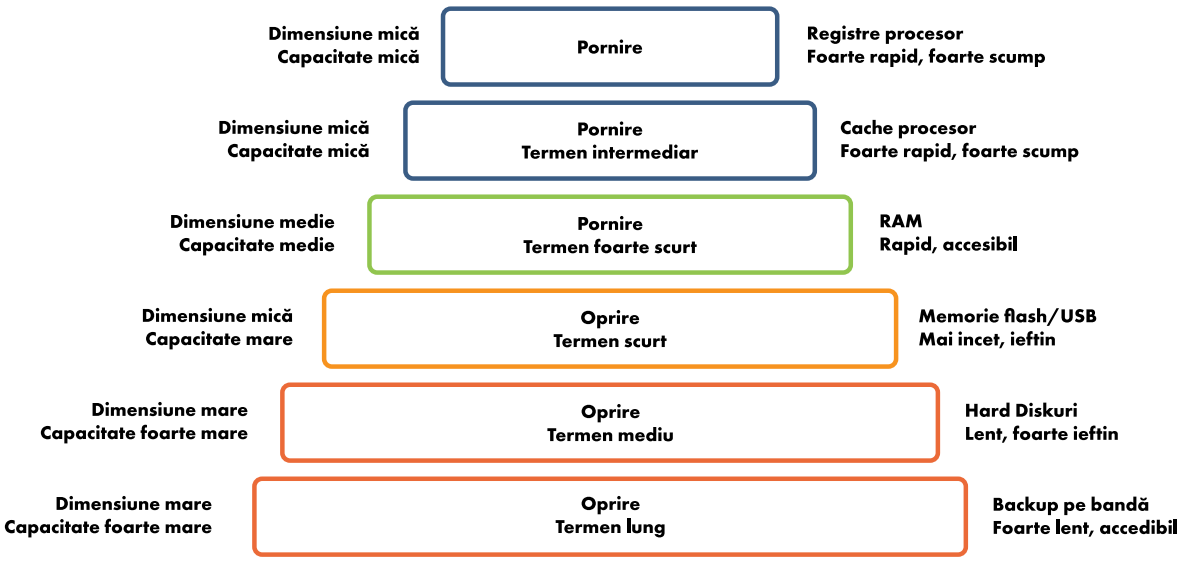


Figura 10.1: Ierarhia de memorie în cadrul unui sistem de calcul

În Figura 10.2 este prezentat un exemplu de transfer de date din Internet în timp ce utilizatorul folosește aplicația *Calculator*. Discurile sunt cele ce stochează efectiv informația. Luăm ca exemplu aplicațiile Google Drive și Dropbox: acestea oferă servicii de stocare a informațiilor pentru utilizatori. Pentru a implementa facilitățile de stocare și partajare a datelor, aceste aplicații folosesc sisteme de stocare. În secțiunea următoare vom descrie și vom realiza o clasificare a discurilor.

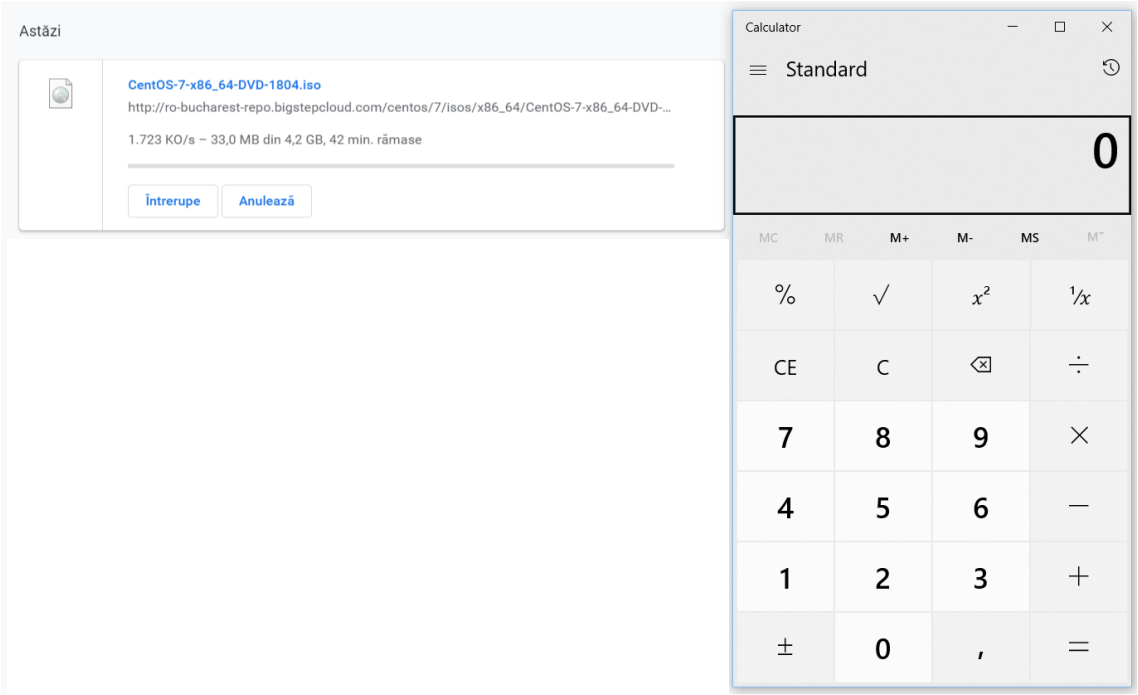


Figura 10.2: Executarea concomitentă a transferului de date și a unei aplicații

10.1 Tipuri de discuri

Subsistemul de stocare din cadrul unui sistem de calcul este format dintr-un controller care furnizează partea de logică a transferului de date și de unul sau mai multe discuri care stochează efectiv informația. Dispozitivele de stocare au următoarele atribute principale:

- spațiul disponibil - câtă informație se poate stoca pe disc (măsurat de obicei în gigabytes - GB sau terabytes - TB)
- viteza de acces - cât de repede se pot transfera datele de pe disc în memoria RAM (măsurat de obicei în megabytes pe secundă - MB/s)
- mod de poziționare - intern (în interiorul unității) sau extern (în exteriorul unității)
- mod de conectare - există două tipuri de conectare pentru discurile interne (SATA vs. SAS) și un tip de conectare pentru cele externe (USB)
- fiabilitate - câte ore de funcționare sau câte citiri/scrieri suportă de-a lungul vieții de funcționare. Aceste numere variază mult în funcție de tipul unității de stocare (HDD vs. SSD) și de gama pentru care a fost proiectat (folosirea în desktopuri vs. servere)

În zilele noastre există două tipuri de discuri pe sistemele de calcul:

- *Hard Disk Drive* (HDD)
- *Solid State Drive* (SSD)

10.1.1 Hard Disk Drive

HDD-ul prin construcție dispune de părți mobile în interiorul acestuia: un braț de citire/scriere si unul sau mai multe platane pe care se stochează datele (vezi Figura 10.3). Astfel HDD-ul este predispus defectării atunci când acesta funcționează și este supus unor anumite șocuri mecanice. De asemenea șocurile mecanice sunt problematice și atunci când HDD-ul nu funcționează (platanele nu se rotesc, iar capul de citire/scriere nu se mișcă). Modelul constructiv al HDD-ului de asemenea nu permite viteze de funcționare ridicate comparativ cu memoria RAM. Parametri prin care se măsoară performanța unui HDD sunt: numărul de rotații pe minut al platanelor și viteza interfeței de conectare la controller în gigabiți pe secundă. În zilele noastre, există două tipuri de interfețe de conectare:

- SATA - Serial ATA (*Serial Advanced Technology Attachment*)
- SAS - Serial Attached SCSI (sau *Serial Attached Small Computer System Interface*)

În funcție de aceste interfețe se clasifică și discurile. Ambele interfețe se interconectează cu controllerul la o viteză uzuală de 6Gbps. Discurile cu interfețe de tip SATA au viteza de rotație mai redusă decât SAS (5400rpm/7200rpm vs. 10000rpm/15000rpm), iar durata de viață, din construcție, este mult mai mică decât a SAS-urilor. De aceea discurile

[1] https://commons.wikimedia.org/wiki/File:Hard_Drive_(11644419853).jpg **CC BY 2.0**

Figura 10.3: HDD[1]

SATA sunt cu mult mai ieftine decât discurile SAS de aceeași capacitate (în cele mai multe cazuri pentru aceeași capacitate prețul se dublează). Un alt deziderat al discurilor SATA este acela că se concentrează pe capacitatea de stocare, iar discurile SAS se concentrează pe viteza de funcționare. Din cauza prețului foarte ridicat al discurilor SAS, acestea se folosesc în servere și sisteme de tip enterprise, iar discurile SATA se folosesc în calculatoarele personale.

În Figura 10.4 sunt reprezentate cele 2 tipuri de interfețe: în stânga sunt interfețe de tip SAS, iar în dreapta de tip SATA.

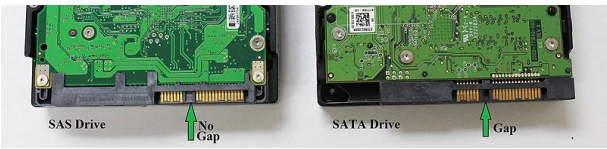


Figura 10.4: SAS vs SATA[2]

Figura 10.5: 2.5" vs 3.5"[3]

O altă clasificare a discurilor este legată de mărimea fizică a acestora:

[2]https://commons.wikimedia.org/wiki/File:SASvsSATA_Connector.JPG **CC BY SA 3.0**

[3]https://commons.wikimedia.org/wiki/File:Comparison_of_3.5_and_2.5_inch_hard_drives.jpg **CC BY SA 3.0**

- 2.5 inch (discurile din partea de sus din Figura 10.5)
- 3.5 inch (discurile din partea de jos din Figura 10.5)

Discurile de 2.5 inch se folosesc de obicei în sisteme enterprise, unde se dorește economisirea spațiului sau acomodarea a cât mai multe HDD-uri într-o unitate de rack (1U).

10.1.2 SSD

SSD-ul a venit ca un răspuns la neajunsurile HDD-ului, încercând să îmbunătățească sensibilitatea la șocuri și viteza de funcționare a acestuia. SSD-ul nu are părți mobile în interiorul acestuia și funcționează pe principiul stickurilor USB: pentru a stoca informația se folosesc chipuri construite din semiconductoare și nu discuri/platane magnetice care prezintă o mișcare pentru a putea citi/scrie datele.

Discurile de tip SSD pot fi conectate atât printr-o interfață SATA ce oferă viteze de până la 6Gbps cât și printr-o interfață SAS ce oferă viteze de până la 12Gbps în acest moment. Forma constructivă a unui SSD este de 2.5 inch sau de 3.5 inch, similar HDD-urilor.

Costul SSD-urilor comparativ cu cel al HDD-urilor, în special la capacități de peste 1TB este semnificativ mai mare (de aproximativ 5 ori). De aceea, în unele dintre sistemele din ziua de astăzi există un disc principal SSD, unde se stochează sistemul de operare și aplicațiile care au nevoie de acces rapid, și un HDD de mare capacitate, pentru stocarea datelor de termen lung și care nu necesită viteză ridicată.

Un aspect de luat în considerare atunci când se achiziționează un disc de tip SSD este durata de viață. Spre deosebire de un HDD, SSD-ul are un număr limitat de scrieri specificat de producător, după care se spune că s-a finalizat durata de viață a acestuia. Din acest motiv, sistemele de operare și sistemul de fișiere întreprind operații specifice, optimizate pentru SSD-uri, pentru a le prelungi durata de viață (de exemplu, defragmentarea generează foarte multe scrieri, scurtând durata de viață a discului, iar aceasta nu este utilă pentru un SSD - viteza de citire a oricăror date aleator de pe disc este aceeași în cazul SSD-urilor). Pentru a afla durata de viață a unui disc SSD există utilitare speciale cum ar fi SSD Life[1].

Pentru a prelungi durata de viață a unui disc, trebuie avute în vedere următoarele facilități:

- defragmentarea (procesul prin care se rearanjează datele pe disc pentru un acces mai eficient) - este recomandată dezactivarea defragmentării pentru discurile SSD (dupa cum am specificat mai sus)
- swapping - atunci când un sistem rămâne fără memorie RAM, acesta va folosi pe post de memorie discul. Este recomandat să dezactivați swappingul pe discurile SSD dacă doriți prelungirea duratei de viață (mai ales dacă swappingul apare des)
- hibernate - atunci când trecem calculatorul în starea Hibernate acesta scrie întregul conținut al memoriei pe disc (ceea ce va genera foarte multe scrieri). Este recomandat să nu treceți sistemul în Hibernate dacă doriți prelungirea duratei de viață (sau să configurați un disc alternativ pentru Hibernate).

[1] https://ssd-life.com

Alte recomandări pentru creșterea duratei de viață a discului SSD specifice sistemului de operare folosit, le puteți găsi la o simplă căutare pe Google[1].

10.2 Partiționarea dispozitivelor de stocare

În secțiunile anterioare au fost prezentate caracteristicile discurilor SSD și HDD. O caracteristică principală a acestora este capacitatea mare de stocare (mult mai care decât a memoriei RAM). Așadar o adresare liniară nu este suficientă pentru a gestiona spațiul de stocare oferit de un disc. Mai mult, un alt motiv este constituit de faptul că administrarea spațiului de stocare este făcută de către utilizator; în schimb gestionarea zonei de memorie RAM este făcută de către sistemul de operare. Astfel administrarea spațiului de stocare trebuie să fie simplă și facilă.

Spațiul de stocare oferit de un dispozitiv trebuie administrat în mod corect pentru a asigura o stocare eficientă (performanță ridicată) și coerentă (să nu corupem date). Există două niveluri de administrare: partiționare și sistemele de fișiere. Aceste niveluri sunt interdependente și se aplică iterativ.

Partiționarea împarte spațiul disponibil în zone continue fizic (numite **zone contigue**), fiecare zonă având un specific definit la creare (partiție de boot, partiție pentru utilizatori, partiție pentru sistemul de de operare). Există două tipuri de partiționări în sistemele din ziua de astăzi, așa cum am prezentat în Capitolul 9:

- Master Boot Record (referit ca MBR) - acesta a fost introdus în 1983 și poartă această denumire deoarece stochează la începutul discului un sector specific procesului de bootare.
- GUID Partition Table (referit ca GPT) - este o metodă mai nouă de partiționare ce va înlocui treptat MBR din cauza limitărilor acestuia.

Schema MBR, după cum se menționează anterior, are alocat exact la începutul unui disc un sector special de boot care reține schema de partiționare precum și o versiune minimală a bootloaderului care va încărca sistemul de operare, așa cum am precizat în Capitolul 9. Schema de partiționare MBR este formată din maxim 4 partiții, denumite **partiții primare**. Pentru a acoperi aceste neajunsuri, au fost introduse **partițiile logice**: una din partițiile primare va fi alocată pe întreg discul rămas liber și va fi marcată ca **partiție extinsă**. În cadrul partiției extinse se pot crea oricâte partiții logice se dorește. Acest lucru îngreunează schema de partiționare, iar unele sisteme de operare nu pot porni folosind aceste partiții. Nu se recomandă folosirea acestor partiții ca partiții de bootare. Întreaga schemă de partiționare a MBR este descrisă și în Figura 10.6.

MBR a început să fie înlocuit de către GPT din cauza limitărilor acestuia:

- dimensiunea maximă a unei partiții este de 2TB
- necesitatea unei partiții extinse pentru a crea mai mult de patru partiții

Pentru a rezolva neajunsurile MBR, a fost creată schema de partiționare GPT. Numele *GUID Partition Table* (GPT) provine de la faptul că fiecare partiție a discului are asociat

[1] de ex. https://www.makeuseof.com/tag/3-top-tips-maintain-performance-extend-life-ssd-si/

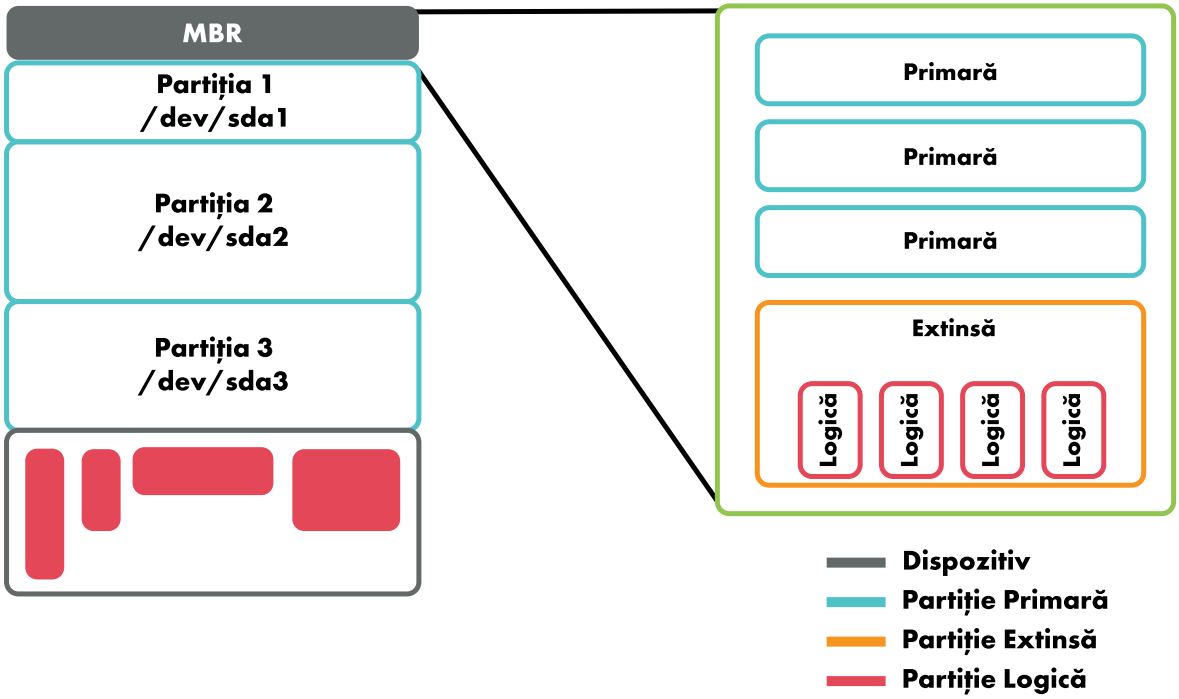


Figura 10.6: Schema de partiționare a MBR

un număr unic de identificare (*guid*), generat aleator și care garantează că fiecare partiție de pe glob va avea propriul identificator unic.

GPT este asociat în general cu standardul UEFI care dorește înlocuirea BIOS în sistemele de calcul, despre care am discutat în Capitolul 9. BIOS și UEFI sunt componente low-level care se execută la pornirea sistemelor de calcul pentru a realiza testarea și inițializarea componentelor. UEFI este un nou standard, menit să înlocuiască BIOS pentru a acoperi neajunsurile acestuia.

În Figura 10.7 este reprezentată schema de partiționare GPT. Se observă asemănarea cu partiționarea de tip MBR: la începutul discului există sectorul de boot. După sectorul de boot se află headerul GPT ce descrie partițiile (maxim 128). Sectorul de boot și header sunt duplicate și la finalul discului: în caz de corupere a primelor sectoare să nu pierdem detaliile despre schema de partiționare. În cazul GPT nu mai avem limitarea la 2TB a unei partiții și nici limitarea la 4 partiții primare (din cauza numărului ridicat disponibil de partiții nu mai există noțiunea de partiții logice).

Pentru a utiliza spațiul de stocare eficient, există recomandări din partea vendorilor sistemelor de operare. Bunele practici în partiționarea unui disc includ:

- Crearea unei partiții de boot (specific Linux `/boot` - conține nucleul sistemului de operare și bootloaderul)
- Crearea unei partiții de swap (specific Linux - spațiu utilizat de sistemul de operare atunci când rămâne fără memorie; în cazul Windows swap-ul este alocat din partiția sistemului de operare)
- Crearea unei partiții pentru sistemul de operare (pentru Linux este `/`, denumită și partiția rădăcină - *root partition*, iar pentru Windows de obicei este discul `C:`)
- Crearea unei partiții pentru utilizatorii sistemului - atunci când reinstalăm sistemul de operare, să putem păstra ușor datele utilizatorilor (pentru Linux este `/home`, iar

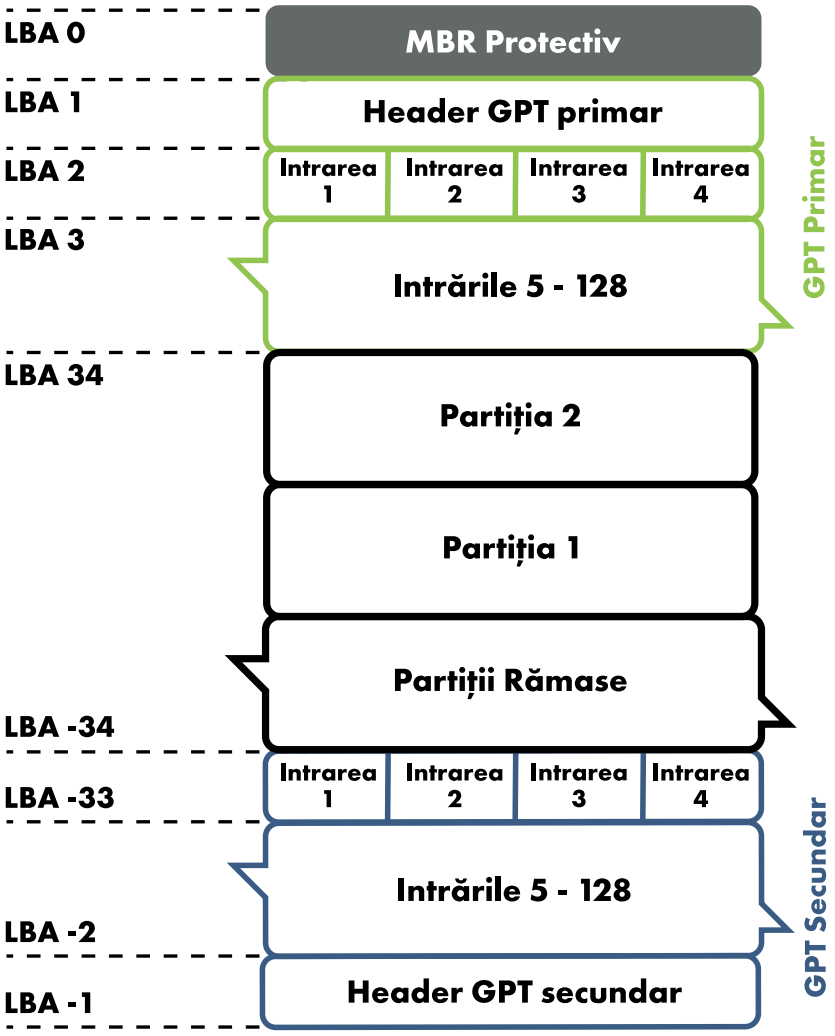


Figura 10.7: Schema de partiționare a GPT

pentru Windows de obicei literele de la `D:` în sus)

- Crearea unei partiții pentru sistemele cu suport EFI/UEFI (specific Linux în `/boot/efi` și conține fișierele necesare să fie executate la încărcarea sistemului de operare de către subsistemul UEFI)

Utilitare ce pot fi folosite pentru administrarea partițiilor sunt:

- Pentru Linux: **`fdisk`** (doar MBR), **`gdisk`** (pentru GPT); **`parted`**/**`gparted`** (suport extins pentru MBR/GPT și operații avansate). O dată creată o partiție, aceasta poate fi văzută în calea `/dev` și poartă numele discului partiționat (ex. `sda`) urmat de o cifră care reprezintă numărul partiției (ex. `sda5`)
- Pentru Windows: **`Disk Management`**. O data creată o partiție, aceasta poate fi văzută în utilitarul menționat.

10.2.1 Sisteme de fișiere

Pentru a putea organiza datele pe o partiție într-un mod facil și ușor de înțeles pentru utilizatorul final al sistemului de calcul, pe aceasta în general se instalează un sistem de fișiere. Procesul de instalare/alocare a unui sistem de fișiere pe o partiție se numește **formatare**. O partiție fără un sistem de fișiere nu poate fi utilizată/explorată de către utilizatorul final. De exemplu, în Windows, un disc USB neformatat nu va putea fi accesat cu dublu-click și va apărea un mesaj către utilizator dacă acceptă formatarea lui ca în Figura 10.8.

Figura 10.8: Fereastră privind formatarea unui disc în Windows

Sistemul de fișiere oferă următoarele obiecte vizibile utilizatorului final:

- fișierul - entitatea care conține date utile în diverse formate
- directorul - entitatea care organizează fișierele într-un formă arborescentă ușor de folosit de către utilizator

Mai multe detalii legate de fișiere și directoare (modul în care se creează, se șterg, permisiuni de acces) se găsesc în Capitolul 2.

În general sistemele de fișiere sunt specifice sistemelor de operare. Astfel avem următoarea clasificare:

- Linux: cel mai folosit este sistemul de fișiere Ext4, iar mai nou acesta ajunge să fie înlocuit cu XFS
- Windows: principalul sistem de fișiere este NTFS

În Linux, pentru a formata (instala) un sistem de fișiere pe o partiție se folosește comandă **`mkfs`** urmată de sufixul sistemului de operare dorit și partiția dorită (ex. **`mkfs.ext4 /dev/sda2`**). **Atenție**: procesul de formatare a unui sistem de fișiere distruge datele prezente pe acea partiție. O greșeală frecventă este aplicarea procesului de formatare pe întreg discul (ex. **`mkfs.ext4 /dev/sda`**). Acest lucru va șterge **toate** partițiile de pe acel disc. Un exemplu complet de folosire a utilitarelor de formatare și partiționare îl puteți găsi la finalul capitolului, la studii de caz.

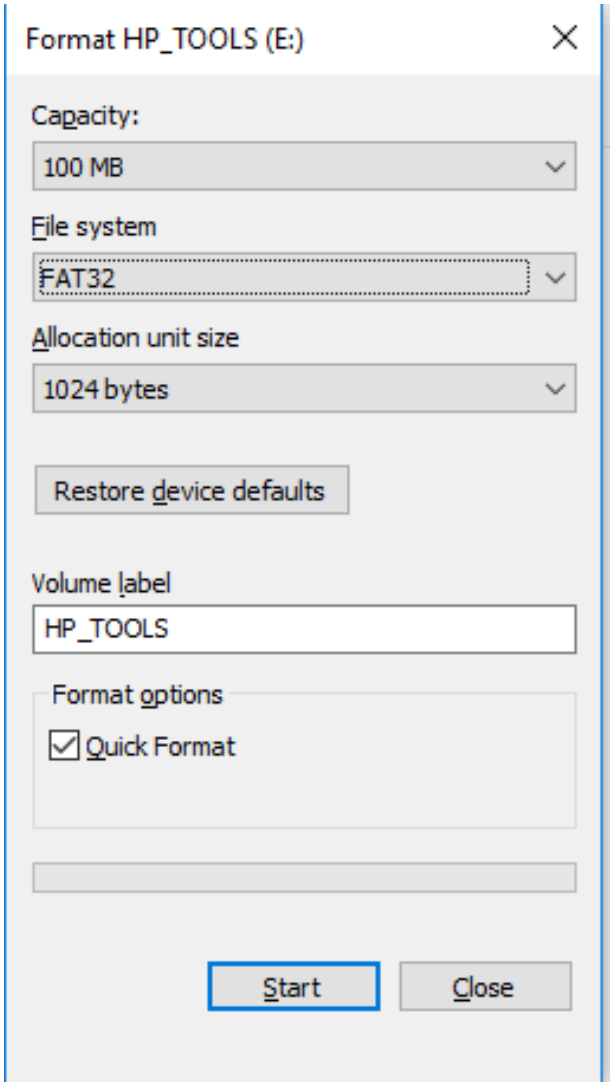


Figura 10.9: Formatarea unei partiții în Windows

În Windows, pentru a formata (instala) un sistem de fișiere pe o partiție, se face click dreapta pe litera aferentă partiției și există opțiunea `Format`: de aici se poate selecta tipul sistemului de fișiere și diverse caracteristici de formatare, ca în Figura 10.9.

Pentru a putea folosi un sistem de fișiere, acesta trebuie făcut disponibil utilizatorului. Procesul prin care acest este disponibil pentru citire/scriere se numește **montare**.

Pe sistemele bazate pe Linux, montarea se realizează cu comanda **`mount`** urmată de partiția care se dorește a fi montată și directorul unde să fie montată în sistemul de fișiere (numit punct de montare - *mount point*):

```
1 student@uso:~$ sudo mount /dev/sdb2 /mnt/sdb2
2 student@uso:~$ mount
```

```
[...]
/dev/sdb2 on /mnt/sdb2 type ext4 (rw,relatime,data=ordered)
```

Demontarea se face cu ajutorul comenzii **`umount`**:

```
student@uso:~$ sudo umount /mnt/sdb2
```

Un lucru specific sistemelor de operare bazate pe Linux este alocarea unui identificator la formatarea unei partiții, numit UUID (*Universally Unique Identifier*). Astfel la montare se poate specifica identificatorul partiției în loc calea către dispozitiv. Acest lucru este utilizat pentru a rezolva conflictele generate de schimbarea ordinii discurilor fizice. De exemplu, în cazul prezentat mai sus, dacă mai inserăm un disc fizic în sistem, acesta poate fi detectat înainte celui prezent și i se va aloca litera `b`, deci va fi `sdb`, iar cel curent va deveni `sdc` Astfel comanda de mai sus devine invalidă și inconsistentă deoarece va monta un cu totul alt disc. Pentru a afla UUID-ul alocat unei partiții noi formatate se folosește comanda **`blkid`**:

```
student@uso:~$ sudo blkid /dev/sdb2
/dev/sdb2: UUID="6ea1370c-b47c-497f-a7b7-556d40d4af97" TYPE="ext4"
PARTUUID="f3aa63a6-02"
```

Acum putem monta partiția folosind UUID-ul:

```
student@uso:~$ sudo mount -U 6ea1370c-b47c-497f-a7b7-556d40d4af97 /mnt/
sdb2
student@uso:~$ mount
[...]
/dev/sdb2 on /mnt/sdb2 type ext4 (rw,relatime,data=ordered)
```

În Windows montarea se realizează în general automat de către sistemul de operare. Dacă se doresc operații avansate, acestea pot fi făcute tot din *Disk Management*: `Start` → `Run` → `diskmgmt.msc`, ca în Figura 10.10.

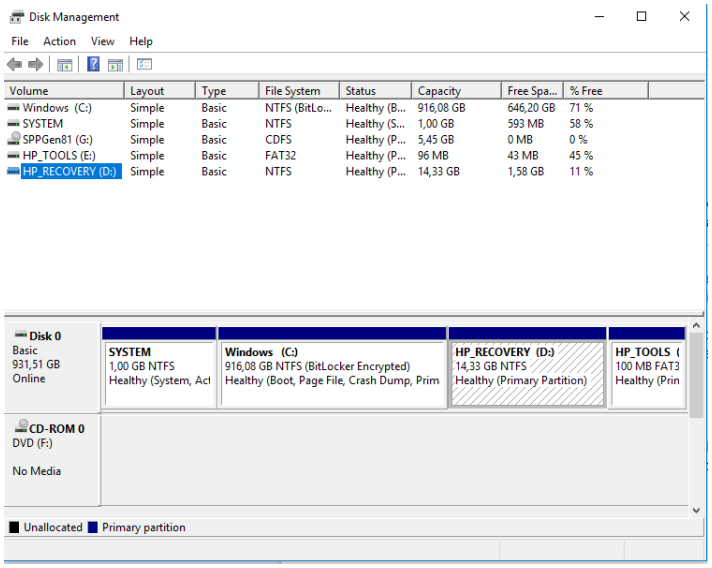


Figura 10.10: Administrarea spațiului de stocare în Windows

Anterior, s-a menționat faptul că sistemele de fișiere sunt specifice sistemului de operare. La nevoie, aceste sisteme de fișiere se pot monta pe alt sistem de operare decât cele pentru care au fost proiectate. Acest lucru nu este întotdeauna recomandat întrucât există penalizări de performanță și risc de corupere a datelor.

Sistemele de fișiere au mecanisme interne de recuperare a datelor corupte. Principalul mecanism prin care se realizează acest lucru este jurnalizarea (*file system journal*). Fiecare sistem de fișiere scrie fiecare operație adusă sistemului de fișiere într-un jurnal. Atunci când sunt probleme la montarea unui sistem de fișiere, acesta folosește jurnalul pentru a încerca să repare problemele de coerență a datelor. De multe ori nu se poate monta sistemul de fișiere fără o verificare completă a integrității sistemului de fișiere. Acest lucru se realizează de obicei automat la pornirea sistemului de operare, atât în Linux cât și în Windows. Este important de menționat faptul că nu toate sistemele de fișiere oferă suport de jurnalizare. În ziua de astăzi, anumite sisteme de fișiere simple nu oferă acest suport; sunt simple și se adresează dispozitivelor specializate (ex. camere video) și cu puține resurse care nu au implementat un mecanism de verificare a integrității; un astfel de sisteme de fișiere este FAT.

În Linux, dacă este necesară verificarea integrității sistemului de fișiere manual, se poate folosi utilitarul **`fsck`**, ca în Listing 10.1. Se observă că nu se poate verifica sistemul de fișiere când acesta este deja montat. Vom demonta sistemul de fișiere iar apoi vom verifica integritatea.

```
1  student@uso:~$ sudo fsck /dev/sdb2
2  fsck from util-linux 2.31.1
3  e2fsck 1.44.1 (24-Mar-2018)
4  /dev/sdb2 is mounted.
5  e2fsck: Cannot continue, aborting.
6
7  student@uso:~$ sudo umount /dev/sdb2
8  student@uso:~$ sudo fsck /dev/sdb2
9  fsck from util-linux 2.31.1
10 e2fsck 1.44.1 (24-Mar-2018)
11 /dev/sdb2: clean, 11/51296 files, 7726/204800 blocks
```

Listing 10.1: Verificarea integrității sistemului de fișiere

Ca și schemele de partiționare, sistemele de fișiere au și ele anumite limitări legate de:

- numărul maxim de fișiere (ex. pentru ext4 - 4 miliarde)
- dimensiunea partiției (ex. pentru ext4 - 1024PB)
- dimensiunea maximă a fișierului (ex. pentru ext4 - 16TB)
- atribute extinse de funcționalitate și securitate

Limitările menționate mai sus variază de la un sistem de operare la altul, sunt precizate în documentația respectivului sistem de fișiere.

Până în acest punct am discutat despre disc și partiție. Discul este partea fizică a sistemului de calcul disponibilă spre a fi folosită de către sistemul de operare. Pentru o folosire eficientă, acesta este partiționat (partiție). Deseori, prin diverse tehnici (RAID hardware sau software), mai multe discuri sunt puse la comun și văzute ca unul singur. Rezultatul poartă numele de *volum*. Un alt termen folosit în administrarea spațiului de stocare este *drive*. Acesta are două semnificații: referă tot un disc fizic sau referă o partiție pe sisteme Windows.

10.3 Anexă: Adăugarea unui disc mașinii virtuale

Pentru partea demonstrativă a acestui capitol, folosim un disc nou adăugat mașinii virtuale. Acest disc va fi vizibil în sistemul de operare (Linux) al mașinii virtuale ca `/dev/sdb`. Spunem că discul este un disc virtual.

Pentru a adăuga un disc mașinii virtuale USO și, în general, pentru a adăuga un disc nou unei mașini virtuale VirtualBox, se urmează pașii:

1. click dreapta pe intrarea mașinii virtuale în VirtualBox și deschiderea meniului contextual
2. selectarea opțiunii `Settings` din meniul contextual deschis
3. în fereastra nou deschisă (`Settings`), navigarea la intrarea `Storage`
4. în noul ecran se folosește iconul `Adds hard disk` pentru adăugarea unui nou disc la controllerul SATA, ca în Figura 10.11
5. se selectează în ordine `Create new disk`, `VDI`, `Dynamically allocated`
6. se alege un nume și o dimensiune; de exemplu, pentru partea demonstrativă a acestui capitol, am ales numele `my.vdi` și dimensiunea de `2GB` ca în Figura 10.12

În acest moment avem un nou disc în cadrul mașinii virtuale. În interfața VirtualBox, din Figura 10.12, discul este văzut ca `my.vdi`. În sistemul de operare Linux din mașina virtuală, discul va apărea ca `/dev/sdb`.

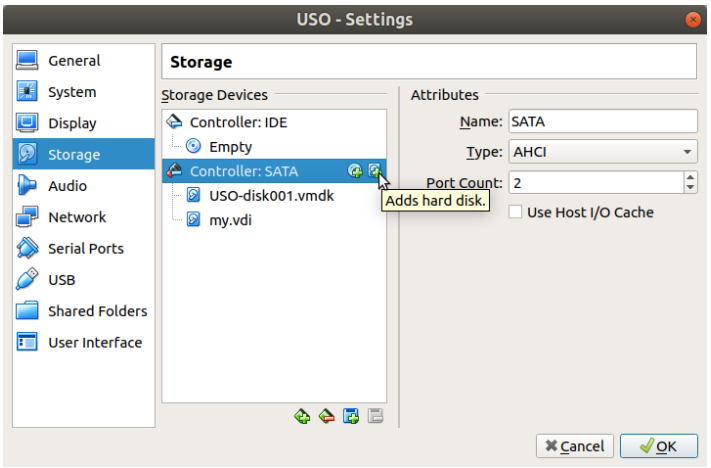


Figura 10.11: Adăugarea unui nou disc mașinii virtuale

10.4 Anexă: Investigarea spațiului de stocare

În Linux, pentru a vizualiza toate discurile din sistem, precum și unde sunt montate, putem folosi utilitarul **`lsblk`** ca în Listing 10.2. Există două discuri în sistem (`sda` și `sdb`) și un dispozitiv de tip CD ROM (`sr0`). Discul `sda` are o partiție (`sda1`) care ocupă tot discul (`16GB`).

```
1 student@uso:~$ lsblk -i
```

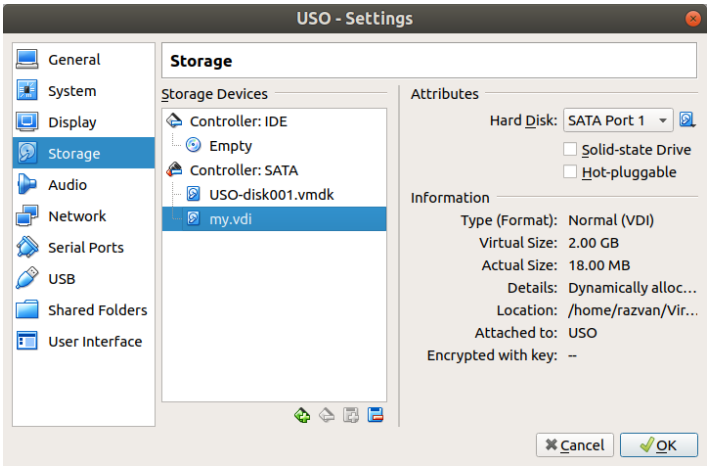


Figura 10.12: Informații despre un disc virtual

```
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
[...]
sda      8:0    0    16G  0 disk
`-sda1   8:1    0    16G  0 part /
sdb      8:16   0     2G  0 disk
sr0     11:0    1  1024M  0 rom
```

Listing 10.2: Afișarea discurilor din sistem (lsblk)

Informații despre partițiile montate ale sistemului și spațiul ocupat de acestea pot fi vizualizate folosind utilitarul **`df`** ca în Listing 10.3. Opțiunea `-h` este folosită pentru a afișa dimensiunea partițiilor în format ușor de citit – *human readable*:

```
student@uso:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
[...]
/dev/sda1        16G   11G  4.2G  73% /
[...]
```

Listing 10.3: Informații despre partiții montate (df)

Pentru a putea investiga spațiul ocupat de un anumit fișier sau director, putem folosi comanda **`du`** ca în Listing 10.4. Opțiunea `-h` este folosită pentru a afișa dimensiunea în format ușor de citit, iar `-s` realizează sumarizarea tuturor fișierelor din acel director. Dacă nu folosim opțiunea `-s`, atunci ni se va afișa fiecare fișier recursiv al acelui director.

```
student@uso:~$ du -hs /opt
95M     /opt

student@uso:~$ du -h /opt
52K     /opt/VBoxGuestAdditions-5.2.18/init
4.7M    /opt/VBoxGuestAdditions-5.2.18/lib
1.8M    /opt/VBoxGuestAdditions-5.2.18/other
1.5M    /opt/VBoxGuestAdditions-5.2.18/bin
288K    /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxvideo
40K     /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxguest/
common/err
176K    /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxguest/
common/log
40K     /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxguest/
common/alloc
```

```
172K    /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxguest/common/string
96K     /opt/VBoxGuestAdditions-5.2.18/src/vboxguest-5.2.18/vboxguest/common/misc
[...]
```

Listing 10.4: Informații despre spațiul ocupat de directoare (du)

Pentru a investiga informații despre partițiile unui disc se folosesc utilitarele **fdisk** (pentru partiții MBR), **gdisk** (pentru partiții GPT) sau **parted**/**gparted**. Comanda **fdisk -l** va lista toate discurile și partițiile unui sistem, ca în Listing 10.5. Dacă se adaugă un parametru suplimentar (calea către discul pe care se dorește a fi inspectat), atunci doar acesta va fi afișat. Informațiile despre partiții sunt informații critice din sistem, așa că rularea trebuie făcut în mod privilegiat, prin prefixarea comenzii cu **sudo**.

```
student@uso:~$ sudo fdisk -l
[...]
Disk /dev/sda: 16 GiB, 17179869184 bytes, 33554432 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xdf4f561b

Device     Boot Start      End  Sectors Size Id Type
/dev/sda1  *     2048 33552383 33550336  16G 83 Linux

Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

student@uso:~$ sudo fdisk -l /dev/sda
Disk /dev/sda: 16 GiB, 17179869184 bytes, 33554432 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xdf4f561b

Device     Boot Start      End  Sectors Size Id Type
/dev/sda1  *     2048 33552383 33550336  16G 83 Linux
```

Listing 10.5: Afișarea partițiilor (fdisk)

Dacă renunțăm la parametrul `-l`, vom intra în modul interactiv al utilitarului **fdisk**, ca în Listing 10.6. Forma interactivă a utilitarului va afișa un prompt unde se vor introduce comenzi. În Listing 10.6 sunt folosite două comenzi:

- comanda `m`: este comanda de ajutor care afișează comenzile disponibile
- comanda `p`: afișează tabela de partiții a discului investigat
- comanda `q`: închide aplicația interactivă **fdisk**

```
student@uso:~$ sudo fdisk /dev/sda
[...]
Command (m for help): m
```

```
4
5  Help:
6
7    DOS (MBR)
8     a   toggle a bootable flag
9     b   edit nested BSD disklabel
10    c   toggle the dos compatibility flag
11
12   Generic
13    d   delete a partition
14    F   list free unpartitioned space
15    l   list known partition types
16    n   add a new partition
17    p   print the partition table
18    t   change a partition type
19    v   verify the partition table
20    i   print information about a partition
21
22   Misc
23    m   print this menu
24    u   change display/entry units
25    x   extra functionality (experts only)
26
27   Script
28    I   load disk layout from sfdisk script file
29    O   dump disk layout to sfdisk script file
30
31   Save & Exit
32    w   write table to disk and exit
33    q   quit without saving changes
34
35   Create a new label
36    g   create a new empty GPT partition table
37    G   create a new empty SGI (IRIX) partition table
38    o   create a new empty DOS partition table
39    s   create a new empty Sun partition table
40
41
42 Command (m for help): p
43 Disk /dev/sda: 16 GiB, 17179869184 bytes, 33554432 sectors
44 Units: sectors of 1 * 512 = 512 bytes
45 Sector size (logical/physical): 512 bytes / 512 bytes
46 I/O size (minimum/optimal): 512 bytes / 512 bytes
47 Disklabel type: dos
48 Disk identifier: 0xdf4f561b
49
50 Device     Boot Start      End  Sectors Size Id Type
51 /dev/sda1  *     2048 33552383 33550336  16G 83 Linux
52
53 Command (m for help): q
54
55 student@uso:~$
```

Listing 10.6: Mod interactiv pentru fdisk

Utilitarul **parted** se folosește în mod asemănător cu **fdisk**. Pentru a administra un disc în mod interactiv se rulează comanda **parted** urmată de calea către dispozitiv, ca în Listing 10.7. Similar **fdisk**, forma interactivă a utilitarului **parted** primește comenzi:

- `help`: afișează un sumar al comenzilor disponibile

- `p` / `print`: afișează tabela de partiții a discului
- `q` / `quit`: închide aplicația interactivă

```
student@uso:~$ sudo parted /dev/sda
GNU Parted 3.2
Using /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) help
  align-check TYPE N                        check partition N for TYPE(min|opt) alignment
  help [COMMAND]                           print general help, or help on COMMAND
  mklabel,mktable LABEL-TYPE               create a new disklabel (partition table)
  mkpart PART-TYPE [FS-TYPE] START END     make a partition
  name NUMBER NAME                         name partition NUMBER as NAME
  print [devices|free|list,all|NUMBER]     display the partition table, available devices, free space, all found partitions, or a particular partition
  quit                                     exit program
  rescue START END                         rescue a lost partition near START and END
  resizepart NUMBER END                    resize partition NUMBER
  rm NUMBER                                delete partition NUMBER
  select DEVICE                            choose the device to edit
  disk_set FLAG STATE                      change the FLAG on selected device
  disk_toggle [FLAG]                       toggle the state of FLAG on selected device
  set NUMBER FLAG STATE                    change the FLAG on partition NUMBER
  toggle [NUMBER [FLAG]]                   toggle the state of FLAG on partition NUMBER
  unit UNIT                                set the default unit to UNIT
  version                                  display the version number and copyright information of GNU Parted
(parted) p
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sda: 17.2GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      1049kB  17.2GB  17.2GB  primary  ext4         boot

(parted) q

student@uso:~$
```

Listing 10.7: Gestiunea discurilor folosind parted

10.5 Anexă: Partiționare, formatare și montare în Linux

Vom realiza pas cu pas partiționarea, formatarea și montarea unui disc în Linux. Vom realiza partiționarea discului folosind forma interactivă a utilitarului **`fdisk`**. Vom realiza

formatarea partițiilor folosind utilitarele din familia **mkfs**. Vom monta partițiile folosind utilitarul **mount**.

Partiționarea discului este prezentată în Listing 10.8. În rularea interactivă a utilitarului **fdisk** am folosit comenzile:

- `p`: afișarea tabelei de partiții
- `n`: crearea unei partiții noi. Când creăm o partiție nouă, intrăm într-un submod interactiv în care precizăm tipul partiției (primară sau logică), indexul ei, și începutul și sfârșitul (în număr de sector)
- `w`: scrierea tabelei de partiții pe disc; permanentizarea modificărilor

În urma comenzilor din Listing 10.8 am creat trei partiții primare: `sdb1` (`500MB`), `sdb2` (`800MB`) și `sdb3` (`747MB`). Pentru afișarea partițiilor am folosit comanda `p` din rularea interactivă a utilitarului **fdisk** și opțiunea `-l` la rularea neinteractivă.

```
1  student@uso:~$ sudo fdisk /dev/sdb
2
3  Welcome to fdisk (util-linux 2.31.1).
4  Changes will remain in memory only, until you decide to write them.
5  Be careful before using the write command.
6
7  Device does not contain a recognized partition table.
8  Created a new DOS disklabel with disk identifier 0xf3aa63a6.
9
10 Command (m for help): p
11 Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
12 Units: sectors of 1 * 512 = 512 bytes
13 Sector size (logical/physical): 512 bytes / 512 bytes
14 I/O size (minimum/optimal): 512 bytes / 512 bytes
15 Disklabel type: dos
16 Disk identifier: 0xf3aa63a6
17
18 Command (m for help): n
19 Partition type
20    p   primary (0 primary, 0 extended, 4 free)
21    e   extended (container for logical partitions)
22 Select (default p): p
23 Partition number (1-4, default 1):
24 First sector (2048-4194303, default 2048):
25 Last sector, +sectors or +size{K,M,G,T,P} (2048-4194303, default 4194303)
   : +500M
26
27 Created a new partition 1 of type 'Linux' and of size 500 MiB.
28
29 Command (m for help): n
30 Partition type
31    p   primary (1 primary, 0 extended, 3 free)
32    e   extended (container for logical partitions)
33 Select (default p): p
34 Partition number (2-4, default 2):
35 First sector (1026048-4194303, default 1026048):
36 Last sector, +sectors or +size{K,M,G,T,P} (1026048-4194303, default
   4194303): +800M
37
38 Created a new partition 2 of type 'Linux' and of size 800 MiB.
39
```

```
Command (m for help): n
Partition type
   p   primary (2 primary, 0 extended, 2 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (3,4, default 3):
First sector (2664448-4194303, default 2664448):
Last sector, +sectors or +size{K,M,G,T,P} (2664448-4194303, default
4194303):

Created a new partition 3 of type 'Linux' and of size 747 MiB.

Command (m for help): p
Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf3aa63a6

Device     Boot   Start     End Sectors  Size Id Type
/dev/sdb1          2048 1026047 1024000  500M 83 Linux
/dev/sdb2       1026048 2664447 1638400  800M 83 Linux
/dev/sdb3       2664448 4194303 1529856  747M 83 Linux

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

student@uso:~$ sudo fdisk -l /dev/sdb
Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf3aa63a6

Device     Boot   Start     End Sectors  Size Id Type
/dev/sdb1          2048 1026047 1024000  500M 83 Linux
/dev/sdb2       1026048 2664447 1638400  800M 83 Linux
/dev/sdb3       2664448 4194303 1529856  747M 83 Linux
```

Listing 10.8: Partiționarea unui disc folosind fdisk

Vom formata cele trei partiții, respectiv cu sistemele de fișiere FAT, Ext4 și NTFS. Vom folosi, respectiv, utilitarele **`mkfs.vfat`**, **`mkfs.ext4`** și **`mkfs.ntfs`** ca în Listing 10.9.

```
student@uso:~$ sudo mkfs.vfat /dev/sdb1
mkfs.fat 4.1 (2017-01-24)

student@uso:~$ sudo mkfs.ext4 /dev/sdb2
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 204800 4k blocks and 51296 inodes
Filesystem UUID: 6ea1370c-b47c-497f-a7b7-556d40d4af97
Superblock backups stored on blocks:
        32768, 98304, 163840

Allocating group tables: done
Writing inode tables: done
```

```
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

student@uso:~$ sudo mkfs.ntfs /dev/sdb3
Cluster size has been automatically set to 4096 bytes.
Initializing device with zeroes: 100% - Done.
Creating NTFS volume structures.
mkntfs completed successfully. Have a nice day.
```

Listing 10.9: Formatarea partițiilor (mkfs)

Pentru a valida sistemele de fișiere cu care sunt formatate partițiile, folosim utilitarele **`lsblk`** sau **`parted`** ca în Listing 10.10.

```
student@uso:~$ sudo parted -l
[...]
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 2147MB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size   Type     File system  Flags
 1      1049kB  525MB   524MB  primary  fat16
 2      525MB   1364MB  839MB  primary  ext4
 3      1364MB  2147MB  783MB  primary  ntfs


student@uso:~$ lsblk -f
NAME   FSTYPE   LABEL UUID                                 MOUNTPOINT
[...]
sdb
|-sdb1 vfat           37F1-DE8B
|-sdb2 ext4           6ea1370c-b47c-497f-a7b7-556d40d4af97
`-sdb3 ntfs           59CABC6B18A0D079
[...]
```

Listing 10.10: Sisteme de fișiere pe partiții

Pentru a putea folosi partițiile formatate, trebuie să le montăm într-un punct de montare (*mount point*) din sistemul de fișiere. Vom monta cele trei partiții, respectiv, în directoarele `/mnt/sdb1`, `/mnt/sdb2` și `/mnt/sdb3`, directoare pe care le vom crea anterior, ca în Listing 10.11. Apoi folosim utilitarul **`mount`** pentru a le monta.

```
student@uso:~$ sudo mkdir /mnt/sdb1
student@uso:~$ sudo mkdir /mnt/sdb2
student@uso:~$ sudo mkdir /mnt/sdb3
student@uso:~$ sudo mount /dev/sdb1 /mnt/sdb1
student@uso:~$ sudo mount /dev/sdb2 /mnt/sdb2
student@uso:~$ sudo mount /dev/sdb3 /mnt/sdb3
```

Listing 10.11: Montarea partițiilor

Pentru a verifica montarea, folosim comanda **`mount`** (fără argumente) sau utilitarul **`df`**, ca în Listing 10.12: Pentru partițiile `/dev/sdb2` și `/dev/sdb3` observăm că există o parte din spațiu ocupat (`Used`). Acest lucru se datorează metadatelor necesare sistemelor de fișiere mai complexe (Ext4, NTFS) pentru a organiza datele efective.

```
student@uso:~$ mount
```

```
[...]
/dev/sdb1 on /mnt/sdb1 type vfat (rw,relatime,fmask=0022,dmask=0022,
codepage=437,iocharset=iso8859-1,shortname=mixed,errors=remount-ro)
/dev/sdb2 on /mnt/sdb2 type ext4 (rw,relatime,data=ordered)
/dev/sdb3 on /mnt/sdb3 type fuseblk (rw,relatime,user_id=0,group_id=0,
allow_other,blksize=4096)
student@uso:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
[...]
/dev/sdb1       500M     0  500M   0% /mnt/sdb1
/dev/sdb2       772M  1.6M  714M   1% /mnt/sdb2
/dev/sdb3       747M  4.2M  743M   1% /mnt/sdb3
```

Listing 10.12: Afișarea partițiilor montate

Un ultim pas pe care trebuie să îl aveți în vedere atunci când configurați partiționarea pe un sistem este dat de atributele unei partiții. Cel mai important atribut este cel care marchează partiția de *boot*. Așa cum este prezentat în Listing 10.13, în forma interactivă a utilitarului **`fdisk`**, comanda **`a`** activează atributul (flagul) de boot pe partiția dorită. Se observă steluța din dreptul partiției **`/dev/sdb2`**.

```
student@uso:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf3aa63a6

Device     Boot   Start     End Sectors  Size Id Type
/dev/sdb1          2048 1026047 1024000  500M 83 Linux
/dev/sdb2       1026048 2664447 1638400  800M 83 Linux
/dev/sdb3       2664448 4194303 1529856  747M 83 Linux

Command (m for help): a
Partition number (1-3, default 3): 2

The bootable flag on partition 2 is enabled now.

Command (m for help): p
Disk /dev/sdb: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf3aa63a6

Device     Boot   Start     End Sectors  Size Id Type
/dev/sdb1          2048 1026047 1024000  500M 83 Linux
/dev/sdb2  *    1026048 2664447 1638400  800M 83 Linux
/dev/sdb3       2664448 4194303 1529856  747M 83 Linux

```

```
39 Command (m for help): w
40 The partition table has been altered.
41 Syncing disks.
```

Listing 10.13: Activarea boot flag

10.6 Anexă: Backup periodic

Un mod de a asigura disponibilitatea datelor este replicarea lor: datele se găsesc în cel puțin două locuri. O formă de replicare este mecanismul RAID (*Redundant Array of Independent / Inexpensive Disks*), iar alta este salvarea periodică (*backup*) a datelor într-un sistem de operare. Vom prezenta RAID în Secțiunea 10.9. În continuare vom prezenta soluții de backup periodic.

10.6.1 Backup periodic în Linux

În Linux, unul dintre cele mai folosite utilitare în replicarea eficientă a fișierelor și directoarelor este **`rsync`**. Acesta are două caracteristici principale:

- poate face replicare incrementală (nu este necesar să copiem întreaga cale de fiecare dată), ceea ce duce la o îmbunătățire semnificativă a timpului de backup
- dispune de un control granular al atributelor replicate: owneri, grupuri, atribute extinse, poate sau nu urmări link-urile simbolice

Comanda **`rsync`** are opțiuni care controlează atributele replicate și are două argumente care specifică sursa și destinația datelor. Sursa și destinația datelor poate fi locală, în sistemul de fișiere curent, sau poate exista peste rețea, iar rsync le accesează folosind protocolul SSH. Vom apela la un exemplu complex pentru backupul întregului sistem de fișiere:

```
1 rsync -aAXv --exclude={"/dev/*","/proc/*","/sys/*","/tmp/*","/run/*","/
mnt/*","/media/*","/lost+found"} / root@nas:/backup
```

Comanda **`rsync`** are opțiunile:

- `a` - modul de arhivare; păstrează și replică toate atributele fișierelor și directoarelor cu excepția linkurilor hard, atributelor extinse și listelor de acces (ACL - *Access Control List*)
- `A` - replică listele de acces (ACL)
- `X` - replică atributele extinse
- `H` - replică linkurile hard
- `--exclude` - se vor exclude din backup căile specificate (toate acele căi nu conțin în mod normal date utile utilizatorului, ci sunt căi virtuale populate la bootare de către sistemul de operare)

Comanda **`rsync`** primește următoarele argumente:

- `/` - sursa datelor (directorul rădăcină)

- `root@nas:/backup` - destinația datelor (un director în rețea pe stația cu numele `nas`)

Dacă rulăm de mai multe ori comanda de mai sus, aceasta va realiza automat backup incremental. `--delete` este un parametru pe care nu l-am folosit anterior, dar este de obicei util: dacă la destinația menționată există fișiere care nu sunt la sursă, acestea vor fi șterse; astfel obținem consistență 100% între sursă și destinație.

Folosirea **`rsync`** din CLI și managementul versiunilor replicate este de obicei greoaie. Pentru a ușura acest lucru, puteți folosi aplicația BackupPC[1] ce are o interfață web prin care puteți controla opțiunile și versiunile replicate. La baza BackupPC stă utilitarul **`rsync`**.

10.6.2 Backup periodic în Windows

În cazul sistemelor Windows, există o multitudine de aplicații de replicare a datelor, atât plătite, cât și gratuite. Un exemplu îl constituie utilitarul Cobian Backup[2]. Cu ajutorul acestuia putem configura replicarea unui director sau a unui disc local pe un stick USB, sau la distanță peste rețea. În Figura 10.13 se află fereastra principală a aplicației. În coloana din stânga sunt sarcinile de backup pe care le creați. În figură avem o sarcină denumită *Backup D*, care va face replicarea întregii partiții `D:`. Printre proprietățile sarcinii de replicare avem:

- `General` - aici se configurează setări generale (dacă vor fi incluse subdirectoarele, ce atribute vor fi replicate)
- `Files` - ce fișiere vor fi replicate și unde
- `Schedule` - când se va realiza replicarea
- `Archive` - dacă datele vor fi arhivate înainte de replicare

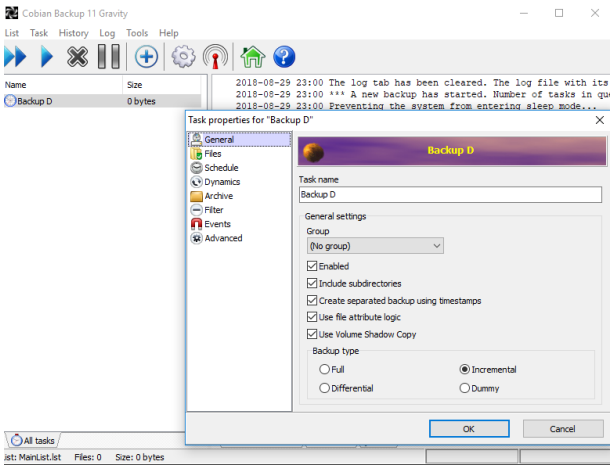


Figura 10.13: Aplicația pentru replicarea datelor în Windows: Cobian Backup

[1] https://backuppc.github.io/backuppc/
[2] http://www.cobiansoft.com/cobianbackup.htm

10.7 Anexă: Montarea dispozitivelor

Pentru a putea folosi un dispozitiv de stocare într-un sistem de calcul, acesta trebuie montat. În cazul sistemelor Windows, dispozitivele sunt montate automat. În cazul sistemelor Linux, acestea nu sunt întotdeauna montate (mai ales dacă sistemul nu dispune de interfață grafică). Cazul cel mai des întâlnit este dat de montarea unui stick USB. La inserarea unui stick USB într-o unitate de calcul, veți observa, în fișierele de jurnalizare ale sistemului (rulați comanda **dmesg**), că acesta a fost detectat. Veți observa și numele intrării pentru gestiunea dispozitivului (în cazul de față observați că este `/dev/sdb`):

```
usb 3-5: new high-speed USB device number 4 using xhci_hcd
usb 3-5: New USB device found, idVendor=abcd, idProduct=1234
usb 3-5: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 3-5: Product: 1
usb 3-5: Manufacturer: 1
usb 3-5: SerialNumber: 60A44C3D7ECDAF710000058E
usb-storage 3-5:1.0: USB Mass Storage device detected
scsi host5: usb-storage 3-5:1.0
usbcore: registered new interface driver usb-storage
usbcore: registered new interface driver uas
scsi 5:0:0:0: Direct-Access     General  UDisk            5.00 PQ: 0 ANSI
: 2
sd 5:0:0:0: Attached scsi generic sg2 type 0
sd 5:0:0:0: [sdb] 15728640 512-byte logical blocks: (8.05 GB/7.50 GiB)
sd 5:0:0:0: [sdb] Write Protect is off
sd 5:0:0:0: [sdb] Mode Sense: 0b 00 00 08
sd 5:0:0:0: [sdb] No Caching mode page found
sd 5:0:0:0: [sdb] Assuming drive cache: write through
 sdb:
sd 5:0:0:0: [sdb] Attached SCSI removable disk
```

Stickurile USB sunt, în general, montate automat de sistemul de operare. În Linux, montarea se face de regulă în subdirectoare din `/media`. Dacă montarea nu se face automat, atunci folosim utilitarul `mount` pentru a monta discul USB (ex. `/dev/sdb`) într-un punct de montare la alegerea noastră.

Deseori companiile care produc software ne pun la dispoziție produsele sub forma unui fișier cu extensia `.iso` destinat scrierii pe un CD/DVD. Pentru a accesa conținutul acestuia fără a-l scrie pe un suport fizic, putem folosi comanda **mount** și opțiunea `-o loop` prin care precizăm faptul că nu vom monta un disc fizic, ca în Listing 10.14.

```
student@uso:~$ sudo mkdir /mnt/iso
student@uso:~$ sudo mount -o loop test.iso /mnt/iso
student@uso:~$ ls -l /mnt/iso
total 108
-rw-rw-r-- 1 root root    14 May  2 14:28 CentOS_BuildTag
drwxr-xr-x 3 root root  2048 May  3 23:34 EFI
-rw-rw-r-- 1 root root   227 Aug 30  2017 EULA
-rw-rw-r-- 1 root root 18009 Dec 10  2015 GPL
drwxr-xr-x 3 root root  2048 May  3 23:45 images
drwxr-xr-x 2 root root  2048 May  3 23:34 isolinux
drwxr-xr-x 2 root root  2048 May  3 23:34 LiveOS
drwxrwxr-x 2 root root 71680 May  4 00:03 Packages
drwxrwxr-x 2 root root  4096 May  4 00:06 repodata
-rw-rw-r-- 1 root root  1690 Dec 10  2015 RPM-GPG-KEY-CentOS-7
```

```
15 -rw-rw-r-- 1 root root  1690 Dec 10  2015 RPM-GPG-KEY-CentOS-Testing-7
16 -r--r--r-- 1 root root  2883 May  4 00:07 TRANS.TBL
```

Listing 10.14: Montarea unui fișier .iso

O altă aplicație a noțiunii de montare o reprezintă accesibilitatea datelor peste rețea din sistemul nostru de fișiere. Dacă dispunem de un server la distanță, ce are serviciul de SSH funcțional, putem monta sistemul acestuia de fișiere folosind utilitarul **`sshfs`**, ca în Listing 10.15. În comenzile din Listing 10.15:

- am instalat pachetul `sshfs` (liniile 1-2)
- am creat punctul de montare `malus-mount/` (linia 3)
- am montat prin SSH directorul home (`/home/malus`) de la distanță (`malus@vmx.cs.pub.ro`) în punctul de montare (linia 4)
- am verificat montarea cu succes prin listarea conținutului din punctul de montare (liniile 5-6) și prin listarea montărilor (liniile 7-9)
- am demontat directorul montat de la distanță (linia 10)

```
1  student@uso:~$ sudo apt install sshfs
2  [...]
3  student@uso:~$ mkdir malus-mount
4  student@uso:~$ sshfs malus@vmx.cs.pub.ro:/home/malus malus-mount/
5  student@uso:~$ ls malus-mount/
6  extra  from-upb-to-ncsu  iOracle.git  ios-privacy-leaks  lib  prolog-data
     prolog-query-work  pwn3  scripts
7  student@uso:~$ mount
8  [...]
9  malus@vmx.cs.pub.ro:/home/malus on /home/student/malus-mount type fuse.
   sshfs (rw,nosuid,nodev,relatime,user_id=1000,group_id=1000)
10 student@uso:~$ sudo umount malus-mount
```

Listing 10.15: Folosirea sshfs

10.7.1 Crearea unui disc ce are ca suport un fișier

În acest studiu de caz ne propunem să creăm un fișier de dimensiune 4GB pe care să îl partiționăm, formatăm și montăm întocmai unui disc fizic. Astfel de fișiere sunt folosite ca discuri pentru mașina virtuală, cum este cazul `my.vdi` din Secțiunea 10.3, sau pentru teste cu sisteme de fișiere.

Pașii pe care îi vom urma sunt prezentați în Listing 10.16:

1. crearea discului - folosind utilitarul **`dd`** (liniile 1-4)
2. formatarea discului - folosind utilitarul **`mkfs.ext4`** (liniile 5-17)
3. montarea discului - folosind utilitarul **`mount`** (liniile 18-19)
4. verificarea montării (liniile 20-24)

```
1 student@uso:~$ dd if=/dev/zero of=usodisk bs=1M count=100
2 100+0 records in
3 100+0 records out
4 104857600 bytes (105 MB, 100 MiB) copied, 0.0706813 s, 1.5 GB/s
```

```
student@uso:~$ mkfs.ext4 usodisk
mke2fs 1.44.1 (24-Mar-2018)
Discarding device blocks: done
Creating filesystem with 102400 1k blocks and 25688 inodes
Filesystem UUID: 23673b79-b27e-4792-b597-9028012586c3
Superblock backups stored on blocks:
        8193, 24577, 40961, 57345, 73729

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

student@uso:~$ mkdir mount-uso
student@uso:~$ sudo mount usodisk mount-uso
student@uso:~$ mount
[...]
/home/student/usodisk on /home/student/mount-uso type ext4 (rw,relatime,
data=ordered)
student@uso:~$ ls mount-uso/
lost+found
student@uso:~$ sudo umount mount-uso
```

Listing 10.16: Disc în fișier

Utilitarul **dd** poate citi și scrie pe/de pe orice dispozitiv fizic sau fișier, având doi parametri centrali: `if=` pentru fluxul de intrare și `of=` pentru fluxul de ieșire. Alți parametri care pot fi specificați pentru a control fluxul de date sunt:

- `bs` (*block size*); dimensiunea unui bloc de date
- `count` - câte blocuri va scrie

10.8 Anexă: Logical Volume Manager

Procesul de partiționare se poate aplica doar asupra unui disc fizic. Dacă în sistem sunt montate două discuri, pentru fiecare din acestea trebuie realizată partiționarea. În sisteme de tip Linux, pentru a rezolva acest neajuns, a fost introdus conceptul de LVM (*Logical Volume Manager*). În cadrul acestuia există următoarele obiecte:

- *Physical Volume* - sunt discurile fizice alte sistemului asociate LVM
- *Volume Group* - format din unul sau mai multe discuri asociate anterior
- *Logical Volume* - alocate din spațiul disponibil într-unul din Volume Group-urile create anterior. Doar acesta este vizibil în sistemul de operare și poate fi formatat cu un sistem de fișiere

Atunci când instalați o distribuție de Linux, aveți opțiunea de a activa LVM. Utilitatea acestuia apare atunci când mai adăugați un disc și doriți să măriți dimensiunea partițiilor existente.

În exemplul următor vom investiga configurația unui sistem (numit `mamba`) care are activat LVM:

- Vom lista discurile fizice:

```
mamba:~# pvs
  PV         VG      Fmt  Attr PSize PFree
  /dev/md2   storage lvm2 a-   1.14t 130.47g
```

- Se observă un disc denumit **`/dev/md2`** cu capacitatea `1.14TB`. Vom lista acum volumele:

```
mamba:~# vgs
  VG      #PV #LV #SN Attr   VSize VFree
  storage   1   6   0 wz--n- 1.14t 130.47g
```

- Se observă un singur volum denumit `storage`. Observați din ce discuri fizice este compus:

```
mamba:~# pvdisplay /dev/md2
  --- Physical volume ---
  PV Name               /dev/md2
  VG Name               storage
  PV Size               1.14 TiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              299641
  Free PE               33401
  Allocated PE          266240
  PV UUID               M9WAFG-pf9x-Uo0j-7r8N-spOy-5evy-tt0cyc
```

- Vom lista acum volumele logice, cele care sunt vizibile sistemului de operare:

```
mamba:~# lvs
  LV       VG      Attr   LSize   Origin Snap%  Move Log Copy%
Convert
  home     storage -wi-ao  20.00g
  labs     storage -wi-ao 200.00g
  projects storage -wi-ao 200.00g
  rosedu   storage -wi-ao 100.00g
  swarm    storage -wi-ao 500.00g
  titan    storage -wi-ao  20.00g
```

- Observați că toate acestea sunt prezente în calea `/dev`:

```
mamba:~# ls -l /dev/mapper/storage-*
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-home ->
../dm-0
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-labs ->
../dm-2
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-projects
-> ../dm-3
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-rosedu ->
../dm-1
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-swarm ->
../dm-5
lrwxrwxrwx 1 root root 7 Aug  9 07:18 /dev/mapper/storage-titan ->
../dm-4
```

În Figura 10.14 este reprezentată grafic incluziunea dintre *Physical Volume*, *Volume Group* și *Logical Volume*.

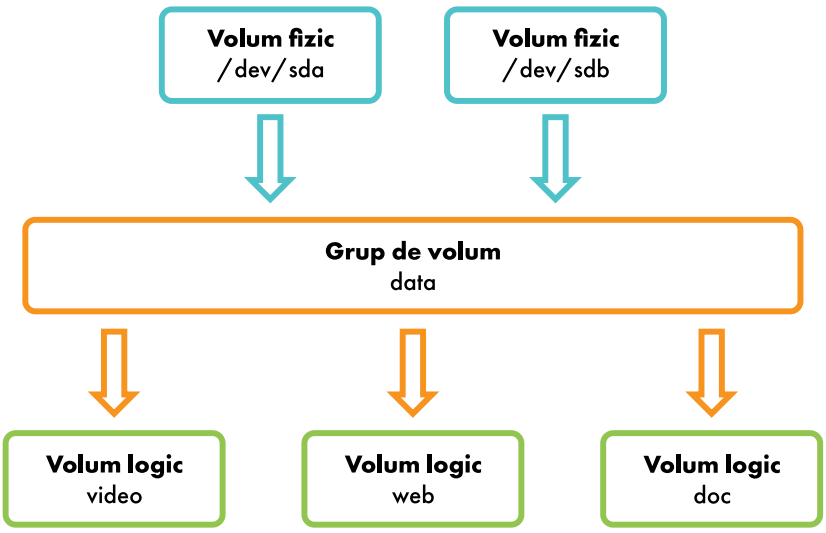


Figura 10.14: Componentele LVM

10.9 Anexă: Replicarea datelor

În general, datele salvate pe sistemele de stocare sunt critice. Defectarea sistemelor de stocare este iminentă mai ales din cauza faptului că HDD-urile sunt sensibile la șocuri, iar SSD-urile au un număr limitat de scrieri din proiectare. Pentru a preveni pierderea iremediabilă a datelor, datele sunt replicate prin diverse mecanisme pe mai multe unități de stocare.

Replicarea datelor pe discuri diferite se poate realiza automat la nivel de bloc de date atunci când acestea sunt trimise de către sistemul de operare. Mecanismul prin care se face este lucru se numește RAID (*Redundant Array of Independent / Inexpensive Disks*). Protocolul RAID poate fi implementat în hardware, de către controllerul de stocare, sau în software, de către sistemul de operare. Implementarea RAID din hardware dispune de un protocol proprietar, cu metadate diferite de la vendor la vendor. Așadar un volum (set de discuri) configurate în RAID hardware nu poate fi mutat pe un alt sistem.

RAID dispune de mai multe moduri de funcționare:

- RAID0 - datele sunt distribuite pe toate discurile configurate. Nu există nici un nivel de replicare (dacă avem două discuri configurate în RAID0 va rezulta un volum ce are capacitatea celor două discuri însumată, fără nici un nivel de replicare)
- RAID1 - datele sunt scrise simultan pe 2 din discurile configurate. Dacă un disc se va defecta, celălalt va deține informația (dacă avem două discuri configurate în RAID1 va rezulta un volum ce are capacitatea celui mai mic dintre discuri, cu un nivel de replicare a datelor)
- RAID5 - datele sunt scrise pe 1 disc și se mai scrie o sumă de control pe un alt disc. Cu ajutorul sumelor de control, datele pot fi recuperate dacă un disc pică. Necesită cel puțin 3 discuri. Poate pica un singur disc, iar datele rămân disponibile.
- RAID10 - combinația dintre RAID0 și RAID1 (datele sunt distribuite și replicate). Acesta asigură unul dintre cele mai bune niveluri de protecție și performanță crescută, cu dezavantajul unui cost mai crescut.

În Figura 10.15 sunt reprezentate grafic modurile de funcționare RAID prezentate mai

sus. Pe lângă acestea mai există și RAID6, care este asemănător RAID5, dar are două discuri pe care se reține sumele de control. Așadar pot pica 2 discuri la un moment dat, iar datele sunt disponibile în continuare.

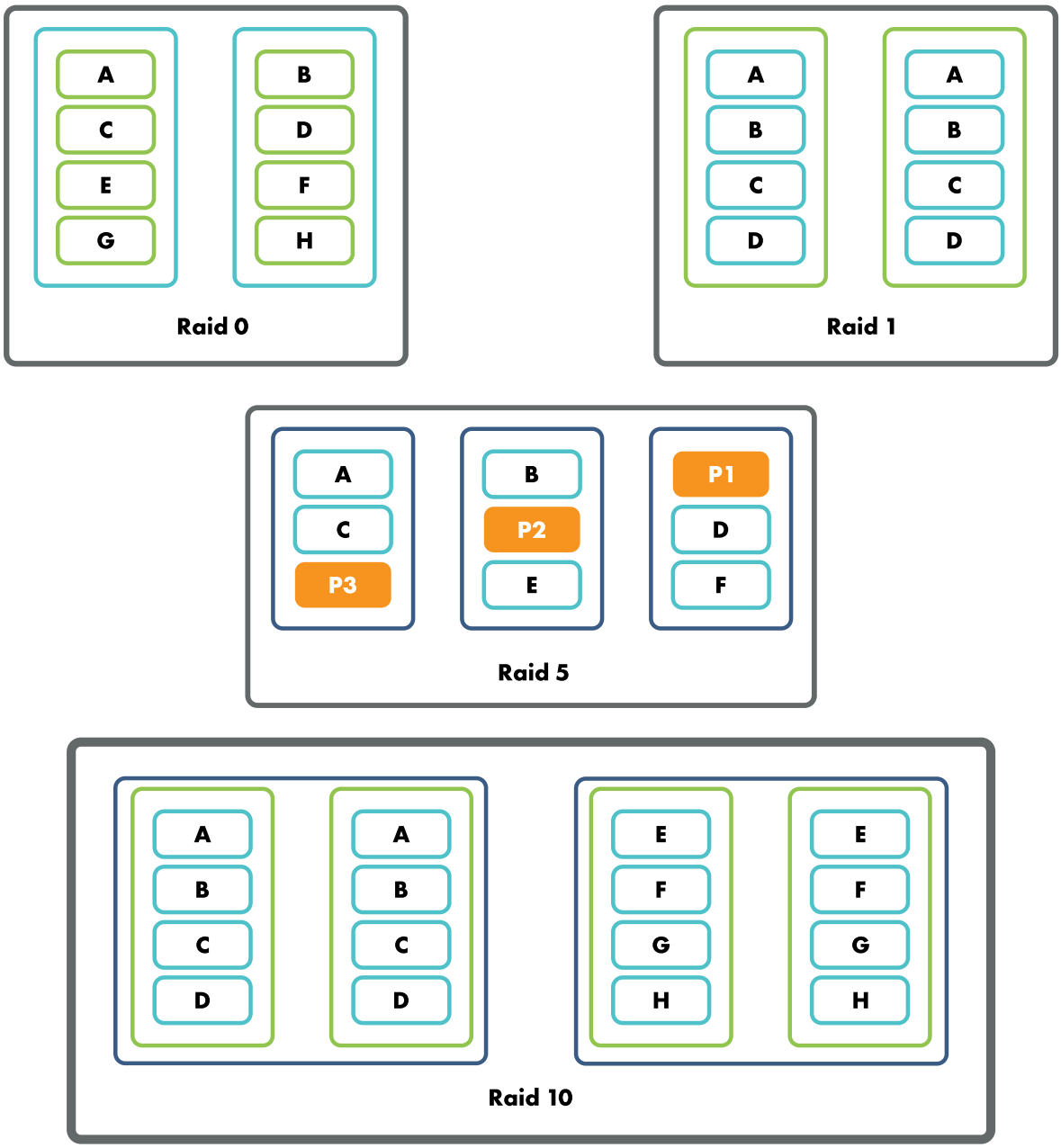


Figura 10.15: Niveluri RAID

În sistemele bazate pe Linux, software RAID este implementat prin modulul `md` (*Multiple Device*). Cu ajutorul acestuia se pot configura două sau mai multe discuri fizice să participe în formarea unei grupări RAID:

```
1 mamba:~# cat /proc/mdstat
2 Personalities : [raid1] [raid6] [raid5] [raid4]
3 md2 : active raid5 sda3[0] sdc3[2] sdb3[1]
4       1227333632 blocks level 5, 64k chunk, algorithm 2 [3/3] [UUU]
```

Se observă dispozitivul *md2*, configurat în mod RAID5, folosind partițiile `sda3`, `sdc3` și `sdb3`, toate fiind disponibile.

Replicarea folosind mecanismul RAID este utilă atunci când sistemul rulează și dorim protecție permanentă a datelor. Acest lucru este costisitor: pierdem spațiu util și performanța este mai scăzută. Mecanismul RAID se folosește cu precădere în sistemele de tip server. Pentru sistemele desktop/laptop se folosesc alte mecanisme de backup / replicare cum ar fi:

- crearea unei imagini a întregului disc
- snapshot folosind sistemul de fișiere
- copierea fișierelor pe un disc extern sau pe un alt dispozitiv peste rețea

Crearea unei imagini a întregului disc este utilă atunci când dorim să refacem rapid un sistem ce a fost afectat de un defect hardware sau de un virus (ex. `crypto-locker` - tot discul a fost criptat). Un alt caz în care este util să folosim imaginea unui disc este atunci când dorim replicarea aceleiași configurații pe mai multe sisteme identice. Un program ce vă poate ajuta să faceți acest lucru atât în Linux, cât și în Windows este Clonezilla[1].

Snapshot folosind sistemul de fișiere se poate face doar dacă acesta suportă. Cel mai cunoscut sistem de fișiere cu această facilitate este ZFS[2], disponibil în general pe sistemele server.

Copierea fișierelor pe un disc extern sau pe un alt dispozitiv peste rețea este o altă modalitate de replicare a datelor. Această metodă este utilă atunci când dorim salvarea unui director din tot sistemul de fișiere, sau atunci când dorim să facem replicare incrementală. Replicarea incrementală va transfera doar diferențele dintre fișiere, iar în acest mod se poate realiza și o versionare a acestora (se pot reține mai multe versiuni ale aceluiași fișier). Cel mai cunoscut utilitar pentru Linux în rezolvarea acestui lucru este **`rsync`**. Un exemplu gratuit pentru Windows îl constituie **`Cobian`**. Am prezentat aceste utilitare în Secțiunea 10.6.1, respectiv Secțiunea 10.6.2.

Copierea datelor se poate realiza pe un disc extern conectat pe USB sau pe un alt dispozitiv în rețea. În general în rețea avem alte servere pe care copiem datele sau avem dispozitive speciale de stocare denumite NAS (*Network Attached Storage*). Un NAS este un sistem de calcul cu un software specializat pentru accesarea și stocarea datelor. Acesta expune în rețea o multitudine de protocoale de acces (HTTP, FTP, NFS, CIFS) compatibile cu aproape toate sistemele de operare.

10.10 Sumar

Sistemul de calcul necesită o formă de stocare persistentă în care să fie reținute datele pentru aplicații și sistemul de operare. Aceste forme de stocare sunt discuri. În vremurile noastre discurile sunt în două forme: HDD (*Hard Disk Drive*) și SSD (*Solid State Drive*).

Discurile sunt partiționate pentru a fi folosite. O partiție reține, în general, informațiile de un anumit tip: informații de sistem, date ale utilizatorilor, informații pentru procesul de boot. Există două scheme de partiționare, folosite împreună cu tipurile de firmware de boot: schema MBR (*Master Boot Record*) și schema GPT (*GUID Partition Table*).

[1] https://clonezilla.org/

[2] https://www.freebsd.org/doc/handbook/zfs.html

O partiție este formatată cu un sistem de fișiere pentru a putea fi folosită. După formatare, sistemul de fișiere poate fi montat și se pot crea, modifica, șterge fișiere.

Pentru că datele sunt prețioase și pentru că pot exista defecte hardware la nivelul discurilor, este utilă replicarea datelor. Replicarea se poate face, în general, la nivelul blocurilor de date, transparent utilizatorului, folosind RAID (*Redundant Array of Independent / Inexpensive Disks*). Sau se poate face selectiv, la nivelul fișierelor / directoarelor, în controlul utilizatorului, folosind tehnici de backup.

Capitolul 11

Rețelistică și Internet

Internetul este o resursă indispensabilă în ziua de azi. Cu ajutorul Internetului putem comunica cu altcineva de pe altă parte a globului în timp real, putem urmări ultimele filme și seriale, putem colabora la distanță, putem descărca cea mai recentă versiune a unei aplicații, putem accesa și stoca cantități uriașe de informații. Internetul ne conectează între noi și la resursele diferitelor companii și organizații, simultan simplificându-ne activitățile personale și profesionale și adăugând un set nou de activități de realizat. Cu ajutorul Internetului putem să achiziționăm un bilet online, fără a fi nevoie de mers la o casă de bilete, așteptat la coadă, plătit cash sau cu cardul. De asemenea, putem apela la serviciul Uber cu plată online și rating în loc de a suna la un serviciu de taximetrie sau la transportul în comun; putem vedea un film direct pe Netflix, în loc de a aștepta achiziționarea unui disc Blu-ray cu filmul. Totodată, acum putem lucra la un proiect software la distanță, folosind GitHub, Slack, Skype, fără a fi nevoie de prezență fizică. Putem să transmitem informații instant unei echipe cu care lucrăm printr-un mesaj pe WhatsApp cu un link la un director partajat Google Drive. În perioada pandemiei Coronavirus (2020-2021), Internetul ne-a ajutat să menținem activitățile profesionale fără a fi necesară interacțiune fizică. Toate acestea și multe altele sunt posibile cu ajutorul Internetului.

Internetul aduce și o serie de provocări. Una dintre acestea este scalabilitatea: modul în care Internetul ajunge la cât mai mulți oameni și cât mai multe dispozitive în lume. În momentul apariției Internetului doar câteva echipamente erau conectate la Internet; apariția World Wide Web-ului a dus la o creștere foarte mare a acestora; dispozitivele mobile au dus la un nou val de creștere; lumea IoT (Internet of Things) (vezi Secțiunea 15.5) cu dispozitive de tot felul (senzori, sisteme de climatizare, becuri inteligente etc.) conectate la Internet e o nouă sursă de creștere. Scalabilitatea Internetului se referă la infrastructura de interconectare, la spațiul de stocare a cantității uriașe de informație și la viteza de transfer a datelor. Vom discuta aspecte legate de infrastructură în acest capitol. O altă provocare este securitatea și viața privată (*privacy*): interconectarea persoanelor înseamnă că informații particulare sunt livrate pe infrastructura Internetului și pot fi capturate la sursă, pe parcurs sau din spațiul unde sunt stocate. Vom discuta aceste aspecte în Capitolul 12.

În general, privim Internetul ca pe un mijloc prin care accesăm date, prin care apelăm la servicii la distanță și prin care comunicăm cu alții. Pentru mulți înseamnă că putem folosi o aplicație precum un browser web pe un sistem desktop sau WhatsApp pe un dispozitiv mobil. În limbaj tehnic, aceste informații și servicii nu fac parte din Internet;

accesul la aceste informații și servicii sunt facilitate de Internet. Companii precum Google sau Facebook au sisteme conectate la Internet care oferă servicii și informații pe care oamenii le pot accesa prin intermediul unor dispozitive conectate la Internet.

Spunem că Internetul este infrastructura care asigură interconectarea între sisteme/dispozitive la nivel planetar, ca în Figura 11.1. Mai precis, Internetul este ansamblul tuturor rețelelor de pe glob, interconectate între ele, ca în Figura 11.2. Fiecare sistem face parte dintr-o rețea. Această rețea este conectată la alte rețele care la rândul său sunt conectate la alte rețele și tot așa, rezultând o rețea planetară de rețele: Internetul.

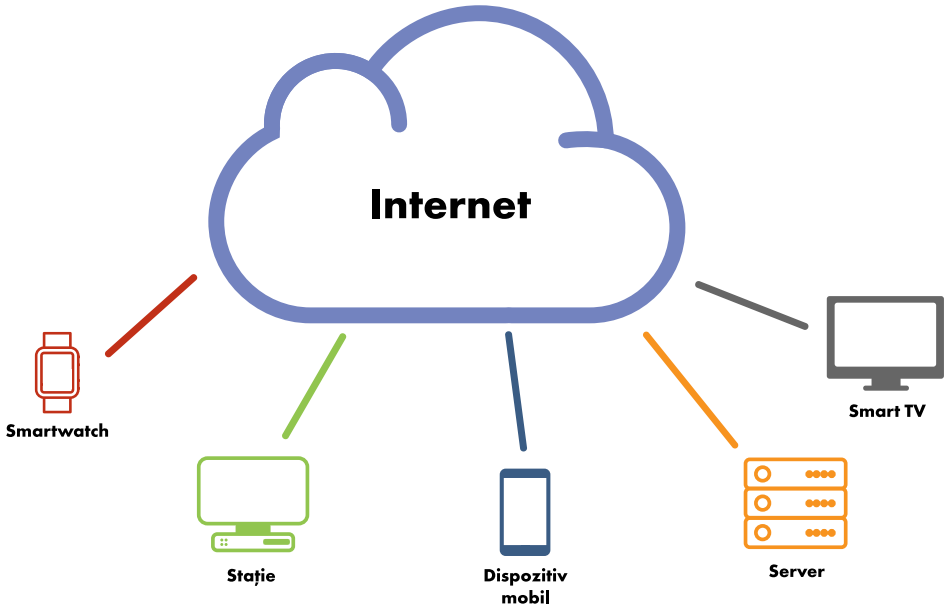


Figura 11.1: Internetul

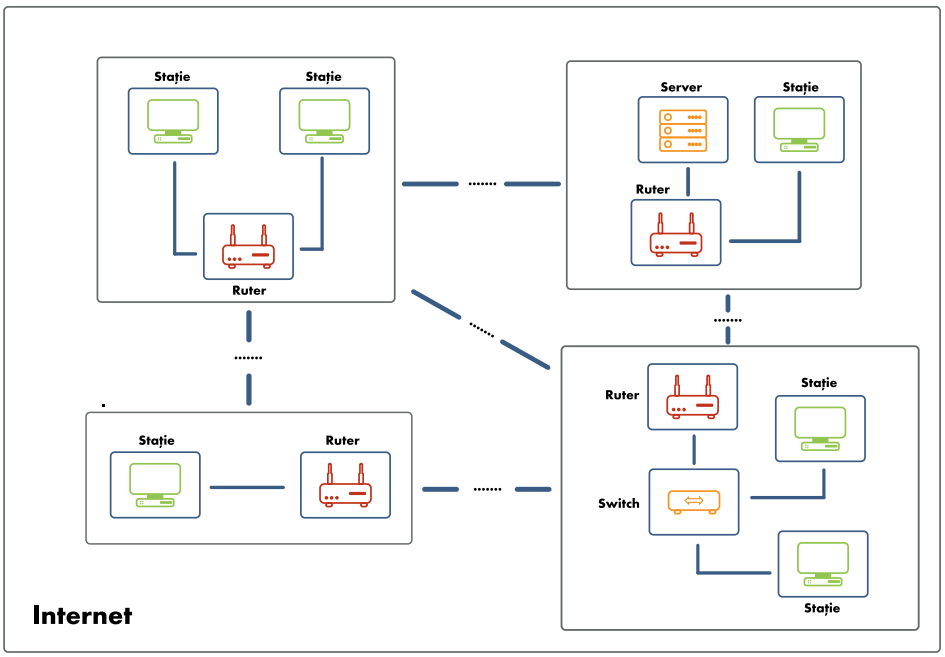


Figura 11.2: Internetul: rețea de rețele

Conectarea la Internet înseamnă conectarea la o rețea care este conectată la Internet, adică la o rețea care este conectată, direct sau indirect, la alte rețele. Atunci când

achiziționăm un ruter wireless și servicii de la un furnizor de servicii Internet, creăm o rețea în care conectăm laptopul, smart TV-ul, telefoanele mobile. Prin intermediul ruterului wireless ne conectăm la rețeaua furnizorului de servicii Internet, care este conectat la alte rețele, și tot așa. În felul acesta, avem legătură la Internet, adică la serviciile și informațiile furnizate de alte persoane sau organizații care au sisteme conectate.

O rețea, precum cea creată între ruterul wireless, laptop, smart TV, telefoane mobile, reprezintă o mulțime de dispozitive care pot comunica între ele, interconectate prin medii de transmisie (cablu, aer, fibră) și echipamente specifice de rețea (ruter, switch, access point). Vom detalia echipamentele de rețea, mediile de transmisie și modul de configurare a rețelei în secțiunile următoare.

11.1 Servicii de Internet

Pentru mulți utilizatori, Internetul este un spațiu de informație și de servicii. Un utilizator folosește o aplicație precum un browser web, care rulează pe un sistem conectat la Internet, pentru a accesa informații sau servicii puse la dispoziție de alte aplicații pe alte sisteme de pe Internet. De exemplu, Figura 11.3 prezintă un scenariu în care un utilizator, folosind un browser web, apelează prin Internet la serviciul Amazon Prime pentru a urmări un film.

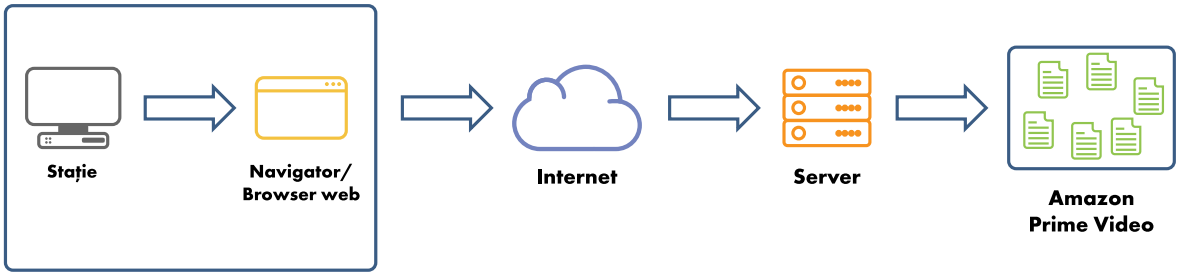


Figura 11.3: Folosirea serviciilor de Internet

Scenariul din Figura 11.3 este modul frecvent de accesare de informații și servicii în Internet. În aceste scenarii, aplicația și sistemul utilizatorului care realizează cererea se numesc la fel: **client**; în mod similar, aplicația și sistemul organizației care oferă serviciul se numesc la fel: **server**. Pentru claritate, se folosesc și termeni precum **aplicație client** sau **sistem client**, respectiv **aplicație server** sau **sistem server**.

Acest mod de comunicare se numește **paradigma client-server** și este forma folosită în comunicarea în Internet. În paradigma client-server o aplicație client solicită informații/un serviciu, iar o aplicație server îi oferă acel serviciu. Numim organizațiile care rulează astfel de servere furnizori de servicii/produse în Internet (*Internet-related services and products*). [1] Companii precum Google, Amazon, Facebook, Apple sunt astfel de organizații.

Prin intermediul infrastructurii furnizate de Internet, putem avea acces la o gamă largă de servicii. Acestea pot fi servicii clasice acum migrate pe infrastructura de Internet (de

[1] A nu se confunda această noțiune cu cea de furnizor de servicii Internet (*Internet Service Provider*). Aceasta se referă la companiile care ne oferă echipamentele și serviciile pentru a conecta la Internet.

exemplu, serviciul de tip video-on-demand furnizat de HBO GO, care înlocuiește achiziția de medii cu conținut video, precum un DVD cu un film; sau un transfer de bani direct prin Revolut în loc de transfer numerar/cash) sau servicii noi apărute în Internet (de exemplu, rețele sociale ca Facebook sau Pinterest sau spații colaborative precum GitHub sau Dropbox). În Tabelul 11.1 prezentăm tipuri de servicii din Internet cu exemple de aplicații și organizații care le furnizează. Acestea sunt doar câteva dintre tipurile de servicii și aplicații și organizații. Dezvoltarea a Internetului duce la o diversificare continuă: apar tipuri noi, apar aplicații și servicii noi, apar funcționalități noi la servicii existente.

Tabelul 11.1: Tipuri de servicii de Internet

Tip de serviciu	**Exemple de servicii**
acces la distanță	SSH (*Secure Shell*), Remote Desktop, VNC
acces de informații	WWW (*World Wide Web*), Wikipedia, Google
livrare de conținut (multimedia)	Netflix, YouTube, Amazon Prime, HBO GO, Spotify
comunicare / messaging	e-mail, forumuri, instant messaging, Facebook Messenger, WhatsApp, Slack, Signal, Telegram, Microsoft Teams
servicii online, intermediere tranzacții	Amazon, eBay, PayPal, Revolut, Glovo, Uber, AirBnb, Booking.com
divertisment	online gaming, Steam
spațiu de stocare	Google Drive, Microsoft One Drive, Dropbox
resurse de calcul, sisteme distribuite	Amazon EC2 / S3, Google Compute Engine, Microsoft Azure, Rackspace
colaborare	GitHub, Microsoft SharePoint, Google Drive, Trello

Așa cum este precizat în Figura 11.3, accesarea unui serviciu din Internet se realizează prin intermediul unei aplicații client de pe un sistem client al unui utilizator. De multe ori aplicația client este un browser web, datorită popularității folosirii serviciului web în Internet. Folosind un browser web putem accesa rețeaua de socializare Facebook, putem face achiziții pe Amazon, putem căuta informații folosind motorul de căutare Google, putem vizualiza cod pe GitHub, putem scrie și edita informație pe Wikipedia, putem partaja documente prin Dropbox, putem vedea conținut video pe YouTube, putem verifica e-mailul și altele. Browserul web devine astfel una dintre cele mai importante aplicații client de Internet.

Alternativa este folosirea unor aplicații client dedicate: un client de e-mail dedicat precum Mozilla Thunderbird, o aplicație (cu rol de client) pentru un joc de rețea precum Starcraft 2, o aplicație client dedicată pentru sincronizarea datelor Dropbox pe sistemul local, o aplicație Slack pentru comunicarea în cadrul echipei și altele.

Folosirea unei aplicații de tip browser web este obișnuită pe sisteme desktop / laptop. (un utilizator pornește un proces browser și accesează serviciile dorite), pe când pe sistemele mobile este mai comună folosirea unor aplicații dedicate (spre exemplu, accesarea Dropbox, Facebook, YouTube, e-mail, WhatsApp presupune o aplicație dedicată). Multe servicii pot fi accesate și prin browser web de pe dispozitivul mobil, dar de cele mai multe ori se folosesc aplicații dedicate.

Pentru mulți utilizatori, accesarea serviciilor de Internet este văzută ca un lucru simplu și gratis. Așteptarea este să poți căuta informație pe Internet folosind motorul de căutare Google, să poți asculta muzică folosind Spotify, să poți publica și vedea conținut Facebook, să poți comunica folosind WhatsApp, să poți stoca informație folosind Dropbox, toate acestea fără cost din partea utilizatorului.

Totuși, pentru organizațiile care oferă aceste servicii interesul este unul financiar, de a capitaliza din folosirea serviciilor pe care le oferă. Aceste organizații dezvoltă modele de afaceri (*business models*) diferite față de cele clasice; în modelul clasic, un client obține un bun de la un furnizor și plătește contravaloarea acelui bun; modelele de afaceri online (*online business models*) au alte forme de monetizare. Mai jos sunt câteva exemple:

- **servicii freemium/premium**: Serviciul de bază este gratis, dar anumite funcționalități sunt permise doar utilizatorilor care plătesc. De exemplu, Spotify Premium îți permite să asculți muzică fără acces la Internet și să selectezi melodiile dorite pe aplicația pe mobil. În mod similar, GitHub poate fi folosit pentru cod public; pentru surse private există un cost[1]. Dropbox este limitat la 2GB de spațiu de stocare în forma gratis (în 2021); pentru mai mult și alte funcționalități există un cost.
- **comision de tranzacții**: Pentru utilizator folosirea serviciului pare gratis, dar serviciul în sine aduce un comision transparent. Aplicații precum Uber, Glovo, Airbnb intermediază tranzacții între furnizori de transport, mâncare, cazare și obțin o cotă parte din tranzacțiile efectuate.
- **reclame țintite pe utilizatori**: Cu ajutorul unei baze largi de utilizator, servicii oferite de Google sau Facebook sau alte companii prezintă reclame utilizatorilor. Reclamele sunt plătite de alte companii care vor să vândă serviciile lor. Un punct cheie în acest model de afaceri este numărul mare de utilizatori, motiv pentru care, pentru organizații de acest fel, este important ca serviciile pe care le oferă să fie cât mai atractive. Un citat celebru în lumea digitală, care face referire la acest model de afaceri, este *If you aren't paying for it, you're the product.*

Multe organizații folosesc modele de afaceri multiple. De exemplu, Revolut încorporează toate modele de mai sus: comision de tranzacții efectuate prin Revolut, servicii premium (însemnând asigurări de călătorie și de transport, card premium, limite mai mari de operațiuni fără comision) și reclame țintite pentru utilizatori.

11.2 Funcționarea Internetului

Internetul este infrastructura ce realizează, la nivel planetar, legătura dintre sisteme și dispozitive. Așa cum indică Figura 11.1, aceste sisteme se găsesc la periferia Internetului. Unele sunt de tip client, altele de tip server. Mai numim aceste sisteme și stații (în engleză *hosts*, *network hosts*, *end nodes*). Internetul cuprinde și echipamentele care realizează conexiunea între aceste stații: dispozitive de interconectare (rutere, switch-uri, access pointuri) și medii de transmisie (cablu, aer, fibră optică).

Peste această infrastructură fizică a Internetului se realizează comunicarea între aplicații client și aplicații server, așa cum este prezentat în Figura 11.4.

Pentru a putea realiza comunicarea între stații în Internet, este nevoie de:

- un mod de adresare: stația A trebuie să poată preciza că vrea să comunice cu stația B sau stația C

[1] GitHub oferă studenților și organizațiilor educaționale posibilitatea de a folosi cod privat fără cost.

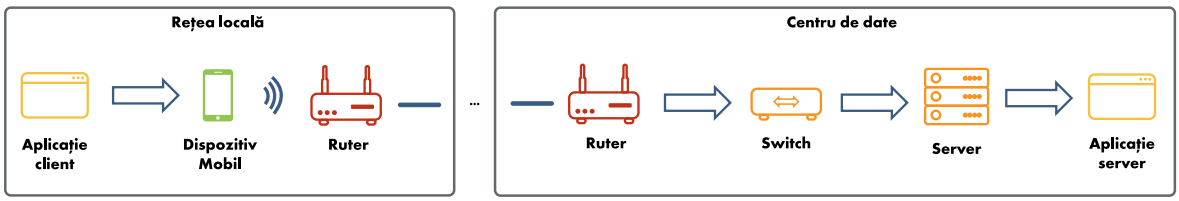


Figura 11.4: Infrastructura de conectare în Internet

- o formă de transmitere a datelor: datele comunicate trebuie să fie compartimentate și așezate pentru a fi transmise între stații
- un mecanism de dirijare a comunicației: pe baza adreselor furnizate, datele trebuie să fie dirijate de la stația sursă A la stația destinație B
- un mod comun acceptat de interacțiune: aplicația client trebuie să aibă un limbaj comun cu aplicația B

Aceste deziderate sunt realizate în Internet prin intermediul protocoalelor de comunicație. Un protocol cunoscut de stația A și stația B va permite interacțiunea între acestea. Un protocol este un set de reguli care guvernează comunicația între două sau mai multe componente interconectate. [1]

La nivelul Internetului se găsesc multe protocoale cu roluri diferite și complementare. Un sistem conectat la Internet va avea implementate mai multe protocoale care să asigure comunicarea cu alte sisteme din Internet. Spunem că, în Internet, sistemele implementează o stivă de protocoale, adică mai multe protocoale care sunt construite unele peste altele pentru a oferi interconectivitate. Stiva de protocoale folosită în Internet (adică stiva de facto în Internet) este **stiva TCP/IP**, numită astfel după cele mai cunoscute două protocoale folosite: TCP și IP. Vom discuta pe scurt protocolul IP în continuare, iar detaliat, împreună cu stiva TCP/IP, în Secțiunea 11.4.

11.2.1 Internet Protocol (IP)

IP (*Internet Protocol*) este cel mai important protocol din Internet. IP oferă **schema de adresare IP**, pe baza căreia se dirijează datele în Internet, și schema de compartimentare a datelor în unități numite **pachete IP** (*IP packets*). Stația sursă compartimentează datele în pachete, le transferă, iar stația destinație le recepționează și le reasamblează. Întrucât pachetele sunt modul universal de transferare a datelor în Internet, de acum încolo vom folosi adesea denumirea de pachete în loc de date.

Dirijarea pachetelor în Internet poartă numele de **rutare**. Rutarea este realizată de echipamente dedicate numite rutere. Pentru a putea dirija un pachet, un ruter are nevoie de informații privind sursa și destinația acestuia: de unde vine și cui îi este adresat. De aceea, orice pachet IP conține metadate (adresa stației sursă și adresa stației destinație) pe lângă datele efective. Un ruter preia un pachet, verifică adresa sursă și adresa destinație și ia decizia unde să îl transfere, în mod normal către alt ruter

[1] Un protocol nu se referă doar la rețele de date și Internet; de exemplu, în interacțiunea cu echipamente periferice un sistem poate folosi protocolul USB; la fel, poate folosi protocolul SCSI pentru a interfața cu un sistem de stocare.

aflat mai aproape de stația destinație. Figura 11.5 prezintă schematic modul de funcționare al unui ruter.

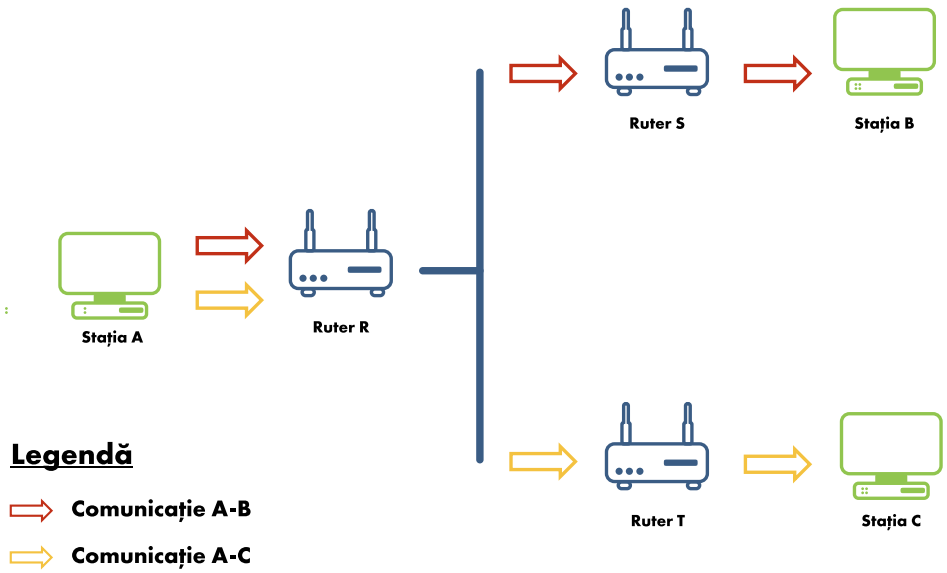


Figura 11.5: Rutarea pachetelor

Fiecare rețea cu stații (numită și **rețea locală**, sau *Local Area Network* - LAN) are un ruter care intermediază accesul la Internet. Acest ruter face primul pas în dirijarea pachetelor către destinație. Acest ruter este numit **gateway** (sau *default gateway*). De exemplu, în Figura 11.5, ruterul `R` este gateway-ul rețelei locale din care face parte stația `A`, ruterul `S` este gateway-ul rețelei locale din care face parte stația `B`, iar ruterul `T` este gateway-ul rețelei locale din care face parte stația `C`.

Așadar, comunicarea în Internet înseamnă compartimentarea datelor în pachete, adăugarea de metadate de adresare și rutarea pe baza metadatelor. Faptul că între stația sursă și stația destinație sunt mai multe echipamente intermediare poate fi un risc de securitate: pachetele pot fi interceptate și analizate sau modificate. De aceea, e important ca mesajele în Internet să fie criptate, folosind, de exemplu, protocolul HTTPS în loc de HTTP. Subiectul securității transferului îl vom trata în Secțiunea 12.4.

11.2.2 Nume în Internet. DNS

Atunci când un utilizator dorește să folosească un serviciu din Internet, trebuie să cunoască adresa destinație corespunzătoare stației server din Internet. Adresele IP sunt în general de forma `141.85.227.118` sau `216.58.207.174` sau `161.148.164.31`. Aceste adrese în format numeric sunt greu de reținut, similar numerelor de telefon. La fel, similar numerelor de telefon, este nevoie de o agendă prin care să avem intrări cu nume ușor de reținut și numere corespunzătoare.

Această „agendă” este implementată în Internet de **serviciul DNS** (*Domain Name System*). Cu ajutorul DNS putem folosi nume de stații (*DNS hostnames*) sau nume de domenii (*DNS domain names*), mai ușor de reținut. De exemplu, în loc de adresa `141.85.227.118` vom folosi numele `swarm.cs.pub.ro`, în loc de adresa `69.171.250.35` vom folosi `facebook.com`, iar în loc de adresa 161.148.164.31 vom folosi www.gov.br.

DNS nu este un termen prea cunoscut, pentru că folosirea DNS este transparentă utilizatorului de Internet. Utilizatorul pornește o aplicație precum un browser web, folosește numele `facebook.com`, iar aplicația va apela, în spate, la serviciul DNS pentru a afla adresa IP corespunzătoare numelui. Apoi aplicația se va conecta la serviciul de la acea adresă IP. Schematic, funcționarea DNS este indicată Figura 11.6.

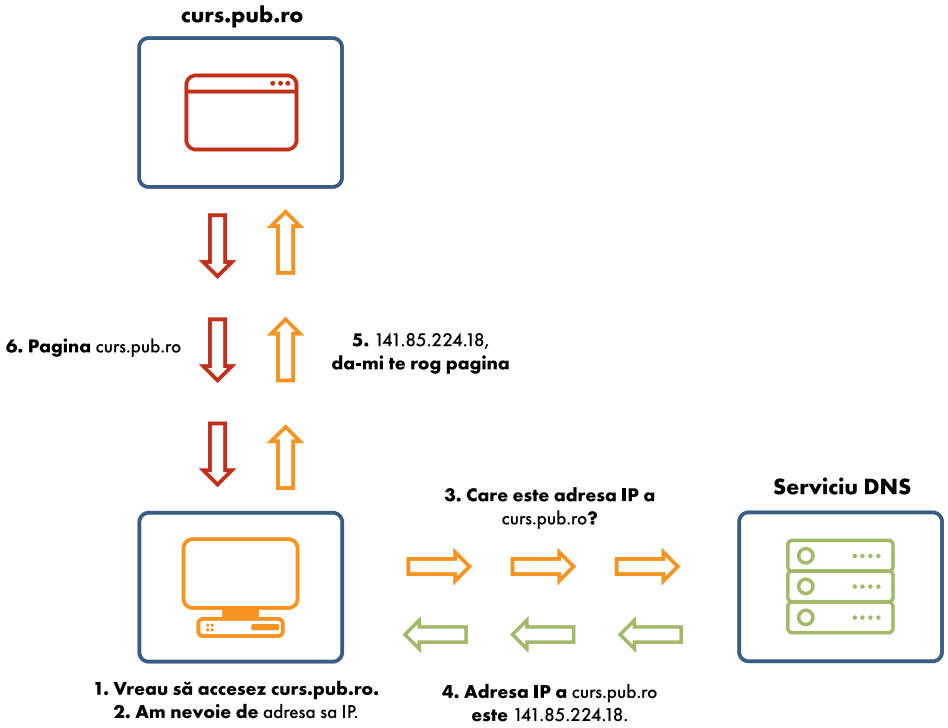


Figura 11.6: Funcționarea DNS

Serviciul DNS este implementat în Internet printr-un set de servere. Aceste servere rețin asocierile între nume și adrese IP. De aceea, fiecare sistem trebuie să aibă configurat un server DNS pe care să îl contacteze pentru informații legate de aceste asocieri. Atunci când o aplicație apelează la un server DNS, spunem că aplicația realizează o interogare DNS (*DNS query*). Serverul DNS configurat pe sistemul pe care rulează aplicația va răspunde interogării dacă are răspunsul, altfel va transmite cererea mai departe altor servere DNS, și tot așa până obține răspunsul, ca în Figura 11.7.

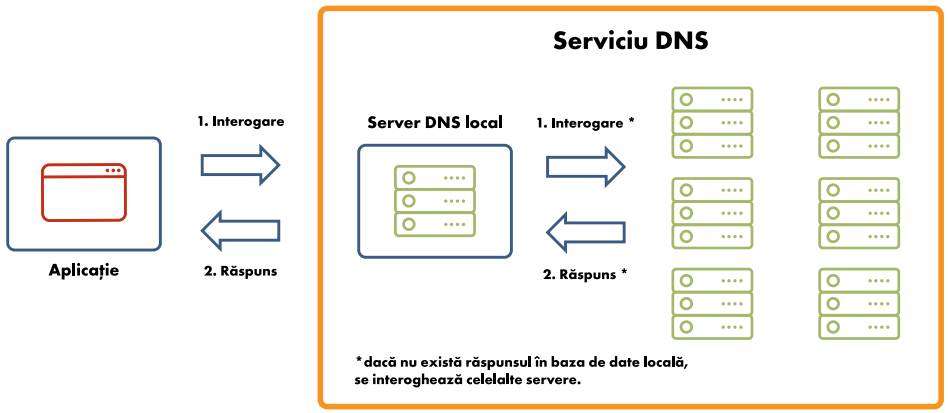


Figura 11.7: Implementarea serviciului DNS

Despre configurarea concretă a serviciului DNS (și a altor parametri de rețea) pe o stație vom vorbi în Secțiunea 11.5.

11.3 Echipamente de rețea

Așa cum am precizat în Secțiunea 11.2 și în Figura 11.4, pentru conectarea fizică a stațiilor între ele, la alte rețele și la Internet se folosesc echipamente de rețea. Clasificăm aceste echipamente în:

- dispozitive de rețea: switch-uri, rutere, access pointuri, firewalluri
- medii de transmisie: aer, cablu, fibră optică

Dispozitivele de rețea lucrează cu pachetele transmise de la stația sursă către stația destinație - le prelucrează și dirijează pentru a ajunge la destinație. De exemplu, un ruter urmărește adresele IP sursă și destinație pentru a ruta un pachet, un access point preia un pachet din mediul aer și îl transmite în mediul cablu, iar un firewall filtrează anumite pachete (nu trec mai departe). Unele dispozitive pot fi multi-rol: dirijare, filtrare, conversie între medii.

Mediile de transmisie sunt folosite pentru a transmite informația digitală (biți). Informația digitală este transmisă sub formă de semnal în funcție de mediul său. Tabelul 11.2 prezintă cele trei tipuri principale de medii de transmisie, tipurile de semnale folosite, vitezele uzuale de transmisie și avantajele și dezavantajele fiecăruia.

Tabelul 11.2: Medii de transmisie

Mediu	**Semnal**	**Viteză maximă**	**Avantaje**	**Dezavantaje**
cablu de cupru	electric	`10 Gbps`	cost redus, individual fiecărui sistem (nepartajat)	incomod
aer	electromagnetic	`300 Mbps`	flexibilitate	viteză redusă, partajat
fibră optică	optic (lumină)	`100 Gbps`	viteză mare	cost mare

O stație comunică printr-un mediu de transmisie cu un echipament de rețea: prin aer cu un access point, prin cablu sau fibră optică cu un switch sau un ruter. Pentru acesta, o stație / un dispozitiv are nevoie de o **placă de rețea** (*network interface card*, NIC). O placă de rețea este o componentă hardware parte a unei stații, componentă care preia datele de la aplicații ce rulează pe stație și le convertește în semnal specific mediului de transmisie. Astfel, o placă de rețea Ethernet va transmite datele în semnal electric pe un cablu de rețea, iar o placă wireless va transmite datele în semnal electromagnetic (unde) prin aer către un ruter wireless sau access point.

În general, stațiile au placă de rețea de tip Ethernet/LAN (*Local Area Network*) sau placă de rețea de tip WiFi/WLAN (*Wireless Area Network*). Dispozitivele mobile au placă de rețea WiFi integrată. Figura 11.8 prezintă o comunicare între un telefon mobil și un sistem desktop, în care telefonul mobil folosește o placă de rețea WiFi, iar sistemul desktop folosește o placă de rețea Ethernet.

11.4 Adresarea IP. Stiva TCP/IP

Peste infrastructura fizică a Internetului, formată din dispozitive de rețea și medii de transmisie, se stabilește legătura între stații. Această legătură, numită și **conexiune**

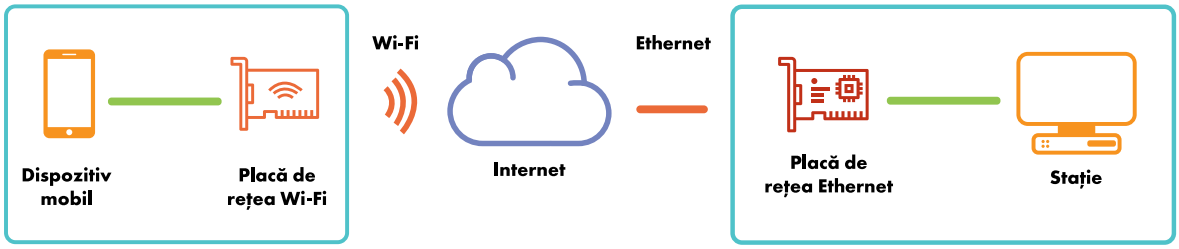


Figura 11.8: Dispozitive și medii folosite în comunicarea în Internet

logică, este realizată cu ajutorul **protocoalelor de rețea** (*network protocols*). Protocoalele de rețea sunt organizate într-o **stivă de rețea**; în cadrul stivei, un protocol de pe un nivel inferior oferă sprijin pentru protocoalele de pe nivelul superior. Stiva folosită în Internet este **stiva TCP/IP**, numită așa de la cele mai folosite protocoale (TCP și IP). Este prezentată în Figura 11.9.

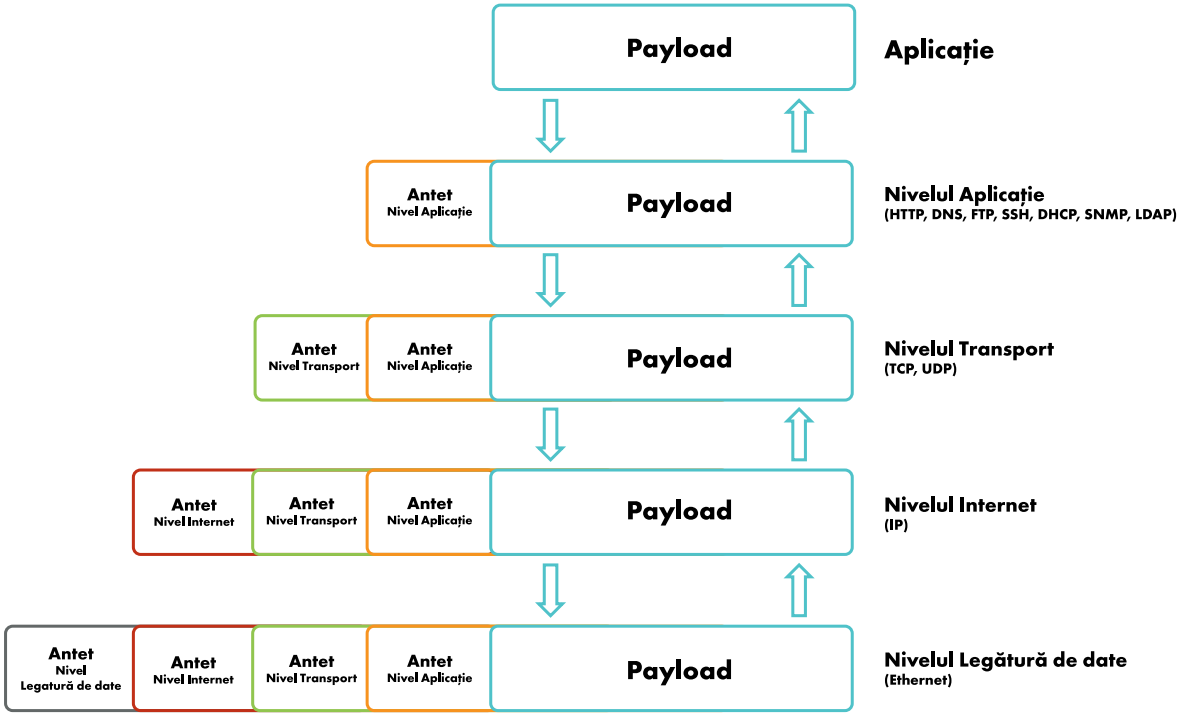


Figura 11.9: Stiva TCP/IP

Fiecare nivel din această stivă are un rol în construirea conexiunii între stația client și stația server. Fiecare nivel din stivă are o formă de adresare:

- Adresa de la nivelul **Legătură de date** este adresa MAC (*Media Access Control*). Este o adresă care identifică unic o placă de rețea, o adresă pe 48 de biți (6 octeți), de obicei definită de fabricantul plăcii de rețea.
- Adresa de la nivelul **Internet** este adresa IP (Internet Protocol), de la numele protocolului. Este o adresă ce poate fi configurată pe stație, identificând stația în Internet și fiind folosită în rutarea pachetelor.
- Adresa de la nivelul **Transport** este portul. Un port este un număr care identifică un proces / o aplicație de rețea pe acel sistem (este diferit de PID-ul procesului). Dacă un sistem are mai multe procese de rețea, sistemul va fi identificat de adresa IP, iar fiecare proces de rețea va fi identificat de port.

- La nivelul **Aplicație** adresele depind de protocol. Uzual, aceste adrese iau forma unui URI (*Uniform Resource Identifier*) care identifică o resursă în sistemul fizic, accesibilă prin protocolul respectiv. Un exemplu de URI este o adresă web de forma https://github.com/longld/peda.

11.4.1 Adresarea IP

Adresa IP este cea mai importantă adresă, pentru că identifică stația în Internet și este folosită pentru rutarea pachetelor. O adresă IP este scrisă, de obicei, în formatul zecimal cu puncte (*dotted decimal notation*) precum `141.85.224.118`. Tehnic, o adresă IP este un număr pe 32 de biți (4 octeți). Forma de reprezentare *dotted decimal* traduce fiecare octet într-un număr în zecimal între 0 și 255 și pune punct între octeți. Un exemplu de formă binară și formă *dotted decimal* a unei adrese IP este în Figura 11.10[1].

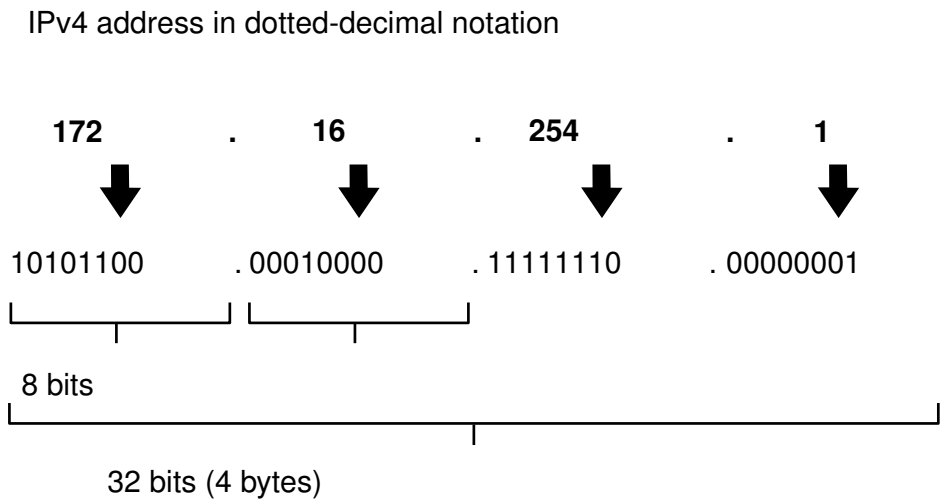


Figura 11.10: Adresă IP în format zecimal și binar

Când vorbim despre protocolul IP sau adresarea IP ne referim, în general, la versiunea 4 a protocolului, numită și IPv4. Aceasta este versiunea în care adresa IP ocupă 32 de biți. În ultimii ani, din cauza limitării adresei IPv4, a început folosirea la scară mai largă a versiunii 6 a protocolului: IPv6. La fel ca IPv4, IPv6 este folosit pentru adresare și rutare. Cea mai mare diferență este adresa, care ocupă 128 de biți (16 octeți) pentru IPv6. Adresele IPv6 nu mai sunt scrise în format *dotted decimal* ci în hexazecimal; se folosesc simplificări pentru a reduce spațiul ocupat de adrese. Un exemplu de adresă IPv6 este `2a00:1450:4016:807::200e`. În cazul IPv6 este cu atât mai necesară folosirea serviciului DNS; dacă în cazul IPv4 putem reține câteva adrese numerice, în cazul IPv6 devine foarte dificil din cauza dimensiunii extinse a acesteia.

Așa cum am precizat la începutul acestui capitol, Internetul este o interconectare a rețelelor la nivel planetar. Spunem că o stație este conectată la Internet, atunci când este conectată la o rețea care este conectată la alte rețele cu care fac parte din Internet. În general, spunem că orice stație face parte dintr-o rețea. Aceasta permite o organizare mai bună și mai scalabilă a Internetului: rutarea se realizează între rețele, nu între stații, urmând ca, într-o rețea, echipamentele specifice (de obicei switch-uri) să transmită pachetele stației destinație.

[1] https://commons.wikimedia.org/wiki/File:Ipv4_address.svg (domeniul public)

Din acest motiv, atunci când configurăm accesul la Internet pentru o stație, îi configurăm o adresă IP și precizăm din ce rețea face parte. O rețea are o adresă, pe care o numim adresa rețelei (*network address*). Adresa IP și adresa rețelei sunt legate: adresa de rețea este adresa IP a stației, dar în care ultimii biți au valoarea 0. De exemplu, într-o rețea avem adresa IP `192.168.53.100` pentru laptop, adresa `192.168.53.71` pentru smart TV și adresa `192.168.53.129` pentru telefon, iar adresa rețelei este `192.168.53.0`. Adică adresa rețelei este obținută din adresa oricărui dispozitiv din rețea înlocuind ultimul octet cu `0`.

Numărul de biți din adresa unei stații înlocuiți cu 0 depinde de la rețea la rețea. Pentru a determina acest lucru, folosim masca de rețea (*network mask*, *netmask*, *subnet mask*). Aceasta spune câți biți nu vor fi modificați și câți biți vor fi puși pe `0`. De exemplu, masca `/24` precizează că primii 24 de biți vor rămâne la fel și restul de 8 biți vor fi puși pe zero. Mai specific:

- Dacă avem adresa `192.168.53.129/24` (adică masca `/24`) înseamnă că adresa rețelei este `192.168.53.0`.
- Dacă avem adresa `192.168.53.129/16` (adică masca `/16`) înseamnă că adresa rețelei este `192.168.0.0`.
- Dacă avem adresa `192.168.53.129/8` (adică masca `/8`) înseamnă că adresa rețelei este `192.0.0.0`.

În general, un utilizator nu este preocupat de adresa de rețea și nici nu va face calcule. Adresa de rețea este determinată de echipamentele de rețea. Utilizatorul trebuie să furnizeze, la configurare, adresa IP și masca de rețea. Vom detalia procesul de configurarea efectuat de utilizator în Secțiunea 11.5.

11.4.2 Adresarea TCP

Adresa IP este folosită pentru a identifica stația în Internet. Pentru identificarea unei aplicații de rețea (client sau server) care rulează pe stație folosim **portul**, o adresă la nivelul Transport din stiva TCP/IP. Cel mai cunoscut protocol de la nivelul Transport este protocolul TCP (*Transmission Control Protocol*), iar adresele corespunzătoare sunt porturile TCP. De exemplu, o aplicație server web care rulează pe o stație folosește portul TCP 80. La fel, o aplicație server SSH (*Secure Shell*) (pentru conexiune sigură la distanță) folosește portul TCP 22.

Noțiunea de port este folosită adesea în reguli de filtrare prezente pe firewalluri, când vrem să blocăm anumite tipuri de trafic știind portul TCP destinație către care pachetele sunt transmise. De exemplu, dacă dorim să blocăm traficul SSH către rețeaua locală, vom configura pe firewall blocarea portului destinație TCP 22. Desigur, aceasta nu previne ca aplicația server SSH să fie configurată să folosească alt port, previne doar configurațiile implicite (care folosesc portul TCP 22).

Dacă dorim să aflăm care este portul implicit folosit de un serviciu de rețea, putem urmări, în Linux, fișierul `/etc/services`. Listing 11.1 conține o parte din fișierul `/etc/services`, cuprinzând cele mai frecvente nume de servicii și porturile pe care acestea le folosesc.

```
1 ftp             21/tcp
```

```
ssh             22/tcp                          # SSH Remote Login Protocol
telnet          23/tcp
smtp            25/tcp          mail
domain          53/tcp                          # Domain Name Server
domain          53/udp
http            80/tcp          www             # WorldWideWeb HTTP
pop3            110/tcp         pop-3           # POP version 3
ntp             123/tcp
ntp             123/udp                         # Network Time Protocol
imap2           143/tcp         imap            # Interim Mail Access P 2 and 4
snmp            161/tcp                         # Simple Net Mgmt Protocol
snmp            161/udp
bgp             179/tcp                         # Border Gateway Protocol
irc             194/tcp                         # Internet Relay Chat
ldap            389/tcp                 # Lightweight Directory Access Protocol
ldap            389/udp
https           443/tcp                         # http protocol over TLS/SSL
```

Listing 11.1: Servicii și porturi în Linux (fișierul /etc/services)

Prin folosirea porturilor, TCP permite coexistența mai multor aplicații de rețea pe aceeași stație. Pachetele care ajung la stația locală având adresa IP destinație corespunzătoare vor fi apoi livrate unei aplicații de rețea ce rulează pe stație pe baza portului: dacă portul destinație este 80, pachetele sunt livrate către serverul web; dacă portul destinație este 22 pachetele sunt livrate către serverul SSH. Spunem că porturile permit **multiplexarea serviciilor** de rețea pe aceeași stație.

Pe lângă beneficiul prezenței mai multor aplicații de rețea pe aceeași stație, TCP oferă și realizarea unui canal virtual de comunicare între două stații în Internet, adică garantează faptul că pachete transmise de la stația sursă la stația destinație și invers vor fi livrate și vor fi livrate în ordine. De unul singur, IP nu garantează aceste lucru, unele pachete putând fi pierdute sau ajungând în altă ordine. TCP are mecanisme interne implementate pe stația client și pe stația server care asigură livrarea corespunzătoare.

11.4.3 Limita de adrese IP

O adresă IPv4 ocupă 32 de biți. Aceasta înseamnă că există un număr maxim de circa 4 miliarde (2^{32}) de adrese IPv4. În realitate, numărul este mult mai mic, unele adrese fiind rezervate, altele fiind pierdute în organizarea adreselor în rețele, altele fiind folosite de echipamentele de rețea din Internet.

Pe măsura creșterii numărului de stații și dispozitive conectate la Internet, adresele IPv4 au început să fie epuizate. În zilele noastre sunt miliarde de dispozitive legate la Internet, în special prin intermediul tehnologiilor de tip IoT (*Internet of Things*), prin care dispozitive precum un bec inteligent sau un frigider sau un senzor au o adresă IP.

Pentru a contracara această limitare a protocolului IPv4, sunt prezente simultan în Internet în ziua de azi două soluții.

Prima soluție este folosirea IPv6 (*Internet Protocol version 6*) cu un spațiu de adresare mult mai generos. Această soluție are dezavantajul că trebuie să fie folosită de ambele părți; dacă serverul are adresă IPv6 dar clientul nu are (nu a primit de la furnizorul de servicii Internet), nu pot comunica.

O a doua soluție, mai puțin disruptivă, este folosirea NAT (*Network Address Translation*). Folosirea NAT înseamnă că o rețea locală primește o singură adresă IP. Această adresă IP este adresa publică a rețelei, o adresă care poate fi folosită în Internet pentru identificarea rețelei și pentru rutare. Toate stațiile din rețea folosesc adrese private, adrese care nu permit identificarea sau rutarea în Internet. Adresele private sunt rezervate în schema de adrese IP și sunt de forma `192.168.X.Y`, `172.16-31.X.Y` și `10.X.Y.Z`.

Când folosim NAT, ruterul rețelei este responsabil de traducerea adreselor. Se realizează două traduceri: din adrese private în adresa publică a rețelei, pentru a permite conectarea la Internet, și, invers, din adresa publică în adrese private pentru fiecare stație. Traducerile se aplică pachetele ce traversează ruterul între stații și Internet, așa cum este indicat în Figura 11.11.

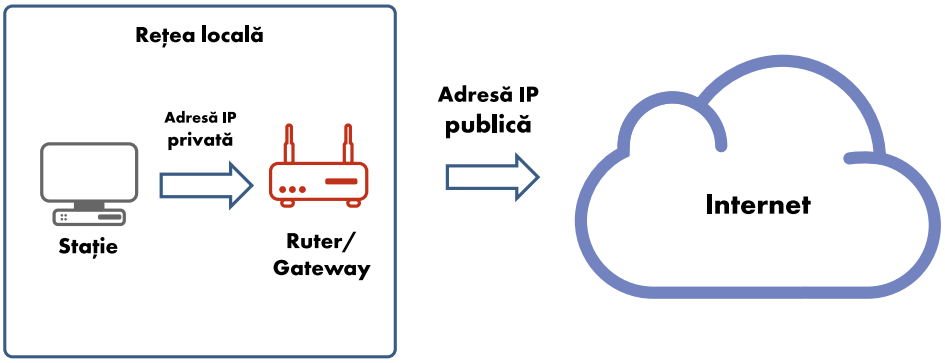


Figura 11.11: NAT: Translatare din adresă privată în publică

NAT are avantajul că se construiește pe infrastructura IPv4 existentă și nu necesită modificări. Dezavantajul este că este limitativ în ceea ce privește interconectarea aplicațiilor de tip server de pe stațiile din rețele private. Cel mai probabil, în rețeaua de acasă sau de la serviciu sau dintr-un local veți întâlni adrese IP private care apoi folosesc NAT pentru conexiunea la Internet. Dacă doriți să aflați adresa IP publică pe care o folosiți, puteți accesa site-ul https://www.whatismyip.com.

11.5 Configurări de rețea

Pentru a conecta o stație la Internet, fie aceasta un sistem laptop sau un telefon mobil sau un smart TV, trebuie să aibă elementele de infrastructură necesare și să configurăm parametrii de rețea corespunzători.

Elementele de infrastructură pentru conectarea unei stații la Internet sunt:

- stația trebuie să aibă o placă de rețea care să fie configurată pentru a permite conectarea
- stația trebuie să fie parte a unei rețele, conectate la Internet printr-un ruter, numit și gateway

- stația trebuie să aibă mediu de transmisie (aer, cablu, fibră optică) care să o conecteze la rețea
- trebuie să existe accesibil un server DNS pe care stația să îl poată interoga pentru translatarea numelor de domeniu în adrese IP

Pe stație apoi trebuie configurați parametrii de rețea corespunzători:

- adresa IP a stației, cu ajutorul căreia va fi identificată în rețea
- masca de rețea a rețelei, pentru determinarea adresei rețelei din care face parte
- adresa IP a gateway-ului care rutează pachetele și asigură legătura la Internet
- adresa IP a serverului DNS

În mod normal, elementele de infrastructură sunt instalate de un tehnician sau un angajat al furnizorului de servicii Internet. Configurarea, adică stabilirea celor patru parametri de rețea de mai sus, este responsabilitatea utilizatorului.

În mod obișnuit, un utilizator nu este conștient de acești parametri de rețea, configurarea acestora realizându-se automat prin intermediul DHCP (*Dynamic Host Configuration Protocol*) despre care vom discuta în secțiunile de mai jos. Pentru un utilizator cu profil tehnic și administrativ este importantă, însă, cunoașterea acestor parametri, a rolului lor și a modurilor de configurare. Cu aceste informații utilizatorul tehnic va putea depana probleme de rețea, va putea realiza configurații particulare, va putea crește nivelul de securitate și de performanță al rețelei.

În cele ce urmează, vom prezenta informații necesare unui utilizator pentru a investiga o configurație de rețea existentă sau pentru a face o configurație nouă.

În vreme ce investigația configurației poate fi făcută de regulă de orice utilizator din sistem, realizarea unei configurații poate fi realizată doar de un un utilizator cu permisiuni administrative. Adică orice utilizator poate afla adresa IP a sistemului sau adresa gateway-ului, dar doar un utilizator cu permisiuni administrative poate modifica adresa IP sau adresa gateway-ului.

11.5.1 Interfețe de rețea. Investigația configurației

Un sistem are nevoie de o placă de rețea pentru a permite conectarea la Internet. Parametrii de rețea vor fi configurați la nivelul acestei plăci de rețea. Pentru a permite configurarea, plăcile de rețea sunt expuse utilizatorului ca interfețe de rețea. O **interfață de rețea** este un mijloc de realizare a configurărilor de rețea, asociată de obicei unei plăci de rețea, și identificată printr-un nume; acest nume este folosit de utilitarele de rețea.

În Linux / Unix (inclusiv macOS), denumiri uzuale pentru interfețele de rețea sunt `eth0` (pentru interfețe Ethernet), `wlan0` (pentru interfețe corespunzătoare unor plăci de rețea WiFi), `wwan0` (pentru interfețe corespunzătoare unor modemuri 3G/4G/5G). Tradițional, în Linux, interfețele de rețea erau numite `eth0`, `eth1`, ... Pentru că denumirea nu era predictibilă între diferite porniri ale sistemului, în ultimii ani s-a optat pentru o denumire

deterministă, care să țină cont de geografia hardware. De exemplu, interfața `enp2s0` înseamnă că placa de rețea corespunzătoare este pe magistrala PCI 2 pe soclul 0[1].

În Windows interfețele sunt numite după formatul *Local Area Connection 1*.

În mod obișnuit, o interfață de rețea corespunde unei plăci de rețea; le mai numim interfețe de rețea fizice. Un sistem poate avea și interfețe de rețea virtuale, care nu au corespunzător o placă de rețea. Aceste interfețe de rețea virtuale pot servi mai multe roluri:

- pentru servicii și conexiuni locale: Interfața de *loopback* este o interfața prezentă pe majoritatea sistemelor de operare care referă sistemul însuși, auto-referință. Este numită `lo` în Linux.
- pentru testare: Dacă dorim să testăm un protocol sau un serviciu dorim să avem interfețe virtuale pentru testare
- pentru virtualizare: Interfața `vboxnet0` este interfața virtuală care face legătura între mașinile virtuale VirtualBox. La fel, interfața `vmnet8` este interfața virtuală care face legătura între mașinile virtuale VMware. Vom discuta despre virtualizare în Capitolul 14.
- pentru servicii și configurări specifice

În Listing 11.2 și Listing 11.3 folosim comenzi pentru a lista interfețele de rețea existente în sistem, împreună cu parametrii de rețea configurați pentru fiecare dintre acestea, respectiv în Linux și în Windows.

```
1  student@uso:~$ ip address show
2  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
    default qlen 1000
3      link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4      inet 127.0.0.1/8 scope host lo
5         valid_lft forever preferred_lft forever
6      inet6 ::1/128 scope host
7         valid_lft forever preferred_lft forever
8  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
9      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
10     inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute
   enp0s3
11        valid_lft 86329sec preferred_lft 86329sec
12     inet6 fe80::7bed:a436:51b8:70f/64 scope link noprefixroute
13        valid_lft forever preferred_lft forever
14 3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
15     link/ether 08:00:27:c0:83:50 brd ff:ff:ff:ff:ff:ff
16     inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic
   noprefixroute enp0s8
17        valid_lft 1125sec preferred_lft 1125sec
18     inet6 fe80::61bb:7aab:13f7:20fa/64 scope link noprefixroute
19        valid_lft forever preferred_lft forever
```

Listing 11.2: Listarea interfețelor de rețea în Linux

[1]Detalii despre denumirea plăcilor de rețea sunt aici: https://www.freedesktop.org/software/systemd/man/systemd.net-naming-scheme.html

```
C:\Users\Student User>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8af:b264:f9fa:f209%15
   IPv4 Address. . . . . . . . . . . : 192.168.56.103
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . : home
   Link-local IPv6 Address . . . . . : fe80::e1ae:5a1c:15a5:2440%14
   IPv4 Address. . . . . . . . . . . : 10.0.2.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.0.2.2
```

Listing 11.3: Listarea interfețelor de rețea în Windows

Din Listing 11.2 observăm că stația are o interfață virtuală (`lo`) cu adresa IP `127.0.0.1`, specifică interfeței virtuale, și două interfețe de rețea fizice: `enp0s3` și `enp0s8`. Cele două interfețe de rețea fizice au adresele IP `10.0.2.15/24` și `192.168.56.101/24`. Adresa IP corespunzătoare fiecărei interfețe cuprinde adresa efectivă și masca de rețea. În Linux, comanda **`ip address show`** afișează doar doi dintre cei patru parametri de rețea: adresa IP și masca de rețea. Pentru afișarea ruterului / gateway-ului, respectiv a serverului DNS, folosim comenzile din Listing 11.4 și Listing 11.5.

```
student@uso:~$ ip route show
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
metric 101
```

Listing 11.4: Afișarea gateway-ului (în Linux)

```
student@uso:~$ cat /etc/resolv.conf
[...]
nameserver 127.0.0.53
```

Listing 11.5: Afișarea serverului DNS (în Linux)

Gateway-ul (sau default gateway-ul) se găsește în rezultatul rulării comenzii **`ip route show`**, pe linia care conține șirul `default`. În rezultatul din Listing 11.4, gateway-ul are adresa IP `10.0.2.2`.

Serverul DNS se găsește în fișierul /etc/resolv.conf. Aici îl putem investiga și tot aici se poate configura. În rezultatul din Listing 11.5, serverul DNS are adresa IP `127.0.0.53`.

Așa cum este cazul pentru majoritatea rețelelor locale, adresa IP a stației este una privată, ruterul / gateway-ul folosind NAT pentru a traduce adresa privată în cea publică, rutabilă în Internet. Pentru a afla adresa IP publică, pe lângă vizitarea site-ului

https://www.whatismyip.com, în Linux se pot folosi comenzi precum cele din Listing 11.6. Ne folosim de o aplicație client web în linia de comandă (**`curl`**) pentru a interoga un server web (`ifconfig.me`, `icanhazip.com`) care ne transmite adresa IP publică. Adică, în cazul de față, `62.121.83.2`.

```
student@uso:~$ curl -s ifconfig.me
62.121.83.2
student@uso:~$ curl -s icanhazip.com
62.121.83.2
```

Listing 11.6: Aflarea adresei IP publice în linia de comandă

Adresa IP publică va diferi în funcție de plasarea stației într-o rețea sau alta. Dacă folosim un laptop acasă, vom avea o adresă IP publică; dacă îl vom folosi într-un local, vom avea o altă adresa IP publică; dacă îl vom folosi într-o universitate, vom avea o altă adresă IP publică.

Pentru a verifica corectitudinea configurației, testăm conectivitatea sistemului la Internet. În mod uzual, acest lucru îl facem pe majoritatea sistemelor de operare folosind comanda **`ping`**, ca în Listing 11.7. În Linux / Unix, comanda **`ping`** rulează neîntrerupt așa că e nevoie să folosim combinația de taste `Ctrl+c` pentru a opri comanda. În Listing 11.7, avem un răspuns pozitiv în care ni se transmite și cât timp a durat transmiterea pachetului de verificare a conectivității; o conexiune bună va afișa timpi de ordinul milisecundelor. În cazul unei probleme de configurare, răspunsul ar fi fost negativ ca în Listing 11.8.

```
student@uso:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
[...]
64 bytes from 8.8.8.8: icmp_seq=42 ttl=63 time=39.2 ms
64 bytes from 8.8.8.8: icmp_seq=43 ttl=63 time=31.0 ms
64 bytes from 8.8.8.8: icmp_seq=44 ttl=63 time=134 ms
64 bytes from 8.8.8.8: icmp_seq=45 ttl=63 time=38.6 ms
64 bytes from 8.8.8.8: icmp_seq=46 ttl=63 time=43.9 ms
64 bytes from 8.8.8.8: icmp_seq=47 ttl=63 time=43.0 ms
C-- 8.8.8.8 ping statistics --48 packets transmitted, 6 received, +15
errors, 87rtt min/avg/max/mdev = 31.056/55.118/134.693/35.829 ms


```

Listing 11.7: Verificarea conectivității (răspuns pozitiv)

```
student@uso:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
[...]
From 10.0.2.2 icmp_seq=27 Destination Net Unreachable
From 10.0.2.2 icmp_seq=28 Destination Net Unreachable
From 10.0.2.2 icmp_seq=29 Destination Net Unreachable
From 10.0.2.2 icmp_seq=30 Destination Net Unreachable
From 10.0.2.2 icmp_seq=31 Destination Net Unreachable
[...]
```

Listing 11.8: Verificarea conectivității (răspuns negativ)

Pentru depanarea problemelor de conectivitate, pe lângă afișarea parametrilor de rețea, putem folosi următoarele acțiuni:

- verificarea conectivității fizice a plăcii de rețea: Pe lângă a vedea dacă există un led activ în portul plăcii de rețea Ethernet unde este conectat cablul de rețea, putem folosi, în Linux, comanda **`ethtool`**, ca în Listing 11.9.
- verificarea rutării până la destinație: Astfel putem vedea dacă un ruter de pe parcurs nu funcționează corespunzător. Folosim, în Linux, utilitarul **`traceroute`** ca în Listing 11.10. În Windows, utilitarul echivalent este **`tracert`**.
- verificarea funcționării serviciului DNS: Pentru a verifica serviciul DNS folosim un utilitar de interogare DNS. În Linux, folosim utilitarul **`host`** ca în Listing 11.11; rol similar au și utilitarele **`dig`** și **`nslookup`**. În Windows, utilitarul echivalent este **`nslookup`**.

```
1 student@uso:~$ sudo ethtool enp0s3
2 [sudo] password for student:
3 Settings for enp0s3:
4   [...]
5         Link detected: yes
```

Listing 11.9: Verificarea conectivității fizice a plăcii de plăcii de rețea

```
1  student@uso:~$ traceroute google.com
2  traceroute to google.com (172.217.23.78), 30 hops max, 60 byte packets
3   1  _gateway (10.0.2.2)  0.379 ms  0.338 ms  0.271 ms
4   2  _gateway (192.168.0.1)  3.417 ms  4.085 ms  3.932 ms
5   3  * * *
6   4  95.77.36.177 (95.77.36.177)  27.792 ms  27.650 ms  28.110 ms
7   5  84.116.216.253 (84.116.216.253)  27.924 ms  37.078 ms  36.923 ms
8   6  213.46.170.134 (213.46.170.134)  60.449 ms  32.514 ms  54.865 ms
9   7  74.125.244.97 (74.125.244.97)  54.630 ms  37.562 ms 74.125.244.81
    (74.125.244.81)  35.782 ms
10  8  172.253.75.141 (172.253.75.141)  35.573 ms  45.178 ms  45.036 ms
11  9  muc12s06-in-f14.1e100.net (172.217.23.78)  66.423 ms  66.274 ms
    66.108 m
```

Listing 11.10: Verificarea rutării până la destinație

```
1 student@uso:~$ host amazon.com
2 amazon.com has address 176.32.103.205
3 amazon.com has address 205.251.242.103
4 amazon.com has address 54.239.28.85
5 amazon.com mail is handled by 5 amazon-smtp.amazon.com.
```

Listing 11.11: Verificarea funcționării serviciului DNS

11.5.2 DHCP

Rareori utilizatorul va configura cei patru parametri de rețea: adresă IP, mască de rețea, adresă IP gateway, server DNS. Pe sistemele și dispozitivele moderne, această configurație este realizată folosind DHCP (*Dynamic Host Configuration Protocol*). DHCP este un protocol prin intermediul căruia un server (numit server DHCP) transmite parametrii de configurare stațiilor dintr-o rețea. Pe lângă avantajul configurării automate a parametrilor, serverul DHCP are grijă ca stațiile să aibă adrese IP distincte, prevenind conflicte de adresare.

Din acest motiv, atunci când dorim să ne conectăm la o rețea pe fir, doar introducem cablul în portul plăcii de rețea, iar când ne conectăm la o rețea wireless doar alegem rețeaua wireless. În spate, stația va solicita serverului DHCP cei patru parametri de rețea care îi asigură conectivitate la Internet. Dezactivarea și reactivarea conexiunii la Internet duce la interogarea serverului DHCP pentru retransmiterea parametrilor. Aceasta este o primă formă de depanare dacă există probleme de conectivitate.

Serverul DHCP este, așadar, o componentă prezentă în majoritatea rețelelor locale.

În concluzie, într-o rețea obișnuită, administratorul unei rețele locale va configura, pe lângă mediile de transmisie necesare stațiilor, un ruter, un server DHCP, un server DNS și, de multe ori, un firewall. În multe rețele (în cam toate cele mici, de casă), serverul DHCP și firewallul sunt integrate pe ruter / gateway.

11.5.3 Tipuri de configurare

Atunci când folosim DHCP spunem că realizăm o **configurare dinamică** (sau automată) a parametrilor de rețea. Alternativa este **configurarea statică** (sau manuală) a parametrilor de rețea, în care utilizatorul precizează parametrii de rețea.

De exemplu, în Windows, pentru configurare, folosim interfața grafică la fel ca în Figura 11.12. Dacă optăm pentru opțiunea *Optain an IP address automatically*, configurarea se va realiza dinamic / automat prin DHCP. Altfel, în cazul opțiunii *Use the following IP address*, configurarea va fi făcută static / manual.

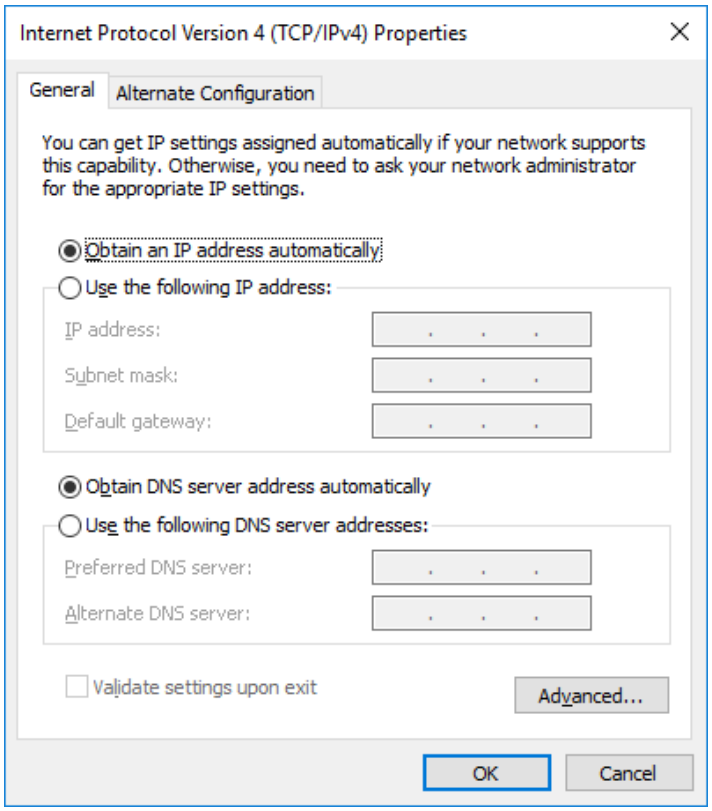


Figura 11.12: Configurarea rețelei în Windows

În mod obișnuit, tipul de configurare (dinamică sau statică) este salvat în sistem; astfel că, la repornirea sistemului, configurația de rețea se menține. Adică, în cazul

configurării manuale salvate, se mențin parametrii de rețea, iar în cazul configurării automate salvate, se interoghează serverul DHCP la repornirea sistemului. Este modul preferat de configurare pentru că, în general, ne dorim să menținem configurația la repornirea sistemului.

Pentru situații în care dorim să testăm configurații, sau pentru depanare, este util să realizăm **configurări temporare**, care pot fi ușor suprascrise și care nu se mențin la repornirea sistemului.

11.6 Configurarea rețelei în Linux

La fel ca în cazul celorlalte sisteme de operare, în Linux, investigarea parametrilor de rețea poate fi realizată de orice utilizator. De cealaltă parte, configurarea parametrilor poate fi realizată doar de un utilizator cu permisiuni administrative (adică de contul **`root`** sau folosind **`sudo`** sau alte forme de obținere de privilegii administrative). În mod tipic, utilizatorul implicit din interfața grafică de pe un sistem Linux are permisiuni de **`sudo`** și poate face astfel de operații.

11.6.1 Inspectarea configurației

Înainte de a realiza o configurație de rețea sau pentru a vedea că o configurație aplicată este corectă, un utilizator va inspecta configurația existentă. Pentru depanarea problemelor, utilizatorul va verifica conectivitatea. Am descris aceste acțiuni și modul de realizare a lor în Linux în Secțiunea 11.5.1. Sumarizăm mai jos cele mai frecvente acțiuni de inspectare a configurației și verificare a depanării:

- listarea interfețelor și a configurației interfețelor de rețea (adresă MAC, adresă IP și mască): Folosim comanda **`ip address show`**, prezentată în Listing 11.2.
- listarea tabelei de rutare (pentru aflarea adresei IP a gateway-ului): Folosim comanda **`ip route show`**: prezentată în Listing 11.4.
- afișarea serverelor DNS: Afișăm conținutul fișierului `/etc/resolv.conf`, prezentat în Listing 11.5.
- verificarea conectivității: Folosim utilitarul **`ping`**, prezentat în Listing 11.7.
- verificarea funcționării serviciului DNS Folosim utilitarul **`host`**, prezentat în Listing 11.11.
- verificarea ruterelor intermediare: Folosim utilitarul **`traceroute`**, prezentat în Listing 11.10.
- verificarea legăturii pe interfață: Folosim utilitarul **`ethtool`**, prezentat în Listing 11.9.

11.6.2 Configurarea grafică. NetworkManager

Pe un sistem desktop un utilizator va folosi de obicei interfața grafică pentru a realiza configurarea rețelei. Configurarea grafică este realizată în Linux cu ajutorul NetworkManager. NetworkManager este un serviciu care permite configurarea facilă a interfeței în Linux. Este compus dintr-un daemon care gestionează configurarea și interfețe grafice care facilitează configurarea, precum interfața din Figura 11.13.

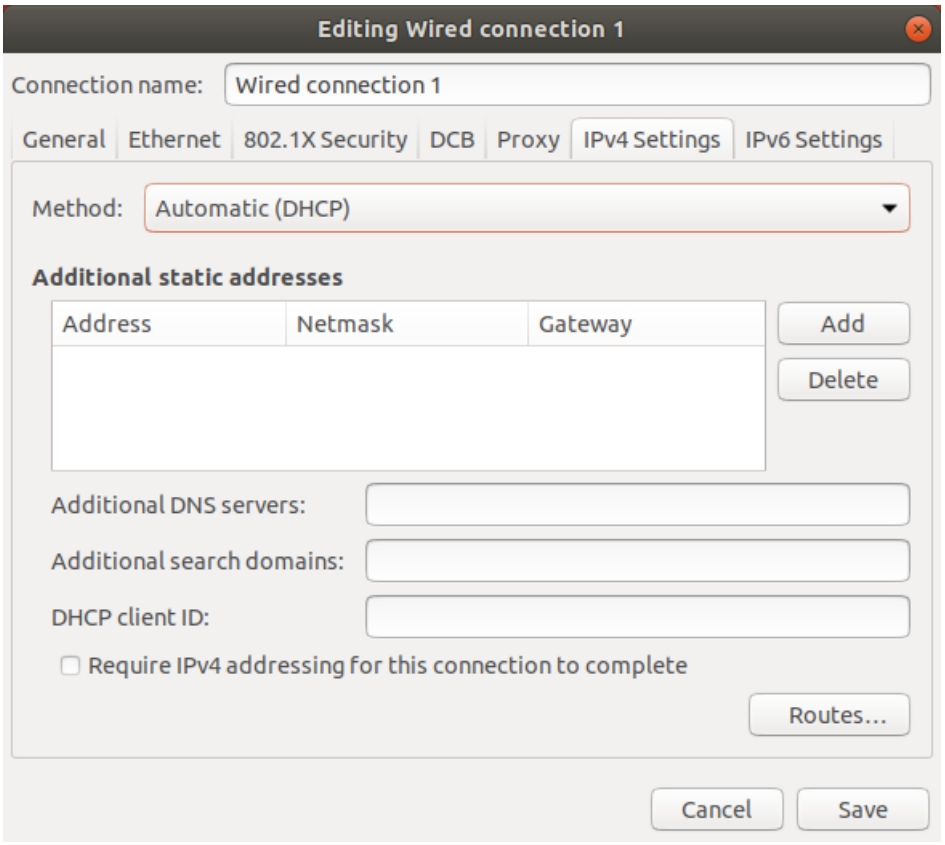


Figura 11.13: Configurarea grafică a rețelei în Linux (NetworkManager)

Interfața din Figura 11.13 poate fi pornită folosind comanda **`nm-connection-editor`**. Configurarea realizată cu ajutorul NetworkManger poate fi statică sau dinamică.

Pe un sistem fără interfață grafică, avem diferite opțiuni de configurare a rețelei, opțiuni care sunt influențate și de distribuție.

11.6.3 Configurarea DNS

În general, utilizatorul nu trebuie să fie preocupat de configurarea DNS. Serverul DNS este, de obicei, obținut automat prin DHCP. Într-un mediu desktop, NetworkManager se ocupă de configurarea DNS.

Pentru situații de depanare sau testare, un utilizator trebuie să aibă în vedere că DNS este configurat în fișierul `/etc/resolv.conf`, așa cum este exemplificat în Listing 11.5. În cadrul fișierului, serverele DNS (poate fi unul sau pot fi mai multe) sunt definite prin adresa IP pe liniile cu șirul `nameserver`.

În teorie, putem modifica sau adăuga noi servere DNS editând liniile cu șirul `nameserver` în fișierul `/etc/resolv.conf`. Dar, pe majoritatea distribuțiilor Linux recente (cu sau fără interfața grafică), fișierul `/etc/resolv.conf` este editat periodic de managerul de rețea (precum NetworkManager) sau de alte utilitare care sunt apelate individual sau de manager: **`resolvconf`**, **`systemd-resolved`**, **`dhclient`**. Modificările realizate în prealabil în fișier sunt suprascrise. De aceea, dacă este nevoie să configurăm manual servere DNS pentru un sistem, va trebui să știm ce alte utilitare modifică fișierul **`/etc/resolv.conf`**. Aceste utilitare vor suprascrie modificările manuale pe care le-am realizat. Avem două opțiuni:

- Configurăm respectivele utilitare (NetworkManager, **`resolvconf`**, **`systemd-resolved`**) pentru a putea configura serverele DNS. Nu mai editǎm manual fișierul `/etc/resolv/conf`, ci configurăm utilitarele / serviciile respective. Configurarea poate înseamna editarea unor fișiere de configurare, specifice utilitarului.
- Dezactivăm respectivele utilitare sau le configurăm să nu modifice fișierul **`/etc/resolv.conf`**. Apoi putem să editǎm manual fișierul `/etc/resolv.conf`.

Nu detaliem aici cum putem configura sau dezactiva utilitarele care editează fișierul `/etc/resolv.conf`. Configurarea depinde de distribuție, de componentele instalate și de configurația sistemului. Astfel de informații se pot găsi pe Internet.[1]

11.6.4 Configurarea în linia de comandă

Atunci când sistemul dispune de interfață grafică, configurăm rețeaua folosind NetworkManager. Configurarea în linia de comandă are loc pe acele sisteme în care nu avem interfață grafică. În general, vom folosi linia de comandă și pe un sistem cu interfața grafică, dar mai degrabă pentru inspectarea configurației, așa cum am precizat în Secțiunea 11.6.1. Cazurile în care vom folosi linia de comandă pentru configurarea rețelei înt-run sistem cu interfață grafică sunt limitate.

Configurarea în linia de comandă poate fi realizată în mai multe moduri, depinzând de opțiunile utilizatorului și de distribuția folosită. Unele metode sunt comune distribuțiilor, altele sunt specifice; fiecare distribuție oferă metode preferate de configurare.

11.6.4.1 nmtui și nmcli

Un prim mod de configurare a rețelei în linia de comandă, disponibil pe toate distribuțiile, este folosind utilitarele **`nmtui`** și **`nmcli`**, parte din NetworkManager. **`nmtui`** oferă o interfață cu ferestre text pentru editarea configurației de rețea, ca în Figura 11.14. **`nmcli`** este folosit preponderent în forma cu argumente în linia de comandă pentru inspectarea și editarea configurației. Cele două sunt utilitare de tip front end pentru serviciul NetworkManager, oferind aceleași opțiuni de configurare precum interfața grafică prezentată în Figura 11.13.

[1] Modurile în care putem configura utilitarele să nu modifice fișierul `/etc/resolv.conf` sunt descrise aici: https://www.ctrl.blog/entry/resolvconf-tutorial.html

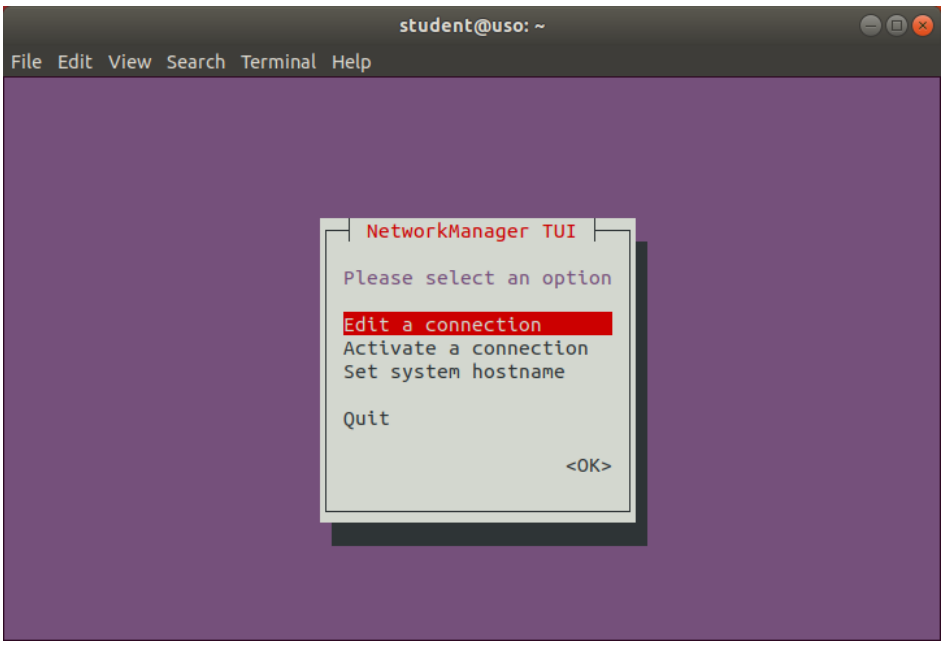


Figura 11.14: Configurarea rețelei în Linux folosind nmtui

11.6.4.2 Netplan (Ubuntu)

Pe distribuțiile Ubuntu recente (începând cu Ubuntu 18.04), configurarea rețelei în linia de comandă se realizează folosind Netplan[1]. Netplan oferă o interfață de fișiere de configurare în format YAML (*YAM Ain't Markup Language*). Utilizatorul / administratorul editează aceste fișiere de configurare, apoi aplică acea configurație. Aplicarea configurației duce la stabilirea parametrilor de rețea. Pasul de aplicare se realizează prin intermediul unui *renderer*. În mod implicit rendererul este NetworkManager.

Configurația Netplan implicită pe o instalare Ubuntu 18.04 cu interfață grafică este cea din fișierul `/etc/netplan/01-network-manager-all.yaml`, indicat în Listing 11.12.

```
1 student@uso:~$ cat /etc/netplan/01-network-manager-all.yaml
2 # Let NetworkManager manage all devices on this system
3 network:
4   version: 2
5     renderer: NetworkManager
```

Listing 11.12: Configure implicită

Dacă dorim o configurație specifică pentru interfața `enp0s3`, vom folosi un conținut precum cel din Listing 11.13. Folosind comanda **`netplan try`** (linia `14`), am verificat validitatea conținutului fișierului de configurare. Și, folosind **`netplan apply`** (linia `24`), am aplicat configurația.

```
1 student@uso:~$ cat /etc/netplan/01-network-manager-all.yaml
2 # Let NetworkManager manage all devices on this system
3 network:
4   version: 2
5   renderer: NetworkManager
6   ethernets:
```

[1] https://netplan.io/

```
    enp0s3:
      addresses:
        - 192.168.0.7/24
      gateway4: 192.168.0.1
      nameservers:
        addresses: [8.8.8.8, 4.4.4.4]

student@uso:~$ sudo netplan try
Do you want to keep these settings?


Press ENTER before the timeout to accept the new configuration


Changes will revert in 119 seconds
Configuration accepted.

student@uso:~$ sudo netplan apply

student@uso:~$ ip a s enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.7/24 brd 192.168.0.255 scope global noprefixroute
enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe3a:a901/64 scope link
       valid_lft forever preferred_lft forever

student@uso:~$ ip r s
default via 192.168.0.1 dev enp0s3 proto static metric 20100
192.168.0.0/24 dev enp0s3 proto kernel scope link src 192.168.0.7 metric
100
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
metric 101
```

Listing 11.13: Configure implicită

Informații extinse despre configurarea rețelei în linia de comandă în Ubuntu se găsesc în documentația oficială (https://ubuntu.com/server/docs/network-configuration).

11.6.4.3 ifupdown (Debian / Ubuntu)

Modul clasic de configurare în linia de comandă în distribuțiile Debian / Ubuntu este folosind pachetul `ifupdown`. În distribuțiile mai noi Ubuntu, se preferă folosirea Netplan. În distribuțiile Debian mai noi, se preferă folosirea Systemd-Networkd[1].

În cazul folosirii `ifupdown`, configurațiile de rețea se rețin în fișiere din directorul `/etc/network/`. Configurațiile fundamentale de rețea se rețin în fișierul `/etc/network/interfaces`. Listing 11.14 conține un exemplu de configurație a interfeței `eth0`. Linia `2` se precizează că este vorba de o configurație statică a interfeței. Apoi se precizează adresa IP (inclusiv masca) și gateway-ul.

```
auto eth0
iface eth0 inet static
```

[1] https://wiki.debian.org/SystemdNetworkd

```
3         address 192.168.0.7/24
4         gateway 192.168.0.1
```

Listing 11.14: Configurație ifupdown în /etc/network/interfaces

Configurațiile sunt aplicate cu ajutorul comenzilor **`ifup`** și **`ifdown`**. Aceste comenzi sunt invocate pentru aplicarea configurației din fișierul **`/etc/network/interfaces`**. **`ifup`** activează configurația și interfața, în vreme ce **`ifdown`** deconfigurează și dezactivează interfața. Ambele comenzi pot primi ca parametru numele interfeței, sau pot primi opțiunea `-a`, caz în care se aplică pe toate interfețele marcate cu `auto` în fișierul `/etc/network/interfaces`, cum este și cazul liniei `1` din Listing 11.14.

`ifupdown` poate coexista cu NetworkManager. În configurația implicită a NetworkManager, orice interfețe prezente în fișierul `/etc/network/interfaces` sunt excluse din configurația NetworkManager.

Informații extinse despre configurarea rețelei în Debian se găsesc în documentația oficială (https://wiki.debian.org/NetworkConfiguration, https://www.debian.org/doc/manuals/debian-reference/ch05.en.html).

11.6.5 ifcfg (Fedora / RedHat)

Modul clasic de configurare în linia de comandă în distribuțiile Fedora / Redhat este folosind `ifcfg`. Similar Netplan și `ifupdown`, `ifcfg` presupune realizarea de configurații în fișiere specifice.

Configurațiile se realizează, de regulă, în fișiere din directorul `/etc/sysconfig/network-scripts/`. Dacă dorim să configurăm interfața `enp0s3` vom crea și edita fișierul `/etc/sysconfig/network-scripts/ifcfg-enp0s3`, ca în Listing 11.15. Configurația conține linii de forma `<cheie>=<valoare>`. De exemplu, `DEVICE` va retine numele interfeței, `BOOTPROTO` tipul configurației, `IPADDR` adresa IP, iar `GATEWAY` adresa IP a gateway-ului.

```
1 [student@uso:~]$ cat /etc/sysconfig/network-scripts/ifcfg-enp0s3
2 DEVICE=enp0s3
3 TYPE=Ethernet
4 BOOTPROTO=non
5 NAME=enp0s3
6 ONBOOT=yes
7 PREFIX=24
8 IPADDR=192.168.0.7
9 GATEWAY=192.168.0.1
```

Listing 11.15: Configurație ifcfg

Activarea și dezactivarea configurației se realizează cu ajutorul serviciului de networking folosind comenzi precum **`sudo service network start`**, **`sudo service network stop`**, **`sudo service network restart`**. Dacă pe sistem rulează NetworkManager, atunci putem dezactiva NetworkManager la nivelul interfeței. Pentru aceasta, în fișierul de configurare al intefeței (de exemplu `/etc/sysconfig/network-scripts/ifcfg-eth0`), adăugăm linia de configurare `NM_CONTROLLED=no`.

Informații extinse despre configurarea rețelei în Fedora / RedHat se găsesc în documentația Fedora (https://docs.fedoraproject.org/en-US/Fedora/25/html/Networking_Guide/ch-Configure_Networking.html, https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/networking_guide/ch-configuring_ip_networking).

11.6.6 Interfețe active/inactive

Pentru a putea folosi o configurație, interfața corespunzătoare trebuie să fie activă. Pentru a activa, dezactiva și verifica o interfață folosim comanda ip link ca în Listing 11.16.

Folosim comanda **`ip link show`** pentru a afișa informații despre legătura interfețelor sistemului: dacă sunt active, adresa MAC, unitatea de transmitere (MTU - *Maximum Transmission Unit*) etc. În mod implicit se afișează informații despre toate interfețele sistemului (liniile `1-7`); se pot afișa informații despre o singură interfață prin transmiterea numelui acelei interfețe ca argument (liniile `11-13` și `17-19`). Atunci când rezultatul rulării comenzii de afișare conține șirul *state UP*, interfața este activă; altfel, dacă șirul conținut este *state DOWN*, interfața este inactivă. Folosind comanda **`ip link set dev enp0s3 down`** (în mod privilegiat, linia `10`) dezactivăm interfața `enp0s3`, iar folosind comanda **`ip link set dev eth0 up`** (în mod privilegiat, linia `15`) activăm interfața. Apoi starea interfeței va fi actualizată în `DOWN`, respectiv `UP`, în rezultatul rulării comenzi ip link show.

```
1  student@uso:~$ ip link show
2  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
   DEFAULT group default qlen 1000
3      link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP mode DEFAULT group default qlen 1000
5      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
6  3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP mode DEFAULT group default qlen 1000
7      link/ether 08:00:27:c0:83:50 brd ff:ff:ff:ff:ff:ff
8
9  student@uso:~$ sudo ip link set dev enp0s3 down
10
11 student@uso:~$ ip link show enp0s3
12 2: enp0s3: <BROADCAST,MULTICAST> mtu 1500 qdisc fq_codel state DOWN mode
   DEFAULT group default qlen 1000
13     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
14
15 student@uso:~$ sudo ip link set dev enp0s3 up
16
17 student@uso:~$ ip link show enp0s3
18 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP mode DEFAULT group default qlen 1000
19     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
```

Listing 11.16: Activarea și dezactivarea unei interfețe

11.6.7 Configurarea temporară. Comanda ip

Configurarea temporară în Linux se realizează în linia de comandă și este universală pe toate tipurile de distribuții. Configurarea temporară se realizează cu ajutorul utilitarul **ip**. Am folosit utilitarul **ip** în Secțiunea 11.5.1 pentru investigarea configurației și în Secțiunea 11.6.6 pentru activarea / dezactivarea unei interfețe.

Comenzile de configurare a interfețelor de rețea sunt comenzi privilegiate. De aceea, în secvențele de comenzi pe care le vom prezenta, acestea sunt prefixate de comanda **sudo**.

Înainte de a configura temporar parametrii de rețea, se recomandă să se șteargă configurațiile anterioare. Acest lucru poate fi făcut cu ajutorul comenzii **ip address flush**, ca în Listing 11.17. După rularea comenzii **ip address flush** (în mod privilegiat), orice configurații anterioare ale interfeței sunt șterse.

```
1  student@uso:~$ ip address show enp0s3
2  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
3      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
4      inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute
   enp0s3
5         valid_lft 76576sec preferred_lft 76576sec
6
7  student@uso:~$ sudo ip address flush enp0s3
8
9  student@uso:~$ ip address show enp0s3
10 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
11     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
```

Listing 11.17: Ștergerea configurațiilor unei interfețe

Pentru configurarea unei adrese IP și a unei măști pe o interfață folosim comanda **ip address add**. În Listing 11.18 avem exemple de configurare a adresei IP **192.168.0.7/24** pe interfața `enp0s3`. În prelabil, am șters configurația interfeței `enp0s3` folosind comanda **ip address flush**.

```
1  student@uso:~$ sudo ip address flush enp0s3
2
3  student@uso:~$ ip address show enp0s3
4  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
5      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
6
7  student@uso:~$ sudo ip address add 192.168.0.7/24 dev enp0s3
8  student@uso:~$ ip address show enp0s3
9  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
10     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
11     inet 192.168.0.7/24 scope global enp0s3
12        valid_lft forever preferred_lft forever
```

Listing 11.18: Configurarea unei adrese pe o interfață

Comanda **ip address add**, așa cum îi spune numele, adaugă o nouă adresă IP interfeței de rețea. O interfață de rețea poate avea astfel mai multe adrese IP. În Listing 11.19, ștergem toate configurația pe interfața de rețea `enp0s3` (linia `1`),

adăugăm două adrese IP pe o interfață de rețea (liniile 7–9) și apoi o ștergem pe una dintre ele folosind comanda **ip address del** (linia 19).

```
1  student@uso:~$ sudo ip address flush enp0s3
2
3  student@uso:~$ ip address show enp0s3
4  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
5      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
6
7  student@uso:~$ sudo ip address add 192.168.0.7/24 dev enp0s3
8
9  student@uso:~$ sudo ip address add 172.16.19.7/24 dev enp0s3
10
11 student@uso:~$ ip address show enp0s3
12 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
13     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
14     inet 192.168.0.7/24 scope global enp0s3
15        valid_lft forever preferred_lft forever
16     inet 172.16.19.7/24 scope global enp0s3
17        valid_lft forever preferred_lft forever
18
19 student@uso:~$ sudo ip address del 172.16.19.7/24 dev enp0s3
20
21 student@uso:~$ ip address show enp0s3
22 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
23     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
24     inet 192.168.0.7/24 scope global enp0s3
25        valid_lft forever preferred_lft forever
```

Listing 11.19: Ștergerea unei adrese de pe o interfață

Pentru configurarea gateway-ului folosim comanda **ip route add default**. În mod similar, dacă dorim să ștergem configurația gateway-ului folosim comanda **ip route del**. În Listing 11.20 avem exemplu de configurare a gateway-ului cu adresa IP **192.168.0.1**.

```
1  student@uso:~$ ip route show
2  192.168.0.0/24 dev enp0s3 proto kernel scope link src 192.168.0.7
3  192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
4
5  student@uso:~$ sudo ip route add default via 192.168.0.1
6
7  student@uso:~$ ip route show
8  default via 192.168.0.1 dev enp0s3
9  192.168.0.0/24 dev enp0s3 proto kernel scope link src 192.168.0.7
10 192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
```

Listing 11.20: Configurarea gateway-ului

Adresa IP a gateway-ului trebuie să se găsească în rețeaua locală, adică să fie similară cu adresele IP configurate pe interfețe. Dacă încercăm să configurăm pentru gateway o adresă IP care nu este în rețeaua locală, primim o eroare precum cea din Listing 11.21: *Nexthop has invalid gateway*.

```
1  student@uso:~$ ip route show
2  192.168.0.0/24 dev enp0s3 proto kernel scope link src 192.168.0.7
```

```
3 192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
4
5 student@uso:~$ sudo ip route add default via 68.212.2.1
6 Error: Nexthop has invalid gateway.
```

Listing 11.21: Configurare gateway-ului cu adresă IP nevalidă

11.6.8 Configurarea DHCP. Comanda dhclient

Configurările de mai sus sunt configurări manuale, cu adresele precizate de utilizator. Dacă dorim să configurăm parametrii de rețea în mod automat, interogând serverul DHCP al rețelei locale, folosim clientul DHCP dat de utilitarul **`dhclient`**, ca în Listing 11.22. Rularea comenzii **`dhclient`** duce la configurarea tuturor parametrilor de rețea furnizați prin DHCP: adresă IP, mască de rețea, gateway, server DNS. Am folosit în prealabil comanda **`ip address flush`** ca să ștergem configurația interfeței `enp0s3`.

```
1  student@uso:~$ sudo ip address flush enp0s3
2
3  student@uso:~$ ip address show enp0s3
4  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
5      link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
6
7  student@uso:~$ ip route show
8  192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
9
10 student@uso:~$ sudo dhclient enp0s3
11
12 student@uso:~$ ip address show enp0s3
13 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   state UP group default qlen 1000
14     link/ether 08:00:27:3a:a9:01 brd ff:ff:ff:ff:ff:ff
15     inet 10.0.2.15/24 brd 10.0.2.255 scope global enp0s3
16        valid_lft forever preferred_lft forever
17
18 student@uso:~$ ip route show
19 default via 10.0.2.2 dev enp0s3
20 10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15
21 192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
```

Listing 11.22: Configurarea automată (prin DHCP) din linia de comandă

11.6.9 Sumar

În Linux, pentru configurarea rețelei, folosim, în general, interfața grafică. Interfața grafică folosește un serviciu de rețea (NetworkManager).

Configurarea în linia de comandă poate fi realizată în mai multe moduri, depinzând de opțiunile utilizatorului și de distribuția folosită. Configurarea în linia de comandă se face folosind fișiere de configurare. Opțiuni pentru configurarea în linia de comandă sunt Netplan (Ubuntu), `ifupdown` (Debian / Ubuntu), `ifcfg` (RedHat).

Pentru diagnosticare, testare sau depanare putem folosi configuare temporară. Configurarea temporară se realizează cu ajutorul utilitarul **`ip`**.

11.7 Aplicații de rețea. Webul

Odată configurat accesul la Internet al unei stații (sistem, dispozitiv), utilizatorul poate accesa servicii din Internet: conținut media, rețele sociale, medii de stocare, soluții colaborative, comunicare online, acces la distanță. Pentru accesarea acestor servicii, utilizatorul folosește aplicații specializate numite aplicații client. O aplicație client este un proces care rulează pe stația locală care folosește un protocol cunoscut pentru a se conecta, prin Internet, la o aplicație server (sau serviciu). Aplicația server este un proces care rulează pe un sistem server și oferă servicii în Internet, servicii ce sunt accesate de utilizatori prin aplicațiile client. De exemplu, atunci când folosim pe un dispozitiv mobil aplicația client WhatsApp, aceasta se va conecta la o aplicația server furnizată de compania Facebook și va putea interacționa cu alte aplicații WhatsApp de pe alte dispozitive. La fel, pe un sistem desktop folosim aplicația client Dropbox, care se va conecta la o aplicație server Dropbox pentru a sincroniza fișierele locale cu cele de la distanță.

În vreme ce pe dispozitivele mobile sau de tip smart TV există este uzual să existe aplicații client dedicate pentru fiecare tip de serviciu (YouTube, Facebook, WhatsApp, Google Hangouts, Google Drive, Maps, Calendar, Mail), pe sistemele desktop multe dintre aceste servicii sunt accesate printr-un navigator (browser) web. Un browser web este un client web (sau client HTTP). Întrucât multe dintre serviciile din Internet sunt accesate prin web / HTTP, un browser web poate fi folosit în rol de client generic. Un browser web poate fi folosit și pe dispozitivele mobile sau smart TV, dar, uzual, există o aplicație dedicată care facilitează accesul serviciului. La fel și pe sistemele desktop: pot exista aplicații dedicate (precum un client Google Drive sau un client de e-mail precum Microsoft Outlook sau Mozilla Thunderbird), dar mulți utilizatori preferă folosirea unei interfețe unice dată de clientul web.

Abordarea folosirii unei aplicații dedicate are avantajul unui acces mai rapid al serviciului: se pornește aplicația și, dacă este configurația realizată, accesează serviciul. De cealaltă parte, avantajul folosirii unui browser web înseamnă că furnizorul serviciului nu trebuie să mai dezvolte o aplicație client dedicată, ci să se concentreze doar pe implementarea serviciului. De asemenea, utilizatorul serviciului nu are nevoie să instaleze o aplicație dedicată, ci folosește browser-ul web pe care îl întâlnește pe orice sistem; poate folosi inclusiv browserul web pe un sistem străin, unde probabil nu are permisiuni de instalare. Browserul web devine astfel una dintre aplicațiile esențiale ale sistemelor conectate la Internet. Browserele web moderne (Safari, Microsoft Edge, Mozilla Firefox) sunt aplicații complexe, cu funcționalități care să permită o experiență cât mai plăcută și performantă utilizatorului care va accesa servicii din Internet.

11.7.1 Serviciul web

Browserele sunt clienți web care folosesc protocolul HTTP (*Hypertext Transfer Protocol*) pentru accesarea serviciilor expuse. Popularitatea webului și a protocolui

HTTP a dus la situația în care furnizorii de servicii de Internet își proiectează serviciile direct să fie accesate prin HTTP, sau să ofere și această interfață. De exemplu, Netflix permite vizualizarea de filme prin intermediul protocolului HTTP, fie din aplicație dedicată (pe un smart TV), fie printr-un browser web.

Protocolul HTTP este în esență un protocol de acces de resurse de la distanță. Pentru accesarea acestor resurse, este nevoie de un mod de identificare a acestora, o adresă. Acest identificator este URL (*Uniform Resource Locator*), numit și adresă web. Este acea adresă pe care o introducem în bara de adrese a unui browser web. Un URL este compus din trei elemente:

- protocolul folosit: uzual HTTP sau HTTPs (HTTP securizat)
- numele sistemului pe care rulează serviciul: uzual este un nume DNS (precum `google.com`), dar poate fi și o adresă IP
- calea către resursă: modul în care serviciul web localizează pe server resursa. Resursa este uzual o pagină web. Pagina web este transferată de la serviciul web la clientul web (browser) unde este redată (*rendered*) în ceea ce vede la final utilizator.

De exemplu, în cadrul URL-ului https://ocw.cs.pub.ro/courses/uso/cursuri/curs-05, protocolul este `https`, numele sistemului este `ocw.cs.pub.ro`, iar calea către resursă (adică pagina web aferentă cursului 5 de USO) este `courses/uso/cursuri/curs-05`.

Folosim un browser web pentru accesarea unor servicii de la distanță, cel mai adesea în formă de pagini web care for redate de browser. Pentru cazul în care dorim descărcarea unei resurse, sau accesarea unui serviciu simplu sau pentru automatizare sau testare putem folosi utilitare de tip clienți web în linia de comandă. Cele mai întâlnite astfel de utilitare în Linux sunt **`wget`** sau **`curl`**. Cele două au funcționalități similare, numai că, implicit, **`wget`** salvează conținutul de la URL transmis ca argument într-un fișier, în vreme ce **`curl`** afișează conținutul la ieșirea standard. O altă diferență este că **`wget`** permite descărcarea recursivă a resurselor; adică link-urile web sunt accesate și apoi se realizează transferul și resurselor respective. **`wget`** folosește protocoalele HTTP, HTTPS, FTP; **`curl`** adaugă suport și pentru alte protocoale. Câteva exemple de funcționare sunt în Listing 11.23.

```
1  student@uso:~$ wget http://acs.pub.ro/~cpop/orare_sem1/Orar1CD.xls
2  --2021-01-16 13:24:51--  http://acs.pub.ro/~cpop/orare_sem1/Orar1CD.xls
3  Resolving acs.pub.ro (acs.pub.ro)... 141.85.227.151
4  [...]
5  2021-01-16 13:24:51 (1.39 MB/s) - 'Orar1CD.xls' saved [50688/50688]
6
7  student@uso:~$ curl icanhazip.com
8  95.76.43.212
9
10 student@uso:~$ wget --user <username> --ask-pass https://repository.grid.
   pub.ro/cs/uso/USO.ova
11 Password for user '<username>':
12 --2021-01-16 13:26:04--  https://repository.grid.pub.ro/cs/uso/USO.ova
13 Resolving repository.grid.pub.ro (repository.grid.pub.ro)...
   141.85.241.222
14 [...]
15 Saving to: 'USO.ova'
```

Listing 11.23: Clienți web în linie de comandă: wget și curl

11.7.2 Alte servicii

Unele servicii nu sunt accesate prin web / HTTP, ci prin alte protocoale, sau și prin alte protocoale. În această situație, există aplicații client dedicate pentru accesarea serviciilor. Exemple de astfel de servicii sunt conexiune la distanță, poștă electronică și transfer de fișiere.

11.7.2.1 Conexiune la distanță

Conexiunea la distanță (*remote connection*) permite accesarea și controlul unui sistem din Internet de pe o stație locală. Prin canalul de comunicație realizat între stația locală și sistemul de la distanță pot fi trimise comenzi. Interacțiunea cu sistemul aflat la distanță poate fi cu interfață grafică (serviciu numit și *remote desktop* / *desktop sharing*) sau interfață în linie de comandă (serviciu numit și *remote shell*). Dat fiind că este sensibilă oferirea accesului la un sistem, serviciile de conexiune la distanță au integrate componente de autentificare și confidențialitate.

Microsoft a dezvoltat protocolul RDP (*Remote Desktop Protocol*) pentru conexiunea la distanță cu interfață grafică; serviciul RDP este încorporat în sistemul de operare Windows. RDP este folosit în principal pentru Windows, dar există aplicații client și server pentru alte sisteme de operare.

O aplicație de tipul *remote desktop* este TeamViewer. Aplicația acționează simultan în rol de server și de client. Un utilizator care instalează aplicația pe sistem propriu o poate folosi pentru a se conecta la alte sisteme, sau pentru a permite altor utilizatori să se conecteze la sistemul său. O aplicație similară este AnyDesk.

VNC (*Virtual Network Computing*) este un alt sistem de tipul *desktop sharing*. Cuprinde un server VNC, un client VNC și protocolul RFB (*Remote Frame Buffer*) folosit pentru comunicare.

Pentru acces la distanță în linia de comandă, cel mai întâlnit protocol este SSH (*Secure Shell*). Protocolul SSH creează un canal sigur de comunicare (criptat). În acest canal se poate opta pentru deschiderea unei sesiuni shell la distanță (*remote shell*) sau pentru transfer de fișier sau pentru tunelarea altor protocoale pentru a le adăuga funcționalități de securitate. Ca să deschidem o sesiune shell la distanță folosim, în Linux, comanda **`ssh`** urmată de numele de cont (*username*) și numele stației, ca în Listing 11.24.

```
1  student@uso:~$ whoami
2  student
3  student@uso:~$ hostname
4  uso
5  razvan@jotunn:~$ ssh asm@elf.cs.pub.ro
6  Linux elf 2.6.32-5-amd64 #1 SMP Tue May 13 16:34:35 UTC 2014 x86_64
7  [...]
8  asm@elf:~$ whoami
9  asm
10 asm@elf:~$ hostname
```

```
11 elf
```

Listing 11.24: Conexiune la distanță folosind SSH

În Listing 11.24, ne-am conectat de pe sistemul cu numele `uso`, din contul `student`, pe sistemul cu numele `elf.cs.pub.ro` în contul `uso`. Avem deschisă o sesiune shell la distanță unde putem rula comenzi și unde putem administra sistemul.

În sistemele Linux, pentru administrarea sistemelor și data centerelor, se folosește aproape exclusiv protocolul SSH. Mai multe detalii despre protocolul SSH vom prezenta în Secțiunea 12.4.4.

11.7.2.2 E-mail

Serviciul de poștă electronică (e-mail) este folosit pentru transmisia de mesaje între utilizatori. Fiecare utilizator dispune de o căsuță poștală electronică (*electronic mailbox*) unde primește mesajele. Pentru accesarea căsuței poștale și pentru trimiterea de mesaje (e-mailuri), folosește protocoale specifice serviciului de e-mail și un client de e-mail care cunoaște aceste protocoale. Exemple de clienți de e-mail sunt Microsoft Outlook sau Mozilla Thunderbird. Aceștia se conectează la servere de e-mail corespunzătoare pentru a citi și a trimite mesaje.

Așa cum am precizat mai sus, multe servicii sunt interfațate prin web. Astfel, și furnizorii de servicii de e-mail oferă, în general, o interfața web care acționează ca un client de e-mail. Este exemplul serviciilor GMail sau ProtonMail care sunt accesate dintr-un browser web.

Serviciul de poștă electronică este similar cu serviciile de mesagerie (*instant messaging*) precum WhatsApp, Facebook Messenger, Telegram, Signal. O bună parte dintre utilizatori preferă folosirea acestor servicii de mesagerie pentru comunicare mai rapidă și creare facilă de grupuri de lucru și conferințe audio și video. Cu toate acestea, serviciul de e-mail rămâne util din câteva motive:

- Are o organizare și accesare mai bună a mesajelor (în căsuța poștală).
- Este de așteptat să nu primească un răspuns imediat.
- Este mai adecvat mediilor profesionale, în care legăturile sunt mai formale.
- Pentru că permite construcția de liste sau grupuri de discuții cu arhive de mesaje care pot fi consultate ulterior.

11.7.2.3 Transfer de fișiere

Transferul de fișiere, după cum îi spune și numele, este un serviciu care permite transmiterea fișierelor (datelor) între o sursă și destinație. Fundamental, protocolul HTTP (*Hypertext Transfer Protocol*), așa cum îi spune și numele, este un protocol de transfer de date.

Protocolul HTTP are utilizări care pot fi mai degrabă clasificate ca fiind acces la distanță, în vreme ce alte protocoale sunt folosite exclusiv pentru transferul de fișiere. Deși poate

face și încărcare de informații (*upload*), protocolul HTTP este folosit, în general, pentru descărcare (*download*).

Protocolul SSH (*Secure Shell*) de care am amintit mai sus permite transferul de fișiere în formă securizată. Atunci când avem un cont pe un sistem la distanță este forma preferată de transfer de fișiere.

Protocolul FTP (*File Transfer Protocol*) are ca rol transferul fișierelor. Spre deosebire de HTTP, protocolul FTP permite mai ușor încărcarea de fișiere (*upload*). Spre deosebire de SSH, protocolul nu necesită un cont la distanță. De aceea, este protocolul preferat folosit de furnizorii de servicii de găzduire (*hosting services*) care nu doresc, din rațiuni de securitate, să ofere un cont pentru acces la distanță.

Pentru transferul de date în rețele mai mari de noduri (numite *swarmuri*) folosim protocolul BitTorrent. Protocolul BitTorrent este un protocol peer-to-peer, în care fiecare nod este simultan și client și server, permițând și descărcarea (*download*) și încărcarea (*upload*) de informații. Este în special util pentru distribuția de fișiere de mari dimensiuni. În perioada 2005-2010 transferurile prin BitTorrent erau printre cele mai consumatoare de lățime de bandă din Internet, în special pentru conținut video, adesea piratat. Între timp, dezvoltarea infrastructurii de Internet și popularitatea serviciilor de video streaming precum YouTube sau Netflix, au dus la diminuarea folosirii BitTorrent.

11.8 Anexă: Configurări de rețea folosind suita ifconfig/route

Pe sistemele Linux folosim suita **iproute2** (care conține utilitarul **ip**) pentru investigarea și configurarea rețelei în linia de comandă. Până la apariția **iproute2**, aceste acțiuni erau realizate cu utilitare precum **ifconfig** și **route**. Aceste utilitare pot fi, în continuare, folosite în Linux, deși preferăm suita **iproute2** care are un număr mai mare de funcționalități.

Un avantaj al cunoașterii utilitarelor de forma **ifconfig** și **route** este că sunt prezente pe toate sistemele din familia Unix. Dacă vom avea de-a face cu sisteme rulând FreeBSD sau macOS, vom putea folosi **ifconfig** și **route** pentru configurarea rețelei.

Folosim **ifconfig** pentru a afișa informații despre interfețele de rețea și folosim **route** pentru a afișa tabela de rutare, incluzând gateway-ul, ca în Listing 11.25. Comanda **ifconfig** afișează implicit doar interfețele active. Dacă dorim să afișăm informații despre toate interfețele (active și inactive) folosim opțiunea **-a** la comanda **ifconfig** (liniile 20–45).

```
1  student@uso:~$ ifconfig
2  enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
3          inet 192.168.56.101  netmask 255.255.255.0  broadcast
   192.168.56.255
4          inet6 fe80::61bb:7aab:13f7:20fa  prefixlen 64  scopeid 0x20<link>
5          ether 08:00:27:c0:83:50  txqueuelen 1000  (Ethernet)
6          RX packets 43105  bytes 5163146 (5.1 MB)
7          RX errors 0  dropped 0  overruns 0  frame 0
8          TX packets 23877  bytes 5381787 (5.3 MB)
9          TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
10
```

```
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 3686  bytes 351404 (351.4 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3686  bytes 351404 (351.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

student@uso:~$ ifconfig -a
enp0s3: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 08:00:27:3a:a9:01  txqueuelen 1000  (Ethernet)
        RX packets 677002  bytes 939796880 (939.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 87608  bytes 5752208 (5.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.56.101  netmask 255.255.255.0  broadcast
192.168.56.255
        inet6 fe80::61bb:7aab:13f7:20fa  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:c0:83:50  txqueuelen 1000  (Ethernet)
        RX packets 43116  bytes 5164052 (5.1 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 23883  bytes 5383391 (5.3 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 3688  bytes 351550 (351.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3688  bytes 351550 (351.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

```

Listing 11.25: Vizualizarea parametrilor de rețea cu ifconfig/route

Activarea sau dezactivarea unei interfețe se face tot cu ajutorul utilitarului **`ifconfig`** ca în Listing 11.26.

```
student@uso:~$ sudo ifconfig enp0s3 down

student@uso:~$ ifconfig enp0s3
enp0s3: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 08:00:27:3a:a9:01  txqueuelen 1000  (Ethernet)
        RX packets 677002  bytes 939796880 (939.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 87616  bytes 5753054 (5.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

student@uso:~$ sudo ifconfig enp0s3 up

student@uso:~$ ifconfig enp0s3
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 08:00:27:3a:a9:01  txqueuelen 1000  (Ethernet)
```

```
        RX packets 677002  bytes 939796880 (939.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 87614  bytes 5752824 (5.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Listing 11.26: Activarea și dezactivarea unei interfețe folosind ifconfig

Pentru configurarea parametrilor de rețea, **`ifconfig`** configurează adresa IP și masca de rețea, iar **`route`** configurează gateway-ul. Astfel, dacă dorim să configurăm adresa IP `192.168.0.7/24` și gateway-ul cu adresa IP `192.168.0.1`, vom folosi comenzile `ifconfig` și `route` ca în Listing 11.27.

```
student@uso:~$ sudo ifconfig enp0s3 192.168.0.7 netmask 255.255.255.0

student@uso:~$ ifconfig enp0s3
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.0.7  netmask 255.255.255.0  broadcast 192.168.0.255
        ether 08:00:27:3a:a9:01  txqueuelen 1000  (Ethernet)
        RX packets 677002  bytes 939796880 (939.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 87623  bytes 5753795 (5.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

student@uso:~$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use
Iface
192.168.0.0     0.0.0.0         255.255.255.0   U     0      0        0
enp0s3
192.168.56.0    0.0.0.0         255.255.255.0   U     0      0        0
enp0s8

student@uso:~$ sudo route add default gw 192.168.0.1

student@uso:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use
Iface
0.0.0.0         192.168.0.1     0.0.0.0         UG    0      0        0
enp0s3
192.168.0.0     0.0.0.0         255.255.255.0   U     0      0        0
enp0s3
192.168.56.0    0.0.0.0         255.255.255.0   U     0      0        0
enp0s8
```

Listing 11.27: Configurarea parametrilor de rețea folosind ifconfig/route

11.9 Sumar

Internetul oferă servicii care îmbunătățesc viața: divertisment, comunicare, stocare de informații, eficiență profesională. Pentru a beneficia de aceste servicii, o persoană dispune de mai multe tipuri de dispozitive cu care accesează Internetul.

Internetul reprezintă totalitatea rețelelor interconectate la nivel planetar. A te conecta la Internet înseamnă să faci parte dintr-o rețea care este parte din Internet. Conectarea la o rețea presupune folosirea unui dispozitiv care are o placă de rețea (laptop, dispozitiv

mobil, smart TV), a unui mediu de transmisie (aer, cablu, fibră optică) și a unor dispozitive de interconectare (switch-uri, rutere, access pointuri).

Pentru a putea comunica în Internet avem nevoie de protocoale care să asigure scheme de adresare (identificare a stațiilor în Internet și a aplicațiilor de rețea ce rulează pe acestea) și care să asigure dirijarea (rutarea) pachetelor.

Protocolul fundamental în Internet este protocolul IP (Internet Protocol). O stație va avea o adresă IP cu care este identificată și pe care o folosește să comunice în Internet. Protocolul IP stă la baza stivei de protocoale TCP/IP, stiva folosită în Internet. Adresele IP sunt limitate și au apărut două soluții: protocolul IPv6, cu un număr potențial foarte mare de adrese, respectiv folosirea de adrese IP private în rețele locale combinată cu translatarea adreselor (NAT).

Odată prezente elementele fizice (placă de rețea, mediu de transmisie, dispozitive de interconectare) pentru conectarea unei stații la Internet, va trebui să o configurăm. Pentru conectarea unei stații la Internet vom configura patru elemente: adresa IP, masca de rețea, gateway-ul și serverul DNS.

În general, pe sistemele și dispozitivele moderne configurarea rețelei este realizată facil cu ajutorul interfeței grafice, prin intermediul NetworkManager. Pe servere și alte sisteme fără interfață grafică folosim configurarea în linia de comandă, ce are la bază fișiere de configurare. Dacă dorim control mai fin sau pentru diagnosticare, depanare sau testare, putem folosi utilitare clasice (în linia de comandă), precum utilitarul **`ip`**.

Utilizatorul are la dispoziție aplicații de tip client de rețea care se conectează pe servere pentru a folosi serviciile acestora. Aplicațiilor vor folosi protocolul specific serviciului: HTTP, FTP, SSH, BitTorrent etc.

Capitolul 12

Securitatea sistemului

Noțiunea de securitate a devenit din ce în ce mai importantă în ultimii ani. Dezvoltarea tehnologiei, împreună cu o creștere alertă a conectivității datorate Internetului, au condus la mărirea numărului de atacuri informatice. În prezența tehnologiei și a conectivității, date digitale și informații confidențiale pot fi disponibile unui atacator la distanță, nefiind nevoie de prezența sa fizică lângă dispozitivul atacat. Interesul crescut pentru securitate vine din partea companiilor, utilizatorilor și entităților guvernamentale pentru protejarea datelor digitale și a infrastructurilor informatice.

În ciuda creșterii interesului legat de securitate, multe noțiuni sunt relativ vagi și neînțelese publicului larg. Anumite informații sunt prea tehnice, altele sunt exagerate și altele sunt prea teoretice. În acest capitol urmărim să clarificăm noțiunile esențiale legate de securitate, cu accent pe securitatea unui sistem de calcul, împreună cu tipurile principale de perspective de securitate de interes pentru fiecare utilizator: securitatea datelor, securitatea accesului, securitatea transferului și securitatea aplicațiilor.

Numim un **sistem informatic sigur** un sistem care furnizează rezultatele așteptate. În contrast, numim un sistem informatic nesigur un sistem care funcționează necorespunzător, însemnând că sunt situații în care rezultatul nu este corect sau sistemul se defectează. Atunci când un sistem funcționează de multe ori conform așteptărilor, nu putem spune cu certitudine că este sigur. Este posibil ca într-un caz de utilizare specific să nu funcționeze corespunzător. Din acest motiv, a spune că un sistem este perfect sigur este o exprimare hazardată, dat fiind că este cvasi-imposibil de garantat că un sistem funcționează conform așteptărilor pentru toate cazurile de utilizare.

Spunem că, în general, **complexitatea unui sistem afectează securitatea acestuia**. Cu cât un sistem este mai complex cu atât este mai greu de validat că pentru un număr mare de cazuri de utilizare, sistemul funcționează corespunzător. Creșterea complexității unui sistem duce la creșterea riscurilor de securitate la adresa acestuia. Riscurile de securitate pot apărea la toate nivelurile unui sistem: pot fi probleme software, probleme hardware, probleme de infrastructură sau probleme de configurare. Aceste probleme sunt riscuri care expun sistemul către atacatori. Un atacator va urmări exploatarea sistemului în beneficiu propriu.

Un sistem care este atacat poate suferi pe trei direcții:

1. **Pierderea controlului**: atacatorul deține controlul sistemului putând accesa date private, rula aplicații malițioase sau abuza alte sisteme.
2. **Furtul de informații**: informații critice ale unui utilizator (cod PIN, date private) sau ale unei companii (conturi de acces, informații despre tranzacții, cod sursă) pot fi extrase de atacator pentru a fi vândute, pentru a șantaja sau pentru a extrage bani din conturi bancare.
3. **Abuzarea resurselor**: atacatorul îngreunează funcționarea sistemului sau îl oprește, sabotând astfel serviciul unei companii și ducând la pierderi de imagine sau pierderi în piață.

Un atacator poate urmări una sau mai multe dintre aceste direcții. Motivația unui atacator este adesea financiară, dar poate fi și una politică, poate fi angajat de o entitate concurentă sau poate ataca sistemul doar pentru distracție. Există, de asemenea, categorii de persoane bine intenționate, așa numiții white-hat hackers (sau ethical hackers), care atacă un sistem pentru a-i descoperi problemele pe care le raportează apoi pentru a fi corectate.

În contrast cu un atacator, perspectiva apărătorului este aceea de a proteja datele sale, de a preveni pierderea controlului și de a face sistemul cât mai robust. O diferență esențială între atacator și apărător este că atacatorul trebuie să găsească o singură problemă de securitate (numită și breșă de securitate - *security hole*) pe când un apărător trebuie să apere toate posibilele breșe din sistem. Un apărător trebuie să aibă în vedere atât mecanisme preventive, cât și mecanisme reactive: adică mecanisme care împiedică sau îngreunează atacatorul să genereze un atac, respectiv mecanisme care minimizează sau izolează daunele în cazul unui atac.

Din aceste considerente, spunem că securitatea nu urmărește crearea unui sistem sigur, lucru imposibil de obținut, ci **reducerea riscurilor de securitate**. Un sistem este mai sigur dacă se investesc resurse în securizarea acestuia: bani, timp, cunoștințe, oameni, proceduri și politici de securitate. Așa cum vom vedea în continuare, riscurile de securitate pentru un sistem cresc în măsura în care există mai multe moduri de a-l accesa, echivalentul numărului de porți într-o cetate sau a unor ziduri slabe. Reducerea numărului de intrări în sistem și verificarea acestor intrări sunt moduri esențiale de a crește securitatea acestuia. Din păcate, interconectarea sistemelor și accesul dispozitivelor de tot felul la Internet înseamnă că au „intrări" prin intermediul Internetului și pot fi atacate de la distanță, o practică frecventă a atacatorilor în zilele noastre. Acesta este un motiv pentru care tehnologiile Internet of Things sunt preocupate continuu de securitate, așa cum vom preciza în Secțiunea 15.5.

În continuare, vom detalia conceptele esențiale ce țin de securitatea informatică. Deși ne concentrăm pe securitatea unui sistem / dispozitiv, conceptele se aplică la zone din întreaga lume IT, precum securitatea aplicațiilor web, securitatea cloud sau securitatea rețelelor.

12.1 Fundamentele securității informatice

Mai sus am precizat că un apărător trebuie să aibă în vedere toate intrările în sistem și toate posibilele breșe de securitate. Măsurile aplicate de un apărător sunt măsuri

preventive și măsuri reactive. Aceste eforturi ale unui apărător sunt permanente. Nu poate să se oprească și să spună „gata, am securizat sistemul”. Sistemul trebuie în permanență monitorizat și actualizat. Mai mult, ținând cont de schimbările frecvente la nivel software (aplicații noi sau versiuni noi de aplicații), hardware (sisteme noi, funcționalități noi la nivel de procesor) sau infrastructură (conexiuni de viteză mai mare, interconectare cu servicii), sistemul este în continuă schimbare.

Spunem că **securitatea este un proces, nu o finalitate**. Eforturile de securizare a unui sistem sunt eforturi continue. Un dezvoltator, un administrator sau un proiectant al unui sistem trebuie să aibă în vedere securitatea acestuia ca un element permanent al activității sale, iar acțiunile sale trebuie să mențină sau să crească securitatea sistemului. Pierderea perspectivei securității pe parcursul proiectării, dezvoltării sau administrării unui sistem poate duce la creșterea riscurilor de securitate și la atacarea sistemului.

Perspectiva securității ca proces, ca acțiune continuă, impune alocarea permanentă de resurse. Aceste resurse pot fi resurse fizice (uși, lacăte, dispozitive de tip firewall), pot fi financiare (achiziția de software / hardware de securitate), de resurse umane (oameni competenți, traininguri de securitate) sau de proces (proceduri și ghiduri de securitate). Un sistem este cu atât mai sigur, cu cât sunt investite resurse mai multe și mai de calitate în securitatea acestuia. Un utilizator obișnuit nu va investi multe resurse în securitate pentru că datele sale și sistemele sale nu sunt atât de interesante pentru un atacator. O companie mare, în special o bancă, va avea însă nevoie de investiții semnificative pentru securitate, fiind o țintă mult mai probabilă a atacatorilor.

12.1.1 Obiectivele securității

A crește securitatea unui sistem, înseamnă a elimina, sau, mai bine zis, a îngreuna atacurile. Securitatea unui sistem înseamnă atingerea unor obiective pentru acel sistem, obiective care sunt compromise în cazul unui atac. În mod ideal, aceste obiective sunt atinse pentru toate sistemele, dar unele pot fi prevalente pentru anumite sisteme. Prezentăm în continuare obiective principale ale securității, cu precizarea că nu sunt exhaustive și nu sunt complet separate.

Unul dintre cele mai importante obiective este **confidențialitatea**. Confidențialitatea se referă, în general, la datele stocate și la datele transferate. Spunem că este asigurată confidențialitatea dacă doar utilizatorii sau entitățile autorizate să acceseze și să vizualizeze datele o pot face. Confidențialitatea este asigurată în general cu ajutorul criptării, așa cum vom clarifica în Secțiunea 12.2.2.

Un alt obiectiv legat de date este **integritatea**. Dacă un atacator nu poate vizualiza date (adică le este asigurată confidențialitatea), în continuare acesta poate modifica datele; spunem că atacatorul corupe datele. Coruperea datelor poate însemna pierderea acestora pentru o entitate de tip victimă sau poate însemna inducerea unui alt comportament: victima crede că datele sunt corecte și ia altă decizie pe baza lor. Integritatea datelor este asigurată, în general, de algoritmi de hashing, așa cum vom detalia în Secțiunea 12.2.3.

Un sistem sigur este un sistem care răspunde cererilor de la utilizator pentru a obține rezultatele dorite de acesta. De aceea un obiectiv al securității este **disponibilitatea** (*availability*). Concepte similare sunt fiabilitatea (*reliability*) și robustețea (*robustness*).

Un sistem este disponibil atunci când oferă în mod constant serviciul corespunzător utilizatorilor, independent de condiții. Adică în cazul unui atac de abuz de resurse de tipul DoS (*Denial of Service*), sistemul funcționează în parametri satisfăcători. Asigurarea disponibilității unui sistem ține de mai multe măsuri, precum monitorizarea sistemului, verificarea intrărilor, redundanță sau fiabilitatea hardware-ului.

Un obiectiv din ce în ce mai prezent în zilele noastre, dominate de rețele sociale și de profilurile virtuale ale utilizatorilor, este **protejarea vieții private** (*privacy*). Noțiunea de privacy se referă la protejarea aspectelor personale ale unui individ: date personale, poze sau relații. Este vorba de informații care nu sunt în mod direct utile altcuiva, dar care pot afecta profilul personal sau public al persoanei. Interesul pentru privacy a crescut întrucât mai multe companii obțin profit având acces la informațiile private ale utilizatorilor, informații care pot fi folosite abuziv. Măsuri pentru asigurarea protejării vieții private sunt măsuri tehnice, precum anonimizarea accesului, reducerea prezenței online, completarea în formulare online doar a informațiilor strict necesare și măsuri legale, precum "Regulamentul general privind protecția datelor" (*General Data Protection Regulation* - GDPR) în Uniunea Europeană[1]. Asigurarea privacy-ului este o responsabilitate importantă a fiecărui utilizator. Pentru a garanta că aspectele private ale unei persoane sunt folosite neabuziv, acea persoană trebuie să fie preocupată în permanență de prezența sa online și de datele pe care le furnizează în mediul online.

12.1.2 Noțiuni de securitate

Cele două perspective pentru securitate sunt perspectiva atacatorului și perspectiva apărătorului. Apărătorul are acces la sistem și urmărește protejarea acestuia. Atacatorul urmărește abuzarea sistemului trecând de măsurile de protecție ale apărătorului. De avut în vedere că apărătorul trebuie să protejeze sistemul în fața unor atacuri, dar și în fața unor situații necorespunzătoare generate, fără intenție, de utilizatori legitimi. O aplicație poate să se defecteze și din cauza unui atac (cu intenții malițioase), dar și din partea unei utilizări neașteptate (cu intenții legitime).

Așa cum am precizat mai sus, rolul de atacator poate fi preluat și de cineva cu intenții bune, care urmărește exploatarea unui sistem pentru a furniza informațiile necesare și pentru a ajuta la rezolvarea problemelor. Numim acest tip de rol **white-hat hacker** sau **ethical hacker**. De partea cealaltă, un atacator malițios este numit simplu atacator, **cracker** sau **black-hat hacker**.

Spunem că un sistem are un **defect** (*bug*, *flaw*) dacă există o situație neașteptată ce duce la funcționarea necorespunzătoare a acelui sistem. Un astfel de bug se poate manifesta la o utilizare legitimă. Adesea vorbim de defecte / buguri în aplicații: în momentul în care folosim o anumită comandă sau o secvență de acțiuni, aplicația își încheie execuția sau generează un comportament inadecvat.

Un bug devine o **vulnerabilitate** în momentul în care acesta poate fi exploatată de un atacator. Exploatarea unei vulnerabilități înseamnă că un atacator poate folosi acea vulnerabilitate în beneficiul său. Un bug nu conduce la un beneficiu personal, dar o vulnerabilitate da. De exemplu, dacă o aplicație afișează, în cazul unei situații neașteptate, informații aleatoare spunem că este un bug; dacă însă o aplicație afișează

[1] https://gdpr-info.eu/

o listă de parole sau date private, informații ce pot fi folosite de atacator, atunci spunem că este o vulnerabilitate.

Un **exploit** este un instrument (poate fi o aplicație, o dată de intrare sau un dispozitiv) care poate fi folosit pentru exploatarea unei vulnerabilități.

Atunci când un atacator știe de prezența uneia sau mai multor vulnerabilități la nivelul unui sistem, acesta va încerca să le exploateze urmărind un beneficiu în direcția celor prezentate la începutul capitolului: obținerea controlului, furtul de informații sau abuzarea resurselor. Pentru aceasta, un atacator construiește un **vector de atac**. Un vector de atac reprezintă setul de pași pe care îl urmează un atacator pentru a obține un beneficiu în sistemul atacat. De obicei, o singură vulnerabilitate și, deci, un singur exploit nu sunt suficiente, așa că atacatorul va lega mai multe exploituri într-un vector de atac. De exemplu, poate fi un exploit pentru a trece de firewall, unul pentru a obține un shell într-un sistem, altul pentru a obține acces la baza de date de parole, alta pentru a accesa un server și, de acolo, de a obține informații critice. În Figura 12.1 este prezentată schematic ideea unui vector de atac.

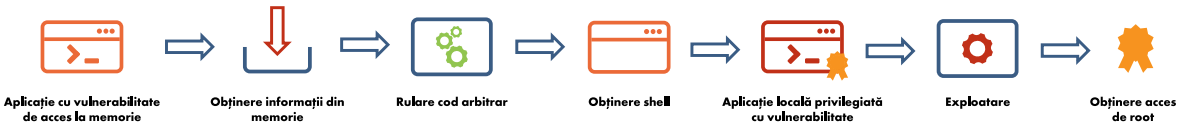


Figura 12.1: Vector de atac

În obiectivul său de a obține beneficii proprii, un atacator are nevoie de un mod de acces la un sistem. În zilele noastre, accesibilitatea prin Internet înseamnă că multe sisteme sunt atacabile de la distanță. Țintele de început ale atacatorilor sunt punctele de intrare în sistem, adică modurile în care se poate accesa un sistem. Spunem că intrările unui sistem reprezintă **suprafața de atac** a unui sistem (*attack surface*); un sistem are suprafața de atac cu atât mai mare cu cât are un număr mai mare de intrări sau de intrări cât mai largi. O suprafață de atac mare înseamnă risc de securitate mare. Așa că, unul dintre modurile prin care un apărător protejează sistemul este reducerea suprafeței de atac.

Principiul reducerii suprafeței de atac se aplică și în alt context. Un sistem are o componentă critică de funcționarea corectă de care depinde întreg sistemul. De exemplu, pe un sistem de calcul obișnuit, această componentă este dată de sistemul de operare și de serviciile privilegiate ale sistemului. Când sistemul de operare sau un serviciu privilegiat este compromis, echivalent cu obținerea contului `root` în Linux sau contul de tip `Administrator` în Windows, spunem că întreg sistemul este compromis. Numim această componentă critică a sistemului **Trusted Computing Base** (TCB), așa cum indicăm în Figura 12.2. TCB-ul trebuie protejat cel mai mult în cadrul sistemului. Componente non-TCB pot fi compromise fără a compromite întreg sistemul, dar compromiterea TCB-ului înseamnă compromiterea sistemului. Dată fiind relevanța acestuia în securizarea sistemului, dimensiunea TCB-ului trebuie să fie cât mai mică, adică TCB-ul să aibă suprafață de atac cât mai redusă, micșorând riscurile critice de securitate.

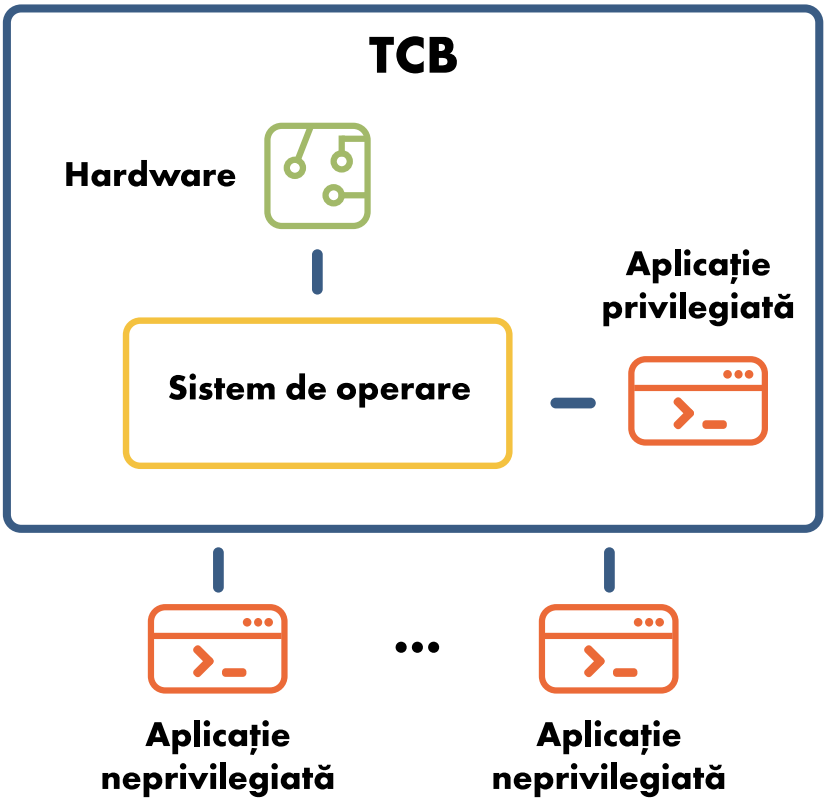


Figura 12.2: Trusted Computing Base (TCB) într-un sistem de calcul

12.1.3 Principii de securitate

În proiectarea, dezvoltarea și administrarea unui sistem, există principii de securitate care se recomandă să fie urmărite pentru a minimiza riscurile de atac sau daunele produse de un atac.

Unul dintre cele mai importante principii este **principiul celui mai mic privilegiu**. Un sistem are componente (hardware, software, de infrastructură) ce interacționează între ele. O anumită componentă, de exemplu un anumit proces, are nevoie de acces la anumite resurse pentru a funcționa. Principiul celui mai mic privilegiu dictează ca implementarea mecanismelor de securitate să nu permită accesul componentei la alte resurse. Astfel că, dacă o aplicație este compromisă, impactul daunelor produse va fi minim: doar la nivelul resurselor accesibile aplicației.

Principiul celui mai mic privilegiu corelează cu **proiectarea modulară a sistemelor**. Atunci când sistemul este monolitic (nu are separații clare între componente), accesul la toate resursele se acordă întregului sistem. Atunci când sistemul este modular, format din mai multe părți, fiecărei părți îi este atribuit setul minim necesar de privilegii: o parte are acces la anumite resurse, o altă parte la alte resurse, limitând potențialul daunelor în cazul compromiterii unei părți. Putem spune că principiul celui mai mic privilegiu, împreună cu o proiectare modulară, duce la reducerea suprafeței de impact în cazul unui atac. În Figura 12.3 este precizată diferența între o proiectare monolitică a unui sistem și una modulară și beneficiul folosirii principiului celui mai mic privilegiu.

Mijloacele defensive care pot fi folosite în securizarea unui sistem nu sunt infailibile. Pentru creșterea securității unui sistem, se recomandă **principiul apărării în adâncime** sau **apărării multi-nivel** (*defense in depth*, *multi-layer security*) cu mai

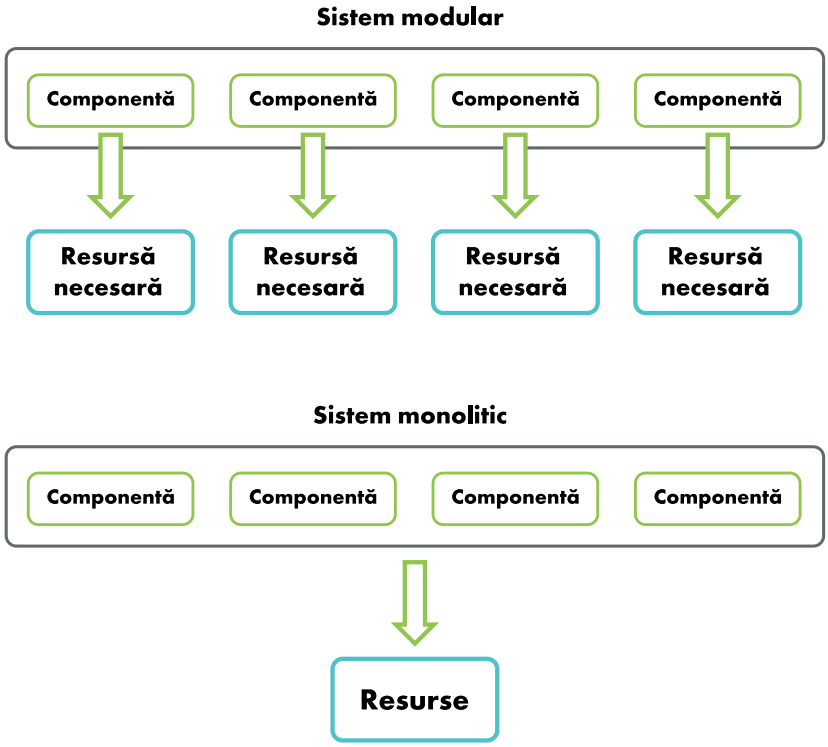


Figura 12.3: Accesul la resurse într-un sistem monolitic și unul modular

multe mijloace defensive prezente simultan în securizarea unui sistem. Aceste mijloace sunt complementare, similare mai multor ziduri de apărare: penetrarea unui mijloc defensiv plasează atacatorul în fața unui alt mijloc defensiv, făcând dificilă crearea unui vector de atac capabil să înfrângă toate mijloacele.

În proiectarea sistemelor trebuie avut în vedere în permanență perspectiva atacatorului. Un atacator va încerca să găsească cel mai simplu mod de a accesa și controla sau abuza sistemul. Numim **veriga cea mai slabă** (*weakest link*) componenta care are un nivel de vulnerabilitate ridicat. Aceasta va fi ținta predilectă a atacatorilor și compromiterea acesteia duce, în general, la compromiterea unei mari părți a sistemului. Un atacator poate porni de la o mică breșă în sistem și o poate expanda, prin exploituri consecutive, până la compromiterea întregului sistem. Modularizarea și principiul celui mai mic privilegiu ajută la izolarea daunelor, în vreme ce securizarea componentelor și asigurarea unui nivel de securitate adecvat celei mai slabe verigi reduce riscul apariției unui atac.

Atunci când se dorește securizarea unui sistem, facem **separația între politica de securitate și mecanismul de securitate** (*security policy*, *security mechanism*). Politica se referă la regulile și principiile de securitate, descrise conceptual. Mecanismul se referă la implementări efective, specifice sistemului, care satisfac regulile și principiile. Principiul separației mecanismului de politică este important pentru a permite implementarea diferitelor mecanisme sau a mai multor mecanisme pentru aceeași politică. Schimbarea mecanismului nu afectează politica, iar cei care decid politica nu trebuie să aibă detalii interne legate de mecanism, concentrându-se astfel pe partea importantă conceptuală.

12.1.4 Modelul subiect-obiect. Permisiuni de acces

În implementarea principiilor de securitate, sistemele și interacțiunea între componentele acestora sunt descrise de **modelul subiect-obiect** (*subject-object model*). În acest model, subiectul, numit și agent, este entitatea activă, entitatea care execută acțiuni. Obiectul, numit și resursă, este entitatea pasivă, entitate asupra căreia se execută acțiunile. Securitatea unui sistem este dată de regulile de acces ale subiecților la obiecte. Un subiect `S1` poate avea acces de scriere la obiectul `O1`, subiectul `S2` poate avea acces de citire la obiectul `O1`, iar subiectul `S3` poate nu să nu aibă acces la nici o resursă. Aceste reguli sunt în general reprezentate în formă de graf ca în Figura 12.4.

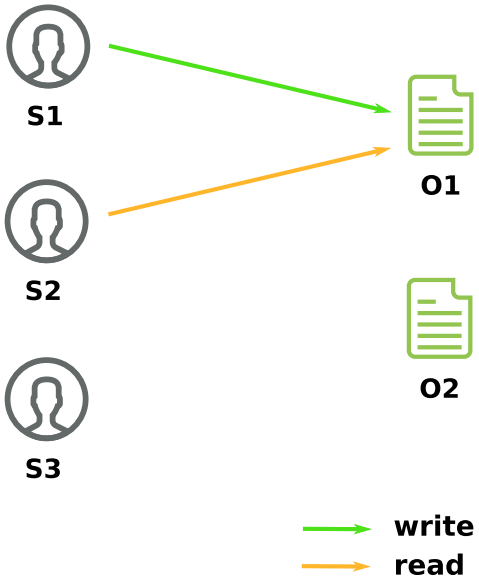


Figura 12.4: Permisiuni de access

O entitate privilegiată, care face parte din TCB-ul sistemului, numită și **monitor de referințe** (*reference monitor*) este cea care permite sau nu accesul subiecților la obiecte. Pentru acesta sunt configurate permisiuni de acces care stabilesc pentru fiecare subiect și obiect ce permisiuni au fiecare. Modul în care monitorul de referințe gestionează accesul subiecților la obiecte este descris în Figura 12.5. Cel mai adesea, monitorul de referințe este sistemul de operare.

O formă de implementare a modelului subiect-obiect o reprezintă permisiunile în sistemul de fișiere despre care vom discuta în Secțiunea 12.2.1. În acest caz subiectul este procesul, iar obiectul este fișierul care se dorește accesat. În cadrul fișierului sunt reținute permisiunile de acces pe baza cărora monitorul de referințe (adică sistemul de operare) oferă accesul. Modelul subiect-obiect este simplificat în cazul permisiunilor pe sistemul de fișiere: nu există intrări pentru fiecare subiect (proces) ci sunt agregate în cele trei clase de entități: utilizator (*user*), grup (*group*), ceilalți (*others*).

În contextul modelului subiect-obiect, vorbim de cele trei tipuri de acțiuni legate: autentificarea, autorizarea și controlul accesului.

Autentificarea este acțiunea prin care un subiect este identificat în sistem. În acel moment, există un element de identificare (*authentication token*) cunoscut sistemului. Un exemplu este autentificarea pe bază de parolă prin care un utilizator poate crea

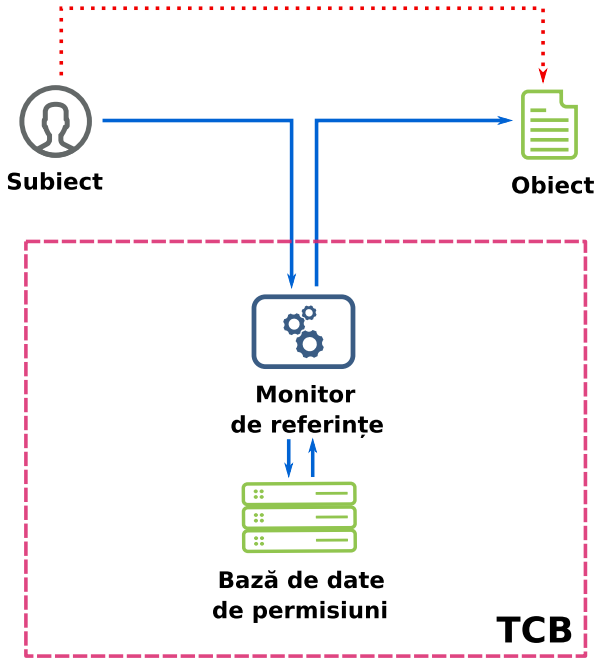


Figura 12.5: Monitorul de referințe (reference monitor)

procese cu un anumit identificator de utilizator (UID). După autentificare, subiectul poate accesa obiecte.

Accesul subiectului la obiect este condiționat de baza de date de permisiuni care asociază un subiect cu un obiect și cu permisiuni de acces. Completarea unei intrări în această bază de date de permisiuni, adică adăugarea unei noi permisiuni pentru un subiect la un obiect poartă numele de **autorizare** (*authorization*). Revocarea autorizării (*unauthorizing*) este operația inversă, de eliminare a unei permisiuni. În Linux, autorizarea în sistemul de fișiere este realizată de utilitarele **`chmod`** și **`chown`**.

În momentul în care un subiect urmărește accesarea unei resurse, accesul este validat de monitorul de referințe prin consultarea bazei de date de permisiuni. Această consultare și permiterea sau interzicerea accesului poartă numele de **controlul accesului** (*access control*). Controlul accesului are loc, în Linux, la rularea oricărei comenzi care accesează într-o formă sau alta un fișier: **`cat`**, **`ls`**, **`vim`**.

12.2 Securitatea datelor

Ținta predilectă a unui atacator o reprezintă datele unei persoane sau ale unei organizații. Aceste date pot conține elemente critice precum parole, date financiare, informații esențiale pentru funcționarea organizației sau planuri care nu trebuie sa ajungă la competitori.

În cazul cel mai comun, datele sunt citite de atacator și acesta le poate folosi ulterior direct sau indirect (le poate vinde unei entități interesate). Altfel, un atacator poate șterge datele sau poate să le suprascrie. În această situație, atacatorul nu găsește o utilitate a informațiilor, dar ștergerea sau suprascrierea lor este problematică pentru victima atacului.

Pentru a preveni accesul unui atacator la date, trebuie configurate permisiuni de acces pe datele respective, stocate în general în fișiere. Pe lângă aceasta, pentru a preveni citirea datelor, mai ales în cazul în care sunt transferate prin rețea, acestea vor fi criptate. Permisiunile de acces pe fișiere și criptarea datelor sunt principalele mijloace de asigurare a confidențialității datelor.

12.2.1 Securitatea în sistemul de fișiere

Pentru accesul corect la fișierele care conțin informații confidențiale, sistemele de operare moderne permit configurarea de **permisiuni de acces** (*access permissions* sau *access rights*). Așa cum am precizat mai sus, în cazul acestor permisiuni, utilizatorii și procesele lor sunt subiecții, iar fișierele sunt obiectele. În forma simplă, permisiunile pe fișiere sunt de citire, scriere și execuție, așa cum am indicat în Secțiunea 5.5.

Astfel, un utilizator neprivilegiat va avea acces de scriere doar în directorul său home, va avea acces de citire la acele fișiere de sistem de care au nevoie procesele sale și nu va avea nici o formă de acces la fișierele cu informații critice ale sistemului. Aceasta reprezintă o implementare a principiului celui mai mic privilegiu. În această implementare, dacă un atacator obține acces la contul unui utilizator neprivilegiat (obține parola acestuia sau exploatează o aplicație a acestuia), va avea acces limitat la fișierele din sistem.

O formă suplimentară de protejare a accesului la sistemul de fișiere este folosirea unui utilitar de tipul **`chroot`**. **`chroot`** este un utilitar Linux care schimbă directorul rădăcină al sistemului de fișiere pentru un proces. Astfel, un proces nu va mai avea `/` (*slash*) ca director rădăcină, ci un subdirector al acestuia, de exemplu `/var/lib/app/`. În această situație, procesul care rulează prin **`chroot`** va avea acces doar la fișierele din ierarhia `/var/lib/app/`, nu la toate fișierele din sistemul de fișiere, limitând daunele care ar fi produse dacă ar fi exploatat. Această tehnică mai poartă numele de **jailing**.

12.2.2 Confidențialitatea datelor

Sistemele și dispozitivele moderne au legătură aproape permanentă la Internet și transferă informații între ele. În această situație, permisiunile pe fișiere în sistemul local sunt nerelevante pentru un atacator care poate să captureze datele când sunt transferate. De aceea avem nevoie de moduri de a ascunde aceste date astfel încât, chiar capturate, să nu fie citite de un atacator, adică să păstrăm **confidențialitatea datelor**. Confidențialitatea datelor este asigurată în general prin **criptare**.

În plus, și pe un sistem local este recomandat ca datele critice să fie criptate. Dacă permisiunile de acces nu sunt configurate corespunzător sau dacă un atacator reușește să treacă de permisiuni sau să exploateze un proces privilegiat care are acces la date, atunci va putea citi datele critice. Criptarea datelor critice pe sistemul local, peste permisiunile de acces corespunzător configurate, este o formă de **securitate în adâncime**.

Criptarea datelor presupune că datele de intrare sunt supuse unui proces (de criptare) din care rezultă date de ieșire ascunse. În criptologie, datele mai sunt numite și **mesaje**.

Avem, așadar, un mesaj de intrare și un mesaj de ieșire. Mesajul de intrare este numit **mesaj inteligibil** (*plaintext*), iar mesajul de ieșire este numit **mesaj ascuns** (*ciphertext*). Pentru criptarea unui mesaj de tip plaintext într-un mesaj de tip ciphertext este nevoie de un **algoritm de criptare**. Algoritmul de criptare este setul de pași prin care mesajul de tipul plaintext este prelucrat în mesajul de tipul ciphertext. Algoritmul de criptare folosește în mod tipic o **cheie de criptare**. Aceasta modifică funcționarea algoritmului astfel încât, în cazul aceluiași mesaj de tip plaintext și aceluiași algoritm de criptare, dar chei de criptare diferite, vor rezultate mesaje de tip ciphertext diferite. Funcționarea criptării cu cele patru componente (mesaj de tip plaintext, mesaj de tip ciphertext, algoritm și cheie) este descrisă schematic în Figura 12.6.

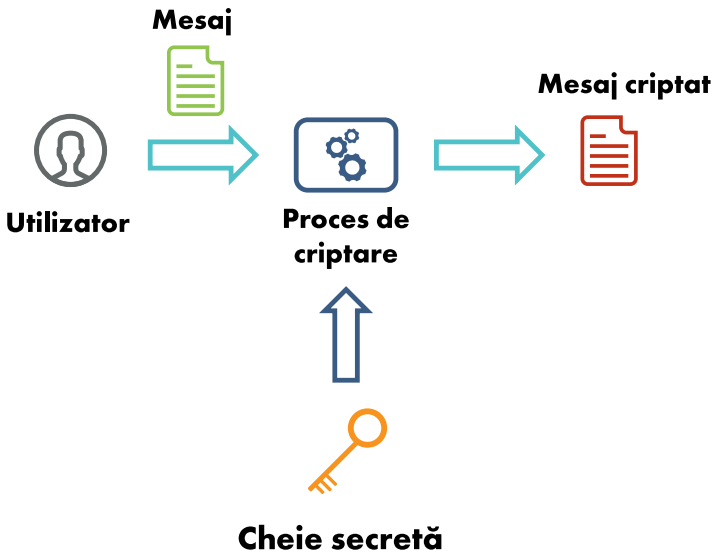


Figura 12.6: Criptarea datelor

Un mesaj ciphertext va fi ascuns atacatorului, pentru că acesta nu cunoaște cheia de criptare. Dar va trebui decriptat de un utilizator valid. **Decriptarea** este operația inversă criptării în care mesajul de intrare este mesajul de tip ciphertext iar mesajul de ieșire este mesajul de tip plaintext. Pentru a putea decripta un fișier de tip ciphertext, un utilizator are nevoie de acces la o cheie de decriptare și să cunoască algoritmul de decriptare. Dacă această cheie este aceeași ca cea de criptare spunem că folosim criptare simetrică, altfel spunem că folosim criptare asimetrică.

În cazul **criptării simetrice**, aceeași cheie este folosită și pentru criptare și decriptare ca în Figura 12.7. Cel mai folosit algoritm de criptare simetrică este AES (*Advanced Encryption Standard*). Cheia de criptare trebuie știută atât de entitatea care face criptarea cât și de entitatea care face decriptarea. În cazul probabil în care aceste entități sunt în locuri diferite în Internet, cheia trebuie transferată prin rețea. Pentru aceasta, în mod uzual, se folosesc algoritmi de schimbare de chei (*key exchange*). De exemplu, un astfel de algoritm este Diffie-Hellman-Merkle.

În cazul **criptării asimetrice**, numită și **criptare cu chei publice** (*public key cryptography*), se folosește o cheie pentru criptare și o alta pentru decriptare. Cele două chei (numite cheie privată / secretă și cheie publică) sunt legate matematic. În general cheia privată este generată aleator, iar cheia publică este generată din cheia privată. Cheia publică este accesibilă întregii lumi, în vreme ce cheia privată este accesibilă doar entității care o deține. Criptarea este realizată folosind cheia publică, iar decriptarea este realizată folosind cheia privată, ca în Figura 12.8. Adică oricine poate

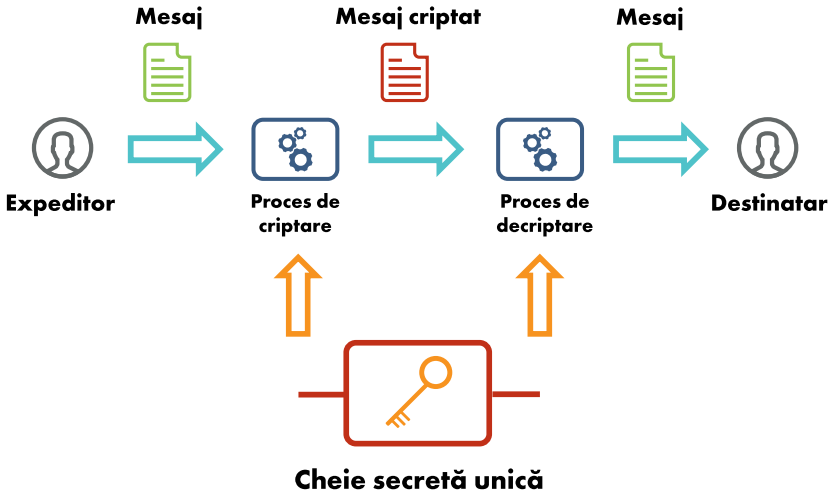


Figura 12.7: Criptarea simetrică

cripta și transmite un mesaj folosind cheia publică, dar doar deținător cheii private poate decripta acel mesaj. Cel mai răspândit algoritm de criptare asimetrică este RSA. Numele vine de la creatorii săi (Rivest–Shamir–Adleman).

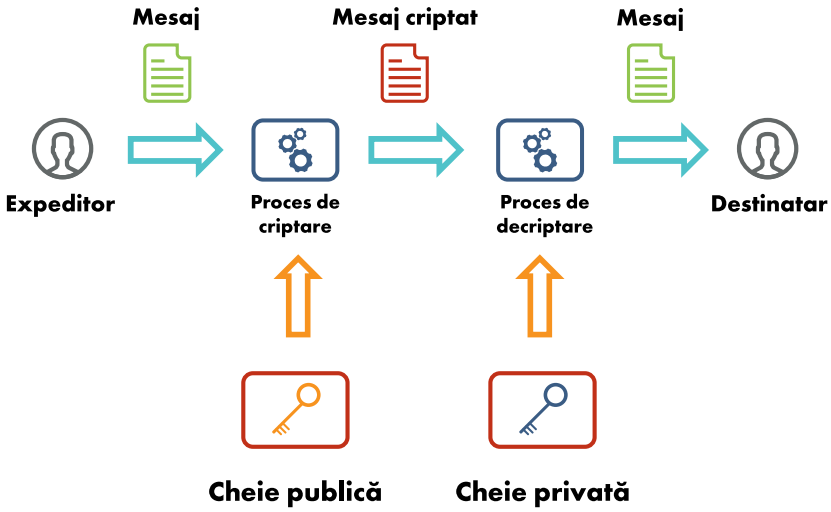


Figura 12.8: Criptarea asimetrică (cu chei publice)

Criptarea asimetrică este avantajoasă pentru că nu este nevoie de transmiterea unei chei. Cheia privată este generată locală și ținută local, iar cheia publică este furnizată publicului larg pentru a o folosi pentru transmiterea de mesaje criptate. În felul acesta evităm neajunsul partajării cheii de la criptarea simetrică. Dezavantajul criptării asimetrice este viteza: este mai puțin performantă decât criptarea simetrică. De aceea, în practică, se folosește criptarea asimetrică pentru stabilirea unui canal sigur de comunicare pe baza căreia se stabilește cheia simetrică (doar pentru acea sesiune) și apoi se folosește criptarea simetrică. Este modul de funcționare al algoritmului Diffie-Hellman-Merkle. Succint, algoritmul urmează pașii de mai jos. Considerăm două entități (un transmițător și un receptor) și un canal public de comunicație (accesibil atacatorului):

1. Ambele entități agreează un număr partajat (transmis pe canalul public).
2. Fiecare entitate are un număr privat și generează o cheie privată din numărul

public și numărul privat. Cele două chei, deși private, au o legătură matematică dată de numărul partajat.

3. Fiecare entitate generează o cheie publică din cheia privată și este transmisă celeilalte entități (pe canalul public).

4. Dată fiind legătura matematică între cele două chei publice, cele două entități pot calcula un număr nou, identic, care va fi cheia pentru criptarea simetrică, cheie ce nu a fost transmisă pe canalul public.

Chiar dacă un atacator nu poate obține cheia de criptate, algoritmii de criptare nu sunt imbatabili. Un algoritm de criptare slab va putea fi „spart" de un atacator (adică poate fi determinată cheia de criptare) dacă atacatorul are acces la suficient de multe mesaje criptate de tip ciphertext (de exemplu, capturate din rețea). Această acțiune este numită **criptanaliză**. Un algoritm de criptare robust rezistă la atacuri criptografice și nu permite descoperirea cheii atunci când atacatorul are acces la multe mesaje de tip ciphertext.

Criptarea poate fi folosită și de atacator în cazul atacurilor de tipul **ransomware**. În acest tip de atac, datele victimei nu sunt utile atacatorului, dar atacatorul știe că aceste date sunt utile victimei. Așadar, acesta le criptează și solicită o sumă de bani pentru ca victima să poată recupera datele. Evident, atacatorul trebuie în primă fază să poată ajunge la date. Așa că protejarea datelor prin permisiuni de acces este importantă. O altă soluție este backupul periodic al datelor (despre care am discutat în Secțiunea 10.6, care rezolvă și probleme de ștergere eronată a datelor sau defecte hardware ale dispozitivelor de stocare.

De obicei, un utilizator nu va folosi direct utilitare de criptare. Algoritmii de criptare sunt, în general, implementați în formă de biblioteci care apoi sunt folosite de aplicații care folosesc funcționalitatea de criptare. De exemplu, un browser web poate folosi o bibliotecă de criptare pentru a cripta parolele salvate de utilizator într-un fișier local sau un utilizator poate decide să își cripteze o partiție de pe disc, fiind responsabilitatea utilitarelor de formatare și montare să folosească o bibliotecă de criptare.

Una dintre cele mai folosite biblioteci de criptare, prezentă în general în sistemele Linux, este OpenSSL. Biblioteca OpenSSL oferă și un utilitar care poate fi folosit în linia de comandă pentru criptarea și decriptarea datelor. În Listing 12.1 folosim utilitarul **`openssl`** și algoritmul AES pentru a cripta fișierul `plain.txt` în fișierul `cipher.dat`, iar apoi decriptăm fișierul `cipher.dat` în fișierul `decrypted.txt`. În final observăm că fișierul `decrypted.txt` și fișierul `plain.txt` sunt identice. Deci ambele operații (criptare și decriptare) au fost realizate cu succes.

```
1  student@uso:~$ cat plain.txt
2  chow time
3  student@uso:~$ openssl enc -aes256 -in plain.txt -out cipher.dat
4  enter aes-256-cbc encryption password:
5  Verifying - enter aes-256-cbc encryption password:
6  *** WARNING : deprecated key derivation used.
7  Using -iter or -pbkdf2 would be better.
8  student@uso:~$ openssl enc -aes256 -in plain.txt -out cipher.dat -pbkdf2
9  enter aes-256-cbc encryption password:
10 Verifying - enter aes-256-cbc encryption password:
11 student@uso:~$ xxd cipher.dat
12 00000000: 5361 6c74 6564 5f5f 094d d919 9e8c 7558  Salted__.M....uX
13 00000010: 711b 9862 4488 ddd2 9332 d5ce 66b2 b91d  q..bD....2..f...
```

```
14 student@uso:~$ openssl enc -aes256 -d -in cipher.dat -out decrypted.txt -
   pbkdf2
15 enter aes-256-cbc decryption password:
16 student@uso:~$ cat decrypted.txt
17 chow time
```

Listing 12.1: Criptare și decriptare folosind openssl

12.2.3 Integritatea datelor

Un atacator poate urmări să citească datele, sau, în cazul în care acestea sunt criptate, poate decide să le modifice. Aceasta va împiedica receptorul datelor să le folosească sau le va folosi într-un mod necorespunzător, benefic atacatorului. Chiar și în absența atacatorului, datele pot fi corupte de defecte hardware ale dispozitivelor de stocare sau a celor de rețea. De aceea este necesar ca, în cazul transferului datelor, să asigurăm integritatea acestora.

Integritatea datelor se realizează în general cu algoritmi de hashing. Un algoritm de hashing generează un rezumat de mici dimensiuni pentru un mesaj de intrare. Un fișier oricât de mare va avea un rezumat de doar câteva zeci de octeți, numit **sumă de control** (*checksum*). Algoritmi întâlniți de hashing sunt MD5 sau SHA. Receptorul unui mesaj va primi și suma de control acestui mesaj și va putea verifica faptul că mesajul este corect aplicând algoritmul de hashing peste mesaj și comparând rezultatul cu suma de control primită. Generarea sumei de control și verificarea sa sunt prezentate în Figura 12.9.

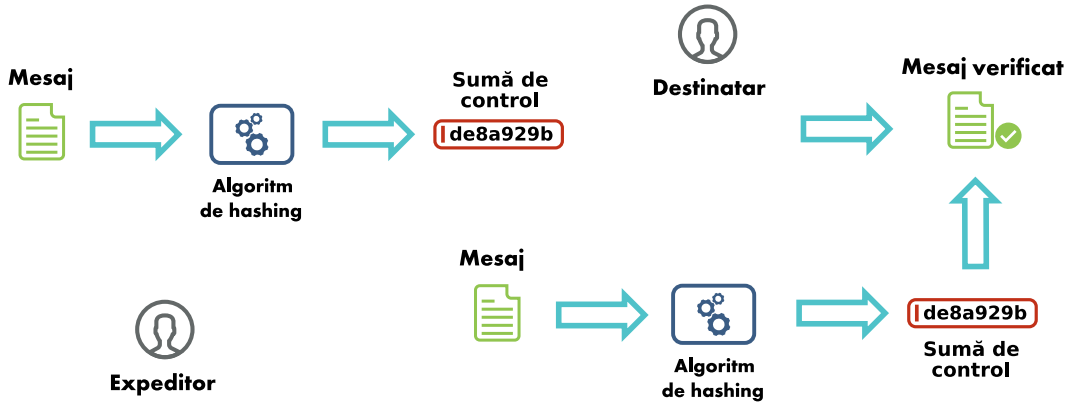


Figura 12.9: Generarea și verificarea sumei de control

În cazul unui algoritm de hashing mai multe mesaje de intrare (de dimensiuni mari) pot genera același rezumat. Numim această situație o **coliziune** (*collision*). Pentru ca această situație să nu fie abuzată, un algoritm de hashing trebuie să fie rezistent la coliziuni (*collision resistance*) și să fie cvasi-imposibil pentru un atacator să furnizeze un mesaj diferit cel inițial care să genereze același rezumat.

La fel ca în cazul algoritmilor de criptare, algoritmii de hashing sunt încorporați în biblioteci precum OpenSSL. Aplicații care transferă date prin rețea, precum clienții BitTorrent, vor folosi implementările de algoritmi de hashing din aceste biblioteci pentru a garanta integritatea datelor transferate. În Linux, există utilitare care implementează algoritmi de hashing pentru lucrul cu fișiere. De exemplu dacă transferăm un fișier de mari dimensiuni, precum o imagine de mașină virtuală, este recomandat să transferăm

și rezumatul acesteia pentru ca cei care o vor descărca să se asigure că datele sunt integre. În Listing 12.2 folosim utilitarele **`md5sum`** și **`sha256sum`** pentru a calcula rezumatul MD5 și SHA-256 pentru o imagine de mașină virtuală de dimensiune mare (`12GB`).

```
1 student@uso:~$ md5sum SSS-Kali-amd64.ova
2 3dbd973d2a331e5e755e7949274da626  SSS-Kali-amd64.ova
3 student@uso:~$ sha256sum SSS-Kali-amd64.ova
4 caefeebfba5bb19c2acb7f77b373653e5a3179ddf46bb68ddbcad612b545debb  SSS-
  Kali-amd64.ova
5 student@uso:~$ ls -sh SSS-Kali-amd64.ova
6 12G SSS-Kali-amd64.ova
```

Listing 12.2: Calcul rezumat (checksum)

12.3 Securitatea accesului. Autentificare

Pentru ca un utilizator să poată crea procese pentru folosirea unui sistem de calcul, acesta trebuie să se autentifice, adică să certifice o **identitate** care poate accesa sistemul. În absența autentificării, un utilizator nu poate folosi un sistem de calcul. Se previne astfel accesul utilizatorilor neautorizați în sistem.

Autentificarea presupune transmiterea unor informații de autentificare compuse uzual dintr-un nume de utilizator (*username*) și un element de autentificare (*authentication token*), adesea o parolă. Este forma descrisă în Secțiunea 5.4. În afara parolei, un utilizator poate folosi date biometrice (precum amprentă digitală, comună la laptopuri sau telefoane mobile, amprentă facială, precum cea folosită de iPhoneX sau recunoașterea retinei), poate folosi autentificare cu chei publice sau certificate digitale sau poate folosi un dispozitiv hardware (precum tokenii folosiți pentru autentificarea în conturile online de bancă). Dispozitivele hardware (*hardware tokens*) sau aplicațiile de pe dispozitivele mobile folosesc parole de tip one-time (*one-time passwords*). Acestea sunt parole care expiră în scurt timp, după expirarea unei parole generându-se una nouă. În acest fel se previne reținerea unei parole de un atacator.

Așa cum este precizat în Figura 12.10, în cadrul procesului de autentificare un utilizator transmite un nume de utilizator și un element de autentificare. Sistemul are o **bază de date de autentificare** (*authentication database*) pe care o consultă și vede dacă există o intrare pentru numele de utilizator furnizat cu elementul de autentificare corespunzător. Accesul este permis dacă ambele sunt găsite în baza de date de autentificare. Altfel, accesul este blocat.

Pentru cazul în care avem mai multe servicii (care se pot localiza pe mai multe sisteme) care folosesc același nume de utilizator și element de autentificare, putem folosi autentificare centralizată, așa cum am prezentat în Secțiunea 5.4.3. În această situație, baza de date de autentificare este gestionată de un serviciu dedicat care este contactat atunci când dorim autentificarea pentru un alt serviciu.

Pentru sporirea nivelului de securitate în cazul autentificării putem folosi **autentificare de tip multi-factor** (*multi-factor authentication*). Forma uzuală este **two-factor authentication** (2FA). În acest caz sunt folosite mai multe moduri de autentificare: o parolă și o parolă de tip one-time sau o parolă și un identificator biometric.

Recomandăm ca, pentru servicii sensibile (precum e-mail, rețele sociale, servicii de stocare de date), să folosiți autentificare multi-factor.

12.3.1 Gestiunea parolelor

Cel mai întâlnit element de autentificare îl reprezintă parolele. Parolele sunt șiruri de caractere furnizate sistemului de autentificare pentru a certifica identitatea utilizatorului. Parolele trebuie să fie păstrate private, altfel un atacator va impersona un utilizator și se va autentifica în sistem pretinzând că este acesta. De aceea parolele trebuie gestionate cu grijă.

Parolele trebuie reținute în baza de date de autentificare descrisă în Secțiunea 5.4.1. Este problematic ca parolele să fie păstrate în format citibil (*plaintext*). Dacă baza de date este compromisă, un atacator va avea acces la parolele tuturor utilizatorilor. De aceea, în general nu sunt stocate parolele în format plaintext în baza de date, ci se stochează un rezumat al acestora obținut cu ajutorul unui algoritm de hashing așa cum am descris în Secțiunea 12.2.3. Algoritmii de hashing sunt numiți algoritmi de tipul one-way, pentru că putem obține rapid rezumatul unui mesaj (în acest caz o parolă), dar funcția nu este reversibilă. Astfel, pentru autentificarea într-un sistem se parcurg pașii de mai jos, descriși și în Figura 12.10.

1. Utilizatorul care dorește autentificarea furnizează un nume de utilizator și o parolă.
2. Pe parolă se aplică algoritmul de hashing și se obține rezumatul.
3. Se caută în baza de date intrarea compusă din numele de utilizator și rezumatul mesajului. Dacă este găsită o astfel de intrare, este permis utilizatorului accesul în sistem.

Figura 12.10: Autentificarea cu parolă

Chiar dacă parola nu este reținută în format plaintext în baza de date, ci ca rezumat, accesul la baza de date trebuie restricționat. Este o formă a principiului de securitate în adâncime. În Linux, așa cum am prezentat în Secțiunea 5.4.1, informațiile despre utilizatori sunt reținute în fișierul `/etc/passwd` care este citibil de către toți utilizatorii. Însă informațiile despre parole sunt reținute în fișierul `/etc/shadow` care este citibil doar de procese privilegiate. În plus, în fișierul `/etc/shadow` parolele sunt reținute ca rezumat, nu în format plaintext.

Obținerea unei parole este, pentru un atacator, modul de a obține acces în sistem. Acesta poate obține parola fie de la utilizator sau acțiunile acestuia (inginerie socială, captură de mesaje, aflarea parolei scrisă pe o foaie sau într-un fișier accesibil, șantaj, amenințare) sau poate încerca ghicirea sau aflarea parolei direct de la sistemul de autentificare. Numim ultima abordare **spargerea parolei** / **contului** (*password cracking*). Spargerea parolei are două forme:

1. *Online password cracking*: atacatorul încearcă pe rând parole și solicită sistemului de autentificare să valideze dacă parola este corectă.
2. *Offline password cracking*: atacatorul obține acces la baza de date de parole (stocate în format rezumat) și calculează rezumate de parole pentru a le compara cu cele din baza de date.

Există utilitare dedicate pentru a asista la spargerea parolei. Utilitare precum THC-Hydra[1] sau Burp Suite[2] sunt folosite pentru online password cracking, iar utilitare precum John the Ripper[3] sunt folosite pentru offline password cracking. Site-ul Crackstation[4] oferă serviciul de offline password cracking, utilizatorul trebuind să furnizeze rezumate de parole pentru ca acesta să poată raporta dacă deține parola inițială. Avantajul online password cracking este că nu necesită accesul la baza de date de parole (adică să existe înainte un atac de tipul *password leak*). Dar nu se pot testa foarte multe parole pe secundă, ca în cazul offline password cracking; în plus, un sistem va raporta situația sau se va dezactiva în cazul unui număr mare de încercări eșuate pe secundă.

Parolele rămân forma principală de autentificare pentru multe servicii în Internet. Într-o astfel de lume interconectată, riscurile de aflare a parolei de un atacator în modurile descrise mai sus sunt mari. De aceea există recomandări de generare, folosire și gestionare a parolelor precum cele descrise mai jos. Multe organizații stabilesc și aplică politici de folosire a parolelor pentru a minimiza riscul de aflare a acestora și pentru a preveni accesul atacatorilor la resursele organizației:

- Parolele să conțină un set de caractere cât mai larg, să nu fie doar litere mici. Pot conține litere mari, cifre, semne de punctuație.
- Parolele trebuie să aibă o lungime de peste 10-12 caractere. Parolele cu dimensiune mai mici pot fi ghicite sau sparte. Este ideală folosirea de passphrase-uri, adică parole compuse din cuvinte, ideal nelegate între ele, mai greu de ghicit și mai ușor de reținut. Așa cum este descrisă într-o celebră ilustrație xkcd[5], o parolă precum `Tr0oub4dor&3` este mai greu de reținut și mai ușor de spart decât passphrase-ul `correct horse battery staple`.
- Este recomandată folosirea multi-factor authentication. În general se folosește telefonul mobil (trimitere de SMS sau aplicații precum Google Authenticator) ca o formă suplimentară de autentificare, nu doar parole. În acest fel, dacă cineva obține parola, nu va putea accesa serviciul fără acces la telefonul mobil al utilizatorului. În plus, utilizatorul va fi notificat de încercarea de autentificare.
- Parolele să fie schimbate periodic. Este ideal ca o parolă să fie schimbată o dată la 6-12 luni. Dacă a fost spartă/ghicită, parola va deveni inutilă odată ce a fost schimbată.

În zilele noastre, avem multe servicii la care avem acces pe bază de parole: sistem desktop, e-mail, online shopping, online media, rețele sociale, servicii de stocare, servicii financiare. Există riscul să ajungem să folosim o parolă de două sau mai multe ori pe același site/serviciu. E important să folosim parole diferite pentru servicii diferite. Dacă

[1] https://github.com/vanhauser-thc/thc-hydra
[2] https://portswigger.net/burp
[3] https://www.openwall.com/john/
[4] https://crackstation.net/
[5] https://xkcd.com/936/

o parolă este ghicită accesul la un singur serviciu este compromis.

Ținând cont de numărul mare de parole folosite, trebuie să fie generate parole sau passphrase-uri diferite și greu de ghicit. Pentru aceasta este util să apelăm la **generatoare de parole**. Browserele web moderne au integrate generatoare de parole care vor completa automat câmpurile din formularele web de creare de conturi și parole. Pe lângă acestea putem folosi utilitare Linux pentru generarea de parole, precum **`pwgen`**, **`apg`** sau **`xkcdpass`**, sau putem folosi un manager de parole.

Pentru gestiunea și stocarea parolelor este recomandată folosirea unui **`manager de parole`** (*password manager*). Un manager de parole reține informații despre serviciul / site-ul folosit și parola corespunzătoare și criptează aceste informații. Pentru accesarea lor este nevoie de o parolă principală (*master password*). Managerii de parole pot fi aplicații care funcționează doar local (precum UPM, KeePass sau PasswordGorrila) sau altele care stochează informațiile în cloud, permițând partajarea lor între diferite dispozitive (precum LastPass, Dashlane sau Keeper). Pe lângă stocarea sigură a parolelor, managerii de parole oferă funcționalități precum generare de parole, completare automată în formulare, partajare de parole între mai multe conturi.

Parolele sunt o componentă critică în accesul utilizatorilor la dispozitive și servicii din Internet. De aceea trebuie protejate de cei care proiectează și administrează sisteme și trebuie gestionate cu grijă de fiecare utilizator. Numărul mare de site-uri și servicii pe care la folosim cu ajutorul parolelor crește riscul pierderii acestora și trebuie urmate recomandări precum cele de mai sus pentru gestiunea lor.

12.4 Securitatea transferului

În zilele noastre, foarte multe date rezidă pe sisteme de tip server accesibile prin Internet. Aceste date sunt transferate între dispozitivele cu care lucrăm și aceste sisteme de tip server. Numim aceste sisteme de tip server cloudul Internetului, de unde și expresia „ținem datele în cloud”. Accesul la serviciile oferite în Internet presupune transferul de informații: un utilizator folosește o aplicație de tip client pe un dispozitiv pentru a face o cerere către o aplicație de tip server de pe un sistem server și primește un răspuns. Așa cum am prezentat în Figura 11.3 și Figura 11.4.

Transferul datelor are loc peste infrastructură de Internet (medii de transmisie și echipamente dedicate) pentru care nu avem garanții. Aceste medii pot fi ostile și datele pot fi capturate, citite, sau modificate. Numim acest tip de atac **man-in-the-middle**. Un atacator cu acces la mediile de transmisie sau echipamentele dedicate va avea acces la datele transferate.

De aceea este important ca în cazul transferului de date să avem două proprietăți:

1. Confidențialitate: să ne asigurăm că datele transferate nu sunt capturate și interpretate de atacatori pe infrastructura de rețea.
2. Autenticitate și integritate: să ne asigurăm că datele transferate sosesc de la / pleacă la entitatea la care ne așteptăm, în formă neschimbată / nemodificată.

Fiecare dintre cele două proprietăți este de regulă implementată în practică prin metode criptografice: criptare în cazul confidențialității și hashing în cazul integrității, așa cum

am descris și în Secțiunea 12.2.2, respectiv Secțiunea 12.2.3.

12.4.1 Identitate

Esențială în cazul transferului de date între două entități este noțiunea de **identitate**. Fără aceasta putem avea o comunicație confidențială și integră, dar cu o entitate care impersonează pe cea cu care chiar dorim să comunicăm. Pentru a putea asigura autenticitatea datelor transmise avem nevoie de un mijloc de a garanta identitatea entității cu care comunicăm. Numim aceasta **certificarea identității**. Certificarea identității este legată de noțiunea de **încredere** (*trust*). Avem nevoie de o entitate în care să avem încredere care să certifice identitatea altor entități. Depinzând de tipul acestei entități de încredere, avem o abordare centralizată sau o abordare descentralizată.

Abordarea centralizată este similară cu obținerea unui act de identitate de la organismele unui stat. Avem încredere că organismul în cauză (de exemplu, Ministerul Administrației) cunoaște identitatea unei persoane și emite un act de tipul certificat care atestă acest lucru. În mod similar, în infrastructurile IT există noțiunea de **autoritate de certificare** (*Certification Authority*) care emite certificate digitale pentru a atesta identitatea unei entități.

De partea cealaltă, abordarea descentralizată este de tipul „comunitate”. Lumea din comunitate știe că cineva este cine spune că este și ne bazăm pe membrii comunității. În infrastructurile IT această abordare este numită **rețea de încredere** (*web of trust*) și se bazează, după cum îi spune și numele, pe construirea unei rețele de încredere și pe schimbul de informații de identitate.

12.4.2 Semnături digitale. Certificate digitale

La baza identității digitale stă conceptul de semnătură digitală. Semnătura digitală este aplicată unei informații pentru a garanta autenticitatea acelei informații. Adică, similar unei semnături fizice pe un document, o semnătură digitală pe un document electronic va certifica acel document.

Semnăturile digitale se realizează cu ajutorul unei perechi cheie publică - cheie privată / secretă pe care le-am prezentat în Secțiunea 12.2.2. În vreme ce, în cazul criptării, criptarea este realizată folosind cheia publică (oricine poate să cripteze) iar decriptarea este realizată folosind cheia privată (doar deținătorul cheii private poate să decripteze), în cazul semnării digitale lucrurile sunt invers: deținătorul cheii private folosește cheia privată ca să semneze un document electronic (doar deținătorul poate semna), iar oricine poate folosi cheia publică pentru a verifica semnătura (orice poate verifica). Aceste două acțiuni (semnare și verificare) sunt descrise în Figura 12.11.

În urma procesului de semnare cu cheia privată rezultă o semnătură digitală, un șir de octeți rezultat în urma semnării. Documentul semnat este apoi expus public, împreună cu semnătura. Cheia publică este deja publică. Oricine poate lua apoi documentul împreună cu semnătura digitală și verifica, folosind cheia publică, autenticitatea documentului. Acesta este un mod care poate fi folosit la mesaje de poștă electronică (e-mail-uri) pentru a garanta că un mesaj este cel corect, având semnătura atașată.

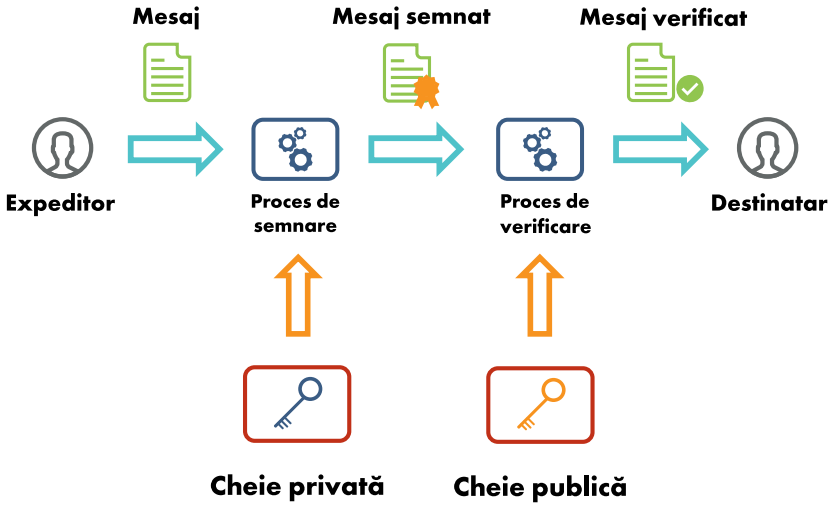


Figura 12.11: Semnare și verificare digitală

Sau este un mod în care documentele pot fi semnate, fără a fi necesară prezența fizică a semnatarilor sau forma fizică a unui document. Pentru semnături în e-mail-uri și pentru semnături de documente, în Linux putem folosi utilitarul GPG (*GNU Privacy Guard*).

În forma prezentată mai sus semnăturile digitale doar certifică autenticitatea mesajului, adică este un mesaj autentic semnat. Dar nu certifică identitatea celui care a semnat. Acesta poate pretinde că este altă persoană, genera o pereche cheie privată - cheie publică și semna mesaje folosind cheia privată iar alții pot verifica folosind cheia publică. Pentru aceasta avem nevoie de un mod de a atașa o identitate fizică (nume, e-mail) de o cheie publică și să avem încredere că acea atașare este validă; adică într-adevăr identitatea este a persoanei pretinse și este cheia publică a acesteia.

Asocierea unei identități cu o cheie publică poartă numele de certificat digital de identitate sau certificat de cheie publică sau pe scurt **certificat digital**. Pentru ca asocierea să fie validă, certificatul trebuie să fie semnat de o entitate în care avem încredere, așa cum am precizat mai sus în Secțiunea 12.4.1. În funcție de natura acestei entități avem o formă centralizată, numită infrastructură cu chei publice (*Public Key Infrastructure*) și o formă descentralizată, numită rețea de încredere (*web of trust*).

12.4.2.1 Public Key Infrastructure

În cazul formei centralizate există o entitate aparte în care avem încredere numită **autoritate de certificare** (*Certificate Authority* - CA). Similar unui minister autorizat care certifică identitatea unei persoane pe un act, autoritatea de certificare este cea care semnează un certificat care asociază identitatea de o cheie publică. Avem astfel o infrastructură cu chei publice (*Public Key Infrastructure* - PKI). Pașii urmați pentru verificarea unui document semnat în PKI sunt prezentați în Figura 12.12:

1. Clientul se conectează la server pentru accesarea resursei dorite.
2. Serverul prezintă certificatul său clientului. Acum clientul are acces la certificatul serverului (și la certificatul autorității de certificare - CA).
3. Clientul extrage cheia publică a autorității de certificare din certificatul acesteia.

4. Clientul verifică, folosind cheia publică a autorității de certificare, certificatul serverului. Dacă verificarea a reușit, acum clientul are încredere în identitatea serverului și poate comunica în siguranță cu acesta.
5. (opțional) Serverul solicită certificatul clientului pentru a verifica identitatea acestuia. În oglindă cu pasul 1.
6. (opțional) Clientul prezintă certificatul său serverului. Acum serverul are acces la certificatul clientului (și la certificatul autorității de certificare - CA). În oglindă cu pasul 2.
7. (opțional) Serverul extrage cheia publică a autorității de certificare din certificatul acesteia. În oglindă cu pasul 3.
8. (opțional) Serverul verifică, folosind cheia publică a autorității de certificare, certificatul clientului. Dacă verificarea a reușit, acum serverul are încredere în identitatea clientului și poate comunica în siguranță cu acesta. În oglindă cu pasul 4. În acest moment ambele entități au identitățile verificate.
9. Clientul (sau serverul) prezintă cheia de criptare (în general, simetrică) folosită pentru asigurarea confidențialității sesiunii curente.
10. Acum este realizat canalul sigur de comunicare și clientul poate accesa resursă dorită.

Pașii 5-8 de mai sus sunt pași opționali de verificare a identității clientului de către server. În mod obișnuit, doar clientul este interesat de identitatea serverului. Identitatea clientului fie nu este relevantă serverului (pentru resurse publice, cum ar fi căutarea folosind Google), fie folosește forme specializate de autentificare pentru demonstrarea identității (pentru resurse private / personale, cum ar fi accesarea contului de Facebook).

În momentul care dorim să verificăm un document, vom obține certificatul digital al semnatarului, vom verifica certificatul digital folosind cheia publică (în care avem încredere) a autorității de certificare. Dacă certificatul digital este valid, vom extrage din acesta cheia publică și vom verifica apoi semnătura. Având certificatul digital verificat de autoritatea de certificare avem încredere că identitatea este cea din certificat.

În schema PKI va trebui să avem încredere în autoritatea de certificare. Vom avea acces la cheia publică a acesteia și vom avea încredere că acea cheie publică este într-adevăr a autorității de certificare. Cheia publică este folosită pentru verificare certificatelor. Această cheie publică a autorității de certificare este ea însăși distribuită în forma unui certificat. Acest certificat poate fi semnat de o altă entitate de tip CA de nivel mai înalt. La un moment dat va exista o autoritate de certificare care nu va mai fi semnată de alta, se spune că am ajuns la rădăcina schemei PKI. Această autoritate de certificare va avea un certificat rădăcină (*root certificate*) care este semnat tot cu el (*self signed*). Adică propria cheie privată este folosită pentru a semna certificatul ce cuprinde propria cheie publică și identitatea.

Schema PKI este folosită în Internet pentru a garanta identitatea site-urilor pe care le vizităm (google.com, facebook.com, amazon.com). Fiecare site are un certificat digital semnat de o autoritate de certificare în care avem încredere. În momentul în care instalăm un browser acesta are deja certificatele diferitelor autorități de certificare și poate verifica autenticitatea certificatelor furnizate de diferitele site-uri și, deci,

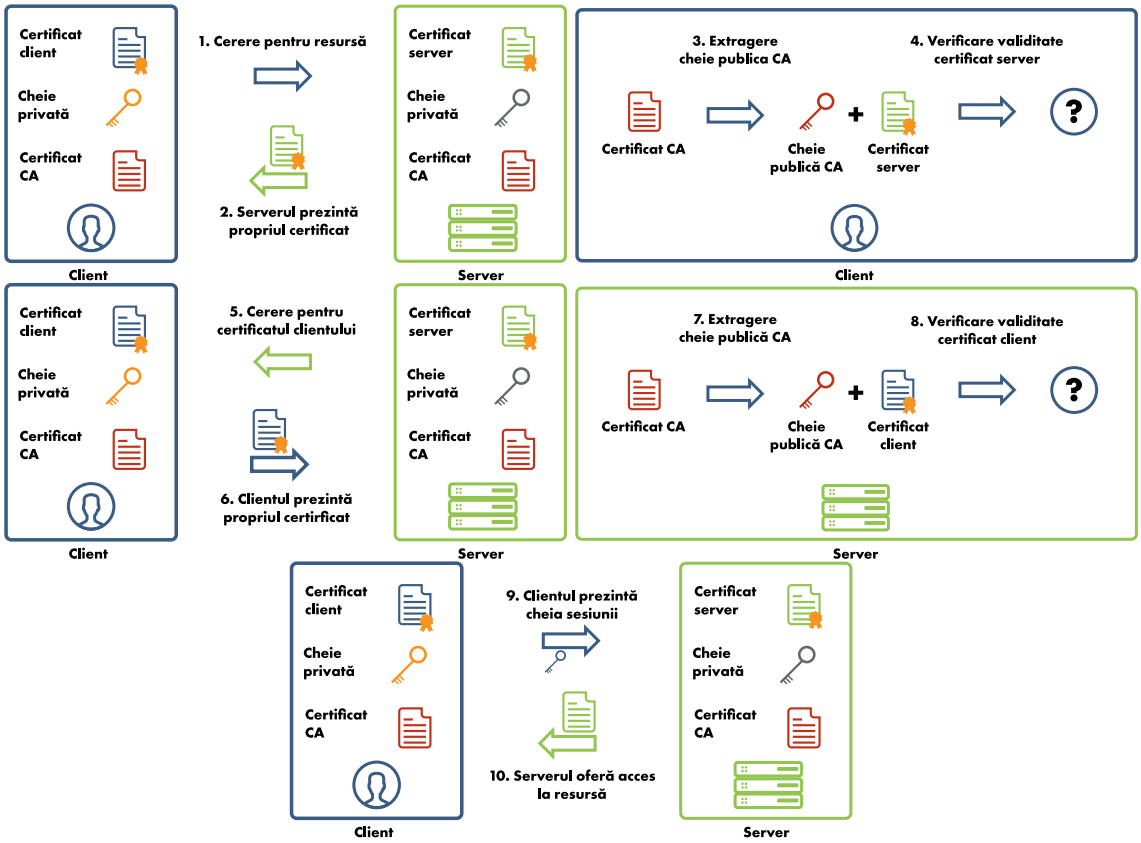


Figura 12.12: Verificarea identității în PKI

identitatea acestora.

Totodată, schema PKI este folosită pentru semnăturile electronice de documente. Forma cea mai puternică este numită **semnătură electronică calificată** (*qualified electronic signature*). Această formă este considerată în legislația multor țări ca fiind echivalentă semnăturii scrise (olografe). În general, obținerea unei semnături electronice calificate necesită obținerea unui dispozitiv (token) de semnătură electronică. Acest token reține cheia privată folosită pentru semnare. Cheia privată nu poate fi folosită în absența acestui token și, deci, semnătura electronică nu poate fi realizată. În general tokenul este obținut contra cost de la o companie autorizată ca furnizor de semnătură electronică. Cel mai adesea tokenul se conectează la portul USB al unui laptop / desktop pentru semnarea documentelor de pe acel sistem.

12.4.2.2 Web of Trust

Schema PKI este o schemă centralizată. Are avantajul organizării și a existenței unui lanț de încredere și răspundere. Cu toate acestea, dacă o autoritate de certificare este compromisă atunci toate certificatele semnate de aceasta sunt compromise. De aceea certificatele au un timp limitat de viața (de obicei 1-2 ani) după care trebuie regenerate.

Alternativa la schema centralizată PKI este schema descentralizată **web of trust**. În această schemă, fiecare utilizator cunoaște direct anumiți utilizatori (îi cunoaște fizic), îi poate garanta și le semnează certificatele fiecăruia. Apoi acest utilizator, pe baza legăturilor directe poate accepta certificate semnate de aceste legături directe. Nu

cunoaște direct utilizatorii finali dar are încrederea în semnăturile legăturilor sale directe. Funcționarea web of trust este descrisă schematic în Figura 12.13.

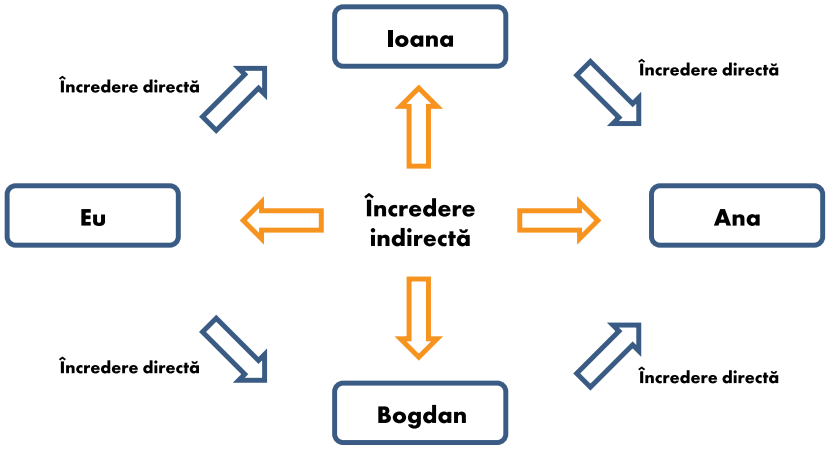


Figura 12.13: Functionarea Web of Trust

Avantajul schemei web of trust față de PKI este flexibilitatea. Neavând o autoritate centrală, nu avem parte de rigiditatea schemei PKI. Un dezavantaj al acesteia este dificultatea verificării funcționării corespunzătoare. În absența unei structuri riguroase, verificarea funcționării corespunzătoare este greoaie. Din acest motiv implementările reale la scară largă ale web of trust sunt mai reduse. O implementare întâlnită este semnarea e-mail-urilor folosind implementări din familia PGP (*Pretty Good Privacy*) precum utilitarul GPG prezentat mai sus.

12.4.3 Transport Layer Security (TLS)

Pentru asigurarea proprietăților amintite la începutul capitolului (confidențialitate, autenticitate / identitate și integritate) protocoalele de rețea moderne încorporează *Transport Layer Security* (TLS), cunoscut în vechea sa denumire și ca *Secure Sockets Layer*. TLS este un framework de securitate criptografică folosit pentru securizarea traficului în rețea. De exemplu, protocolul HTTP (*Hypertext Transfer Protocol*) nu are caracteristici de securitate, însă, dacă îi atașăm TLS, protocolul devine securizat în varianta sa HTTPS (*Hypertext Transfer Protocol Secure*).

TLS încorporează cele mai multe din conceptele pe care le-am descris în cazul acestei secțiuni: folosește certificate digitale pentru a certifica identitatea partenerului în conversație, folosește protocoale de schimb de chei precum Diffie-Hellman, folosește protocoale de cheie simetrică precum AES, folosește algoritmi de hashing precum SHA-2. În cadrul unei sesiuni TLS clientul și serverul agreează ce algoritmi să folosească în funcție de posibilitățile fiecăruia.

Versiunea curentă de TLS este 1.3 (din August 2018) și este considerată sigură. Versiunile anterioare prezintă vulnerabilități ale algoritmilor. În general, o specificație mai nouă de TLS înseamnă că anumiți algoritmi nu mai sunt permiși, sau cheile de criptare sunt mai lungi, tocmai pentru a preveni exploatarea lor. De obicei, se recomandă ca un server să refuze conexiuni care solicită algoritmi mai slabi care pot fi apoi exploatați. Acest tip de atac poartă numele de *connection downgrade attack* și

este folosit pentru a impune un algoritm mai slab pentru care se cunoaște că are vulnerabilități.

Dacă dorim să verificăm nivelul de securitate TLS pentru un server web pe care l-am configurat (sau altul) putem folosi aplicații de test precum SSL Server Test[1], așa cum este prezentat în Figura 12.14. Într-un astfel de test ne sunt raportate punctele slabe ale unui server, precum faptul că acceptă conexiuni TLS 1.1 și TLS 1.2 care pot avea probleme pentru anumiți algoritmi.

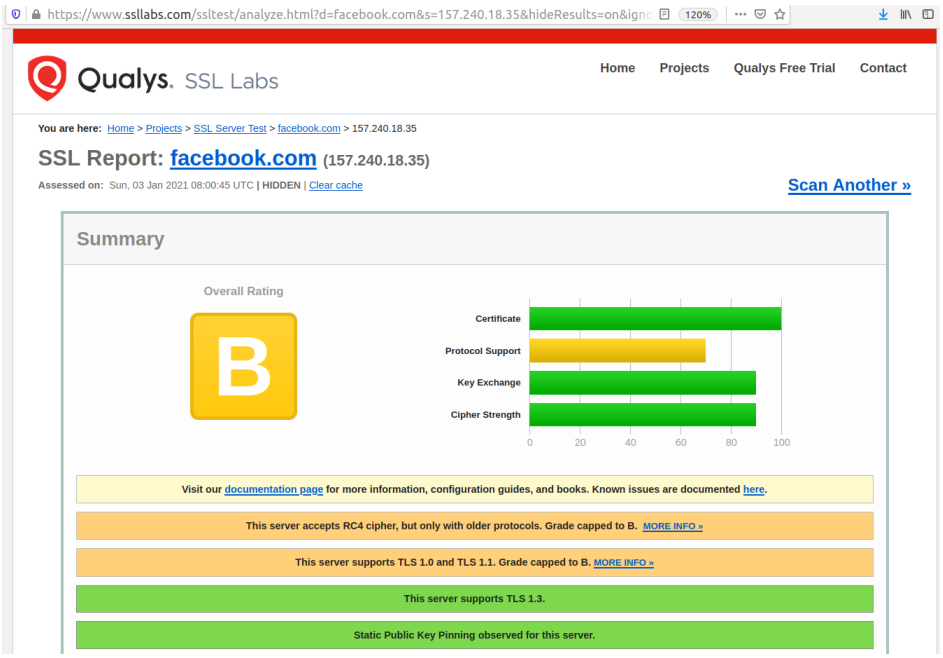


Figura 12.14: Verificarea nivelului de securitate pentru un domeniu

Similar funcționalităților de criptare, TLS este implementat de obicei în forma unor biblioteci criptografice. Protocoalele nesecurizate folosesc TLS pentru a obține forma lor securizată: este vorba de HTTP / HTTPS (pentru acces web), IMAP / IMAPS (pentru acces la căsuța poștală), LDAP / LDAPS (pentru interogarea directoarelor de informații).

Biblioteca OpenSSL, de care am amintit în Secțiunea 12.2.2 implementează TLS și este folosită de multe aplicații de rețea pentru a comunica securizat. Utilitarul în linia de comandă **`openssl`** permite, pentru scenarii de test, construirea sau verificarea unei conexiuni securizate. De exemplu, comanda din Listing 12.3 obține informații despre certificatul digital folosit de serverul `google.com`.

```
1  student@uso:~$ openssl s_client -connect google.com:443
2  CONNECTED(00000005)
3  depth=2 OU = GlobalSign Root CA - R2, O = GlobalSign, CN = GlobalSign
4  verify return:1
5  depth=1 C = US, O = Google Trust Services, CN = GTS CA 1O1
6  verify return:1
7  depth=0 C = US, ST = California, L = Mountain View, O = Google LLC, CN =
   *.google.com
8  verify return:1
9  ---
10 Certificate chain
```

[1] https://www.ssllabs.com/ssltest/

```
 0 s:C = US, ST = California, L = Mountain View, O = Google LLC, CN = *.
google.com
   i:C = US, O = Google Trust Services, CN = GTS CA 1O1
 1 s:C = US, O = Google Trust Services, CN = GTS CA 1O1
   i:OU = GlobalSign Root CA - R2, O = GlobalSign, CN = GlobalSign
---
Server certificate
-----BEGIN CERTIFICATE-----
[...]
```

Listing 12.3: Obținerea de informații despre certificatul unui domeniu

12.4.4 Secure Shell

Transport Layer Security (TLS) permite adăugarea proprietăților de securitate pentru protocoale într-o implementare în formă de bibliotecă. O alternativă la TLS pentru crearea unui canal sigur de comunicare este protocolul SSH (*Secure Shell*). Protocolul SSH, la fel ca TLS, oferă confidențialitate, integritate și autenticitate. Protocolul SSH este folosind în general pentru deschiderea unei conexiuni shell sigure la distanță, pe un canal criptat. Permite astfel administrarea unui sistem la distanță. Este, astfel, principalul utilitar folosit în lumea Linux / Unix pentru administrarea sistemelor și este prezent pe majoritatea sistemelor de tip server.

În ciuda numelui, protocolul SSH nu este folosit numai pentru deschiderea unei sesiuni de shell sigure la distanță; protocolul SSH creează un canal sigur de comunicare prin care se pot realiza acțiuni precum deschiderea unei sesiuni de shell la distanță, transferul de fișiere sau tunelarea unui protocol, descrise și în Figura 12.15. Tunelarea unui protocol nesigur (de tip *plaintext*) înseamnă că mesajele specifice acestui protocol sunt trecute prin canalul sigur de conexiune realizat de SSH. În felul acesta SSH oferă funcționalitate similară TLS, de securizare a unui protocol. Totuși, în TLS, protocolul în forma securizată este deja implementat, pe când în cazul SSH este nevoie de crearea canalului sigur de comunicare prin care apoi este tunelat protocolul nesigur. Întrucât este mai simplu de folosit, preferăm folosirea protocolului securizat folosind TLS; SSH rămâne util însă în situațiile în care nu dispunem de forma securizată a protocolului prin TLS, cum ar fi tunelarea protocolului X pentru mediul grafic în Linux, așa cum am prezentat în Secțiunea 1.2.2.

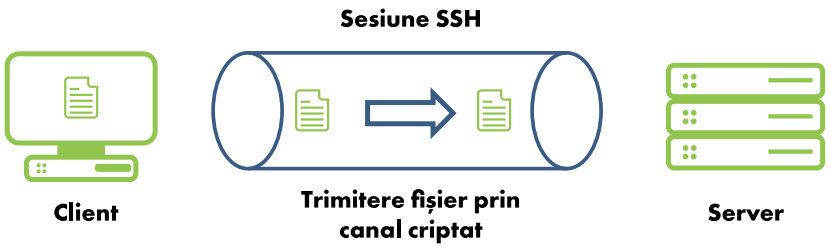


Figura 12.15: Canal securizat de comunicație cu SSH

Pentru a funcționa, protocolul SSH are nevoie de un cont de utilizator la sursă și la destinație. Practic, realizăm un canal de comunicare sigur între două conturi de utilizator, uzual pentru deschiderea unei sesiuni de shell la distanță, ca în Figura 12.16. Crearea canalului se realizează numai după ce se realizează autentificarea pe contul destinație. Autentificarea se poate realiza cu parolă sau folosind cheie publică, așa cum

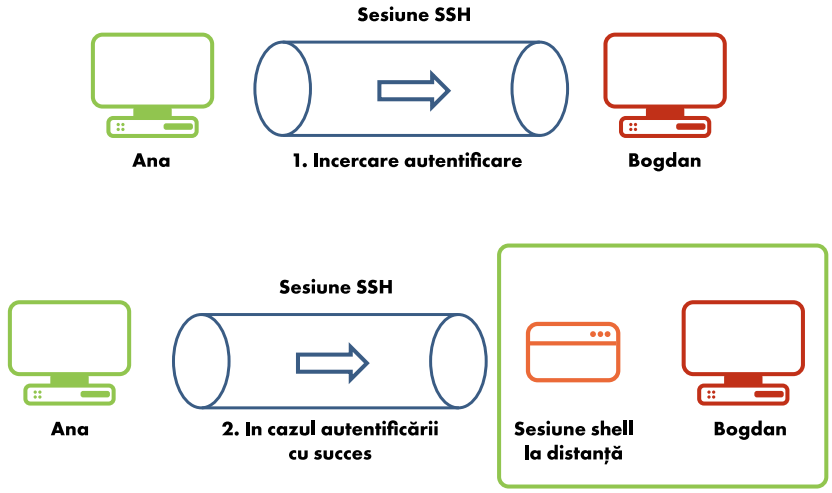


Figura 12.16: Conexiune SSH

cum descriem în continuare. O dată realizată autentificarea, avem un canal de comunicare prin care putem tunela comenzi (în cazul shellului), fișiere (în cazul transferului de fișiere) sau protocoale.

Pentru realizarea conexiunii SSH, la destinație trebuie să ruleze un server SSH, numit și daemon SSH. Serverul SSH rulează uzual pe portul 22, prezența sa putând fi verificată folosind utilitarul **`netstat`** ca în Listing 12.4.

```
1 student@uso:~$ netstat -tln | grep 22
2 tcp        0      0 0.0.0.0:22              0.0.0.0:*
  LISTEN
3 tcp6       0      0 :::22                   :::*
  LISTEN
```

Listing 12.4: Serverul SSH

Dacă nu este prezent serverul SSH este posibil ca acesta să fie oprit sau să nu fie instalat. Pentru instalarea și pornire / repornirea serverului SSH folosim comenzile din Listing 12.5. Comenzile de instalare sunt respectiv pentru sistemele Debian / Ubuntu și pentru sistemele RedHat / Fedora.

```
1 # Install SSH server on Debian / Ubuntu.
2 student@uso:~$ sudo apt install openssh-server
3
4 # Install SSH server on Debian / Ubuntu.
5 student@uso:~$ sudo dnf install openssh-server
6
7 # Restart SSH server.
8 student@uso:~$ sudo systemctl restart ssh
```

Listing 12.5: Instalare și pornire server SSH

Pentru deschiderea unei sesiuni shell la distanță folosim comanda **`ssh`** urmată de destinație: numele contului de utilizator și adresa stației destinație; adresa stației destinație poate fi adresă IP sau nume DNS. În Listing 12.6 avem exemple de conexiune la distanță. Se cere parola utilizatorului la distanță și, în cazul unei conexiuni reușite, se obține un shell care rulează la distanță cu permisiunile utilizatorului la distanță. Pentru închiderea conexiunii, închidem procesul shell folosind comenzile **`exit`** sau **`logout`** sau combinația de taste `Ctrl+d`.

```
student@uso:~$ ssh malus@vmx.cs.pub.ro
malus@vmx.cs.pub.ro's password:
[...]
malus@vmx:~$ ls
extra  from-upb-to-ncsu  iOracle.git  ios-privacy-leaks  lib  prolog-data
  prolog-query-work  pwn3  scripts
malus@vmx:~$ exit
logout
Connection to vmx.cs.pub.ro closed.
student@uso:~$
```

Listing 12.6: Acces shell la distanță folosind SSH

Comanda de conexiune la distanță poate fi urmată de o altă comandă, caz în care nu se va deschide o sesiune de shell interactivă, ci se va rula acea comandă și apoi se va închide conexiunea. În Listing 12.7 rulăm la distanță comanda **`ls`** sau o comandă mai complexă pentru a afla rapid informații de pe acel sistem fără a mai fi nevoie să se deschidă o sesiune de shell interactivă.

```
student@uso:~$ ssh malus@vmx.cs.pub.ro ls
malus@vmx.cs.pub.ro's password:
extra
from-upb-to-ncsu
iOracle.git
ios-privacy-leaks
lib
prolog-data
prolog-query-work
pwn3
scripts
student@uso:~$ ssh malus@vmx.cs.pub.ro hostname
malus@vmx.cs.pub.ro's password:
vmx.cs.pub.ro
```

Listing 12.7: Rulare comenzi la distanță prin SSH

Atunci când realizăm pentru prima oară o conexiune la un sistem la distanță vom primi un mesaj de avertizare că nu este cunoscută identitatea acelui sistem și suntem întrebați dacă dorim să continuăm. În cazul în care răspundem afirmativ și continuăm, cheia publică (identitatea) acelui sistem va fi adăugată într-un fișier de configurare specific (`~/.ssh/known_hosts`) și nu vom primi întrebarea mai târziu. În Listing 12.8 fișierul `~/.ssh/known_hosts` nu există, apoi la prima conexiune primim întrebarea și răspundem afirmativ; fișierul `~/.ssh/known_hosts` are acum intrarea corespunzătoare sistemului la distanță (cheia publică), iar la următoarea conexiune nu mai primim întrebarea, întrucât cunoaștem identitatea serverului.

```
student@uso:~$ ssh malus@vmx.cs.pub.ro
The authenticity of host 'vmx.cs.pub.ro (141.85.227.139)' can't be
established.
RSA key fingerprint is SHA256:CrRMD7nrflw7/KEZ4z7lksvEd8tXxkjuVXoBqsG5Vdc
.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'vmx.cs.pub.ro,141.85.227.139' (RSA) to the
list of known hosts.
malus@vmx.cs.pub.ro's password:
[...]
malus@vmx:~$ logout
```

```
9  Connection to vmx.cs.pub.ro closed.
10
11 student@uso:~$ ssh malus@vmx.cs.pub.ro
12 malus@vmx.cs.pub.ro's password:
13 [...]
14 malus@vmx:~$
```

Listing 12.8: Verificarea identității serverului prin SSH

12.4.4.1 Transferul fișierelor prin SSH

Pe lângă deschiderea unei conexiuni shell la distanță, protocolul SSH este folosit pentru transferul securizat de fișiere folosind utilitarul `scp` (de la *secure copy*). Comanda **`scp`** primește ca argument o cale locală și un nume de utilizator, adresă de stație și cale de la distanță. În Listing 12.9, avem un exemplu de descărcare de fișier (*download*) de la destinație (*remote*) către sursă (*local*) și unul de încărcare de fișier (*upload*) de la sursă (*local*) către destinație (*remote*).

```
1  student@uso:~$ mkdir malus
2  student@uso:~$ scp malus@vmx.cs.pub.ro:scripts/sync* malus/
3  malus@vmx.cs.pub.ro's password:
4  sync_from_ncsu_to_upb
5  sync.log
6  student@uso:~$ scp plain.txt malus@vmx.cs.pub.ro:
7  malus@vmx.cs.pub.ro's password:
8  plain.txt
9  student@uso:~$ scp malus@vmx.cs.pub.ro:scripts/sync.log .
10 malus@vmx.cs.pub.ro's password:
11 sync.log
12 student@uso:~$ ls sync.log
13 sync.log
14 student@uso:~$ scp -r malus@vmx.cs.pub.ro:scripts .
15 malus@vmx.cs.pub.ro's password:
16 .sync_from_ncsu_to_upb.un~
17 sync.log
18 sync_from_ncsu_to_upb
```

Listing 12.9: Transfer de fișiere prin SSH (scp)

De multe ori în cazul descărcării unui fișier destinația este **`.`** (punct) adică realizăm transferul de la distanță în directorul curent, ca în liniile 9-13 din Listing 12.9. Similar comenzii **`cp`**, putem folosi comanda **`scp`** pentru a transfera recursiv o ierarhie de directoare și fișiere ca în liniile 14-18 din Listing 12.9.

12.4.4.2 Autentificarea cu cheie publică prin SSH

În mod implicit, realizarea unei conexiuni SSH necesită introducerea parolei contului utilizatorului de la distanță. Așa cum am precizat în Secțiunea 12.3.1, autentificarea cu parole poate fi problematică dacă parolele nu sunt gestionate corespunzător. De aceea, în multe situații dorim să înlocuim folosirea autentificării cu parole cu altă formă de autentificare pentru SSH.

O formă alternativă de autentificare pentru SSH este autentificarea cu cheie publică. Această autentificare presupune existența unei perechi cheie publică / cheie privată, ca

în cazul semnăturilor digitale, descrise în Secțiunea 12.4.2. Cheia privată este deținută de utilizator local. Cheia publică trebuie să fie adăugată în contul utilizatorului de la distanță. În momentul în care se realizează autentificarea, protocolul SSH folosește cheia privată a utilizatorului local pentru a semna un mesaj. Mesajul și semnătură sunt transmise la distanță, unde se folosește cheia publică deja adăugată în contul utilizatorului de la distanță pentru a verifica mesajul. Autentificarea reușește dacă utilizatorul de la distanță deține cheia publică a utilizatorului local. În acel moment canalul SSH este creat.

Astfel, pentru realizarea unei conexiuni SSH cu autentificare prin cheie publică, trebuie urmați pașii următori:

1. Se generează perechea cheie privată / cheie publică în contul utilizatorului local.
2. Se adaugă cheia publică în contul utilizatorului de la distanță.
3. Se folosește cheia privată din contul utilizatorului local pentru crearea unei conexiuni SSH. Aceasta se va realiza fără parolă.

În mod obișnuit, generarea perechii cheie privată / cheie publică se realizează o singură dată. Apoi cheia publică se adaugă în conturile de utilizator de la distanță pentru a permite autentificarea fără parolă. Cheia publică SSH este configurată și în alte servicii care se bazează pe SSH precum GitHub.

În Linux, pentru generarea unei perechi cheie privată / cheie publică se folosește utilitarul **`ssh-keygen`**, ca în Listing 12.10. Așa cum se afișează și la promptul rulării **`ssh-keygen`**, în mod implicit cheile sunt generate în directorul `~/.ssh/`: cheia privată este în fișierul `~/.ssh/id_rsa`, iar cheia publică este în fișierul `~/.ssh/id_rsa.pub`. Întrucât deja existau cheile implicite, în Listing 12.10, am generat o nouă pereche cheie privată / cheie publică în fișierele `~/.ssh/new_id`, respectiv `~/.ssh/new_id.pub`.

```
1  student@uso:~$ ls ~/.ssh/
2  id_rsa  id_rsa.pub  known_hosts
3  student@uso:~$ ssh-keygen
4  Generating public/private rsa key pair.
5  Enter file in which to save the key (/home/student/.ssh/id_rsa): /home/
   student/.ssh/new_id
6  Enter passphrase (empty for no passphrase):
7  Enter same passphrase again:
8  [...]
9  student@uso:~$ ls ~/.ssh/
10 id_rsa  id_rsa.pub  known_hosts  new_id  new_id.pub
```

Listing 12.10: Generarea unei perechi cheie privată / cheie publică SSH

Adăugarea cheii publice în contul utilizatorului de la distanță se realizează în fișierul `~/.ssh/authorized_keys`. Acest fișier conține toate cheile publice pentru care se permite autentificare (cu cheie publică) la contul utilizatorului. Adăugarea se face prin copierea conținutului cheii publice în cadrul fișierului. Sau se poate folosi utilitarul **`ssh-copy-id`** pentru a copia cheia publică a utilizatorului curent în contul utilizatorului de la distanță, ca în Listing 12.11.

```
1  student@uso:~$ ssh-copy-id malus@vmx.cs.pub.ro
2  /usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/
   student/.ssh/id_rsa.pub"
```

```
3  /usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to
   filter out any that are already installed
4  /usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
    prompted now it is to install the new keys
5  malus@vmx.cs.pub.ro's password:
6
7  Number of key(s) added: 1
8
9  Now try logging into the machine, with:   "ssh 'malus@vmx.cs.pub.ro'"
10 and check to make sure that only the key(s) you wanted were added.
```

Listing 12.11: Copierea cheii publice la distanță

În urma copierii cheii publice în contul utilizatorului de la distanță, se poate realiza autentificarea fără parolă, ca în Listing 12.12, liniile 1-4. Dacă dorim să folosim o anumită cheie privată (perechea cheii publice) pentru autentificare, transmite opțiunea `-i` (de la *identity*) comenzii **`ssh`** (sau **`ssh-copy-id`**, ca în Listing 12.12, liniile 6-21; opțiunea este urmată de calea către fișierul ce conține cheia privată.

```
1  student@uso:~$ ssh malus@vmx.cs.pub.ro
2  [...]
3  malus@vmx:~$ logout
4  Connection to vmx.cs.pub.ro closed.
5
6  student@uso:~$ ssh -i ~/.ssh/new_id malus@vmx.cs.pub.ro
7  malus@vmx.cs.pub.ro's password:
8
9  student@uso:~$ ssh-copy-id -i ~/.ssh/new_id.pub malus@vmx.cs.pub.ro
10 /usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/
   student/.ssh/new_id.pub"
11 /usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to
   filter out any that are already installed
12 /usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
    prompted now it is to install the new keys
13
14 Number of key(s) added: 1
15
16 Now try logging into the machine, with:   "ssh 'malus@vmx.cs.pub.ro'"
17 and check to make sure that only the key(s) you wanted were added.
18
19 student@uso:~$ ssh -i ~/.ssh/new_id malus@vmx.cs.pub.ro
20 [...]
21 malus@vmx:~$
```

Listing 12.12: Autentificarea SSH cu cheie publică

12.4.4.3 SSH în Windows

Deși specific lumii Linux / Unix, protocolul SSH este folosit și în lumea Windows. În general, folosim SSH în Windows în aplicații client, pentru a ne conecta la sisteme Linux / Unix la distanță. Totuși, se poate instala un server SSH și în Windows, de exemplu Bitvise SSH Server[1].

[1] https://www.bitvise.com/ssh-server

Cele mai întâlnite aplicații de tip client SSH pe Window sunt PuTTY[1], WinSCP[2] și Bitvise SSH Client[3].

PuTTY (în Figura 12.17) este folosit pentru realizarea unei conexiuni SSH la distanță. Conține și utilitare asociate pentru generarea de perechi cheie privată / cheie publică (PuTTYgen) și pentru transferul de informații la distanță (PSCP).

WinSCP (în Figura 12.18) oferă o interfață de tip *commander* pentru transferul de fișiere prin SSH.

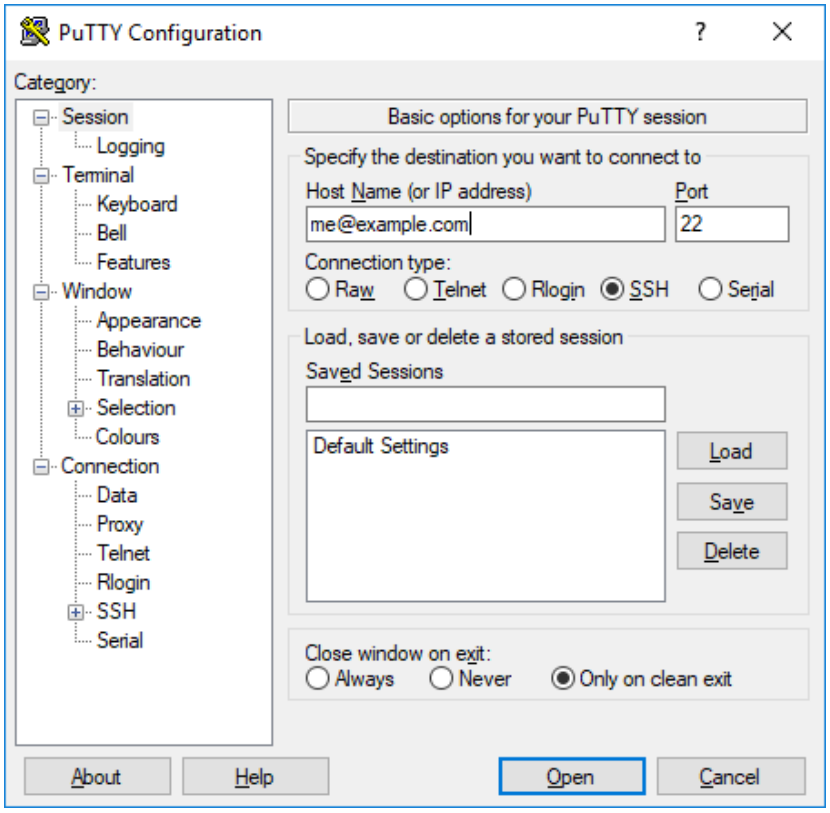


Figura 12.17: PuTTY

12.5 Securitatea proceselor

Un utilizator folosește resursele unui sistem (hardware, fișiere, servicii) cu ajutorul proceselor. La fel, un atacator care dorește să abuzeze un sistem (să-l controleze, să fure informații sau să-i afecteze funcționarea) va dori capturarea unor procese. Un mod direct pentru capturarea unui proces este ca atacatorul să obțină credențialele de acces și să impersoneze un utilizator valid în sistem, obținând permisiunile acestuia, așa cum am descris în Secțiunea 12.3. Un alt mod este exploatând vulnerabilități ale procesului și capturând astfel procesul. Alt mod este păcălind utilizatorul să instaleze aplicații create de atacator care să pornească procese care să abuzeze sistemele. Aceste aplicații malițioase sunt numite **malware**: virușii, troienii, viermii (*worms*), spyware sunt exemple de malware.

[1] https://www.putty.org/
[2] https://winscp.net/eng/download.php
[3] https://www.bitvise.com/ssh-client

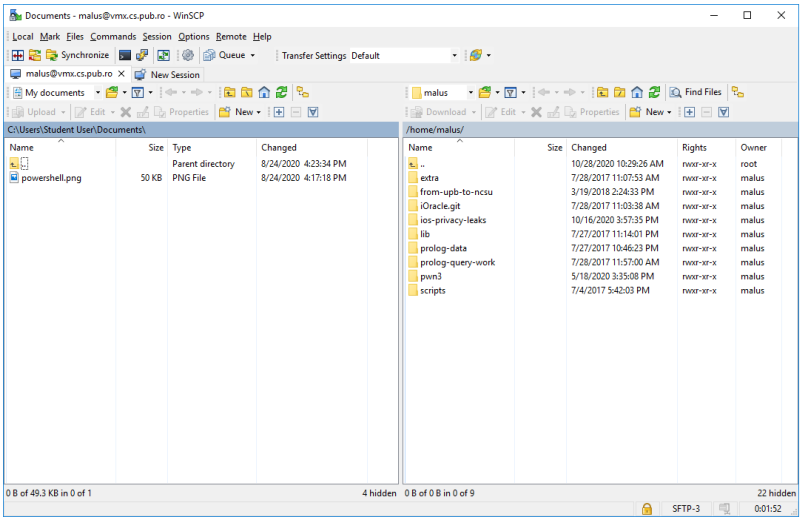


Figura 12.18: WinSCP

În oricare dintre variantele de mai sus, sistemul de operare trebuie să aibă în vedere măsuri preventive și măsuri reactive pentru protejarea proceselor.

Măsurile preventive vizează împiedicarea unui atacator să poată ajunge la procese din sistem. Aceste măsuri pot fi:

- securitatea accesului la sistem (descrisă în Secțiunea 12.3)
- tehnici specializate (precum protejarea memoriei) care să împiedice exploatarea vulnerabilităților
- verificarea aplicațiilor instalate (folosind rezumate / sume de control) și verificarea sursei de proveniență. Aceasta este foarte importantă mai ales în cazul dispozitivelor mobile unde magazinele online de aplicații (*online application stores*) precum Apple AppStore sau Google Play stochează foarte multe aplicații

Măsurile reactive presupun că atacul s-a produs sau se va produce și vizează limitarea daunelor produse de un atac. Astfel de măsuri sunt: plasarea procesului contaminat în carantină, investigația atacului și depistarea problemei, actualizarea aplicației vulnerabile (*update*, *patching*) sau înlăturarea aplicației vulnerabile.

Măsurile reactive trebuie să fie luate cât mai repede după producerea unui atac pentru a limita daunele. De aceea, este importantă **monitorizarea proceselor** și resurselor unui sistem. Un utilizator tehnic sau un administrator de sistem sau de rețea va folosi suite de aplicații specifice pentru monitorizare și va depista comportamente anormale care pot fi efectele unui atac și va reacționa rapid. În absența monitorizării, un atacator va avea mai mult timp pentru a obține beneficii în urma atacului sau va putea extinde atacul la alte sisteme sau resurse. Monitorizarea are rolul de a depista atât atacurile, cât și abuzul de resurse sau comportamente neobișnuite datorate unei utilizări necorespunzătoare (dar neintenționat abuzive) a sistemului. De exemplu, sistemele de tip antivirus monitorizează periodic sistemul de fișiere pentru a detecta prezența fișierelor infectate. Utilitare precum Fail2ban[1] monitorizează fișiere de tip jurnal pentru a detecta și a bloca încercări de acces nevalid în sistem. Sisteme complexe precum Nagios[2] monitorizează infrastructura IT a unei organizații: rețea, aplicații, resurse hardware.

[1] https://www.fail2ban.org/wiki/index.php/Main_Page
[2] https://www.nagios.org/

Pentru prevenirea daunelor produse de un proces compromis, este esențială respectarea principiului celui mai mic privilegiu, descris în Secțiunea 12.1.3. Acesta presupune ca un proces să aibă acces doar la resursele de care are nevoie: fișiere, interacțiune cu alte procese, resurse hardware.

Un prim mod de implementarea a principiului celui mai mic privilegiu îl reprezintă permisiunile de acces la sistemul de fișiere. Un proces va putea accesa doar acele fișiere la care are acces utilizatorul de care aparține procesul. Astfel, un proces al unui utilizator neprivilegiat va putea scrie doar în directorul home propriu, va putea doar citi fișierul `/etc/passwd` și nu va avea nici o formă de acces la fișierul `/etc/shadow`.

O măsură suplimentară este folosirea de mecanisme de jailing de tipul chroot, așa cum am descris în Secțiunea 12.2.1. Cu un astfel de mecanism, un proces va putea accesa doar fișierele dintr-o parte a ierarhiei sistemului de fișiere.

Mecanismele de tipul chroot sunt mecanisme de **izolare a procesului**. Cu ajutorul chroot izolăm accesul procesului la sistemul de fișiere. Pentru a extinde izolarea și în alte zone (comunicarea cu alte procese, accesul la rețea, accesul la hardware), putem folosi **sandboxing**. Sandboxing presupune crearea unor reguli de acces la resurse și atașarea acelor reguli la un proces; procesul va putea accesa doar resursele permise în acele reguli. Sandboxingul este implementat în sistemele de operare de pe dispozitivele mobile (Android, iOS) pentru a limita potențialele daune create de aplicații instalate de un utilizator pe dispozitivul mobil.

O formă extinsă de izolare este **folosirea containerelor sau mașinilor virtuale**, descrise în Capitolul 14. Containerele oferă reguli complexe de izolare a unei aplicații sau a unui set de aplicații, extinzând astfel mecanismul de sandboxing. Mașinile virtuale adaugă, față de containere, partiționarea resurselor hardware și izolarea inclusiv a sistemului de operare al mașinii virtuale; în cazul unui sistem de operare compromis, doar mașina virtuală respectivă va fi afectată.

Prezența metodelor de izolare duce la diminuarea daunelor produse de un potențial atac. Dar izolarea unui proces limitează plaja de acțiuni a acestuia. Este posibil ca un proces să aibă nevoie, la un moment dat, de o acțiune privilegiată care nu poate fi realizată conform regulilor de izolare. Pentru această acțiuni este nevoie de escaladarea nivelului de privilegiu (*privilege escalation*), adică obținerea (temporară) de privilegii care să permită acțiunea.

În Linux, forma clasică de escaladare a privilegiilor este marcarea unui executabil cu bitul *set-user-ID-on-execution* (numit și *setuid*). O formă configurabilă de escaladare de privilegii este cu ajutorul utilitarului **`sudo`**. Am detaliat bitul setuid și utilitarul **`sudo`** în Secțiunea 5.2.3.

Întrucât procesele sunt modul de utilizare a sistemului și accesarea a resurselor acestuia, trebuie să avem în vedere securitatea acestora prin măsuri preventive și măsuri reactive. Este important sa avem grijă ce aplicații instalăm pe sistemele și dispozitivele noastre și să folosim aplicații care monitorizează funcționarea sistemului și folosirea resurselor acestuia.

12.6 Sumar

Un sistem este sigur dacă este folosit așa cum a fost proiectat. Întrucât este cvasi-imposibil să acoperim toate cazurile în care un sistem poate fi folosit, nu putem spune că un sistem este perfect sigur. Spunem că securitatea este un proces, nu o finalitate.

Un atacator poate urmări să controleze un sistem, să fure informații sau să abuzeze resurse. Un apărător urmărește să prevină existența atacurilor sau reacționeze cât mai rapid pentru a limita daunele unui atac. Un apărător va urmări să asigure confidențialitatea datelor, integritate datelor, disponibilitatea serviciilor și protejarea vieții private (*privacy*).

Pentru protejarea unui sistem, există principii de securitate pe care utilizatorii și organizațiile să le urmeze: cel mai mic privilegiu, securitate în adâncime, separarea mecanismului de politică, proiectare modulară.

Un utilizator sau administrator se va preocupa de securitatea accesului (cine are acces la sistem și la ce resurse), securitatea transferului (cum se transmit informațiile în Internet) si securitatea aplicațiilor (o aplicație să fie protejată de atacuri sau, dacă este atacată, să nu afecteze alte aplicații).

Există protocoale, algoritmi, tehnologii și aplicații pentru creșterea securității unui sistem. Un utilizator / administrator va fi în permanență informat, va aloca resurse și va depune efort constant pentru a garanta un nivel optim de securitate.

Capitolul 13

Automatizarea sarcinilor

În utilizarea și administrarea sistemului de operare sau a unei rețele sau infrastructuri informatice mai ample, avem adesea acțiuni pe care le repetăm. Fie că avem de compilat o aplicație, de pornit o mașină virtuală, de prelucrat datele dintr-o bază de date, de testat o aplicație desktop, mobilă sau web, de generat un raport de folosire a rețelei, este util să avem moduri prin care să automatizăm aceste aceste acțiuni.

> Automatizarea unei acțiuni înseamnă executarea acesteia fără intermedierea unei persoane, fără ca aceasta să fie interactivă. O acțiune interactivă solicită intervenția utilizatorului în vreme ce o acțiune neinteractivă nu are nevoie de intervenția utilizatorului. Automatizarea înseamnă eliminarea sau înlocuirea părților interactive dintr-o acțiune sau un set de acțiuni.

Un exemplu de automatizare este asigurat de utilitarul Make, prezentat în Secțiunea 6.6.7. Fără utilitarul Make, un dezvoltator ar trebui să compileze toate fișierele obiect, apoi să le link-editeze împreună cu biblioteci și, eventual, să le compună cu alte fișiere într-un pachet software. Folosind Make și fișiere Makefile, dezvoltatorul automatizează procesul și permite ca toți pașii să fie executați la rularea comenzii **`make`**. Mai mult, comanda **`make`** poate fi declanșată la un moment de timp sau la un alt eveniment; este cazul proceselor de tip *nightly build* despre care vom discuta mai târziu.

Un alt exemplu de automatizare, de această dată la nivelul unui administrator de datacenter, este crearea de mașini virtuale. Dacă un administrator are nevoie să creeze sute de mașini virtuale, nu le va crea pe rând, interactiv. Ar dura prea mult și ar fi repetitiv și plictisitor. Administratorul va automatiza procesul creării unei mașini virtuale și apoi va generaliza comanda pentru mai multe instanțe. În final, prin rularea unei singure comenzi vor fi create toate mașinile virtuale.

Un utilizator tehnic, în special unul care are rol administrativ într-o organizație, are nevoie de cunoștințe și abilități de automatizare pentru a își eficientiza timpul de lucru. Un utilizator care face mereu aceleași acțiuni în mod repetat va consuma timp care ar putea fi investit în alte acțiuni. În momentul când automatizează acțiuni, acel timp consumat de repetiții va putea fi folosit în scopuri creative. Activitățile creative nu pot fi automatizate, însă cele repetitive da, și un utilizator înțelept le automatizează pe cele din urmă ca să aibă cât mai multă energie și timp pentru celelalte. Într-o notă mai amuzantă, după spusele unui prieten, un administrator de infrastructură informatică ar trebui plătit cu

atât mai bine cu cât stă mai mult timp degeaba: înseamnă că acțiunile uzuale sunt automatizate iar cele critice apar rar și sunt bine tratate.

Termenul de automatizare este folosit în sens mai larg și la domenii diferite de domeniul IT: irigații automate, folosirea de roboți pentru automatizarea creării de componente, mașini autonome. Oriunde este nenecesară intervenția unei persoane în execuția unei acțiuni, spunem că avem automatizare. Automatizarea va necesita în continuare controlul din partea unei persoane, dar partea de execuție nu.

În domeniul IT, termenul de automatizare este asociat de obicei cu noțiunea de **scripting**. Acțiunea se mai numește „a scripta", întrucât acțiunile automatizate sunt formate din comenzi și instrucțiuni care se trec într-un fișier numit script. Acest script este apoi interpretat și rulat la un moment convenabil, având loc astfel automatizarea acțiunii.

Un script este, de obicei, asociat interfeței în linie de comandă; vorbim de scripturi shell, scripturi Python, scripturi Perl, scripturi Lua. Există însă și moduri de automatizare pentru interfețe grafice sau web. Există aplicații[1] care testează interfețe grafice automatizând plasarea cursorului de mouse și evenimente grafice precum click, selectare, glisare. Similară este tehnologia RPA (*Robotic Process Automation*) care permite automatizarea unei activități pe care o execută cineva folosind interfața grafică, precum emiterea unei facturi care să cuprindă anumite date, sau citirea unor date dintr-un document și completarea acestora într-un formular web. În cele ce urmează ne vom concentra pe automatizarea în interfața în linia de comandă, modul uzual de automatizare în sistemele de operare și infrastructurile IT. Este de reținut că există tehnologii și posibilități de automatizare și în interfața grafică.

Așa cum am spus mai sus, automatizarea în linia de comandă folosește un script care conține comenzi și instrucțiuni pentru executarea unei acțiuni. Un conținut similar are însă și un program și poate apărea întrebarea cu ce diferă un program de un script. Nu există o linie clară de demarcație între un script și un program. În general, un program se dorește a fi eficient în consumarea resurselor și execută o acțiune bine determinată. Pe de altă parte, un script este ceva creat și folosit rapid (*quick and dirty*) și folosește programe deja existente pentru executarea unei acțiuni combinate, care leagă mai multe componente. De exemplu, un program va fi acela care generează un fișier PDF dintr-un fișier de tip `.doc`; un script este acela care parcurge o ierarhie ce conține fișiere `.doc`, selectează fișierele care sunt mai noi de o anumită dată, le convertește în fișiere PDF și apoi creează o arhivă cu fișierele PDF pe care o stochează într-un director dat. Păstrând măsura, putem folosi interschimbabil noțiunea de script și program, preferând termenul de script pentru situațiile în care urmărim o formă de automatizare.

Într-un sistem informatic multe aplicații au interfață grafică, care nu poate fi automatizată într-un script. Pentru automatizare acțiunilor corespunzătoare, avem două opțiuni:

1. să folosim aplicații alternative, în interfață de comandă
2. să folosim forme de rulare neinteractivă a acestora

Din prima categorie, de exemplu, folosim utilitarele **`wget`** sau **`curl`** în locul unui browser web grafic precum Firefox sau Chrome. Sau folosim utilitarul **`bc`** în locul unei aplicații grafice de calculator.

[1] https://en.wikipedia.org/wiki/Graphical_user_interface_testing

Din a doua categorie fac parte aplicațiile care oferă o formă de rulare numită *headless*. Forma headless înseamnă o aplicație cu funcționalități similare dar fără interfață grafică. În general, aplicații headless sunt acele aplicații care nu au interfața grafică. De exemplu, clientul Bittorrent Transmission are varianta headless **`transmission-cli`**. Soluția de virtualizare VirtualBox are varianta headless **`VBoxManage`**, pentru controlul mașinilor virtuale. Folosind aplicații alternative sau forme headless ale unor aplicații putem automatiza acțiunile aferente acestora. Putem spune că, în privința automatizării rulării aplicațiilor, *where there is a will, there is a way*.

În cele ce urmează vom prezenta modul în care putem crea scripturi pentru automatizarea de acțiuni în linia de comandă, cu accentul pe cea mai simplă și accesibilă formă: **shell scripting**.

13.1 Perspective pentru scripting

Am precizat mai sus că forma de automatizare în linia de comandă este folosind scripturi ce conțin comenzi și instrucțiuni. Atunci când automatizăm acțiuni în linia de comandă folosim scripturi în două cazuri mari de utilizare:

1. automatizarea unor acțiuni repetitive
2. prelucrarea datelor

În prima categorie intră acțiuni precum compilarea unei suite de aplicații, crearea de mai multe mașini virtuale sau copierea unor fișiere pe o platformă la distanță pentru actualizarea conținutului. În general, aceste scripturi conțin comenzi sau programe pe care utilizatorul le-ar rula pe rând în linia de comandă. Scriptul are avantajul de a agrega acele comenzi, iar utilizatorul doar rulează scriptul.

În a doua categorie intră acțiuni precum crearea unui raport sau a unei diagrame din datele existente într-o bază de date, dintr-un fișier jurnal sau din date afișate de o aplicație. În general aceste scripturi conțin comenzi specifice pentru prelucrarea datelor (text), adică filtre de text, așa cum le-am denumit în Capitolul 7.

Putem spune că ambele cazuri automatizează acțiuni și că acele acțiuni sunt repetitive. Cea mai importantă diferență între cele două este însă tipul de comenzi și utilitare pe care scripturile le folosesc: comenzi obișnuite, dar acum folosite agregat, în cazul automatizării de acțiuni repetitive, și filtre de text în cazul prelucrării datelor. Atunci când automatizăm o acțiune, ne putem da seama de tipul de comenzi și utilitare pe care le vom folosi în funcție de cazul de utilizare al acțiunii.

Atunci când dezvoltăm un script pentru automatizare, este recomandat să urmărim idei de bună proiectare, pe care le descriem mai jos.

Există mai multe moduri de a face o acțiune. De exemplu, pentru a afișa informații despre un proces putem folosi utilitarul **`ps`** sau putem urmări intrările din `/proc`. Pentru a scrie informație într-un fișier putem folosi comanda **`echo`** și redirectare sau putem folosi utilitarul **`dd`**. Și în cazul general, și când automatizăm, **alegem soluția cea mai adecvată pentru cazul respectiv de utilizare**. De exemplu utilitarul **`sed`** face tot ce face utilitarul **`tr`** și în plus; cu toate acestea vom folosi utilitarul **`tr`** pentru cazuri de utilizare

simple. Nu există soluții universale, iar autorul scriptului va alege cele mai adecvate soluții în funcție de cazul de utilizare.

Când creăm un script folosim cât mai mult utilitare și scripturi existente. **Nu reinventăm roata**. Dacă o comandă face deja ce ne dorim, nu facem un script. Dacă o comandă cu anumiți parametri face deja ceea ce ne dorim, nu folosim două sau trei comenzi. De exemplu, dacă dorim să aflăm dimensiunea unui fișier, avem cele două variante din Listing 13.1.

```
1 student@uso:~$ ls -l vm-actions-log.txt  | tr -s ' ' | cut -d ' ' -f 5
2 4827
3 student@uso:~$ stat -c "%s" vm-actions-log.txt
4 4827
```

Listing 13.1: Afișarea informațiilor despre un fișier (ls vs stat)

Vom prefera a doua variantă care este mai simplă și mai clară și mai puțin expusă la posibile erori de formatare ale utilitarului **`ls`**. Pentru aceasta, este nevoie de o cunoaștere temeinică a comenzilor, utilitarelor și facilităților existente pentru a nu ajunge să reinventăm roata.

Scripturile sunt folosite pentru acțiuni de tipul *quick and dirty*. Scripturile nu urmăresc să fie eficiente din punct de vedere al consumului de resurse (rapide, memorie redusă) ci urmăresc să fie eficace (să ducă treaba la bun sfârșit) și să eficientizeze munca utilizatorului, lăsându-i timp pentru alte activități. De aceea nu vom ajunge ca în scripturi să facem calcule numerice complexe sau acțiuni intens computaționale. Scripturile pot automatiza apelarea de programe scrise special pentru acțiuni intens computaționale, dar nu vor ajunge ele să facă astfel de acțiuni.

În lumea Unix, **modul de transfer de informație este textul**, conținut ușor de citit de om. Acest lucru permite stocarea informației în fișiere text și, la nevoie, editarea lor de către utilizator. De aceea, scripturile, mai ales cele de prelucrare de date, este recomandat să folosească informații text: ușor de citit de utilizator, ușor de modificat. Cele mai multe utilitare shell lucrează cu informații text, de exemplu filtrele de text.

În linia de comandă avem utilitare care raportează informații de la nivelul sistemului. De exemplu, utilitarul **`ps`** raportează informații despre procese, iar utilitarul **`ls`** afișează informații despre fișiere. Un rol oarecum similar îl au, respectiv, utilitarele **`pgrep`** și **`stat`**. Ne punem problema când folosim pe fiecare. Pentru aceasta clasificăm utilitarele în două:

1. utilitare pentru afișarea de informații, pe ecran, pentru a fi accesibile utilizatorului
2. utilitare care afișează informația în format prelucrabil, util pentru legarea de alte comenzi, adică pentru automatizare

Utilitarele **`ps`** și **`ls`** fac parte din prima categorie. La va folosi utilizatorul atunci dorește să afișeze informații despre fișiere și procese. Dacă însă utilizatorul va avea nevoie de PID-ul anumitor procese sau dimensiunea anumitor fișiere, va fi complicată folosirea utilitarelor **`ps`** și **`ls`**; atunci se recomandă folosirea utilitarelor **`pgrep`** și **`stat`**, ca în Listing 13.2. Observăm că folosirea **`pgrep`** este mai curată, extrăgând exact informația dorită: PID-urile proceselor cu numele `VBoxClient`. Folosirea **`ps`** este mai anevoioasă, necesitând operații suplimentare de prelucrare a rezultatului.

```
1 student@uso:~$ ps -ef | grep VBoxClient | tr -s ' ' | cut -d ' ' -f 2
```

```
1300
1301
1310
1311
1321
1322
1328
1329
10819
student@uso:~$ pgrep VBoxClient
1300
1301
1310
1311
1321
1322
1328
1329
```

Listing 13.2: Utiltare pentru afișare și pentru prelucrare

În secțiunile ce urmează vom prezenta informații despre crearea de scripturi shell și vom furniza exemple în care vom ține cont de principiile enunțate mai sus.

13.2 Dezvoltarea scripturilor

Un script este un fișier care conține comenzi, instrucțiuni și construcții care execută o anumită acțiune. Din punctul de vedere al conținutului, un script este similar unui program. Din punctul de vedere al execuției, un script este interpretat. Așa cum multe limbaje de programare sunt compilate și executate, un script este interpretat de un interpretor. În cazul unui script shell, interpretorul este chiar shellul. Alte interpretoare pot fi utilitarul **`sed`** sau **`awk`** sau un interpretor de limbaj Python, Perl, Ruby, Lua etc.

13.2.1 „Hello, World!” în scripting și shebang

Cel mai cunoscut exemplu în orice limbaj este un program care afișează mesajul „Hello, World!”. Vom exemplifica cele mai simple scripturi cu același rezultat: afișarea mesajul „Hello, World!”.

Dacă am creat fișierul `hello.sh` având conținutul din Listing 13.3 (liniile 1-4), atunci vom putea apela shellul Bash pentru interpretarea sa (liniile 5-6). În mod similar, dacă vom crea fișierul `hello.py` având conținutul din Listing 13.3 (liniile 7-10), atunci vom putea apela Python pentru interpretarea sa (liniile 11-12). La fel ar fi cazul și cu scripturi Perl sau Ruby sau alte limbaje de scripting.

```
student@uso:~/.../code/13-auto/hello$ cat hello.sh
#!/bin/bash

echo "Hello, World!"
student@uso:~/.../code/13-auto/hello$ bash hello.sh
Hello, World!
student@uso:~/.../code/13-auto/hello$ cat hello.py
```

```
8  #!/usr/bin/env python3
9
10 print("Hello, World!")
11 student@uso:~/.../code/13-auto/hello$ python hello.py
12 Hello, World!
```

Listing 13.3: Hello, World! în script shell

Pentru a simplifica rularea unui script putem să oferim fișierului permisiuni de execuție și să îl rulăm ca pe un program. Pentru ca acest lucru să funcționeze, shellul va trebui să știe ce interpretor să folosească. Pentru a stabili interpretorul, folosim o construcție numită shebang (`#!`). Această construcție este folosită la începutul scriptului pentru a indica interpretorul folosit. Este folosită la începutul scripturilor `hello.sh` și `hello.py`, așa cum se vede în Listing 13.3, liniile 2, respectiv 8.

Dacă este prezentă construcția shebang (`#!`) pentru a indica interpretorul folosit (`/bin/bash`, respectiv `/usr/bin/env python3`), atunci scripturile pot fi executate ca un program obișnuit, după ce le vom acorda permisiuni de execuție, ca în Listing 13.4.

```
1 student@uso:~/.../code/13-auto/hello$ chmod a+x hello.sh
2 student@uso:~/.../code/13-auto/hello$ chmod a+x hello.py
3 student@uso:~/.../code/13-auto/hello$ ./hello.sh
4 Hello, World!
5 student@uso:~/.../code/13-auto/hello$ ./hello.py
6 Hello, World!
```

Listing 13.4: Rularea directă a scripturilor

În scripturile pe care le vom construi în continuare vom folosi mereu shebang.

13.2.2 Conținutul unui script shell

În acest capitol ne vom concentra pe automatizare în linia de comandă, adică pe scripturi, și, dintre acestea, pe cele mai simple și rapide: scripturile shell.

Scripturile shell sunt scripturi interpretate direct de shell. În mod uzual, acestea vor începe cu simbolul shebang și apoi calea către executabilul aferent shellului. Pentru Bash, shellul predominant pe Linux, calea este `/bin/bash`, așa cum am văzut în exemplul din Listing 13.3. Astfel că prima linie într-un script shell / Bash va fi `#!/bin/bash`.

Informațiile pe care le vom furniza și exemplele de scripturi vor fi specifice shellului Bash. Cu toate acestea, multe dintre funcționalități sunt prezente și în alte shelluri. În general, cu modificări minore un script Bash poate ajunge să ruleze și pe un shell Zsh sau Tcsh. Nu vom intra în aceste detalii în această carte, ne vom concentra pe shellul Bash; lăsăm utilizatorului interesat să folosească resursele puse la dispoziție pe Internet legate de scripting pentru shellul Zsh sau Tcsh. În continuare, când folosim shell, vom înțelege implicit Bash, iar când vom folosi script shell, vom înțelege script Bash.

Un script shell conține în cea mai mare parte comenzi pe care utilizatorul le folosește în linia de comandă, agregate pentru o nouă funcționalitate. Pe lângă aceste comenzi, un script conține construcții specifice shellului așa cum au fost descrise în Secțiunea 7.2:

operatori, expandare, escaping. Cu ajutorul acestora, un script combină comenzi pentru a obține noi funcționalități.

Pe lângă comenzile din linia de comandă, un script shell conține și comenzi interne (despre care am discutat în Secțiunea 7.1.4) folosite pentru controlul fluxului, funcții și parametri în linia de comandă.

13.2.3 Depanarea unui script shell

Întrucât este conceput să fie *quick and dirty*, un script shell este de obicei mic, iar depanarea se face, în general, folosind *print debugging*: rularea sa de mai multe ori și afișarea de mesaje folosind comanda **`echo`**. Afișăm fie locuri în care a ajuns scriptul în cadrul execuției comenzilor, fie valorile unor variabile.

La nevoie, dacă sunt foarte multe informații afișate pe ecran, scriem informațiile într-un fișier pentru a le consulta ulterior.

Întrucât un script este format din comenzi de utilitare deja existente, putem rula pas cu pas fiecare comandă și apoi să vedem efectul ei. În fapt, este recomandată rularea comenzilor dorite înainte de adăugarea lor într-un script pentru a ne asigura de funcționarea corectă.

Pentru depanare mai avansată, putem folosi comanda **`set -x`** așa cum vom descrise în Secțiunea 13.4.5.

13.2.4 Exemple de scripturi shell

În această secțiune prezentăm câteva exemple practice de scripturi shell și ce funcționalități sunt folosite în fiecare.

Listing 13.5 prezintă un script simplu folosit pentru actualizarea sistemului. Scriptul este folosit pentru a rula o singură comandă pentru a actualiza pachetele din sistem. Scriptul folosește:

- `#!` (*shebang*) pentru precizarea interpretorului (`/bin/bash`)
- utilitarul **`apt`** pentru operare cu pachete
- înlănțuirea comenzilor (operatorul `&&`); am discutat în Secțiunea 7.2.1

```
1 student@uso:~/uso-book/code/13-auto$ cat system-update
2 #!/bin/bash
3
4 sudo apt update && sudo apt -y upgrade && sudo apt -y dist-upgrade
```

Listing 13.5: Script: Actualizarea systemului

Listing 13.6 prezintă folosirea scriptului `wiki-cat`. Scriptul este folosit în laboratoarele didactice pentru a agrega secțiuni de tip wiki din fișiere diferite într-un singur fișier. Există directoare pentru fiecare laborator (`01-fs`, `02-process`, `03-user`, `04-appdev`) care conțin secțiuni de wiki în fișiere diferite. Scriptul realizează fișiere agregate pentru fiecare laborator: `01-fs.wiki`, `02-process.wiki`, `03-user.wiki`, `04-appdev.wiki`. Scriptul folosește:

- `#!` (*shebang*) pentru precizarea interpretorului (`/bin/bash`)
- `for` pentru parcurgerea elementelor din directorul curent; variabila `i` reține fiecare fișier sau director; detalii în Secțiunea 13.3.2.1
- `*` (*star*), metacaracter care expandează la toate intrările din directorul curent; am discutat în Secțiunea 7.2.4.1
- `if` și `test` pentru a verifica elementele de tipul director (`-d "$i"`); detalii în Secțiunea 13.3.2.1
- expandarea opțiunilor (cu acolade, construcția `{opt1,opt2,opt3}`); am discutat în Secțiunea 7.2.4
- redirectarea ieșirii (operatorul `>`); am discutat în Secțiunea 7.2.1
- expandarea comenzii `basename` (construcția `$(...)`); am discutat în Secțiunea 7.2.4.2

```
1  student@uso:~/.../code/13-auto/wiki-cat$ ls
2  01-fs  02-process  03-user  04-appdev  wiki-cat
3
4  student@uso:~/.../code/13-auto/wiki-cat$ cat wiki-cat
5  #!/bin/bash
6
7  for i in *; do
8      if test -d "$i"; then
9          cat "$i"/{concepts,demo,recap,basics,need-to-know,nice-to-know,
   get-a-life,summary}.wiki > $(basename "$i").wiki
10     fi
11 done
12
13 student@uso:~/.../code/13-auto/wiki-cat$ ls 01-fs 02-process 03-user 04-
   appdev
14 01-fs:
15 basics.wiki  concepts.wiki  demo.wiki  get-a-life.wiki  need-to-know.wiki
     nice-to-know.wiki  recap.wiki  setup.wiki  summary.wiki
16
17 02-process:
18 basics.wiki  concepts.wiki  demo.wiki  get-a-life.wiki  need-to-know.wiki
     nice-to-know.wiki  recap.wiki  setup.wiki  summary.wiki
19
20 03-user:
21 basics.wiki  concepts.wiki  demo.wiki  get-a-life.wiki  need-to-know.wiki
     nice-to-know.wiki  recap.wiki
22
23 04-appdev:
24 basics.wiki  concepts.wiki  demo.wiki  get-a-life.wiki  need-to-know.wiki
     nice-to-know.wiki  recap.wiki
25
26 student@uso:~/.../code/13-auto/wiki-cat$ ./wiki-cat
27 cat: 03-user/summary.wiki: No such file or directory
28 cat: 04-appdev/summary.wiki: No such file or directory
29
30 student@uso:~/.../code/13-auto/wiki-cat$ ls
31 01-fs  01-fs.wiki  02-process  02-process.wiki  03-user  03-user.wiki
   04-appdev  04-appdev.wiki  wiki-cat
```

Listing 13.6: Script: Agregarea secțiunilor de tip wiki

Listing 13.7 prezintă folosirea scriptului `gen-passwd`. Scriptul generează o parolă și afișează parola și formatul ei hash MD5. Parolele sunt reținute în format hash din rațiuni de securitate, așa cum am precizat în Secțiunea 12.3.1 și în Secțiunea 5.4.1. Scriptul folosește:

- `#!` (*shebang*) pentru precizarea interpretorului (`/bin/bash`)
- expandarea comenzilor `pwgen` și `openssl` pentru a reține rezultatul lor (construcția `$(...)`); am discutat în Secțiunea 7.2.4.2
- variabilele `salt`, `password` și `crypt` pentru a reține informații generate din expandarea comenzilor

```
1  student@uso:~/uso-book/code/13-auto$ cat gen-passwd
2  #!/bin/bash
3
4  salt=$(pwgen -N 1 -c 8)
5  password=$(pwgen -N 1 -c 10)
6  crypt=$(openssl passwd -1 -salt "$salt" "$password")
7
8  echo "password: $password"
9  echo "crypt: $crypt"
10
11 student@uso:~/uso-book/code/13-auto$ ./gen-passwd
12 password: fie3kac6Ta
13 crypt: $1$Boongoh8$RPo3/ckltzviC4TXgxcAJ0
14
15 student@uso:~/uso-book/code/13-auto$ ./gen-passwd
16 password: Gei8ieno2i
17 crypt: $1$Pei4Beak$zHBDNjWMj3ozZW/cV.rKj/
```

Listing 13.7: Script: Generator de parole și hash

13.3 Funcționalități pentru scripturi shell

Scripturile shell sunt secvențe de comenzi existente combinate pentru o funcționalitate. Shellul oferă și funcționalități similare unui limbaj de programare precum variabile, comenzi de control al fluxului, funcții. Aceste funcționalități ajută la crearea de scripturi mai complexe și care să țină cont de diferitele condiții în care se rulează un script.

13.3.1 Variabile

Variabilele au un rol similar altor limbaje de programare. Au un nume și o valoare. În shell (și în scripturi shell) variabilele nu au tip și interpretarea lor depinde de modul de utilizare. O variabilă cu valoarea `12` poate fi număr sau doar un simplu string depinzând de cazul de folosire. Exemple de inițializare și folosire a unei variabile sunt prezentate în Listing 13.8. Variabila `f` reține numele fișierului `/etc/passwd`. Pentru a folosi valoarea variabilei `f`, prefixăm cu simbolul `$` (*dollar*). Este recomandat să folosim valorile variabilelor între ghilimele, pentru escaping, așa cum am prezentat în Secțiunea 7.2.5. La fel, variabila `num` reține numărul de linii din fișierul reținut de variabila `f`, folosind expandarea comenzii **`wc`**.

```
student@uso:~/uso-book/code/13-auto$ f=/etc/passwd
student@uso:~/uso-book/code/13-auto$ wc -l < "$f"
46
student@uso:~/uso-book/code/13-auto$ num=$(wc -l < "$f")
student@uso:~/uso-book/code/13-auto$ echo "$num"
46
```

Listing 13.8: Folosirea variabilelor în shell

Detalii despre variabile și folosirea lor în shell am prezentat în Secțiunea 7.2.3.

Variabilele sunt utile în scripturi shell pentru a reține o valoare. Acea valoare ar putea fi folosită și direct, dar e mai lizibil și mai ușor de înțeles scriptul când folosim un nume de variabilă. Este cazul scriptului `gen-passwd` prezentat în Listing 13.7. Altfel, variabilele sunt folosite implicit în comenzi / construcții shell, precum **`for`**, așa cum am folosit variabila `i` în scriptul `wiki-cat` din Listing 13.6. Adesea, vom folosi variabile pentru menține o informație comună mai multor comenzi, similar unui limbaj de programare. De exemplu, variabila `i` din scriptul `wiki-cat` din Listing 13.6 este folosită împreună cu **`for`**, cu **`if`** / **`test`**, cu **`cat`** și cu **`basename`**.

Valoarea unei variabile poate fi inițializată și folosită prin operația simplă de atribuire, respectiv prin prefixarea ei cu simbolul `$` (*dolar*). Pe lângă acestea, inițializarea și folosirea unei variabile poate fi realizată cu diferitele forme de expandare (aritmetică, a comenzilor, parametrică, cu opțiuni) pe care le-am prezentat în Secțiunea 7.2.4 (cu exemple de folosire în Listing 7.18).

Un tip special de variabile sunt parametrii unui script shell. Un shell poate primi unul sau mai mulți parametri care pot afecta funcționarea shellului. În Listing 13.9 avem o actualizare a scriptului `wiki-cat` din Listing 13.6. Scriptul `wiki-cat-param` primește un parametru: numele directorului de laborator ale cărui secțiuni wiki să fie unificate. Dacă scriptul este rulat fără argumente, se afișează un mesaj de utilizare corectă. La o rulare corectă, după transmiterea numelui unui director de laborator ca parametru (`01-fs/`), scriptul generează fișierul agregat corespunzător (`01-fs.wiki`). Scriptul `wiki-cat-param` folosește construcțiile scriptului `wiki-cat` și parametri de script:

- `$#` (împreună cu **`if`**) pentru numărul de parametri ai scriptului
- `$0` pentru afișarea numelui scriptului atunci când nu se transmit numărul corect de parametri
- `$1` pentru primul parametru al scriptului: director al cărui conținut va fi agregat

În script am fi putut să nu folosim variabila `lab` ci construcția `$1` (primul parametru al scriptului). Am preferat să definim o nouă variabilă pentru lizibilitate, fiind mai clar ce înseamnă construcția `$lab` în loc de construcția `$1`.

```
student@uso:~/.../code/13-auto/wiki-cat$ ls
01-fs  02-process  03-user  04-appdev  wiki-cat  wiki-cat-param

student@uso:~/.../code/13-auto/wiki-cat$ cat wiki-cat-param
#!/bin/bash

if test $# -ne 1; then
    echo "Usage: $0 <lab-folder>" 1>&2
    exit 1
```

```
10 fi
11
12 lab="$1"
13 if test -d "$lab"; then
14     cat "$lab"/{concepts,demo,recap,basics,need-to-know,nice-to-know,get-
   a-life,summary}.wiki > $(basename "$lab").wiki
15 fi
16
17 student@uso:~/.../code/13-auto/wiki-cat$ ./wiki-cat-param
18 Usage: ./wiki-cat-param <lab-folder>
19
20 student@uso:~/.../code/13-auto/wiki-cat$ ./wiki-cat-param 01-fs/
21
22 student@uso:~/.../code/13-auto/wiki-cat$ ls
23 01-fs  01-fs.wiki  02-process  03-user  04-appdev  wiki-cat  wiki-cat-
   param
```

Listing 13.9: Script: Agregarea secțiunilor de tip wiki (cu parametru)

Construcțiile aferente parametrilor unui script shell sunt prezentate în Tabelul 13.1. Construcțiile `$*` și `$@` au, în aparență același efect. Diferență constă în escaparea celor două construcții prin ghilimele. Astfel, construcția `$*` se va expanda la o singură valoare care conține spații între parametrii scriptului, în vreme ce construcția `$@` se va expanda la o listă de valori escapate fiecare prin ghilimele.

Tabelul 13.1: Construcții pentru parametrii unui script shell

Nume construcție	Rol / Efect
`$#`	numărul de parametri ai scriptului (echivalentul `argc` în C)
`$0`	numele scriptului (echivalentul `argv[0]` în C)
`$1`, `$2`, ...	primul, al doilea, ... parametru
`$*`	toți parametrii scriptului
`$@`	toți parametrii scriptului

În lucrul cu parametrii, shellul oferă comanda **`shift`**. Comanda **`shift`** este folosită pentru a deplasa parametri o poziție la stânga. După o folosire a comenzii **`shift`**, construcția `$1` va avea acum valoarea celui de-al doilea parametru, `$2` valoarea celui de-al treilea parametru ș.a.m.d.

13.3.2 Controlul fluxului

Așa cum am văzut și în exemplele de mai sus, shellul oferă construcții de controlul fluxului: comenzi pentru decizii condiționale (**`if`**) și comenzi de ciclare (**`for`**).

De avut în vedere că pot exista scripturi care nu folosesc aceste comenzi; un script nu este un program în sine ci o înlănțuire de funcționalități existente care pot să nu necesite comenzi de control al fluxului, cum este cazul scripturilor `system-update` (din Listing 13.5) sau `gen-passwd` (din Listing 13.7).

13.3.2.1 if

Comanda **if** este folosită pentru controlul fluxului. Comanda primește ca argument o altă comandă și interpretează codul de ieșire (*exit code*) al acesteia, în funcție de care ia o decizie.

Sintaxa comenzii **if** este indicată în Listing 13.10. Secțiunile care cuprind `elif` (liniile `6-9`) și `else` (liniile `10-13`) pot lipsi. Uzual, `then` apare pe aceeași linie cu **if**, separată cu operatorul de secvențiere (`;`), la modul `if command_to_check1; then`, așa cum apare și în Listing 13.6 și Listing 13.9. Comenzile `command_to_check1` și `command_to_check2` vor fi executate și va fi verificat codul de ieșire. Dacă acest cod de ieșire înseamnă rulare cu succes, atunci se execută `command_to_execute1`, `command_to_execute2` și așa mai departe.

```
1  if command_to_check1
2  then
3      command_to_execute1
4      command_to_execute2
5      ...
6  elif command_to_check2
7      command_to_execute3
8      command_to_execute4
9      ...
10 else
11     command_to_execute5
12     command_to_execute6
13     ...
14 fi
```

Listing 13.10: Sintaxa comenzii if

Cel mai adesea, comanda primită de comanda **if**, comandă al cărei cod de ieșire va fi verificat, este comanda **test**, ca în Listing 13.6 și Listing 13.9. Comanda **test** permite verificări de șiruri de caractere, numere, fișiere și întoarce succes / insucces dacă verificarea e adevărată sau nu. De exemplu, în Listing 13.9, în linia 7 comanda **test** face o verificare a numărului de parametri ai scriptului, iar în linia 13 comanda **test** verifică dacă valoarea variabilei `lab` este un director.

Un cod de ieșire egal cu `0` înseamnă succes. Un cod de ieșire diferit de `0` înseamnă insucces. Codul de ieșire al unei comenzi este reținut de construcția `$?`, așa cum am precizat în Secțiunea 7.2.3. Astfel că cele trei construcții din Listing 13.11 sunt echivalente.

```
1  grep "ana" /etc/passwd > /dev/null && echo "found"
2
3  if grep "ana" /etc/passwd > /dev/null; then
4      echo "found"
5  fi
6
7  grep "ana" /etc/passwd > /dev/null
8  if test $? -eq 0; then
9      echo "found"
10 fi
```

Listing 13.11: Verificarea codului de ieșire al unei comenzi

Folosim **if** în locul operatorilor de secvențiere condiționată (&&, ||) atunci când executăm mai multe comenzi sau pentru lizibilitate. Folosim **if** cu **test** și verificarea codului de ieșire $? din rațiuni de lizibilitate; dar e discutabil și probabil chestiune de gust care dintre variantele (liniile 3–5, respectiv 7–10) este preferabilă.

Comanda **test** primește ca argument o specificare de condiție care poate avea diferite forme. Condiția poate fi comparație între numere, între șiruri de caractere, verificarea tipului unui fișier și altele. Tabelul 13.2 conține o selecție a condițiilor folosite de comanda **test**. O listă completă găsiți în pagina de manual a comenzii **test**.

Tabelul 13.2: Condiții folosite de comanda test

Exemplu condiție	Rol / Efect
test -f /etc/passwd	verifică dacă /etc/passwd este fișier
test -d "$d"	verifică dacă valoarea variabilei d este director
test -x my_script	verifică dacă my_script are permisiuni de execuție
test "$#" -gt 2	verifică dacă numărul de parametri ai scriptului este mai mare (strict) decât 2
test "$v" -eq 10	verifică dacă valoarea variabilei v este egală cu 10
test "$v" -eq 10	verifică dacă valoarea variabilei v este egală cu 10
test "$1" = "/etc/passwd"	verifică dacă primul parametru al scriptului are ca valoare șirul /etc/passwd
test -z "$store"	verifică dacă variabila store nu conține nici o informație (este un șir vid)
test ! -z "$store"	verifică opusul condiției de mai sus

Comanda **test** are ca echivalent comanda [(paranteză dreaptă deschisă). Cele două comenzi sunt echivalente: orice verificare se face cu **test**, se poate face și cu [, ca în Listing 13.12. Diferența este că, în cazul comenzii [, la sfârșitul condiției de verificare, trebuie adăugată construcția]. Din cauză că această construcție este adesea omisă sau este „lipită” de condiția de verificare, în loc să fie separată prin spațiu, recomandăm folosirea comenzii **test** în defavoarea comenzii [.

```
1  if test 3 -eq 3; then
2      echo "equal"
3  fi
4
5  if [ 3 -eq 3 ]; then
6      echo "equal"
7  fi
8
9  if test -f /etc/passwd; then
10   echo "exists"
11 fi
```

```
12
13 if [ -f /etc/passwd ]; then
14   echo "exists"
15 fi
```

Listing 13.12: Comanda test și comanda]

O altă comandă pentru verificarea condițiilor este comanda [[. Comanda [[diferă în două moduri de comenzile **test** și [:

- este o comandă internă shellului (cum am discutat în Secțiunea 7.1.4), în vreme ce **test** și [sunt comenzi externe
- este proprie shellului Bash; este posibil să se găsească și în alte shelluri, dar nu ne putem baza pe acest lucru

Comanda [[poate fi considerată o extensie a comenzilor **test** și [. Aceleași verificări realizate cu **test** și [pot fi realizate și cu [[. Pe lângă acestea, [[permite verificări suplimentare, ca cele prezentate în Listing 13.13:

- liniile 1-2: verificare de condiții compuse, folosind operatorul && (ȘI logic - *logical AND*)
- liniile 4-5: comparația unui șir cu o construcție de tip pattern / globbing (prezentate în Secțiunea 7.2.4.1), folosind operatorul ==
- liniile 7-8: comparația unui șir cu o construcție de tip expresie regulată (prezentate în Secțiunea 7.4), folosind operatorul =∼

```
1 student@uso:~/.../code/13-auto$ if [[ 3 -eq 3 && 9 -eq 9 ]]; then echo "
  equal"; fi
2 equal
3
4 student@uso:~/.../code/13-auto/wiki-cat$ if [[ /etc/passwd == *c/p* ]];
  then echo "match"; fi
5 match
6
7 student@uso:~/.../code/13-auto/wiki-cat$ if [[ /etc/passwd =~ /.*/.*d ]];
   then echo "match"; fi
8 match
```

Listing 13.13: Folosirea comenzii interne [[

În general recomandăm folosirea **test** pentru condiții uzuale și întotdeauna atunci când scriptul se dorește portabil între mai multe shelluri, așa cum vom discuta în Secțiunea 13.4.4. Folosim [[pentru situații de potrivire cu expresii regulate sau patternuri. Dacă avem nevoie de potrivire cu expresii regulate sau patternuri și este nevoie de portabilitatea scriptului, putem crea construcții echivalente folosind comenzi de lucru cu expresii regulate precum **grep**.

13.3.2.2 case

Comanda **case** este echivalentul instrucțiunii switch din C; are rolul de a potrivi o variabilă la mai multe variante și de a executa comenzile echivalente. Potrivirea se face folosind globbing / patternuri. La fel ca instrucțiunea switch în C, comanda **case** este mai puțin întâlnită în scripturi.

Sintaxa comenzii **case** este indicată în Listing 13.14. Valoarea variabilei `var` este comparată cu diferite patternuri, precum `abc*` sau `*[de]fg*`. În cazul fiecărei potriviri, se execută comenzile corespunzătoare. De avut în vedere construcția `;;` pentru închiderea unei opțiuni **case**. Construcția `*` definește echivalentul etichetei `default` din programarea în C: se potrivește cu orice șir. În mod obișnuit, se plasează ca ultima opțiune pentru **case**, iar comenzile corespunzătoare sunt executate dacă nu s-au potrivit cu nici o altă opțiune.

```
case var in
    abc*)
        command1
        command2
        ...
        ;;
    *[de]fg*)
        command3
        command4
        ...
        ;;
    *)
        command5
        command6
        ...
        ;;
esac
```

Listing 13.14: Sintaxa comenzii case

Un exemplu de utilizare a comenzii **case** este în fișierul de startup `/etc/init.d/ssh`, prezentat în Listing 13.15. În cadrul fișierului, primul parametru al scriptului (`$1`) este comanda de executat a scriptului cu valori precum `start`, `stop`, `reload`. În cazul fiecărei valori a parametrului se execută anumite comenzi. În cazul implicit (opțiunea *default*: `*`) se afișează un mesaj legat de modul de utilizare a scriptului.

```
student@uso:~/uso-book/code/13-auto$ cat /etc/init.d/ssh
[...]
case "$1" in
  start)
        check_privsep_dir
        check_for_no_start
        [...]
        ;;

  stop)
        log_daemon_msg "Stopping OpenBSD Secure Shell server" "sshd" ||
true
        [...]
        ;;

  reload|force-reload)
        check_for_no_start
        check_config
        [...]
        ;;

  [...]
  *)
```

```
23          log_action_msg "Usage: /etc/init.d/ssh {start|stop|reload|force-
    reload|restart|try-restart|status}" || true
24          exit 1
25  esac
26
27  exit 0
```

Listing 13.15: Exemplu de folosire case: /etc/init.d/ssh

13.3.2.3 for

Atunci când dorim să trecem, pe rând, printr-o listă de valori posibile pentru o variabilă folosim comanda **`for`**. O formă uzuală de folosire a comenzii for este pentru a trece printr-o listă de fișiere, cum este linia `7` din Listing 13.6. Folosind construcția `for i in *`, variabila `i` primește, pe rând, ca valoare, numele fiecărei intrări din directorul curent.

Dacă dorim să folosim comanda **`for`** pentru a parcurge o listă de indecși, similar instrucțiunii `for` din C, folosim una dintre construcțiile din Listing 13.16. Prima construcție (linia `1`) este o formă de expandare aritmetică. A doua construcție (linia `4`) folosește expandarea comenzii **`seq`** pentru a genera o listă de numere care va fi parcursă de variabila `i`. A treia construcție (linia `7`) folosește o formă de expandare cu acolade (*brace expansion*) pentru a genera lista de numere. Folosirea **`for`** pentru a parcurge o listă de indecși este mai puțin frecventă în scripturi shell față de alte scenarii. În general operațiile numerice / aritmetice, deși prezente în scripturi shell, sunt mai adecvate unor limbaje de programe complete, precum Python, Perl sau C.

```
1  student@uso:~/uso-book/code/13-auto$ for ((i=1; i<=10; i++)); do echo "$i
   "; done
2  [...]
3
4  student@uso:~/uso-book/code/13-auto$ for i in $(seq 1 10); do echo "$i";
   done
5  [...]
6
7  student@uso:~/uso-book/code/13-auto$ for i in {1..10}; do echo "$i"; done
8  [...]
```

Listing 13.16: Parcurgere numerică folosind for

Un mod de folosire a comenzii **`for`** este trecerea prin parametrii unui script, ca în Listing 13.17. În linia `9` am folosit **`for`** pentru a parcurge parametrii scriptului, indicați de construcția `$*`.

```
1  student@uso:~/uso-book/code/13-auto$ cat show-ids
2  #!/bin/bash
3
4  if test $# -eq 0; then
5      echo "Usage: $0 username1 [username2 ...]" 1>&2
6      exit 1
7  fi
8
9  for i in $*; do
10     echo -n "$i: "
11     id -u "$i"
12 done
13
```

```
14 student@uso:~/uso-book/code/13-auto$ ./show-ids
15 Usage: ./show-ids username1 [username2 ...]
16
17 student@uso:~/uso-book/code/13-auto$ ./show-ids student root
18 student: 1000
19 root: 0
```

Listing 13.17: Parcurgerea parametrilor unui script

În general, comanda **for** îmbracă una dintre formele din Listing 13.18. În prima formă, `expression` este cel mai adesea o expresie aritmetică, precum cea din linia `1` din Listing 13.16. În a doua formă, variabila `var` primește ca valoare, pe rând, elementele din lista `list`. Lista `list` poate fi definită explicit, prin scrierea de elemente separate prin spațiu, sau implicit, prin expandări care conduc la obținerea unei liste, cum am prezentat în exemplele de mai sus.

```
1  for ((expression)); do
2      command_to_execute1
3      command_to_execute2
4      ...
5  done
6
7  for var in list; do
8      command_to_execute1
9      command_to_execute2
10     ...
11 done
```

Listing 13.18: Sintaxa comenzii for

De avut în vedere că folosirea **for** poate fi problematică sau ineficientă. Folosirea **for** este problematică atunci când există spații în elementele listei. În acel caz, în loc să fie preluate elementele efective, vor fi preluate părțile din fiecare element separate cu spații. Vom discuta mijloace de rezolvare a spațiilor în listă în Secțiunea 13.4.1.

Folosirea **for** poate fi ineficientă când sunt alte forme de a folosi lista, fără a trece prin **for**, cum este cazul în Listing 13.19. În primul exemplu, rulăm comanda **for** și creăm mai multe procese **stat**, câte unul pentru fiecare fișier. În a doua variantă pur și simplu trimitem ca argument fișierele dorite comenzii **stat**. Prima variantă rămâne, însă, utilă pentru cazul în care dorim să executăm mai multe comenzi care folosesc un element al unei liste (intrare din directorul curent). În cazul nostru, am folosit doar comanda **stat** pentru fiecare element al listei. Dacă am fi dorit să folosim mai multe comenzi, atunci nu am fi putut folosi a doua variantă.

```
1  student@uso:~/uso-book/code/13-auto$ for i in *; do stat -c "%n,%s" "$i";
    done
2  gen-passwd,168
3  hello,4096
4  show-ids,158
5  system-update,80
6  wiki-cat,4096
7
8  student@uso:~/uso-book/code/13-auto$ stat -c "%n,%s" *
9  gen-passwd,168
10 hello,4096
11 show-ids,158
12 system-update,80
```

```
13 wiki-cat,4096
```

Listing 13.19: Alternativă la folosirea for

Ca de obicei, în shell scripting, există mai multe moduri de a realiza un lucru. Ideal, cumulat cu experiența, vom prefera cea mai bună opțiune pentru o situație dată: opțiunea cea mai robustă, în primul rând, și cea mai eficientă, în al doilea rând. Vom discuta aceste aspecte în Secțiunea 13.4.

13.3.2.4 while

Similar comenzii **for** și a instrucțiunii `while` din C, comanda **while** poate fi folosită pentru a bucla în așteptarea unei condiții, ca în Listing 13.20. În a doua formă, am folosit expandare aritmetică pentru compararea valorii variabilei `i` cu `10`.

```
1 student@uso:~/uso-book/code/13-auto$ i=1; while test $i -le 10; do echo "
  $i"; ((i++)); done
2 [...]
3 student@uso:~/uso-book/code/13-auto$ i=1; while ((i <= 10)); do echo "$i
  "; ((i++)); done
4 [...]
```

Listing 13.20: Folosirea numerică a comenzii while

La fel ca în cazul comenzii **for**, forma de parcurgere numerică a comenzii `while` este mai puțin frecventă. Astfel de operații numerice sunt preferate unor limbaje de programare precum Python, Perl sau C. Frecvent, comanda **while** este folosită pentru a face parsing la fișiere text structurate tabelar (linii și coloane), folosind comanda **read**, ca în scriptul `extract-10` din Listing 13.21. Scriptul afișează numele și grupa studenților care au nota finală 10. În script am folosit comanda **while** pentru a parcurge fiecare linie din fișierul `students-surnames-only.txt`; fișierul este redirectat către comanda **while**. Parsarea fiecărui element din linie este realizat de comanda **read**; fiecare element este reținut în câte o variabilă `name`, `group`, `final_grade`, ...

```
1  student@uso:~/.../code/13-auto/students$ ls
2  extract-10  extract-10-csv  extract-10-tab  students.csv  students-
   surnames-only.txt  students.txt
3
4  student@uso:~/.../code/13-auto/students$ cat extract-10
5  #!/bin/bash
6
7  while read name group final_grade test_grade practical_grade; do
8      if test "$final_grade" -eq 10; then
9          echo "$name,$group"
10     fi
11 done < students-surnames-only.txt
12
13 student@uso:~/.../code/13-auto/students$ ./extract-10
14 GEORGIU,311CC
15 MUȘATESCU,311CC
16 RADU,311CC
17 GONDOȘ,312CC
18 NECULAI,313CC
19 MUHCINĂ,313CC
```

```
GRAMA,314CC
VĂSII,314CC
RĂDOI,314CC
NEACȘU,314CC
BARBU,315CC
MĂNOIU,315CC
ȘTIRBĂȚ,315CC
```

Listing 13.21: Script: Extragerea studenților cu nota 10

Sintaxa uzuală pentru **`while`**, în combinație cu **`read`** este indicată în Listing 13.22. În general intrarea este dată de la un fișier (prin redirectare) de la o altă comandă (prin înlănțuire cu operatorul *pipe* - |).

```
while read field1 field2 ...; do
    command1
    command2
done < file_to_redirect

command_to_redirect | while read field1 field2 ...; do
    command1
    command2
done
```

Listing 13.22: Sintaxa comenzii while (cu read)

Pentru a face parsing, comanda **`read`** folosește variabila shell `IFS` (*Input Field Separator*). În mod implicit, variabila `IFS` este inițializată la caracterele albe (spațiu, Tab, linie nouă). Dacă dorim să folosim alt separator pentru parsare, vom actualiza variabila IFS ca în scripturile `extract-10-tab` și `extract-10-csv` din Listing 13.23, respectiv Listing 13.24. Listing 13.23 folosește Tab pe post de separator și fișierul de intrare `students.txt`, ale cărui elemente sunt separate de Tab în fiecare linie. Listing 13.24 folosește virgulă (`,`) pe post de separator și fișierul de intrare `students.csv` (de tip CSV - *Comma Separated Values*), ale cărui elemente sunt separate de virgulă pe fiecare linie.

```
student@uso:~/.../code/13-auto/students$ cat extract-10-tab
#!/bin/bash

IFS=$'\t'
while read name group final_grade test_grade practical_grade; do
    if test "$final_grade" -eq 10; then
        echo "$name,$group"
    fi
done < students.txt

student@uso:~/.../code/13-auto/students$ ./extract-10-tab
GEORGIU V. Alexandra-Maria,311CC
MUȘATESCU V. Alexandru-Petrișor,311CC
RADU L. Alina,311CC
GONDOȘ I. Gabriel,312CC
NECULAI M. Alexandru-George,313CC
MUHCINĂ V. Silviu-Mihai,313CC
GRAMA C. Serban-Alexandru,314CC
VĂSII I. Bogdan-Cristian,314CC
RĂDOI D. Ghennadi,314CC
NEACȘU C. Florin-Mărgărit,314CC
BARBU I. Ștefan,315CC
```

```
MĂNOIU I. Ioana-Veronica,315CC
ȘTIRBĂȚ A. Steliana,315CC
```

Listing 13.23: Script: Extragerea studenților cu nota 10 (Tab ca separator)

```
student@uso:~/.../code/13-auto/students$ cat extract-10-csv
#!/bin/bash

IFS=','
while read name group final_grade test_grade practical_grade; do
    if test "$final_grade" -eq 10; then
        echo "$name,$group"
    fi
done < students.csv

student@uso:~/.../code/13-auto/students$ ./extract-10-csv
GEORGIU V. Alexandra-Maria,311CC
MUȘATESCU V. Alexandru-Petrișor,311CC
RADU L. Alina,311CC
GONDOȘ I. Gabriel,312CC
NECULAI M. Alexandru-George,313CC
MUHCINĂ V. Silviu-Mihai,313CC
GRAMA C. Serban-Alexandru,314CC
VĂSII I. Bogdan-Cristian,314CC
RĂDOI D. Ghennadi,314CC
NEACȘU C. Florin-Mărgărit,314CC
BARBU I. Ștefan,315CC
MĂNOIU I. Ioana-Veronica,315CC
ȘTIRBĂȚ A. Steliana,315CC
```

Listing 13.24: Script: Extragerea studenților cu nota 10 (CSV)

Prin inițializarea variabilei `IFS` la un separator dat (Tab sau virgulă) am putut să afișăm numele complet al studenților, nu doar numele de familie, cum era cazul în scriptul `extract-10`. În scriptul `extract-10` am folosit o versiune modificată a fișierului de intrare, în care prima coloană a fișierului conține doar numele de familie. Dacă ar fi conținut și inițiala tatălui și prenumele, atunci acestea ar fi fost câmpuri parsate de comanda **`read`** la folosirea implicită a separatorului `IFS` (care conține caractere albe: spațiu, Tab, linie nouă); caz în care valorile câmpurilor `group`, `final_grade` și altele ar fi fost necorespunzătoare.

Așa cum am văzut în exemplele de mai sus, construcția **`while read`** este utilă atunci când dorim să prelucrăm date în format tabelar, putând rula comenzi pe câmpurile parsate din fiecare linie de la intrare. Putem privi construcția **`while read`** ca o comandă **`cut`** extinsă. **`cut`** doar permite extragerea uneia sau mai multor câmpuri / coloane dar fără a permite execuția de comenzi pentru fiecare element extras. **`while read`** permite prelucrarea suplimentară (nu doar extragerea) câmpurilor parsate de pe fiecare linie.

La fel ca **`for`**, comanda **`while`** poate fi folosită necorespunzător într-un mod în care robustețea, lizibilitatea sau performanța sunt afectate. În general, folosirea buclelor, în special pentru prelucrarea text poate fi sursă de probleme în scripturi shell[1].

[1] https://unix.stackexchange.com/a/169765/227174

13.3.3 Funcții

Atunci când avem secvențe de comenzi pe care vrem să le folosim modular, putem folosi funcții. La fel ca într-un limbaj de programare, o funcție are un nume și are un conținut; o funcție rulează când este apelată. O funcție este apelată folosind numele său, ca în Listing 13.25, care prezintă secvențe din scriptul `/etc/init.d/ssh`. Sunt definite funcțiile `check_dev_null`, `check_privsep_dir` și `check_config`, care sunt apoi apelate pe parcursul programului.

```
student@uso:~/uso-book/code/13-auto$ cat /etc/init.d/ssh
[...]
check_dev_null() {
    if [ ! -c /dev/null ]; then
        if [ "$1" = log_end_msg ]; then
            log_end_msg 1 || true
        fi
        if ! run_by_init; then
            log_action_msg "/dev/null is not a character device!" || true
        fi
        exit 1
    fi
}

check_privsep_dir() {
    # Create the PrivSep empty dir if necessary
    if [ ! -d /run/sshd ]; then
        mkdir /run/sshd
        chmod 0755 /run/sshd
    fi
}

check_config() {
    if [ ! -e /etc/ssh/sshd_not_to_be_run ]; then
        /usr/sbin/sshd $SSHD_OPTS -t || exit 1
    fi
}

export PATH="${PATH:+$PATH:}/usr/sbin:/sbin"

case "$1" in
  start)
        check_privsep_dir
        [...]
        check_dev_null
        [...]
        ;;

  [...]
  reload|force-reload)
        check_for_no_start
        check_config
        [...]
        ;;
[...]
```

Listing 13.25: Folosirea funcțiilor (/etc/init.d/ssh)

Din punctul de vedere al folosirii, o funcție sau o comandă sunt echivalente:

- pentru apelarea unei funcții, se scrie numele funcției (fără paranteze, cum e cazul limbajelor de programare)
- dacă funcția primește parametri, aceștia sunt transmiși în continuarea numelui funcției, separați prin spații
- o funcție poate întoarce un cod de ieșire (folosind comanda `return`), cod ce poate fi investigat
- o funcție poate afișa mesaje ce pot fi reținute prin construcția de expandare a comenzilor (`$(...)`)

Doar uitându-ne la un script care folosește comenzi sau funcții, nu am putea să ne diferențiem între cele două.

Așa cum am observat și în Listing 13.25, funcțiile ajung să fie folosite în scripturi shell mai complexe, mai puțin în scripturi uzuale. Scripturi mai complexe sunt scripturile de inițializare din directorul `/etc/init.d/`, cum este și cazul scriptului `/etc/init.d/ssh` din Listing 13.25.

13.4 Facilități avansate și bune practici în shell scripting

Așa cum am precizat anterior, un script shell este o cale de mijloc între a rula comenzile în linia de comandă și a face un program într-un limbaj de programare. Pe lângă acestea, în shell sunt comenzi similare și funcționalități diferite care pot conduce la același rezultat. Fiind mai multe modurile de a ajunge la un rezultat în shell, cel mai bine este să creăm scripturi shell folosind cea mai bună abordare dintre cele posibile.Pentru aceasta, pentru fiecare script vom urmări:

- **robustețea**: scriptul să ruleze indiferent de condiții, să nu aibă erori de rulare sau rezultate imprevizibile
- **lizibilitate**: scriptul să fie ușor de citit și de înțeles de altcineva; anumite construcții în shell sunt întortocheate și pot face dificilă înțelegerea scriptului
- **eficiența și performanța**: deși nu poate ajunge la nivelul de performanță al unui program, e indicat ca scriptul să ruleze cât mai repede și să nu consume resurse în mod exagerat
- **portabilitatea**: există mai multe tipuri de shell (Bash, Dash, ksh, tcsh, zsh) sau diferite versiuni de shell; dacă se știe că un script va rula pe shelluri și în medii diferite trebuie să fie portabil și să ruleze determinist

Mai jos prezentăm construcții specifice acestor deziderate și recomandări de folosire în scripturi shell.

13.4.1 Robustețe

Robustețea unui script înseamnă că acesta funcționează în condiții cât mai diverse. Acest lucru înseamnă că având intrări diferite, sau pe configurații diferite sau cu intrări în sistemul de fișiere diferite, scriptul va funcționa corespunzător.

Mai jos sunt indicate câteva surse de probleme care pot afecta robustețea scripturilor. Vom detalia ulterior soluții la acestea:

- **inconsecvența formatului datelor prelucrate**: Unele comenzi afișează informațiile în formate care pot varia și care pot produce probleme. Soluția este folosirea de comenzi sau fișiere cu informație consecventă sau a avea în vedere, cât de mult se poate, toate cazurile.
- **configurații diferite de sistem**: Un script presupune anumite configurații care nu sunt prezente pe toate sistemele. Sau necesită rularea în anumite condiții în absența cărora rularea duce la erori.
- **caractere speciale**: Intrările în sistemul de fișiere pot conține caractere speciale (precum spații sau chiar caractere newline) care nu sunt luate în considerare de script.

Așa cum am precizat în Secțiunea 7.5.1 și în Secțiunea 13.1, utilitarele generatoare de text sunt de două tipuri, în funcție de utilitatea rezultatului rulării:

1. utilitare pentru afișarea de informații, pe ecran, pentru a fi accesibile utilizatorului
2. utilitare care afișează informația în format prelucrabil, util pentru legarea de alte comenzi, adică pentru automatizare

Pentru a nu fi afectate de inconsecvența datelor prelucrate, în scripturi e recomandată folosirea utilitarelor care au ca obiectiv prelucrarea, adică a doua categorie de mai sus. Astfel, dacă dorim să obținem pentru prelucrare dimensiunea unui fișier vom folosi **`stat`** în loc de **`ls`**, ca în Listing 13.1. La fel, vom folosi **`pidof`** sau **`pgrep`** în loc de **`ps`**, ca în Listing 13.2.

Forme precum **`ps`** | **`grep`** rămân în continuare utile atunci când vrem să fie afișate utilizatorului doar anumite informații din rezultatul comenzii **`ps`**. Dacă însă dorim prelucrarea acelor informații de alte comenzi, vom folosi **`pidof`** sau **`pgrep`**.

În partea de configurații diferite putem avea variabila de mediu `PATH` cu valori diferite. Drept urmare, e posibil ca o comandă să poată fi rulată doar furnizând calea completă. Adică să funcționeze **`/bin/ls`** dar nu **`ls`**. Mai mult, anumite utilitare pot să nu fie instalate. Dacă proiectăm scripturi de sistem (rulate cu permisiuni administrative) sau scripturi cu șanse de a a rula pe configurații diferite, vom specifica o comandă folosind calea completă. În felul acesta, prevenim cazurile în care variabila de mediu `PATH` nu are conținutul așteptat.

Dacă un script se dorește rulat ca `root` trebuie verificat acest lucru printr-o construcție de genul celei din Listing 13.26. În caz contrar scriptul va fi rulat ca utilizator obișnuit (neprivilegiat) și vor apărea erori la rularea comenzilor privilegiate.

```
1 if test $(id -u) -ne 0; then
2     echo "You have to be root to run this script". 1>&2
3     exit 1
4 fi
```

Listing 13.26: Verificare root în script

Unele scripturi vor folosi **`sudo`** sau o conexiune SSH sau alte comenzi care pot cere (sau nu) introducerea unei parole. Este posibil ca pe un sistem să avem configurată autentificarea SSH sau rularea **`sudo`** fără cererea parolei iar pe alte sistem să nu fie

configurată. Scriptul trebuie să aibă în vedere acest lucru și să configureze sistemul corespunzător sau să folosească un utilitar de automatizare a comenzilor neinteractive, așa cum vom preciza în Secțiunea 13.4.5.

Când rulăm un script trebuie să avem în vedere din ce director îl rulăm. În script este posibil să existe căi relative la directorul din care rulăm. Din păcate, nu există nici o soluție care să funcționeze în toate cazurile[1]. Sunt recomandate două abordări:

1. folosirea căilor absolute către fișierele folosite în script, precum plasarea fișierelor de configurare într-un director cu o cale fixă
2. rularea scriptului doar din directorul în care se găsește scriptul

Atunci când lucrăm cu fișiere trebuie să ținem cont de numele acestora. Numele fișierelor pot conține caractere speciale precum spațiu (*blank*), folosit de shell pentru separarea argumentelor. De exemplu, în Listing 13.27, avem o situație în care un director conține două subdirectoare (`movies/` și `books/`), fiecare subdirector conținând două fișiere al căror nume conțin spații; așa cum se vede în liniile `1-5`. La o rulare a comenzii **`for`** în combinație cu **`find`**, nu vom obține o parcurgere corespunzătoare a fișierelor ce conțin spații; așa cum se vede în liniile `7-16`. Soluția este fie folosirea **`while`** (liniile `18-22`), fie inițializarea separatorului din variabila `IFS` (*Input Field Separator*) la caracterul linie nouă (*newline*), prin construcția `$'\n'` (liniile `24-28`).

```
1  student@uso:~/.../code/13-auto/filenames$ find -type f
2  ./movies/The Godfather
3  ./movies/Shawshank Redemption
4  ./books/The Great Gatsby
5  ./books/The Stand
6
7  student@uso:~/.../code/13-auto/filenames$ for f in $(find -type f); do
   stat -c "%n,%s" "$f"; done
8  stat: cannot stat './movies/The': No such file or directory
9  stat: cannot stat 'Godfather': No such file or directory
10 stat: cannot stat './movies/Shawshank': No such file or directory
11 stat: cannot stat 'Redemption': No such file or directory
12 stat: cannot stat './books/The': No such file or directory
13 stat: cannot stat 'Great': No such file or directory
14 stat: cannot stat 'Gatsby': No such file or directory
15 stat: cannot stat './books/The': No such file or directory
16 stat: cannot stat 'Stand': No such file or directory
17
18 student@uso:~/.../code/13-auto/filenames$ find -type f | while read f; do
    stat -c "%n,%s" "$f"; done
19 ./movies/The Godfather,14
20 ./movies/Shawshank Redemption,12
21 ./books/The Great Gatsby,20
22 ./books/The Stand,13
23
24 student@uso:~/.../code/13-auto/filenames$ IFS=$'\n'; for f in $(find -
   type f); do stat -c "%n,%s" "$f"; done
25 ./movies/The Godfather,14
26 ./movies/Shawshank Redemption,12
27 ./books/The Great Gatsby,20
28 ./books/The Stand,13
```

Listing 13.27: Tratarea caracterelor speciale în numele fișierelor

[1] http://mywiki.wooledge.org/BashFAQ/028

Chiar și soluțiile care urmăresc citirea linie cu linie a elementelor (precum folosirea **while** sau inițializarea `IFS` la *newline*) pot avea probleme dacă un fișier conține caracterul linie nouă (*newline* - `\n`) în numele său. Deși ciudat, fișierele pot conține caracterul *newline* în nume. Nu este uzual să existe astfel de nume de fișiere, dar un atacator poate crea fișiere cu astfel de nume pentru a sabota funcționarea scriptului. Trei dintre soluțiile posibile pentru cazul în care există *newline* în numele unui fișier sunt prezentate în Listing 13.28:

- Folosim globbing împreună cu **for**, ca în liniile `1-5`. Forma cu globbing este robustă la prezența *newline* în numele fișierelor. Dar este problematică dacă avem nevoie de o parcurgere în adâncime (recursivă) a unor fișiere, caz în care preferăm să folosim `find`.
- Folosim **find** (de care am discutat în Secțiunea 7.6.1) împreună cu opțiunea `-exec`, ca în liniile `7-11`. Această formă permite rularea unei singure comenzi pe intrare. Funcționează în cazul nostru, pentru că dorim rularea comenzii **stat** pentru fiecare intrare.
- Folosim **find** cu afișarea intrărilor separate de terminatorul de șir (`\0`), ca în liniile `13-17`. Caz în care separatorul `IFS` este resetat, la fel și delimitatorul folosit de comanda **read**.

Aici[1] sunt descrise un spectru larg de soluții la situația caracterelor problematice în numele de fișiere.

```
1  student@uso:~/.../code/13-auto/filenames$ for f in ./*/*; do stat -c "%n
   ,%s" "$f"; done
2  ./books/The Great Gatsby,20
3  ./books/The Stand,13
4  ./movies/Shawshank Redemption,12
5  ./movies/The Godfather,14
6
7  student@uso:~/.../code/13-auto/filenames$ find -type f -exec stat -c "%n
   ,%s" {} \;
8  ./movies/The Godfather,14
9  ./movies/Shawshank Redemption,12
10 ./books/The Great Gatsby,20
11 ./books/The Stand,13
12
13 student@uso:~/.../code/13-auto/filenames$ find -type f -print0 | while
   IFS="" read -r -d "" f; do stat -c "%n,%s" "$f"; done
14 ./movies/The Godfather,14
15 ./movies/Shawshank Redemption,12
16 ./books/The Great Gatsby,20
17 ./books/The Stand,13
```

Listing 13.28: Tratarea prezenței newline în numele fișierelor

13.4.2 Lizibilitate

Scripturile sunt *quick and dirty* și trebuie să fie lizibile, ușor de înțeles. Pentru aceasta este de preferat să nu se folosească construcții în care se poate greși repede sau care sunt greu de înțeles.

[1] https://www.dwheeler.com/essays/filenames-in-shell.html

De exemplu, pentru expandarea unei comenzi sunt posibile două forme, ca în Listing 13.29:

- Folosim caracterul *backtick* (`) în sintaxa `` `command` `` ca în liniile 1–5.
- Folosim construcția `$(command)` ca în liniile 7–11.

Între cele două forme, recomandăm a doua formă, care folosește construcția `$(command)`. Prima formă are probleme de lizibilitate: caracterul *backtick* poate fi confundat cu apostrof; pe lângă aceasta, este mai dificil de localizat pe tastatură. De partea cealaltă, construcția `$(command)` este mai ușor de citit și tastat.

```
1  student@uso:~/.../code/13-auto/filenames$ IFS=$'\n'; for f in `find -type
    f`; do stat -c "%n,%s" "$f"; done
2  ./movies/The Godfather,14
3  ./movies/Shawshank Redemption,12
4  ./books/The Great Gatsby,20
5  ./books/The Stand,13
6
7  student@uso:~/.../code/13-auto/filenames$ IFS=$'\n'; for f in $(find -
   type f); do stat -c "%n,%s" "$f"; done
8  ./movies/The Godfather,14
9  ./movies/Shawshank Redemption,12
10 ./books/The Great Gatsby,20
11 ./books/The Stand,13
```

Listing 13.29: Moduri de expandare a comenzilor

Similar, între folosirea comenzii **`test`** și folosirea comenzii `[` preferăm folosirea comenzii **`test`**, așa cum am precizat și în Secțiunea 13.3.2.1. Întrucât construcția `[` poate să fie văzută ca un operator, nu o comandă, și se pot omite spații, recomandăm folosirea comenzii **`test`**.

Atunci când avem de furnizat un mesaj unei comenzi putem folosi *here documents* și *here strings*, forme mai expresive de transmitere a informației. *Here documents* și *here strings* sunt forme „inline” de transmitere a informației așa cum sunt prezentate în Listing 13.30. Liniile 1–4 arată cum putem scrie 2 linii în fișierul `~/.inputrc` (*here document*). Linia 6 arată cum putem efectua calculul 2^{10}, folosind utilitarul **`bc`** (*here string*). Ambele forme pot fi realizate și folosind redirectări / *pipe*; dar, din rațiuni de lizibilitate, sau pentru a nu crea un proces nou, putem folosi *here documents* și *here strings*

```
1  cat > ~/.inputrc <<END
2  set show-all-if-ambiguous On
3  set completion-ignore-case On
4  END
5
6  bc <<< "2^10"
```

Listing 13.30: Here document și here string

13.4.3 Eficiență

Acolo unde avem mai multe opțiuni și avem robustețea asigurată, e de preferat să folosim soluția cea mai eficientă și performantă. De exemplu, între variantele din Listing 13.28,

am prefera ultima variantă, care folosește doar **find**, fără **while** sau **for**. Este cea mai simplă și, deci, cea mai eficientă. Pe lângă problemele inerente de robustețe posibile a procesului de parsing, o variantă precum cea cu **while** durează mai mult, trebuind să citească ieșirea comenzii **find** și apoi să o transfere către comandă. Variantele folosind **while** și **for** rămân utile atunci când vrem să rulăm mai multe comenzi; comanda **find** permite, împreună cu `-exec` rularea unei singure comenzi[1].

Altfel, atunci când folosim **find** pentru a executa o comandă, avem opțiunile din Listing 13.31. Prima variantă (liniile `1-5`) execută comanda **stat** pentru fiecare intrare; sunt create astfel 4 procese **stat**, câte una pentru fiecare fișier. A doua (liniile `7-11`) și a treia (liniile `13-17`) variantă execută o singură comandă **stat** pentru toate intrările, fiind evident mai performantă. A doua și a treia variantă sunt aproape echivalente. Prima variantă creează câte un proces pentru fiecare fișier fiind mai costisitoare ca timp. Lucrurile de acest fel încep să conteze când scripturile prelucrează cantități mari de informații: multe fișiere sau multe date de intrare.

```
1  student@uso:~/.../code/13-auto/filenames$ find -type f -exec stat -c "%n
   ,%s" {} \;
2  ./movies/The Godfather,14
3  ./movies/Shawshank Redemption,12
4  ./books/The Great Gatsby,20
5  ./books/The Stand,13
6
7  student@uso:~/.../code/13-auto/filenames$ find -type f -exec stat -c "%n
   ,%s" {} +
8  ./movies/The Godfather,14
9  ./movies/Shawshank Redemption,12
10 ./books/The Great Gatsby,20
11 ./books/The Stand,13
12
13 student@uso:~/.../code/13-auto/filenames$ find -type f -print0 | xargs -0
    stat -c "%n,%s"
14 ./movies/The Godfather,14
15 ./movies/Shawshank Redemption,12
16 ./books/The Great Gatsby,20
17 ./books/The Stand,13
```

Listing 13.31: Forme de rulare de comenzi pe fișiere (cu find)

> **Important:** Cu toate acestea, urmărind robustețea, există o situație în care prima forma a comenzii **find** (liniile `1-5` din Listing 13.31) este preferată: atunci când sunt foarte multe intrări descoperite, celelalte forme pot eșua din cauza limitării numărului de argumente pe care le poate lua o comandă.

Ca un plus de performanță, dintre a doua și a treia variantă, varianta a treia (care folosește comanda **xargs**) poate fi utilă atunci când folosim opțiunea `-P`. Această opțiune permite precizarea numărului de core-uri folosite pentru rularea comenzilor în paralel, ceea ce poate duce la creșterea performanței.

[1] Se poate trece peste această limitare prin rularea unui shell nou sau a unui script, dar se pierde lizibilitate.

13.4.4 Portabilitate

Anumite construcții din scripturi shell sunt specifice unui tip de shell (precum Bash). Le mai numim *bashisms*. Dacă dorim să rulăm un script pe mai multe tipuri de shell[1] sau pe diferite versiuni de shell, trebuie să avem în vedere ce construcții sunt standard și portabile. Pentru scripturi portabile recomandarea este folosirea de funcționalități care sunt recunoscute de standardul POSIX. Un shell compatibil POSIX[2] va rula un script construit conform standardului.

De exemplu, pentru redirectarea simultană a ieșirii standard și ieșirii de eroare standard avem două moduri: `&> file` sau `> file 2>&1`. Preferăm opțiunea a doua, care este portabilă. Prima variantă nu funcționează, de exemplu, pe `sh` (Bourne Shell).

O altă formă de bashism este construcția [[pentru testarea condițiilor, despre care am discutat în Secțiunea 13.3.2.1, o construcție specifică Bash. Construcția se găsește și pe alte shelluri, dar nu este garantată prezența ei pe toate shellurile. Dacă dorim să avem scripturi portabile, vom evita folosirea [[. Dacă avem nevoie de comparația valorii unei variabile cu o expresie regulată, vom apela la o soluție alternativ, folosind **`grep`**, ca în Listing 13.32. A doua variantă folosește **`grep`** pentru a realiza același lucru: verificarea că valoarea `ana` corespunde expresiei regulate `.*$`. Varianta cu **`grep`** este mai complicată și mai puțin lizibilă, dar este portabilă.

```
1 student@uso:~/uso-book/code/13-auto$ if [[ "ana" =~ .*a$ ]]; then echo "
  match"; fi
2 match
3
4 student@uso:~/uso-book/code/13-auto$ if grep '.*a$' <<< "ana" > /dev/null
   2>&1; then echo "match"; fi
5 match
```

Listing 13.32: Alternativă la folosirea [[(grep)

Am văzut în Listing 13.28 și în Listing 13.31 că, pentru a folosi robust prelucrarea fișierelor cu **`find`**, trebuie să folosim opțiunea `-print0`. Opțiunea `-print0` nu este, însă, în standardul POSIX și este posibil să nu fie prezentă pe anumite sisteme. Din păcate, folosirea **`find`** cu opțiunea `-print0` poate ajunge sa fie singura soluție pentru prelucrarea robustă a fișierelor dintr-o ierarhie; așa că aici trebuie optat între portabilitate 100% și robustețe.

13.4.5 Alte funcționalități

Atunci când dorim să facem debugging în shell, soluția cea mai directă este afișarea unui mesaj sau a valorilor unei variabile folosind comenzile **`printf`** sau **`echo`**. O altă variantă este să folosim în cadrul unui script shell construcțiile `set -x` și `set +x` care activează, respectiv dezactivează, afișarea comenzilor atunci când se execută. Acest lucru permite să vedem dacă o comandă se execută sau nu și toate expandările realizate (variabile, expandare de comenzi, expandare aritmetică). În scriptul `gen-passwd-dbg` din Listing 13.33 activăm afișarea comenzilor pentru cele trei linii care folosesc expandarea comenzilor. Observăm că se afișează și comenzile executate

[1] https://stackoverflow.com/questions/19428418/what-is-the-use-of-portable-shell-scripts
[2] http://pubs.opengroup.org/onlinepubs/009695399/utilities/xcu_chap02.html

și atribuirile de variabile. Dacă avem probleme într-un script, acest mod de depanare este util să depistăm rapid problemele.

```
student@uso:~/uso-book/code/13-auto$ cat gen-passwd-dbg
#!/bin/bash

set -x
salt=$(pwgen -N 1 -c 8)
password=$(pwgen -N 1 -c 10)
crypt=$(openssl passwd -1 -salt "$salt" "$password")
set +x

echo "password: $password"
echo "crypt: $crypt"

student@uso:~/uso-book/code/13-auto$ ./gen-passwd-dbg
++ pwgen -N 1 -c 8
+ salt=bohJ0noj
++ pwgen -N 1 -c 10
+ password=uu2tiiGhoc
++ openssl passwd -1 -salt bohJ0noj uu2tiiGhoc
+ crypt='$1$bohJ0noj$30nn0fhwx2RTHDS5SxmK30'
+ set +x
password: uu2tiiGhoc
crypt: $1$bohJ0noj$30nn0fhwx2RTHDS5SxmK30
```

Listing 13.33: Permisiuni pe fișiere cu parole

Se întâmplă adesea să folosim comenzi care necesită intrare de la utilizator într-un script shell. Aceste comenzi sunt interactive și nu pot fi folosite în această formă într-un script shell în care se dorește automatizare completă. De exemplu comenzi precum **`update-alternatives`**, **`passwd`**, **`ftp`**, **`curl`**, **`wget`**, **`ssh`**, **`sudo`**. Pentru unele există forme de rulare neinteractivă: în loc de **`passwd`** se poate folosi **`chpasswd`**; **`curl`** și **`wget`** au opțiuni de furnizare de nume de utilizator și parolă. Atunci când nu există forme de rulare neinteractivă, se pot folosi utilitare din categoria **`expect`** care permit transmiterea de informații în mod automat către comenzi interactive. Le vom detalia în Secțiunea 13.6.

13.4.6 Resurse suplimentare

Pentru sfaturi practice despre dezvoltarea shell scripturilor (și folosirea Bash) recomandăm ghidul BashFAQ: https://mywiki.wooledge.org/BashFAQ

13.5 Automatizarea la nivelul sistemelor informatice

13.5.1 Gestiunea serviciilor

În Capitolul 9 am precizat că procesul `init` (de obicei implementat folosind `systemd` în distribuțiile Linux moderne) este responsabil pentru pornirea primelor procese ale sistemului, majoritatea fiind serviciile. Serviciile sunt procese de tip daemon, care rulează în background și care gestionează sistemul sau oferă funcționalități pentru alte

aplicații. Serviciile sunt gestionate de `systemd` fie în modul compatibil System V (folosind intrările din directorul `/etc/init.d/`) fie în modul specific `systemd`, așa cum am precizat în Secțiunea 9.6.

Dacă dorim să adăugăm un serviciu nou în sistem putem face acest lucru folosind `systemd`. Aceasta presupune crearea unui fișier de configurare pentru serviciu (numit `unit` în `systemd`) și apoi încărcarea sa în `systemd`[1]. Putem face acest lucru cu un serviciu la nivelul utilizatorului (nu la nivelul sistemului), folosind instanța de **`systemd`** specifică utilizatorului (**`systemd --user`**)[2].

Listing 13.34 conține o secvență de comenzi pentru activarea unui serviciu `systemd` la nivelul utilizatorului. Serviciul este definit în fișierul `dater.service`, fișier care va fi plasat în directorul de configurare `systemd` de la nivelul utilizatorului: `~/.config/systemd/user/`. În fișierul serviciului, directiva de configurare `ExecStart` precizează care este comanda executată de serviciu, în cazul de față rularea scriptului `/home/student/uso-book/code/13-auto/systemd/dater`. Acest script este unul didactic, care afișează data curentă în fișierul `/home/student/date` la fiecare `60` de secunde. După activarea și pornirea serviciului, verificăm fișierul `/home/student/date` (liniile `40-44`) și validând, astfel, funcționarea corectă a scriptului. O dată instalat serviciul în directorul `~/.config/systemd/user/` rulăm comenzile de listare / verificare (linia `31`) și de activare, pornire, oprire, dezactivare.

```
1  student@uso:~/.../code/13-auto/systemd$ ls
2  dater  dater.service
3
4  student@uso:~/.../code/13-auto/systemd$ cat dater.service
5  [Unit]
6  Description=My test dater service
7
8  [Service]
9  Type=simple
10 WorkingDirectory=/home/student
11 ExecStart=/bin/bash /home/student/uso-book/code/13-auto/systemd/dater
12 Restart=always
13 PrivateTmp=true
14 NoNewPrivileges=true
15
16 [Install]
17 WantedBy=default.target
18
19 student@uso:~/.../code/13-auto/systemd$ cat /home/student/uso-book/code
   /13-auto/systemd/dater
20 #!/bin/bash
21
22 while true; do
23     date > /home/student/date
24     sleep 60
25 done
26
27 student@uso:~/.../code/13-auto/systemd$ mkdir -p ~/.config/systemd/user/
28
29 student@uso:~/.../code/13-auto/systemd$ cp dater.service ~/.config/
   systemd/user/
```

[1] https://www.devdungeon.com/content/creating-systemd-service-files
[2] https://www.brendanlong.com/systemd-user-services-are-amazing.html

```
30
31 student@uso:~/.../code/13-auto/systemd$ systemctl --user list-unit-files
   | grep dater
32 dater.service                          disabled
33
34 student@uso:~/.../code/13-auto/systemd$ systemctl --user enable dater
35 Created symlink /home/student/.config/systemd/user/default.target.wants/
   dater.service -> /home/student/.config/systemd/user/dater.service.
36
37 student@uso:~/.../code/13-auto/systemd$ systemctl --user start dater
38 dater.service                          enabled
39
40 student@uso:~/.../code/13-auto/systemd$ cat /home/student/date
41 joi 7 ianuarie 2021, 05:39:24 +0200
42
43 student@uso:~/.../code/13-auto/systemd$ cat /home/student/date
44 joi 7 ianuarie 2021, 05:42:24 +0200
45
46 student@uso:~/.../code/13-auto/systemd$ systemctl --user stop dater
47
48 student@uso:~/.../code/13-auto/systemd$ systemctl --user disable dater
49 Removed /home/student/.config/systemd/user/default.target.wants/dater.
   service.
50
51 student@uso:~/.../code/13-auto/systemd$ systemctl --user list-unit-files
   | grep dater
52 dater.service                          disabled
```

Listing 13.34: Creare serviciu systemd la nivelul utilizatorului

13.5.1.1 supervisor

Supervisor este un daemon care permite configurarea de servicii și monitorizarea acestora. Are funcționalitate similară serviciilor configurate cu `systemd`. În general, configurările `systemd` pot fi realizate și în supervisor și invers[1].

Supevisor conține un daemon (`supervisord`), un utilitar de gestiune (**`supervisorctl`**) și fișiere de configurare. Față de `systemd`, supervisor deține fișiere de configurare ceva mai simple de scris și poate fi configurat ușor să ruleze servicii ca alt utilizator.

Listing 13.35 conține un exemplu configurare de serviciu supervisor pentru soluția vmchecker de verificare automată a submisiilor de teme[2]. Fișierul de configurare `/etc/supervisor/conf.d/vmchecker.conf` conține mai multe servicii configurate. Unul dintre aceste servicii este numit `vmchecker-ts` și rulează o comandă specifică pentru verificarea submisiilor pentru cursul de Teoria Sistemelor (TS). Folosind **`supervisorctl`** putem afla starea acestui serviciu, îl putem opri sau porni.

```
1 root@checker:~# cat /etc/supervisor/conf.d/vmchecker.conf
2 [...]
3 [program:vmchecker-ts]
```

[1] https://gunes.io/2017/08/25/systemd-vs-supervisor/
[2] https://vmchecker.cs.pub.ro/

```
4  command=/usr/local/bin/vmchecker-queue-manager --course_id=TS --stdin=/
   dev/null --stdout=/home/ts/tester/queue-manager.log --stderr=/home/ts/
   tester/queue-manager.log
5  directory=/srv
6  autostart=true
7  autorestart=true
8  startretries=3
9  stderr_logfile=/dev/null
10 stdout_logfile=/dev/null
11 user=ts
12
13 root@checker:~# supervisorctl status vmchecker-ts
14 vmchecker-ts                    RUNNING    pid 29463, uptime 59 days,
   7:31:15
15 root@checker:~# supervisorctl stop vmchecker-ts
16 vmchecker-ts: stopped
17 root@checker:~# supervisorctl status vmchecker-ts
18 vmchecker-ts                    STOPPED    Jan 07 05:52 AM
19 root@checker:~# supervisorctl start vmchecker-ts
20 vmchecker-ts: started
21 root@checker:~# supervisorctl status vmchecker-ts
22 vmchecker-ts                    RUNNING    pid 27707, uptime 0:00:03
```

Listing 13.35: Configurare supervisor

13.5.2 Planificarea sarcinilor

De multe ori dorim rularea de sarcini periodice sau la un moment dat. Pentru aceasta în Linux avem servicii specifice: **cron** și **at**. **cron** este folosit pentru rularea periodică a unei comenzi sau script. **at** este folosit pentru rularea unei comenzi sau a unui script la un moment dat.

O formă apropiată ideii de rulare planificată este rularea după un anumit eveniment cum este cazul procesului de *continuous integration* (CI). În cazul continuous integration, orice actualizare a unui proiect software declanșează în spate rularea unor scripturi de verificare. Exemple de utilitare de continuous integration sunt Travis CI, Jenkins, GitLabCI.

13.5.2.1 cron

cron este un serviciu care duce la rularea periodică a unei comenzi. Un caz de utilizare pentru **cron** sunt scenariile de *nightly builds* în care la momente bine determinate se verifică infrastructura, sau se compilează aplicații. Sau situații în care dorim o reindexare a fișierelor de pe sistemul local sau un backup periodic care beneficiază de folosirea **cron**.

cron poate fi configurat la nivel global în fișiere de configurare sau la nivelul unui utilizator.

La nivel global, **cron** este configurat în directoare / fișiere de configurare precum cele din linia 2 din Listing 13.36. O linie de configurare are formatul celor din liniile 10–13. Liniile de configurare precizează momentul de periodicitate, utilizatorul folosit și comanda ce va fi executată. Momentul de periodicitate stabilește minutul, ora, ziua din

lună, luna, ziua din săptămână. În felul acesta putem configura periodicități diverse de la rulare la fiecare minut până la rulare o dată pe an.

```
student@uso:~/uso-book/code/13-auto$ ls -dF /etc/cron*
/etc/cron.d/  /etc/cron.daily/  /etc/cron.hourly/  /etc/cron.monthly/  /
etc/crontab  /etc/cron.weekly/

student@uso:~/uso-book/code/13-auto$ cat /etc/crontab
[...]
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts
--report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts
--report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts
--report /etc/cron.monthly )
#
```

Listing 13.36: Configurarea cron

La nivel unui utilizator, configurarea **cron** se realizează prin intermediul comenzii **crontab -e**. Adică nu se editează direct un fișier, ci se apelează o comandă, similar configurării **sudo** folosind comanda **visudo**. Listing 13.37 conține un exemplu de configurare **cron** la nivelul utilizatorului `student`. Folosind comanda **crontab -l** afișăm configurarea curentă a utilizatorului, inițial vidă. Folosind comanda **crontab -e** ni se prezintă un ecran de editare în care adăugăm o linie de configurare precum cea de la linia `10`. Linia de configurare va instrui **cron** să ruleze scriptul `/home/student/uso-book/code/13-auto/systemd/dater` la fiecare 10 secunde. Dacă dorim să ștergem configurația **cron** a unui utilizator, folosim comanda **crontab -r**.

```
student@uso:~/uso-book/code/13-auto$ crontab -l
no crontab for student

student@uso:~/uso-book/code/13-auto$ crontab -e
[...]

student@uso:~/uso-book/code/13-auto$ crontab -l
[...]
# m h  dom mon dow   command
*/10 * * * * /home/student/uso-book/code/13-auto/systemd/dater
```

Listing 13.37: Configurarea cron la nivelul utilizatorului

13.5.2.2 at

Comanda **at** (care folosește în spate daemonul `atd`) permite rularea unei comenzi la un moment dat în viitor. Comanda primește ca argument timpul când să aibă loc execuția și la intrarea standard comanda care să fie executată, ca în Listing 13.38 (liniile `3` și `7`). Am instalat în prealabil pachetul **at**. Similar opțiunilor comenzii **crontab**, folosind opțiunea `-l` putem lista planificările curente, iar folosind opțiunea `-r` putem șterge planificări.

```
student@uso:~/uso-book/code/13-auto$ sudo apt install at

student@uso:~/uso-book/code/13-auto$ echo /home/student/uso-book/code/13-auto/systemd/dater | at midnight
warning: commands will be executed using /bin/sh
job 1 at Fri Jan  8 00:00:00 2021

student@uso:~/uso-book/code/13-auto$ echo /home/student/uso-book/code/13-auto/systemd/dater | at 2021-01-20
warning: commands will be executed using /bin/sh
job 2 at Wed Jan 20 06:18:00 2021

student@uso:~/uso-book/code/13-auto$ at -l
1       Fri Jan  8 00:00:00 2021 a student
2       Wed Jan 20 06:18:00 2021 a student

student@uso:~/uso-book/code/13-auto$ at -r 1
student@uso:~/uso-book/code/13-auto$ at -r 2
student@uso:~/uso-book/code/13-auto$ at -l
student@uso:~/uso-book/code/13-auto$
```

Listing 13.38: Planificarea unei acțiuni folosind at

13.6 Anexă: Utilitare pentru interacțiunea cu programe interactive

Atunci când dorim să automatizăm și să folosim în scripturi programe interactive, lucrurile devin problematice. În general, ne propunem ca un script să fie complet automatizat și neinteractiv. Dacă putem, căutăm variante neinteractive ale acelor programe. Altfel, unde se poate, folosim utilitarul **`expect`**.

Utilitarul **`expect`** permite transmiterea de informații în mod automat la intrarea standard a unor programe interactive, ca în scriptul `ftp.exp` din Listing 13.39. Acest script deschide o conexiune FTP folosind utilitarul **`nctftp`** și transmite parola și comenzi FTP (`dir` și `quit`) în mod neinteractiv folosind **`expect`**.

```
student@uso:~/uso-book/code/13-auto$ cat ftp.exp
#!/usr/bin/expect

set ftp_host repository.grid.pub.ro
set ftp_user clusterncituso
set ftp_password "XXXXXXXXXXXXXXXX"

spawn ncftp -u $ftp_user $ftp_host
expect "Password: "
send -- $ftp_password\r
expect ">"
send -- "dir\r"
expect ">"
send -- "quit\r"
expect eof

student@uso:~/uso-book/code/13-auto$ ./ftp.exp
spawn ncftp -u clusterncituso repository.grid.pub.ro
```

```
[...]
Password requested by 141.85.241.222 for user "clusterncituso".

    User clusterncituso OK. Password required

Password: *********

OK. Current restricted directory is /
Logged in to repository.grid.pub.ro.
ncftp / > dir
drwxr-xr-x                Aug 31  2018   2015-2016
drwxr-xr-x                Aug 31  2018   2016-2017
drwxr-xr-x                Oct 13  2017   2017-2018
drwxr-xr-x                Nov  4  2018   2018-2019
drwxr-xr-x                Sep 26 13:55   2019-2020
drwxr-xr-x                Feb 22  2020   experimental
drwxr-xr-x                Jan  4 22:45   old
-rw-r-----   5239211520   Jan  7  2019   USO-CTF.ova
-rw-r-----          124   Jan  7  2019   USO-CTF.ova.sums
-rw-r-----   1090779648   Nov 22  2015   USO Demo.ova
-rw-r-----   4818603008   Oct 27 00:04   USO.ova
-rw-rw-r--           74   Oct 26 23:36   USO.ova.sums
-rw-r-----   1607888384   Oct 29 21:14   USOTeme.ova
-rw-rw-r--           78   Oct 29 21:10   USOTeme.ova.sums
-rw-r-----   3370503680   Aug 31  2018   USO_tom_jerry.ova
-rw-r-----          136   Aug 31  2018   USO_tom_jerry.ova.sums
ncftp / > quit


        Thank you for using NcFTP Client.
        Ask your system administrator to try NcFTPd Server!
        http://www.ncftp.com/ncftpd/
```

Listing 13.39: Folosire expect pentru programe interactive

Comenzile **expect** creează un proces (**spawn**) căruia apoi îi pot controla intrarea standard și ieșirea standard, putând astfel realiza o comunicare neinteractivă. Îi sunt transmise comenzi intrării standard a procesului interactiv folosind comanda **send**. Așa trimitem parola și comenzile FTP (`dir` și `quit`). Ieșirea procesului este capturată și analizată folosind comanda **expect**. Cu ajutorul comenzii **expect** putem realiza acțiuni în funcție de ieșirea procesului; de exemplu, să știm ce trebuie să afișeze / transmită procesul pentru ca noi să-i transmitem o intrare specifică.

Pentru o variantă programatică a utilitarului **expect**, putem folosi **pexpect**[1]. **pexpect** oferă o interfață programatică Python în care pot fi operate funcționalitățile expect.

O variantă similară **pexpect**, folosită în comunitatea de securitate (hacking, competiții de tip CTF - *Capture the Flag*) este **pwntools**[2]. Pe lângă modul de comunicare neinteractivă cu aplicații interactive, **pwntools** oferă funcționalități specifice pentru securitate și hacking: analiză de executabile, construire de payloaduri pentru atacuri de securitate, formatări de date, integrare cu depanatoare etc.

[1] https://pexpect.readthedocs.io/en/stable/
[2] http://docs.pwntools.com/en/stable/

13.7 Anexă: Gestiunea configurației

Un administrator de sistem și infrastructură care gestionează mai multe sisteme urmărește un mod rapid, automatizat de configurare a acelor sisteme. În general acest lucru înseamnă ca un set de configurații de pe un sistem central să ajungă pe sistemele administrate în mod rapid și scalabil. De obicei, astfel de soluții oferă administratorului o interfață programatică, pentru a configura ce informații de configurare ajung pe sisteme (numite uzual noduri).

Soluții de gestiune a configurației (*Configuration Management*) sunt Chef, Puppet, Ansible, SaltStack, Terraform.

Astfel de soluții sunt folosite de cei care ocupă poziții de *DevOps* (*Development and Operations*): persoane responsabile cu buna funcționare a infrastructurii IT a unei organizații (*Operations*) care folosesc facilități programatice (*Development*) furnizate de utilitare precum cele din categoria *Configuration Management*.

13.8 Anexă: Bune practici în automatizare și scripting

Atunci când folosim scripting sau, în general, când automatizăm, e util să avem în vedere bune practici și, mai ales, perspective de lucru. Aceste bune practici asigură o rulare rapidă, automată și robustă de sarcini repetitive astfel încât să obținem mai mult pentru activități creative.

Pentru persoana care lucrează cu un sistem de operare, perspectiva trebuie să fie „Orice fac poate fi automatizat. Are sens să automatizez?” Dacă o acțiune se întâmplă frecvent, acea acțiune este candidat la automatizare pentru a eficientiza realizarea acțiunii: timp mai scurt pentru acțiune repetitivă, timp mai mult pentru alte acțiuni.

Atunci când automatizăm o acțiune, avem în vedere că sunt mai multe moduri de realizare a acesteia. Nu există un mod unic și recomandat de rezolvare. Pentru cazuri simple folosim comenzi simple, pentru cazuri complexe folosim utilitare avansate. Este nevoie de o cunoaștere bună a comenzilor, utilitarelor și facilităților oferite pentru a putea face alegerea optimă în situația respectivă.

Pentru o situație dată, când avem mai multe moduri de a realiza un lucru, urmărim criterii precum: robustețe, lizibilitate, eficiență, portabilitate. Scriptul sau utilitarul folosit trebuie să funcționeze pe cât mai multe sisteme, în cât mai multe situații, cât mai performant și să fie cât mai ușor de înțeles.

În Linux, formatul uzual de lucru este formatul text. Majoritatea utilitarelor citesc text și afișează text, ceea ce le face ușor de folosit de un utilizator uman; și ușor de combinat folosind operatorul *pipe* (`|`). Atunci când folosim scripturi, ne folosim de filtre de text care prelucrează text. Iar dacă dezvoltăm noi utilitare, urmărim ca intrarea, ieșirea și configurarea lor să fie făcute tot folosind format text.

Multe utilitare fac un tip de acțiune. Acea acțiune înseamnă de multe ori prelucrarea unei intrări și afișarea unui rezultate. Astfel de utilitare pot fi combinate prin operatorul pipe sau prin funcționalități precum expandarea comenzii. Când automatizăm în

scripturi shell, e recomandat să avem perspectiva combinării de comenzi care pot duce la rezultatul dorit.

Nu folosim shell scripting ca alternativă la un limbaj de programare. Shell scripting este o formă de automatizare, de combinare rapidă de comenzi pentru a obține un rezultat cât mai rapid. Pentru calcule de înaltă performanță, pentru aplicații complexe, pentru eficiență în folosirea resurselor, folosim un limbaj de programare.

13.9 Sumar

Folosim automatizare pentru sarcini repetitive sau care necesită rulare neinteractivă la un moment dat, lăsând mai mult timp pentru sarcini creative.

Avem mai multe limbaje pentru scripting, cu scenariile avantajoase fiecăruia.

Preferăm folosirea shell scripting pentru acțiuni de tipul *quick and dirty*.

Scripturile shell combină comenzi existente, comenzi interne shellului și alte funcționalități ale shellului: înlănțuire comenzi, variabile, expandări, globbing, controlul fluxului.

Când creăm scripturi shell avem în vedere că sunt mai multe opțiuni posibile și le alegem urmărim robustețe, lizibilitate, eficiență, portabilitate.

Pentru gestiunea unui sistem sau infrastructuri avem utilitare dedicate pentru automatizare.

Capitolul 14

Mașini virtuale

În operațiunile de zi cu zi, deseori avem nevoie să rulăm mai multe tipuri de sisteme de operare (ex. Linux și Windows) și avem la dispoziție un singur sistem de calcul. Rularea mai multor tipuri de sisteme de operare este utilă atunci când unele aplicații funcționează doar pe unul sau pe altul.

Pentru a rula mai multe sisteme de operare pe același sistem de calcul, trebuie să instalăm fiecare sistem de operare pe câte partiție într-o configurație dual-boot (vezi Capitolul 9), și când dorim să comutăm de pe unul pe altul, trebuie să resetăm calculatorul și să alegem sistemul de operare ce va porni. Această operațiune este consumatoare de timp, iar tot lucrul efectuat până în acel moment trebuie salvat. Pentru a preîntâmpina necesitatea folosirii unei configurații dual-boot, se poate folosi tehnologia de virtualizare, principală în partea de IT&C în zilele noastre. Prin intermediul virtualizării se creează o abstractizare a hardware-ului, dând posibilitatea mai multor sisteme de operare să ruleze în același timp, pe același sistem hardware. Abstractizarea hardware-ului se referă la crearea unei instanțe virtuale (ea nu există fizic) a fiecărei componente centrale a unui sistem de calcul (procesor, memorie, disc) pentru fiecare sistem de operare care rulează.

În următoarele secțiuni vom extinde modul în care se realizează virtualizarea, diferite tipuri de virtualizare, precum și tehnologiile care implementează virtualizarea.

14.1 Concepte de virtualizare

Rularea mai multor sisteme de operare diferite pe același sistem de calcul se justifică atât din punctul de vedere al funcționalității, cât și al securității. Funcționalitatea este deseori dată de faptul că aplicațiile nu sunt compatibile cu sistemul de operare actual. Avem nevoie de o versiune de sistem de operare mai vechi (de ex. unele aplicații scrise în Java funcționează doar pe Windows 7 cu o versiune veche de Java) sau de alt tip de sistem de operare (de ex. jocurile au fost scrise în general doar pentru sistemul de operare Windows, iar noi avem instalat pe calculator Linux).

O altă folosință a virtualizării este proeminentă în domeniul securității. În ziua de astăzi, programele malițioase sunt distribuite prin diverse canale, deseori împreună cu aplicații ce sunt legitime. Dacă aveți o astfel bănuială, acele aplicații pot fi rulate pe un sistem de

operare diferit de sistemul ce rulează direct pe sistemul de calcul. Daunele provocate de programele malițioase vor afecta doar sistemul de operare respectiv, nu și sistemul de operare principal. Se observă, astfel, că există nenumărate aplicații ale rulării mai multor sisteme de operare în același timp, pe același sistem de calcul.

În nomenclatura virtualizării, avem următoarele cuvinte cheie:

- *sistem gazdă* sau *host* sau *mașină fizică*: cel care a fost instalat prima dată și deasupra căruia rulăm un alt sistem de operare.
- *sistem oaspete* sau *guest* sau *mașină virtuală*: sistem de operare ce îl rulăm în cadrul sistemului gazdă.

Necesitățile prezentate anterior pentru rularea concomitentă a mai multor sisteme de operare pe același sistem de calcul au existat încă de la începutul unităților de calcul, dar resursele limitate nu au permis acest lucru (dacă ar fi fost implementată, sistemul de operare ce rula pe frameworkul de virtualizare ar fi fost imposibil de folosit).

Un prim pas spre rularea unui sistem de operare într-un altul (conceptul de virtualizare) a fost făcut în anii 2000 atunci când puterea de calcul a procesoarelor a crescut făcând posibil acest lucru. Una dintre primele implementări a fost dezvoltată de compania VMware, produsul denumit Workstation. *VMware Workstation* oferea posibilitatea rulării unui alt tip de sistem de operare alături de cel de bază (de exemplu rularea unei distribuții Linux pe un calculator ce are instalat Windows). Pentru a oferi izolarea corespunzătoare, acesta rula ca un nou proces în sistemul de operare gazdă (sistemul fizic). Astfel, va fi izolat de celelalte procese sau sisteme de operare gazdă care ar putea rula.

Problema cu abordarea VMware Workstation o constituie accesul la resursele hardware (instrucțiunile privilegiate): acest lucru se poate întâmpla doar din cadrul nucleului sistemului de operare gazdă, nu și din cadrul procesului / aplicației. Pentru a rezolva acest lucru, s-a creat un modul de kernel care este inserat / instalat împreună cu aplicația VMware Workstation. La fiecare acces la instrucțiunile privilegiate ale sistemului oaspete, modulul de kernel va trata excepțiile și le va executa în numele sistemului gazdă.

Intermedierea oferită de sistemul de operare pentru rularea instrucțiunilor privilegiate determină o viteză scăzută a mașinii virtuale, mai ales când aceasta execută multe astfel de instrucțiuni. Pentru a veni în întâmpinarea acestui neajuns, producătorii de hardware (procesoare) au introdus o nouă facilitate numită virtualizare nativă (în cazul procesoarelor Intel, acesta se numește VT-X - *Virtualization Extensions*, iar în cazul procesoarelor AMD, poartă numele de SVM - *Secure Virtual Machine*). Acest lucru a permis implementarea unor module software suplimentare pentru a diminua overheadul indus de excepțiile cauzate de execuția instrucțiunilor privilegiate. De aceea, în zilele noastre, atunci când porniți o aplicație de virtualizare (fie că e vorba de VMware, fie că e vorba de VirtualBox), primiți o notificare în cazul în care nu aveți activate din BIOS aceste extensii de virtualizare (vedeți Capitolul 9 pentru detalii despre configurarea BIOS-ului). Majoritatea versiunilor aplicațiilor de virtualizare nu mai oferă suport de rulare fără prezența acestor extensii.

Virtualizarea nu este folosită doar pe calculatoarele personale pentru a acoperi diverse deficiențe de securitate sau incompatibilități între diverse versiuni de software și sisteme de operare. Virtualizarea este folosită și în aria serverelor pentru a rezolva

câteva probleme importante:

- **consolidarea resurselor**: de multe ori un server este alocat doar unei aplicații, iar această aplicație nu folosește întreaga capacitate a serverului. Astfel, resursele rămân nefolosite. Rularea unei alte aplicații pe același server este considerată de multe ori o problemă de securitate (dacă una din aplicații e compromisă, o afectează inevitabil și pe cealaltă). Folosind conceptul de virtualizare împreună cu extensiile hardware de virtualizare, putem rula mai multe mașini virtuale, câte una pentru fiecare aplicație dorită.
- **securitate**: izolarea fiecărei aplicații într-o mașină virtuală pentru a preveni furtul de date de la una la alta.
- **mentenanță**: fără a întrerupe funcționarea acestora, mașinile virtuale pot fi migrate (mutate) de pe o mașină fizică pe alta. Astfel, se pot aplica actualizările necesare pentru sistemul fizic (e.g. *Cluster Aware Updating* de la Microsoft) sau se poate face mentenanța hardware dorită (ex. upgrade de memorie).

14.1.1 Clasificarea virtualizării

În terminologia virtualizării, toate operațiunile necesare acestui proces sunt administrate de o entitate denumită *hipervizor* (*hypervisor*). Hipervizorul este componenta din sistemul de operare ce se ocupă de virtualizare. O altă denumire des întâlnită a hipervizorului, mai ales în diagramele arhitecturale, este *Virtual Machine Monitor* (VMM); numele este intuitiv întrucât rolul unei hipervizor este acela de a monitoriza toate operațiile privilegiate ale unei mașini virtuale și de a le executa în numele acesteia. În continuare, vom realiza o clasificare a hipervizoarelor strâns legată de diferite tipuri de virtualizare.

De-a lungul timpului, procedeele de virtualizare au evoluat, existând în ziua de astăzi două tipuri de virtualizare:

- **hosted** - virtualizarea se realizează în cadrul unui sistem de operare existent. În cadrul acestui tip de virtualizare, hipervizorul se mai numește și *Type-2 hypervisor* (hipervizor de tipul 2). Este tipul de virtualizare folosit de VirtualBox, VMware Workstation.
- **baremetal** - este dezvoltat un nou sistem de operare, numai pentru a realiza virtualizarea, care rulează deasupra hardware-ului. În cadrul acestui tip de virtualizare, hipervizorul se mai numește și *Type-1 hypervisor* (hipervizor de tipul 1). Este tipul de virtualizare realizat de VMware ESX, Xen.

În Figura 14.1 sunt reprezentate cele două tipuri de virtualizare, iar în Figura 14.2 sunt reprezentate cele două tipuri de hipervizoare (se observă echivalența).

În partea stângă din Figura 14.1 și, respectiv, din Figura 14.2, este reprezentată virtualizarea de tip hosted în care avem sistemul de operare ce rulează deasupra hardware-ului, iar nivelul de virtualizare a fost dezvoltat deasupra acestuia, având un hipervizor de tipul 2 (Type-2 Hypervisor). De asemenea, în paralel cu mașinile virtuale pot rula și alte aplicații obișnuite. Acest tip de virtualizare este deseori folosit în sistemele desktop.

În partea dreaptă din Figura 14.1 și, respectiv, din Figura 14.2, este reprezentată o arhitectură folosind virtualizarea de tip baremetal: codul de virtualizare rulează deasupra hardware-ului, având un hipervizor de tipul 1 (Type-1 Hypervisor). Acest tip de virtualizare este folosit de obicei în servere (unde nu avem aplicații ce trebuie să ruleze în paralel) și este apreciat pentru faptul că hipervizorul are un code-base (numărul de linii de cod) mult mai mic decât al unui sistem de operare și are riscuri mai mici de securitate.

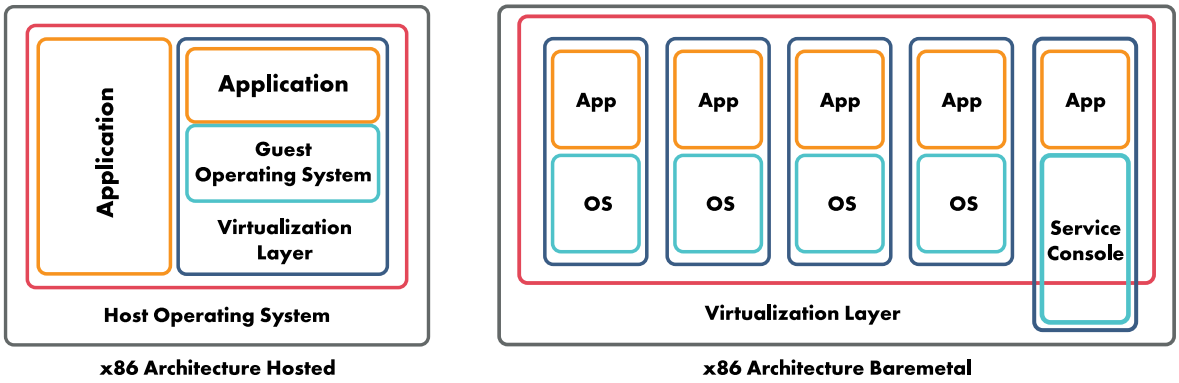


Figura 14.1: Virtualizare de tip hosted vs. baremetal

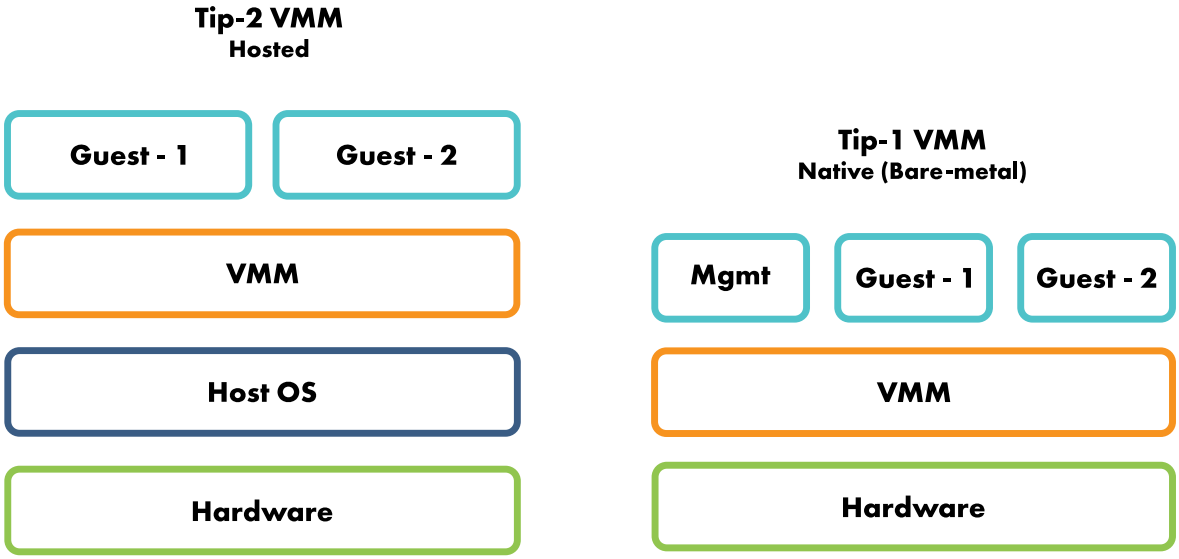


Figura 14.2: Hipervizor tip 2 vs. tip 1

În ambele figuri, se observă că pentru virtualizarea baremetal și hipervizorul de tip 1 există o mașină virtuală de management. Aceasta este necesară pentru a putea administra hipervizorul și pentru a implementa funcționalitățile non-critice din acesta. Faptul că rulează ca o mașină virtuală diferită este un lucru bun întrucât dacă există probleme în funcționarea acestuia, nu va afecta întregul nod (implicit și celelalte mașini fizice).

14.1.2 Aplicații/hipervizoare pentru rularea mașinilor virtuale

În continuare, vom prezenta cele mai importante (cu o cotă de utilizare semnificativă) hipervizoare din ziua de astăzi. Astfel, avem hipervizoare de tipul 2:

- *Virtual Box* - este un hipervizor de tipul 2 adresat sistemelor desktop. Acesta se instalează pe majoritatea sistemelor de operare (Windows, Linux sau MacOS). Poate rula orice sistem de operare gazdă, are opțiuni de snapshot (salvarea stării unei mașini virtuale), posibilități avansate de a controla configurațiile de rețea. Poate fi folosit în mod gratuit.

- *VMware Workstation/Player/Fusion* - este un hipervizor de tipul 2 adresat, de asemenea, sistemelor desktop. VMware Workstation se instalează pe sistemele Windows și Linux și oferă aceleași facilități ca și VirtualBox. VMware Fusion se adresează sistemelor de operare gazdă MacOS oferind aceleași funcționalități ca VMware Workstation. Ambele versiuni necesită achiziționarea unei licențe. Cu ajutorul versiunii VMware Player se pot rula mașinile virtuale create cu VMware Workstation, dar aceasta nu deține facilități avansate (ex. snapshot). VMware Player este distribuit gratuit.

Hipervizoarele de tip 1 sunt folosite și au aplicabilitate în domeniul centrelor de date (datacenter). Printre acestea putem enumera:

- *Xen* - dezvoltat de Universitatea din Cambridge, rulează direct peste hardware și are nevoie de o mașină virtuală pentru management și pentru a rula driverele virtualizate. Mașina de management poartă denumirea de `Dom0` (*Domain 0* - în terminologia Xen mașinile virtuale poartă denumirea de domenii). Xen oferă suport de virtualizare atât pentru procesoarele cu arhitectură x86, cât și ARM (vezi Capitolul 8).

- *VMware ESXi* - hipervizorul de tip 1 al celor de la VMware. Acesta se instalează direct peste hardware și oferă suport doar pentru procesoarele cu arhitectură x86. Varianta gratuită oferă un set limitat de funcționalități (e.g. nu oferă posibilitatea de a migra o mașină virtuală de pe un nod fizic pe altul). Pentru a beneficia de toate facilitățile, trebuie achiziționată varianta comercială denumită *VMware vShpere* împreună cu aplicația de management *VMware vCenter*.

Două soluții de virtualizare folosite frecvent în mediile Linux, respectiv, Windows, sunt *KVM* (Kernel Virtual Machine), respectiv, *Microsoft Hyper-V*. Modul în care sunt implementate, ca o componentă adăugată în nucleul sistemului de operare, le face să aibă caracteristici similare și cu hipervizoarele de tipul 1 (rulează direct peste hardware, ca parte din sistemul de operare nativ) și cu cele de tipul 2 (hipervizorul deține și celelalte caracteristici ale sistemului de operare).

KVM este implementat ca un modul de kernel în cadrul sistemelor Linux și folosește extensiile hardware necesare pentru a realiza virtualizarea. Există suport de virtualizarea KVM atât pentru procesoarele cu arhitectură x86 (Intel și AMD), cât și pentru procesoarele ARM, la fel ca în cazul Xen.

Microsoft Hyper-V este implementat ca un rol (*Role*) instalabil în cadrul distribuțiile Windows Server. Prima versiune la care a apărut este Windows Server 2008, fiind ulterior introdus în 2008R2, 2012, 2012R2, 2016 și, în acest moment, în 2019. Hyper-V este livrat în mod gratuit ca un sistem de operare de sine stătător (practic, este un sistem de operare Microsoft Windows fără interfață grafică și cu rolul Hyper-V instalat).

14.1.3 Containere (lightweight virtualization)

Virtualizarea aduce un overhead important. Pentru aplicațiile care au nevoie să ruleze doar într-un mediu izolat, dar să folosească același nucleu a fost introdus conceptul de containerizare sau *lightweight virtualization*.

Containerele sunt reprezentate printr-o nouă ierarhie de fișiere separată de cea a mașinii gazdă și care folosește nucleul mașinii gazdă pentru a efectua apeluri privilegiate către hardware și nu numai. Containerele, în comparație cu mașinile virtuale, sunt mai rapide, consumă mai puține resurse întrucât nu trebuie decât să pornească o nouă ierarhie de procese (practic, un nou proces *init*, precum și procesele daemon aferente - vezi Capitolul 4). Dezavantajul containerelor îl reprezintă faptul că nu putem rula sisteme de operare diferite (Windows în cadrul unei mașini fizice ce rulează Linux) întrucât este nevoie de același tip de nucleu.

În Figura 14.3 este reprezentată diferența între mașini virtuale și containere. Se observă că, în primul caz (mașini virtuale - stânga), avem un hipervizor deasupra căruia rulează mașinile virtuale cu nucleele aferente, iar în cel de-al doilea (containere - dreapta), avem un mecanism de containerizare (în exemplul din figură tehnologia se numește Docker) peste care rulează direct aplicațiile. Se observă că mecanismul de container nu conține propriul nucleu.

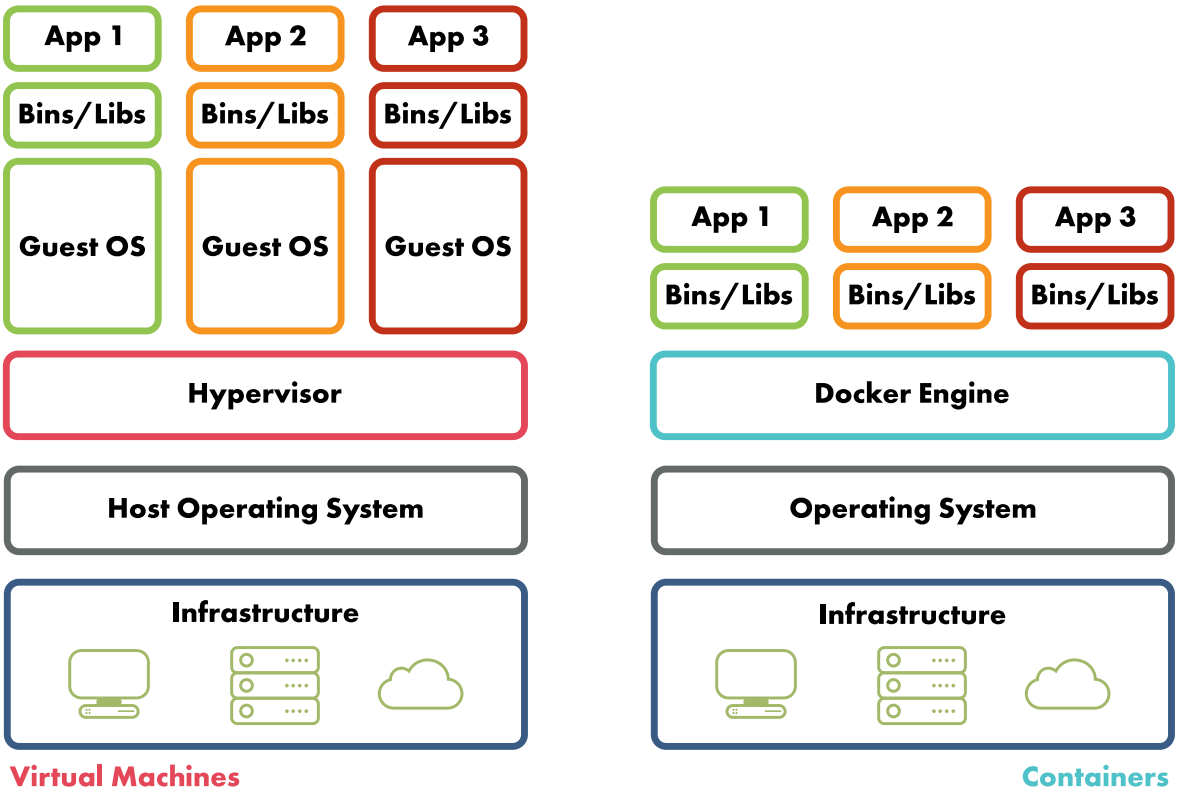


Figura 14.3: Mașini virtuale vs. containere

Tehnologii ce implementează mecanismul de container în sistemele Linux sunt:

- LXC (Linux Containers) - oferă posibilitatea rulării unor servicii intr-un mediu izolat de sistemul de bază.
- OpenVZ - similar LXC, dar nu este prezent în mod implicit în nucleul Linux.

- Docker - oferă posibilitatea rulării într-un container doar a unei singure aplicații.

14.2 Operații cu mașini virtuale

Crearea mașinii virtuale se face de obicei printr-o comandă CLI sau folosind interfața GUI a soluției de virtualizare aleasă. La creare se precizează mai mulți parametri care să descrie configurația virtualizată a hardware-ului acesteia:

- numele mașinii virtuale;
- numărul de procesoare (nuclee/core-uri);
- cantitatea de memorie care va fi disponibilă;
- mărimea discului;
- dacă va avea CD-ROM și ce imagine .iso va fi asociată (întocmai unui CD/DVD fizic);
- tipul de rețea (detaliat în Secțiunea 14.2.1).

O dată creată, mașina virtuală poate fi pornită. În momentul primei porniri a mașinii virtuale, după încheierea procesului de creare, aceasta nu are un sistem de operare instalat. Dacă a fost configurat și un CD-ROM cu imaginea unui sistem de operare atașată, se poate realiza instalarea sistemului de operare. De obicei, soluția de virtualizare oferă un ecran direct la mașina virtuală prin care puteți interacționa cu aceasta. Acest ecran poartă numele de consolă. După instalarea sistemului de operare, mașina virtuală poate fi folosită.

Dacă se dorește modificarea configurației hardware, trebuie oprită mașina virtuală, apoi modificate resursele alocate (număr procesoare, cantitate memorie, mărime disc, adăugare disc nou). Dacă a fost modificată mărimea discului, aceasta nu se va reflecta automat în sistemul de operare întrucât partiționarea a fost făcută folosind dimensiunea inițială. De asemenea, adăugarea unui nou disc implică partiționarea și formatarea acestuia, așa cum este prezentat în Capitolul 10. În general, modificarea numărului de procesoare, precum și a cantității de memorie se va reflecta automat în mașina virtuală după pornire.

O altă operație ce poate fi efectuată în decursul rulării unei mașini virtuale este realizarea unui snapshot. Snapshotul este operația prin care se salvează starea mașinii virtuale (atât a memorie, cât și a discului) cu scopul de a ne întoarce înapoi la aceasta în cazul în care se întâmplă ceva în funcționarea sistemului. Această funcție este asemănătoare comenzii *Hibernate* din Windows atunci când dorim să închidem calculatorul și să păstrăm toate aplicațiile deschise, dar oferă și posibilitatea refacerii unei mașini virtuale folosind snapshotul creat. Un caz aplicat al acestei facilități o reprezintă chiar partea didactică: pregătim o mașina virtuală cu o configurație dată, creăm un snapshot și dăm acces de administrare studenților. Aceștia pot să execute și să testeze absolut orice comandă, iar dacă se corupe sistemul de operare sau mediul de lucru, se pot întoarce la snapshotul creat.

14.2.1 Configurarea rețelei virtuale

Ca orice sistem fizic, o mașină virtuală necesită o conexiune la rețea, respectiv la Internet (pentru instalare de aplicații, actualizări, comunicare cu alte stații din rețeaua locală). Pentru acest lucru, orice soluție de virtualizare oferă opțiunea de a adăuga o placă de rețea virtuală. Acestă placă de rețea face în general legătura între mașina virtuală și mașina fizică (este, practic, un fir logic între acestea două). Mașina fizică trebuie să ofere o modalitate de conectare a firului logic la rețeaua fizică, precum și alocarea unei adrese IP, așa cum am precizat în Capitolul 11. Soluțiile de virtualizare oferă, în general, trei tipuri de configurații (sau moduri) ale plăcii de rețea virtuale, ilustrate și în Figura 14.4:

- *NAT* (*Network Address Translation*);
- *host-only*;
- *bridge*.

Modul de funcționare NAT alocă o adresă IP automată (prin DHCP) mașinii virtuale și, în același timp, realizează și funcția de NAT pentru a acorda acces la Internet mașinii virtuale. După cum se poate vedea și în Figura 14.4, traficul care iese în afara rețelei sau cel care intră este colorat cu roșu, în comparație cu cel din interior: acest lucru arată că la ieșire se face o translatare de adrese (nu se păstrează adresa sursă a mașinii virtuale) folosind mecanismul NAT.

Modul de funcționare *host-only* asigură o adresă IP prin intermediului serviciului DHCP, dar comunicația este limitată doar între mașina fizică (*host*) și mașina virtuală după cum se poate vedea în Figura 14.4 (se observă că traficul nu va ieși în afara sistemului).

Modul de funcționare bridge unifică rețeaua externă cu placa de rețea a mașinii virtuale. Adresarea IP este asigurată de rețeaua externă. Mașina virtuală este văzută ca o altă stație normală în cadrul rețelei externe și poate fi accesată în mod direct. Se observă în Figura 14.4 desenat cu roșu faptul că pachetele nu sunt modificate.

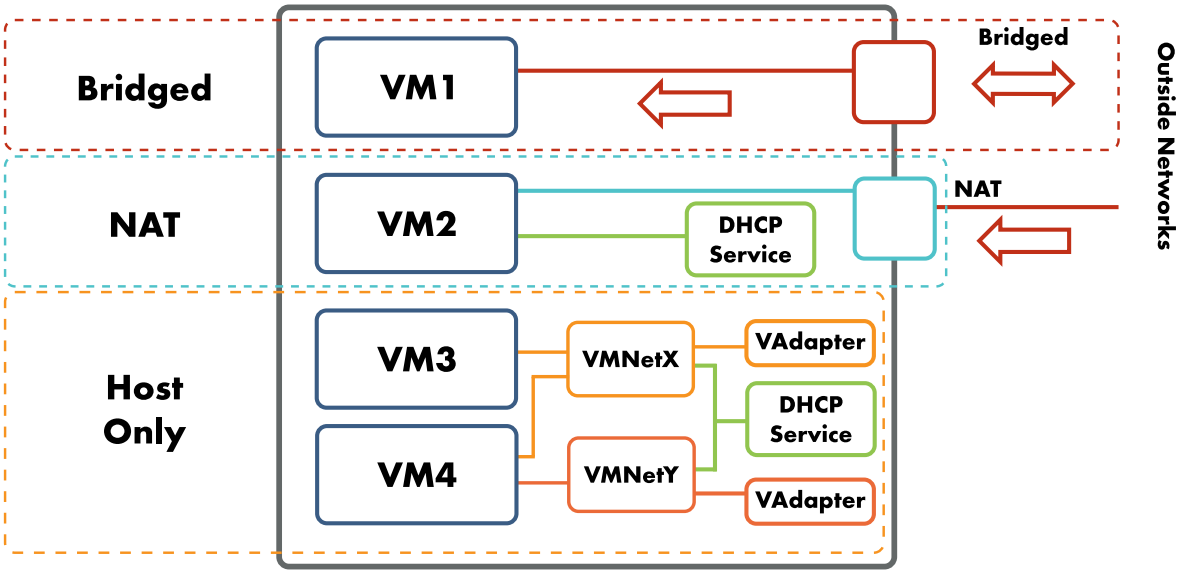


Figura 14.4: Moduri de funcționare a plăcii de rețea virtualizate

14.2.2 Servicii de integrare

În mod normal, sistemul de operare al mașinii virtuale nu este conștient de faptul că rulează într-un mediu virtualizat. Acest lucru limitează anumite funcționalități și aduce penalități de performanță. Un exemplu îl constituie facilitatea de copy-paste din host (mașina fizică) în ecranul mașinii virtuale. În mod normal acest lucru nu este posibil întrucât mașina virtuală nu are la cunoștință faptul că rulează într-un mediu virtualizat.

Pentru a rezolva această problemă, soluțiile de virtualizare oferă aplicații specializate (agenți software) care trebuie instalate în cadrul mașinii virtuale pentru ca aceasta să învețe faptul că rulează într-un mediu virtualizat și să comunice cu hipervizorul cu scopul de a aduce noi facilități de utilizare și performanță crescută. Aceste aplicații software sau agenți poartă numele de servicii de integrare (*integration services*). Serviciile de integrare sunt specifice fiecărei soluții de virtualizare și sunt oferite, în general, în mod gratuit. Puteți observa un exemplu de instalare a serviciile de integrare pe Virtual Box în Secțiunea 14.5.

14.3 Virtualizarea și tehnologiile cloud

În comparație cu aplicațiile de virtualizare pentru calculatoarele personale, instalarea și configurarea serviciilor de virtualizare pe servere este un proces complicat, necesită cunoștințe tehnice avansate atât de sisteme de operare, cât și de calcul distribuit și rețelistică. Chiar și după procesul de configurare a mediului virtualizat, crearea, instalarea și pornirea unei mașini virtuale pe un sistem server este anevoioasă. Pentru a rezolva aceste lucruri și a facilita un management ușor de înțeles pentru utilizatori, au fost dezvoltate frameworkuri și aplicații intermediare. Acestea poartă denumirea de *tehnologii cloud*.

Cloudul reprezintă o abstractizare a resurselor pentru utilizatorul final: acesta nu are nevoie să știe unde se află serverele, ce tipuri de servere sunt, dacă mai e loc pe ele sau nu, ș.a.m.d. Utilizatorul doar solicită resursele (procesor, memorie, disc, adresă IP), iar acestea sunt automat alocate. Un alt avantaj al tehnologiilor *cloud* îl reprezintă elasticitatea: dacă la un moment dat un utilizator dorește mai multe resurse (mai multe core-uri, mai multă memorie), poate solicita acest lucru furnizorului și va plăti doar pentru timpul în care le folosește efectiv.

Există două tipuri de servicii cloud:

- **private** (numite și *on-premise*): o organizație deține multe servere pe care dorește să ruleze mașini virtuale. Pentru a facilita managementul acestora va instala o soluție de cloud și o va configura să ruleze pe acele servere. Printre soluțiile de cloud private sunt OpenStack și OpenNebula. În general, soluțiile de cloud privat pot interacționa cu oricare tip de hipervizor prezentat anterior (KVM, Xen, Hyper-V).
- **publice**: organizațiile care nu dețin suficient hardware și nu vor să cheltuie bani pe achiziția și mentenanța acestuia, pot apela la serviciile cloud publice. Companii specializate care oferă sevicii publice de cloud sunt Amazon AWS-EC2[1], Google

[1] https://aws.amazon.com/ec2/

Cloud[1], Oracle OCI[2], DigitalOcean[3]. Acestea oferă posibilitatea creării unor mașini virtuale, rulând sistemul de operare dorit și având alocate resursele dorite.

Cloudul nu oferă doar mașini virtuale, ci oferă și servicii specifice: servicii de web (Amazon Web Services), servicii de stocare (Dropbox, Google Drive), servicii de computing (Google Compute Engine). Astfel, se oferă o interfață către utilizator care oferă doar serviciul dorit. Acest lucru are avantajul de a scuti utilizatorul de configurarea mașinii virtuale pentru ce ar avea nevoie (stocare, calcul).

Pornind de la cele enumerate mai sus, serviciile de cloud poate fi clasificate în:

- IaaS (*infrastructure as a service*) - oferirea atât de mașini virtuale utilizatorilor, precum și a unui mijloc prin care pot să își gestioneze singuri infrastructura de cloud pusă la dispoziție. Ca exemplu, aici intră operatorii de public cloud prezentați anterior (AWS-EC2, OCI, DigitalOcean).
- PaaS (*platform as a service*) - servicii ce pun la dispoziție un framework pentru dezvoltarea aplicațiilor (tool-uri de dezvoltare) fără a mai fi nevoie să realizăm instalarea acestora local. Un astfel de serviciu este Google App Engine[4], cu ajutorul căruia dezvoltatorii pot crea aplicații web în limbaje de programare precum PHP, Python, Ruby, JavaScript, Go.
- SaaS (*software as a service*) - oferirea de servicii/aplicații care sunt găzduite în medii cloud și utilizatorul nu trebuie să știe detalii despre platformă, sisteme de operare, mod de stocare, etc. Un astfel de serviciu este Overleaf[5], un serviciu pentru scrierea de documente în LaTeXîn mod colaborativ.

Secțiunea 14.6 va descrie pașii pe care un utilizator trebuie să îi urmeze pentru a-și crea un cont și a face operații cu mașini virtuale (creare, autentificare, ștergere) într-o infrastructură cloud: cea folosită curent de UPB, bazată pe OpenStack.

14.4 Emulare și virtualizare

Prin intermediul virtualizării, putem rula mai multe sisteme de operare pe aceeași mașină fizică. Sistemele de operare ale diferitelor mașini virtuale sunt proiectate să ruleze pe aceeași arhitectură de procesor (x86, ARM). Prin intermediul virtualizării nu putem rula pe același sistem fizic mai multe sisteme de operare concepute pentru diverse arhitecturi: nu putem rula un sistem de operare Android compilat pentru arhitectura ARM ca o mașină virtuală pe un laptop cu procesor x86. Acest lucru poate fi realizat prin emulare.

Emularea este procedeul prin care fiecare instrucțiune este tradusă (interpretată) de un software specializat denumit emulator. Instrucțiunile nu se execută în mod direct pe procesor; fiecare instrucțiune este interpretată de către emulator și emulatorul execută instrucțiunile necesare pentru a întreprinde efectul dorit. Emularea este un procedeu

[1] https://cloud.google.com/
[2] https://cloud.oracle.com/cloud-infrastructure
[3] https://www.digitalocean.com
[4] https://cloud.google.com/appengine
[5] https://www.overleaf.com/

mai lent ca virtualizarea. În cadrul virtualizării, sunt interpretate doar instrucțiunile privilegiate, restul sunt executate direct de către unitatea hardware.

QEMU[1] (*Quick EMUlator*) este una dintre cele mai cunoscute soluții de emulare pentru un număr variat de arhitecturi: x86, ARM, Sparc, PowerPC, ș.a. BOCHS[2] poate emula arhitectura x86 și este util în a face debugging în cadrul sistemelor de operare atunci când sunt portate pe arhitectura x86. Are suport inclusiv pentru extensiile de virtualizare (să le emuleze).

Wine[3] (*Wine Is Not an Emulator*) este un tip de emulator care permite rularea aplicațiilor Windows pe sistemele de operare Linux. După cum se poate vedea și în acronim acesta nu este un emulator în adevăratul sens al cuvântului. Wine pune la dispoziție o bibliotecă (`Winelib`) care să fie folosită în compilarea aplicațiilor Windows pentru a fi rulate pe Linux.

14.5 Anexă: Crearea unei mașini virtuale în VirtualBox

Operațiile cu mașinile virtuale se pot face ușor folosind aplicația de virtualizare VirtualBox[4]. VirtualBox asigură o interfață grafică utilizatorului pentru a crea, configura și șterge o mașină virtuală. VirtualBox se poate descărca și instala gratuit pe orice sistem de Operare (Windows, Linux, MacOS). Este întreținut și dezvoltat de Oracle.

În Figura 14.5 este reprezentată fereastra principală a aplicației VirtualBox. Se observă că există o mașină virtuală deja creată denumită Windows 7 care este închisă (*Powered Off*).

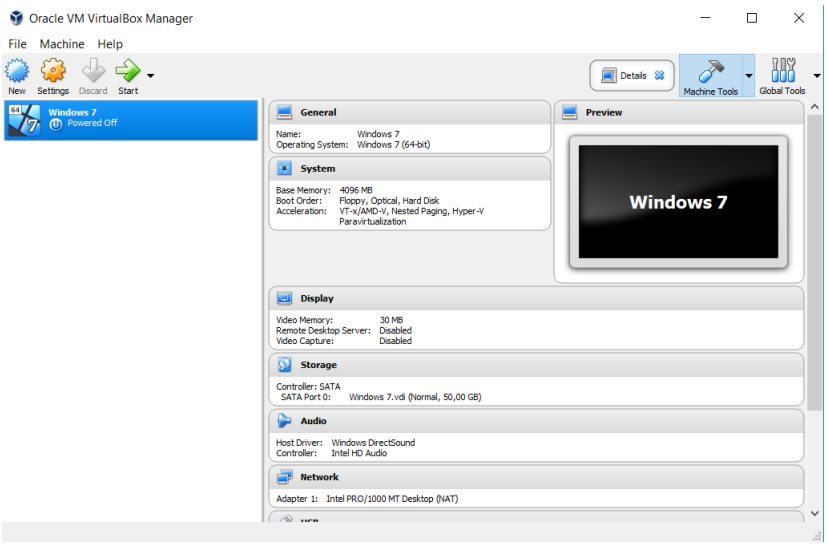


Figura 14.5: Fereastra principală VirtualBox

Pentru a crea o mașină virtuală nouă, mergeți pe meniul `Machine` → `New`. O nouă fereastră va apărea, fereastră în care trebuie să introduceți un nume pentru mașina

[1] https://www.qemu.org/
[2] http://bochs.sourceforge.net/
[3] https://www.winehq.org/
[4] https://www.virtualbox.org/

virtuală, precum și tipul sistemului de operare și varianta dorită. În Figura 14.6 este exemplificată crearea unei mașini virtuale ce va rula un sistem de operare Linux, baza pe distribuția Debian pe 64 de biți.

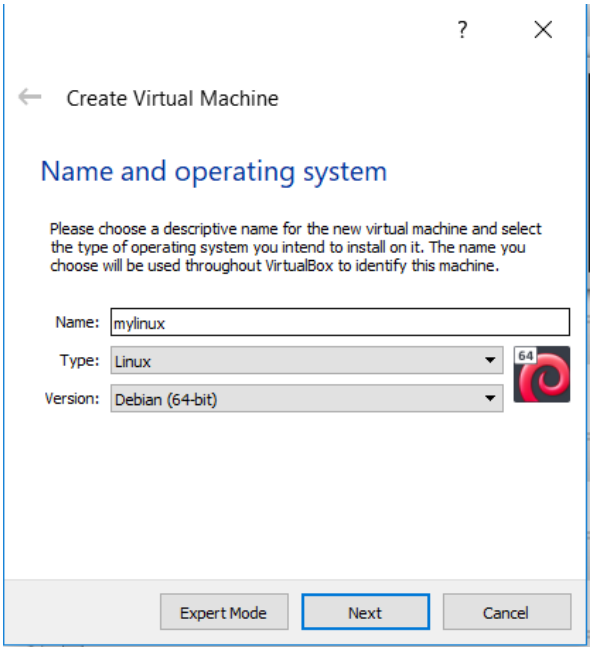


Figura 14.6: Meniul de creare a unei mașini virtuale în VirtualBox

După selectarea sistemului de operare, trebuie să specificați cantitatea de memorie la care va avea acces mașina virtuală (poate fi modificată și după creare), precum și dimensiunea discului unde se va instala sistemul de operare. Există trei tipuri de discuri: VDI (specific VirtualBox), VHDX (specific Hyper-V), VMDK (specific VMware). Recomandăm folosirea tipului de disc specific VirtualBox (VDI) întrucât aceasta va fi aplicația folosită. După selectarea tipului discului, avem două alte opțiuni:

- *fixed size* (dimensiune fixă): tot spațiul va fi alocat pe disc de la crearea mașinii virtuale. Acest mod oferă performanță mai bună, dar spațiul trebuie să fie disponibil de la creare.
- *dynamically allocated* (alocat dinamic): spațiul va fi alocat pe disc pe măsură ce mașina virtuală va scrie date. În acest fel putem crea discuri oricât de mari la început, fără să avem spațiu disponibil pe discul fizic.

În ultimă fază, trebuie să indicați dimensiunea pe care o doriți pentru disc.

Pașii pentru crearea unui disc sunt cei pe care i-am întâlnit și în Secțiunea 10.3.

După acest ecran, mașina virtuală va fi creată. Înainte de a o porni, trebuie să mai realizăm setări privind rețeaua. Folosiți click dreapta pe mașina virtuală creată și accesați, din meniul contextual deschis, opțiunea *Settings*. În ecranul nou deschis puteți configura fiecare componentă hardware virtualizată (vezi Figura 14.7).

Printre cele mai relevante opțiuni de configurare sunt:

- `System` - cantitatea de memoriei, numărul de procesoare.
- `Display` - cantitatea de memorie utilizată de placa video.

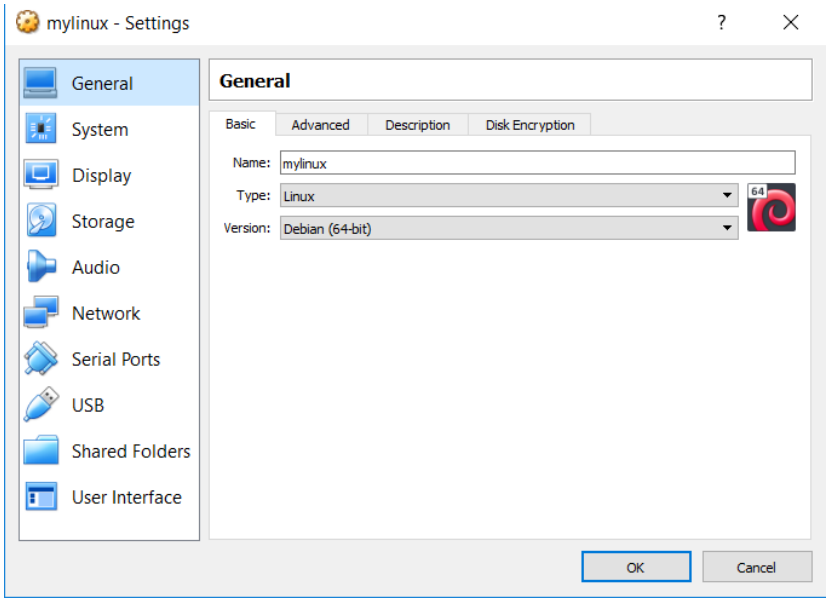


Figura 14.7: Configurarea unei mașini virtuale VirtualBox

- `Storage` - discurile atașate și proprietățile lor. Tot aici putem adăuga și un disc virtual necesar instalării mașinii virtuale. În Figura 14.8 este reprezentat modul prin care puteți selecta o imagine de tip ISO pentru a fi montată ca un disc virtual în mașina virtuală cu scopul de a instala sistemul de operare.
- `Audio` - activarea / dezactivarea plăcii audio pentru mașina virtuală.
- `Network` - configurarea plăcii / plăcilor de rețea pe care mașina virtuală le poate folosi. Implicit doar prima placă de rețea virtuală (*Adapter 1*) este activată în modul NAT (vedeți Secțiunea 14.2.1 pentru mai multe detalii despre modurile de funcționare). Recomandăm folosirea parametrilor impliciți.
- `USB` - configurarea dispozitivelor USB disponibile mașinii virtuale. Se poate oferi acces dispozitivelor USB hardware direct mașinii virtuale. Procesul se numește *USB passthrough*.

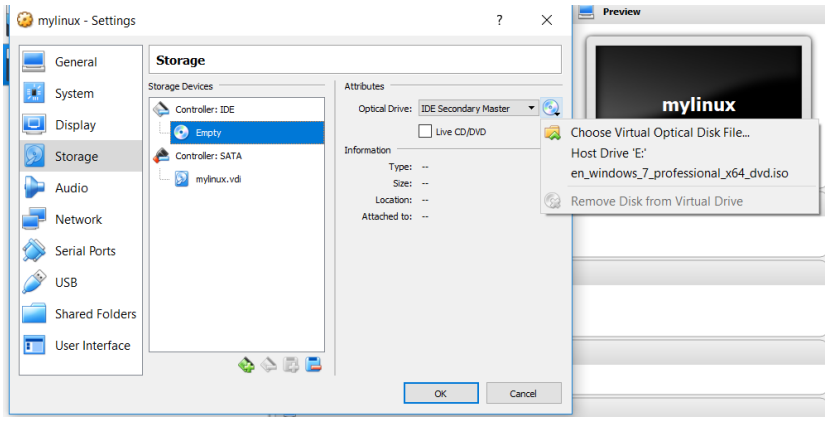


Figura 14.8: Introducere unei imagini ISO ca disc virtual într-o mașină virtuală

După configurarea mașinii virtuale, aceasta poate fi pornită. Selectați mașina virtuală și apăsați butonul `Start` din stânga sus. Aceasta va porni și va boota de pe discul virtual inserat. În Figura 14.9 este ilustrată pornirea mașinii virtuale și pornirea sistemului de

operare Windows în vedere instalării.

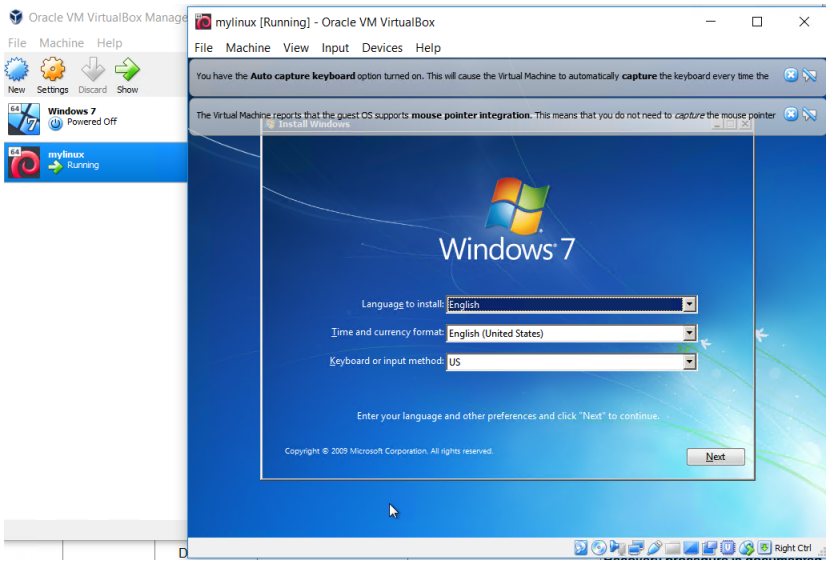


Figura 14.9: Instalarea unui sistem de operare în VirtualBox

După instalarea sistemului de operare, este important să instalați serviciile de integrare pe care le-am prezentat în Secțiunea 14.2.2. În cazul VirtualBox, acestea poartă numele de *VirtualBox Guest Additions*. Pentru a le instala, accesați meniul `Devices → Insert Guest Additions CD Image`. Această comandă va monta automat în mașina virtuală un disc virtual ce conține software-ul de instalat pentru serviciile de integrare. În Figura 14.10 se poate observa noul disc virtual și conținutul acestuia. De exemplu, după instalarea serviciilor de integrare, veți putea face copy/paste din/în mașina virtuală atât a fișierelor cât și a textului.

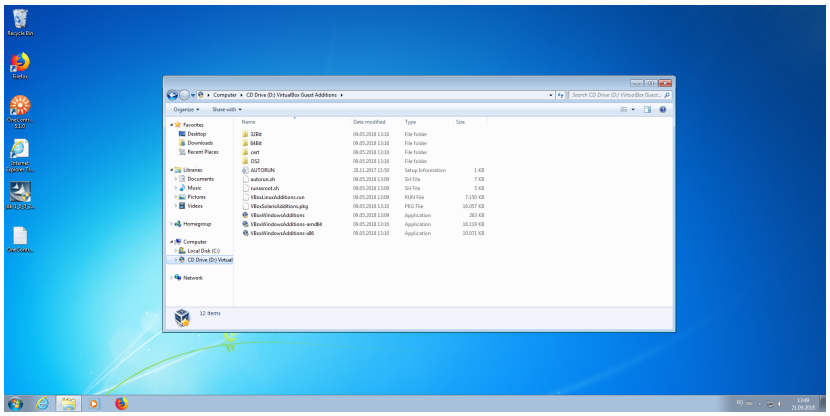


Figura 14.10: Guest Additions pentru mașini virtuale VirtualBox

O operație foarte utilă cu o mașină virtuală este cea de snapshot, prezentată în Secțiunea 14.2. Pentru a realiza un snapshot în VirtualBox, accesați meniul *Machine Tools* din dreapta sus și selectați butonul *Snapshot*, ca în Figura 14.11. Pentru a realiza un snapshot apăsați butonul *Take*.

Va apărea un meniu în care trebuie să introduceți numele snapshotului și o descriere relevantă. În Figura 14.12 se poate observa snapshotul creat. Pentru a ne întoarce la snapshotul dorit, folosim click dreapta și selectăm opțiunea *Restore*. Se poate observa după aceea că mașina a revenit la starea anterioară.

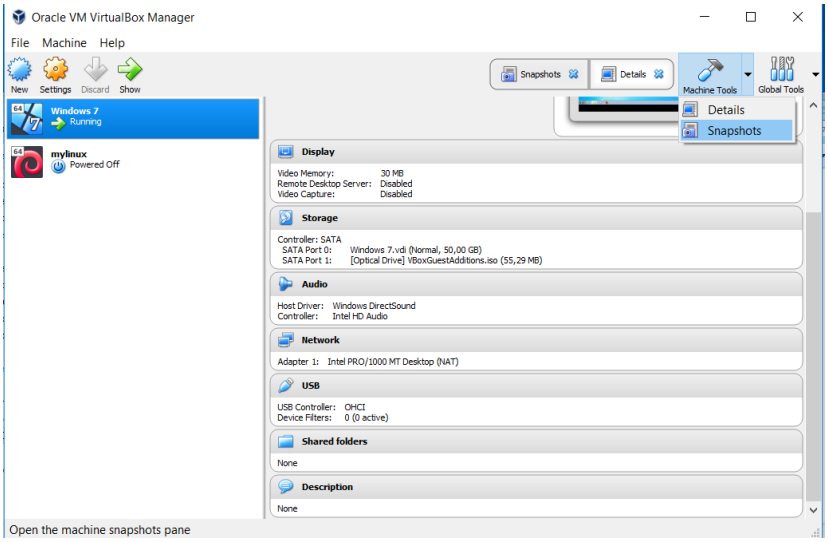


Figura 14.11: Snapshotul unei mașini virtuale în VirtualBox

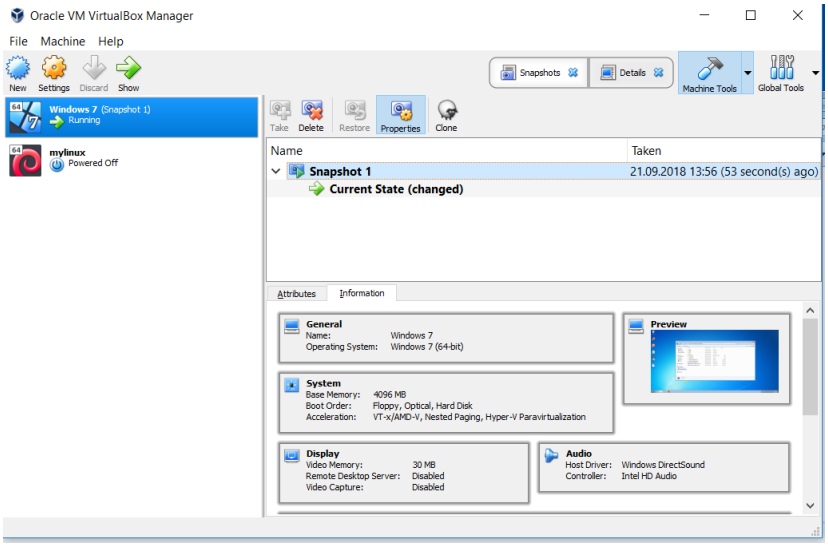


Figura 14.12: Snapshot creat în cadrul VirtualBox

14.6 Anexă: OpenStack în UPB

Tehnologia cloud folosită la nivelul Universității POLITEHNICA din București este OpenStack. Din perspectiva OpenStack, un student face parte dintr-un *tenant*. În acest context, noțiunea de tenant poate fi asociata cu un proiect. Crearea proiectului se face prin apăsarea butonului `Create user`, buton aflat în blocul OpenStack de pe platforma Moodle a Universității POLITEHNICA din București [1], ca în Figura 14.13. Odată creat tenantul, putem accesa `Dashboard` pentru crearea mașinilor virtuale.

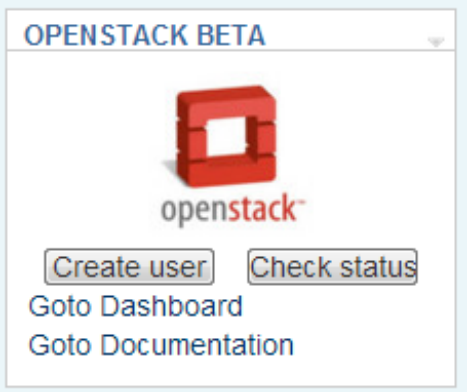


Figura 14.13: Crearea proiect OpenStack

14.6.1 Accesarea OpenStack Dashboard

Interfața de administrare a OpenStack (*OpenStack Dashboard*) este indicată în Figura 14.14 și se poate accesa folosind linkul https://cloud-controller.grid.pub.ro/. Autentificarea se face folosind utilizatorul și parola furnizate de universitate.

14.6.2 Crearea unei perechi de chei SSH

Autentificarea în cadrul unei mașini virtuale se face folosind chei SSH, așa cum am prezentat în Secțiunea 12.4.4.2. Acest mod de autentificare asigură cel mai înalt grad de securitate fiind folosit și de Amazon EC2 în mod implicit.

Mașinile virtuale pot fi accesate doar din cadrul unui server dedicat, cu numele `fep.grid.pub.ro`. Fiecare cont UPB are corespondent un nume de utilizator pe `fep.grid.pub.ro`. Pentru accesarea acestui cont și crearea perechii de chei, folosim comenzile din Listing 14.1. În cadrul comenzilor am folosit construcția `<upb_account>` ca substitut pentru numele contului UPB. Cheia publică este generată în fișierul `~/.ssh/openstack.pub` din directorul home al contului de utilizator de pe `fep.grid.pub.ro`.

```
1 student@uso:~$ ssh <upb_account>@fep.grid.pub.ro
2 [...]
3 <upb_account>@fep7-1:~$ ssh-keygen -f ~/.ssh/openstack
4 Generating public/private rsa key pair.
```

[1] https://curs.upb.ro/

Figura 14.14: Interfața de admininstrare OpenStack

```
5 [...]
6 <upb_account>@fep7-1:~$ ls ~/.ssh/
7 [...]  openstack  openstack.pub
```

Listing 14.1: Crearea unei perechi de chei SSH pe fep.grid.pub.ro

O dată generate cheile SSH, putem trece la pasul următor, de adăugare a cheii publice în configurația OpenStack.

14.6.3 Crearea unui keypair

Un keypair reprezintă o asociere între un nume și o cheie publică. În OpenStack, cheia publică se va identifica printr-un nume, specificat la creare. Cheia publică creată la pasul anterior trebuie încărcată în OpenStack. Acest lucru se face din OpenStack Dashboard, accesând `Project` → `Compute` → `Access & Security` → `Key Pairs` → `Import Key Pair`, ca în Figura 14.15.

14.6.4 Crearea unei mașini virtuale

Crearea unei mașini virtuale se poate face atât din OpenStack Dashboard cât și din linia de comandă. În acest tutorial va fi prezentată varianta folosind OpenStack Dashboard. Pentru a lansa o instanță, accesați, în OpenStack Dashboard, opțiunea `Project` → `Compute` → `Instances` → `Launch Instance` ca în Figura 14.16.

În Figura 14.16, am folosit template-ul `Ubuntu 14.04`. Un flavor reprezintă mărimea unei instanțe virtuale din punct de vedere al resurselor: număr de procesoare, memorie și spatiu pe disk.

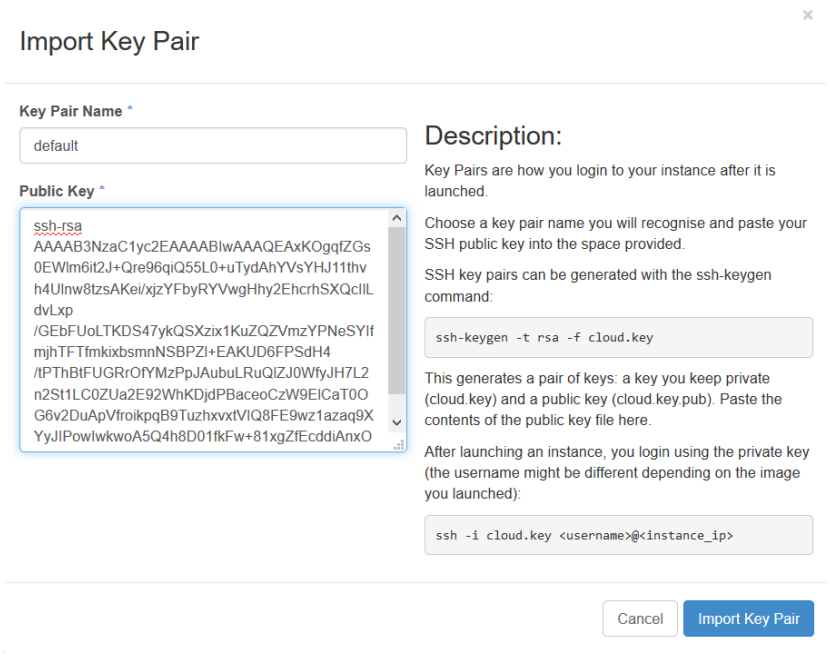


Figura 14.15: Importarea unei perechi de chei în OpenStack

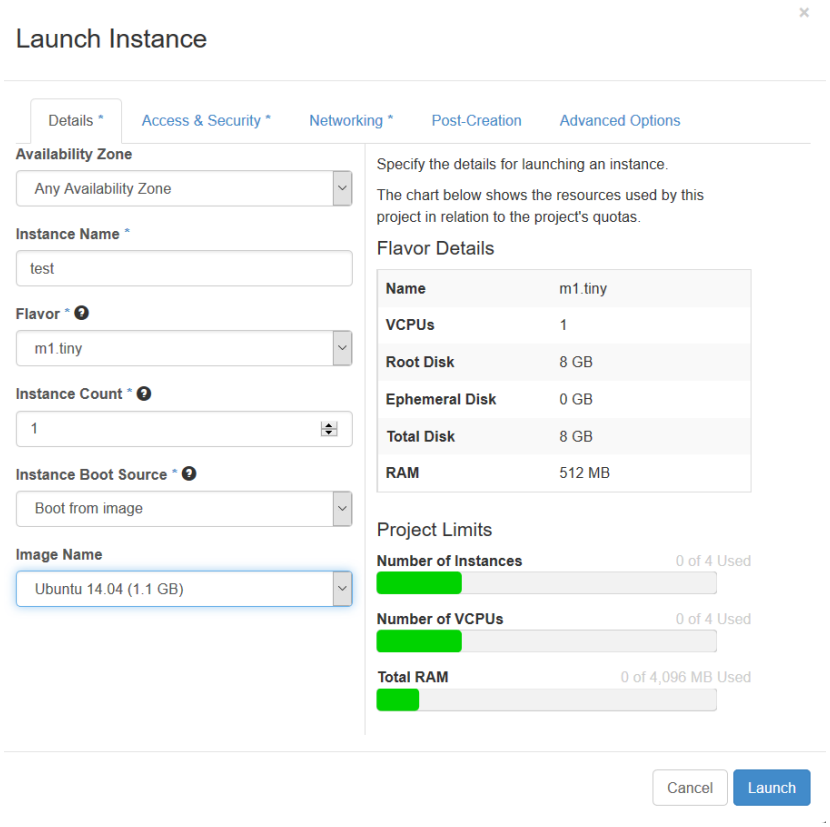


Figura 14.16: Crearea unei instanțe OpenStack

În final, din tabul `Access & Security`, alegeți keypairul creat anterior, ca în Figura 14.17. În final, apăsați pe `Launch` pentru a crea mașina virtuală.

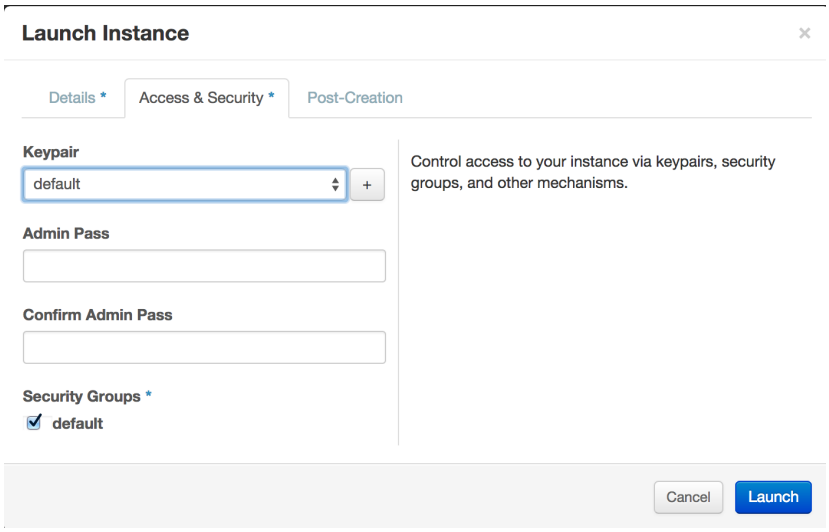


Figura 14.17: Alegerea unui keypair la crearea unei instanțe

14.6.5 Accesarea mașinii virtuale

Accesarea mașinii virtuale se face prin SSH de pe `fep.grid.pub.ro` ca în Listing 14.2. În loc de `<vm_ip_address>` folosim adresa IP a mașinii virtuale, adresă pe care o obținem din Dashboardul OpenStack, ca în Figura 14.18. Utilizatorul folosit la autentificare este `student`.[1]

```
1 student@uso:~$ ssh <upb_account>@fep.grid.pub.ro
2 [...]
3 <upb_account>@fep7-1:~$ ssh student@<vm_ip_address>
```

Listing 14.2: Accesarea mașinii virtuale prin intermediul fep.grid.pub.ro

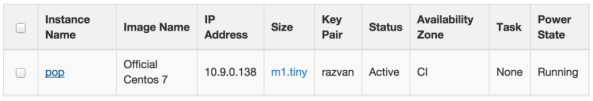


Figura 14.18: Informații despre mașina virtuală

14.6.6 Ștergerea mașinii virtuale

Odată ce ați finalizat procesul de folosire a mașinii virtuale, trebuie să o ștergeți. Acest lucru se face folosind butonul `Terminate Instance`, după ce ați selectat instanța pe care doriți să o ștergeți. În urma apăsării butonului `Terminate Instance`, toate datele stocate în acea instanță se vor pierde.

[1] Alte template-uri pot folosi alte nume de utilizator. Lista lor este la adresa https://cloud.curs.pub.ro/2014/12/17/username-uri-implicite/

14.7 Anexă: Rularea unui sistem de operare compilat pentru ARM pe x86

Pentru rularea unui sistem de operare compilat pentru arhitectura ARM pe un sistem de operare ce rulează pe o arhitectură x86, avem nevoie de un emulator. Vom folosi emulatorul QEMU ce are suportul de emulare pentru arhitectura ARM. Vom instala pachetele QEMU corespunzătoare.

```
student@uso~:$ sudo apt-get install qemu qemu-kvm qemu-system-arm
```

Pentru a rula un sistem de operare, avem nevoie de un nucleu (kernel) precum și de un binarele aferente (sistemul de fișiere). O imagine de kernel Linux pentru arhitectura ARM se poate descărca de la adresa http://uso.cs.pub.ro/virtualizare/kernel-qemu. Pentru sistemul de fișiere vom folosi o imagine de numită Raspbian (mașină virtuală de Debian care funcționează pe un sistem de fișiere tip Raspberry Pi), ce poate fi descărcată de la adresa http://uso.cs.pub.ro/virtualizare/2012-10-28-wheezy-raspbian.zip. După ce descărcați fișierul, îl dezarhivați pentru a obține sistemul de fișiere. Vom porni sistemul de operare folosind următoarea comandă:

```
student@uso~:$ qemu-system-arm -kernel kernel-qemu -cpu arm1176 -m 256 -M
 versatilepb -no-reboot -serial stdio -append "root=/dev/sda2 panic=1
rootfstype=ext4 rw" -hda 2012-10-28-wheezy-raspbian.img
```

Printre parametrii relevanți folosiți în comandă, putem enumera:

- `-kernel` - specifică imaginea de kernel de va rula;
- `-cpu` - specifică modelul procesorului pe care îl va emula;
- `-m` - definește cantitatea de memorie ce va fi alocată;
- `-hda` - specifică imaginea sistemului de fișiere ce va fi utilizată;

În Figura 14.19, se observă cum bootează sistemul de operare emulat. Pornirea acestuia durează în jur de 60 de secunde, mai lent decât pornirea unei mașini virtuale.

Pentru a vă autentifica în consola sistemului de operare emulat, folosiți utilizatorul *pi* și parola *raspberry*:

```
raspberry pi login: pi
Password:
...
pi@raspberrypi:~$
```

În acest fel, avem acces la un sistem care rulează (emulat) o arhitectură de procesor ARM.

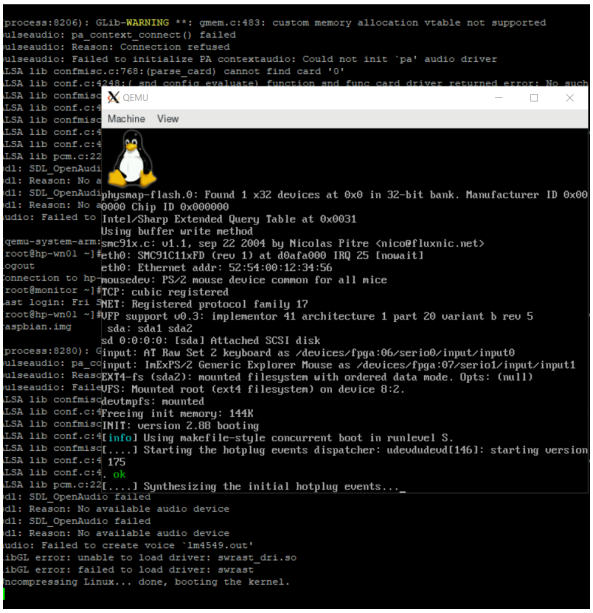


Figura 14.19: Pornirea unui sistem de operare emulat

14.8 Sumar

Tehnologia de virtualizare se folosește de creșterea puterii de calcul din hardware-ul modern pentru a rula sisteme de operare multiple (și diferite) pe același sistem fizic. O mașină fizică (numită *host*) va rula un hipervizor care va gestiona accesul la resursele hardware pentru mai multe mașini virtuale. Virtualizarea este de tip hosted (VirtualBox, VMware Workstation) sau de tip baremetal (VMware ESX, Xen).

O formă intermediară este containerizarea, cu ajutorul căreia aplicațiile pot rula izolat pe același sistem de operare, cu beneficii de performanță.

Virtualizarea permite rularea de mașini virtuale care folosesc aceeași arhitectură de procesor ca mașina fizică. Dacă dorim să rulăm aplicații care rulează pe o arhitectură de procesor diferită (de exemplu aplicații ARM pe sistem cu arhitectură x86), folosim un emulator, precum QEMU.

Tehnologiile de virtualizare stau la baza tehnologiilor cloud. Tehnologiile cloud permit stocarea de informații și rularea de aplicații prin Internet, fără ca utilizatorul să fie preocupat de locul unde se întâmplă acest lucru. Tehnologiile cloud sunt, în general, clasificate ca soluții de tip IaaS (*Infrastructure as a Service*), PaaS (*Platform as a Service*) și SaaS (*Software as a Service*).

Capitolul 15

Sisteme încorporate

După epoca calculatoarelor mainframe, urmată de o perioadă în care computerele personale au condus lumea tehnologiei, ne îndreptăm spre perioada calculatoarelor omniprezente (termenul a fost introdus pentru prima oară în 1988 de Mark D. Weiser, cercetător la Xerox PARC), când suntem înconjurați de calculatoare de dimensiuni tot mai mici, comparabile chiar cu dimensiunile unei monede.

Această evoluție se reflectă în popularitatea pe care a câștigat-o domeniul Internet of Things (IoT) și evoluția dispozitivelor inteligente. În prezent, dezvoltările tehnologiei se concentrează pe realizarea unor dispozitive care să se adapteze cât mai mult necesităților noastre și care să acționeze autonom, preluând cât mai multe din sarcinile noastre. Suntem înconjurați de reclame la aspiratoare inteligente care își construiesc o hartă a casei, parcurg întreaga suprafață și se conectează singure la încărcător. Purtăm pe încheietura mâinii ceasuri și brățări care ne monitorizează activitatea zilnică și ne avertizează când nu am făcut destulă mișcare sau stăm pe scaun de prea mult timp.

Ca să ne facem o idee mai clară asupra popularității acestor dispozitive, un studiu al World Economic Forum[2] indică folosirea a peste 14 miliarde de dispozitive în 2019, cu o estimare de 25 de miliarde în 2021.

În acest context, e ușor de observat o nevoie accelerată de a folosi sisteme de calcul tot mai mici. Toate aceste sisteme inteligente pe care le integrăm în viața noastră sunt, de fapt, niște roboți sau niște calculatoare de dimensiuni reduse. Fiecare astfel de dispozitiv conține o unitate de procesare și o memorie internă; mai specific, în componența dispozitivelor inteligente intră sistemele încorporate.

Un sistem încorporat (*embedded system*) este un sistem de calcul care controlează un proces. Spre deosebire de calculatoarele clasice, aceste sisteme nu sunt dezvoltate pentru a fi conectate la un monitor și o tastatură și controlate pentru operații diverse. Sistemele încorporate sunt create cu un scop bine definit și sunt adaptate pentru a deservi cât mai eficient acel scop, fie că sunt folosite pentru a controla o bandă de asamblare sau o brățară care măsoară activitatea fizică a unei persoane.

Sistemele încorporate stau la baza dispozitivelor inteligente și a platformelor IoT, ele având dimensiuni, memorie, capacitate de procesare și conectivitate adaptate sistemelor în componența cărora intră.

[2] https://www.weforum.org/agenda/2019/04/why-securing-the-internet-of-things-is-crucial-to-the-fourth-industrial-revolu

Un sistem IoT este un sistem de obiecte, dispozitive sau mașini interconectate care este conectat și la Internet. Rețeaua permite acestor „lucruri” să comunice între ele și să furnizeze informații valoroase despre mediul înconjurător în care sunt plasate.

15.1 Microcontrolere și calculatoare

Dispozitivele care controlează sisteme inteligente sau anumite operații specializate sunt numite dispozitive încorporate. Acestea sunt în principal unități de calcul având, de obicei, caracteristici reduse, mai puțină putere de procesare și mai puțină memorie față de calculatoarele cu care suntem obișnuiți. Multe dintre aceste sisteme sunt similare cu calculatoarele pe care foloseam cu 15 sau 20 de ani în urmă, dar care sunt proiectate să funcționeze în condiții speciale. Aceste dispozitive integrate trebuie să funcționeze continuu timp de luni sau chiar ani, deoarece ele sunt încorporate într-un sistem autonom. În plus, multe dispozitive trebuie să reziste la condiții dure de funcționare, cum ar fi temperaturi extreme, umiditate sau chiar ploaie. Cu cât dispozitivele încorporate sunt mai robuste, cu atât sunt și mai costisitoare. Există o mulțime de dispozitive încorporate având un preț de aproximativ 15-20 dolari. De cele mai multe ori, acestea sunt folosite pentru prototipare și în educație. Pe de altă parte, sisteme cu capacități similare, dar adaptate unor condiții dificile, pot atinge prețuri de 100 sau chiar și de 1000 de ori mai mari.

Unele dintre cele mai comune dispozitive integrate, folosite în special în educație, sunt Raspberry Pi sau Arduino. Aceste două placi sunt cunoscute datorită prețului lor scăzut și a rezistenței la scurtcircuite.

Un aspect important de menționat în ceea ce privește dispozitivele încorporate este că aceste sisteme, deși aparent asemănătoare cu calculatoarele personale, folosesc arhitecturi diferite. În timp ce calculatoarele personale se bazează pe procesoare Intel (x86, x64), aceste dispozitive încorporează diverse procesoare cum ar fi Intel, ARM sau MIPS. Astfel, aplicațiile pentru dispozitivele integrate sunt uneori mai greu de portat.

15.1.1 Microcontrolere

Cele mai simple sisteme încorporate sunt microcontrolerele.

După cum sugerează și numele, un microcontroler este un sistem de control de mici dimensiuni, folosit într-un număr mare de obiecte sau aparate. Dintr-o perspectivă de ansamblu, un microcontroler seamănă cu un mic computer care vine cu periferice, memorie proprie și cel puțin un procesor (CPU - Unitatea centrală de procesare) capabil să execute un program (numit *firmware*). De obicei, în cadrul unui sistem de control, senzorii și servomotoarele sunt conectate direct la microcontroler. Acest lucru se datorează faptului că microcontrolerele sunt concepute special pentru a oferi suport pentru conexiuni diferite la dispozitive periferice variate. În același timp, aceste plăcuțe sunt capabile să comunice cu un dispozitiv mai complex, cum ar fi un computer. În plus, microcontrolerele sunt optimizate și din alte puncte de vedere: dimensiune, consum de energie, costuri etc. În principiu, ele sunt simple, accesibile și utilizate cu un scop clar definit.

Deoarece microcontrolerele sunt dispozitive simple, concepute pentru un scop anume, acestea rulează o singură aplicație. Software-ul rulat de microcontrolere se numește *firmware*; îl putem defini ca „software creat special pentru hardware”. Firmware-ul este stocat, de regulă, în memoria flash (ROM), care este non-volatilă (adică păstrează datele chiar dacă dispozitivul este oprit și repornit) și conține informații relevante despre modul în care ar trebui să funcționeze sistemul. Putem deci să deducem că odată programat, un microcontroler poate fi integrat într-un sistem și el va rula acel firmware chiar dacă sistemul este rebootat.

Datorită faptului că aceste dispozitive sunt capabile să ruleze doar o singură bucată de software (firmware-ul), putem realiza aplicații în timp real (aplicații care necesită un răspuns instant la un declanșator extern). Practic, aceste dispozitive nu au un sistem de operare, deci orice întreruperi provenite de la un periferic sunt declanșate și procesate într-un timp exact stabilit. Aceasta înseamnă că procesorul nu mai execută sarcinile curente și începe executarea rutinelor legate de întrerupere. Dacă nu sunteți familiarizat cu conceptul de întreruperi, vă puteți gândi la ele ca la semnalele provenite de la dispozitivele periferice. Astfel, odată ce un dispozitiv periferic trimite un semnal, acesta este tratat instantaneu. În plus, un alt avantaj de a avea doar o singură bucată de software care rulează pe dispozitiv este faptul că putem prezice când se execută o anumită instrucțiune.

15.1.1.1 Conectori

Principalul scop al microcontrolerelor este de a le conecta la senzori și actuatori. Acest lucru se face de obicei prin intermediul unor conectori speciali; în cazul dispozitivelor educaționale, conectorii sunt niște pini. Din firmware, programatorii pot citi sau controla starea acestor pini. Un avantaj al microcontrolerelor este faptul că expun pini foarte specializați. În general, un microcontroler are în primul rând un set de pini meniți să îndeplinească sarcini de bază: ei permit conexiuni la periferice simple și, în fond, alimentează cu energie perifericele. Chiar dacă acest „pachet” de pini de bază este găsit pe aproape fiecare microcontroler, numărul și amplasarea lor diferă de la dispozitiv la dispozitiv. Un singur pin este capabil să efectueze mai multe funcții în același timp (acest fenomen este numit „multiplexare”).

Cei mai simpli pini pe care îi putem găsi pe dispozitivele încorporate sunt:

- `Vcc` - Acești pini sunt practic o sursă de tensiune cu o valoare fixă. Ne putem gândi la ei ca la polul pozitiv al unei baterii. Nu pot fi controlați din firmware.
- `GND` - Acești pini sunt cei opuși celor *Vcc*. Ei sunt masa dispozitivului, de aceea se numesc pini *ground*; ei sunt referința față de care se calculează orice cădere de tensiune. Nu pot fi controlați din firmware.
- `GPIO` (*General Purpose Input Output*) - Sunt pini simpli care pot fi controlați din firmware. În funcție de starea lor, ei pot să funcționeze fie ca pini `Vcc` sau `GND`, fie ca un voltmetru, citind date de la senzori.
- `ADC` (Analog to Digital Converter) - Sunt pini care preiau date de la senzori și le transformă în valori digitale. Valorile pot fi citite din firmware.
- `DAC` (Digital to Analog Converter) - Sunt pini asemănători cu cei de `Vcc`, dar care pot fi controlați din firmware să expună un voltaj între `0` și `Vcc`; sunt folosiți pentru

a controla periferice, cum ar fi motoare.

- `PWM` (*Pulse Width Modulation*) - Sunt pini care simulează comportamentul pinilor DAC, fiind mai ieftini. Aceștia oscilează între GND și Vcc, simulând o tensiune variabilă.

Pe de altă parte, există pini care controlează periferice mai complexe, ce transmit și primesc date folosind diverse protocoale:

- `SDA` + `SCL` - câte doi pini pentru fiecare canal de comunicație I^2C; detalii despre I^2C în Secțiunea 15.3.1.3
- `SCK` + `MOSI` + `MISO` - câte trei pini pentru fiecare canal de comunicație SPI suportat; detalii despre SPI în Secțiunea 15.3.1.2
- `TX` + `RX` - câte doi pini pentru fiecare canal serial suportat.

15.1.1.2 Exemple de microcontrolere

Cum am menționat deja, există o varietate foarte mare de microcontrolere pe piață, de la cele dezvoltate pentru a fi integrate în sisteme industriale, la plăcuțe simple care se folosesc pentru prototipare sau în educație:

- Arduino - Este o placă cu un microcontroler ATMega produs de o companie italiană. Este cel mai popular sistem de microcontroler utilizat pentru prototipuri și în scopuri educaționale. De fapt, Arduino constă atât în plăcuța efectivă, cât și în mediul integrat de dezvoltare (IDE), care ușurează procesul de programare al dispozitivului. Arduino și-a câștigat popularitatea chiar datorită IDE-ul foarte ușor de folosit, nu neapărat datorită produsului hardware.
- ESP32 - Este un microcontroler foarte ieftin cu WiFi integrat care este folosit în general ca *placa de rețea pentru microcontrolere*. Acestă placă se conectează la alte microcontrolere prin portul serial și permite acestora accesul la Internet.
- MicroBit - Este o placă dezvoltata de BBC în scopuri educaționale ce are un microcontroler nRF52 produs de Nordic Semiconductor. Este mai adecvat pentru a începe programarea pe microcontrolere, dar poate fi folosit și pentru proiecte mai mari.

15.1.2 Calculatoare integrate

Am descris deja microcontrolerele ca fiind dispozitive simple utilizate cu un scop clar definit. Tocmai datorită capabilităților reduse ale microcontrolerelor, acestea nu au de obicei acces la rețea. În plus, chiar dacă am avea acces la un microcontroler care suportă o conexiune la rețea, integrarea acestuia într-un sistem IoT ar reprezenta un risc de securitate. Din cauza memoriei reduse, microcontrolerele nu sunt capabile să implementeze protocoale securizate sau alte mecanisme de securitate. Prin urmare, folosind un microcontroler am obține un sistem care transferă date nesecurizate. Ținând cont de acest lucru, este clar că nu putem proiecta o soluție IoT bazată exclusiv pe microcontrolere.

Un calculator încorporat reprezintă un sistem de calcul de dimensiuni mici, de obicei foarte personalizat pentru a deservi unor scopuri specifice. Acest tip de dispozitive este utilizat în mod obișnuit ca parte a unui alt sistem (cum ar fi un ceas inteligent, un bancomat sau chiar un autoturism) care îndeplinește sarcini complexe. Deși aceste dispozitive sunt de fapt calculatoare, în majoritatea cazurilor, acestea nu suportă o interfață de utilizator, consumă cantități mici de energie și dispun de resurse limitate de calcul.

Fiind calculatoare, spre deosebire de microcontrolere, dispozitivele rulează un sistem de operare care gestionează resursele.

Unul din scopurile principale ale calculatoarelor integrate este de a trimite sau de a prelua date din rețeaua locală sau chiar de pe Internet. De aceea, aceste dispozitive au, în general, integrată o placă de rețea. Datorită memoriei și puterii de procesare crescute față de un microcontroler, calculatoarele integrate implementează mecanisme de securitate pentru transferul de date și pentru aplicațiile rulate. Acest lucru permite dispozitivelor să devină „inteligente” și să comunice între ele.

În plus, calculatoarele încorporate sunt capabile să execute aplicații în paralel. Pe de altă parte, datorită sistemului de operare care gestionează resursele hardware și software, aceste dispozitive încorporate nu pot rula aplicații în timp real. Acesta este un dezavantaj față de microcontrolere.

Pentru anumite proiecte simple, putem folosi calculatoare integrate în locul microcontrolerelor. Acest lucru se datorează faptului că și aceste dispozitive expun anumiți pini. Scopul pinilor este de a controla câteva periferice simple, pentru interacțiunea cu utilizatorul (de exemplu, unele LED-uri care semnalizează starea sistemului). De obicei, numărul de pini expuși de calculatoarele integrate este limitat, majoritatea fiind pini GPIO. Aceste dispozitive nu expun mulți pini pentru că scopul lor principal nu este acela de a interacționa cu mediul, ci de a aduna, procesa și transporta date. Adăugarea unor module de conectare, cum ar fi ADC sau DAC, crește prețul dispozitivului fără a-i crește în mod semnificativ valoarea. Pe de altă parte, aproape toate sistemele de calcul încorporate dispun de conexiuni avansate cum ar fi SPI, I^2C, USB, UART sau chiar HDMI. Pentru a fi ușor de controlat, unele calculatoare integrate pot fi conectate la un monitor, o tastatură și un mouse, similar unui calculator obișnuit.

15.1.2.1 Exemple de calculatoare integrate

Avem o diversitate mare de calculatoare integrate pe care le putem folosi în cadrul sistemelor inteligente:

- Raspberry Pi - Deși nu a fost primul calculator integrat, Raspberry Pi este cel mai cunoscut și a fost dispozitivul care a revoluționat domeniul IoT. Este primul calculator integrat dedicat publicului larg, având un preț scăzut, ceea ce a dus la o scădere generală a prețurilor la alte dispozitive similare și a făcut domeniul IoT accesibil pasionaților de tehnologie și în educație. Dispozitivul costă aproximativ 35 de dolari.
- BeagleBone Black - BeagleBone Black este un dispozitiv cu capabilități de procesare mai scăzute în comparație cu Raspberry Pi. Cu toate acestea, dispozitivul expune un număr mai mare și mai variat de pini. BeagleBone Black

poate fi folosit cu ușurință pentru prototipuri, dar poate fi integrat și în aparate inteligente, cum ar fi cabine foto sau frigidere inteligente. Dispozitivul costă în jur de 45 de dolari.

- Raspberry Pi Compute Module - Este un Raspberry Pi pentru uz industrial.

15.1.2.2 Sisteme de operare pentru calculatoarele integrate

După cum am precizat anterior, calculatoarele integrate sunt mai mici și mai ieftine, dar cu o putere de calcul și o memorie mai mică comparativ cu un calculator de dimensiuni mari. Raspberry Pi, de exemplu, are 1GB de memorie RAM în timp ce BeagleBone Black are 512 MB. Microprocesoarele acestor dispozitive au frecvențe la o treime din frecvența unui calculator personal. Acest lucru le permite să utilizeze o cantitate mică de energie și să fie accesibile ca preț. Pe de altă parte, acest aspect duce la nevoia de a avea sisteme de operare specializate, care să gestioneze cât mai eficient resursele disponibile. Aceste dispozitive au nevoie de un sistem de operare foarte eficient în ceea ce privește consumul de energie, deoarece funcționează de obicei pe baterie și trebuie să fie autonome pentru perioade lungi de timp.

Linux

Știm deja că Linux e o platformă foarte utilizată și pentru calculatoarele personale. În timp, odată cu evoluția IoT, s-au dezvoltat foarte multe distribuții Linux pentru calculatoare integrate:

- Raspbian - Este sistemul de operare recomandat pentru Raspberry Pi. A fost dezvoltat de Fundația Raspberry Pi. Ca orice alt sistem de operare, Raspbian suportă controlul dispozitivului prin Shell, folosind comenzile standard Linux. Raspbian vine în două versiuni: Desktop și Lite.
- Desktop este bazat pe Debian Linux și este optimizat pentru Raspberry Pi. Distribuția oferă inclusiv o interfață grafică de utilizator (GUI) și câteva aplicații, cum ar fi Chromium, LibreOffice și Claws Mail. Există și o versiunea Desktop care include preinstalate mai multe aplicații.
- Lite este versiunea Stretch fără GUI, având dimensiuni reduse. Interacțiunea cu sistemul se realizează doar prin linie de comandă.
- Ubuntu Server - Este sistemul Ubuntu Server compilat pentru Raspberry Pi, practic identic cu sistemul destinat calculatoarelor server sau desktop.
- Ubuntu Core - Este un sistem de operare bazat pe Ubuntu. Spre deosebire de Ubuntu Server, Core este special realizat pentru sisteme IoT. Acesta include doar sistemul de pachete *snap* si oferă posibilitatea aducerii la zi a sistemului de la distanță. Este în general utilizat în scopuri industriale.
- Yocto - Yocto este un set de instrumente open source pentru crearea de imagini de sisteme de operare bazate pe Linux pentru dispozitivele încorporate. Sistemul include medii de emulare și depanatoare. Yocto a devenit popular pe piața hobbyștilor când Intel a lansat dispozitivele Edison și Galileo, ambele având o distribuție Yocto. Este un sistem compatibil cu Raspberry Pi.

Sisteme de operare în timp real (Real-time Operating Systems - RTOS)

Această categorie constă în sisteme de operare construite de la zero, special pentru dispozitive încorporate. După cum sugerează și numele acestora, aceste sisteme de operare vizează susținerea aplicațiilor în timp real. Ele implementează algoritmi de programare speciali care se asigură că orice eveniment extern este tratat imediat. Sistemele de operare în timp real existente nu au nimic în comun, cu excepția faptului că toate sunt proiectate în același scop. În contrast cu cele două categorii anterioare, exemplele pe care le vom menționa nu împărtășesc același nucleu:

- FreeRTOS este un sistem de operare open source menținut de către Amazon. Este cel mai vechi sistem de acest gen, considerat ca fiind cel mai potrivit pentru uz industrial. Este scris in C și este axat pe performanță.
- Zephyr Project este un sistem de operare open source insiprat din Linux și realizat inițial de către Intel. Este considerat *fratele mai mic al Linuxului*. Deși este inspirat din Linux, funcționarea acestuia este foarte diferită, acesta fiind conceput ca un sistem de operare în timp real. Fiind mai nou decât FreeRTOS și având un API similar cu Linux, acesta este recomandarea actuală pentru folosirea în industrie.
- RIOT este un sistem de operare open source. Arhitectura acestuia îl face ușor de adaptat la sistemele cu memorie redusă. RIOT este compatibil cu Raspberry Pi, Beagle Board, unele tipuri de Arduino (modele mai avansate care sunt mai mult decât un microcontroler și au o putere de calcul mai mare).
- Contiki - Contiki este un sistem de operare open source dezvoltat special pentru sisteme IoT de Texas Instruments. Principalul său avantaj față de alte sisteme de operare similare este consumul redus de energie și de memorie.
- Tock este primul sistem de operare folosit în industrie scris într-un alt limbaj decât C, mai exact Rust. De asemenea, este primul sistem de operare pentru micrcontrolere care face o separare clară între kernel și aplicații. Tock este folosit de către Google în proiectul OpenSK și de către proiectul OpenTitan.

15.2 Sisteme simple de intrare ieșire

Având în vedere faptul că sistemele inteligente au drept scop extragerea de informații despre mediul înconjurător și interacțiunea cu acesta, este necesară integrarea senzorilor și a actuatorilor în aceste sisteme.

Un senzor este o componentă electronică sau un modul destinat să urmărească evenimente sau modificări din mediu și să transmită datele colectate unui alt dispozitiv. Exemple sunt senzori de temperatura, de umiditate, butoane, senzori de lumina etc.

Un actuator este un element de acționare care aduce modificări asupra mediului. Un exemplu foarte bun este un LED.

Aceste sisteme de intrare/ieșire sunt perifericele specifice dispozitivelor încorporate, deci una din caracteristicile principale ale unui dispozitiv încorporat este capacitatea de realiza conexiuni la aceste periferice.

În dezvoltarea sistemelor inteligente putem alege între mai multe tipuri de periferice în funcție de modul în care acestea se conectează și schimbă date cu dispozitivul încorporat. Există periferice simple care comunică direct cu dispozitivul la care sunt

conectate sau există unele mai complexe care au o unitate de procesare integrată (un microcontroler), permițând un schimb mai complex de mesaje.

Din punct de vedere al datelor furnizate dispozitivului la care sunt conectate, perifericele pot să fie digitale sau analogice (se trimit semnale digitale sau analogice).

Semnalele care sunt continue în timp sunt considerate semnale analogice, în timp ce semnalele care iau valori discrete sunt considerate semnale digitale. Considerăm că un semnal este continuu dacă acesta poate să ia orice valoare într-un anumit interval.

Dacă, de exemplu, semnalul nostru poate lua orice valoare în intervalul [2; 5] (2; 2.1; 2.11; 2.11111; 3.(4); și așa mai departe) este un semnal analogic. Dacă avem un semnal discret, el poate lua doar anumite valori. De exemplu, dacă semnalul nostru poate lua doar valorile 1; 0 (pornit / oprit) sau poate lua doar valorile 2; 3; 4; 5; 100, el este digital.

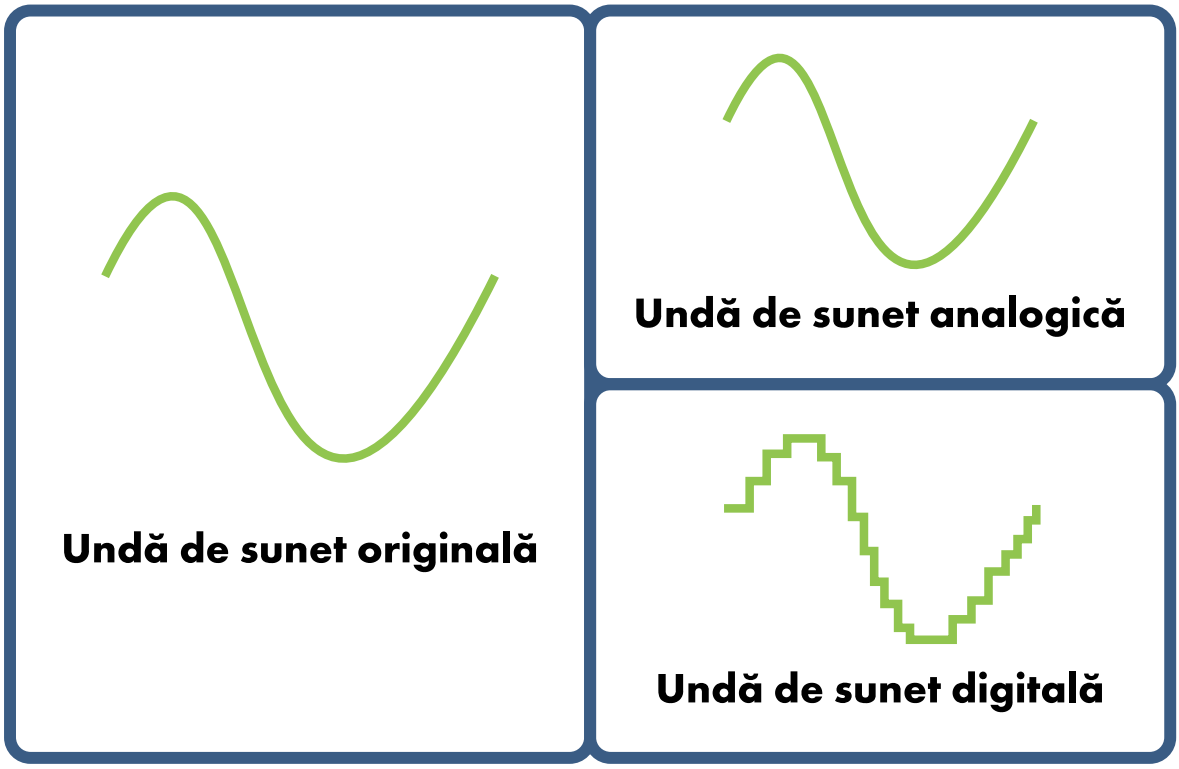


Figura 15.1: Semnal analogic vs. semnal digital

Cum am precizat mai sus, dispozitivele încorporate, cum ar fi Raspberry Pi sau Arduino, expun pini la care aceste sisteme de intrare/ieșire se pot conecta. În funcție de semnalul pe care perifericul îl transmite, el trebuie conectat la anumiți pini (nu putem conecta un senzor care transmite un semnal analogic la un pin care suportă doar semnale digitale).

Cei mai simpli pini sunt cei care sunt folosiți pentru semnale digitale. Ei se numesc pini GPIO (*General Purpuse Input/Output*) și pot să funcționeze în două moduri, ca intrare sau ca ieșire.

În cazul în care un pin GPIO funcționează ca o ieșire digitală, el poate fi conectat la un sistem de acționare pe care îl controlăm prin intermediul pinului. În acest caz, pinul funcționează ca o baterie și poate să înlocuiască fie borna pozitivă, fie borna negativă a bateriei. În general, vom seta pinul în starea *HIGH* (1), în care va funcționa ca o bornă pozitivă, sau *LOW* (0), în care va funcționa ca o borna negativă (sau ca un pin de ground).

Cel mai simplu exemplu de folosire pentru acești pini este controlul unui LED. Un LED este o componentă electronică (o diodă) care generează lumină când este străbătută de curent. Deci, pentru a face un LED să lumineze, e suficient să îl conectăm la o baterie.

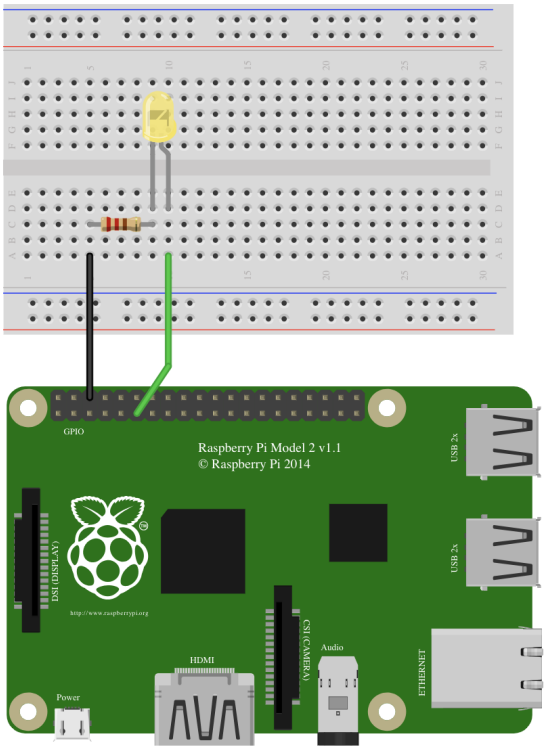


Figura 15.2: LED conectat la Raspberry Pi

Figura 15.2 conține un LED conectat la pinii unui Raspberry Pi. Unul din pini este conectat la ground (borna negativă a bateriei), iar celălalt este conectat la un pin GPIO. Dacă scriem valoarea 1 pe pin, acesta va genera un voltaj și LED-ul va începe să lumineze. Dacă scriem valoarea 0 pe pin, acesta va fi practic conectat la două borne negative, deci nu va lumina. Jonglând cu aceste două stări putem face diverse jocuri de lumini.

Pe de altă parte, putem folosi un pin digital ca intrare. În acest caz, el se comportă ca un voltmetru care măsoară diferența de potențial relativ la groundul plăcii, adică tensiunea la nivelul pinului. Dacă tensiunea este mai mică decât jumătate din valoarea maximă suportată de placă (`Vcc/2`), pinul va raporta valoarea 0. În caz contrar, se va raporta valoarea 1.

Putem, de exemplu, să conectăm un buton la un astfel de pin și să citim starea butonului. În general, dacă butonul va fi apăsat vom citi o tensiune apropiată de tensiunea maximă a plăcii, iar dacă butonul nu este apăsat, tensiunea va fi aproape 0.

Pinii GPIO sunt utili în interacțiunea cu periferice simple cum ar fi LED-uri sau butoane, dar majoritatea senzorilor și a elementelor de acționare nu funcționează atât de ușor. Pentru perifericele mai complexe avem la dispoziție pini analogici și PWM.

În general, senzorii sunt rezistențe variabile care își modifică valoarea în funcție de factorii de mediu. Astfel, căderea de tensiune de pe senzor se schimbă.

Pinii analogici funcționează ca un voltmetru care măsoară căderea de tensiune de pe senzor și o transformă într-o valoare finită. Dispozitivul care face conversia din voltaj într-o valoare întreagă, transmisă plăcii, se numește **convertor analog-digital** (ADC). Valorile întregi returnate depind de numărul de biți integrați în ADC. Dacă considerăm n numărul de biți al ADC-ului, valorile sunt în intervalul $0-2^{n-1}$. De exemplu, convertorul analog-digital integrat în plăcuțele Arduino are 10 biți, deci valorile citite de pe pinii analogici sunt în intervalul `0-1023`.

Putem astfel să conectăm un fotorezistor la un pin analog, iar în funcție de luminozitatea mediului vom citi valori mai mici sau mai mari. Dacă identificăm o luminozitate prea mică, putem scrie valoarea 1 pe un pin GPIO conectat la un LED și să aprindem LED-ul, obținând un prototip al unui sistem de iluminare inteligent.

Am stabilit că pentru a citi semnale analogice vom folosi pini conectați la un ADC, dar ce facem pentru a trimite semnale analogice mediului?

În primul rând, există posibilitatea de a folosi un convertor digital-analogic (DAC). El transformă o valoarea digitală într-un semnal analogic. Putem deci să scriem o valoare pe un astfel de pin pentru a controla tensiunea de ieșire a pinului. Astfel de pini sunt folosiți pentru a controla motoare.

Pe de altă parte, convertoarele de la digital la analogic sunt destul de scumpe, de aceea multe dispozitive încorporate nu expun astfel de pini. În schimb, ele folosesc o metodă mai accesibilă de simulare a semnalelor analogice, PWM.

Pulse Width Modulation (PWM) este o modalitate de a simula semnale analogice prin mijloace digitale. Putem realiza acest lucru prin alternarea rapidă între semnale care pot avea doar două valori (0/1), dar prin modificarea factorului de umplere, adică timpul în care pinul are valoarea 1. De multe ori, receptorul semnalului va face o medie a celor două valori alternative.

De exemplu, dacă avem un LED care clipește foarte repede, ochiul uman îl va percepe ca un LED ce luminează la o anumită intensitate. În funcție de raportul între perioada în care LED-ul este aprins versus perioada de timp în care el este stins, lumina va fi percepută la o intensitate mai mare sau mai joasă.

Acești pini pot fi folosiți pentru a controla LED-uri sau chiar motoare.

15.3 Magistrale de comunicare

Până acum am enumerat modalitățile de conectare a perifericelor simple, folosind pini digitali sau analogici. Cum am menționat deja, multe din perifericele existente sunt sisteme complexe, care au un procesor integrat și care trimit date structurate. În plus, există multe periferice care comunică wireless cu dispozitivele încorporate.

Pentru a interacționa cu aceste dispozitive de intrare/ieșire complexe, există diverse protocoale prin fir (ex. serial, SPI (*Serial Pripheral Interface*), I^2C, Modbus, CAN (*Controller Area Network*)) sau wireless (ex. Bluetooth, Wi-Fi, LoRa, Z-Wave). Unele din aceste protocoale sunt valabile și pentru calculatoarele clasice. De exemplu, porturile USB implementează o conexiune serială. De asemenea, utilizăm zilnic mouse-uri wireless, unele folosind Bluetooth pentru comunicare. Pe de altă parte, multe

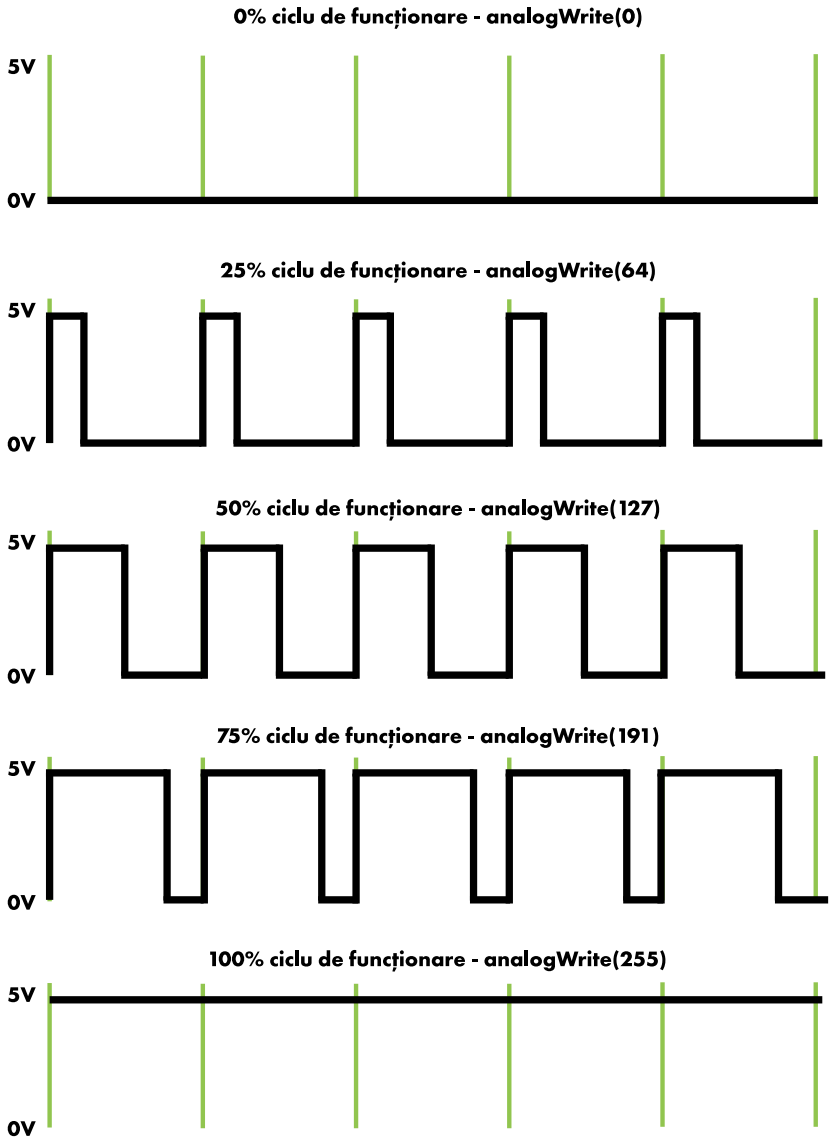


Figura 15.3: Pulse Width Modulation (PWM)

din protocoalele pe care le vom descrie mai departe au fost implementate special pentru comunicarea între dispozitivele încorporate și periferice.

15.3.1 Comunicare prin fir

Unele din primele periferice complexe au fost dezvoltate pentru a fi conectate direct la calculatoare, transmisia făcându-se prin intermediul firelor. Cea mai cunoscută astfel de conexiune este conexiunea serială.

În continuare vom descrie câteva din protocoalele pe care mulți senzori și alte periferice le implementează.

15.3.1.1 Comunicarea serială

Comunicarea serială (Figura 15.4) e unul dintre cele mai cunoscute și simple protocoale. Protocolul presupune trimiterea datelor între dispozitive bit cu bit, pe rând. În contrast, există linii de comunicație paralele, unde datele sunt trimise în paralel. Avantajul comunicării seriale este simplitatea.

Pentru a implementa un protocol serial sunt necesare minim două linii de comunicație, RX și TX. Pe fiecare linie se trimit date într-o direcție. Fiecare dispozitiv trimite date pe TX și primește date pe linia RX. Astfel, când conectăm două dispozitive, linia RX a unuia se va conecta la linia TX a celuilalt și invers.

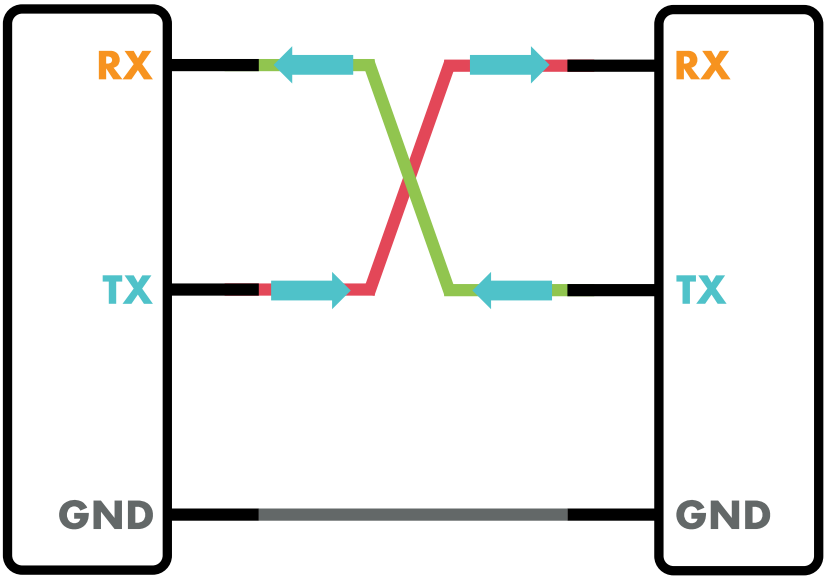


Figura 15.4: Conexiune serială

Pentru a se asigura un transfer corect de date, protocolul serial se bazează pe următoarele elemente:

- biți de date - datele efective transmise între dispozitive;
- biți de sincronizare - sunt biți transmiși înaintea și după biții de date, pentru a marca începutul și sfârșitul blocului de date;

- biți de paritate - e o modalitate simplă de a face verificarea datelor transmise (o sumă de control foarte simplă); dacă trimitem 8 biți de date și din acei 8 biți 5 conțin valoarea 1, datele trimise conțin un număr impar de valori de 1, deci bitul de paritate va fi setat la 1, dacă numărul de valori de 1 este par, bitul de paritate va fi setat la 0;
- baud rate - specifică viteza de transmitere a datelor; este foarte important ca ambele dispozitive conectate să transmită date la aceeași viteză.

Protocolul serial poate fi folosit doar la comunicarea între două dispozitive. Asta înseamnă că fiecare dispozitiv periferic conectat are nevoie de un port serial separat, făcând conectarea multor dispozitive nepractică.

15.3.1.2 SPI

SPI e un protocol mai complex, care necesită 3 linii pentru comunicare. Datele sunt transmise pe liniile MISO (*Master In Slave Out*) și MOSI (*Master Out Slave In*). Cea de-a treia linie este folosită pentru sincronizare (`SCLK`).

SPI vine ca o îmbunătățire a conexiunii seriale, încercând să asigure sincronizarea celor două dispozitive care comunică, transmisia de date realizându-se într-un mod mai eficient prin reducerea erorilor rezultate de la transmisie. Astfel, SPI se folosește de o a treia linie de comunicare dedicată sincronizării transmisiei de date.

Un alt avantaj al SPI e că are suport pentru mai multe periferice, denumite *slave*, conectate la un dispozitiv principal, numit *master*. Fiecare dispozitiv este activat sau dezactivat la un moment dat folosind un pin GPIO oarecare setat la valoare 0 sau 1.

15.3.1.3 I^2C

I^2C, sau IIC (*Inter-Integrated Circuit*) e un alt protocol care necesită două linii pentru transmiterea de date, SDA și SCL. SDA e linia pe care se transmit efectiv datele, în timp ce SCL e linia de sincronizare.

Avantajul protocolului I^2C este că putem conecta mai multe dispozitive simultan. Avem dispozitive master și slave. În cazul I^2C este foarte importantă sincronizarea dintre dispozitive. Avem o singură linie pe care se trimit toate datele. De aceea, comunicarea este mereu inițiată de un master care cere informații de la unul din dispozitivele slave conectate.

Fiecare dispozitiv are o adresă formată din 7 biți, adresă înregistrată în periferic. Sistemul master interoghează pe rând fiecare adresă, putând astfel să descopere adresele tuturor dispozitivelor legate la sistem.

15.3.1.4 Alte protocoale folosite industrial

Modbus

Un protocol folosit destul de mult in industrie este ModBus. Spre deosebire de cele enumerate anterior, acesta este un protocol software. El definește cum arată datele

transmise între dispozitive, fără a defini însă o infrastructură de comunicație. Există diferite variante, în funcție de ce mijloc de transport de date se folosește. Exemple sunt o legătură serială sau Ethernet (rețea). Similar cu I^2C, fiecare periferic conectat la Modbus are o adresă unică.

CAN

Un protocol folosit în industria Auto este CAN (*Car Area Network*). Acesta leagă sistemele de control din autoturisme de periferice. Aproape toate sistemele electronice din autoturisme sunt legate la magistrala CAN. Fiecare dispozitiv conectat la magistrală se numește nod. Fiecare nod are un ID (adresă) unic. Protocolul are o variantă de viteză redusă, tolerantă la erori. Protocolul a fost gândit pentru transmisie în timp real.

15.3.2 Comunicarea wireless

Pe măsură ce platformele IoT au evoluat, ele au fost incluse în multe sisteme întinse pe arii largi, cum ar fi sistemele agricole. Pentru astfel de cazuri, conectarea fizică a senzorilor la sistemele de procesare este costisitoare și consumă mult timp. Drept urmare, au fost implementate o mulțime de microcontrolere și calculatoare integrate care suportă conexiuni WiFi. Cu toate acestea, deoarece multe dintre dispozitivele utilizate în sistemele inteligente funcționează pe baterie, a existat necesitatea unui protocol care să implice consum redus de energie. Astfel, au apărut mai multe protocoale wireless, special concepute pentru IoT.

15.3.2.1 Bluetooth

Acesta a fost unul dintre primele protocoale implementate în sistemele IoT. Pe măsură ce domeniul s-a dezvoltat și Bluetooth a rămas principalul protocol de comunicare între dispozitive, acesta a fost extins, implementându-se Bluetooth 4.0, sau *Bluetooth Low Energy* (BLE) . Acesta este special conceput pentru a consuma puțină energie în procesul de transfer al datelor.

15.3.2.2 Z-wave

Este un protocol utilizat în principal pentru sistemele de automatizare a caselor (întrerupătoare, termostate, încuietori inteligente). Comunicația între dispozitive se face prin unde radio, iar dispozitivele trebuie să fie amplasate la o distanță de maxim 100 m.

15.3.2.3 Sigfox

Este un protocol conceput pentru transmiterea de date între dispozitive situate la distanțe considerabile unul de celălalt. Principalul avantaj al Sigfox este că poate fi utilizat în agricultură sau în alte domenii în care dispozitivele trebuie să transmită date pe distanțe mari. Folosind Sigfox, putem transmite date pe distanțe de până la 50 km în câmp deschis.

15.3.2.4 LoRa

Numele protocolului provine de la sintagma Long Range, rază lungă. Este similar cu Sigfox, permițând schimbul de pachete pe distanțe lungi, până la 50 km în câmp deschis, folosind foarte puțină energie pentru transmiterea de date.

15.3.3 Sisteme Gateway

Pentru a permite acestor periferice să trimită și să primească date de la dispozitivele integrate, cele două părți trebuie să implementeze același protocol. Prin urmare, construirea unei soluții IoT care este, de exemplu, compatibilă atât cu periferice LoRa, cât și cu periferice Sigfox, necesită implementarea celor două protocoale în logica aplicației, iar introducerea unui al treilea protocol necesită alt efort. De aceea, pentru comunicarea cu astfel de periferice se folosesc, de obicei, gateway-uri. În general, gateway-urile sunt dispozitivele care implementează protocolul de comunicație periferic, pe de o parte, și un protocol TCP / IP standard, pe de altă parte, pentru a comunica cu dispozitivul încorporat. Rezultatul este că aplicația care rulează pe dispozitivul încorporat comunică doar cu gateway-urile, folosind un singur protocol, și este independentă de senzorii conectați la sistem.

15.4 Dezvoltarea programelor pentru sisteme integrate

În procesul de dezvoltare al aplicațiilor pentru sisteme integrate trebuie să ținem cont de faptul că aplicația dezvoltată va funcționa pe o perioadă îndelungată și pe un sistem autonom. Astfel, trebuie să ne asigurăm că orice posibile erori sunt tratate cu atenție, fără a necesita intervenție umană.

Putem identifica următorii pași în dezvoltarea programelor pentru aceste sisteme:

1. scrierea programului
2. testarea pe mai multe dispozitive
3. instalarea aplicației pe toate dispozitivele necesare
4. aducerea la zi a aplicației

În ceea ce privește scrierea programului, procesul nu este diferit de cel pentru aplicațiile obișnuite. În schimb procesul de rulare a aplicației pe dispozitiv este diferit. Având în vedere că ne referim la dispozitive care nu au fost concepute pentru interacțiunea directă cu utilizatorul, acestea nu au, în general, interfață grafică. Practic, noi vom scrie programul folosind un calculator obișnuit, după care vom încărca executabilul pe dispozitivul integrat.

Cel mai important aspect când vine vorba de rularea unei aplicații pe un astfel de dispozitiv este că, de cele mai multe ori, arhitectura pe care vom rula nu este aceea pe care compilăm aplicația. De exemplu, dacă scriem un program pentru un Raspberry Pi, care are un procesor ARM, va trebui să generăm un executabil pentru acest procesor. Având în vedere că executabilul este generat pe calculatorul personal, cu arhitectură de

procesor x86, avem nevoie de utilitare care permit compilarea programelor pentru alte arhitecturi.

Procesul de compilare a unui program pentru a rula pe o altă arhitectură se numește **cross-compiling**.

Odată generat executabilul, acesta trebuie transferat pe sistemul integrat, sistem la care nu avem o conexiune directă. Astfel, sunt necesare utilitare precum `scp`, care implementează copierea prin SSH, pentru a transfera fișierul executabil pe dispozitive. În plus, avem nevoie de acces la dispozitiv pentru a putea rula efectiv executabilul. Cea mai accesibilă metodă de a face acest lucru este să deschidem o conexiune SSH la dispozitiv.

Odată ce avem versiunea finală a executabilului, acesta trebuie transferat pe toate dispozitivele pe care urmează să le dăm în producție. Acest procedeu este mai complicat, de aceea există platforme speciale pentru asa ceva, cum ar fi Resin.io sau Android Things. Pentru a le folosi, trebuie să înregistrăm toate dispozitivele în cadrul platformei, după care se folosește un sistem de distribuție pentru a instala aplicația pe fiecare dispozitiv. În plus, aceste platforme oferă posibilitatea de a aduce îmbunătățiri aplicației și de a aduce la zi toate dispozitivele.

În ceea ce privește aducerea la zi, trebuie să ținem cont de faptul că dispozitivele pe care lucrăm sunt în producție, integrate în diverse obiecte sau sisteme în funcțiune. De aceea, este foarte important ca procesul de aducere la zi să decurgă fără întreruperi și, dacă intervine vreo problemă, să ne asigurăm că aceasta nu rezultă în imposibilitatea de a mai folosi aparatul în cauză.

15.5 Internet of Things

În prezent, una din cele mai răspândite noțiuni din domeniul IT este cea de Internetul Lucrurilor (*Internet of Things* sau pe scurt IoT). Acest cuvânt cheie se referă la ideea de inteligență ambientală, unde suntem înconjurați de obiecte și aparate inteligente care se adaptează nevoilor noastre și acționează în consecință. Exemple de astfel de obiecte și aparate inteligente sunt lumini inteligente care funcționează pe baza luminozității exterioare, asistenți controlați vocal sau mașini autonome. Scopul IoT este să trăim într-un mediu în care aceste dispozitive inteligente sunt integrate în jurul nostru, devenind imperceptibile, dar îmbunătățindu-ne stilul de viață.

Cel mai potrivit termen care să descrie această situație spre care tinde Internetul Lucrurilor este cel de calcul omniprezent (*ubiquitous computing*). Este conceptul care a precedat IoT-ul și care a fost introdus în jurul anului 1988 de Mark D. Weiser, cercetător la Xerox PARC. Chiar dacă în perioada respectivă existau niște limitările hardware considerabile, Weiser a reușit să anticipeze progresul care va veni. De asemenea, a publicat mai multe lucrări în care descrie concepte similare cu cele din IoT, precum și riscurile pe care un asemenea progres le aduce. În cadrul lucrării intitulată „Computerul pentru secolul XXI”, Weiser menționează ideea că un computer omniprezent nu se referă la un lucru pe care îl poți purta cu ușurință, ci mai degrabă la un lucru care este o parte a traiului zilnic la un asemenea nivel încât nu îi mai observăm prezența. Integrarea perfectă a computerelor în activitățile noastre zilnice rămâne și în prezent o provocare continuă, la care evoluția tehnologică se luptă să facă față. Unele din

previziunile lui Mark Weiser menționează lupta de a obține o tehnologie calmă, o problemă cu care ne confruntăm acum prin asaltul de informații sau prin încălcarea frecventă a intimității noastre prin cunoscutele atacuri cibernetice. Deși suntem departe de a atinge calmul tehnologic, evoluțiile tehnologice încă au dificultăți în a îndeplini această nevoie.

Unul dintre primele dispozitive de uz zilnic care a fost conectat la Internet a fost automatul de Coca-Cola de la Universitatea Carnegie Mellon. Aparatul a fost dezvoltat la începutul anilor 1980 de un grup de studenți. Scopul studenților a fost de a dezvolta o modalitate prin care să se poată verifica dacă automatul este gol fără a necesita deplasarea inutilă. În zilele noastre, acest aparat este considerat unul din primele sisteme inteligente.

În anul 2000, Kevin Ashton, co-fondator al Centrului Auto-ID din cadrul Massachusetts Institute of Technology, lucra la ceea ce avea să devină Internetul Lucrurilor (IoT), termen pe care l-a brevetat în anul 2002, împreună cu David L. Brock. În acea perioadă, echipa lui Kevin Ashton lucra la o modalitate de standardizare a identificării prin radiofrecvență (RFID - *Radio-frequency identification*) prin conectarea informațiilor obținute la Internet. În prezent există la nivel global laboratoare Auto-ID care lucrează împreună la dezvoltarea de sisteme RFID și IoT.

Popularitatea în creștere a IoT poate fi explicată prin dorința oamenilor de a automatiza cât mai multe din sarcinile zilnice. De altfel, răspunsul este potrivit nu doar pentru evoluția IoT, ci pentru evoluția tehnologică în general. De secole am căutat metode de automatizare a muncii noastre prin intermediul mașinilor, la fel cum în momentul de față ideea de a avea aspiratoare autonome sau frigidere care fac singure cumpărăturile on-line este una foarte atrăgătoare. Ca rezultat, companiile IT au investit mult în cercetare și dezvoltare în acest domeniu, ceea ce a dus la scăderea prețului senzorilor, microcontrolerelor, gateway-urilor etc., făcând soluțiile inteligente accesibile publicului larg.

Un aspect important pe care să îl luăm în calcul când discutăm despre dispozitive inteligente este acela că un obiect inteligent nu e neapărat un sistem IoT. Să luăm exemplul unor becuri inteligente, care luminează mai puternic sau mai slab în funcție de lumina ambientală. Deși este evident că avem un dispozitiv inteligent, care își adaptează comportamentul în funcție de ambient și de nevoile noastre, sistemul nu este unul IoT, nu avem conectivitate la Internet și nu conectăm mai multe sisteme diferite.

Fiind vorba despre dispozitive care comunică, pentru Internet of Things se poate defini o stivă pentru a reprezenta traseul pe care datele sunt transmise. Deși nu este definită ca un standard, se consideră, în general, că majoritatea sistemelor Internet of Things au următoarea structură:

- senzori și actuatori
- procesare și stocare locală
- conexiunea la Internet
- cloud

Acum, că suntem familiarizați cu caracteristicile unui sistem IoT, să aruncăm o privire la unele dintre cele mai utilizate dispozitive inteligente din zilele noastre. Este important să observăm modul în care urmăresc stiva IoT și care este scopul final al acestora.

15.5.1 Implementări de sisteme IoT

15.5.1.1 Amazon Alexa

Figura 15.5: Amazon Alexa

La o primă privire, soluția Alexa (în Figura 15.5), dezvoltată de Amazon, nu pare nimic mai mult decât un difuzor wireless. De fapt, sistemul este capabil să facă mult mai mult decât redarea unor melodii, fiind un sistem complex de control vocal. Prin intermediul Alexa, utilizatorii pot seta alarme, căuta articole pe Wikipedia, pot obține informații în timp real despre condițiile meteorologice și de trafic. În plus, Alexa poate funcționa și ca un sistem de control al casei, fiind compatibil cu o gamă largă de dispozitive (becuri, televizoare etc.).

Din punct de vedere tehnic, Alexa este un dispozitiv inteligent care utilizează un microfon pentru a obține informații de la utilizator, trimite datele în cloud, unde acestea sunt procesate - un asistent virtual (*virtual assistant*). Odată ce sistemul din cloud „înțelege” comanda, acesta generează răspunsul, fie că este vorba despre anumite informații pe care le primește de pe Internet, fie despre unele acțiuni pe care dispozitivul trebuie să le efectueze, și trimite răspunsul înapoi pe dispozitiv.

15.5.1.2 Ceasuri inteligente

Câteva din caracteristicile principale ale ceasurilor inteligente sunt comenzile vocale, ghidarea pe hartă, redarea muzicii sau afișarea de mesaje text. Aceste dispozitive pot realiza sarcini complexe, majoritatea fiind dotate cu sisteme de operare mobile precum Android. Dimensiunile reduse și utilitatea le transformă în „calculatoare portabile” și chiar „telefoane care pot fi purtate”, ultimele versiuni fiind capabile să răspundă și să efectueze apeluri vocale și video.

În realitate, aceste sisteme sunt în general alcătuite din două componente: ceasul inteligent și telefonul la care acesta se conectează. Ceasurile sunt de fapt senzorii și ecranul care redă imagini, în timp ce telefoanele inteligente sunt adevăratele dispozitive de procesare. Cele două comunică de obicei prin Bluetooth 4.0. Provocarea principală în proiectarea acestui tip de dispozitive este de a asigura consumul de energie cât mai redus posibil.

15.5.1.3 Electrocasnice inteligente

În ultima perioadă, electrocasnicele au devenit „inteligente”, ceea ce înseamnă că se pot conecta la Internet într-un fel sau altul și, bineînțeles, pot comunica între ele pentru a optimiza cantitatea de energie utilizată. Este vorba de frigidere, mașini de spălat vase, aspiratoare sau termostate inteligente. În general, funcționarea lor poate fi controlată la distanță, printr-o aplicație telefonică.

Pentru toate aceste dispozitive inteligente, principiul de funcționare este același: avem senzori care trimit date către o unitate centrală. Datele ajung în cloud, astfel încât utilizatorii să poată vizualiza datele în orice moment și de oriunde. După aceea, orice comandă a utilizatorului este trimisă de către utilizator în cloud, de unde este transmisă calculatorului integrat, care comandă actuatorii (ex. pornește aspiratorul).

15.5.2 Edge/Fog Computing

La început, majoritatea sistemelor IoT funcționau într-un mod foarte simplu. Ele adunau date din mediul înconjurător, după care trimiteau datele în cloud, unde se făceau toate procesările și se luau toate deciziile. În timp, lumea și-a dat seama că o astfel de arhitectură are un mare dezavantaj: se trimit cantități imense de date redundante în cloud. Acesta este motivul pentru care a apărut un concept destul de nou, cel de „Fog Computing” (fog = ceață) sau „Edge Computing” (edge = margine). „Fog Computing” este un termen introdus de Cisco în noiembrie 2015. El face referire la cuvântul „ceață” din prisma faptului că în acest caz prelucrarea datelor se face parțial local și parțial în cloud, adică în ceață. În același mod, Intel a făcut referire la acest concept folosind sintagma „Edge computing”, deoarece prelucrarea se realizează la marginea cloud-ului.

Indiferent de sintagma folosită, ceea ce face ca această arhitectură să iasă în evidență este capacitatea ei de a procesa rapid datele pe loc, reducând semnificativ cantitatea de date transmise în cloud.

Pentru a înțelege mai bine importanța prelucrării locale a datelor, vom lua un exemplu simplu, cel al unei stații meteorologice.

Avem o stație meteorologică care monitorizează viteza vântului. Să presupunem că se măsoară viteza vântului o dată la 2 minute, ceea ce înseamnă 30 de măsurători într-o oră sau 720 de măsurători într-o singură zi. Cantitatea de date înregistrate în săptămâni sau ani ar fi vastă și dificil de procesat sau stocat. Să ne imaginăm acum că stația meteo face măsurători o dată la două minute și înregistrează datele în cloud numai dacă viteza vântului s-a schimbat, deoarece, datorită procesării locale, are capacitatea de a decide dacă noile măsurători sunt egale cu cele anterioare sau nu. Acest lucru înseamnă același număr de măsurători, însă datele stocate sunt mult mai puține.

15.5.3 Securitatea IoT

Una din vulnerabilitățile sistemelor IoT provine chiar din caracteristica principală a acestor sisteme: conectivitatea. Avem de-a face cu platforme care partajează cantități mari de date sensibile, ceea ce le face vulnerabile la atacuri cibernetice. Când ne gândim la dispozitive IoT trebuie să ne amintim că unele controlează fabrici întregi, mașini autonome sau sisteme de securitate a unei clădiri de birouri. De aceea, este esențial să conștientizăm importanța securizării acestor sisteme.

Desigur, o mulțime de soluții menite să rezolve vulnerabilitățile de securitate sunt în curs de dezvoltare și implementate în infrastructura dispozitivelor IoT. În acest timp se cercetează continuu tehnologii noi și îmbunătățite pentru stocarea și securizarea unor cantități mari de date.

15.6 Sumar

Sistemele integrate sunt folosite pentru un scop bine stabilit, au o interfață limitată cu utilizatorul și rulează sisteme de operare special adaptate pentru sarcinile sale. Se definesc două tipuri de sisteme integrate: microcontrolere și calculatoare. Prima categorie este un sistem de procesare limitat, însă care funcționează în timp real, și este folosit pentru conectarea hardware-ului. Calculatoarele, pe de altă parte, rulează un sistem de operare, în general nu unul în timp real, pot procesa o cantitate mai mare de date și pot fi conectate la rețea.

Conectivitatea dintre periferice și sistemele integrate se realizează prin interfețe dedicate, cu fir sau fără fir. Un element important în conectarea dispozitivelor periferice și a celor integrate îl reprezintă gateway-urile, care aduc la un protocol comun conexiunea.

Sistemele încorporate moderne sunt conectate la Internet pentru a permite colectarea facilă de informație și controlul lor. Ansamblul sistemelor încorporate legate la Internet formează Internetul Lucrurilor (*Internet of Things* - IoT).

Bibliografie

[1] Brian W. Kernighan and Rob Pike. *The Practice of Programming*. Addison-Wesley Professional, February 1999.

Glosar

Automatizarea unei acțiuni, 367

Cale absolută, 33
Cale relativă, 34

director, 28

fișier, 28

Loading, 94

Metadatele unui fișier, 37

Permisiuni și privilegii, 125
proces, 82

sistem de fișiere, 29

Utilizatori privilegiați și neprivilegiați, 125